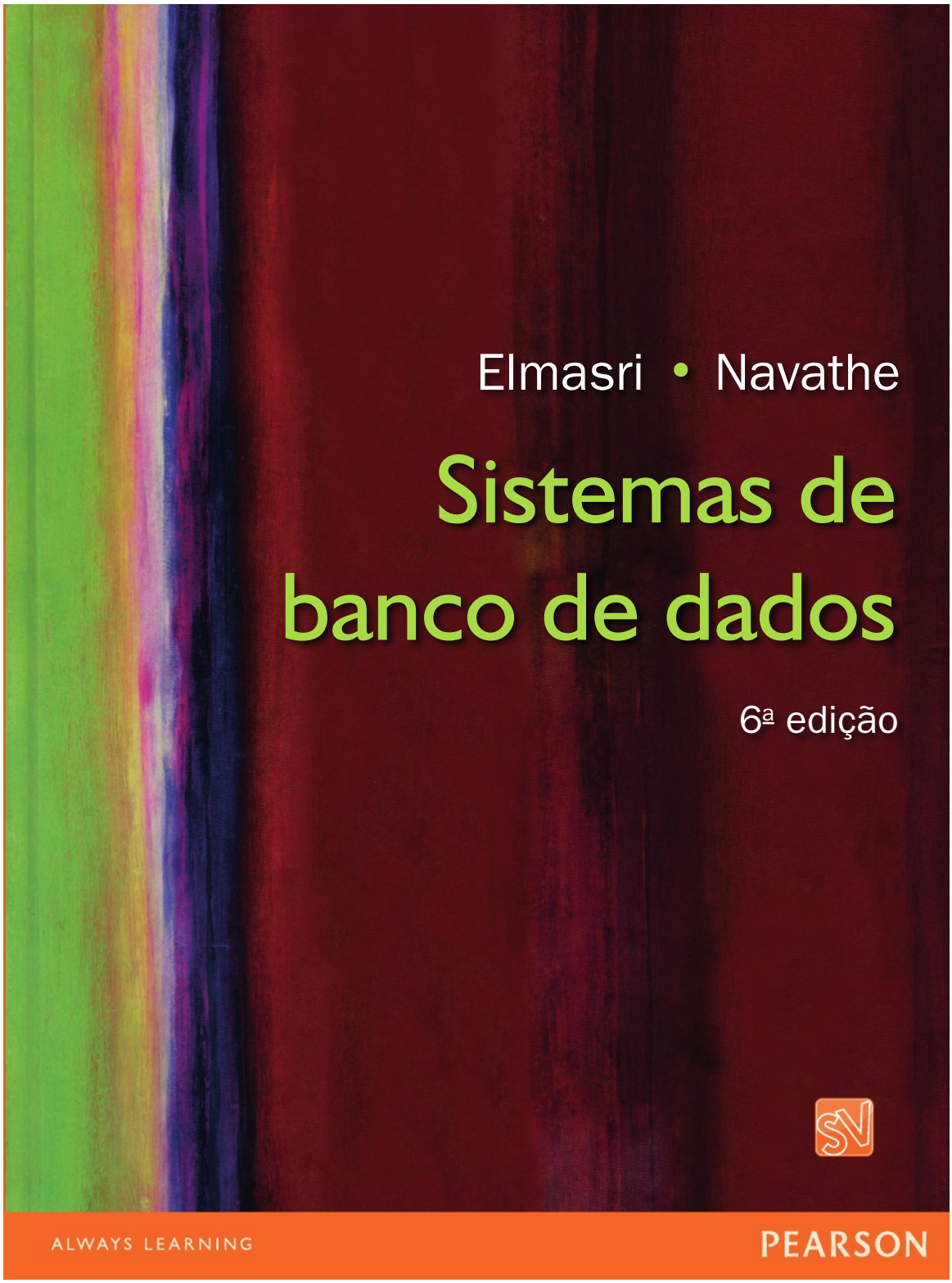
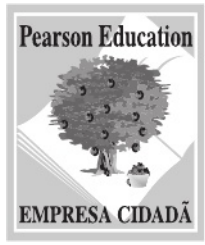

Elmasri • Navathe
Sistemas de banco de dados
6ª edição
ALWAYS LEARNING
PEARSON

Sistemas de banco de dados

6ª edição



Elmasri • Navathe

Sistemas de banco de dados

6ª edição

Ramez Elmasri
Departamento de Ciência da Computação e Engenharia
Universidade do Texas em Arlington
Shamkant B. Navathe
Faculdade de Computação
Georgia Institute of Technology

Tradução:
Daniel Vieira
Revisão técnica:
Enzo Seraphim
Thatyana de Faria Piola Seraphim

Professores Doutores do Instituto de Engenharia de Sistemas e Tecnologias da Informação — Universidade Federal de Itajubá

PEARSON

abdr
Respeite o direito autoral

© 2011 by Pearson Education do Brasil.
© 2011, 2007, 2004, 2000, 1994 e 1989 Pearson Education, Inc.

Tradução autorizada a partir da edição srcinal, em inglês, *Fundamentals of Database Systems* , 6th edition, publicada pela P earson Education, Inc., sob o selo Addison-Wesley.

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, da Pearson Education do Brasil.

Diretor editorial: Roger Trimer
Gerente editorial: Sabrina Cairo
Supervisor de produção editorial: Marcelo Françozo
Editora plena: Thelma Babaoka
Editora de texto: Sabrina Levensteinas
Preparação: Paula Brandão Perez Mendes
Revisão: Elisa Andrade Buzzo
Capa: Thyago Santos sobre o projeto srcinal de Lou Gibbs/Getty Images
Projeto gráfico e diagramação: Globaltec Artes Gráficas Ltda.

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Elmasri, Ramez
Sistemas de banco de dados / Ramez Elmasri e Shamkant B. Navathe ; tradução Daniel Vieira ; revisão técnica Enzo Seraphim e Thatyana de Faria Piola Seraphim. -- 6. ed. -- São Paulo : Pearson Addison Wesley, 2011.

Título srcinal: Fundamentals of database systems.

ISBN 978-85-4301-381-7

1. Banco de dados I. Navathe, Shamkant B..II. Título.

10-11462 CDD-005.75

Índices para catálogo sistemático:
1. Banco de dados : Sistemas : Processamento de dados 005.75
2. Banco de dados : Fundamentos : Processamento de dados 005.75

3ª reimpressão – Julho 2014
Direitos exclusivos para a língua portuguesa cedidos à
Pearson Education do Brasil Ltda.,
uma empresa do grupo Pearson Education
Rua Nelson Francisco, 26
CEP 02712-100 – São Paulo – SP – Brasil
Fone: (11) 2178-8686 – Fax: (11) 2178-8688
vendas@pearson.com

Para Katrina, Thomas e Dora (e também para Vicky)
R.E.

Para minha esposa Aruna, mãe Vijaya e para minha família inteira, por seu amor e apoio
S.B.N.

Sumário

Parte 1: Introdução aos bancos de dados 1

Capítulo 1 Bancos de dados e usuários de banco de dados 2

1.1 Introdução 2
1.2 Um exemplo 4
1.3 Características da abordagem de banco de dados 6
1.4 Atores em cena 9
1.5 Trabalhadores dos bastidores 10
1.6 Vantagens de usar a abordagem de SGBD 11
1.7 Uma breve história das aplicações de banco de dados 15
1.8 Quando não usar um SGBD 17
Resumo 18
Perguntas de revisão 18
Exercícios 18
Bibliografia selecionada 18

Capítulo 2 Conceitos e arquitetura do sistema de banco de dados 19

2.1 Modelos de dados, esquemas e instâncias 19
2.2 Arquitetura de três esquemas e independência de dados 22
2.3 Linguagens e interfaces do banco de dados ... 24
2.4 O ambiente do sistema de banco de dados ... 26
2.5 Arquiteturas centralizadas e cliente/servidor para SGBDs 29
2.6 Classificação dos sistemas gerenciadores de banco de dados 32
Resumo 35
Perguntas de revisão 36
Exercícios 36
Bibliografia selecionada 36

Parte 2: Modelo de dados relacional e SQL .. 37

Capítulo 3 O modelo de dados relacional e as restrições em bancos de dados relacionais 38

3.1 Conceitos do modelo relacional 39
3.2 Restrições em modelo relacional e esquemas de bancos de dados relacionais 44
3.3 Operações de atualização, transações e tratamento de violações de restrição 50
Resumo 52
Perguntas de revisão 53
Exercícios 53
Bibliografia selecionada 56

Capítulo 4 SQL básica 57

4.1 Definições e tipos de dados em SQL 58
4.2 Especificando restrições em SQL 61
4.3 Consultas de recuperação básicas em SQL .. 63
4.4 Instruções INSERT, DELETE e UPDATE em SQL 70
4.5 Recursos adicionais da SQL 72
Resumo 73
Perguntas de revisão 73
Exercícios 74
Bibliografia selecionada 75

Capítulo 5 Mais SQL: Consultas complexas, triggers, views e modificação de esquema 76

5.1 Consultas de recuperação SQL mais complexas 76

5.2 Especificando restrições como asserções e ações como triggers 87
5.3 Visões (views) — Tabelas virtuais em SQL ..88
5.4 Instruções de alteração de esquema em SQL ..91
Resumo 92
Perguntas de revisão 93
Exercícios 94
Bibliografia selecionada 94

Capítulo 6 Álgebra e cálculo relacional 96

6.1 Operações relacionais unárias: SELEÇÃO e PROJEÇÃO 97
6.2 Operações de álgebra relacional com base na teoria dos conjuntos 101
6.3 Operações relacionais binárias: JUNÇÃO e DIVISÃO 105
6.4 Outras operações relacionais 110
6.5 Exemplos de consultas na álgebra relacional 115
6.6 O cálculo relacional de tupla 116
6.7 O cálculo relacional de domínio 123
Resumo 124
Perguntas de revisão 125
Exercícios 125
Exercícios de laboratório 128
Bibliografia selecionada 129

Parte 3: Modelagem conceitual e projeto de banco de dados 130

Capítulo 7 Modelagem de dados usando o modelo Entidade-Relacionamento (ER) 131

7.1 Usando modelos de dados conceituais de alto nível para o projeto do banco de dados .. 132
7.2 Exemplo de aplicação de banco de dados ..133
7.3 Tipos de entidade, conjuntos de entidades, atributos e chaves 134
7.4 Tipos e conjuntos de relacionamentos, papéis e restrições estruturais 140
7.5 Tipos de entidade fraca 144
7.6 Refinando o projeto de ER para o banco de dados EMPRESA 145
7.7 Diagramas ER, convenções de nomes e questões de projeto 146
7.8 Exemplo de outra notação: diagramas de classes UML 148
7.9 Tipos de relacionamento de grau maior que dois 151
Resumo 154
Perguntas de revisão 154
Exercícios 155
Exercícios de laboratório 159
Bibliografia selecionada 160

Capítulo 8 O modelo Entidade-Relacionamento Estendido (EER) 161

8.1 Subclasses, superclasses e herança 162
8.2 Especialização e generalização 163
8.3 Restrições e características das hierarquias de especialização e generalização 165
8.4 Modelagem dos tipos UNIÃO usando categorias 170
8.5 Um exemplo de esquema UNIVERSIDADE de EER, opções de projeto e definições formais 172
8.6 Exemplo de outra notação: representando especialização e generalização em diagramas de classes em UML 175
8.7 Conceitos de abstração de dados, representação do conhecimento e ontologia 176
Resumo 181
Perguntas de revisão 181
Exercícios 181
Exercícios de laboratório 186
Bibliografia selecionada 188

Capítulo 9 Projeto de banco de dados relacional por mapeamento ER e EER para relacional 189

9.1 Projeto de banco de dados relacional usando o mapeamento ER para relacional 189
9.2 Mapeando construções do modelo EER para relações 195
Resumo 198
Perguntas de revisão 199
Exercícios 199
Exercícios de laboratório 200
Bibliografia selecionada 200

Capítulo 10 Metodologia prática de projeto de banco de dados e uso de diagramas UML 201

10.1 O papel dos sistemas de informação nas organizações 202
10.2 O projeto de banco de dados e o processo de implementação 205

10.3 Uso de diagramas UML como recurso para especificação de projeto de banco de dados ... 219
10.4 Rational Rose: uma ferramenta de projeto baseada em UML ... 225
10.5 Ferramentas de projeto automatizado de banco de dados ... 229
Resumo ... 231
Perguntas de revisão ... 232
Bibliografia selecionada ... 233

Parte 4: Objeto, objeto-relacional e XML: conceitos, modelos, linguagens e padrões ... 235

Capítulo 11 Bancos de dados de objeto e objeto-relacional ... 236
11.1 Visão geral dos conceitos de banco de dados de objeto ... 237
11.2 Recursos objeto-relacional: extensões do banco de dados de objeto para SQL ... 247
11.3 O modelo de objeto ODMG e a Object Definition Language (ODL) ... 252
11.4 Projeto conceitual de banco de dados de objeto ... 267
11.5 A linguagem de consulta de objeto (OQL — Object Query Language) ... 269
11.6 Visão geral de binding da linguagem C++ no padrão ODMG ... 275
Resumo ... 276
Perguntas de revisão ... 277
Exercícios ... 277
Bibliografia selecionada ... 278

Capítulo 12 XML: Extensible Markup Language ... 279
12.1 Dados estruturados, semiestruturados e não estruturados ... 280
12.2 Modelo de dados hierárquico (em árvore) da XML ... 283
12.3 Documentos XML, DTD e esquema XML ... 284
12.4 Armazenando e extraindo documentos XML de bancos de dados ... 291
12.5 Linguagens XML ... 292
12.6 Extraindo documentos XML de bancos de dados relacionais ... 295
Resumo ... 300
Perguntas de revisão ... 300
Exercícios ... 300
Bibliografia selecionada ... 300

Parte 5: Técnicas de programação de banco de dados ... 301

Capítulo 13 Introdução às técnicas de programação SQL ... 302
13.1 Programação de banco de dados: técnicas e problemas ... 303
13.2 SQL embutida, SQL dinâmica e SQLJ ... 305
13.3 Programação de banco de dados com chamadas de função: SQL/CLI e JDBC ... 314
13.4 Procedimentos armazenados de banco de dados e SQL/PSM ... 320
13.5 Comparando as três técnicas ... 322
Resumo ... 323
Perguntas de revisão ... 323
Exercícios ... 323
Bibliografia selecionada ... 324

Capítulo 14 Programação de banco de dados Web usando PHP ... 325
14.1 Um exemplo simples em PHP ... 325
14.2 Visão geral dos recursos básicos da PHP ... 327
14.3 Visão geral da programação de banco de dados em PHP ... 332
Resumo ... 335
Perguntas de revisão ... 335
Exercícios ... 335
Bibliografia selecionada ... 335

Parte 6: Teoria e normalização de projeto de banco de dados ... 336

Capítulo 15 Fundamentos de dependências funcionais e normalização para bancos de dados relacionais ... 337
15.1 Diretrizes de projeto informais para esquemas de relação ... 338
15.2 Dependências funcionais ... 346
15.3 Formas normais baseadas em chaves primárias ... 347
15.4 Definições gerais da segunda e terceira formas normais ... 353
15.5 Forma normal de Boyce-Codd ... 355
15.6 Dependência multivalorada e quarta forma normal ... 357
15.7 Dependências de junção e quinta forma normal ... 359
Resumo ... 360
Perguntas de revisão ... 360

Exercícios 361
Exercício de laboratório 363
Bibliografia selecionada 363

Capítulo 16 Algoritmos de projeto de banco de dados relacional e demais dependências 364

16.1 Outros tópicos em dependências funcionais: regras de inferência, equivalência e cobertura mínima 365
16.2 Propriedades de decomposições relacionais 369
16.3 Algoritmos para projeto de esquema de banco de dados relacional 374
16.4 Sobre nulos, tuplas suspensas e projetos relacionais alternativos 377
16.5 Discussão adicional sobre dependências multivaloradas e 4FN 381
16.6 Outras dependências e formas normais .. 383
Resumo 385
Perguntas de revisão 386
Exercícios 386
Exercícios de laboratório 387
Bibliografia selecionada 387

Parte 7: Estruturas de arquivo, indexação e hashing 388

Capítulo 17 Armazenamento de disco, estruturas de arquivo básicas e hashing 389

17.1 Introdução 389
17.2 Dispositivos de armazenamento secundários 392
17.3 Buffering de blocos 397
17.4 Gravando registros de arquivo no disco .. 397
17.5 Operações em arquivos 401
17.6 Arquivos de registros desordenados (arquivos de heap) 403
17.7 Arquivos de registros ordenados (arquivos classificados) 404
17.8 Técnicas de hashing 406
17.9 Outras organizações de arquivo primárias 414
17.10 Paralelizando o acesso de disco usando tecnologia RAID 415
17.11 Novos sistemas de armazenamento ... 418
Resumo 419
Perguntas de revisão 420
Exercícios 421
Bibliografia selecionada 423

Capítulo 18 Estruturas de indexação para arquivos 424

18.1 Tipos de índices ordenados de único nível .. 424
18.2 Índices multiníveis 433
18.3 Índices multiníveis dinâmicos usando B-trees e B+-trees 435
18.4 Índices em chaves múltiplas 445
18.5 Outros tipos de índices 447
18.6 Algumas questões gerais referentes à indexação 450
Resumo 452
Perguntas de revisão 453
Exercícios 453
Bibliografia selecionada 455

Parte 8: Processamento de consulta, otimização e ajuste de banco de dados 456

Capítulo 19 Algoritmos para processamento e otimização de consulta 457

19.1 Traduzindo consultas SQL para álgebra relacional 459
19.2 Algoritmos para ordenação externa 459
19.3 Algoritmos para operações SELEÇÃO e JUNÇÃO 461
19.4 Algoritmos para operações PROJEÇÃO e de conjunto 469
19.5 Implementando operações de agregação e JUNÇÃO EXTERNA 470
19.6 Combinando operações com pipelining .. 472
19.7 Usando a heurística na otimização da consulta 472
19.8 Usando seletividade e estimativas de custo na otimização da consulta 479
19.9 Visão geral da otimização da consulta no Oracle 487
19.10 Otimização de consulta semântica 487
Resumo 488
Perguntas de revisão 488
Exercícios 488
Bibliografia selecionada 489

Capítulo 20 Projeto físico e ajuste de banco de dados 490

20.1 Projeto físico em bancos de dados relacionais 490
20.2 Visão geral do ajuste de banco de dados em sistemas relacionais 494
Resumo 498

Perguntas de revisão 498
Bibliografia selecionada 498

Parte 9: Processamento de transações, controle de concorrência e recuperação 499

Capítulo 21 Introdução aos conceitos e teoria de processamento de transações ... 500

21.1 Introdução ao processamento de transações 500
21.2 Conceitos de transação e sistema 506
21.3 Propriedades desejáveis das transações 508
21.4 Caracterizando schedules com base na facilidade de recuperação 509
21.5 Caracterizando schedules com base na facilidade de serialização 511
21.6 Suporte para transação em SQL 519
Resumo 520
Perguntas de revisão 520
Exercícios 521
Bibliografia selecionada 522

Capítulo 22 Técnicas de controle de concorrência 523

22.1 Técnicas de bloqueio em duas fases para controle de concorrência 523
22.2 Controle de concorrência baseado na ordenação de rótulo de tempo (timestamp) 531
22.3 Técnicas de controle de concorrência multiversão 533
22.4 Técnicas de controle de concorrência de validação (otimista) 535
22.5 Granularidade dos itens de dados e bloqueio de granularidade múltiplo 536
22.6 Usando bloqueios para controle de concorrência em índices 539
22.7 Outras questões de controle de concorrência 539
Resumo 540
Perguntas de revisão 541
Exercícios 542
Bibliografia selecionada 542

Capítulo 23 Técnicas de recuperação de banco de dados 543

23.1 Conceitos de recuperação 543
23.2 Recuperação NO-UNDO/REDO baseada em atualização adiada 549
23.3 Técnicas de recuperação baseadas em atualização imediata 550
23.4 Paginação de sombra 552
23.5 O algoritmo de recuperação ARIES 553
23.6 Recuperação em sistemas de múltiplos bancos de dados 556
23.7 Backup e recuperação de banco de dados contra falhas catastróficas 557
Resumo 557
Perguntas de revisão 558
Exercícios 558
Bibliografia selecionada 560

Parte 10: Tópicos adicionais de banco de dados: segurança e distribuição 561

Capítulo 24 Segurança de banco de dados ... 562

24.1 Introdução a questões de segurança de banco de dados 562
24.2 Controle de acesso discricionário baseado na concessão e revogação de privilégios 567
24.3 Controle de acesso obrigatório e controle de acesso baseado em papel para segurança multinível 570
24.4 Injeção de SQL 575
24.5 Introdução à segurança do banco de dados estatístico 577
24.6 Introdução ao controle de fluxo 579
24.7 Criptografia e infraestruturas de chave pública 580
24.8 Questões de privacidade e preservação 582
24.9 Desafios da segurança do banco de dados 583
24.10 Segurança baseada em rótulo no Oracle 584
Resumo 586
Perguntas de revisão 587
Exercícios 587
Bibliografia selecionada 588

Capítulo 25 Bancos de dados distribuídos ... 589

25.1 Conceitos de banco de dados distribuído 590
25.2 Tipos de sistemas de bancos de dados distribuídos 593
25.3 Arquiteturas de banco de dados distribuídas 596
25.4 Técnicas de fragmentação, replicação e alocação de dados para projeto de banco de dados distribuído 601
25.5 Processamento e otimização de consulta em bancos de dados distribuídos 605
25.6 Visão geral do gerenciamento de transação em bancos de dados distribuídos 611

25.7 Visão geral do controle de concorrência e recuperação em bancos de dados distribuídos 613
25.8 Gerenciamento de catálogo distribuído .. 615
25.9 Tendências atuais em bancos de dados distribuídos 615
25.10 Bancos de dados distribuídos em Oracle 617
Resumo 620
Perguntas de revisão 621
Exercícios 622
Bibliografia selecionada 623

Parte 11: Modelos, sistemas e aplicações de bancos de dados avançados 625

Capítulo 26 Modelos de dados avançados para aplicações avançadas ... 626
26.1 Conceitos de banco de dados ativo e triggers 627
26.2 Conceitos de banco de dados temporal .. 635
26.3 Conceitos de banco de dados espacial ... 645
26.4 Conceitos de banco de dados multimídia . 650
26.5 Introdução aos bancos de dados dedutivos 653
Resumo 663
Perguntas de revisão 664
Exercícios 665
Bibliografia selecionada 667

Capítulo 27 Introdução à recuperação de informações e busca na Web. 669
27.1 Conceitos de recuperação de informações (RI) 669
27.2 Modelos de recuperação 674
27.3 Tipos de consultas em sistemas de RI .. 679
27.4 Pré-processamento de textos 680
27.5 Indexação invertida 682
27.6 Medidas de avaliação de relevância da pesquisa 684
27.7 Pesquisa e análise na Web 686
27.8 Tendências na recuperação de informações 693
Resumo 694
Perguntas de revisão 695
Bibliografia selecionada 696

Capítulo 28 Conceitos de mineração de dados 698
28.1 Visão geral da tecnologia de mineração de dados 698
28.2 Regras de associação 701
28.3 Classificação 709
28.4 Agrupamento 712
28.5 Abordagens para outros problemas de mineração de dados 713
28.6 Aplicações de mineração de dados 715
28.7 Ferramentas comerciais de mineração de dados 716
Resumo 717
Perguntas de revisão 718
Exercícios 718
Bibliografia selecionada 719

Capítulo 29 Visão geral de data warehousing e OLAP 720
29.1 Introdução, definições e terminologia .. 720
29.2 Características dos data warehouses 721
29.3 Modelagem de dados para data warehouses 722
29.4 Criando um data warehouse 726
29.5 Funcionalidade típica de um data warehouse 728
29.6 Data warehouses *versus* visões 729
29.7 Dificuldades de implementação de data warehouses 729
Resumo 730
Perguntas de revisão 731
Bibliografia selecionada 731

Apêndice A Notações diagramáticas alternativas para modelos ER .. 732

Apêndice B Parâmetros de discos 735

Apêndice C Visão geral da linguagem QBE ... 737
C.1 Recuperações básicas em QBE 737
C.2 Agrupamento, agregação e modificação de banco de dados em QBE 740

Bibliografia 744

Índice remissivo 766

Prefácio

Este livro é uma introdução aos conceitos fundamentais necessários para projetar, usar e implementar sistemas de banco de dados e aplicações de banco de dados. Nossa apresentação enfatiza os fundamentos da modelagem e do projeto de banco de dados, as linguagens e os modelos fornecidos pelos sistemas de gerenciamento de banco de dados e as técnicas de implementação do sistema de banco de dados. O propósito do livro é que ele seja usado como um livro-texto para a disciplina de sistemas de banco de dados e como livro de referência. Nosso objetivo é oferecer uma apresentação profunda e atualizada dos aspectos mais importantes dos sistemas e aplicações de banco de dados, bem como das tecnologias relacionadas. Consideramos que os leitores estejam acostumados aos conceitos elementares da programação e estruturação de dados, e que eles tenham tido algum contato com os fundamentos de organização do computador.

Novidades

Os recursos-chave a seguir foram acrescentados nessa edição:

- Uma reorganização da ordem dos capítulos, permitindo que os professores comecem com projetos e exercícios de laboratório bem mais cedo no curso.
- O material sobre SQL, padrão de banco de dados relacional, foi antecipado no livro para os capítulos 4 e 5, para permitir que os professores focalizem esse importante assunto no início do curso.
- O material sobre bancos de dados objeto-relacional e orientado a objeto foi atualizado para que se adapte aos padrões SQL e ODMG mais recentes, e consolidado em um único capítulo (Capítulo 11).
- A apresentação da XML foi expandida e atualizada, sendo antecipada no livro para o Capítulo 12.
- Os capítulos sobre teoria de normalização foram reorganizados de modo que o primeiro capítulo (Capítulo 15) focaliza os conceitos de normalização intuitivos, enquanto o segundo capítulo (Capítulo 16) aborda as teorias formais e os algoritmos de normalização.
- A apresentação das ameaças à segurança do banco de dados foi atualizada com uma discussão sobre ataques de Injeção de SQL e técnicas de prevenção no Capítulo 24, e uma visão geral da segurança baseada em rótulo, com exemplos.
- Nossa apresentação sobre bancos de dados espaciais e bancos de dados de multimídia foi expandida e atualizada no Capítulo 26.
- Um novo Capítulo 27, sobre técnicas de recuperação de informações, foi acrescentado, discutindo sobre modelos e técnicas de recuperação, consulta, navegação e indexação de informações a partir de documentos da Web; apresentamos as etapas de processamento típicas em um sistema de recuperação de informações, as medidas de avaliação e como as técnicas de recuperação de informação estão relacionadas aos bancos de dados e à pesquisa na Web.

Organização desta edição

Existem mudanças organizacionais significativas nesta edição, bem como melhoria nos capítulos indi-

viduais. O livro agora é dividido em onze partes, da seguinte forma:

- Parte 1 (capítulos 1 e 2) inclui os capítulos introdutórios.
- A apresentação sobre bancos de dados relacionais e SQL foi movida para a Parte 2 (capítulos de 3 a 6) do livro; o Capítulo 3 apresenta o modelo relacional formal e as restrições do banco de dados relacional; o material sobre SQL (capítulos 4 e 5) agora é apresentado antes da nossa apresentação sobre álgebra e cálculo relacional no Capítulo 6, para permitir que os professores iniciem projetos em SQL mais cedo em um curso, se desejarem (essa reordenação também é baseada em um estudo que sugere que os alunos dominam SQL melhor quando ela é ensinada antes das linguagens relacionais formais).
- A apresentação sobre modelagem de entidade-relacionamento e projeto de banco de dados agora está na Parte 3 (capítulos 7 a 10), mas ainda pode ser abordada antes da Parte 2, se o foco do curso for o projeto de banco de dados.
- A Parte 4 aborda o material atualizado sobre bancos de dados objeto-relacional e orientado a objeto (Capítulo 11) e XML (Capítulo 12).
- A Parte 5 inclui os capítulos sobre técnicas de programação de banco de dados (Capítulo 13) e programação de banco de dados na Web usando PHP (Capítulo 14, que foi antecipado no livro).
- A Parte 6 (capítulos 15 e 16) inclui os capítulos sobre normalização e teoria de projeto (passamos todos os aspectos formais dos algoritmos de normalização para oCapítulo 16).
- A Parte 7 (capítulos 17 e 18) contém os capítulos sobre organizações de arquivo, indexação e hashing.
- A Parte 8 inclui os capítulos sobre técnicas de processamento e otimização de consulta (Capítulo 19) e ajuste de banco de dados (Capítulo 20).
- A Parte 9 inclui o Capítulo 21 sobre conceitos de processamento de transação; o Capítulo 22 sobre controle de concorrência; e o Capítulo 23 sobre recuperação do banco de dados contra falhas.
- A Parte 10, sobre tópicos adicionais de banco de dados, inclui o Capítulo 24 sobre segurança de banco de dados e o Capítulo 25 sobre bancos de dados distribuídos.
- A Parte 11 sobre modelos e aplicações avançadas de banco de dados inclui o Capítulo 26 sobre modelos de dados avançados (bancos de dados ativos, temporais, espaciais, multimídia e dedutivos); o novo Capítulo 27 sobre recuperação de informações e busca na Web; e os capítulos sobre mineração de dados (Capítulo 28) e data warehousing (Capítulo 29).

Conteúdo desta edição

A Parte 1 descreve os conceitos introdutórios básicos necessários para um bom conhecimento dos modelos de banco de dados, sistemas e linguagens. Os capítulos 1 e 2 introduzem bancos de dados, usuários típicos e conceitos, terminologia e arquitetura de SGBD.

A Parte 2 descreve o modelo de dados relacional, o padrão SQL e as linguagens relacionais formais. O Capítulo 3 descreve o modelo relacional básico, suas restrições de integridade e operações de atualização. O Capítulo 4 descreve algumas partes básicas do padrão SQL para bancos de dados relacionais, incluindo definição de dados, operações de modificação de dados e consultas SQL simples. O Capítulo 5 apresenta consultas SQL mais complexas, bem como os conceitos SQL de triggers, asserções, visões e modificação de esquema. O Capítulo 6 descreve as operações da álgebra relacional e introduz o cálculo relacional.

A Parte 3 aborda vários tópicos relacionados à modelagem conceitual do banco de dados e o projeto de banco de dados. No Capítulo 7, os conceitos do modelo Entidade-Relacionamento (ER) e diagramas ER são apresentados e usados para ilustrar o projeto conceitual do banco de dados. O Capítulo 8 focaliza os conceitos de abstração de dados e modelagem semântica dos dados, mostrando como o modelo ER pode ser estendido para incorporar essas ideias, levando ao modelo de dados ER-Estendido (EER) e diagramas EER. Os conceitos apresentados no Capítulo 8 incluem subclasses, especialização, generalização e tipos (categorias) de união. A notação para os diagramas de classe da UML também é apresentada nos capítulos 7 e 8. O Capítulo 9 discute o projeto de banco de dados relacional usando o mapeamento ER e EER para relacional. Terminamos a Parte 3 com o Capítulo 10, que apresenta uma visão geral das diferentes fases do processo de projeto de banco de dados nas empresas para aplicações de banco de dados de tamanho médio e grande.

A Parte 4 aborda os modelos de dados orientados a objeto, objeto-relacional e XML, e suas linguagens e padrões afiliados. O Capítulo 11 introduz os

conceitos para bancos de dados de objeto e mostra como eles foram incorporados ao padrão SQL a fim de acrescentar capacidades de objeto aos sistemas de bancos de dados relacionais. Depois, aborda o padrão do modelo de objeto ODMG e sua definição de objeto e linguagens de consulta. O Capítulo 12 aborda o modelo e linguagens XML (eXtensible Markup Language), discutindo como a XML está relacionada aos sistemas de banco de dados. Apresenta os conceitos e linguagens do modelo XML, comparando-o com modelos de banco de dados tradicionais. Também mostra como os dados podem ser convertidos entre a XML e representações relacionais.

A Parte 5 é sobre técnicas de programação de banco de dados. O Capítulo 13 aborda os tópicos de programação SQL, como SQL embutida, SQL dinâmica, ODBC, SQLJ, JDBC e SQL/CLIENTE. O Capítulo 14 introduz a programação de banco de dados na Web, usando a linguagem de scripting PHP em nossos exemplos.

A Parte 6 aborda a teoria da normalização. Os capítulos 15 e 16 abordam os formalismos, as teorias e os algoritmos desenvolvidos para o projeto de banco de dados relacional por normalização. Esse material inclui dependências funcionais e outros tipos de dependências e formas normais das relações. A normalização intuitiva passo a passo é apresentada no Capítulo 15, que também define dependências multivaloradas e de junção. Os algoritmos de projeto relacional baseados na normalização, junto com o material teórico em que os algoritmos são baseados, são apresentados no Capítulo 16.

A Parte 7 descreve as estruturas de arquivo físicas e os métodos de acesso usados nos sistemas de banco de dados. O Capítulo 17 descreve os principais métodos de organização de arquivos de registros em disco, incluindo o hashing estático e dinâmico. O Capítulo 18 descreve as técnicas de indexação para arquivos, incluindo estruturas de dados em árvore B e B^+ e arquivos de grade.

A Parte 8 focaliza o processamento de consulta e o ajuste de desempenho de banco de dados. O Capítulo 19 apresenta os fundamentos do processamento e otimização de consulta, e o Capítulo 20 discute sobre o projeto físico e ajuste de banco de dados.

A Parte 9 discute o processamento de transações, controle de concorrência e técnicas de recuperação, incluindo discussões de como esses conceitos são realizados em SQL. O Capítulo 21 introduz as técnicas necessárias para os sistemas de processamento de transação e define os conceitos de facilidade de recuperação e serialização dos schedules. O Capítulo 22 oferece uma visão geral dos vários tipos de protocolos de controle de concorrência, com foco no bloqueio em duas fases. Também trata das técnicas de ordenação de timestamp (rótulos de tempo) e controle de concorrência otimista, além do bloqueio de granularidade múltiplo. Finalmente, o Capítulo 23 focaliza os protocolos de recuperação de banco de dados e oferece uma visão geral dos conceitos e técnicas que são usadas nessa recuperação.

As partes 10 e 11 abordam diversos tópicos avançados. O Capítulo 24 oferece uma visão geral da segurança do banco de dados, incluindo o modelo de controle de acesso discricionário com comandos SQL para o GRANT e o REVOKE de privilégios, o modelo de controle de acesso obrigatório com categorias de usuários e poli-instanciação, uma discussão sobre privacidade de dados e seu relacionamento com segurança, e uma visão geral dos ataques de Injeção de SQL. O Capítulo 25 oferece uma introdução aos bancos de dados distribuídos e discute a arquitetura cliente/servidor em três camadas. O Capítulo 26 introduz vários modelos de banco de dados avançados para aplicações avançadas. Estes incluem bancos de dados ativos e triggers, além de bancos de dados temporais, espaciais, multimídia e dedutivos. O Capítulo 27 é um capítulo novo, sobre técnicas de recuperação de informações e como elas estão relacionadas a sistemas de banco de dados e a métodos de pesquisa na Web. O Capítulo 28, sobre mineração de dados (*data mining*), oferece uma visão geral do processo de mineração de dados e descoberta de conhecimento, discute algoritmos para mineração, classificação e agrupamento de regra de associação, e aborda rapidamente outras técnicas e ferramentas comerciais. O Capítulo 29 introduz os conceitos de data warehousing e OLAP.

O Apêndice A oferece uma série de notações diagramáticas alternativas para exibir um esquema ER ou EER conceitual. Estas podem ser usadas em substituição à notação que usamos, se o professor preferir. O Apêndice B oferece alguns parâmetros físicos importantes de discos. O Apêndice C oferece uma visão geral da linguagem de consulta gráfica QBE. Os apêndices D e E (disponíveis na Sala Virtual, sv.pearson.com.br) abordam sistemas de banco de dados legados, com base nos modelos de banco de dados hierárquico e de rede. Eles têm sido usados há mais de trinta anos como base para muitas aplicações de banco de dados e sistemas de processamento de transações comerciais. Consideramos importante expor os alunos de gerenciamento de banco de dados a essas técnicas legadas de modo que possam ter uma ideia melhor de como a tecnologia de banco de dados progrediu.

Orientações

Existem muitas maneiras diferentes de ministrar um curso de banco de dados. Os capítulos das partes 1 a 7 podem ser usados em um curso introdutório sobre sistemas de banco de dados, na ordem em que aparecem ou na ordem preferida dos professores. Capítulos e seções selecionadas podem ser omitidas, e o professor pode acrescentar outros capítulos do restante do livro, dependendo da ênfase do curso. Ao final da seção inicial de muitos dos capítulos do livro, listamos seções que são candidatas a serem omitidas sempre que uma discussão menos detalhada do assunto for desejada. Sugerimos incluir até o Capítulo 15 em um curso introdutório de banco de dados e incluir partes selecionadas de outros capítulos, dependendo da base dos alunos e da cobertura desejada. Para uma ênfase em técnicas de implementação de sistemas, os capítulos das partes 7, 8 e 9 devem substituir alguns dos capítulos anteriores.

Os capítulos 7 e 8, que abordam a modelagem conceitual usando os modelos ER e EER, são importantes para um bom conhecimento conceitual dos bancos de dados. Porém, eles podem ser abordados parcialmente ou mais adiante em um curso ou até mesmo omitidos, se a ênfase for sobre a implementação do SGBD. Os capítulos 17 e 18, sobre organizações de arquivos e indexação, também podem ser abordados mais cedo, mais tarde ou ainda omitidos, se a ênfase for sobre modelos de banco de dados e linguagens. Para alunos que concluíram um curso sobre organização de arquivos, partes desses capítulos podem ser indicadas como material de leitura ou alguns exercícios podem ser passados como revisão para esses conceitos.

Se a ênfase de um curso for em projeto de banco de dados, então o professor deverá abordar os capítulos 7 e 8 mais cedo, seguidos pela apresentação dos bancos de dados relacionais. Um curso sobre o ciclo de vida completo do projeto e implementação de bancos de dados incluiria o projeto conceitual (capítulos 7 e 8), bancos de dados relacionais (capítulos 3, 4 e 5), mapeamento do modelo de dados (Capítulo 9), normalização (Capítulo 15) e implementação de programas de aplicação com SQL (Capítulo 13).O Capítulo 14 também deverá ser abordado se a ênfase for em programação e aplicações de banco de dados naWeb. A documentação adicional sobre linguagens de programação e SGBDRs específicos seria necessária.

O livro foi escrito de modo a possibilitar a abordagem de tópicos em várias sequências. O gráfico de dependência dos capítulos, a seguir, mostra as principais dependências entre os capítulos. Conforme o diagrama ilustra, é possível começar com vários tópicos diferentes após os dois capítulos introdutórios. Embora o gráfico possa parecer complexo, é importante observar que, se os capítulos forem usados na ordem, as dependências não serão perdidas. O gráfico pode ser consultado por aqueles que desejam usar uma ordem de apresentação alternativa.

Para um curso de um semestre baseado neste livro, capítulos selecionados podem ser atribuídos como material de leitura. O livro também pode ser usado para uma sequência de curso em dois semestres. O primeiro curso, *Introdução ao Projeto e Sistemas de Bancos de Dados* pode abranger a maioria dos capítulos 1 a 15. O segundo curso, *Modelos e Técnicas de Implementação de Bancos de Dados* pode abranger a maioria dos Capítulos de 16 a 29. A sequência de dois semestres também pode ser elaborada de várias outras maneiras, dependendo da preferência de cada professor.

Materiais adicionais

Na Sala Virtual (sv.pearson.com.br), professores e estudantes podem acessar materiais adicionais 24 horas por dia.

Para professores:

- Apresentações em PowerPoint.
- Banco de imagens.
- Manual de soluções (em inglês).

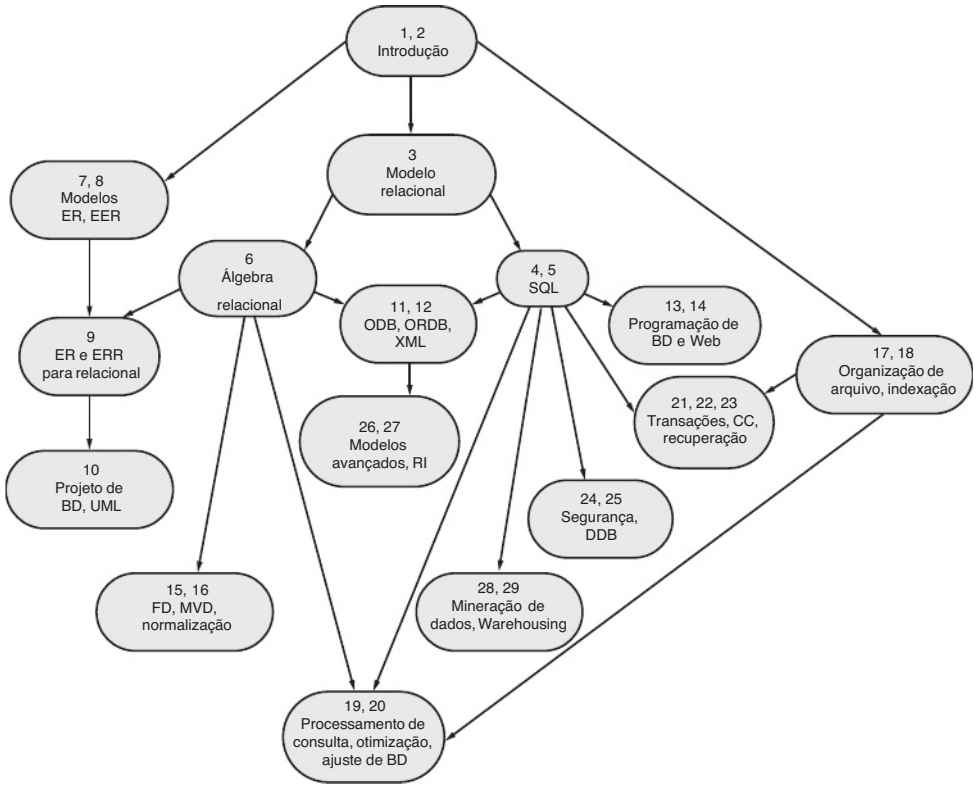
Esse material é de uso exclusivo para professores e está protegido por senha. Para ter acesso a ele, os professores que adotam o livro devem entrar em contato com seu representante Pearson ou enviar e-mail para universitarios@pearson.com.

Para estudantes:

- Manual de laboratório (em inglês).
- Apêndices D e E (em inglês).

Agradecimentos

É um grande prazer reconhecer o auxílio e as contribuições de muitos indivíduos para este esforço. Primeiro, gostaríamos de agradecer ao nosso editor, Matt Goldstein, por sua orientação, encorajamento e apoio. Gostaríamos de agradecer o excelente trabalho de Gillian Hall, pela gerência de produção, e Rebecca Greenberg, por uma revisão completa do livro. Agradecemos às seguintes pessoas da Pearson, que contribuíram para a sexta edição: Jeff Holcomb, Marilyn Lloyd, Margaret Waples e Chelsea Bell.



Sham Navathe gostaria de agradecer a contribuição significativa de Saurav Sahay para o Capítulo 27. Vários alunos atuais e do passado também contribuíram para os diversos capítulos nesta edição: Rafi Ahmed, Liora Sahar, Fariborz Farahmand, Nalini Polavarapu e Wanxia Xie (ex-alunos); e Bharath Rengarajan, Narsi Srinivasan, Parimala R. Pranesh, Neha Deodhar, Balaji Palanisamy e Hariprasad Kumar (alunos atuais).

Discussões com seus colegas Ed Omiecinski e Leo Mark, da Georgia Tech, e Venu Dasigi, da SPSU, Atlanta, também contribuíram para a revisão do material.

Gostaríamos de repetir nossos agradecimentos àqueles que revisaram e contribuíram de alguma maneira para as edições anteriores de *Sistemas de banco de dados*.

- **Primeira edição.** Alan Apt (editor), Don Batory, Scott Downing, Dennis Heimbinger, Julia Hodges, Yannis Ioannidis, Jim Larson, Per-Ake Larson, Dennis McLeod, Rahul Patel, Nicholas Roussopoulos, David Stemple, Michael Stonebraker, Frank Tompa e Kyu-Young Whang.
- **Segunda edição.** Dan Joraanstad (editor), Rafi Ahmed, Antonio Albano, David Beech, Jose Blakeley, Panos Chrysanthis, Suzanne Dietrich, Vic Ghorpadey, Goetz Graefe, Eric Hanson, Junguk L. Kim, Roger King, Vram Kouramajian, Vijay Kumar, John Lowther, Sanjay Manchanda, Toshimi Minoura, Inderpal Mumick, Ed Omiecinski, Girish Pathak, Raghu Ramakrishnan, Ed Robertson, Eugene Sheng, David Stotts, Marianne Winslett e Stan Zdonick.

- **Terceira edição.** Maite Suarez-Rivas e Katherine Harutunian (editores); Suzanne Dietrich, Ed Omiecinski, Rafi Ahmed, Francois Bancilhon, Jose Blakeley, Rick Cattell, Ann Chervenak, David W. Embley, Henry A. Etlinger, Leonidas Fegaras, Dan Forsyth, Farshad Fotouhi, Michael Franklin, Sreejith Gopinath, Goetz Craefe, Richard Hull, Sushil Jajodia, Ramesh K. Karne, Harish Kotbagi, Vijay Kumar, Tarcisio Lima, Ramon A. Mata-Toledo, Jack McCaw, Dennis McLeod, Rokia Missaoui, Magdi Morsi, M. Nakhaiswamy, Carlos Ordonez, Joan Peckham, Betty Salzberg, Ming-Chien Shan, Junping Sun, Rajshekhar Sunderraman, Aravindan Veerasamy e Emilia E. Villareal.
- **Quarta edição.** Maite Suarez-Rivas, Katherine Harutunian, Daniel Rausch e Juliet Silveri (editores); Phil Bernhard, Zhengxin Chen, Jan Chomicki, Hakan Ferhatosmanoglu, Len Fisk, William Hankley, Ali R. Hurson, Vijay Kumar, Peretz Shoval, Jason T. L. Wang (revisores); Ed Omiecinski (que contribuiu para o Capítulo 27). Os colaboradores da Universidade do Texas em Arlington são Jack Fu, Hyoil Han, Babak Hojabri, Charley Li, Ande Swathi e Steven Wu; os colaboradores da Georgia Tech são Weimin Feng, Dan Forsythe, Angshuman Guin, Abrar Ul-Haque, Bin Liu, Ying Liu, Wanxia Xie e Waigen Yee.
- **Quinta edição.** Matt Goldstein e Katherine Harutunian (editores); Michelle Brown, Gillian Hall, Patty Mahtani, Maite Suarez-Rivas, Bethany Tidd e Joyce Cosentino Wells (da Addison-Wesley); Hani Abu-Salem, Jamal R. Alsabbagh, Ramzi Bualuan, Soon Chung, Sumali Conlon, Hasan Davulcu, James Geller, J. B. Graubmald, Latifur Khan, Herman Lam, Byung S. Lee, Donald Sanderson, Jamil Saquer, Costas Tsatsoulis e Jack C. Wileden (revisores); Raj Sunderraman (que contribuiu com os projetos de laboratório); Salman Azar (que contribuíram com alguns exercícios novos); Gaurav Bhatia, Fariborz Farahmand, Ying Liu, Ed Omiecinski, Nalini Polavarapu, Liora Sahar, Saurav Sahay e Wanxia Xie (da Georgia Tech).

Por último, mas não menos importante, gostaríamos de agradecer nossas famílias pelo apoio, pelo encorajamento e pela paciência.

R.E.
S.B.N.

parte 1

Introdução aos bancos de dados

capítulo 1

Bancos de dados e usuários de banco de dados

Bancos de dados e sistemas de banco de dados são um componente essencial da vida na sociedade moderna; a maioria de nós encontra diariamente diversas atividades que envolvem alguma interação com um banco de dados. Por exemplo, quando vamos ao banco para depositar ou retirar fundos, fazemos uma reserva de hotel ou de voo, acessamos o catálogo de uma biblioteca virtual para procurar uma referência bibliográfica, ou compramos algo on-line — como um livro, um brinquedo ou um computador —, provavelmente essas atividades envolverão alguém ou algum programa de computador que acessa um banco de dados. Até mesmo a compra de produtos em um supermercado atualiza automaticamente o banco de dados que mantém o controle de estoque dos itens.

Essas interações são exemplos do que podemos chamar de **aplicações de banco de dados tradicionais**, em que a maior parte da informação armazenada e acessada é textual ou numérica. Nos últimos anos, os avanços na tecnologia levaram a interessantes novas aplicações dos sistemas de banco de dados. A nova tecnologia de mídia tornou possível armazenar imagens, clipes de áudio e streams de vídeo digitalmente. Esses tipos de arquivo estão se tornando um componente importante dos **bancos de dados de multimídia** . Os **sistemas de informações geográficas** (**GIS** — Geographic Information Systems) podem armazenar e analisar mapas, dados sobre o clima e imagens de satélite. Sistemas de **data warehousing** e de **processamento analítico on-line** (**OLAP** — On-Line Analytical Processing) são usados em muitas empresas para extrair e analisar informações comerciais úteis de bancos de dados muito grandes, para ajudar na tomada de decisão. A **tecnologia de tempo real** e **banco de dados ativo** é usada para controlar processos industriais e de manufatura. Além disso, técnicas de pesquisa de banco de dados estão sendo aplicadas à World Wide Web para melhorar a busca por informações necessárias feita pelos usuários que utilizam a Internet.

No entanto, para entender os fundamentos da tecnologia de banco de dados, devemos começar das aplicações básicas de banco de dados tradicional. Na Seção 1.1, começamos definindo um banco de dados, e depois explicamos outros termos básicos. Na Seção 1.2, oferecemos um simples exemplo de banco de dados UNIVERSIDADE para ilustrar nossa discussão. A Seção 1.3 descreve algumas das principais características dos sistemas de banco de dados, e as seções 1.4 e 1.5 classificam os tipos de pessoas cujas funções envolvem o uso e a interação com sistemas de banco de dados. As seções 1.6, 1.7 e 1.8 oferecem uma discussão mais profunda sobre as diversas capacidades oferecidas pelos sistemas de banco de dados e discutem algumas aplicações típicas. No final do capítulo é apresentado um resumo.

O leitor que quiser uma introdução rápida aos sistemas de banco de dados pode estudar as seções 1.1 a 1.5, depois pular ou folhear as seções 1.6 a 1.8 e seguir para o Capítulo 2.

1.1 Introdução

Os bancos de dados e sua tecnologia têm um impacto importante sobre o uso crescente dos computadores. É correto afirmar que os bancos de dados desempenham um papel crítico em quase todas as áreas em que os computadores são usados, incluindo negócios, comércio eletrônico, engenharia, medicina, genética, direito, educação e biblioteconomia. O termo *banco de dados* (do srcinal *database*) é tão utilizado que precisamos começar por sua definição. E nossa definição inicial é bastante genérica.

Um **banco de dados** é uma coleção de dados[1] relacionados. Com **dados**, queremos dizer fatos conhecidos que podem ser registrados e possuem significado implícito. Por exemplo, considere os nomes, números de telefone e endereços das pessoas que você conhece. Você pode ter registrado esses dados em uma agenda ou, talvez, os tenha armazenado em um disco rígido, usando um computador pessoal e um software como Microsoft Access ou Excel. Essa coleção de dados relacionados, com um significado implícito, é um banco de dados.

Essa definição de banco de dados é bastante genérica; por exemplo, a coleção de palavras que compõem esta página de texto pode ser considerada dados relacionados e, portanto, constitui um banco de dados. Porém, o uso comum do termo *banco de dados* normalmente é mais restrito e tem as seguintes propriedades implícitas:

- Um banco de dados representa algum aspecto do mundo real, às vezes chamado de **minimundo** ou de **universo de discurso** (**UoD** — Universe of Discourse). As mudanças no minimundo são refletidas no banco de dados.
- Um banco de dados é uma coleção logicamente coerente de dados com algum significado inerente. Uma variedade aleatória de dados não pode ser corretamente chamada de banco de dados.
- Um banco de dados é projetado, construído e populado com dados para uma finalidade específica. Ele possui um grupo definido de usuários e algumas aplicações previamente concebidas nas quais esses usuários estão interessados.

Em outras palavras, um banco de dados tem alguma fonte da qual o dado é derivado, algum grau de interação com eventos no mundo real e um público que está ativamente interessado em seu conteúdo. Os usuários finais de um banco de dados podem realizar transações comerciais (por exemplo, um cliente compra uma câmera) ou eventos podem acontecer (por exemplo, uma funcionária tem um filho), fazendo que a informação no banco de dados mude. Para que um banco de dados seja preciso e confiável o tempo todo, ele precisa ser um reflexo verdadeiro do minimundo que representa; portanto, as mudanças precisam ser refletidas no banco de dados o mais breve possível.

Um banco de dados pode ter qualquer tamanho e complexidade. Por exemplo, a lista de nomes e endereços referenciados anteriormente pode consistir em apenas algumas centenas de registros, cada um com uma estrutura simples. Por sua vez, o catálogo computadorizado de uma grande biblioteca pode conter meio milhão de entradas organizadas sob diferentes categorias — por sobrenome do autor principal, por assunto, por título do livro —, com cada uma das categorias organizada alfabeticamente. Um banco de dados de tamanho e complexidade ainda maior é mantido pela Receita Federal para monitorar formulários de imposto de renda preenchidos pelos contribuintes. Se considerarmos que existem 100 milhões de contribuintes e que cada um deles preenche uma média de cinco formulários com aproximadamente 400 caracteres cada um, teríamos um banco de dados de $100 \times 10^6 \times 400 \times 5$ caracteres (bytes) de informação. Se a Receita Federal mantém o registro dos três últimos anos de cada contribuinte, além do ano atual, teríamos um banco de dados de 8×10^{11} bytes (800 gigabytes). Essa imensa quantidade de informações precisa ser organizada e gerenciada de modo que os usuários possam consultar, recuperar e atualizar os dados quando necessário.

Um exemplo de um grande banco de dados comercial é a Amazon.com. Ela contém dados de mais de 20 milhões de livros, CDs, vídeos, DVDs, jogos, eletrônicos, roupas e outros itens. O banco de dados ocupa mais de dois terabytes (um terabyte é 10^{12} bytes de armazenamento) e está armazenado em 200 computadores diferentes (denominados servidores). Cerca de 15 milhões de visitantes acessam a Amazon.com todos os dias e utilizam o banco de dados para fazer compras. O banco de dados é continuamente atualizado à medida que novos livros e outros itens são acrescentados ao estoque e as quantidades em estoque são atualizadas à medida que as compras são feitas. Cerca de cem pessoas são responsáveis por manter o banco de dados da Amazon atualizado.

Um banco de dados pode ser gerado e mantido manualmente, ou pode ser computadorizado. Por exemplo, um catálogo de cartão de biblioteca é um banco de dados que pode ser criado e mantido manualmente. Um banco de dados computadorizado pode ser criado e mantido por um grupo de programas de aplicação escritos especificamente para essa tarefa ou por um sistema gerenciador de banco de dados. Vamos tratar apenas dos bancos de dados computadorizados neste livro.

Um **sistema gerenciador de banco de dados** (**SGBD** — Database Management System) é uma coleção de programas que permite aos usuários criar e manter um banco de dados. O SGBD é um *sistema de software de uso geral* que facilita o processo de

[1]O livro srcinal utiliza a palavra *data* em singular e plural, pois isso é comum na literatura de banco de dados; o contexto determinará se ela está no singular ou no plural. (Em inglês padrão, *data* é usado para o plural e *datum*, para o singular.)

definição, *construção*, *manipulação* e *compartilhamento* de bancos de dados entre diversos usuários e aplicações. **Definir** um banco de dados envolve especificar os tipos, estruturas e restrições dos dados a serem armazenados. A definição ou informação descritiva do banco de dados também é armazenada pelo SGBD na forma de um catálogo ou dicionário, chamado de **metadados**. A **construção** do banco de dados é o processo de armazenar os dados em algum meio controlado pelo SGBD. A **manipulação** de um banco de dados inclui funções como consulta a banco de dados para recuperar dados específicos, atualização do banco de dados para refletir mudanças no minimundo e geração de relatórios com base nos dados. O **compartilhamento** de um banco de dados permite que diversos usuários e programas acessem-no simultaneamente.

Um **programa de aplicação** acessa o banco de dados ao enviar consultas ou solicitações de dados ao SGBD. Uma **consulta**[2] normalmente resulta na recuperação de alguns dados; uma **transação** pode fazer que alguns dados sejam lidos e outros, gravados no banco de dados.

Outras funções importantes fornecidas pelo SGBD incluem *proteção* do banco de dados e sua *manutenção* por um longo período. A **proteção** inclui *proteção do sistema* contra defeitos (ou falhas) de hardware ou software e *proteção de segurança* contra acesso não autorizado ou malicioso. Um banco de dados grande pode ter um ciclo de vida de muitos anos, de modo que o SGBD precisa ser capaz de **manter** o sistema, permitindo que ele evolua à medida que os requisitos mudam com o tempo.

Não é absolutamente necessário utilizar software de SGBD de uso geral para implementar um banco de dados computadorizado. Poderíamos escrever nosso próprio conjunto de programas para criar e manter o banco de dados, com efeito criando nosso próprio software de SGBD de *uso especial*. Em ambos os casos — se usarmos um SGBD de uso geral ou não —, em geral temos de implementar uma quantidade considerável de software complexo. De fato, a maioria dos SGBDs é constituída de sistemas de software muito complexos.

Para completar nossas definições iniciais, chamaremos a união do banco de dados com o software de SGBD de **sistema de banco de dados**. A Figura 1.1 ilustra alguns dos conceitos que discutimos até aqui.

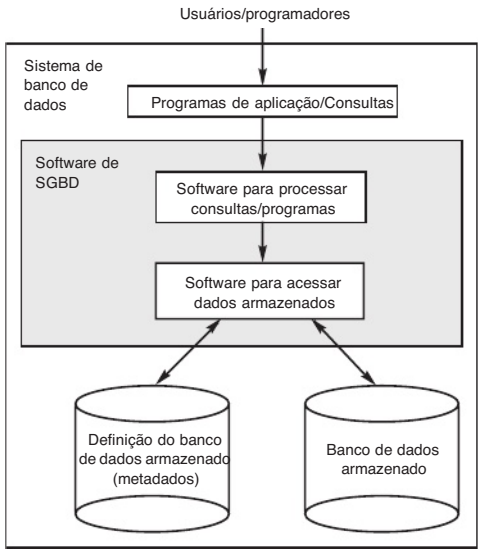


Figura 1.1
Diagrama simplificado de um ambiente de sistema de banco de dados.

1.2 Um exemplo

Vamos considerar um exemplo simples ao qual a maioria dos leitores pode estar acostumada: um banco de dados UNIVERSIDADE para manter informações referentes a alunos, disciplinas e notas em um ambiente universitário. A Figura 1.2 mostra a estrutura e alguns exemplos de dados para o banco de dados UNIVERSIDADE. O banco de dados está organizado como cinco arquivos, e cada um armazena **registros de dados** do mesmo tipo[3]. O arquivo ALUNO armazena dados sobre cada aluno, o arquivo DISCIPLINA armazena dados sobre cada disciplina, o arquivo TURMA armazena dados sobre cada turma de uma disciplina, o arquivo HISTORICO_ESCOLAR armazena as notas que os alunos recebem nas várias turmas que eles concluíram, e o arquivo PRE_REQUISITO armazena os pré-requisitos de cada disciplina.

Para *definir* esse banco de dados, precisamos especificar a estrutura dos registros de cada arquivo, determinando os diferentes tipos **elementos de dados** a serem armazenados em cada registro. Na Figura 1.2, cada registro de ALUNO contém os dados que represen-

[2] O termo *consulta* (ou *query*), que srcinalmente significa uma pergunta ou uma pesquisa, é usado livremente para todos os tipos de interações com bancos de dados, incluindo a modificação dos dados.

[3] Usamos o termo *arquivo* informalmente aqui. Em um nível conceitual, um *arquivo* é uma *coleção* de registros que podem ou não estar ordenados.

ALUNO

Nome	Numero_aluno	Tipo_aluno	Curso
Silva	17	1	CC
Braga	8	2	CC

DISCIPLINA

Nome_disciplina	Numero_disciplina	Creditos	Departamento
Introd. à ciência da computação	CC1310	4	CC
Estruturas de dados	CC3320	4	CC
Matemática discreta	MAT2410	3	MAT
Banco de dados	CC3380	3	CC

TURMA

Identificacao_turma	Numero_disciplina	Semestre	Ano	Professor
85	MAT2410	Segundo	07	Kleber
92	CC1310	Segundo	07	Anderson
102	CC3320	Primeiro	08	Carlos
112	MAT2410	Segundo	08	Chang
119	CC1310	Segundo	08	Anderson
135	CC3380	Segundo	08	Santos

HISTORICO_ESCOLAR

Numero_aluno	Identificacao_turma	Nota
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PRE_REQUISITO

Numero_disciplina	Numero_pre_requisito
CC3380	CC3320
CC3380	MAT2410
CC3320	CC1310

Figura 1.2
Exemplo de banco de dados que armazena informações de aluno e disciplina.

tam o Nome, Numero_aluno, Tipo_aluno (como novato igual a '1', segundo ano igual a '2', e assim por diante) e Curso (como matemática igual a 'MAT' e ciência da computação igual a 'CC'); cada registro de DISCIPLINA contém os dados que representam o Nome_disciplina, Numero_disciplina, Creditos e Departamento (o departamento que oferece a disciplina); e assim por diante. Também precisamos especificar um **tipo de dado** para cada elemento de dado em um registro. Por exemplo, podemos especificar que o Nome de ALUNO é uma sequência de caracteres alfabéticos; Numero_aluno de ALUNO é um inteiro, e Nota de HISTORICO_ESCOLAR é um único caractere do conjunto {'A', 'B', 'C', 'D', 'F'}. Também podemos usar um esquema de codificação para representar os valores de um item de dados. Por exemplo, na Figura 1.2, representamos Tipo_aluno de um ALUNO como 1 para novato, 2 para segundo ano, 3 para júnior, 4 para sênior e 5 para aluno formado.

Para *construir* o banco de dados UNIVERSIDADE, armazenamos dados para representar cada aluno, disciplina, turma, histórico escolar e pré-requisito como um registro no arquivo apropriado. Observe que os registros nos diversos arquivos podem estar relacionados. Por exemplo, o registro para Silva no arquivo ALUNO está relacionado a dois registros no arquivo HISTORICO_ESCOLAR, que especifica as notas de Silva em duas turmas. De modo semelhante, cada registro no arquivo PRE_REQUISITO relaciona-se a dois registros de disciplina; um representando a disciplina e o outro representando o pré-requisito. A maioria dos bancos de dados de tamanho médio e grande inclui muitos tipos de registros e possui *muitos relacionamentos* entre os registros.

A *manipulação* do banco de dados envolve consulta e atualização. Alguns exemplos de consultas são os seguintes:

- Recuperar uma lista de todas as disciplinas e notas de 'Silva'.
- Listar os nomes dos alunos que realizaram a disciplina 'Banco de dados' oferecida no segundo semestre de 2008 e suas notas nessa turma.
- Listar os pré-requisitos do curso de 'Banco de dados'.

Alguns exemplos de atualizações incluem:

- Alterar o tipo de aluno de 'Silva' para segundo ano.
- Criar outra turma para a disciplina 'Banco de dados' para este semestre.
- Inserir uma nota 'A' para 'Silva' na turma 'Banco de dados' do último semestre.

Essas consultas e atualizações informais precisam ser especificadas corretamente na linguagem de consulta do SGBD antes de serem processadas.

Nesse estágio, é útil descrever o banco de dados como parte de uma tarefa maior conhecida como sistema de informação dentro de qualquer organização.

O departamento de Tecnologia da Informação (TI) de uma empresa projeta e mantém um sistema de informações que consiste em vários computadores, sistemas de armazenamento, software de aplicação e bancos de dados. O projeto de uma nova aplicação para um banco de dados existente ou de um novo banco de dados começa com uma fase chamada **especificação e análise de requisitos** Esses requisitos são documentados com detalhes e transformados em um **projeto conceitual**, que pode ser representado e manipulado usando algumas ferramentas computadorizadas para que possa ser facilmente mantido, modificado e transformado em uma implementação de banco de dados. (Apresentaremos um modelo denominado Entidade-Relacionamento, no Capítulo 7, que é usado para essa finalidade.) O projeto é então traduzido para um **projeto lógico**, que pode ser expresso em um modelo de dados implementado em um SGBD comercial. (Neste livro, vamos enfatizar um modelo de dados conhecido como Modelo de Dados Relacional a partir do Capítulo 3. Essa é atualmente a técnica mais popular para projetar e implementar bancos de dados usando SGBDs relacionais.) O estágio final é o **projeto físico**, durante o qual outras especificações são fornecidas para armazenar e acessar o banco de dados. O projeto de banco de dados é implementado, alimentado com dados reais e mantido continuamente para refletir o estado do minimundo.

1.3 Características da abordagem de banco de dados

Diversas características distinguem a abordagem de banco de dados da abordagem muito mais antiga de programação com arquivos. No **processamento de arquivo** tradicional, cada usuário define e implementa os arquivos necessários para uma aplicação de software específica como parte da programação da aplicação. Por exemplo, um usuário, o *departamento de registro acadêmico*, pode manter arquivos sobre os alunos e suas notas. Os programas para imprimir o histórico escolar de um aluno e inserir novas notas são implementados como parte da aplicação. Um segundo usuário, o *departamento de finanças*, pode registrar as mensalidades dos alunos e seus pagamentos. Embora ambos os usuários estejam interessados em dados sobre alunos, cada um mantém arquivos separados — e programas para manipular esses arquivos —, pois cada usuário requer dados não disponíveis nos arquivos do outro. Essa redundância na definição e no armazenamento de dados resulta em desperdício no espaço de armazenamento e em esforços redundantes para manter os dados comuns atualizados.

Na abordagem de banco de dados, um único repositório mantém dados que são definidos uma vez e depois acessados por vários usuários. Nos sistemas de arquivo, cada aplicação é livre para nomear os elementos de dados independentemente. Ao contrário, em um banco de dados, os nomes ou rótulos de dados são definidos uma vez, e usados repetidamente por consultas, transações e aplicações. As principais características da abordagem de banco de dados *versus* a abordagem de processamento de arquivo são as seguintes:

- Natureza de autodescrição de um sistema de banco de dados.
- Isolamento entre programas e dados, e abstração de dados.
- Suporte de múltiplas visões dos dados.
- Compartilhamento de dados e processamento de transação multiusuário.

Descrevemos cada uma dessas características em uma seção distinta. Discutiremos características adicionais dos sistemas de banco de dados nas seções 1.6 a 1.8.

1.3.1 Natureza de autodescrição de um sistema de banco de dados

Uma característica fundamental da abordagem de banco de dados é que seu sistema contém não apenas o próprio banco de dados, mas também uma definição ou descrição completa de sua estrutura e restrições. Essa definição é armazenada no catálogo do SGBD, que possui informações como a estrutura de cada arquivo, o tipo e o formato de armazenamento de cada item de dados e diversas restrições sobre os dados. A informação armazenada no catálogo é chamada de **metadados**, e descreve a estrutura do banco de dados principal (Figura 1.1).

O catálogo é usado pelo software de SGBD e também pelos usuários do banco de dados que precisam de informações sobre a estrutura do banco de dados. Um pacote de software de SGBD de uso geral não é escrito para uma aplicação de banco de dados específica. Portanto, ele precisa consultar o catálogo para conhecer a estrutura dos arquivos em um banco de dados específico, como o tipo e o formato dos dados que ele acessará. O software de SGBD precisa trabalhar de forma satisfatória com *qualquer quantidade de aplicações de banco de dados* — por exemplo, um banco de dados de universidade, de banco ou de uma empresa —, desde que sua definição esteja armazenada no catálogo.

No processamento de arquivos tradicional, a definição de dados normalmente faz parte dos próprios programas de aplicação. Logo, esses programas são forçados a trabalhar com apenas *um banco de dados específico*, cuja estrutura é declarada nos programas de aplicação. Por exemplo, um programa de aplicação

escrito em C++ pode ter declarações de estrutura ou classe, e um programa em COBOL tem instruções da divisão de dados para definir seus arquivos. Enquanto o software de processamento de arquivos pode acessar apenas bancos de dados específicos, o software de SGBD pode acessar diversos bancos de dados extraindo e usando as definições do catálogo de banco de dados.

Para o exemplo mostrado na Figura 1.2, o catálogo do SGBD armazenará as definições de todos os arquivos mostrados. A Figura 1.3 mostra alguns exemplos de entradas em um catálogo de banco de dados.

Essas definições são especificadas pelo projetista antes da criação do banco de dados real e armazenadas no catálogo. Sempre que é feita uma solicitação para acessar, digamos, o Nome de um registro de ALUNO, o software de SGBD consulta o catálogo para determinar a estrutura do arquivo ALUNO e a posição e o tamanho do item de dado Nome dentro do registro de ALUNO. Ao contrário, em uma aplicação típica de processamento de arquivo, a estrutura do arquivo e, no caso extremo, a localização exata do Nome dentro de um registro de ALUNO já estão codificadas dentro de cada programa que acessa esse item de dados.

RELACOES

Nome_relacao	Numero_de_colunas
ALUNO	4
DISCIPLINA	4
TURMA	5
HISTORICO_ESCOLAR	3
PRE_REQUISITO	2

COLUNAS

Nome_coluna	Tipo_de_dado	Pertence_a_relacao
Nome	Caractere(30)	ALUNO
Numero_aluno	Caractere(4)	ALUNO
Tipo_aluno	Inteiro (1)	ALUNO
Curso	Tipo_curso	ALUNO
Nome_disciplina	Caractere(10)	DISCIPLINA
Numero_disciplina	XXXXNNNN	DISCIPLINA
....	....	....
....	....	....
....	....	....
Numero_pre_requisito	XXXXNNNN	PRE-REQUISITO

Figura 1.3
Exemplo de um catálogo para o banco de dados na Figura 1.2.

Nota: Tipo_curso é definido como um tipo enumerado com todas as matérias conhecidas. XXXXNNNN é usado para definir um tipo com quatro características alfanuméricas seguidas por quatro dígitos.

1.3.2 Isolamento entre programas e dados, e abstração de dados

No processamento de arquivos tradicional, a estrutura dos arquivos de dados está embutida nos programas de aplicação, de modo que quaisquer mudanças em sua estrutura podem exigir *alteração em todos os programas* que acessam esse arquivo. Ao contrário, os programas que acessam o SGBD não exigem tais mudanças na maioria dos casos. A estrutura dos arquivos de dados é armazenada no catálogo do SGBD separadamente dos programas de acesso. Chamamos essa propriedade de **independência de dados do programa.**

Por exemplo, um programa de acesso a arquivo pode ser escrito para acessar apenas registros de ALUNO da estrutura mostrada na Figura 1.4. Se quisermos acrescentar outro dado a cada registro de ALUNO, digamos Data_nascimento, esse programa não funcionará mais, e precisará ser alterado. Ao contrário, em um ambiente de SGBD, só precisamos mudar a descrição dos registros de ALUNO no catálogo (Figura 1.3) para refletir a inclusão do novo item de dado Data_nascimento; nenhum programa é alterado. Da próxima vez que o programa de SGBD consultar o catálogo, a nova estrutura dos registros de ALUNO será acessada e usada.

Em alguns tipos de sistemas de banco de dados, como os orientados a objeto e objeto-relacional (ver Capítulo 11), os usuários podem definir operações sobre dados como parte das definições do banco de dados. Uma **operação** (também chamada de *função* ou *método*) é especificada em duas partes. A *interface* (ou *assinatura*) de uma operação inclui o nome da operação e os tipos de dados de seus argumentos (ou parâmetros). A *implementação* (ou *método*) da operação é especificada separadamente e pode ser alterada sem afetar a interface. Os programas de aplicação do usuário podem operar sobre os dados invocando essas operações por meio de seus nomes e argumentos, independentemente de como as operações são implementadas. Isso pode ser chamado de **independência da operação do programa**.

Nome do item de dados	Posicionamento inicial no registro	Tamanho em caracteres (bytes)
Nome	1	30
Numero_aluno	31	4
Tipo_aluno	35	1
Curso	36	4

Figura 1.4
Formato de armazenamento interno para um registro de ALUNO, baseado no catálogo do banco de dados da Figura 1.3.

A característica que permite a independência de dados do programa e a independência da operação do programa é chamada de **abstração de dados**. Um SGBD oferece aos usuários uma **representação conceitual** de dados que não inclui muitos dos detalhes de como os dados são armazenados ou como as operações são implementadas. De maneira informal, um **modelo de dados** é um tipo de abstração de dados usado para oferecer essa representação conceitual. O modelo de dados usa conceitos lógicos, como objetos, suas propriedades e seus inter-relacionamentos, que podem ser mais fáceis para os usuários entenderem. Em bloco os conceitos de armazenamento de computador. Logo, o modelo de dados *oculta* os detalhes de armazenamento e implementação que não são do interesse da maioria dos usuários de banco de dados.

Por exemplo, reconsidere as figuras 1.2 e 1.3. A implementação interna de um arquivo pode ser definida por seu tamanho de registro — o número de caracteres (bytes) em cada registro — e cada item de dados pode ser especificado pelo byte inicial dentro de um registro e seu tamanho em bytes. O registro de ALUNO, assim, seria representado como mostra a Figura 1.4. Mas um típico usuário de banco de dados não está preocupado com a localização de cada item de dados dentro de um registro ou em seu tamanho; em vez disso, o usuário se preocupa seo valor é retornado corretamente, quando for feita uma referência ao Nome do ALUNO. Uma representação conceitual dos registros de ALUNO aparece na Figura 1.2. Muitos outros detalhes da organização do armazenamento do arquivo — como os caminhos de acesso especificados em um arquivo — podem ser ocultados dos usuários do banco de dados pelo SGBD. Discutiremos detalhes de armazenamento nos capítulos 17 e 18.

Na abordagem de banco de dados, a estrutura detalhada e a organização de cada arquivo são armazenadas no catálogo. Os usuários do banco de dados e os programas de aplicação se referem à representação conceitual dos arquivos, e o SGBD extrai os detalhes do armazenamento do arquivo do catálogo quando estes são necessários para os módulos de acesso a arquivo do SGBD. Muitos modelos de dados podem ser usados para oferecer essa abstração aos usuários do banco de dados. A primeira parte deste livro é dedicada à apresentação dos vários modelos de dados e dos conceitos que eles utilizam para abstrair a representação dos dados.

Nos bancos de dados orientados a objeto e objeto-relacional, o processo de abstração inclui não apenas a estrutura dos dados, mas também as operações sobre os dados. Essas operações oferecem uma abstração das atividades do minimundo comumente entendidas pelos usuários. Por exemplo, uma operação CALCULA_MEDIA pode ser aplicada ao objeto ALUNO para calcular a média das notas. Essas operações podem ser solicitadas pelas consultas do usuário ou por programas de aplicação sem ter que saber os detalhes de como as operações são implementadas. Nesse sentido, uma abstração da atividade do minimundo se torna disponível ao usuário como uma **operação abstrata**.

1.3.3 Suporte para múltiplas visões dos dados

Um banco de dados em geral tem muitos usuários, cada um podendo exigir um ponto de vista ou **visão** diferente do banco de dados. Uma visão (ou *view*) pode ser um subconjunto do banco de dados ou conter **dado virtual** que é derivado dos arquivos do banco de dados, mas não estão armazenados explicitamente. Alguns usuários não precisam saber se os dados a que se referem estão armazenados ou se são derivados. Um SGBD multiusuário, cujos usuários têm uma série de aplicações distintas, precisa oferecer facilidades para definir múltiplas visões. Por exemplo, um usuário do banco de dados da Figura 1.2 pode estar interessado apenas em acessar e imprimir o histórico escolar de cada aluno; a visão para esse usuário é mostrada na Figura 1.5(a). Um segundo usuário, que está interessado apenas em verificar se os alunos possuem todos os pré-requisitos de cada disciplina para a qual se inscreveram, pode requerer a visão apresentada na Figura 1.5(b).

1.3.4 Compartilhamento de dados e processamento de transação multiusuário

Um SGBD multiusuário, como o nome sugere, precisa permitir que múltiplos usuários acessem o banco de dados ao mesmo tempo. Isso é essencial se o dado para múltiplas aplicações está sendo integrado e mantido em um único banco de dados. O SGBD precisa incluir um software de **controle de concorrência** para garantir que vários usuários tentando atualizar o mesmo dado faça isso de uma maneira controlada, de modo que o resultado dessas atualizações seja correto. Por exemplo, quando vários agentes de viagem tentam reservar um assento em um voo de uma companhia aérea, o SGBD precisa garantir que cada assento só possa ser acessado por um agente de cada vez para que seja atribuído a um único passageiro. Esses tipos de aplicações geralmente são chamados de aplicações de **processamento de transação on-line** (**OLPT** — On-Line Transaction Processing). Um papel fundamental do software SGBD multiusuário é garantir que as transações concorrentes operem de maneira correta e eficiente.

(a)

DADO_ESCOLAR

Nome_aluno	Historico_escolar_aluno				
	Numero_disciplina	Nota	Semestre	Ano	Identificacao_turma
Silvah	CC1310	C	Segundo	08	119
	MAT2410	B	Segundo	08	112
Braga	MAT2410	A	Segundo	07	85
	CC1310	A	Segundo	07	92
	CC3320	B	Primeiro	08	102
	CC3380	A	Segundo	08	135

(b)

PRE_REQUISITO_DISCIPLINA

Nome_disciplina	Numero_disciplina	Pre_requisitos
Bancodedados	CC3380	CC3320
		MAT2410
Estruturadedados	CC3320	CC1310

Figura 1.5
Duas visões derivadas do banco de dados da Figura 1.2. (a) A visão do HISTORICO_ESCOLAR. (b) A visão do PRE_REQUISITO_DISCIPLINA.

O conceito de **transação** tem se tornado fundamental para muitas aplicações de banco de dados. Uma transação é um *programa em execução* ou *processo* que inclui um ou mais acessos ao banco de dados, como a leitura ou atualização de seus registros. Uma transação executa um acesso logicamente correto a um banco de dados quando ela é executada de forma completa e sem interferência de outras transações. O SGBD precisa impor várias propriedades da transação. A propriedade de **isolamento** garante que cada transação pareça executar isoladamente das demais, embora centenas de transações possam estar executando concorrentemente. A propriedade de **atomicidade** garante que todas as operações em uma transação sejam executadas ou que nenhuma seja. Discutiremos sobre transações em detalhes na Parte 9.

As características anteriores são importantes para distinguir um SGBD de um software tradicional de processamento de arquivo. Na Seção 1.6, discutiremos recursos adicionais que caracterizam um SGBD. Primeiro, porém, vamos categorizar os diferentes tipos de pessoas que trabalham em um ambiente de sistema de banco de dados.

1.4 Atores em cena

Para um pequeno banco de dados pessoal, como a lista de endereços discutida na Seção 1.1, uma pessoa normalmente define, constrói e manipula o banco de dados, sem compartilhamento. Porém, em grandes organizações, muitas pessoas estão envolvidas no projeto, no uso e na manutenção de um grande banco de dados, com centenas de usuários. Nesta seção, identificamos as pessoas cujas funções envolvem o uso diário de um grande banco de dados; nós os chamamos de *atores em cena*. Na Seção 1.5, consideraremos as pessoas que podem ser chamadas de *trabalhadores dos bastidores* — aqueles que trabalham para manter o ambiente do sistema de banco de dados, mas que não estão ativamente interessados em seu conteúdo como parte de sua função diária.

1.4.1 Administradores de banco de dados

Em qualquer organização onde muitas pessoas utilizam os mesmos recursos, há uma necessidade de um administrador principal para supervisionar e gerenciar tais recursos. Em um ambiente de banco de dados, o recurso principal é o próprio banco de dados, e o recurso secundário é o SGBD e os softwares relacionados. A administração desses recursos é de responsabilidade do **administrador de banco de dados** (**DBA —** *database administrator*). O DBA é responsável por autorizar o acesso ao banco de dados, coordenar e monitorar seu uso e adquirir recursos de software e hardware conforme a necessidade. Também é responsável por problemas como falhas na segurança e demora no tempo de resposta do sistema. Em grandes organizações, ele é auxiliado por uma equipe que executa essas funções.

1.4.2 Projetistas de banco de dados

Os **projetistas de banco de dados** são responsáveis por identificar os dados a serem armazenados e escolher estruturas apropriadas para representar e armazenar esses dados. Essas tarefas são realizadas principalmente antes que o banco de dados seja realmente implementado e populado com dados. É responsabilidade dos projetistas de banco de dados se comunicar com todos os potenciais usuários a fim de entender suas necessidades e criar um projeto que as atenda. Em muitos casos, os projetistas estão na equipe de DBAs e podem receber outras responsabilidades após o projeto do banco de dados estar concluído. Os projetistas de banco de dados normalmente interagem com cada potencial grupo de usuários e desenvolvem **visões** do banco de dados que cumprem os requisitos de dados e processamento desses grupos. Cada visão é então analisada e *integrada* às visões de outros grupos de usuários.

O projeto final do banco de dados precisa ser capaz de atender às necessidades de todos os grupos de usuários.

1.4.3 Usuários finais

Os **usuários finais** são pessoas cujas funções exigem acesso ao banco de dados para consultas, atualizações e geração de relatórios. O banco de dados existe primariamente para atender os usuários finais. Existem várias categorias de usuários finais:

- **Usuários finais casuais** ocasionalmente acessam o banco de dados, mas podem precisar de diferentes informações a cada vez. Utilizam uma linguagem sofisticada de consulta ao banco de dados para especificar suas necessidades e normalmente são gerentes de nível intermediário ou alto, ou outros usuários ocasionais.
- **Usuários finais iniciantes** ou **paramétricos** compõem uma grande parte dos usuários finais do banco de dados. Sua função principal gira em torno de consultar e atualizar o banco de dados constantemente, usando tipos padrão de consultas e atualizações — denominadas **transações programadas** — que foram cuidadosamente programadas e testadas. As tarefas que esses usuários realizam são variadas:
  - Caixas de banco verificam saldos de conta e realizam saques, depósitos, pagamentos etc.
  - Agentes de companhias aéreas, hotéis e locadoras de automóveis verificam a disponibilidade de determinada solicitação e fazem reservas.
  - Funcionários nas estações de recebimento de transportadoras inserem identificações de pacotes por códigos de barra e informações descritivas por meio de etiquetas para atualizar um banco de dados central de pacotes recebidos e em trânsito.
- **Usuários finais sofisticados** incluem engenheiros, cientistas, analistas de negócios e outros que estão profundamente familiarizados com as facilidades do SGBD a ponto de implementar as próprias aplicações para que atendam a suas necessidades complexas.
- **Usuários isolados** mantêm bancos de dados pessoais usando pacotes de programas prontos, que oferecem interfaces de fácil utilização, baseadas em menus ou gráficos. Um exemplo é o usuário de um pacote de cálculos de impostos, que armazena uma série de dados financeiros pessoais para fins de declaração de imposto.

Um SGBD típico oferece múltiplas facilidades para acessar um banco de dados. Usuários iniciantes precisam aprender muito pouco sobre as facilidades oferecidas pelo SGBD; eles simplesmente têm de entender as interfaces de usuário das transações padrão projetadas e implementadas para seu uso. Os usuários casuais aprendem apenas algumas facilidades que podem usar repetidamente. Usuários sofisticados tentam aprender a maioria das facilidades do SGBD para satisfazer suas necessidades complexas. Usuários isolados costumam se tornar especialistas no uso de um pacote de software específico.

1.4.4 Analistas de sistemas e programadores de aplicações (engenheiros de software)

Analistas de sistemas identificam as necessidades dos usuários finais, especialmente os iniciantes e paramétricos, e definem as especificações das transações padrão que atendam a elas. Os **programadores de aplicações** implementam essas especificações como programas; depois, eles testam, depuram, documentam e mantêm essas transações programadas. Esses analistas e programadores — também conhecidos como **engenheiros de software** e **desenvolvedores de sistemas de software** — devem estar familiarizados com todo o conjunto de capacidades fornecido pelo SGBD para realizarem suas tarefas.

1.5 Trabalhadores dos bastidores

Além daqueles que projetam, usam e administram um banco de dados, há outros associados ao projeto, desenvolvimento e operação do *software e ambiente de sistema* do SGBD. Essas pessoas normalmente não estão interessadas no conteúdo do banco de dados em si. Vamos chamá-las de *trabalhadores dos bastidores*, e elas estão incluídas nas seguintes categorias:

- **Projetistas e implementadores de sistema de SGBD** projetam e implementam os módulos e as interfaces do SGBD como um pacote de software. Um SGBD é um sistema muito complexo, que consiste em muitos componentes, ou **módulos**, incluindo módulos para implementação do catálogo, processamento de linguagem de consulta, processamento de interface, acesso e buffering de dados, controle de concorrência e tratamento de recuperação e segurança de dados. O SGBD precisa realizar a interface com outros sistemas de software, como o sistema operacional, e compiladores para diversas linguagens de programação.

- **Desenvolvedores de ferramentas** projetam e implantam **ferramentas** — os pacotes de software que facilitam a modelagem e o projeto do banco de dados, o projeto do sistema de banco de dados e a melhoria no desempenho. Ferramentas são pacotes opcionais que, em geral, são adquiridos separadamente. Elas incluem pacotes para projeto de banco de dados, monitoramento de desempenho, linguagem natural ou interfaces gráficas, protótipo, simulação e geração de dados de teste. Em muitos casos, fornecedores de software independentes desenvolvem e comercializam essas ferramentas.
- **Operadores e pessoal de manutenção** (pessoal de administração de sistemas) são responsáveis pela execução e manutenção do ambiente de hardware e software para o sistema de banco de dados.

Embora essas categorias de trabalhadores dos bastidores sejam instrumento para tornar o sistema de banco de dados disponível aos usuários finais, eles não costumam utilizar o conteúdo do banco de dados para fins pessoais.

1.6 Vantagens de usar a abordagem de SGBD

Nesta seção, discutiremos algumas das vantagens de usar um bom SGBD e as capacidades que ele deve possuir. Essas capacidades estão além das quatro principais características discutidas na Seção 1.3. O DBA deve utilizá-las para cumprir uma série de objetivos relacionados ao projeto, à administração e ao uso de um grande banco de dados multiusuário.

1.6.1 Controlando a redundância

No desenvolvimento de software tradicional, utilizando processamento de arquivo, cada grupo de usuários mantém os próprios arquivos para tratamento de suas aplicações de processamento de dados. Por exemplo, considere o exemplo do banco de dados UNIVERSIDADE da Seção 1.2; aqui, dois grupos de usuários podem ser o pessoal do departamento de registro acadêmico e departamento de finanças. Na técnica tradicional, cada grupo mantém de maneira independente os arquivos sobre os alunos. O departamento financeiro mantém dados sobre o registro e informações relacionadas a faturas, enquanto o departamento de registro acadêmico acompanha as disciplinas e as notas dos alunos. Outros grupos podem duplicar ainda mais alguns ou todos os dados nos próprios arquivos.

Essa **redundância** causada ao armazenar os mesmos dados várias vezes gera diversos problemas. Primeiro, é preciso realizar uma única atualização lógica — como a entrada de dados sobre um novo aluno — várias vezes: uma para cada arquivo onde o dado do aluno é registrado. Isso ocasiona uma *duplicação de esforço*. Segundo, o *espaço de armazenamento é desperdiçado* quando o mesmo dado é armazenado repetidamente, e esse problema pode ser sério para grandes bancos de dados. Terceiro, os arquivos que representam os mesmos dados podem tornar-se *inconsistentes*. Isso porque uma atualização é aplicada a alguns dos arquivos, mas não a outros. Mesmo que uma atualização — como a inclusão de um novo aluno — seja aplicada a todos os arquivos apropriados, os dados referentes a alunos ainda podem ser inconsistentes porque as atualizações são aplicadas de maneira independente pelos grupos de usuários. Por exemplo, um grupo de usuários pode entrar com a data de nascimento de um aluno incorretamente como '19/01/1988', enquanto outros grupos de usuários podem inserir o valor correto '29/01/1988'.

Na abordagem de banco de dados, as visões de diferentes grupos de usuários são integradas durante o projeto. O ideal é que tenhamos um projeto que armazena cada item de dados lógico — como o nome ou a data de nascimento de um aluno — em *apenas um lugar* no banco de dados. Isso é conhecido como **normalização de dados**, e garante consistência e economia de espaço de armazenamento (a normalização de dados é descrita na Parte 6 do livro). Porém, na prática, às vezes é necessário usar a **redundância controlada** para melhorar o desempenho das consultas. Por exemplo, podemos armazenar Nome_aluno e Numero_disciplina redundantemente em um arquivo HISTORICO_ESCOLAR [Figura 1.6(a)] porque, sempre que recuperamos um registro de HISTORICO_ESCOLAR, queremos recuperar o nome do aluno e o número da disciplina juntamente com a nota, o número do aluno e o identificador de turma. Colocando todos os dados juntos, não precisamos pesquisar vários arquivos para coletar esses dados. Isso é conhecido como **desnormalização**. Nesses casos, o SGBD deve ter a capacidade de *controlar* essa redundância a fim de proibir inconsistências entre os arquivos. Isso pode ser feito verificando automaticamente se os valores de Nome_aluno-Numero_aluno em qualquer registro de HISTORICO_ESCOLAR na Figura 1.6(a) combinam com um dos valores de Nome-Numero_aluno de um registro de ALUNO (Figura 1.2). De modo semelhante, os valores de Identificacao_turma-Numero_disciplina de HISTORICO_ESCOLAR podem ser verificados em registros de TURMA. Essas verificações podem ser especificadas no SGBD durante o projeto do banco de dados e impostas automaticamente pelo SGBD sempre que o arquivo HISTORICO_ESCOLAR for atualizado. A Figura 1.6(b) mostra um registro de HISTORICO_ESCOLAR incoerente com o arquivo ALUNO na Figura 1.2; esse tipo de erro pode ser inserido se a redundância *não for controlada*. Você consegue identificar a parte inconsistente?

(a)

HISTORICO_ESCOLAR

Numero_aluno	Nome_aluno	Identificacao_turma	Numero_disciplina	Nota
17	Silva	112	MAT2410	B
17	Silva	119	CC1310	C
8	Braga	85	MAT2410	A
8	Braga	92	CC1310	A
8	Braga	102	CC3320	B
8	Braga	135	CC3380	A

(b)

HISTORICO_ESCOLAR

Numero_aluno	Nome_aluno	Identificacao_turma	Numero_disciplina	Nota
17	Braga	112	MAT2410	B

Figura 1.6
Armazenamento redundante de Nome_aluno e Nome_disciplina em HISTORICO_ESCOLAR. (a) Dados consistentes. (b) Registro inconsistente.

1.6.2 Restringindo o acesso não autorizado

Quando vários usuários compartilham um grande banco de dados, é provável que amaioria deles não esteja autorizada a acessar todas as informações nele contidas. Por exemplo, dados financeiros normalmente são considerados confidenciais, e somente pessoas autorizadas têm permissão para acessá-los. Além disso, alguns usuários só podem ter permissão para recuperar dados, enquanto outros podem recuperar e atualizar. Logo, o tipo de operação de acesso — recuperação ou atualização — também deve ser controlado. Em geral, os usuários ou grupos de usuários recebem números de conta protegidos por senhas, que podem usar para acessar o banco de dados. Um SGBD deve oferecer um **subsistema de segurança e autorização**, que o DBA utiliza para criar contas e especificar suas restrições. Então, o SGBD deve impor essas restrições automaticamente. Observe que podemos aplicar controles semelhantes ao software de SGBD. Por exemplo, somente o DBA está autorizado a usar certo **software privilegiado**, como o software para criar contas. De modo semelhante, usuários paramétricos podem ter permissão para acessar o banco de dados apenas por meio de transações programadas predefinidas, desenvolvidas para seu uso.

1.6.3 Oferecendo armazenamento persistente para objetos do programa

Os bancos de dados podem ser usados para oferecer **armazenamento persistente** para objetos e estruturas de dados do programa. Esse é um dos principais motivos para a existência dos **sistemas de banco de dados orientados a objeto**. Linguagens de programação normalmente possuem estruturas de dados complexas, como tipos de registro em Pascal ou definições de classe em C++ ou Java. Os valores das variáveis de programa ou estruturas dos objetos são descartados quando um programa termina, a menos que o programador os armazene explicitamente em arquivos permanentes, o que, em geral, envolve converter essas estruturas complexas em um formato adequado para armazenamento de arquivo. Quando surge a necessidade de ler esses dados mais tarde, o programador precisa converter do formato de arquivo para a variável de programa ou estrutura de objeto. Os sistemas de banco de dados orientados a objeto são compatíveis com linguagens de programação, como C++ e Java, e o software de SGBD realiza automaticamente quaisquer conversões necessárias. Assim, um objeto complexo em C++ pode ser armazenado de forma permanente em um SGBD orientado a objeto. Esse objeto é considerado **persistente**, pois sobrevive ao término da execução e pode ser recuperado mais tarde diretamente por outro programa C++.

O armazenamento persistente de objetos de programa e estruturas de dados é uma função importante dos sistemas de banco de dados. Os sistemas tradicionais sofrem com frequência do chamado **problema de divergência de impedância**, pois as estruturas de dados fornecidas pelo SGBD são incompatíveis com as estruturas de dados da linguagem de programação. Os sistemas de banco de dados orientados a objeto em geral oferecem **compatibilidade** da estrutura de dados com uma ou mais linguagens de programação orientadas a objeto.

1.6.4 Oferecendo estruturas de armazenamento e técnicas de pesquisa para o processamento eficiente de consulta

Os sistemas de banco de dados precisam oferecer capacidades para *executar consultas e atualizações de modo eficiente*. Como o banco de dados costuma ser armazenado em disco, o SGBD precisa oferecer estruturas de dados e técnicas de pesquisa especializadas para agilizar a busca dos registros desejados no disco. Arquivos auxiliares, denominados **índices**, são usados para essa finalidade. Os índices normalmente são baseados em estruturas de dados em árvore ou estrutura de dados em *hash*, que são modificadas de maneira adequada para a pesquisa no disco. Para processar os registros de banco de dados necessários por uma consulta em particular, eles precisam ser copiados do disco para a memória principal. Portanto,

o SGBD frequentemente tem um módulo de **buffering** ou **caching** que mantém partes do banco de dados nos buffers de memória principais. Em geral, o sistema operacional é responsável pelo buffering do disco para a memória. Contudo, como o buffering de dados é essencial para o desempenho do SGBD, a maioria desses sistemas realiza o próprio buffering de dados.

O módulo de **processamento e otimização de consulta** do SGBD é responsável por escolher um plano de execução eficiente para cada consulta, com base nas estruturas de armazenamento existentes. A escolha de quais índices criar e manter faz parte do *projeto e ajuste de banco de dados físico*, que é uma das responsabilidades da equipe de DBAs. Discutiremos sobre processamento de consulta, otimização e ajuste na Parte 8 do livro.

1.6.5 Oferecendo backup e recuperação

Um SGBD precisa oferecer recursos para recuperar-se de falhas de hardware ou software. Seu **subsistema de backup e recuperação** é responsável por isso. Por exemplo, se o sistema do computador falhar no meio de uma transação de atualização complexa, o subsistema de recuperação é responsável por garantir que o banco de dados seja restaurado ao estado em que estava antes da transação ser executada. Como alternativa, o subsistema de recuperação poderia garantir que a transação seja reiniciada no ponto em que foi interrompida, de modo que seu efeito completo seja registrado no banco de dados. O backup de disco também é necessário no caso de uma falha de disco catastrófica. Discutiremos a respeito do backup e recuperação no Capítulo 23.

1.6.6 Oferecendo múltiplas interfaces do usuário

Uma vez que muitos tipos de usuários, com diversos níveis de conhecimento técnico, utilizam um banco de dados, um SGBD deve oferecer uma variedade de interfaces de usuário. Essas incluem linguagens de consulta para usuários casuais, interfaces de linguagem de programação para programadores de aplicação, formulários e códigos de comando para usuários paramétricos e interfaces controladas por menu e de linguagem natural para usuários isolados. As interfaces no uso de formulários e de menus normalmente são conhecidas como **interfaces gráficas do usuário** (**GUIs** – Graphical User Interfaces). Existem muitas linguagens e ambientes especializados para especificar GUIs. Recursos para oferecer interfaces GUI para um banco de dados na Web — ou habilitar um banco de dados para a Web — também são muito comuns.

1.6.7 Representando relacionamentos complexos entre dados

Um banco de dados pode incluir muitas variedades de dados que estão inter-relacionados de diversas maneiras. Considere o exemplo mostrado na Figura 1.2. O registro de 'Braga' no arquivo ALUNO está relacionado a quatro registros no arquivo HISTORICO_ESCOLAR. De modo semelhante, cada registro de turma está relacionado a um registro de disciplina e a uma série de registros de HISTORICO_ESCOLAR — um para cada aluno que concluiu a turma. Um SGBD precisa ter a capacidade de representar uma série de relacionamentos complexos entre os dados, definir novos relacionamentos à medida que eles surgem e recuperar e atualizar dados relacionados de modo fácil e eficaz.

1.6.8 Impondo restrições de integridade

A maioria das aplicações de banco de dados possui certas **restrições de integridade** que devem ser mantidas para os dados. Um SGBD deve oferecer capacidades para definir e impor tais restrições. O tipo mais simples de restrição de integridade envolve especificar um tipo de dado para cada item de dado. Por exemplo, na Figura 1.3, especificamos que o valor do item de dados Tipo_aluno em cada registro de ALUNO deve ser um inteiro de um dígito e que o valor de Nome precisa ser um alfanumérico de até 30 caracteres. Para restringir o valor de Tipo_aluno entre 1 e 5, seria preciso uma restrição adicional, que não aparece no catálogo atual. Um tipo de restrição mais complexo, que ocorre com frequência, envolve especificar que um registro em um arquivo deve estar relacionado a registros em outros arquivos. Por exemplo, na Figura 1.2, podemos especificar que *cada registro de turma deve estar relacionado a um registro de disciplina* Isso é conhecido como restrição de **integridade referencial** Outro tipo de restrição especifica a exclusividade sobre valores de item de dados, como *cada registro de disciplina deverá ter um valor exclusivo para Numero_disciplina* Isso é conhecido como uma restrição de **chave** ou **singularidade**. Tais restrições são derivadas do significado ou da **semântica** dos dados e do minimundo que eles representam. É responsabilidade dos projetistas do banco de dados identificar restrições de integridade durante o projeto. Algumas restrições podem ser especificadas ao SGBD e impostas automaticamente. Outras podem ter que ser verificadas por programas de atualização ou no momento da entrada de dados. Em geral, para grandes aplicações, é comum chamar essas restrições de **regras de negócio**

Um item de dados pode ser inserido erroneamente e ainda satisfazer as restrições de integridade especificadas. Por exemplo, se um aluno recebe uma nota 'A', mas uma nota 'C' é inserida no banco de dados, o SGBD *não pode* descobrir esse erro automaticamente,

pois 'C' é um valor válido para o tipo de dado Nota. Esses erros de entrada de dados só podem ser descobertos manualmente (quando o aluno recebe a nota e reclama) e corrigidos posteriormente, atualizando o banco de dados. Porém, uma nota 'Z' seria rejeitada automaticamente pelo SGBD, pois 'Z' não é um valor válido para o tipo de dado Nota. Quando discutirmos cada modelo de dados nos próximos capítulos, vamos apresentar regras que pertencem a esse modelo de maneira implícita. Por exemplo, no modelo Entidade-Relacionamento, no Capítulo 7, um relacionamento deve envolver pelo menos duas entidades. Essas regras são **inerentes** do modelo de dados e assumidas de maneira automática, para garantir a validade do modelo.

1.6.9 Permitindo dedução e ações usando regras

Alguns sistemas oferecem capacidades para definir *regras de dedução* (ou *inferência*) para *deduzir* novas informações com base nos fatos armazenados no banco de dados. Esses sistemas são chamados de **sistemas de banco de dados dedutivos**. Por exemplo, pode haver regras complexas na aplicação do minimundo para determinar quando um aluno está em época de prova. Estas podem ser especificadas *declarativamente* como **regras** que, quando compiladas e mantidas pelo SGBD, podem determinar todos os alunos em época de prova. Em um SGBD tradicional, um *código de programa de procedimento* explícito teria de ser escrito para dar suporte a tais aplicações. Mas, se as regras do minimundo mudarem, geralmente é mais conveniente mudar as regras de dedução declaradas do que recodificar programas de procedimento. Nos sistemas de banco de dados relacionais de hoje é possível associar **gatilhos** (ou **triggers**) a tabelas. Um gatilho é uma forma de regra ativada por atualizações na tabela, que resulta na realização de algumas operações adicionais em algumas outras tabelas, envio de mensagens, e assim por diante. Procedimentos mais elaborados para impor regras são popularmente chamados de **procedimentos armazenados** (ou **stored procedures**); eles se tornam parte da definição geral de banco de dados e são chamados de forma apropriada quando certas condições são atendidas. A funcionalidade mais poderosa é fornecida por **sistemas de banco de dados ativos**, que oferecem regras ativas que podem automaticamente iniciar ações quando ocorrem certos eventos e condições.

1.6.10 Implicações adicionais do uso da abordagem de banco de dados

Esta seção discute algumas implicações adicionais do uso da abordagem de banco de dados que pode beneficiar a maioria das organizações.

Potencial para garantir padrões. A técnica de banco de dados permite que o DBA defina e imponha o uso de padrões entre os usuários de banco de dados em uma grande organização. Isso facilita a comunicação e a cooperação entre seus vários departamentos, projetos e usuários dentro da organização. Podem ser definidos padrões para nomes e formatos dos elementos de dados, formatos de exibição, estruturas de relatório, terminologia, e assim por diante. O DBA pode impor padrões em um ambiente de banco de dados centralizado mais facilmente do que em um ambiente onde cada grupo de usuários tem controle sobre os próprios arquivos de dados e software.

Tempo reduzido para desenvolvimento de aplicação Um importante recurso de venda da abordagem de banco de dados é que o desenvolvimento de uma nova aplicação — como a recuperação de certos dados para impressão de um novo relatório — leva muito pouco tempo. Projetar e implementar um grande banco de dados multiusuário do zero pode levar mais tempo do que escrever uma única aplicação de arquivo especializada. Porém, quando um banco de dados está pronto e funcionando, geralmente é preciso muito menos tempo para criar outras aplicações usando as facilidades do SGBD. O tempo de desenvolvimento usando um SGBD é estimado como sendo um sexto a um quarto daquele para um sistema de arquivo tradicional.

Flexibilidade. Pode ser necessário mudar a estrutura de um banco de dados à medida que as necessidades mudam. Por exemplo, pode aparecer um novo grupo de usuários precisando de informações atualmente não incluídas no banco de dados. Em resposta, pode ser preciso acrescentar um arquivo ao banco de dados ou estender os elementos de dados em um arquivo existente. Os SGBDs modernos permitem certos tipos de mudanças evolucionárias na estrutura do banco de dados sem afetar os dados armazenados e os programas de aplicação existentes.

Disponibilidade de informações atualizadas. Um SGBD torna o banco de dados disponível a todos os usuários. Assim que a atualização de um usuário é aplicada ao banco de dados, todos os outros usuários podem vê-la imediatamente. Essa disponibilidade de informações atualizadas é essencial para muitas aplicações de processamento de transação, como sistemas de reserva ou bancos de dados bancários, e ela é possibilitada pelos subsistemas de controle de concorrência e recuperação de um SGBD.

Economias de escala. A técnica de SGBD permite a consolidação de dados e aplicações, reduzindo assim a quantidade de sobreposição desperdiçada entre as atividades do pessoal de processamento de dados em diferen-

tes projetos ou departamentos, bem como as redundâncias entre as aplicações. Isso permite que a organização inteira invista em processadores, dispositivos de armazenamento ou mecanismos de comunicação mais poderosos, em vez de cada departamento ter de comprar o próprio equipamento (menor desempenho), o que reduz os custos gerais de operação e gerenciamento.

1.7 Uma breve história das aplicações de banco de dados

Agora, vamos apresentar uma breve visão histórica das aplicações que usam SGBDs e como elas motivaram o desenvolvimento de novos tipos de sistemas de banco de dados.

1.7.1 Antigas aplicações de banco de dados usando sistemas hierárquicos e de rede

Muitas aplicações de banco de dados antigas mantinham os registros em grandes organizações, como corporações, universidades, hospitais e bancos. Em muitas delas, existia grande quantidade de registros com estrutura semelhante. Por exemplo, em uma aplicação de universidade, informações semelhantes seriam mantidas para cada aluno, cada disciplina, cada histórico escolar, e assim por diante. Também existiam muitos tipos de registros e muitos inter-relacionamentos entre eles.

Um dos principais problemas com os sistemas de banco de dados antigos era a mistura de relacionamentos conceituais com o armazenamento e posicionamento físico dos registros no disco. Logo, esses sistemas não ofereciam capacidades suficientes para *abstração de dados* e *independência entre dados e programas*. Por exemplo, as notas de determinado aluno poderiam ser armazenadas fisicamente próximas do registro do aluno. Embora isso fornecesse um acesso muito eficiente para as consultas e transações corrinais com as quais o banco de dados foi projetado para lidar, não oferecia flexibilidade suficiente para acessar registros de modo eficiente quando novas consultas e transações fossem identificadas. Em particular, novas consultas que exigiam uma organização de armazenamento diferente para o processamento eficiente eram muito difíceis de implementar de modo eficaz. Também era muito trabalhoso reorganizar o banco de dados quando eram feitas mudanças nos requisitos da aplicação.

Outra deficiência dos sistemas antigos era que eles ofereciam apenas interfaces da linguagem de programação. Isso tornava demorada e cara a implementação de novas consultas e transações, pois novos programas tinham de ser escritos, testados e depurados. A maioria desses sistemas de banco de dados era implantada em computadores mainframes grandes e caros, começando em meados da década de 1960 e continuando nos anos 1970 e 1980. Os principais tipos dos primeiros sistemas eram baseados em três paradigmas principais: sistemas hierárquicos, sistemas baseados em modelo de rede e sistemas de arquivo invertido.

1.7.2 Oferecendo abstração de dados e flexibilidade de aplicação com bancos de dados relacionais

Os bancos de dados relacionais foram propostos corrinalmente para separar o armazenamento físico dos dados de sua representação conceitual e para fornecer uma base matemática para a representação e a consulta dos dados. O modelo de dados relacional também introduziu linguagens de consulta de alto nível, que ofereciam uma alternativa às interfaces de linguagem de programação, tornando muito mais rápida a escrita de novas consultas. A representação relacional dos dados é semelhante ao exemplo que apresentamos na Figura 1.2. Os sistemas relacionais visavam inicialmente atender às mesmas aplicações dos sistemas mais antigos, e forneciam flexibilidade para desenvolver rapidamente novas consultas e reorganizar o banco de dados à medida que os requisitos mudavam. Logo, a *abstração de dados* e a *independência entre dados e programas* eram mais desenvolvidas em comparação com os sistemas anteriores.

Os sistemas relacionais experimentais, desenvolvidos no final da década de 1970, e os sistemas de gerenciamento de bancos de dados relacionais (SGBDR), introduzidos na década de 1980, eram muito lentos, pois não usavam ponteiros de armazenamento físico ou posicionamento de registro para acessar registros de dados relacionados. Com o desenvolvimento de novas técnicas de armazenamento houve uma melhora no desempenho do processamento e otimização de consulta. Por fim, os bancos de dados relacionais se tornaram o tipo de sistema de banco de dados dominante para aplicações tradicionais. Eles agora existem em quase todos os tipos de computadores, desde os menores modelos pessoais até grandes servidores.

1.7.3 Aplicações orientadas a objeto e a necessidade de bancos de dados mais complexos

O surgimento de linguagens de programação orientadas a objeto no final da década de 1980 e a necessidade de armazenar e compartilhar objetos complexos e estruturados levou ao desenvolvimento

de Bancos de Dados Orientados a Objeto (BDOOs). Inicialmente, os BDOOs eram considerados um concorrente dos bancos de dados relacionais, pois forneciam estruturas de dados mais gerais. Eles também incorporavam muitos dos paradigmas úteis orientados a objeto, como tipos de dados abstratos, encapsulamento de operações, herança e identidade de objeto. Porém, a complexidade do modelo e a falta de um padrão inicial contribuíram para seu uso limitado. Eles agora são usados principalmente em aplicações especializadas, como projeto de engenharia, aplicações multimídia e sistemas de manufatura. Apesar das expectativas de que eles causariam um grande impacto, atualmente sua penetração geral no mercado de produtos de banco de dados permanece abaixo dos cinco por cento. Além disso, muitos conceitos orientados a objeto foram incorporados nas versões mais recentes dos SGBDs relacionados, levando a sistemas de gerenciamento de banco de dados objeto-relacional, conhecidos como SGBDORs.

1.7.4 Intercâmbio de dados na Web para comércio eletrônico usando XML

A World Wide Web oferece uma grande rede de computadores interconectados. Os usuários podem criar documentos usando uma linguagem de publicação na Web, como HyperText Markup Language (HTML ou, em português, linguagem de marcação de hipertexto), e armazenar esses documentos em servidores Web, onde outros usuários (clientes) podem acessá-los. Os documentos podem ser vinculados por meio de **hyperlinks**, que são indicadores para outros documentos. Na década de 1990, o comércio eletrônico (e-commerce) surgiu como uma importante aplicação da Web. Rapidamente ficou visível que partes da informação nas páginas Web de e-commerce eram, com frequência, dados extraídos dinamicamente de SGBDs. Diversas técnicas foram desenvolvidas para permitir o intercâmbio de dados na Web. Atualmente, a eXtended Markup Language (XML, em português, linguagem de marcadores extensível) é considerada o principal padrão para intercâmbio entre diversos tipos de bancos de dados e páginas Web. A XML combina conceitos dos modelos usados nos sistemas de documentos com os conceitos de modelagem de banco de dados. O Capítulo 12 é dedicado à discussão sobre a XML.

1.7.5 Estendendo as capacidades do banco de dados para novas aplicações

O sucesso dos sistemas de banco de dados nas aplicações tradicionais encorajou os desenvolvedores de outros tipos de aplicações a tentarem utilizá-los. Essas aplicações tradicionalmente usavam suas próprias estruturas especializadas de arquivo e dados. Os sistemas de banco de dados agora oferecem extensões para dar melhor suporte às necessidades especializadas para algumas dessas aplicações. A seguir estão alguns exemplos dessas aplicações:

- Aplicações **científicas** que armazenam grande quantidade de dados resultantes de experimentos científicos em áreas como a física de alta energia, o mapeamento do genoma humano e a descoberta de estruturas de proteínas.
- Armazenamento e recuperação de **imagens**, incluindo notícias escaneadas e fotografias pessoais, imagens fotográficas de satélite e imagens de procedimentos médicos, como raios X e IRMs (imagens por ressonância magnética).
- Armazenamento e recuperação de **vídeos**, como filmes, e **clipes de vídeo** de notícias ou de câmeras digitais pessoais.
- Aplicações de **mineração de dados** (ou **data mining**), que analisam grande quantidade de dados procurando as ocorrências de padrões ou relacionamentos específicos, e identificação de padrões incomuns em áreas como uso de cartão de crédito.
- Aplicações **espaciais**, que armazenam a localização espacial de dados, como informações de clima, mapas usados em sistemas de informações geográficas e em sistemas de navegação de automóveis.
- Aplicações de **série temporais**, que armazenam informações como dados econômicos em pontos regulares no tempo, como vendas diárias e valores mensais do Produto Interno Bruto (PIB).

Logo ficou aparente que os sistemas relacionais básicos não eram muito adequados para muitas dessas aplicações, em geral por um ou mais dos seguintes motivos:

- Estruturas de dados mais complexas eram necessárias para modelar a aplicação do que a representação relacional simples.
- Novos tipos de dados eram necessários além dos tipos básicos numéricos e alfanuméricos.
- Novas operações e construtores de linguagem de consulta eram necessários para manipular os novos tipos de dados.
- Novas estruturas de armazenamento e indexação eram necessárias para a pesquisa eficiente sobre os novos tipos de dados.

Isso levou os desenvolvedores de SGBD a acrescentarem funcionalidade a seus sistemas. Alguma funcionalidade era de uso geral, como a incorporação de conceitos dos bancos de dados orientados a objeto aos sistemas relacionais. Outras eram de uso especial, na forma de módulos opcionais que poderiam ser usados para aplicações específicas. Por exemplo, os usuários poderiam comprar um módulo de séries temporal para usar com seu SGBD relacional para aplicações de séries temporais.

Muitas organizações de grande porte utilizam vários pacotes de aplicação de software que trabalham intimamente com o **banco de dados de back-ends**. O banco de dados do back-end. representa um ou mais bancos de dados, possivelmente de diferentes fornecedores e usando diferentes modelos de dados, que mantêm dados manipulados por esses pacotes para dar suporte a transações, gerar relatórios e responder a consultas ocasionais. Um dos sistemas mais utilizados inclui o **ERP** (Enterprise Resource Planning, planejamento de recursos empresariais), que serve para consolidar diversas áreas funcionais dentro de uma organização, incluindo produçã o, vendas, distribuição, marketing, finanças, recursos humanos, e assim por diante. Outro tipo de sistema popular é o **CRM** (Customer Relationship Management, gerenciamento do relacionamento com o cliente), que compreende funções de processamento de pedidos, propaganda, suporte ao cliente. Uma das razões para a popularidade das aplicações de Web é que elas habilitam as **portas Web** para que usuários internos e externos recebem uma série de interfaces de portal Web para interagir com os bancos de dados de back-end.

1.7.6 Bancos de dados *versus* recuperação de informações

Tradicionalmente, a tecnologia de banco de dados se aplica a dados estruturados e formatados, que surgem em aplicações de rotina no governo, no comércio e na indústria. Ela é bastante utilizada nos setores de manufatura, varejo, bancos, seguros, finanças e saúde, onde dados estruturados são coletados por meio de formulários, como faturas ou documentos de registro de paciente. Uma área relacionada à tecnologia de banco de dados é a **Recuperação de Informação (RI)**, que lida com livros, manuscritos e diversas formas de artigos baseados em biblioteca. O dado é indexado, catalogado e anotado usando palavras-chave. A RI está relacionada à busca por conteúdo com base nessas palavras-chave e a muitos problemas que lidam com processamento de documento e processamento de texto em forma livre. Muito trabalho tem sido feito sobre busca em texto baseada em palavras-chave, localização de documentos e sua classificação conforme a relevância, categorização automática de texto, classificação de documentos de texto por tópicos, e assim por diante. Com o advento da Web e a proliferação de páginas HTML na faixa dos bilhões, é preciso aplicar muitas técnicas de RI para processar os dados na Web. Os dados dessas páginas normalmente contêm imagens, texto e objetos que são ativos e mudam de maneira dinâmica. A recuperação de informações na Web é um problema novo que exige que técnicas de bancos de dados e RI sejam aplicadas a uma série de combinações e novas. Discutiremos os conceitos da Web na recuperação de informações e páginas Web no Capítulo 27.

1.8 Quando não usar um SGBD

Apesar das vantagens de usar um SGBD, existem algumas situações em que esse sistema pode envolver custos adicionais desnecessários, que não aconteceriam no processamento de arquivos tradicional. Os custos adicionais do uso de um SGBD devem-se aos seguintes fatores:

- Alto investimento inicial em hardware, software e treinamento.
- A generalidade que um SGBD oferece para a definição e o processamento de dados.
- Esforço adicional para oferecer funções de segurança, controle de concorrência, recuperação e integridade.

Portanto, pode ser mais desejável usar arquivos comuns sob as seguintes circunstâncias:

- Aplicações de banco de dados simples e bem definidas, para as quais não se espera muitas mudanças.
- Requisitos rigorosos, de tempo real, para alguns programas de aplicação, que podem não ser atendidos devido as operações extras executadas pelo SGBD.
- Sistemas embarcados com capacidade de armazenamento limitada, onde um SGBD de uso geral não seria apropriado.
- Nenhum acesso de múltiplos usuários aos dados.

Certos setores e aplicações decidiram não utilizar SGBDs de uso geral. Por exemplo, muitas ferramentas de projeto auxiliado por computador (CAD) usadas por engenheiros civis e mecânicos possuem software proprietário para gerenciamento de arquivos e dados, preparado para as manipulações internas dos desenhos e objetos 3D. De modo semelhante, sistemas de comunicação e comutação projetados por empresas como a AT&T foram manifestações

iniciais do software de banco de dados preparado para executar de forma muito rápida com dados organizados hierarquicamente, para agilizar o acesso e o roteamento das chamadas. De modo semelhante, implementações dos sistemas de informações geográficas (SIG) normalmente usam os próprios esquemas de organização de dados, a fim de implantar, de modo eficiente, funções relacionadas a processamento de mapas, contornos físicos, linhas, polígonos, e assim por diante. Os SGBDs de uso geral são inadequados para essa finalidade.

Resumo

Neste capítulo, definimos um banco de dados como uma coleção de dados relacionados, onde *dados* significam fatos gravados. Um banco de dados típico representa algum aspecto do mundo real e é usado para fins específicos por um ou mais grupos de usuários. Um SGBD é um pacote de software generalizado para implementar e manter um banco de dados computadorizado. Juntos, o banco de dados e o software formam um sistema de banco de dados. Identificamos várias características que distinguem a técnica de banco de dados das aplicações tradicionais de processamento de arquivo, e discutimos as principais categorias de usuários de banco de dados, ou os *atores em cena*. Observamos que, além dos usuários em ambiente de banco de dados, existem várias categorias de pessoal de suporte, ou *trabalhadores de bastidores*, em um ambiente de banco de dados.

Apresentamos uma lista de capacidades que devem ser fornecidas pelo software de SGBD ao DBA, aos projetistas de banco de dados e aos usuários finais para ajudá-los a projetar, administrar e usar um banco de dados. Depois, mostramos uma rápida perspectiva histórica da evolução das aplicações de banco de dados. Indicamos o casamento da tecnologia de banco de dados com a tecnologia da recuperação de informações, que desempenhará um papel importante devido à popularidade da Web. Finalmente, discutimos os custos adicionais do uso de um SGBD e algumas situações em que sua utilização pode não ser vantajosa.

Perguntas de revisão

1.1. Defina os seguintes termos: *dados*, *banco de dados*, *SGBD*, *sistema de banco de dados*, *catálogo de banco de dados*, *independência entre dados e programas*, *visão do usuário*, *DBA*, *usuário final*, *transação programada*, *sistema de banco de dados dedutivo*, *objeto persistente*, *metadados* e *aplicação para processamento de transação*.

1.2. Quais os quatro tipos principais de ações que envolvem bancos de dados? Discuta cada tipo rapidamente.

1.3. Discuta as principais características da abordagem de banco de dados e como ela difere dos sistemas de arquivo tradicionais.

1.4. Quais são as responsabilidades do DBA e dos projetistas de banco de dados?

1.5. Quais são os diferentes tipos de usuários finais de banco de dados? Discuta as principais atividades de cada um.

1.6. Discuta as capacidades que devem ser fornecidas por um SGBD.

1.7. Discuta as diferenças entre sistemas de banco de dados e sistemas de recuperação de informações.

Exercícios

1.8. Identifique algumas operações informais de consulta e atualização que você esperaria aplicar ao banco de dados mostrado na Figura 1.2.

1.9. Qual é a diferença entre redundância controlada e não controlada? Dê exemplos.

1.10. Especifique todos os relacionamentos entre os registros do banco de dados mostrado na Figura 1.2.

1.11. Mostre algumas visões adicionais que podem ser necessárias a outros grupos de usuários do banco de dados mostrado na Figura 1.2.

1.12. Cite alguns exemplos de restrições de integridade que você acredita que possam se aplicar ao banco de dados mostrado na Figura 1.2.

1.13. Dê exemplos de sistemas em que pode fazer sentido usar o processamento de arquivos tradicional em vez da técnica de banco de dados.

1.14. Considere a Figura 1.2.

- **a.** Se o nome do departamento 'CC' (Ciência da computação) mudar para 'CCES' (Ciência da computação e engenharia de software) e o prefixo correspondente para o número da disciplina também mudar, identifique as colunas no banco de dados que precisariam ser atualizadas.
- **b.** Você consegue reestruturar as colunas nas tabelas DISCIPLINA, TURMA e PRE_REQUISITO de modo que somente uma delas precise de atualização?

Bibliografia selecionada

A edição de outubro de 1991 de *Communications of the ACM* e Kim (1995) incluem vários artigos que descrevem SGBDs da próxima geração. Muitos dos recursos de banco de dados discutidos no início agora estão disponíveis comercialmente. A edição de março de 1976 de *ACM Computing Surveys* oferece uma introdução aos sistemas de banco de dados, e pode fornecer uma perspectiva histórica para o leitor interessado.

capítulo 2

Conceitos e arquitetura do sistema de banco de dados

A arquitetura dos SGBDs tem evoluído desde os primeiros sistemas monolíticos, nos quais todo o software SGBD era um sistema altamente integrado, até os mais modernos, que têm um projeto modular, com arquitetura de sistema cliente/servidor. Essa evolução espelha as tendências na computação, em que grandes computadores mainframes centralizados estão sendo substituídos por centenas de estações de trabalho distribuídas e computadores pessoais, conectados por redes de comunicações a vários tipos de máquinas servidoras — servidores Web, servidores de banco de dados, servidores de arquivos, servidores de aplicações, e assim por diante.

Em uma arquitetura básica de SGBD cliente/servidor a funcionalidade do sistema é distribuída entre dois tipos de módulos.[1] O **módulo cliente** normalmente é projetado para executar em uma estação de trabalho ou computador pessoal. Em geral, os programas de aplicação e interfaces com o usuário que acessam o banco de dados executam no módulo cliente. Logo, esse módulo se encarrega da interação do usuário e oferece interfaces amigáveis, como formulários ou GUIs (interfaces gráficas do usuário) baseadas em menu. O outro tipo de módulo, chamado **módulo servidor**, é normalmente responsável pelo armazenamento de dados, acesso, pesquisa e outras funções. Discutiremos sobre arquiteturas cliente/servidor com mais detalhes na Seção 2.5. Primeiro, temos de estudar mais os conceitos básicos, que nos darão um melhor conhecimento das arquiteturas de banco de dados modernas.

Neste capítulo, apresentamos a terminologia e os conceitos básicos que serão usados no decorrer do livro. A Seção 2.1 discute os modelos de dados e define os conceitos de esquemas e instâncias, que são fundamentais para o estudo dos sistemas de banco de dados. Depois, discutimos a arquitetura do SGBD de três esquemas e a independência de dados na Seção 2.2; isso oferece um ponto de vista do usuário sobre o que um SGBD deve realizar. Na Seção 2.3, descrevemos os tipos de interfaces e linguagens que, normalmente, são fornecidas por um SGBD. A Seção 2.4 discute o ambiente do software de um sistema de banco de dados. A Seção 2.5 oferece uma visão geral de vários tipos de arquiteturas cliente/servidor. Finalmente, a Seção 2.6 apresenta uma classificação dos tipos de pacotes de SGBD. No final do capítulo é apresentado um resumo.

O material nas seções 2.4 a 2.6 oferece conceitos mais detalhados, que podem ser considerados suplementares ao material introdutório básico.

2.1 Modelos de dados, esquemas e instâncias

Uma característica fundamental da abordagem de banco de dados é que ela oferece algum nível de abstração de dados. A **abstração de dados**, geralmente, se refere à supressão de detalhes da organização e armazenamento dos dados, destacando recursos essenciais para um melhor conhecimento desses dados. Uma das principais características da abordagem de banco de dados é possibilitar a abstração de dados, de modo que diferentes usuários possam percebê-los em seu nível de detalhe preferido. Um **modelo de dados** — uma coleção de conceitos que podem ser usados para descrever a estrutura de um banco de dados — oferece os meios necessários para

[1] Conforme veremos na Seção 2.5, existem variações sobre essa simples arquitetura cliente/servidor em duas camadas.

alcançar essa abstração. [2] Com *estrutura de um banco de dados*, queremos dizer os tipos, relacionamentos e restrições que se aplicam aos dados. A maioria dos modelos de dados também inclui um conjunto de **operações básicas** para especificar recuperações e atualizações no banco de dados.

Além das operações básicas fornecidas pelo modelo de dados, está se tornando mais comum incluir conceitos no modelo de dados para especificar o **aspecto dinâmico** ou **comportamento** de uma aplicação de banco de dados. Isso permite ao projetista do banco de dados especificar um conjunto de operações válidas, definidas pelo usuário, sobre os objetos do banco de dados.[3] Um exemplo de uma operação definida pelo usuário poderia ser CALCULA_MEDIA, que pode ser aplicada a um objeto ALUNO. Por sua vez, operações genéricas para inserir, excluir, modificar ou recuperar qualquer tipo de objeto normalmente estão incluídas nas *operações básicas do modelo de dados*. Conceitos para especificar o comportamento são fundamentais para os modelos de dados orientados a objeto (ver Capítulo 11), mas também estão sendo incorporados em modelos de dados mais tradicionais. Por exemplo, modelos objeto-relacional (ver Capítulo 11) estendem o modelo relacional básico para incluir tais conceitos, entre outros. No modelo de dados relacional básico, existe um recurso para conectar um comportamento às relações na forma de módulos de armazenamento persistentes, popularmente conhecidos como procedimentos armazenados ou *stored procedures* (ver Capítulo 13).

2.1.1 Categorias de modelos de dados

Muitos modelos de dados foram propostos, e podemos classificá-los de acordo com os tipos de conceitos que eles utilizam para descrever a estrutura do banco de dados. **Modelos de dados de alto nível** ou **conceituais** oferecem conceitos que são próximos ao modo como muitos usuários percebem os dados, enquanto os **modelos de dados de baixo nível** ou **físicos** oferecem conceitos que descrevem os detalhes de como os dados são armazenados no computador, em geral, em discos magnéticos. Os conceitos oferecidos pelos modelos de dados de baixo nível costumam ser voltados para especialistas de computadores, não para usuários finais. Entre esses dois extremos está uma classe de **modelos de dados representativos** (ou de **implementação**),[4] que oferece conceitos que podem ser facilmente entendidos pelos usuários finais, mas que não está muito longe do modo como os dados são organizados e armazenados no computador. Modelos de dados representativos ocultam muitos detalhes do armazenamento de dados em disco, mas podem ser implementados diretamente em um sistema de computador.

Os modelos de dados conceituais utilizam conceitos como entidades, atributos e relacionamentos. Uma **entidade** representa um objeto ou conceito do mundo real, como um funcionário ou um projeto do minimundo que é descrito no banco de dados. Um **atributo** representa alguma propriedade de interesse que descreve melhor uma entidade, como o nome ou o salário do funcionário. Um **relacionamento** entre duas ou mais entidades representa uma associação entre elas — por exemplo, um relacionamento trabalha-em entre um funcionário e um projeto. O Capítulo 7 apresentará o **modelo Entidade-Relacionamento** — um modelo de dados conceitual popular de alto nível. O Capítulo 8 descreverá abstrações adicionais usadas para a modelagem avançada, como generalização, especialização e categorias (tipos de união).

Os modelos de dados representativos ou de implementação são os usados com mais frequência nos SGBDs comerciais tradicionais. Estes incluem o amplamente utilizado **modelo de dados relacional**, bem como os chamados modelos de dados legados — os **modelos de rede e hierárquicos** — que foram bastante usados no passado. A Parte 2 é dedicada ao modelo de dados relacional e suas restrições, operações e linguagens.[5] O padrão SQL para bancos de dados relacionais será descrito nos capítulos 4 e 5. Os modelos de dados representativos mostram os dados usando estruturas de registro e, portanto, às vezes são denominados **modelos de dados baseados em registro**.

Podemos considerar o **modelo de dados de objeto** como um exemplo de uma nova família de modelos de dados de implementação de nível mais alto e que são mais próximos dos modelos de dados conceituais. Um padrão para bancos de dados de objeto, chamado modelo de objeto ODMG, foi proposto pelo **grupo de gerenciamento de dados objeto** (**ODMG** — Object Data Management Group). O Capítulo 11 descreve as características gerais dos bancos de dados de objeto e o padrão proposto do modelo de

[2] Às vezes, a palavra *modelo* é usada para indicar uma descrição de banco de dados específica, ou esquema — por exemplo, *o modelo de dados de marketing*. Não usaremos essa interpretação.

[3] A inclusão de conceitos para descrever um comportamento reflete uma tendência por meio da qual as atividades do projeto do banco de dados e do projeto de software estão, cada vez mais, sendo combinadas em uma única atividade. Tradicionalmente, especificar um comportamento é algo associado ao projeto de software.

[4] O termo *modelo de dados de implementação* não é um termo padrão; ele foi apresentado para nos referirmos aos modelos de dados disponíveis nos sistemas de banco de dados comerciais.

[5] Um resumo dos modelos de dados hierárquico e de rede consta dos apêndices D e E (em inglês). Eles podem ser acessados no site de apoio do livro.

objeto. Os modelos de dados de objeto também são frequentemente utilizados como modelos conceituais de alto nível, em particular no domínio da engenharia de software.

Os modelos de dados físicos descrevem o armazenamento dos dados como arquivos no computador, com informações como formatos de registro, ordenações de registro e caminhos de acesso. Um **caminho de acesso** é uma estrutura que torna eficiente a busca por registros de um banco de dados em particular. Discutiremos as técnicas de armazenamento físico e as estruturas de acesso nos capítulos 17 e 18. Um **índice** é um exemplo de um caminho que permite o acesso direto aos dados usando um termo de índice ou uma palavra-chave. Ele é semelhante ao índice no final deste livro, com a exceção de que pode ser organizado de forma linear, hierárquica (estruturada em árvore) ou de alguma outra maneira.

2.1.2 Esquemas, instâncias e estado do banco de dados

Em qualquer modelo de dados, é importante distinguir entre a *descrição* do banco de dados e o *próprio banco de dados*. Tal descrição é chamada de **esquema do banco de dados**, que é especificado durante o projeto do banco de dados e não se espera que mude com frequência. [6] A maioria dos modelos de dados tem certas convenções para representar esquemas como diagramas.[7] A representação de um esquema é chamada de **diagrama de esquema**. A Figura 2.1 mostra um diagrama de esquema para o banco de dados da Figura 1.2; o diagrama apresenta a estrutura de cada tipo de registro, mas não as instâncias reais dos registros. Chamamos cada objeto no esquema — por exemplo, ALUNO ou DISCIPLINA — de **construtor do esquema**.

Um diagrama de esquema representa apenas *alguns aspectos* de um esquema, como os nomes de tipos de registro e itens de dados, e alguns tipos de restrições. Outros aspectos não são especificados nele; por exemplo, a Figura 2.1 não mostra nem o tipo de dado de cada item de dados nem os relacionamentos entre os diversos arquivos. Muitos tipos de restrições não são apresentados nos diagramas de esquema. Uma restrição do tipo *alunos prestes a se formarem em ciência da computação precisam realizar o curso CC1310 antes do final de seu segundo ano* é muito difícil de representar em forma de diagrama.

ALUNO

Nome	Numero_aluno	Tipo_aluno	Curso

DISCIPLINA

Nome_ disciplina	Numero_ disciplina	Creditos	Departamento

PRE_REQUISITO

Numero_disciplina	Numero_pre_requisito

TURMA

Identificacao_ turma	Numero_ disciplina	Semestre	Ano	Professor

HISTORICO_ESCOLAR

Numero_aluno	Identificacao_turma	Nota

Figura 2.1
Diagrama de esquema para o banco de dados da Figura 1.2.

Os dados reais armazenados em um banco de dados podem mudar com muita frequência. Por exemplo, o banco de dados mostrado na Figura 1.2 muda toda vez que acrescentamos um novo aluno ou inserimos uma nova nota. Os dados no banco de dados em determinado momento no tempo são chamados de **estado** ou **instante do banco de dados**. Também são chamados de conjunto *atual* de **ocorrências** ou **instâncias** no banco de dados. Em determinado estado do banco de dados, cada construtor de esquema tem o próprio *conjunto de instâncias atuais*; por exemplo, o construtor ALUNO terá o conjunto de entidades de cada aluno (registros) como suas instâncias. Muitos estados de banco de dados podem ser construídos para corresponder a um esquema de banco de dados em particular. Toda vez que inserimos ou excluímos um registro ou alteramos o valor de um item de dados em um registro, mudamos de um estado do banco de dados para outro.

A distinção entre esquema e estado de banco de dados é muito importante. Quando **definimos** um novo banco de dados, especificamos seu esquema apenas para o SGBD. Nesse ponto, o estado do banco de dados correspondente é o *estado vazio*, sem dados. Obtemos o *estado inicial* do banco de dados quando ele é **populado** ou **carregado** com os dados iniciais. Daí em diante, toda vez que uma operação de atualização

[6] Mudanças no esquema normalmente são necessárias à medida que os requisitos das aplicações do banco de dados mudam. Sistemas de banco de dados mais novos incluem operações para permitir mudanças de esquema, embora esse processo seja mais complicado do que as simples atualizações no banco de dados.

[7] Em terminologia de banco de dados, em inglês, é comum usar *schemas* como plural para *schema*, embora *schemata* seja a forma no plural apropriada. A palavra *scheme* também é usada para se referir a um esquema.

é aplicada, obtemos outro estado do banco de dados. Em qualquer ponto no tempo, o banco de dados tem um *estado atual*[8] O SGBD é parcialmente responsável por garantir que todo estado do banco de dados seja um **estado válido**— ou seja, um estado que satisfaça a estrutura e as restrições especificadas no esquema. Logo, especificar um esquema correto para o SGBD é extremamente importante, e o esquema deve ser projetado com extremo cuidado. O SGBD armazena as descrições das construções e restrições do esquema — também denominadas **metadados** — no catálogo do SGBD, de modo que o software do SGBD possa às vezes é chamado de **intenção**, e um estado do banco de dados é chamado de **extensão** do esquema.

Embora, como já mencionamos, o esquema não deva mudar com frequência, não é raro que as mudanças, ocasionalmente, precisem ser aplicadas ao esquema, à medida que os requisitos da aplicação mudam. Por exemplo, podemos decidir que outro item de dados precisa ser armazenado para cada registro em um arquivo, como a inclusão de Data_nascimento ao esquema ALUNO da Figura 2.1. Isso é conhecido como **evolução do esquema**. A maioria dos SGBDs modernos possui algumas operações para evolução de esquema que podem ser aplicadas enquanto o banco de dados está em funcionamento.

2.2 Arquitetura de três esquemas e independência de dados

Três das quatro características importantes da abordagem de banco de dados, listadas na Seção 1.3, são (1) uso de um catálogo para armazenar a descrição (esquema) do banco de dados de modo a torná-lo autodescritivo, (2) isolamento de programas e dados (independência entre dados do programa e operação do programa) e (3) suporte para múltiplas visões do usuário. Nesta seção, especificamos uma arquitetura para sistemas de banco de dados, chamada **arquitetura de três esquemas**,[9] que foi proposta para ajudar a alcançar e visualizar essas características. Depois, vamos discutir melhor o conceito de independência de dados.

2.2.1 A arquitetura de três esquemas

O objetivo da arquitetura de três esquemas, ilustrada na Figura 2.2, é separar as aplicações do usuário do banco de dados físico. Nessa arquitetura, os esquemas podem ser definidos nos três níveis a seguir:

1. O **nível interno** tem um **esquema interno**, que descreve a estrutura do armazenamento físico do banco de dados. O esquema interno usa um modelo de dados físico e descreve os detalhes completos do armazenamento de dados e caminhos de acesso para o banco de dados.
2. O **nível conceitual** tem um **esquema conceitual**, que descreve a estrutura do banco de dados inteiro para uma comunidade de usuários. O esquema conceitual oculta os detalhes das estruturas de armazenamento físico e se concentra na descrição de entidades, tipos de dados, relacionamentos, operações de usuário e restrições. Normalmente, um modelo de dados representativo é usado para descrever o esquema conceitual quando um sistema de banco de dados é implementado. Esse *esquema conceitual de implementação* costuma estar baseado em um *projeto de esquema conceitual* em um modelo de dados de alto nível.
3. O **nível externo** ou **de visão** inclui uma série de **esquemas externos** ou **visões do usuário**. Cada esquema externo descreve a parte do banco de dados em que um grupo de usuários em particular está interessado e oculta o restante do banco de dados do grupo de usuários. Como no nível anterior, cada esquema externo é comumente implementado usando um modelo de dados representativo, possivelmente baseado em um projeto de esquema externo em um modelo de dados de alto nível.

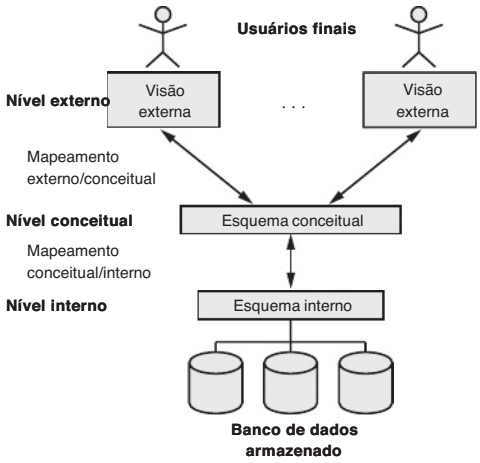


Figura 2.2
A arquitetura de três esquemas.

[8] O estado atual também é chamado de *instante atual* do banco de dados e de *instância do banco de dados*, mas preferimos usar o termo *instância* para nos referir a registros individuais.

[9] Isso também é conhecido como arquitetura ANSI/SPARC, mesmo nome do comitê que a propôs (Tsichritzis e Klug, 1978).

A arquitetura de três esquemas é uma ferramenta com a qual o usuário pode visualizar os níveis de esquema em um sistema de banco de dados. A maioria dos SGBDs não separa os três níveis completa e explicitamente, mas dá suporte a eles de alguma forma. Alguns sistemas mais antigos podem incluir detalhes de nível físico no esquema conceitual. A arquitetura ANSI de três níveis tem um lugar importante no desenvolvimento da tecnologia de banco de dados, pois separa, claramente, o nível externo dos usuários, o nível conceitual do banco de dados e o nível de armazenamento interno do projeto de um banco de dados. Ainda hoje ela é bastante aplicável no projeto de SGBDs. Na maioria dos SGBDs que têm suporte a visões do usuário, os esquemas externos são especificados no mesmo modelo de dados que descreve a informação no nível conceitual (por exemplo, um SGBD relacional como o Oracle utiliza SQL para isso). Alguns SGBDs permitem que diferentes modelos de dados sejam usados nos níveis conceitual e externo. Um exemplo é a Base de Dados Universal (Universal Data Base — UDB), um SGBD da IBM que utiliza o modelo relacional para descrever o esquema conceitual, mas que pode usar um modelo orientado a objeto para descrever um esquema externo.

Observe que os três esquemas são apenas *descrições* dos dados; os dados armazenados que *realmente* existem estão apenas no nível físico. Em um SGBD baseado na arquitetura de três esquemas, cada grupo de usuários recorre ao seu próprio esquema externo. Assim, o SGBD precisa transformar uma solicitação especificada em um esquema externo em uma solicitação no esquema conceitual, e depois em uma solicitação no esquema interno para o processamento no banco de dados armazenado. Se a solicitação for uma recuperação, os dados extraídos do banco de dados armazenado devem ser reformatados para corresponder à visão externa do usuário. Os processos de transformação de requisições e os resultados entre os níveis são chamados de **mapeamentos**. Esses mapeamentos podem ser demorados, de modo que alguns SGBDs — especialmente aqueles que servem para dar suporte a pequenos bancos de dados — não suportam visões externas. Porém, mesmo em tais sistemas, uma certa quantidade de mapeamento é necessária para transformar solicitações entre os níveis conceitual e interno.

2.2.2 Independência de dados

A arquitetura de três esquemas pode ser usada para explicar melhor o conceito de **independência de dados**, que pode ser definida como a capacidade de alterar o esquema em um nível do sistema de banco de dados sem ter de alterar o esquema no nível mais alto. Podemos definir dois tipos de independência de dados:

1. **Independência lógica de dados** é a capacidade de alterar o esquema conceitual sem ter de alterar os esquemas externos ou os programas de aplicação. Podemos alterar o esquema conceitual para expandir o banco de dados (acrescentando um tipo de registro ou item de dado), para alterar restrições ou para reduzir o banco de dados (removendo um tipo de registro ou item de dado). No último caso, esquemas externos que se referem apenas aos dados restantes não seriam afetados. Por exemplo, o esquema externo da Figura 1.5(a) não deverá ser afetado pela alteração do arquivo HISTORICO_ESCOLAR (ou tipo de registro), mostrado na Figura 1.2, para aquele mostrado na Figura 1.6(a). Somente a definição da visão e os mapeamentos precisam ser alterados em um SGBD que suporta a independência lógica de dados. Depois que o esquema conceitual passa por uma reorganização lógica, os programas de aplicação que referenciam as construções do esquema externo devem trabalhar da mesma forma que antes. As mudanças nas restrições podem ser aplicadas ao esquema conceitual sem afetar os esquemas externos ou os programas de aplicação.

2. **Independência física de dados** é a capacidade de alterar o esquema interno sem ter de alterar o esquema conceitual. Logo, os esquemas externos também não precisam ser alterados. Mudanças no esquema interno podem ser necessárias porque alguns arquivos físicos foram reorganizados — por exemplo, ao criar estruturas de acesso adicionais — para melhorar o desempenho da recuperação ou atualização. Se os mesmos dados de antes permanecerem no banco de dados, provavelmente não teremos de alterar o esquema conceitual. Por exemplo, oferecer um caminho de acesso para melhorar a velocidade de recuperação dos registros de turma (Figura 1.2) por semestre e ano não deverá exigir que uma consulta como *listar todas as turmas oferecidas no segundo semestre de 2008* seja alterada, embora a consulta seja executada com mais eficiência pelo SGBD ao utilizar o novo caminho de acesso.

Em geral, a independência física de dados existe na maioria dos bancos de dados e ambientes de arquivo, nos quais detalhes físicos, como a localização exata dos dados no disco, e detalhes de hardware sobre codificação do armazenamento, posicionamento, compactação, divisão, mesclagem de registros, e

assim por diante, são ocultados do usuário. As demais aplicações ignoram esses detalhes. Por sua vez, a independência lógica de dados é mais difícil de ser alcançada porque permite alterações estruturais e de restrição sem afetar os programas de aplicação — um requisito muito mais estrito.

Sempre que temos um SGBD de múltiplos níveis, seu catálogo deve ser expandido para incluir informações sobre como mapear solicitações e dados entre os diversos níveis. O SGBD usa software adicional para realizar esses mapeamentos, recorrendo à informação de mapeamento no catálogo. A independência de dados ocorre porque, quando o esquema é alterado em algum nível, o esquema no próximo nível mais alto permanece inalterado; somente o *mapeamento* entre os dois níveis é alterado. Logo, os programas de aplicação que fazem referência ao esquema de nível mais alto não precisam ser alterados.

A arquitetura de três esquemas pode tornar mais fácil obter a verdadeira independência de dados, tanto física quanto lógica. Porém, os dois níveis de mapeamentos criam uma sobrecarga durante a compilação ou execução de uma consulta ou programa, levando a baixa eficiência do SGBD. Por causa disso, poucos SGBDs implementaram a arquitetura completa de três esquemas.

2.3 Linguagens e interfaces do banco de dados

Na Seção 1.4, discutimos a variedade de usuários atendidos por um SGBD. O sistema precisa oferecer linguagens e interfaces apropriadas para cada categoria de usuário. Nesta seção, discutimos os tipos de linguagens e interfaces oferecidas por um SGBD e as categorias de usuário alvo de cada interface.

2.3.1 Linguagens do SGBD

Quando o projeto de um banco de dados é finalizado e um SGBD é escolhido para implementá-lo, o primeiro passo é especificar esquemas conceituais e internos para o banco de dados e quaisquer mapeamentos entre os dois. Em muitos SGBDs, onde não é mantida nenhuma separação estrita de níveis, uma linguagem, chamada **linguagem de definição de dados** (**DDL** — Data Definition Language), é usada pelo DBA e pelos projetistas de banco de dados para definir os dois esquemas. O SGBD terá um compilador da DDL cuja função é processar instruções da DDL a fim de identificar as descrições dos construtores de esquema e armazenar a descrição de esquema no catálogo do SGBD.

Nos SGBDs que mantêm uma separação clara entre os níveis conceitual e interno, a DDL é usada para especificar apenas o esquema conceitual. Outra linguagem, a **linguagem de definição de armazenamento** (**SDL** — Storage Definition Language), é utilizada para especificar o esquema interno. Os mapeamentos entre os dois esquemas podem ser especificados em qualquer uma dessas linguagens. Na maioria dos SGBDs relacionais, *não existe uma linguagem específica* que realiza o papel de SDL. Em vez disso, o esquema interno é especificado por uma combinação de funções, parâmetros e especificações relacionadas ao armazenamento, que permitem aos DBAs controlar opções de indexação e mapeamentos dos dados que serão armazenados. Para uma verdadeira arquitetura de três esquemas, precisaríamos de uma terceira linguagem, a **linguagem de definição de visão** (**VDL** — View Definition Language), para especificar visões do usuário e seus mapeamentos ao esquema conceitual, mas na maioria dos SGBDs *a DDL é usada para definir tanto o esquema conceitual como o externo*. Nos SGBDs relacionais, a SQL é usada pela VDL para definir **visões** do usuário ou da aplicação como resultados de consultas predefinidas (ver capítulos 4 e 5).

Quando os esquemas são compilados e o banco de dados é populado, os usuários precisam de alguma forma de manipulá-lo. As manipulações típicas incluem recuperação, inserção, exclusão e modificação dos dados. O SGBD oferece um conjunto de operações ou uma linguagem chamada **linguagem de manipulação de dados** (**DML** — Data Manipulation Language) para essas finalidades.

Nos SGBDs atuais, esses tipos de linguagens normalmente *não são considerados linguagens distintas*; ao contrário, uma linguagem integrada e abrangente é usada na definição do esquema conceitual, definição de visão e manipulação de dados. A definição do armazenamento, em geral, é mantida em separado, pois serve para definir estruturas de armazenamento físicas, para ajustar o desempenho do sistema de banco de dados, o que é normalmente feito pelos DBAs. Um exemplo típico de linguagem de banco de dados abrangente é a linguagem de banco de dados relacional SQL (ver capítulos 4 e 5), que representa uma combinação de DDL, VDL e DML, bem como as instruções para especificação de restrição, evolução de esquema e outros recursos. A SDL era um componente nas primeiras versões da SQL, mas foi removida da linguagem para mantê-la apenas nos níveis conceitual e externo.

Existem dois tipos de DMLs. Uma DML de **alto nível** ou **não procedural** pode ser utilizada para espe-

cificar operações de banco de dados complexas de forma concisa. Muitos SGBDs permitem que instruções DML de alto nível sejam inseridas interativamente em um monitor ou terminal ou sejam embutidas em uma linguagem de programação de uso geral. Nesse último caso, as instruções DML precisam ser identificadas dentro do programa, de modo que possam ser extraídas por um pré-compilador e processadas por um SGBD. Uma DML de **baixo nível** ou **procedural** *deve* ser embutida em uma linguagem de programação de uso geral. Esse tipo de DML, em geral, recupera registros individuais ou objetos do banco de dados e processa cada um deles separadamente. Portanto, precisa de construções de linguagem de programação, como o looping, para recuperar e processar cada registro de um conjunto de registros. DMLs de baixo nível também são chamadas de DMLs que tratam **um registro de cada vez**, devido a essa propriedade. A DL/1, uma DML projetada para o modelo hierárquico, é uma DML de baixo nível que emprega comandos como GET UNIQUE, GET NEXT ou GET NEXT WITHIN PARENT para navegar de um registro para outro em uma hierarquia de registros no banco de dados. As DMLs de alto nível, como a SQL, podem especificar e recuperar muitos registros em uma única instrução DML; portanto, elas são chamadas de DMLs de **um conjunto de cada vez** ou **orientadas a conjunto**. Uma consulta em uma DML de alto nível normalmente especifica *quais* dados recuperar, em vez de *como* recuperá-los; portanto, essas linguagens também são chamadas **declarativas**.

Sempre que comandos DML, sejam eles de alto ou de baixo nível, são incorporados em uma linguagem de programação de uso geral, ela é chamada de **linguagem hospedeira** e a DML é chamada de **sublinguagem de dados**.[10] Por sua vez, uma DML de alto nível usada em uma maneira interativa é chamada **linguagem de consulta**. Em geral, comandos de recuperação e atualização de uma DML de alto nível podem ser usados de maneira interativa e, portanto, são considerados parte da linguagem de consulta.[11]

Usuários finais casuais costumam usar uma linguagem de consulta de alto nível para especificar suas solicitações, enquanto os programadores usam a DML em sua forma embutida. Para usuários comuns e paramétricos, normalmente existem **interfaces amigáveis ao usuário** para interagir com o banco de dados; estas também podem ser usadas por usuários casuais ou outros que não querem aprender os detalhes de uma linguagem de consulta de alto nível. Discutiremos esses tipos de interfaces a seguir.

2.3.2 Interfaces de SGBD

As interfaces amigáveis ao usuário oferecidas por um SGBD podem incluir:

Interfaces baseadas em menu para clientes Web ou de navegação. Essas interfaces apresentam ao usuário uma lista de opções (chamadas **menus**) que acompanham o usuário na formulação de uma solicitação. Os menus acabam com a necessidade de memorizar os comandos específicos e a sintaxe de uma linguagem de consulta; em vez disso, a consulta é composta passo a passo ao escolher opções de um menu que é exibido pelo sistema. Os menus pull-down são uma técnica muito popular nas **interfaces de usuário baseadas na Web**. Eles também são usados com frequência em **interfaces de navegação**, que permitem a um usuário examinar o conteúdo de um banco de dados de uma maneira exploratória e desestruturada.

Interfaces baseadas em formulário. Uma interface baseada em formulário apresenta um formulário para cada usuário. Os usuários podem preencher todas as entradas do **formulário** para inserir novos dados ou preencher apenas certas entradas, neste caso o SGBD recuperará os dados correspondentes para as entradas restantes. Os formulários, normalmente, são projetados e programados para usuários finais como interfaces para transações já programadas. Muitos SGBDs possuem **linguagens de especificação de formulários**, que são linguagens especiais que ajudam os programadores a especificar tais formulários. A SQL*Forms é uma linguagem baseada em formulário que especifica consultas usando um formulário projetado em conjunto com o esquema de banco de dados relacional. O Oracle Forms é um componente do pacote de produtos da Oracle que oferece um extenso conjunto de recursos para projetar e montar aplicações usando formulários. Alguns sistemas possuem utilitários para definir um formulário, permitindo que ousuário final construa interativamente um formulário de amostra na tela.

Interfaces gráficas com o usuário. Uma GUI normalmente apresenta um esquema para o usuário em formato de diagrama. O usuário pode então especificar uma consulta manipulando o diagrama. Em

[10] Em bancos de dados de objeto, as sublinguagens hospedeiras e de dados formam uma linguagem integrada — por exemplo, C++ com algumas extensões, para dar suporte à funcionalidade de banco de dados. Alguns sistemas relacionais também oferecem linguagens integradas — por exemplo, a PL/SQL do Oracle.

[11] De acordo com seu significado em inglês, a palavra *consulta* (*query*), na realidade, deveria ser usada para descrever apenas leituras, e não atualizações.

muitos casos, as GUIs utilizam menus e formulários. A maioria delas utiliza um **dispositivo de apontamento**, como um mouse, para selecionar certas partes do diagrama de esquema apresentado.

Interfaces de linguagem natural. Essas interfaces aceitam solicitações escritas em inglês, ou em outro idioma, e tentam *entendê-las*. Uma interface de linguagem natural costuma ter o próprio *esquema*, que é semelhante ao esquema conceitual do banco de dados, bem como um dicionário de palavras importantes. Essa interface recorre às palavras em seu esquema, bem como ao conjunto de palavras-padrão em seu dicionário, para interpretar a solicitação. Se a interpretação for bem-sucedida, a interface gera uma consulta de alto nível correspondente à solicitação de linguagem natural e a submete ao SGBD para processamento; caso contrário, um diálogo é iniciado com o usuário para esclarecer a solicitação. As funcionalidades das interfaces de linguagem natural não têm avançado rapidamente. Hoje, vemos mecanismos de busca que aceitam sequências de palavras de linguagem natural (como inglês) e as combinam com documentos em sites específicos (para mecanismos de busca locais) ou páginas na Web em geral (para mecanismos como Google ou Ask). Eles utilizam índices pré-definidos sobre palavras e funções de pontuação (ranking) para recuperar e apresentar documentos resultantes em um grau decrescente de combinação. Essas interfaces de consulta textual 'em forma livre' ainda não são comuns nos bancos de dados de modelo relacional estruturado ou legado, embora uma área de pesquisa chamada **consulta baseada em palavra-chave** tenha surgido recentemente para os bancos de dados relacionais.

Entrada e saída de voz. O uso limitado da voz como entrada para uma consulta e como resposta para uma pergunta ou resultado de uma solicitação está se tornando comum. Aplicações com vocabulários limitados, como consultas do catálogo telefônico, chegada/saída de voo e informações de conta de cartão de crédito, estão permitindo que a voz, para entrada e saída, facilite o acesso a essas informações pelos clientes. A entrada de voz é detectada usando uma biblioteca de palavras predefinidas e usadas para configurar os parâmetros fornecidos para as consultas. Para a saída, acontece uma conversão semelhante de texto ou de números para voz.

Interfaces para usuários paramétricos. Os usuários paramétricos, como caixas de banco, em geral possuem um pequeno conjunto de operações que precisam realizar repetidamente. Por exemplo, um caixa pode usar teclas de função isoladas para fazer transações de rotina e repetitivas, como depósitos em conta ou saques, ou consultas de saldo. Os analistas de sistemas e programadores projetam e implementam uma interface especial para cada classe conhecida de usuários finais. Normalmente, um pequeno conjunto de comandos abreviados é incluído, com o objetivo de minimizar o número de toques de teclas exigidos para cada solicitação. Por exemplo, as teclas de função em um terminal podem ser programadas para iniciar diversos comandos. Isso permite que o usuário paramétrico prossiga com um número mínimo de toques de teclas.

Interfaces para o DBA. A maioria dos sistemas de banco de dados contém comandos privilegiados que podem ser usados apenas pelos DBAs. Estes incluem comandos para criar contas, definir parâmetros do sistema, conceder autorização de conta, alterar um esquema e reorganizar as estruturas de armazenamento de um banco de dados.

2.4 O ambiente do sistema de banco de dados

Um SGBD é um sistema de software complexo. Nesta seção, vamos discutir os tipos de componentes de software que constituem um SGBD e os tipos de software de sistemas de computação com os quais ele interage.

2.4.1 Módulos componentes do SGBD

A Figura 2.3 ilustra, de uma forma simplificada, os componentes típicos do SGBD. A figura está dividida em duas partes. A parte superior refere-se aos vários usuários do ambiente de banco de dados e suas interfaces. A parte inferior mostra os detalhes internos do SGBD, responsáveis pelo armazenamento de dados e processamento de transações.

O banco de dados e o catálogo do SGBD, habitualmente, são armazenados em disco. O acesso ao disco é controlado, em especial, pelo **sistema operacional (SO)**, que escalona a leitura/escrita em disco. Muitos SGBDs possuem o próprio módulo de **gerenciamento de buffer** para planejar a leitura/escrita em disco, pois isso tem um efeito considerável sobre o desempenho. A redução da leitura/escrita em disco melhora o desempenho de maneira considerável. Um módulo **gerenciador de dados armazenados** controla o acesso às informações do SGBD que são armazenadas em disco, não importando se elas fazem parte do banco de dados ou do catálogo.

Primeiramente, vamos considerar a parte superior da Figura 2.3. Ela mostra as interfaces para DBAs, usuários casuais que trabalham com interfaces interativas para formular consultas, programadores de aplicação que criam programas usando algumas linguagens de programação hospedeira, e usuários

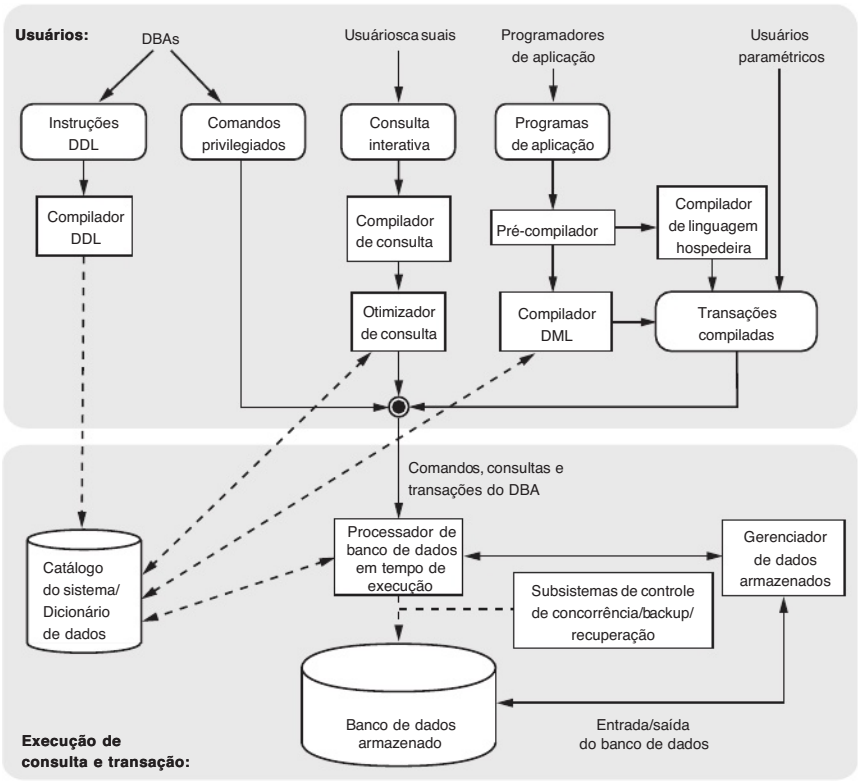


Figura 2.3
Módulos componentes de um SGBD e suas interações.

paramétricos que realizam a entrada dos dados fornecendo parâmetros para transações predefinidas. Os DBAs definem o banco de dados e realizam ajustes, alterando sua definição por meio da DDL e de outros comandos privilegiados.

O compilador da DDL processa as definições de esquema especificadas e armazena as descrições dos esquemas (metadados) no catálogo do SGBD. O catálogo inclui informações como os nomes e os tamanhos dos arquivos, nomes e tipos de dados dos itens de dados, detalhes de armazenamento de cada arquivo, informações de mapeamento entre esquemas e restrições. Além disso, o catálogo armazena muitos outros tipos de informações essenciais para os módulos do SGBD, que podem, então, utilizar as informações do catálogo, conforme a necessidade.

Usuários casuais do banco de dados interagem usando alguma forma de interface, que chamamos de interface de **consulta interativa,** na Figura 2.3. Não mostramos explicitamente qualquer interação baseada em menu ou baseada em formulário que possa ser usada para gerar a consulta interativa de maneira automática. Essas consultas são analisadas e validadas pela exatidão da sintaxe da consulta, os nomes de arquivos e elementos de dados, e assim por diante, por um **compilador de consulta**, que as compila para um formato interno. Essa consulta interna está sujeita à otimização de consultas (discutida nos capítulos 19 e 20). Entre outras coisas, o **otimizador de consulta** preocupa-se com o rearranjo e a possível reordenação de operações, com a eliminação de redundâncias e uso dos algoritmos e índices corretos durante a execução. Ele consulta o catálogo do sistema em busca de informações estatísticas e outras informações físicas sobre os dados armazenados, gerando um código executável que realiza as operações necessárias para

a consulta e faz chamadas ao processador em tempo de execução.

Os programadores de aplicação escrevem programas em linguagens hospedeiras, como Java, C ou C++, que são submetidas a um pré-compilador. O **pré-compilador** extrai comandos DML do programa de aplicação escrito em uma linguagem de programação hospedeira. Esses comandos são enviados ao compilador DML para serem compilados em código objeto para o acesso ao banco de dados. O restante do programa é enviado ao compilador da linguagem hospedeira. Os códigos objeto para os comandos DML e o restante do programa são ligados ("linkados"), formando uma transação programada, cujo código executável inclui chamadas para o processador de banco de dados em tempo de execução. As transações programadas são executadas repetidamente pelos usuários paramétricos, que apenas fornecem os parâmetros para as transações. Cada execução é considerada uma transação separada. Um exemplo é uma transação de saque bancário, no qual o número da conta e o valor podem ser fornecidos como parâmetros.

Na parte inferior da Figura 2.3, o **processador de banco de dados em tempo de execução** executa (1) os comandos privilegiados, (2) os planos de consulta executáveis e (3) as transações programadas com parâmetros em tempo de execução. Ele trabalha com o **catálogo do sistema** e pode atualizá-lo com estatísticas. Também trabalha com o **gerenciador de dados armazenados**, que, por sua vez, utiliza os serviços básicos do sistema operacional para executar operações de entrada/saída (leitura/esc rita) de baixo nível entre o disco e a memória principal. O processador de banco de dados em tempo de execução cuida de outros aspectos da transferência de dados, como o gerenciamento de buffers na memória principal. Alguns SGBDs têm o próprio módulo de gerenciamento de buffer, enquanto outros dependem do SO para fazê-lo. Mostramos os **sistemas de controle de concorrência** e **backup e recuperação** separadamente como um módulo nessa figura. Eles são integrados ao processador de banco de dados em tempo de execução para fins de gerenciamento de transação.

Agora, é comum que o **programa cliente** acesse o SGBD executando em um computador separado do computador em que o banco de dados reside. O primeiro é chamado **computador cliente**, que executa um software cliente do SGBD, e o segundo é chamado **servidor de banco de dados**. Em alguns casos, o cliente acessa um computador intermediário, conhecido como **servidor de aplicações**, que, por sua vez, acessa o servidor de banco de dados. Abordaremos melhor esse assunto na Seção 2.5.

A Figura 2.3 não pretende descrever um SGBD específico; ao contrário, ela ilustra os módulos básicos de SGBD. O SGBD interage com o sistema operacional quando os acessos ao disco — ao banco de dados ou ao catálogo — são necessários. Se o computador for compartilhado por muitos usuários, o SO escalonará as solicitações de acesso ao disco do SGBD e o processamento do SGBD juntamente com outros processos. Além disso, se o computador estiver, principalmente, dedicado a executar o servidor de banco de dados, o SGBD controlará a memória principal bufferizando as páginas de disco. O sistema também realiza a interface com compiladores das linguagens de programação hospedeiras de uso geral, e com os servidores de aplicação e programas cliente rodando em máquinas separadas, por meio da interface de rede do sistema.

2.4.2 Utilitários do sistema de banco de dados

Além dos componentes de software que descrevemos, a maioria dos SGBDs possui **utilitários de banco de dados** que ajudam o DBA a gerenciar o sistema. Os utilitários têm, basicamente, os seguintes tipos de funções:

- **Carga.** Um utilitário de carga é usado para carregar os arquivos de dados existentes — como arquivos de texto ou arquivos sequenciais — no banco de dados. Normalmente o formato atual do arquivo de dado (srcem) e a estrutura do arquivo do banco de dados desejado (destino) são especificados pelo utilitário que reformata automaticamente os dados e os armazena no banco de dados. Com a proliferação de SGBDs, a transferência de dados de um SGBD para outro está se tornando comum em muitas organizações. Alguns fornecedores de SGBDs estão oferecendo produtos que geram os programas de carga apropriados, tendo como base descrições de armazenamento de banco de dados de srcem e destino (esquemas internos). Essas ferramentas também são chamadas de **ferramentas de conversão** Para o SGBD hierárquico chamado IMS (da IBM) e para muitos SGBDs de rede, incluindo o IDMS (da Computer Associates), SUPRA (da Cincom) e Image (da HP), os fornecedores ou empresas de terceiros estão disponibilizando uma série de ferramentas de conversão (por exemplo, o SUPRA Server SQL da Cincom) para transformar os dados para o modelo relacional.
- **Backup.** Um utilitário de backup cria uma cópia de segurança do banco de dados, nor-

malmente copiando o banco de dados inteiro para fita ou outro meio de armazenamento em massa. A cópia backup pode ser usada para restaurar o banco de dados no caso de uma falha catastrófica no disco. Os backups incrementais também costumam ser utilizados, e registram apenas as mudanças ocorridas após o backup anterior. O backup incremental é mais complexo, mas economiza espaço de armazenamento.

- **Reorganização do armazenamento do banco de dados.** Esse utilitário pode ser usado para reorganizar um conjunto de arquivos do banco de dados em diferentes organizações de arquivo, e cria novos caminhos de acesso para melhorar o desempenho.
- **Monitoração de desempenho.** Esse utilitário monitora o uso do banco de dados e oferece estatísticas ao DBA. O DBA usa as estatísticas para tomar decisões se deve ou não reorganizar arquivos ou se deve incluir ou remover índices para melhorar o desempenho.

Podem estar disponíveis outros utilitários para classificar arquivos, tratar da compactação de dados, monitorar o acesso pelos usuários, realizar a interface com a rede e desempenhar outras funções.

2.4.3 Ferramentas, ambientes de aplicação e facilidades de comunicações

Outras ferramentas estão frequentemente disponíveis aos projetistas de bancos de dados, usuários e ao SGBD. Ferramentas CASE[12] são usadas na fase de projeto dos sistemas de banco de dados. Outra ferramenta que pode ser muito útil em grandes organizações é um **sistema de dicionário de dados** (ou de **repositório de dados**) expandido. Além de armazenar informações de catálogo sobre esquemas e restrições, o dicionário de dados armazena decisões do projeto, padrões de uso, descrições de programa de aplicação e informações do usuário. Esse sistema também é chamado de **repositório de informação**. Essa informação pode ser acessada *diretamente* pelos usuários ou pelo DBA, quando necessário. Um utilitário de dicionário de dados é semelhante ao catálogo do SGBD, mas inclui uma variedade maior de informações e é acessado principalmente pelos usuários, em vez de ser acessada pelo software de SGBD.

Ambientes de desenvolvimento de aplicação, como o PowerBuilder (Sybase) ou o JBuilder (Borland), são muito populares. Esses sistemas oferecem um ambiente para desenvolver aplicações de banco de dados e incluem facilidades que ajudam em muitas facetas dos sistemas, incluindo projeto de banco de dados, desenvolvimento GUI, consulta e atualização, e desenvolvimento de programas de aplicação.

O SGBD também precisa realizar a interface com o **software de comunicações**, cuja função é permitir que os usuários em locais remotos do sistema de banco de dados acessem o banco de dados por meio de terminais de computador, estações de trabalho ou computadores pessoais. Estes são conectados ao local do banco de dados por meio de hardware de comunicações de dados, como roteadores da Internet, linhas telefônicas, redes de longa distância, redes locais ou dispositivos de comunicação por satélite. Muitos sistemas de banco de dados do mercado possuem pacotes de comunicação que trabalham com o SGBD. O sistema integrado de SGBD e comunicações de dados é chamado de sistema **DB/DC**. Além disso, alguns SGBDs estão fisicamente distribuídos em várias máquinas. Nesse caso, redes de comunicações são necessárias para conectar as máquinas. Estas, com frequência, são **redes locais** (**LANs** — Local Area Networks), mas também podem ser outros tipos de redes.

2.5 Arquiteturas centralizadas e cliente/servidor para SGBDs

2.5.1 Arquitetura de SGBDs centralizada

As arquiteturas para SGBDs têm seguido tendências semelhantes àquelas dos sistemas de computação em geral. As arquiteturas anteriores usavam computadores mainframe para oferecer o processamento principal para todas as funções do sistema, incluindo programas de aplicação do usuário e programas de interface com o usuário, bem como toda a funcionalidade do SGBD. O motivo é que a maioria dos usuários acessava tais sistemas por terminais de computador, que não tinham poder de processamento e apenas ofereciam capacidades de exibição. Portanto, todo o processamento era realizado remotamente no computador central, e somente informações de exibição e controles eram enviados do computador para os terminais de vídeo, que eram conectados ao computador central por meio de vários tipos de redes de comunicações.

À medida que os preços do hardware caíram, a maioria dos usuários substituiu seus terminais com PCs e estações de trabalho. No início, os sistemas

[12] Embora CASE signifique engenharia de software auxiliada por computador, muitas de suas ferramentas são usadas, principalmente, para o projeto de banco de dados.

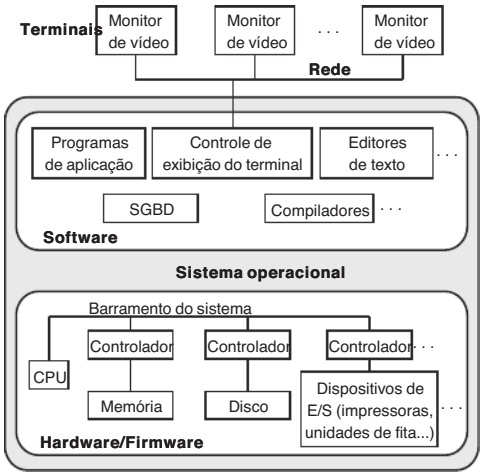


Figura 2.4
Uma arquitetura física centralizada.

de banco de dados usavam esses computadores de modo semelhante à forma que usavam terminais de vídeo, de maneira que o SGBD em si ainda era um SGBD **centralizado,** em que toda sua funcionalidade, execução de programas de aplicação e processamento de interface do usuário eram executados em uma máquina. A Figura 2.4 ilustra os componentes físicos em uma arquitetura centralizada. Gradualmente, os sistemas de SGBD começaram a explorar o poder de processamento disponível no lado do usuário, o que levou às arquiteturas de SGBD cliente/servidor.

2.5.2 Arquiteturascliente/servidor básicas

Primeiro, vamos discutir a arquitetura cliente/servidor em geral, e depois veremos como ela se aplica aos SGBDs. A **arquitetura cliente/servidor** foi desenvolvida para lidar com ambientes de computação em que um grande número de PCs, estações de trabalho, servidores de arquivos, impressoras, servidores de banco de dados, servidores Web, servidores de correio (e-mail) e outros softwares e equipamentos são conectados por uma rede. A ideia é definir **servidores especializados** com funcionalidades específicas. Por exemplo, é possível conectar uma série de PCs ou pequenas estações de trabalho como clientes a um **servidor de arquivos** que mantém os arquivos das máquinas clientes. Outra máquina pode ser designada como **servidor de impressão** , sendo conectada a várias impressoras; todas as solicitações de impressão pelos clientes são encaminhadas a essa máquina. Os **servidores Web** ou **servidores de correio** também se encontram na categoria de servidor especializado. Os recursos fornecidos pelos servidores especializados podem ser acessados por muitas máquinas cliente. As **máquinas cliente** oferecem ao usuário as interfaces apropriadas para utilizar esses servidores, bem como poder de processamento local para executar aplicações locais. Esse conceito pode ser transportado para outros pacotes de software, com programas especializados — como o CAD (Computer-Aided Design) — sendo armazenados em máquinas servidoras específicas e acessíveis a múltiplos clientes. A Figura 2.5 mostra a arquitetura cliente/servidor no nível lógico; a Figura 2.6 é um diagrama simplificado que mostra a arquitetura física. Algumas máquinas seriam apenas locais do cliente (por exemplo, estações de trabalho sem disco ou estações de trabalho/PCs com discos, que têm apenas software cliente instalado). Outras máquinas seriam servidores dedicados, e outras ainda teriam funcionalidade de cliente e servidor.

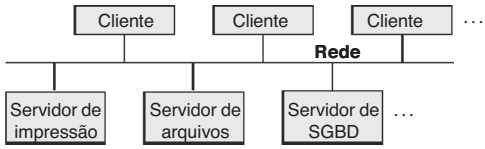


Figura 2.5
Arquitetura cliente/servidor lógica em duas camadas.

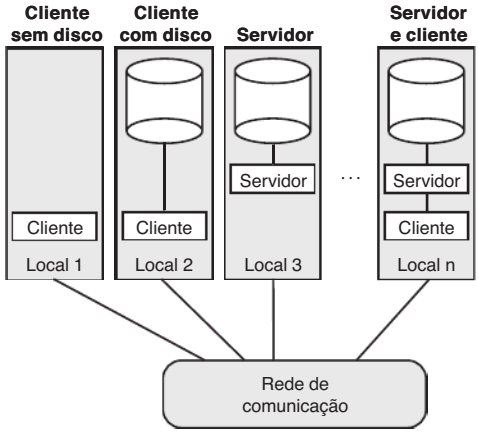


Figura 2.6
Arquitetura cliente/servidor física em duas camadas.

O conceito de arquitetura cliente/servidor assume uma estrutura básica composta por muitos PCs e estações de trabalho, além de um número menor de máquinas de grande porte (mainframes), conectados por LANs e outros tipos de redes de computadores. Um **cliente** nessa estrutura normalmente é uma máquina que oferece capacidades de interface com o usuário e processamento local. Quando um cliente requer acesso a alguma funcionalidade adicional — como acesso ao banco de dados — que não existe nessa máquina, ele se conecta a um servidor que oferece a funcionalidade necessária. Um **servidor** é um sistema com hardware e software que pode oferecer serviços às máquinas cliente, como acesso a arquivo, impressão, arquivamento ou acesso a banco de dados. Em geral, algumas máquinas têm instalados apenas softwares cliente, outras, apenas software servidor, e ainda outras podem incluir software cliente e servidor, conforme ilustrado na Figura 2.6. Porém, é mais comum que o software cliente e servidor seja executado em máquinas separadas. Dois tipos principais de arquiteturas de SGBD foram criados nessa estrutura cliente/servidor básica: **duas camadas** e **três camadas**.[13] Vamos explicar esses dois tipos a seguir.

2.5.3 Arquiteturas cliente/servidor de duas camadas para SGBDs

Em sistemas de gerenciamento de banco de dados relacional (SGBDRs), muitos dos quais começaram como sistemas centralizados, os componentes do sistema movidos inicialmente para o lado do cliente foram a interface com o usuário e os programas de aplicação. Como a SQL (ver capítulos 4 e 5) fornecia uma linguagem-padrão para os SGBDRs, isso criou um ponto de divisão lógico entre cliente e servidor. Assim, as funcionalidades de consulta e de transação relacionadas ao processamento da SQL permaneceram no lado do servidor. Em tal arquitetura, o servidor frequentemente é chamado **servidor de consulta** ou **servidor de transação**, pois oferece essas duas funcionalidades. Em um SGBDR, o servidor também é chamado de **servidor SQL**.

Os programas da interface com o usuário e os programas de aplicação podem ser executados no lado do cliente. Quando é necessário acessar o SGBD, o programa estabelece uma conexão com o SGBD (que está no lado do servidor); quando a conexão é criada, o programa cliente pode se comunicar com o SGBD. Um padrão denominado **Conectividade de Banco de Dados Aberta (ODBC —** Open Database Connectivity) oferece uma **interface de programação de aplicações (API** — Application Programming Interface), que permite que os programas do cliente chamem o SGBD, desde que as máquinas cliente e servidor tenham o software necessário instalado. A maioria dos vendedores de SGBD oferece drivers ODBC para seus sistemas. Um programa cliente pode se conectar a vários SGBDRs e enviar solicitações de consulta e transação usando a API da ODBC, que são processadas nos servidores. Quaisquer resultados de consulta são enviados de volta ao programa cliente, que pode processar e exibir os resultados conforme a necessidade. Foi definido também um padrão para a linguagem de programação Java, chamado **JDBC**. Isso permite que programas cliente em Java acessem um ou mais SGBDs por meio de uma interface-padrão.

A abordagem diferente para a arquitetura cliente/servidor em duas camadas foi seguida por alguns SGBDs orientados a objeto, em que os módulos de software do sistema foram divididos entre cliente e servidor de uma maneira mais integrada. Por exemplo, o **nível do servidor** pode incluir a parte do software do SGBD responsável por tratar do armazenamento de dados nas páginas do disco, controle de concorrência local e recuperação, buffering e caching de páginas do disco, e outras funções semelhantes. Enquanto isso, o **nível do cliente** pode tratar da interface com o usuário; funções de dicionário de dados; interações do SGBD com os compiladores da linguagem de programação; otimização global da consulta, controle de concorrência e recuperação entre vários servidores; estruturação de objetos complexos com base nos dados dos buffers; e outras funções semelhantes. Nessa abordagem, a interação cliente/servidor é mais acoplada e é feita internamente pelos módulos de SGBD — alguns dos quais residem no cliente e alguns no servidor —, e não pelos usuários/programadores. A divisão exata da funcionalidade pode variar de um sistema para outro. Em tal arquitetura cliente/servidor, o servidor é chamado de **servidor de dados**, pois oferece dados organizados em páginas de disco ao cliente. Esses dados podem então ser estruturados pelos programas clientes pelo software cliente do SGBD em objetos.

As arquiteturas descritas aqui são chamadas **arquiteturas de duas camadas** porque os componentes de software são distribuídos por dois sistemas: cliente e servidor. As vantagens dessa arquitetura são sua simplicidade e compatibilidade transparente com os sistemas existentes. O surgimento da Web mudou os papéis de clientes e servidores, levando à arquitetura de três camadas.

[13] Existem muitas outras variações de arquiteturas cliente/servidor. Discutiremos aqui as duas mais básicas.

2.5.4 Arquiteturas de três camadas e *n* camadas para aplicações Web

Muitas aplicações Web utilizam uma arquitetura chamada **arquitetura de três camadas**, que acrescenta uma camada intermediária entre o cliente e o servidor de banco de dados, conforme ilustrado na Figura 2.7(a).

Essa **camada intermediária** é chamada de **servidor de aplicação** ou **servidor Web**, dependendo da aplicação. Esse servidor desempenha um papel intermediário pela execução de programas de aplicação e armazenamento de regras de negócios (procedimentos ou restrições), que são usados para acessar os dados do servidor de banco de dados. Ela também pode melhorar a segurança do banco de dados, verificando as credenciais de um cliente antes de encaminhar uma solicitação ao servidor de banco de dados. Os clientes têm interfaces GUI e algumas regras de negócios adicionais, específicas da aplicação. O servidor intermediário aceita e processa solicitações do cliente, e envia consultas e comandos do banco de dados ao servidor de banco de dados, e depois atua como um canal para passar (parcialmente) dados processados do servidor de banco de dados aos clientes, onde podem ainda ser processados e filtrados para serem apresentados aos usuários no formato da GUI. Assim, a *interface com o usuário*, as *regras da aplicação* e o *acesso aos dados* atuam como três camadas. A Figura 2.7(b) mostra outra arquitetura usada pelos fornecedores de bancos de dados e de outras aplicações. A camada de apresentação exibe informações ao usuário e permite a entrada de dados. A camada de lógica de negócios cuida das regras e restrições intermediárias antes dos dados serem passados para o usuário ou devolvidos ao SGBD. A camada inferior inclui todos os serviços de gerenciamento de dados. A camada do meio também pode atuar como um servidor Web, que recupera resultados das consultas do servidor de banco de dados e os formata para as páginas Web dinâmicas, que são vistas pelo navegador Web no lado do cliente.

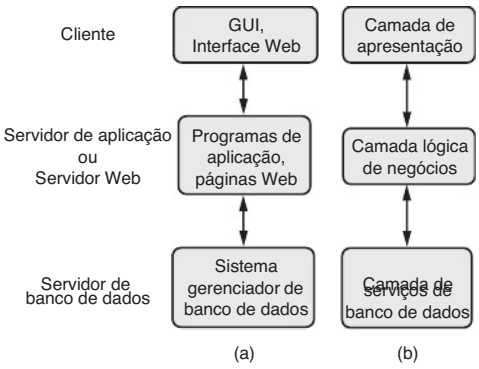


Figura 2.7
Arquitetura cliente/servidor lógica de três camadas, com algumas nomenclaturas comumente utilizadas.

Outras arquiteturas também foram propostas. É possível dividir as camadas entre o usuário e os dados armazenados em outros componentes mais detalhados, resultando, assim, em arquiteturas de *n* camadas, onde *n* pode ser quatro ou cinco camadas. Em geral, a camada da lógica de negócios é dividida em várias camadas. Além de distribuir a programação e os dados pela rede, as aplicações de *n* camadas têm a vantagem de que qualquer camada poder ser executada em um processador ou plataforma de sistema operacional adequado e ser tratada independentemente. Os fornecedores de pacotes de ERP (Enterprise Resource Planning) e CRM (Customer Relationship Management) costumam utilizar uma *camada de middleware*, que é responsável pelos módulos de front-end (clientes) que se comunicam com uma série de bancos de dados de back-end (servidores).

Os avanços na tecnologia de criptografia tornam mais seguro transferir dados confidenciais em formato codificado do servidor ao cliente, onde será decodificado. Isso pode ser feito pelo hardware ou por um software avançado. Essa tecnologia oferece níveis mais altos de segurança de dados, mas as questões de segurança da rede continuam sendo uma preocupação constante. Diversas tecnologias de compactação de dados também ajudam a transferir grande quantidade de dados dos servidores aos clientes por redes com e sem fio.

2.6 Classificação dos sistemas gerenciadores de banco de dados

Vários critérios são normalmente utilizados para classificar os SGBDs. O primeiro é o **modelo de dados** no qual o SGBD é baseado. O principal modelo de dados usado atualmente em muitos SGBDs comercial é o **modelo de dados relacional**. O **modelo de dados de objeto** foi implementado em alguns sistemas comerciais, mas não tem seu uso generalizado. Muitas aplicações legadas ainda rodam em sistemas de banco de dados baseados nos **modelos de dados hierárquico** e de **rede**. Alguns exemplos de SGBDs hierárquicos são o IMS (da IBM) e alguns outros sistemas, como o System 2K (da SAS Inc.) e o TDMS.

O IMS ainda é usado em instalações do governo e industriais, incluindo hospitais e bancos, embora muitos de seus usuários tenham passado para sistemas relacionais. O modelo de dados de rede foi usado por muitos fornecedores, e os produtos resultantes, como IDMs (da Cullinet — atualmente, Computer Associates), DMS 1100 (da Univac — atualmente, Unisys), IMAGE (da Hewlett-Packard), VAX-SGBD (da Digital — depois Compaq e atualmente HP) e SUPRA (da Cincom), ainda têm seguidores e seus grupos de usuários estão ativamente organizados. Se você usa IBM, a pode ou o fudin sistema de arquivos VSAM da IBM, podemos facilmente dizer que uma porcentagem razoável dos dados computadorizados do mundo inteiro ainda estão nesses chamados **sistemas de banco de dados legados**.

Os SGBDs relacionais estão evoluindo continuamente e, em particular, têm incorporado muitos dos conceitos que foram desenvolvidos nos bancos de dados de objeto. Isso tem levado a uma nova classe de SGBDs, chamada **SGBDs objeto-relacionais**. Assim, podemos categorizar os SGBDs com base no modelo de dados: relacionais, objeto, objeto-relacional, hierárquico, rede, entre outros.

Mais recentemente, alguns SGBDs experimentais se baseiam no modelo XML (eXtended Markup Language), que é um modelo de dados estruturado em árvore (hierárquico). Estes têm sido chamados de **SGBDs XML nativos**. Vários SGBDs relacionais comerciais acrescentaram interfaces e armazenamento XML a seus produtos.

O segundo critério usado para classificar SGBDs é o **número de usuários** suportados pelo sistema. **Sistemas monousuário** admitem apenas um usuário de cada vez, e são usados principalmente com PCs. **Sistemas multiusuário**, que incluem a maioria dos SGBDs, admitem múltiplos usuários simultaneamente.

O terceiro critério é o **número de locais** sobre os quais o banco de dados está distribuído. Um SGBD é **centralizado** se os dados estiverem armazenados em um único computador. Um SGBD centralizado pode atender a vários usuários, mas o SGBD e o banco de dados residem integralmente em um único computador. Um SGBD **distribuído** (SGBDD) pode ter o banco de dados real e o software de SGBD distribuídos por vários locais, conectados por uma rede de computadores. Os SGBDDs **homogêneos** usam o mesmo software de SGBD em todos os locais, ao passo que SGBDDs **heterogêneos** podem usar um software de SGBD diferente em cada local. Também é possível desenvolver **software de middleware** para acessar vários bancos de dados autônomos pré-existentes, armazenados sob SGBDs heterogêneos. Isso leva a um SGBD **federado** (ou **sistema multibanco de dados**), em que os sistemas participantes são fracamente acoplados e possuem um certo grau de autonomia local. Muitos SGBDDs utilizam arquitetura cliente/servidor, conforme descrito na Seção 2.5.

O quarto critério é o custo. É difícil propor uma classificação dos SGBDs com base no custo. Hoje, temos SGBDs de código aberto (gratuito), como MySQL e PostgreSQL, que têm suporte de fornecedores terceirizados com serviços adicionais. Os principais SGBDRs estão disponíveis em versões gratuitas para testes durante 30 dias, além de versões pessoais, que podem custar menos de US$ 100 e permitir uma funcionalidade muito maior. Os sistemas gigantes estão sendo vendidos em formato modular, com componentes para lidar com distribuição, replicação, processamento paralelo, capacidade móvel, e assim por diante, e com um grande número de parâmetros que precisam ser definidos para configuração. Além do mais, eles são vendidos na forma de licenças — as licenças por local permitem uso ilimitado do sistema de banco de dados com qualquer número de cópias rodando na instalação definida pelo comprador. Outro tipo de licença limita o número de usuários simultâneos ou o número total de usuários em determinado local. As versões de alguns sistemas para um único usuário isolado, como Microsoft Access, são vendidas por cópia ou incluídas na configuração geral do computador desktop ou laptop. Além disso, recursos de data warehousing e mineração de dados, bem como o suporte para tipos de dados adicionais, estão disponíveis a um custo extra. É possível pagar milhões de dólares anualmente pela instalação e manutenção de grandes sistemas de banco de dados.

Também podemos classificar um SGBD com base nas opções de **tipos de caminho de acesso** para armazenar arquivos. Uma família bem conhecida de SGBDs é baseada em estruturas invertidas de arquivo. Por fim, um SGBD pode ser de **uso geral** ou de **uso especial**. Quando o desempenho é um aspecto importante, um SGBD de uso especial pode ser projetado e construído para uma aplicação específica; esse sistema não pode ser usado para outras aplicações sem mudanças relevantes. Muitos sistemas de reservas aéreas e catálogo telefônico desenvolvidos no passado são SGBDs de uso especial. Eles se encontram na categoria de sistemas de **processamento de transação on-line** (**OLTP** — Online Transaction Processing), que precisam dar suporte a um grande número de transações simultâneas sem causar atrasos excessivos.

Vamos detalhar rapidamente o critério principal para classificar SGBDs: o modelo de dados. O **modelo de dados relacional** básico representa um banco de dados como uma coleção de tabelas, onde cada

tabela pode ser armazenada como um arquivo separado. O banco de dados na Figura 1.2 se assemelha a uma representação relacional. A maior parte dos bancos de dados relacionais utiliza a linguagem de consulta de alto nível, chamada SQL, e admite uma forma limitada de visões do usuário. Discutiremos o modelo relacional e suas linguagens e operações nos capítulos 3 a 6, e as técnicas para programar aplicações relacionais nos capítulos 13 e 14.

O **modelo de dados de objeto** define um banco de dados em termos de objetos, suas propriedades e operações. Os objetos com a mesma estrutura e comportamento pertencem a uma **classe**, e as classes são organizadas em **hierarquias** (ou **grafos acíclicos**). As operações de cada classe são especificadas com procedimentos predefinidos, chamados **métodos**. Os SGBDs relacionais têm estendido seus modelos para incorporar conceitos de banco de dados de objeto e outras funcionalidades; esses sistemas são chamados de **sistemas objeto-relacional** ou **relacional estendido**. Discutiremos os sistemas de bancos de dados de objeto e objeto-relacional no Capítulo 11.

O **modelo XML** surgiu como um padrão para troca de dados pela Web, e foi usado como base para implementar vários protótipos de sistemas com XML nativa. A XML utiliza estruturas de árvore hierárquicas e combina conceitos de banco de dados com conceitos dos modelos de representação de documentos. Os dados são representados como elementos; com o uso de tags, os dados podem ser aninhados para criar estruturas hierárquicas complexas. Esse modelo é conceitualmente semelhante ao modelo de objeto, mas usa uma terminologia diferente. Funcionalidades XML têm sido acrescentadas a muitos produtos de SGBD comercial. Apresentaremos uma visão geral da XML no Capítulo 12.

Dois modelos de dados mais antigos, historicamente importantes, agora conhecidos como **modelos de dados legados**, são os modelos de rede e hierárquico. O **modelo de rede** representa dados como tipos de registro e também representa um tipo limitado de relacionamento 1:N, chamado **tipo de conjunto**. Um relacionamento 1:N, ou um-para-muitos, relaciona uma instância de um registro a muitas instâncias de registros usando algum mecanismo de ligação com ponteiros nesses modelos. A Figura 2.8 mostra um diagrama de esquema de rede para o banco de dados da Figura 2.1, em que os tipos de registros aparecem como retângulos e os tipos de conjuntos aparecem como setas direcionadas e rotuladas.

O modelo de rede, também conhecido como modelo CODASYL DBTG,[14] possui uma linguagem que trata um registro por vez e que deve estar embutida em uma linguagem de programação hospedeira. A DML do modelo de rede foi proposta no Database Task Group (DBTG) Report de 1971 como uma extensão da linguagem COBOL. Ela oferece comandos para localizar registros diretamente (por exemplo, FIND ANY <tipo-registro> USING <lista-de-campos> ou FIND DUPLICATE <tipo-registro> USING <lista-de-campos>). Tem comandos para navegar dentro dos tipos conjunto (por exemplo, GET OWNER, GET {FIRST, NEXT, LAST} MEMBER WITHIN <tipo-conjunto> WHERE <condição>). Tem também comandos para armazenar novos dados (por exemplo, STORE <tipo-registro>) e torná-los parte de um tipo conjunto (por exemplo, CONNECT <tipo-registro> TO <tipo-conjunto>). A linguagem também trata diversos aspectos adicionais, como a posição dos tipos de

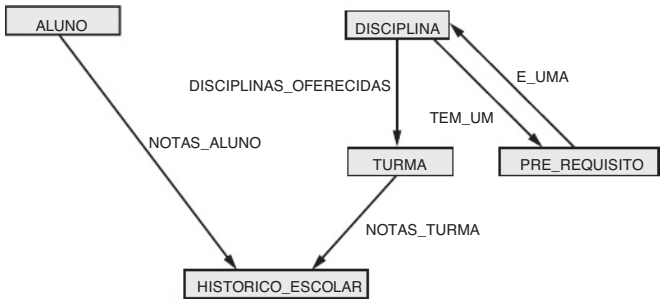


Figura 2.8
O esquema da Figura 2.1 na notação de modelo de rede.

[14] CODASYL DBTG significa Conference on Data Systems Languages Database Task Group, que é o nome do comitê que especificou o modelo de rede e sua linguagem.

registro e dos tipos de conjunto, que são definidos pela posição atual do processo de navegação no banco de dados. Ela é usada predominantemente pelos sistemas IDMS, IMAGE e SUPRA nos dias de hoje.

O **modelo hierárquico** representa os dados como estruturas de árvore hierárquicas. Cada hierarquia simboliza uma série de registros relacionados. Não existe uma linguagem-padrão para o modelo hierárquico. Uma DML hierárquica popular é a DL/1 do sistema IMS. Ela dominou o mercado de SGBD por mais de 20 anos, entre 1965 e 1985, e ainda é um sistema muito usado no mundo inteiro, gerenciando uma grande porcentagem dos dados dos bancos de dados do governo, de serviços de saúde, de bancos e do setor de seguros. Sua DML, chamada DL/1, foi um padrão de fato na indústria por um longo tempo. Ela tem comandos para localizar um registro (por exemplo, GET {UNIQUE, NEXT} <tipo-registro> WHERE <condição>) e facilidades para navegar nas hierarquias (por exemplo, GET NEXT WITHIN PARENT ou GET {FIRST, NEXT} PATH<especificação-caminho-hierárquico> WHERE <condição>). Possui funcionalidades para armazenar e atualizar registros (por exemplo, INSERT <tipo-registro>, REPLACE <tipo-registro>). As questões relacionadas à posição do registro durante a navegação também são tratadas com outros recursos na linguagem.

Resumo

Neste capítulo, apresentamos os principais conceitos usados em sistemas de banco de dados. Definimos um modelo de dados e destacamos três categorias principais:

- Modelos de dados de alto nível ou conceituais (baseados em entidades e relacionamentos).
- Modelos de dados de baixo nível ou físicos.
- Modelos de dados representativos ou de implementação (baseados em registro, orientados a objeto).

Destacamos a separação do esquema, ou descrição de um banco de dados, do próprio banco de dados. O esquema não muda com muita frequência, enquanto o estado do banco de dados muda todas as vezes em que dados são inseridos, excluídos ou modificados. Depois, descrevemos a arquitetura de SGBD, que permite três níveis de esquema:

- Um esquema interno que descreve a estrutura física do armazenamento do banco de dados.
- Um esquema conceitual que é uma descrição de alto nível do banco de dados inteiro.
- Esquemas externos que descrevem as visões de diferentes grupos de usuários.

Um SGBD que separa nitidamente os três níveis precisa ter mapeamentos entre os esquemas para transformar solicitações e resultados das consultas de um nível para o seguinte. A maioria dos SGBDs não separa os três níveis completamente. Usamos a arquitetura de três esquemas para definir os conceitos de independência lógica e física dos dados.

Depois, discutimos os principais tipos de linguagens e interfaces que os SGBDs possuem. Uma linguagem de definição de dados (DDL) é usada para definir o esquema conceitual do banco de dados. Na maioria dos SGBDs, a DDL também define visões do usuário e, às vezes, estruturas de armazenamento; em outros, existem linguagens ou funções separadas para especificar estruturas de armazenamento. Atualmente, essa distinção está desaparecendo nas implementações relacionais, com a SQL servindo como uma linguagem geral para realizar vários papéis, incluindo definição de visões. A parte de definição de armazenamento (SDL) foi incluída nas primeiras versões da SQL, mas agora costuma ser implementada como comandos especiais para o DBA nos SGBDs relacionais. O SGBD compila todas as definições do esquema e armazena suas descrições no catálogo do sistema.

Uma linguagem de manipulação de dados (DML) é usada para especificar leituras e atualizações no banco de dados. As DMLs podem ser de alto nível (orientada a conjunto, não procedural) ou de baixo nível (orientada a registro, procedural). Uma DML de alto nível pode ser embutida em uma linguagem de programação hospedeira, ou pode ser usada como uma linguagem independente; nesse último caso, ela costuma ser chamada de linguagem de consulta.

Discutimos os diferentes tipos de interfaces fornecidas pelos SGBDs e os tipos de usuários com os quais cada interface está associada. Depois, abordamos o ambiente do sistema de banco de dados, módulos de software de SGBD típicos e utilitários de SGBD para ajudar os usuários e os DBAs a realizar suas tarefas. Continuamos com uma visão geral das arquiteturas de duas e três camadas para aplicações de banco de dados, progressivamente passando para *n* camadas, que agora são comuns em muitas aplicações, em especial nas aplicações de banco de dados da Web.

Por fim, classificamos os SGBDs de acordo com vários critérios: modelo de dados, número de usuários, número de locais, tipos de caminhos de acesso e custo. Discutimos a disponibilidade dos SGBDs e módulos adicionais — desde nenhum custo, na forma de software de código aberto, até configurações que custam anualmente milhões de dólares para serem mantidas. Também indicamos um conjunto variável de licença para os SGBDs e produtos relacionados. A principal classificação dos SGBDs é baseada no modelo de dados. Discutimos rapidamente os principais modelos de dados usados nos SGBDs disponíveis atualmente no mercado.

Perguntas de revisão

2.1. Defina os seguintes termos: *modelo de dados*, *esquema de banco de dados*, *estado de banco de dados*, *esquema interno*, *esquema conceitual*, *esquema externo*, *independência de dados*, *DDL*, *DML*, *SDL*, *VDL*, *linguagem de consulta*, *linguagem hospedeira*, *sublinguagem de dados*, *utilitário de banco de dados*, *catálogo*, *arquitetura cliente/servidor*, *arquitetura de três camadas* e *arquitetura de n camadas*.

2.2. Discuta as principais categorias de modelos de dados. Quais são as diferenças básicas entre o modelo relacional, o de objeto e a XML?

2.3. Qual é a diferença entre um esquema de banco de dados e um estado de banco de dados?

2.4. Descreva a arquitetura de três camadas. Por que precisamos de mapeamentos entre os níveis de esquema? Como diferentes linguagens de definição de esquema dão suporte a essa arquitetura?

2.5. Qual é a diferença entre a independência lógica e a independência física dos dados? Qual é a mais difícil de se alcançar? Por quê?

2.6. Qual é a diferença entre DMLs procedurais e não procedurais?

2.7. Discuta os diferentes tipos de interfaces de fácil utilização e os tipos de usuários que normalmente utilizam cada tipo.

2.8. Com que outro software um SGBD interage?

2.9. Qual é a diferença entre as arquiteturas cliente/servidor de duas e três camadas?

2.10. Discuta alguns tipos de utilitários e ferramentas de banco de dados e suas funções.

2.11. Qual é a funcionalidade adicional na arquitetura de n camadas ($n > 3$)?

Exercícios

2.12. Pense nos diferentes usuários para o banco de dados mostrado na Figura 1.2. De que tipos de aplicações cada usuário precisaria? A que categoria de usuário cada um pertenceria e de que tipo de interface cada um precisaria?

2.13. Escolha uma aplicação de banco de dados com a qual você esteja acostumado. Crie um esquema e mostre um exemplo de banco de dados para essa aplicação, usando a notação das figuras 1.2 e 2.1. Que tipos de informações e restrições adicionais você gostaria de representar no esquema? Pense nos diversos usuários de seu banco de dados e crie uma visão para cada tipo.

2.14. Se você estivesse criando um sistema baseado na Web para fazer reservas e vender passagens aéreas, qual arquitetura de SGBD você escolheria, com base na Seção 2.5? Por quê? Por que as outras arquiteturas não seriam uma boa escolha?

2.15. Considere a Figura 2.1. Além das restrições relacionando os valores das colunas de uma tabela às colunas de outra tabela, também existem restrições que impõem limitações sobre valores de uma coluna ou uma combinação de colunas de uma tabela. Uma restrição desse tipo impõe que uma coluna ou um grupo de colunas deva ser exclusivo em todas as linhas na tabela. Por exemplo, na tabela ALUNO, a coluna numero_aluno deve ser exclusiva (para impedir que dois alunos diferentes tenham o mesmo numero_aluno). Identifique a coluna ou o grupo de colunas das outras tabelas que precisam ser exclusivos em todas as linhas na tabela.

Bibliografia selecionada

Muitos livros-texto de banco de dados, incluindo Date (2004), Silberschatz et al. (2006), Ramakrishnan e Gehrke (2003), Garcia-Molina et al. (2000, 2009) e Abiteboul et al. (1995), oferecem uma discussão sobre os fundamentos teóricos de bancos de dados apresentados aqui. Tsichritzis e Lochovsky (1982) é um livro-texto mais antigo sobre modelos de dados. Tsichritzis e Klug (1978) e Jardine (1977) apresentam a arquitetura de três esquemas, que foi sugerida inicialmente no relatório CODASYL DBTG de 1971 e, mais tarde, em um relatório do American National Standards Institute (ANSI, instituto norte-americano de padrões) de 1975. Uma análise profunda do modelo de dados relacional e algumas de suas possíveis extensões são apresentadas em Codd (1990). O padrão proposto para bancos de dados orientados a objeto é descrito em Cattell et al. (2000). Muitos documentos descrevendo XML estão disponíveis na Web, como XML (2005).

Alguns exemplos de utilitários de banco de dados são as ferramentas Connect, Analyze e Transform, da ETI (<http://www.eti.com>), e a ferramenta de administração de banco de dados, DBArtisan, da Embarcadero Technologies (<http://www.embarcadero.com>).

parte 2

Modelo de dados relacional e SQL

capítulo 3

O modelo de dados relacional e as restrições em bancos de dados relacionais

Este capítulo abre a Parte 2 do livro, que aborda os bancos de dados relacionais. O modelo de dados relacional foi introduzido inicialmente por Ted Codd, da IBM Research, em 1970, em um artigo clássico (Codd, 1970), que atraiu atenção imediata devido a sua simplicidade e base matemática. O modelo usa o conceito de *relação matemática* — que se parece com uma tabela de valores — como seu bloco de montagem básico, e sua base teórica reside em uma teoria de conjunto e lógica de predicado de primeira ordem. Neste capítulo, discutiremos as características básicas do modelo e suas restrições.

As primeiras implementações comerciais do modelo relacional se tornaram disponíveis no início da década de 1980, como o sistema SQL/DS no sistema operacional MVS, da IBM, e o SGBD, da Oracle. Desde então, o modelo foi implantado em uma grande quantidade de sistemas comerciais. Os SGBDs relacionais (SGBDRs) populares atuais incluem o DB2 e Informix Dynamic Server (da IBM), o Oracle e Rdb (da Oracle), o Sybase SGBD (da Sybase) e o SQLServer e Access (da Microsoft). Além disso, vários sistemas de código aberto, como MySQL e PostgreSQL, estão disponíveis.

Por causa da importância do modelo relacional, toda a Parte 2 é dedicada a esse modelo e algumas das linguagens associadas a ele. Nos capítulos 4 e 5, descreveremos a linguagem de consulta SQL, que é o *padrão* para SGBDs relacionais comerciais. O Capítulo 6 abordará as operações da álgebra relacional e introduzirá o cálculo relacional — essas são duas linguagens formais associadas ao modelo relacional. O cálculo relacional é considerado a base para a linguagem SQL, e a álgebra relacional é usada nos detalhes internos de muitas implementações de banco de dados para processamento e otimização de consulta (ver Parte 8 do livro).

Outros aspectos do modelo relacional são apresentados em outras partes do livro. O Capítulo 9 abordará as estruturas de dados do modelo relacional para construções dos modelos ER e EER (apresentados nos capítulos 7 e 8), e apresentará algoritmos para projetar um esquema de banco de dados relacional mapeando um esquema conceitual no modelo ER ou EER para uma representação relacional. Esses mapeamentos são incorporados em muitas ferramentas de projeto de banco de dados e CASE.[1] Os capítulos 13 e 14, na Parte 5, discutirão as técnicas de programação usadas para acessar sistemas de banco de dados e a noção de conexão com bancos de dados relacionais por meio dos protocolos-padrão ODBC e JDBC. O Capítulo 14 também apresentará o tópico de programação de banco de dados na Web. Os capítulos 15 e 16, na Parte 6, apresentarão outro aspecto do modelo relacional, a saber, as restrições formais das dependências funcionais e multivaloradas. Essas dependências são usadas para desenvolver uma teoria de projeto de banco de dados relacional baseada no conceito conhecido como *normalização*.

Os modelos de dados que precederam o relacional incluem os modelos hierárquico e de rede. Eles foram propostos na década de 1960 e implementados nos primeiros SGBDs durante o final da década

[1] CASE significa Computer-Aided Software Engineering (engenharia de software auxiliada por computador).

de 1960 e início da década de 1970. Por causa de sua importância histórica e da existência da base de usuários para esses SGBDs, incluímos um resumo dos destaques desses modelos nos apêndices D e E, que estão disponíveis no site de apoio do livro (em inglês). Esses modelos e sistemas agora são conhecidos como *sistemas de banco de dados legados*.

Neste capítulo, concentramo-nos em descrever os princípios básicos do modelo de dados relacional. Começamos definindo os conceitos de modelagem e a notação do modelo relacional na Seção 3.1. A Seção 3.2 é dedicada a uma discussão das restrições relacionais que são consideradas uma parte importante do modelo relacional e automaticamente impostas na maioria dos SGBDs relacionais. A Seção 3.3 define as operações de atualização do modelo relacional, discute como as violações de restrições de integridade são tratadas e apresenta o conceito de uma transação. No final há um resumo do capítulo.

3.1 Conceitos do modelo relacional

O modelo relacional representa o banco de dados como uma coleção de *relações*. Informalmente, cada relação é semelhante a uma tabela de valores ou, até certo ponto, a um arquivo *plano* de registros. Ele é chamado de **arquivo plano** porque cada registro tem uma simples estrutura linear ou *plana*. Por exemplo, o banco de dados de arquivo mostrado na Figura 1.2 é semelhante à representação do modelo relacional básico. No entanto, existem diferenças importantes entre relações e arquivos, conforme veremos em breve.

Quando uma relação é considerada uma **tabela** de valores, cada linha na tabela representa uma coleção de valores de dados relacionados. Uma linha representa um fato que normalmente corresponde a uma entidade ou relacionamento do mundo real. Os nomes da tabela e de coluna são usados para ajudar a interpretar o significado dos valores em cada linha. Por exemplo, a primeira tabela da Figura 1.2 é chamada de ALUNO porque cada linha representa fatos sobre uma entidade particular de aluno. Os nomes de coluna Nome, Numero_aluno, Tipo_aluno e Curso especificam como interpretar os valores de dados em cada linha, com base na coluna em que cada valor se encontra. Todos os valores em uma coluna são do mesmo tipo de dado.

Na terminologia formal do modelo relacional, uma linha é chamada de *tupla*, um cabeçalho da coluna é chamado de *atributo* e a tabela é chamada de *relação*. O tipo de dado que descreve os tipos de valores que podem aparecer em cada coluna é representado por um *domínio* de valores possíveis. Agora, vamos definir esses termos — *domínio*, *tupla*, *atributo* e *relação* — de maneira formal.

3.1.1 Domínios, atributos, tuplas e relações

Um **domínio** *D* é um conjunto de valores atômicos. Com **atômico**, queremos dizer que cada valor no domínio é indivisível em se tratando do modelo relacional formal. Um método comum de especificação um domínio é definir um tipo de dado do qual são retirados os valores de dados que formam o domínio. Também é útil especificar um nome para o domínio, para ajudar na interpretação de seus valores. Alguns exemplos de domínios são:

- Numeros_telefone_nacional. O conjunto de números de telefone com dez dígitos válidos no Brasil.
- Numeros_telefone_local. O conjunto de números de telefone de oito dígitos válidos dentro de um código de área em particular no Brasil. O uso de números de telefone locais está rapidamente se tornando obsoleto, sendo substituído por números padrão de dez dígitos.
- Cadastro_pessoa_fisica. O conjunto de números do CPF com onze dígitos. (Esse é um identificador exclusivo atribuído a cada pessoa no Brasil para fins de emprego, imposto e benefícios.)
- Nomes. O conjunto de cadeia de caracteres que representam nomes de pessoas.
- Medias_nota. Possíveis valores para calcular a média das notas; cada um deve ser um número real (ponto flutuante) entre 0 e 4.
- Idades_funcionario. Idades possíveis dos funcionários em uma empresa; cada um deve ser um valor inteiro entre 15 e 80.
- Nomes_departamento_academico. O conjunto de nomes de departamentos acadêmicos em uma universidade, como Ciência da Computação, Economia e Física.
- Codigos_departamento_academico. O conjunto de códigos de departamentos acadêmicos, como 'CC', 'ECON' e 'FIS'.

Estas são chamadas definições *lógicas* de domínios. Um **tipo de dado** ou **formato** também é especificado para cada domínio. Por exemplo, o tipo de dado para o domínio Numeros_telefone_nacional pode ser declarado como uma sequência de caracteres na forma (*dd*)*dddd*-*dddd*, onde cada *d* é um dígito numérico (decimal) e os dois primeiros dígitos formam um código de área de telefone válido. O tipo de dado para Idades_funcionario é um número inteiro entre 15 e 80. Para Nomes_departamento_academico, o tipo de dado é o conjunto de todas as cadeias de

caracteres que representam nomes de departamento válidos. Um domínio, portanto, recebe um nome, tipo de dado e formato. Informações adicionais para interpretar os valores de um domínio também podem ser dadas; por exemplo, um domínio numérico como Pesos_pessoa deveria ter as unidades de medida, como gramas ou quilos.

Um **esquema**[2] **relacional** R, indicado por $R(A_1, A_2, ..., A_n)$, é composto de um nome de relação R e uma lista de atributos, $A_1, A_2, ..., A_n$. Cada **atributo** A_i é o nome de um papel desempenhado por algum domínio D no esquema de relação R. D é chamado de **domínio** de A_i, e indicado por $\mathbf{dom}(A_i)$. Um esquema de relação é usado para *descrever* uma relação; R é chamado de **nome** dessa relação. O **grau** (ou **aridade**) de uma relação é o número de atributos n desse esquema de relação.

Uma relação de grau sete, que armazena informações sobre alunos universitários, teria sete atributos descrevendo cada aluno, da seguinte forma:

ALUNO(Nome, Cpf, Telefone_residencial, Endereco, Telefone_comercial, Idade, Media)

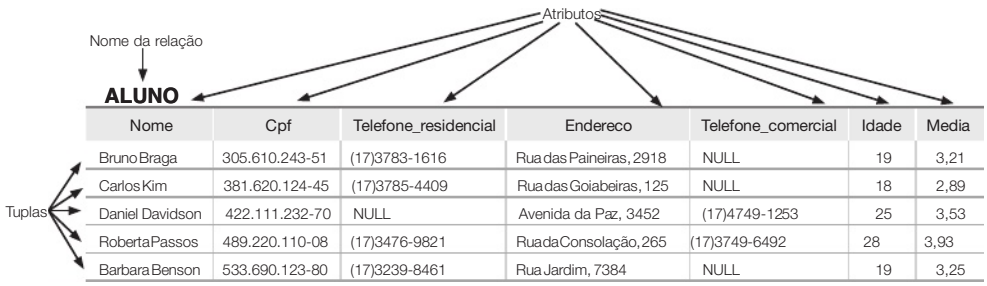
Usando o tipo de dado de cada atributo, a definição às vezes é escrita como:

ALUNO(Nome: string, Cpf: string, Telefone_residencial: string, Endereco: string, Telefone_comercial: string, Idade: integer, Media: real)

Para este esquema de relação, ALUNO é o nome da relação, que tem sete atributos. Na definição anterior, mostramos a atribuição de tipos genéricos, como string ou inteiro, aos atributos. Mais precisamente, podemos especificar os seguintes domínios já definidos para alguns dos atributos da relação ALUNO: dom(Nome) = Nomes; dom(Cpf) = Cadastro_pessoa_fisica; dom(Telefone_residencial) = Numeros_telefone_nacional,[3] dom(Endereco), dom(Telefone_comercial) = Numeros_telefone_nacional e dom(Media) = Media. Também é possível referenciar atributos de um esquema de relação por sua posição dentro da relação; assim, o segundo atributo da relação ALUNO é Cpf, enquanto o quarto atributo é Endereco.

Uma **relação** (ou **estado de relação**)[4] r do esquema de relação $R(A_1, A_2, ..., A_n)$, também indicada por $r(R)$, é um conjunto de n tuplas $r = \{t_1, t_2, ..., t_m\}$. Cada n **tuplas** t é uma lista ordenada de n valores $t = <v_1, v_2, ..., v_n>$, em que cada valor v_i, $1 \leq i \leq n$, é um elemento de $\mathrm{dom}(A_i)$ ou é um valor especial NULL. (Valores NULL serão discutidos mais adiante, na Seção 3.1.2.) O valor i-ésimo na tupla t, que corresponde ao atributo A_i, é referenciado como $t[A_i]$ ou $t.A_i$ (ou $t[i]$ se usarmos a notação posicional). Os termos **intenção da relação** para o esquema R e **extensão da relação** para o estado de relação $r(R)$ também são comumente utilizados.

A Figura 3.1 mostra um exemplo de uma relação ALUNO, que corresponde ao esquema ALUNO já espe-



Nome	Cpf	Telefone_residencial	Endereco	Telefone_comercial	Idade	Media
Bruno Braga	305.610.243-51	(17)3783-1616	Rua das Paineiras, 2918	NULL	19	3,21
Carlos Kim	381.620.124-45	(17)3785-4409	Rua das Goiabeiras, 125	NULL	18	2,89
Daniel Davidson	422.111.232-70	NULL	Avenida da Paz, 3452	(17)4749-1253	25	3,53
Roberta Passos	489.220.110-08	(17)3476-9821	Rua da Consolação, 265	(17)3749-6492	28	3,93
Barbara Benson	533.690.123-80	(17)3239-8461	Rua Jardim, 7384	NULL	19	3,25

Figura 3.1
Os atributos e tuplas de uma relação ALUNO.

[2] Um esquema de relação às vezes é chamado de **scheme de relação**

[3] Nos EUA, com o grande aumento nos números de telefone causado pela proliferação dos telefones móveis, a maioria das áreas metropolitanas agora possui vários códigos de área, de modo que a discagem local com sete dígitos foi descontinuada na maioria das áreas. Mudamos esse domínio para Numeros_telefone_nacional em vez de Numeros_telefones_local, que seria uma opção mais geral. Isso ilustra como os requisitos do banco de dados podem mudar com o tempo.

[4] Isso também tem sido chamado de **instância da relação** Não usaremos esse termo porque *instância* também é usado para se referir a uma única tupla ou linha.

cificado. Cada tupla na relação representa uma entidade de aluno em particular (ou objeto). Apresentamos a relação como uma tabela, onde cada tupla aparece como uma *linha* e cada atributo corresponde aum *cabeçalho de coluna*, indicando um papel ou interpretação dos valores nesta coluna. *Valores NULL* representam atributos cujos valores são desconhecidos ou não existem para alguma tupla individual de ALUNO.

A definição anterior de uma relação pode ser *refeita* de maneira mais formal usando os conceitos da teoria de conjunto, como se segue. Uma relação (ou estado de relação) $r(R)$ é uma **relação matemática** de grau n sobre os domínios $\text{dom}(A_1)$, $\text{dom}(A_2)$, ..., $\text{dom}(A_n)$, que é um **subconjunto** do **produto cartesiano** (indicado por $\times$) dos domínios que definem R:

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times ... \times \text{dom}(A_n))$$

O produto cartesiano especifica todas as combinações possíveis de valores dos domínios subjacentes. Logo, se indicarmos o número total de valores, ou **cardinalidade**, em um domínio D como $|D|$ (considerando que todos os domínios são finitos), o número total de tuplas no produto cartesiano é

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times ... \times |\text{dom}(A_n)|$$

Esse produto de cardinalidades de todos os domínios representa o número total de possíveis instâncias ou tuplas que poderão existir em qualquer estado de relação $r(R)$. De todas as combinações possíveis, um estado de relação em determinado momento — o **estado de relação atual** — reflete apenas as tuplas válidas que representam um estado em particular do mundo real. Em geral, à medida que o estado do mundo real muda, também muda o estado de relação, sendo transformado em outro estado de relação. Contudo, o esquema R é relativamente estático e muda com *muito pouca* frequência — por exemplo, como resultado da inclusão de um atributo para representar novas informações que não estavam armazenadas srcinalmente na relação.

É possível que vários atributos *tenham o mesmo domínio*. Os nomes de atributo indicam diferentes **papéis**, ou interpretações, do domínio. Por exemplo, na relação ALUNO, o mesmo domínio Numeros_telefone_nacional desempenha o papel de Telefone_residencial, referindo-se ao *telefone residencial de um aluno*, e o papel de Telefone_comercial, referindo-se ao *telefone comercial do aluno*. Um terceiro atributo possível (não mostrado) com o mesmo domínio poderia ser Telefone_celular.

3.1.2 Características das relações

A definição dada de relações implica certas características que tornam uma relação diferente de um arquivo ou uma tabela. Agora, discutiremos algumas dessas características.

Ordenação de tuplas em uma relação. Uma relação é definida como um *conjunto* de tuplas. Matematicamente, os elementos de um conjunto *não possuem ordem* entre eles; logo, as tuplas em uma relação não possuem nenhuma ordem em particular. Em outras palavras, uma relação não é sensível à ordenação das tuplas. Porém, em um arquivo, os registros estão fisicamente armazenados no disco (ou na memória), de modo que sempre existe uma ordem entre eles. Essa ordenação indica o primeiro, segundo, *i-nésimo* e último registros no arquivo. De modo semelhante, quando exibimos uma relação como uma tabela, as linhas são exibidas em certa ordem.

A ordenação da tupla não faz parte da definição da relação porque uma relação tenta representar fatos em um nível lógico ou abstrato. Muitas ordens de tupla podem ser especificadas na mesma relação. Por exemplo, as tuplas na relação ALUNO da Figura 3.1 poderiam ser ordenadas pelos valores de Nome, Cpf, Idade ou algum outro atributo. A definição de uma relação não especifica ordem alguma: *não existe preferência* para ordenação de qualquer outra relação. Logo, a relação apresentada na Figura 3.2 é conside-

ALUNO

Nome	Cpf	Telefone_residencial	Endereco	Telefone_comercial	Idade	Media
Daniel Davidson	422.111.232-70	NULL	Avenida da Paz, 3452	(17)4749-1253	25	3,53
Barbara Benson	533.690.123-80	(17)3239-8461	Rua Jardim, 7384	NULL	19	3,25
Roberta Passos	489.220.110-08	(17)3476-9821	Rua da Consolação, 265	(17)3749-6492	28	3,93
Carlos Kim	381.620.124-45	(17)3785-4409	Rua das Goiabeiras, 125	NULL	18	2,89
Bruno Braga	305.610.243-51	(17)3783-1616	Rua das Paineiras, 2918	NULL	19	3,21

Figura 3.2
A relação ALUNO da Figura 3.1 com uma ordem de tuplas diferente.

rada *idêntica* àquela mostrada na Figura 3.1. Quando uma relação é implementada como um arquivo ou exibida como uma tabela, uma ordenação em particular pode ser especificada sobre osregistros do arquivo ou das linhas da tabela.

Ordem dos valores dentro de uma tupla e uma definição alternativa de uma relação. De acordo com a definição anterior de uma relação, uma tupla *n* é uma *lista ordenada* de *n* valores, de modo que a ordem dos valores em uma tupla — e, portanto, dos atributos em um esquema de relação — é importante. No entanto, em um nível mais abstrato, a ordem dos atributos e seus valores *não* é tão importante, desde que a correspondência entre atributos e valores seja mantida.

Uma **definição alternativa** de uma relação pode ser dada, tornando a ordem dos valores em uma tupla *desnecessária*. Nessa definição, um esquema de relação $R = \{A_1, A_2, ..., A_n\}$ é um *conjunto* de atributos (em vez de uma lista), e um estado de relação $r(R)$ é um conjunto finito de mapeamentos $r = \{t_1, t_2, ..., t_m\}$, onde cada tupla t_i é um **mapeamento** de R para D, e D é a **união** (indicada por ∪) dos domínios de atributo; ou seja, $D = \text{dom}(A_1) \cup \text{dom}(A_2) \cup ... \cup \text{dom}(A_n)$. Nessa definição, $t[A_i]$ deve estar em $\text{dom}(A_i)$ para $1 \leq i \leq n$ para cada mapeamento t em r. Cada mapeamento t_i é chamado de tupla.

De acordo com essa definição de tupla como um mapeamento, uma **tupla** pode ser considerada um **conjunto** de pares (<atributo>, <valor>), em que cada par dá o valor do mapeamento a partir de um atributo A_i para um valor v_i de $\text{dom}(A_i)$. A ordem dos atributos *não* é importante, pois o *nome do atributo* aparece com seu *valor*. Por essa definição, as duas tuplas mostradas na Figura 3.3 são idênticas.

t = <(Nome, Daniel Davidson),(Cpf,422.111.232-70),(Telefone_residencial, NULL),(Endereco, Avenida da Paz, 3452), (Telefone_comercial, (17)4749-1253),(Idade, 25),(Media, 3,53)>

t = <(Endereco, Avenida da Paz, 3452),(Nome, Daniel Davidson),(Cpf, 422.111.232-70),(Idade, 25), (Telefone_comercial,(17)4749-1253),(Media, 3,53),(Telefone_residencial,NULL)>

Figura 3.3
Duas tuplas idênticas quando a ordem dos atributos e valores não faz parte da definição da relação.

Isso faz sentido em um nível abstrato, já que não há realmente motivo para preferir ter um valor de atributo aparecendo antes de outro em uma tupla.

Quando uma relação é implementada como um arquivo, os atributos são fisicamente ordenados como campos dentro de um registro. Geralmente, usaremos a **primeira definição** da relação, onde os atributos e os valores dentro das tuplas *são ordenados*, porque isso simplifica grande parte da notação. Porém, a definição alternativa dada aqui é mais geral[5].

Valores e NULLs nas tuplas. Cada valor em uma tupla é um valor **atômico**; ou seja, ele não é divisível em componentes dentro da estrutura do modelo relacional básica. Logo, atributos compostos ou multivalorados (ver Capítulo 7) não são permitidos. Esse modelo às vezes é chamado de **modelo relacional plano**. Grande parte da teoria por trás do modelo relacional foi desenvolvida com essa suposição em mente, que é chamada pressuposto da **primeira forma normal**.[6] Assim, atributos multivalorados precisam ser representados por relações separadas, e os atributos compostos são representados apenas por seus atributos de componentes simples no modelo relacional básico.[7]

Um conceito importante é o dos valores NULL, que são usados para representar os valores de atributos que podem ser desconhecidos ou não se aplicam a uma tupla. Um valor especial, chamado NULL, é usado nesses casos. Por exemplo, na Figura 3.1, algumas tuplas ALUNO têm NULL para seus telefones comerciais, pois eles não trabalham (ou seja, o telefone comercial *não se aplica* a esses alunos). Outro aluno tem um NULL para o telefone residencial, talvez porque ele não tenha um telefone residencial ou ele tem, mas nós não o conhecemos (o valor é *desconhecido*). Em geral, podemos ter vários significados para valores NULL, como *valor desconhecido*, *valor existe mas não está disponível* ou *atributo não se aplica* a esta tupla (também conhecido como *valor indefinido*). Um exemplo do último tipo de NULL ocorrerá se acrescentarmos um atributo Tipo_visto (tipo do visto) à relação ALUNO, que se aplica apenas a tuplas que representam alunos estrangeiros. É possível criar diferentes códigos para diversos significados de valores NULL. A incorporação de diferentes tipos de valores NULL nas operações do modelo relacional (ver Capítulo 6) provou ser muito difícil e, portanto, está fora do escopo de nossa apresentação.

[5] Como veremos, a definição alternativa da relação será útil quando discutirmos o processamento e a otimização da consulta no Capítulo 19.

[6] Discutiremos esse pressuposto da primeira forma normal com mais detalhes no Capítulo 15.

[7] Extensões do modelo relacional removem essas restrições. Por exemplo, os sistemas objeto-relacional (Capítulo 11) permitem atributos estruturados complexos, assim como os modelos relacionais **não de primeira forma normal** ou **aninhados**

O significado exato de um valor NULL determina como ele será aplicado durante agregações aritméticas ou comparações com outros valores. Por exemplo, uma comparação de dois valores NULL leva a ambiguidades — se os Clientes A e B têm endereços NULL, isso *não significa* que eles têm o mesmo endereço. Durante o projeto do banco de dados, é melhor evitar ao máximo valores NULL. Discutiremos isso melhor nos capítulos 5 e 6, no contexto de operações e consultas, e no Capítulo 15, no contexto do projeto e normalização de banco de dados.

Interpretação (significado) de uma relação. O esquema de relação pode ser interpretado como uma declaração ou um tipo de **afirmação** (ou **asserção**). Por exemplo, o esquema da relação ALUNO da Figura 3.1 afirma que, em geral, uma entidade de aluno tem um Nome, Cpf, Telefone_residencial, Endereco, Telefone_comercial, Idade e Media. Cada tupla na relação pode então ser interpretada como um **fato** ou uma instância em particular da afirmação. Por exemplo, a primeira tupla na Figura 3.1 afirma que existe um ALUNO cujo Nome é Bruno Braga, o Cpf é 305.610.243-51, a Idade é 19, e assim por diante.

Observe que algumas relações podem representar fatos sobre *entidades*, enquanto outras podem representar fatos sobre *relacionamentos*. Por exemplo, um esquema de relação CURSAR (Cpf_aluno, Codigo_disciplina) afirma que os alunos cursaram disciplinas acadêmicas. Uma tupla nessa relação relaciona um aluno a disciplina cursada. Logo, o modelo relacional representa fatos sobre entidades e relacionamentos *uniformemente* como relações. Isso às vezes compromete a compreensão, pois é preciso descobrir se uma relação representa um tipo de entidade ou um tipo de relacionamento. Apresentaremos o modelo Entidade-Relacionamento (ER) com detalhes no Capítulo 7, no qual os conceitos de entidade e relacionamento serão descritos minuciosamente. Os procedimentos de mapeamento no Capítulo 9 mostram como diferentes construções dos modelos de dados conceituais ER e EER (modelo ER estendido, abordado no Capítulo 8, na Parte 3) são convertidas em relações.

Uma interpretação alternativa de um esquema de relação é como um **predicado**; nesse caso, os valores em cada tupla são interpretados como valores que *satisfazem* o predicado. Por exemplo, o predicado ALUNO (Nome, Cpf, ...) é verdadeiro para as cinco tuplas na relação ALUNO da Figura 3.1. Essas tuplas representam cinco proposições ou fatos diferentes no mundo real. Essa interpretação é muito útil no contexto das linguagens de programação lógicas, como Prolog, pois permite que o modelo relacional seja usado nessas linguagens (ver Seção 26.5). Um pressuposto, chamado **pressuposto do mundo fechado**, afirma que os únicos fatos verdadeiros no universo são aqueles presentes dentro da extensão (estado) da(s) relação(ões). Qualquer outra combinação de valores torna o predicado falso.

3.1.3 Notação do modelo relacional

Usaremos a seguinte notação em nossa representação:

- Um esquema de relação R de grau n é indicado por $R(A_1, A_2, ..., A_n)$.
- As letras maiúsculas Q, R, S indicam nomes de relação.
- As letras minúsculas q, r, s indicam estados de relação.
- As letras t, u, v indicam tuplas.
- Em geral, o nome de um esquema de relação, como ALUNO, também indica o conjunto atual de tuplas nessa relação — o *estado de relação atual* —, enquanto ALUNO(Nome, Cpf, ...) refere-se *apenas* ao esquema de relação.
- Um atributo A pode ser qualificado com o nome de relação R ao qual pertence usando a notação de ponto $R.A$ — por exemplo, ALUNO.Nome ou ALUNO.Idade. Isso porque o mesmo nome pode ser usado para dois atributos em relações diferentes. Contudo, todos os nomes de atributo *em uma relação em particular* precisam ser distintos.
- Uma n-tupla t em uma relação $r(R)$ é indicada por $t = <v_1, v_2, ..., v_n>$, onde v_i é o valor correspondente ao atributo A_i. A notação a seguir refere-se a **valores componentes** de tuplas:
- Tanto $t[A_i]$ quanto $t.A_i$ (e às vezes $t[i]$) referem-se ao valor v_i em t para o atributo A_i.
- Tanto $t[A_u, A_w, ..., A_z]$ quanto $t.(A_u, A_w, ..., A_z)$, onde $A_u, A_w, ..., A_z$ é uma lista de atributos de R, que referem-se à subtupla de valores $<v_u, v_w, ..., v_z>$ de t correspondentes aos atributos especificados na lista.

Como um exemplo, considere a tupla t = <'Barbara Benson', '533.690.123-80', '(17)3239-8461', 'Rua Jardim, 7384', NULL, 19, 3,25> da relação ALUNO na Figura 3.1; temos t[Nome] = <'Barbara Benson'> e t[Cpf, Media, Idade] = <'533.690.123-80', 3,25, 19>.

3.2 Restrições em modelo relacional e esquemas de bancos de dados relacionais

Até aqui, discutimos as características de relações isoladas. No banco de dados relacional, normalmente haverá muitas relações, e as tuplas nessas relações costumam estar relacionadas de várias maneiras. O estado do banco de dados inteiro corresponderá aos estados de todas as suas relações em determinado ponto no tempo. Em geral, existem muitas **restrições** (ou *constraints*) sobre os valores reais em um estado do banco de dados. Essas restrições são derivadas das regras no minimundo que o banco de dados representa, conforme discutimos na Seção 1.6.8.

Nesta seção, discutiremos as diversas restrições sobre os dados que podem ser especificadas em um banco de dados relacional na forma de restrições. As restrições nos bancos de dados geralmente podem ser divididas em três categorias principais:

1. Restrições que são inerentes no modelo de dados. Chamamos estas de **restrições inerentes baseadas no modelo** ou **restrições implícitas**
2. Restrições que podem ser expressas diretamente nos esquemas do modelo de dados, em geral especificando-as na DDL (linguagem de definição de dados; ver Seção 2.3.1). Chamamos estas de **restrições baseadas em esquema** ou **restrições explícitas**.
3. Restrições que *não podem* ser expressas diretamente nos esquemas do modelo de dados, e, portanto, devem ser expressas e impostas pelos programas de aplicação. Chamamos estas de **restrições baseadas na aplicação** ou **restrições semânticas** ou **regras de negócios**.

As características das relações que discutimos na Seção 3.1.2 são as restrições inerentes do modelo relacional e pertencem à primeira categoria. Por exemplo, a restrição de que uma relação não pode ter tuplas duplicadas é uma restrição inerente. As restrições que discutimos nesta seção são da segunda categoria, a saber, restrições que podem ser expressas no esquema do modelo relacional por meio da DDL. As restrições da terceira categoria são mais gerais, relacionam-se ao significado e também ao comportamento dos atributos, e são difíceis de expressar e impor dentro do modelo de dados, de modo que normalmente são verificadas nos programas de aplicação que realizam as atualizações no banco de dados.

Outra categoria importante de restrições é a de *dependências de dados*, que incluem *dependências funcionais* e *dependências multivaloradas*. Elas são usadas principalmente para testar a 'virtude' do projeto de um banco de dados relacional e em um processo chamado *normalização*, que será discutido nos capítulos 15 e 16.

As restrições baseadas em esquema incluem restrições de domínio, restrições de chave, restrições sobre NULLs, restrições de integridade de entidade e restrições de integridade referencial.

3.2.1 Restrições de domínio

As restrições de domínio especificam que, dentro de cada tupla, o valor de cada atributo A deve ser um valor indivisível do domínio dom(A). Já discutimos as maneiras como os domínios podem ser especificados na Seção 3.1.1. Os tipos de dados associados aos domínios normalmente incluem os tipos de dados numéricos padrão para inteiros (como short integer, integer e long integer) e números reais (float e double). Caracteres, booleanos, cadeia de caracteres de tamanho fixo e cadeia de caracteres de tamanho variável também estão disponíveis, assim como data, hora, marcador de tempo, moeda ou outros tipos de dados especiais. Outros domínios possíveis podem ser descritos por um subintervalo dos valores de um tipo de dados ou como um tipo de dado enumerado, em que todos os valores possíveis são explicitamente listados. Em vez de descrevê-los com detalhes aqui, discutiremos os tipos de dados oferecidos pelo padrão relacional SQL na Seção 4.1.

3.2.2 Restrições de chave e restrições sobre valores NULL

No modelo relacional formal, uma *relação* é definida como um *conjunto de tuplas*. Por definição, todos os elementos de um conjunto são distintos; logo, todas as tuplas em uma relação também precisam ser distintas. Isso significa que duas tuplas não podem ter a mesma combinação de valores para *todos* os seus atributos. Normalmente, existem outros **subconjuntos de atributos** de um esquema de relação R com a propriedade de que duas tuplas em qualquer estado de relação r de R não deverão ter a mesma combinação de valores para esses atributos. Suponha que indiquemos um subconjunto de atributos desse tipo como SCh; então, para duas tuplas *distintas* quaisquer t_1 e t_2 em um estado de relação r de R, temos a restrição de que:

$$t_1[\text{SCh}] \neq t_2[\text{SCh}]$$

Qualquer conjunto de atributos SCh desse tipo é chamado de **superchave** do esquema de relação R. Uma superchave SCh especifica uma *restrição de exclusividade* de que duas tuplas distintas em qualquer estado r de R não podem ter o mesmo valor de SCh. Cada relação tem pelo menos uma superchave padrão — o

conjunto de todos os seus atributos. Contudo, uma superchave pode ter atributos redundantes, de modo que um conceito mais útil é o de uma *chave*, que não tem redundância. Uma **chave** *Ch* de um esquema de relação *R* é uma superchave de *R* com a propriedade adicional de que a remoção de qualquer atributo *A* de *Ch* deixa um conjunto de atributos *Ch'* que não é mais uma superchave de *R*. Logo, uma chave satisfaz duas propriedades:

1. Duas tuplas distintas em qualquer estado da relação não podem ter valores idênticos para (todos) os atributos na chave. Essa primeira propriedade também se aplica a uma superchave.
2. Ela é uma *superchave mínima* — ou seja, uma superchave da qual não podemos remover nenhum atributo e ainda mantemos uma restrição de exclusividade na condição 1. Essa propriedade não é exigida por uma superchave.

Embora a primeira propriedade se aplique a chaves e superchaves, a segunda propriedade é exigida apenas para chaves. Assim, uma chave também é uma superchave, mas não o contrário. Considere a relação ALUNO da Figura 3.1. O conjunto de atributos {Cpf} é uma chave de ALUNO porque duas tuplas de aluno não podem ter o mesmo valor para Cpf.[8] Qualquer conjunto de atributos que inclua Cpf — por exemplo, {Cpf, Nome, Idade} — é uma superchave. No entanto, a superchave {Cpf, Nome, Idade} não é uma chave de ALUNO, pois remover Nome ou Idade, ou ambos, do conjunto ainda nos deixa com uma superchave. Em geral, qualquer superchave formada com base em um único atributo também é uma chave. Uma chave com múltiplos atributos precisa exigir que *todos* os seus atributos juntos tenham uma propriedade de exclusividade.

O valor de um atributo de chave pode ser usado para identificar exclusivamente cada tupla na relação. Por exemplo, o valor de Cpf 305.610.243-51 identifica exclusivamente a tupla correspondente a Bruno Braga na relação ALUNO. Observe que um conjunto de atributos constituindo uma chave é uma propriedade do esquema de relação; essa é uma restrição que deve ser mantida sobre *cada* estado do relação válido do esquema. Uma chave é determinada com base no significado dos atributos, e a propriedade é *invariável no tempo* : ela precisa permanecer verdadeira quando inserimos novas tuplas na relação. Por exemplo, não podemos e não devemos designar o atributo Nome da relação ALUNO da Figura 3.1 como uma chave porque é possível que dois alunos com nomes idênticos existam em algum ponto em um estado válido.[9]

CARRO

Placa	Numero_chassi	Marca	Modelo	Ano
Itatiaia ABC-7039	A6935207586	Volkswagen	Gol	02
Itu TVP-3470	B4369668697	Chevrolet	Corsa	05
Santos MPO-2902	X8355447376	Fiat	Uno	01
Itanhaem TFY-6858	C4374268458	Chevrolet	Celta	99
Itatiba RSK-6279	Y8293586758	Renault	Clio	04
Atibaia RSK-6298	U0283657858	Volkswagen	Parati	04

Figura 3.4
A relação CARRO, com duas chaves candidatas: Placa e Numero_chassi.

Em geral, um esquema de relação pode ter mais de uma chave. Nesse caso, cada uma das chaves é chamada de **chave candidata** . Por exemplo, a relação CARRO na Figura 3.4 tem duas chaves candidatas: Placa e Numero_chassi . É comum designar uma das chaves candidatas como **chave primária** da relação. Essa é a chave candidata cujos valores são usados para *identificar* tuplas na relação. Usamos a convenção de que os atributos que formam a chave primária de um esquema de relação são sublinhados, como mostra a Figura 3.1. Observe que, quando um esquema de relação tem várias chaves candidatas, a escolha de uma para se tornar a chave primária é um tanto quanto arbitrária; porém, normalmente é melhor escolher uma chave primária com um único atributo ou um pequeno número de atributos. As outras chaves candidatas são designadas como **chaves únicas** (*unique keys*), e não são sublinhadas.

Outra restrição sobre os atributos especifica se valores NULL são permitidos ou não. Por exemplo, se cada tupla de ALUNO precisar ter um valor válido, diferente de NULL, para o atributo Nome, então Nome de ALUNO é restrito a ser NOT NULL.

3.2.3 Bancos de dados relacionais e esquemas de banco de dados relacional

As definições e restrições que discutimos até aqui se aplicam a relações isoladas e seus atributos. Um

[8] Observe que Cpf também é uma superchave.

[9] Os nomes às vezes são usados como chaves, mas, nesse caso, algum artefato — como anexar um número ordinal — precisa ser usado para distinguir esses nomes idênticos.

banco de dados relacional costuma conter muitas relações, com tuplas nas relações que estão relacionadas de várias maneiras. Nesta seção, definimos um banco de dados relacional e um esquema de banco de dados relacional.

Um **esquema de banco de dados relacional** S é um conjunto de esquemas de relação $S = R_1, R_2, ..., R_m\}$ e um conjunto de **restrições de integridade** RI. Um **estado de banco de dados relacional**[10] DB de S é um conjunto de estados de relação $DB = r_1, r_2, ..., r_m\}$, tal que cada r_i é um estado de R_i e tal que os estados da relação r_i satisfazem as restrições de integridade especificadas em RI. A Figura 3.5 mostra um esquema de banco de dados relacional que chamamos de EMPRESA = {FUNCIONARIO, DEPARTAMENTO, LOCALIZACAO_DEP, PROJETO, TRABALHA_EM, DEPENDENTE}. Os atributos sublinhados representam chaves primárias. A Figura 3.6 mostra um estado de banco de dados relacional correspondente ao esquema EMPRESA. Usaremos esse esquema e estado de banco de dados neste capítulo e nos capítulos 4 a 6 para desenvolver consultas de exemplo em diferentes linguagens relacionais.

Quando nos referimos a um banco de dados relacional, implicitamente incluímos seu esquema e seu estado atual. Um estado de banco de dados que não obedece a todas as restrições de integridade é chamado de **estado inválido**, e um estado que satisfaz a todas as restrições no conjunto definido de restrições de integridade RI é chamado de **estado válido**.

Na Figura 3.5, o atributo Dnumero em DEPARTAMENTO e LOCALIZACAO_DEP significa o conceito do mundo real — o número dado a um departamento. O mesmo conceito é chamado Dnr em FUNCIONARIO e Dnum em PROJETO. Os atributos que representam o mesmo conceito do mundo real podem ou não ter nomes idênticos em diferentes relações. Como alternativa, os atributos que representam diferentes conceitos podem ter o mesmo nome em diferentes relações. Por exemplo, poderíamos ter usado o nome de atributo Nome para Projnome de PROJETO e Dnome de DEPARTAMENTO; nesse caso, teríamos dois atributos compartilhando o mesmo nome, mas representando diferentes conceitos do mundo real — nomes de projeto e nomes de departamento.

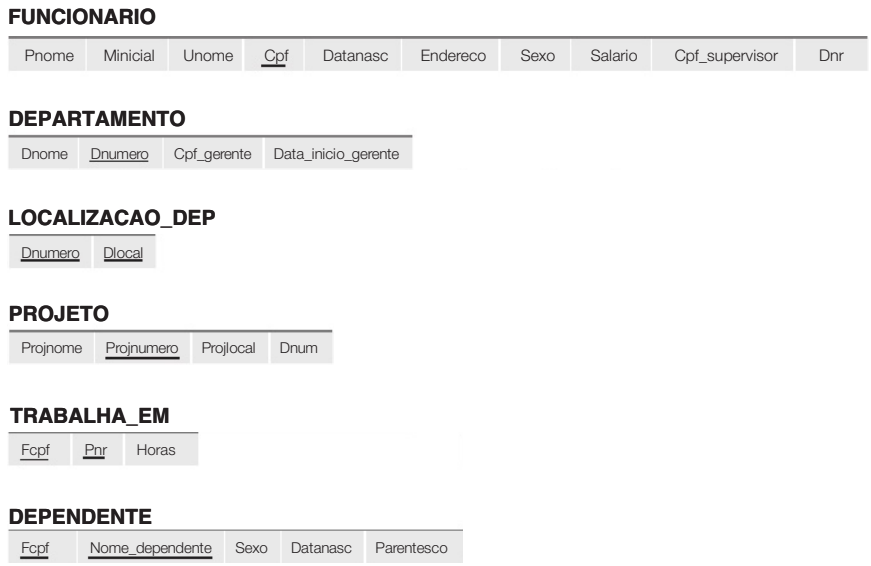


Figura 3.5
Diagrama de esquema para o esquema de banco de dados relacional EMPRESA.

[10] Um *estado* de banco de dados relacional às vezes é chamado de *instância* de banco de dados relacional. No entanto, como mencionamos anteriormente, não usaremos o termo *instância* porque ele também se aplica a tuplas isoladas.

Em algumas versões antigas do modelo relacional, era feita uma suposição de que o mesmo conceito do mundo real, quando representado por um atributo, teria nomes de atributo *idênticos* em todas as relações. Isso cria problemas quando o mesmo conceito do mundo real é usado em diferentes papéis (significados) na mesma relação. Por exemplo, o conceito de Cadastro de Pessoa Física aparece duas vezes na relação FUNCIONARIO da Figura 3.5: uma no papel do CPF do funcionário e outra no papel do CPF do supervisor. Precisamos dar-lhes nomes de atributo distintos — Cpf e Cpf_supervisor, respectivamente — porque eles aparecem na mesma relação e a fim de distinguir seu significado.

Cada SGBD relacional precisa ter uma linguagem de definição de dados (DDL) para estabelecer um esquema de banco de dados relacional. Os SGBDs relacionais atuais costumam usar principalmente SQL para essa finalidade. Apresentaremos a DDL SQL nas seções 4.1 e 4.2.

Restrições de integridade são especificadas em um esquema de banco de dados e espera-se que sejam mantidas em cada estado de banco de dados válido desse esquema. Além das restrições de domínio, chave e NOT NULL, dois outros tipos de restrições são considerados parte do modelo relacional: integridade de entidade e integridade referencial.

3.2.4 Integridade, integridade referencial e chaves estrangeiras

A **restrição de integridade de entidade** afirma que nenhum valor de chave primária pode ser NULL. Isso porque o valor da chave primária é usado para identificar tuplas individuais em uma relação. Ter valores NULL para a chave primária implica que não podemos identificar algumas tuplas. Por exemplo, se duas ou mais tuplas tivessem NULL para suas chaves primárias, não conseguiríamos distingui-las ao tentar referenciá-las por outras relações.

As restrições de chave e as restrições de integridade de entidade são especificadas sobre relações individuais. A **restrição de integridade referencial** é especificada entre duas relações e usada para manter a consistência entre tuplas nas duas relações. Informalmente, a restrição de integridade referencial afirma que uma tupla em uma relação que referencia outra relação precisa se referir a uma *tupla existente* nessa relação. Por exemplo, na Figura 3.6, o atributo Dnr de FUNCIONARIO fornece o número de departamento para o qual cada funcionário trabalha; logo, seu valor em cada tupla FUNCIONARIO precisa combinar com o valor de Dnumero de alguma tupla na relação DEPARTAMENTO.

Para definir a integridade referencial de maneira mais formal, primeiro estabelecemos o conceito de uma *chave estrangeira* (*ChE — foreign key*). As condições para uma chave estrangeira, dadas a seguir, especificam a restrição de integridade referencial entre os dois esquemas de relação R_1 e R_2. Um conjunto de atributos ChE no esquema de relação R_1 é uma **chave estrangeira** de R_1 que **referencia** a relação R_2 se ela satisfizer as seguintes regras:

1. Os atributos em ChE têm o mesmo domínio (ou domínios) que os atributos de chave primária ChP de R_2; diz-se que os atributos ChE **referenciam** ou **referem-se à** relação R_2.
2. Um valor de ChE em uma tupla t_1 do estado atual $r_1(R_1)$ ocorre como um valor de ChE para alguma tupla t_2 no estado atual $r_2(R_2)$ *ou é NULL*. No primeiro caso, temos t_1[ChE] = t_2[ChP], e dizemos que a tupla t_1 **referencia** ou **refere-se à** tupla t_2.

Nessa definição, R_1 é chamada de **relação que referencia** e R_2 é a **relação referenciada**. Se essas condições se mantiverem, diz-se que é mantida uma **restrição de integridade referencial** de R_1 para R_2. Em um banco de dados de muitas relações, normalmente existem muitas restrições de integridade referencial.

Para especificar essas restrições, primeiro devemos ter um conhecimento claro do significado ou papel que cada atributo, ou conjunto de atributos, que fazem parte nos diversos esquemas de relação do banco de dados. As restrições de integridade referencial surgem com frequência dos *relacionamentos entre as entidades* representadas pelos esquemas de relação. Por exemplo, considere o banco de dados mostrado na Figura 3.6. Na relação FUNCIONARIO, o atributo Dnr refere-se ao departamento para o qual um funcionário trabalha; portanto, designamos Dnr para ser a chave estrangeira de FUNCIONARIO que referencia a relação DEPARTAMENTO. Isso significa que um valor de Dnr em qualquer tupla t_1 da relação FUNCIONARIO precisa combinar com um valor da chave primária de DEPARTAMENTO — o atributo Dnumero — em alguma tupla t_2 da relação DEPARTAMENTO, ou o valor de Dnr *pode ser NULL* se o funcionário não pertencer a um departamento ou for atribuído a um departamento mais tarde. Por exemplo, na Figura 3.6, a tupla para o funcionário 'João Silva' referencia a tupla para o departamento 'Pesquisa', indicando que 'João Silva' trabalha para esse departamento.

Observe que uma chave estrangeira pode *se referir a sua própria relação*. Por exemplo, o atributo Cpf_supervisor em FUNCIONARIO refere-se ao supervisor de um funcionário; este é outro funcionário, representado por uma tupla na relação FUNCIONARIO.

FUNCIONARIO

Pnome	Minicial	Unome	Cpf	Datanasc	Endereco	Sexo	Salario	Cpf_supervisor	Dnr
João	B	Silva	12345678966	09-01-1965	Rua das Flores, 751, São Paulo, SP	M	30.000	33344555587	5
Fernando	T	Wong	33344555587	08-12-1955	Rua da Lapa, 34, São Paulo, SP	M	40.000	88866555576	5
Alice	J	Zelaya	99988777767	19-01-1968	Rua Souza Lima, 35, Curitiba, PR	F	25.000	98765432168	4
Jennifer	S	Souza	98765432168	20-06-1941	Av. Arthur de Lima, 54, Santo André, SP	F	43.000	88866555576	4
Ronaldo	K	Lima	66688444476	15-09-1962	Rua Rebouças, 65, Piracicaba, SP	M	38.000	33344555587	5
Joice	A	Leite	45345345376	31-07-1972	Av. Lucas Obes, 74, São Paulo, SP	F	25.000	33344555587	5
André	V	Pereira	98798798733	29-03-1969	Rua Timbira, 35, São Paulo, SP	M	25.000	98765432168	4
Jorge	E	Brito	88866555576	10-11-1937	Rua do Horto, 35, São Paulo, SP	M	55.000	NULL	1

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Data_inicio_gerente
Pesquisa	5	33344555587	22-05-1988
Administração	4	98765432168	01-01-1995
Matriz	1	88866555576	19-06-1981

LOCALIZACAO_DEP

Dnumero	Dlocal
1	São Paulo
4	Mauá
5	Santo André
5	Itu
5	São Paulo

TRABALHA_EM

Fcpf	Pnr	Horas
12345678966	1	32,5
12345678966	2	7,5
66688444476	3	40,0
45345345376	1	20,0
45345345376	2	20,0
33344555587	2	10,0
33344555587	3	10,0
33344555587	10	10,0
33344555587	20	10,0
99988777767	30	30,0
99988777767	10	10,0
98798798733	10	35,0
98798798733	30	5,0
98765432168	30	20,0
98765432168	20	15,0
88866555576	20	NULL

PROJETO

Projnome	Projnumero	Projlocal	Dnum
ProdutoX	1	Santo André	5
ProdutoY	2	Itu	5
ProdutoZ	3	São Paulo	5
Informatização	10	Mauá	4
Reorganização	20	São Paulo	1
Novosbenefícios	30	Mauá	4

DEPENDENTE

Fcpf	Nome_dependente	Sexo	Datanasc	Parentesco
33344555587	Alicia	F	05-04-1986	Filha
33344555587	Tiago	M	25-10-1983	Filho
33344555587	Janaína	F	03-05-1958	Esposa
98765432168	Antonio	M	28-02-1942	Marido
12345678966	Michael	M	04-01-1988	Filho
12345678966	Alicia	F	30-12-1988	Filha
12345678966	Elizabeth	F	05-05-1967	Esposa

Figura 3.6

Um estado de banco de dados possível para o esquema de banco de dados relacional EMPRESA.

Logo, Cpf_supervisor é uma chave estrangeira que referencia a própria relação FUNCIONARIO. Na Figura 3.6, a tupla que o funcionário 'Joã o Silva' referencia é a tupla do funcionário 'Fernando Wong', indicando que 'Fernando Wong' é o supervisor de 'João Silva'.

Podemos *exibir em forma de diagrama as restrições de integridade referencial*, desenhando um arco direcionado de cada chave estrangeira para a relação que ela referencia. Para ficar mais claro, a ponta da seta pode apontar para a chave primária da relação referenciada. A Figura 3.7 mostra o esquema da Fi-

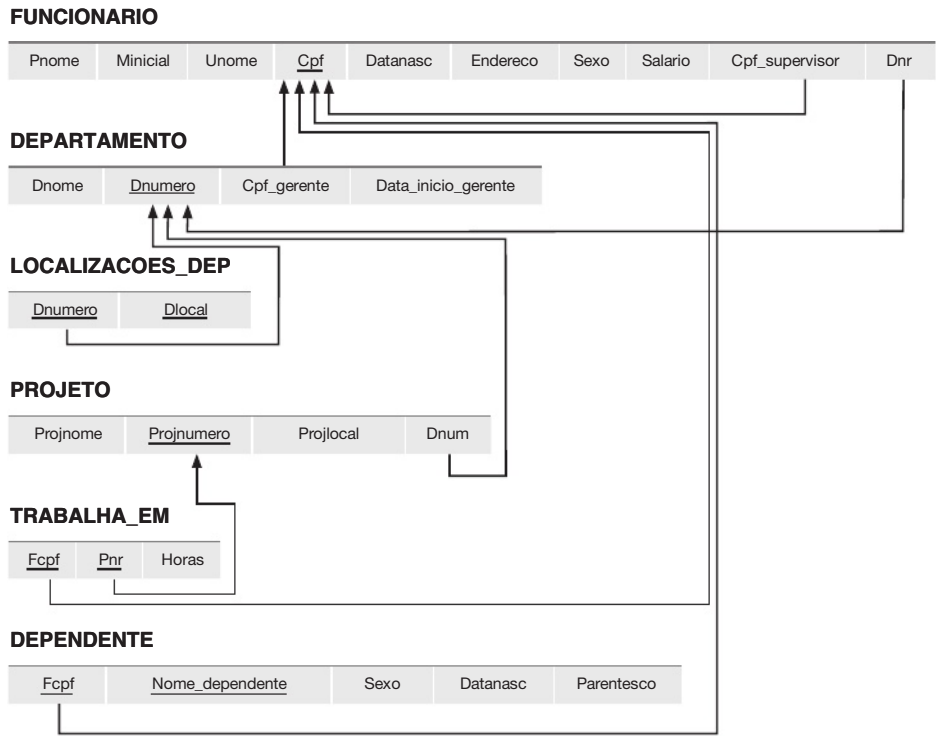


Figura 3.7
Restrições de integridade referencial exibidas no esquema de banco de dados relacional EMPRESA.

gura 3.5 com as restrições de integridade referencial mostradas dessa maneira.

Todas as restrições de integridade deverão ser especificadas no esquema de banco de dados relacional (ou seja, definidas como parte de sua definição) se quisermos impor essas restrições sobre os estados do banco de dados. Logo, a DDL inclui meios para especificar os diversos tipos de restrições de modo que o SGBD possa impô-las automaticamente. A maioria dos SGBDs relacionais admite restrições de chave, integridade de entidade e integridade referencial. Essas restrições são especificadas como uma parte da definição de dados na DDL.

3.2.5 Outros tipos de restrições

As restrições de integridade anteriores estão incluídas na linguagem de definição de dados porque ocorrem na maioria das aplicações de banco de dados. No entanto, elas não incluem uma grande classe de restrições gerais, também chamadas de *restrições de integridade semântica*, que podem ter de ser especificadas e impostas em um banco de dados relacional. Alguns exemplos dessas restrições são *o salário de um funcionário não deve ser superior ao salário de seu supervisor* e *o número máximo de horas que um funcionário pode trabalhar em todos os projetos por semana é 56*. Essas restrições podem ser especificadas e impostas em programas de aplicação que atualizam o banco de dados, ou usando uma **linguagem de especificação de restrição** de uso geral. Mecanismos chamados **triggers** (gatilhos) e **assertions** (afirmações) podem ser usados. Em SQL, os comandos CREATE ASSERTION e CREATE TRIGGER podem ser usados para essa finalidade (ver Capítulo 5). É mais comum verificar esses tipos de restrições em programas de aplicação do que usar linguagens de especificação de restrição, pois estas às vezes são difíceis e complexas de se usar, conforme discutiremos na Seção 26.1.

Outro tipo de restrição é a de *dependência funcional*, que estabelece um relacionamento funcional

entre dois conjuntos de atributos X e Y. Essa restrição especifica que o valor de X determina um valor exclusivo de Y em todos os estados de uma relação; isso é indicado como uma dependência funcional $X \rightarrow Y$. Usaremos dependências funcionais e outros tipos de dependências nos capítulos 15 e 16 como ferramentas para analisar a qualidade dos projetos relacionais e 'normalizar' relações para melhorar sua qualidade.

Os tipos de restrições que discutimos até aqui podem ser chamados de **restrições de estado**, pois definem as restrições às quais um *estado válido* do banco de dados precisa satisfazer. Outro tipo de restrição, chamadas **restrições de transição**, pode ser definido para lidar com mudanças de estado no banco de dados.[11] Um exemplo de uma restrição de transição é: 'o salário de um funcionário só pode aumentar'. Tais restrições costumam ser impostas pelos programas de aplicação ou especificadas usando regras ativas e triggers, conforme discutiremos na Seção 26.1.

3.3 Operações de atualização, transações e tratamento de violações de restrição

As operações do modelo relacional podem ser categorizadas em *recuperações* e *atualizações*. As operações da álgebra relacional, que podem ser usadas para especificar **recuperação**, serão discutidas com detalhes no Capítulo 6. Uma expressão da álgebra relacional forma uma nova relação após a aplicação de uma série de operadores algébricos a um conjunto existente de relações; seu uso principal é consultar um banco de dados a fim de recuperar informações. O usuário formula uma consulta que especifica os dados de interesse, e uma nova relação é formada aplicando operadores relacionais para recuperar esses dados. Esta **relação resultado** torna-se a resposta para a (ou resultado da) consulta do usuário. O Capítulo 6 também introduz a linguagem chamada cálculo relacional, que é usada para definir a nova relação de forma declarativa sem dar uma ordem específica das operações.

Nesta seção, concentramo-nos nas operações de **modificação** ou **atualização** do banco de dados. Existem três operações básicas que podem mudar os estados das relações no banco de dados: Inserir, Excluir e Alterar (ou Modificar). Elas inserem novos dados, excluem dados antigos ou modificam registros de dados existentes. **Insert** é usado para inserir uma ou mais novas tuplas em uma relação, **Delete** é usado para excluir tuplas, e **Update** (ou **Modify**) é usado para alterar os valores de alguns atributos nas tuplas existentes. Sempre que essas operações são aplicadas, as restrições de integridade especificadas sobre o esquema de banco de dados relacional não devem ser violadas. Nesta seção, discutimos os tipos de restrições que podem ser violadas por cada uma dessas operações e os tipos de ações que podem ser tomados se uma operação causar uma violação. Usamos o banco de dados mostrado na Figura 3.6 para os exemplos e discutimos apenas as restrições de chave, restrições de integridade de entidade e as restrições de integridade referencial mostradas na Figura 3.7. Para cada tipo de operação, damos alguns exemplos e discutimos as restrições que cada operação pode violar.

3.3.1 A operação Inserir

A operação **Inserir** oferece uma lista de valores de atributo para que uma nova tupla t possa ser inserida em uma relação R. Ela pode violar qualquer um dos quatro tipos de restrições discutidos na seção anterior. As restrições de domínio podem ser violadas se for dado um valor de atributo que não aparece no domínio correspondente ou não é do tipo de dado apropriado. As restrições de chave podem ser violadas se um valor de chave na nova tupla t já existir em outra tupla na relação $r(R)$. A integridade de entidade pode ser violada se qualquer parte da chave primária da nova tupla t for NULL. A integridade referencial pode ser violada se o valor de qualquer chave estrangeira em t se referir a uma tupla que não existe na relação referenciada. Aqui estão alguns exemplos para ilustrar essa discussão.

- *Operação*:

  Inserir <'Cecilia', 'F', 'Ribeiro', NULL, '05-04-1960', 'Rua Esmeraldas, 35, Bueno Brandão, MG', F, 28.000, NULL, 4> em FUNCIONARIO.

  Resultado: Esta inserção viola a restrição de integridade de entidade (NULL para a chave primária Cpf), de modo que é rejeitada.

- *Operação*:

  Inserir <'Alice', 'J', 'Zelaya', '99988777767', '05-04-1960', 'Rua Souza Lima, 35, Curitiba, PR', F, 28.000, '98765432168', 4> em FUNCIONARIO.

  Resultado: Esta inserção viola a restrição de chave porque outra tupla com o mesmo valor de Cpf já existe na relação FUNCIONARIO, e, portanto, é rejeitada.

[11] As restrições de estado também podem ser chamadas de ***restrições estáticas***, e as restrições de transição também são chamadas de ***restrições dinâmicas***.

- *Operação*:

  Inserir <'Cecilia','F','Ribeiro','67767898976', '05-04-1960', 'Rua Esmeraldas, 35, Bueno Brandão, MG', F, 28.000, '98765432168', 7> em FUNCIONARIO.

  Resultado: Esta inserção viola a restrição de integridade referencial especificada sobre Dnr em FUNCIONARIO porque não existe uma tupla referenciada correspondente em DEPARTAMENTO com Dnumero = 7.

- *Operação*:
  Inserir <'Cecilia','F','Ribeiro','67767898976', '05-04-1960', 'Rua Esmeraldas, 35, Bueno Brandão, MG', F, 28.000,NULL, 4> em FUNCIONARIO.

  Resultado: Esta inserção satisfaz todas as restrições, de modo que é aceitável.

Se uma inserção violar uma ou mais restrições, a opção padrão é *rejeitar a inserção* . Nesse caso, seria útil se o SGBD pudesse oferecer um motivo ao usuário sobre a rejeição da inserção. Outra opção é tentar *corrigir o motivo da rejeiç ão da inserção* , mas isso normalmente não é usado para violações causadas pela operação Inserir; em vez disso, é usado com mais frequência na correção de violações das operações Excluir e Alterar. Na primeira operação, o SGBD poderia pedir ao usuário para oferecer um valor para Cpf, e poderia então aceitar a inserção se um valor de Cpf válido fosse fornecido. Na operação 3, o SGBD poderia pedir que o usuário mudasse o valor de Dnr para algum valor válido (ou defini-lo como NULL), ou poderia pedir ao usuário para inserir uma tupla DEPARTAMENTO com Dnumero = 7 e poderia aceitar a inserção srcinal somente depois que uma operação fosse aceita. Observe que, no último caso, a violação de inserção pode se **propagar** de volta à relação FUNCIONARIO se o usuário tentar inserir uma tupla para o departamento 7 com um valor para Cpf_gerente que não existe na relação FUNCIONARIO .

3.3.2 A operação Excluir

A operação **Excluir** pode violar apenas a integridade referencial. Isso ocorre se a tupla que está sendo excluída for referenciada por chaves estrangeiras de outras tuplas no banco de dados. Para especificar a exclusão, uma condição sobre os atributos da relação seleciona a tupla (ou tuplas) a ser(em) excluída(s). Aqui estão alguns exemplos.

- *Operação*:

  Excluir a tupla em TRABALHA_EM com Fcpf = '99988777767' e Pnr = 10.

  Resultado: Esta exclusão é aceitável e exclui exatamente uma tupla.

- *Operação*:

  Excluir a tupla em FUNCIONARIO com Cpf = '99988777767'.

  Resultado: Esta exclusão não é aceitável, pois existem tuplas em TRABALHA_EM que se referenciam a esta tupla. Logo, se a tupla em FUNCIONARIO for excluída, haverá violações de integridade referencial.

- *Operação*:

  Excluir a tupla em FUNCIONARIO com Cpf = '33344555587'.

  Resultado: Esta exclusão resultará em ainda mais violações de integridade referencial, pois a tupla envolvida é referenciada por tuplas das relações FUNCIONARIO, DEPARTAMENTO, TRABALHA_EM e DEPENDENTE.

Várias opções estão disponíveis se uma operação de exclusão causar uma violação. A primeira opção, chamada **restrict**, é *rejeitar a exclusão*. A segunda opção, chamada **cascade**, é *tentar propagar* (ou *gerar em cascata*) a *exclusão* às tuplas que referenciam aquela que está sendo excluída. Por exemplo, na operação 2, o SGBD poderia excluir automaticamente as tuplas problemáticas de TRABALHA_EM com Fcpf = '99988777767'. Uma terceira opção, chamada **set null** ou **set default**, é *modificar os valores de atributo que referenciam* a causa da violação; cada valor desse tipo é definido para NULL ou alterado para referenciar outra tupla de valor válido. Observe que, se um atributo referenciando que causa uma violação faz *parte da chave primária*, ele *não pode* ser definido como NULL; caso contrário, ele violaria a integridade de entidade.

Combinações dessas três opções também são possíveis. Por exemplo, para evitar que a operação 3 cause uma violação, o SGBD pode excluir automaticamente todas as tuplas de TRABALHA_EM e DEPENDENTE com Fcpf = '33344555587'. As tuplas em FUNCIONARIO com Cpf_supervisor = '33344555587' e DEPARTAMENTO com Cpf_gerente = '33344555587' podem ter seus valores Cpf_supervisor e Cpf_gerente alterados para outros valores válidos ou para NULL. Embora possa fazer sentido excluir automaticamente as tuplas de TRABALHA_EM e DEPENDENTE que se referem a uma tupla de FUNCIONARIO, pode não fazer sentido excluir outras tuplas de FUNCIONARIO ou uma tupla de DEPARTAMENTO.

Em geral, quando uma restrição de integridade referencial é especificada na DDL, o SGBD permitirá que o projetista de banco de dados *especifique qual das opções* se aplica no caso de uma violação da restrição. Discutiremos como especificar essas opções na DDL SQL no Capítulo 4.

3.3.3 A operação Alterar

A operação **Alterar** (ou **Modificar**) é usada para alterar os valores de um ou mais atributos em uma tupla (ou tuplas) de alguma relação *R*. É necessário especificar uma condição sobre os atributos da relação para selecionar a tupla ou tuplas a serem modificadas. Aqui estão alguns exemplos.

- *Operação*:

  Alterar o salário da tupla em FUNCIONARIO com Cpf = '99988777767' para 28.000.

  Resultado: Aceitável.

- *Operação*:

  Alterar o Dnr da tupla em FUNCIONARIO com Cpf = '99988777767' para 1.

  Resultado: Aceitável.

- *Operação*:

  Alterar o Dnr da tupla em FUNCIONARIO com Cpf = '99988777767' para 7.

  Resultado: Inaceitável, pois viola a integridade referencial.

- *Operação*:

  Alterar o Cpf da tupla em FUNCIONARIO com Cpf = '99988777767' para '98765432168'.

  Resultado: Inaceitável, pois viola a restrição de chave primária, repetindo um valor que já existe como chave primária em outra tupla; isso viola as restrições de integridade referencial porque existem outras relações que se referem ao valor existente de Cpf.

Atualizar um atributo que *nem faz parte de uma chave primária nem de uma chave estrangeira* em geral não causa problemas; o SGBD só precisa verificar para confirmar se o novo valor é do tipo de dado e domínio corretos. Modificar um valor de chave primária é semelhante a excluir uma tupla e inserir outra em seu lugar, pois usamos a chave primária para identificar tuplas. Logo, as questões discutidas anteriormente nas seções 3.3.1 (Operação Inserir) e 3.3.2 (Operação Excluir) entram em cena. Se um atributo de chave estrangeira for modificado, o SGBD deverá garantir que o novo valor referencia a uma tupla existente na relação referenciada (ou que seja definido com NULL). Existem opções semelhantes para lidar com as violações de integridade referencial causadas pela operação Alterar, como as opções discutidas para a operação Excluir. De fato, quando uma restrição de integridade referencial for especificada na DDL, o SGBD permitirá que o usuário escolha opções separadas para lidar com uma violação causada pela operação Excluir e uma violação causada pela operação Alterar (ver a Seção 4.2).

3.3.4 O conceito de transação

Um programa de aplicação de banco de dados que executa com um banco de dados relacional normalmente executa uma ou mais *transações*. Uma **transação** é um programa em execução que inclui algumas operações de banco de dados, como fazer a leitura do banco de dados ou aplicar inserções, exclusões ou atualizações a ele. Ao final da transação, ela precisa deixar o banco de dados em um estado válido ou coerente, que satisfaça todas as restrições especificadas no esquema do banco de dados. Uma única transação pode envolver qualquer número de operações de recuperação (a serem discutidas como parte da álgebra e cálculo relacional no Capítulo 6, e como uma parte da linguagem SQL nos capítulos 4 e 5) e qualquer número de operações de atualização. Essas recuperações e atualizações juntas formarão uma unidade atômica de trabalho no banco de dados. Por exemplo, uma transação para aplicar um saque bancário costuma ler o registro da conta do usuário, verificar se existe saldo suficiente e depois atualizar o registro pelo valor do saque.

Um grande número de aplicações comerciais, que executam com bancos de dados relacionais em sistemas de **processamento de transação on-line** (**OLTP — Online Transaction Processing**), executa transações que atingem taxas de centenas por segundo. Os conceitos de processamento de transação, execução concorrente de transações e recuperação de falhas serão discutidos nos capítulos 21 a 23.

Resumo

Neste capítulo, apresentamos os conceitos de modelagem, estruturas de dados e restrições, fornecidos pelo modelo relacional de dados. Começamos apresentando os conceitos de domínios, atributos e tuplas. Depois, definimos um esquema de relação como uma lista de atributos que descrevem a estrutura de uma relação. Uma relação, ou estado de relação, é um conjunto de tuplas que correspondem ao esquema.

Várias características diferenciam relações das tabelas ou arquivos comuns. A primeira é que uma relação não é sensível à ordem das tuplas. A segunda envolve a ordem dos atributos em um esquema de relação e a ordem correspondente dos valores dentro de uma tupla. Oferecemos uma definição alternativa de relação que não exige essas duas ordens, mas continuamos a usar a

primeira definição, que requer que atributos e valores de tupla sejam ordenados, por conveniência. Depois, discutimos os valores nas tuplas e apresentamos os valores NULL para representar informações faltantes ou desconhecidas. Enfatizamos que valores NULL devem ser evitados ao máximo.

Classificamos as restrições do banco de dados em restrições inerentes baseadas no modelo, restrições explícitas baseadas no esquema e restrições baseadas na aplicação, também conhecidas como restrições semânticas ou regras de negócios. Depois, discutimos as restrições de esquema pertencentes ao modelo relacional, começando com as restrições de domínio, depois as restrições de chave, incluindo os conceitos de superchave, chave candidata e chave primária, e a restrição NOT NULL sobre atributos. Definimos bancos de dados relacionais e esquemas de banco de dados relacionais. Outras restrições relacionais incluem a restrição de integridade de entidade, que proíbe que atributos de chave primária sejam NULL. Descrevemos a restrição de integridade referencial entre relações, que é usada para manter a consistência das referências entre tuplas de diferentes relações.

As operações de modificação no modelo relacional são Inserir, Excluir e Alterar. Cada operação pode violar certos tipos de restrições (consulte a Seção 3.3). Sempre que uma operação é aplicada, o estado do banco de dados após a operação ser executada deve ser verificado para garantir que nenhuma restrição seja violada. Finalmente, apresentamos o conceito de transação, que é importante nos SGBDs relacionais porque permite o agrupamento de ações em uma unidade atômica. Uma transação pode ser definida como uma unidade de trabalho lógica sobre o banco de dados.

Perguntas de revisão

3.1. Defina os termos a seguir quando se aplicam ao modelo de dados relacional: *domínio*, *atributo*, *tupla n*, *esquema de relação*, *estado de relação*, *grau da relação*, *esquema de banco de dados relacional* e *estado de banco de dados relacional*.

3.2. Por que as tuplas em uma relação não são ordenadas?

3.3. Por que as tuplas duplicadas não são permitidas em uma relação?

3.4. Qual é a diferença entre uma chave e uma superchave?

3.5. Por que designamos uma das chaves candidatas de uma relação como sendo a chave primária?

3.6. Discuta as características de relações que as tornam diferentes das tabelas e arquivos comuns.

3.7. Discuta os diversos motivos que levam à ocorrência de valores NULL nas relações.

3.8. Discuta as restrições de integridade de entidade e integridade referencial. Por que são consideradas importantes?

3.9. Defina a *chave estrangeira*. Para que esse conceito é usado?

3.10. O que é uma transação? Como ela difere de uma operação atualização?

Exercícios

3.11. Suponha que cada uma das seguintes operações de atualização seja aplicada diretamente ao estado do banco de dados mostrado na Figura 3.6. Discuta *todas* as restrições de integridade violadas por cada operação, se houver, e as diferentes maneiras de lidar com essas restrições.

a. Inserir <'Roberto','F','Santos','943775543 5', '21-06-1972', 'Rua Benjamin, 34, Santo André, SP', M, 58.000, '88866555576', 1> em FUNCIONARIO.

b. Inserir <'ProdutoA', 4, 'Santo André', 2> em PROJETO.

c. Inserir <'Producao', 4, '94377554355', '01-10-2007'> em DEPARTAMENTO.

d. Inserir <'67767898944', NULL, '40,0'> em TRABALHA_EM.

e. Inserir <'45345345376', 'João', 'M', '12-12-1990', 'marido'> em DEPENDENTE.

f. Excluir as tuplas de TRABALHA_EM com Fcpf = '33344555587'.

g. Excluir a tupla de FUNCIONARIO com Cpf = '98765432168'.

h. Excluir a tupla de PROJETO com Projnome = 'ProdutoX'.

i. Modificar Cpf_gerente e Data_inicio_gerente da tupla DEPARTAMENTO com Dnumero = 5 para '12345678966' e '01-10-2007', respectivamente.

j. Modificar o atributo Cpf_supervisor da tupla FUNCIONARIO com Cpf = '99988777767' para '94377554355'.

k. Modificar o atributo Horas da tupla TRABALHA_EM com Fcpf = '99988777767' e Pnr = 10 para '5,0'.

3.12. Considere o esquema do banco de dados relacional COMPANHIA AEREA mostrado na Figura 3.8, que descreve um banco de dados para informações de voo. Cada VOO é identificado por um Numero_voo, e consiste em um ou mais TRECHOs_VOO com Numero_trecho 1, 2, 3, e assim por diante. Cada TRECHO_VOO tem agendados horários de chegada e saída, aeroportos e uma ou mais INSTANCIAs_TRECHO — uma para cada Data em que o voo ocorre. TARIFAs são mantidas para cada VOO. Para cada instância de TRECHO_VOO, RESERVAs_ASSENTO são mantidas, assim como a AERONAVE usada no trecho e os horários de chegada e saída e aeroportos reais. Uma AERONAVE é identificada por um Codigo_aeronave e tem um TIPO_AERONAVE em particular. PODE_POUSAR relaciona os

TIPOs_AERONAVE aos AEROPORTOs em que eles podem aterrissar. Um AEROPORTO é identificado por um Codigo_aeroporto. Considere uma atualização para o banco de dados COMPANHIA AEREA entrar com uma reserva em um voo em particular ou trecho de voo em determinada data.

a. Indique as operações para esta atualização.

b. Que tipos de restrições você esperaria verificar?

c. Quais dessas restrições são de chave, de integridade de entidade e de integridade referencial, e quais não são?

d. Especifique todas as restrições de integridade referencial que se mantêm no esquema mostrado na Figura 3.8.

AEROPORTO

Codigo_aeroporto	Nome	Cidade	Estado

VOO

Numero_voo	Companhia aerea	Dias da semana

TRECHO_VOO

Numero_voo	Numero_trecho	Codigo_aeroporto_partida	Horario_partida_previsto
		Codigo_aeroporto_chegada	Horario_chegada_previsto

INSTANCIA_TRECHO

Numero_voo	Numero_trecho	Data	Numero_assentos_disponiveis	Codigo_aeronave
Codigo_aeroporto_partida	Horario_partida	Codigo_aeroporto_chegada	Horario_chegada	

TARIFA

Numero_voo	Codigo_tarifa	Quantidade	Restricoes

TIPO_AERONAVE

Nome_tipo_aeronave	Qtd_max_assentos	Companhia

PODE_POUSAR

Nome_tipo_aeronave	Codigo_aeroporto

AERONAVE

Codigo_aeronave	Numero_total_assentos	Tipo_aeronave

RESERVA_ASSENTO

Numero_voo	Numero_trecho	Data	Numero_assento	Nome_cliente	Telefone_cliente

Figura 3.8
O esquema do banco de dados relacional COMPANHIA AEREA.

3.13. Considere a relação AULA(Num_disciplina, Num_turma, Nome_professor, Semestre, Codigo_edificio, Num_Sala, Turno, Dias_da_semana, Creditos). Isso representa as aulas lecionadas em uma universidade, com Num_Turma única. Identifique quais você acha que devem ser as diversas chaves candidatas e escreva, com suas palavras, as condições ou suposições sob as quais as chaves candidatas seriam válidas.

3.14. Considere as seis relações a seguir para uma aplicação de banco de dados de processamento de pedido em uma empresa:

CLIENTE(Num_~~cliente~~, Nome_cliente, Num_cidade)
PEDIDO(Num_pedido, Data_pedido, Num_cliente, Num_pedido)
ITEM_PEDIDO(Num_pedido, Num_item, Quantidade)
ITEM(Num_item, Preco_unitario)
EXPEDICAO(Num_pedido, Num_deposito, Data_envio)
DEPOSITO(Num_deposito#, Cidade)

Aqui, Valor_pedido refere-se ao valor total em reais de um pedido; Data_pedido é a data em que o pedido foi feito; e Data_envio é a data em que um pedido (ou parte de um pedido) é despachado do depósito. Suponha que um pedido possa ser despachado de vários depósitos. Especifique chaves estrangeiras para esse esquema, indicando quaisquer suposições que você faça. Que outras restrições você imagina para esse banco de dados?

3.15. Considere as seguintes relações para um banco de dados que registra viagens de negócios de vendedores em um escritório de vendas:

VENDEDOR(Cpf, Nome, Ano_inicio, Numero_departamento)
VIAGEM(Cpf, Cidade_srcem, Cidade_destino, Data_partida, Data_retorno, Cod_viagem)
DESPESA(Cod_viagem, Num_conta, Valor)

Uma viagem pode ser cobrada de uma ou mais contas (Num_conta). Especifique as chaves estrangeiras para esse esquema, indicando quaisquer suposições que você faça.

3.16. Considere asseguintes relaçõespara um banco de dados que registra a matrícula do aluno nas disciplinas e os livros adotados para cada disciplina:

ALUNO(Cpf, Nome, Curso, Datanasc)
DISCIPLINA(Num_Disciplina, Dnome, Dept)
INSCRICAO(Cpf, Num_disciplina, Semestre, Nota)
LIVRO_ADOTADO(Num_disciplina, Semestre, ISBN_livro)
LIVRO(ISBN_livro, Titulo_livro, Editora, Autor)

Especifique as chaves estrangeiras para este esquema, indicando quaisquer suposições que você faça.

3.17. Considere as seguintes relações para um banco de dados que registra vendas de automóveis em um revendedor de carros (OPCAO refere-se a algum equipamento opcional instalado em um automóvel):

CARRO(Numero_chassi, Modelo, Fabricante, Preco)
OPCAO(Numero_chassi, Nome_opcional, Preco)
VENDA(Cod_vendedor, Numero_chassi, Data, Preco_venda)
VENDEDOR(Cod_vendedor, Nome, Telefone)

Primeiro, especifique as chaves estrangeiras para este esquema, indicando quaisquer suposições que você faça. Depois, preencha as relações com algumas tuplas de exemplo, e então mostre um exemplo de uma inserção nas relações VENDA e VENDEDOR que *viola* as restrições de integridade referencial e de outra inserção que não viola.

3.18. O projeto de banco de dados normalmente envolve decisões sobre o armazenamento de atributos. Por exemplo, o Cadastro de Pessoa Física pode ser armazenado como um atributo ou dividido em quatro atributos (um para cada um dos quatro grupos separados por hífen em um Cadastro de Pessoa Física — XXX-XXX-XXX-XX). Porém, os números do Cadastro de Pessoa Física costumam ser representados como apenas um atributo. A decisão é baseada em como o banco de dados será usado. Este exercício pede para você pensar nas situações específicas onde a divisão do CPF é útil.

3.19. Considere uma relação ALUNO em um banco de dados UNIVERSIDADE com os seguintes atributos (Nome, Cpf, Telefone_local, Endereco, Telefone_celular, Idade, Media). Observe que o telefone celular pode ser de uma cidade e estado diferentes do telefone local. Uma tupla possível da relação é mostrada a seguir:

Nome	Cpf	Telefone_local	Endereco	Telefone_celular	Idade	Media
Jorge Pereira William Ribeiro	123-459-678-97	5555-1234	Rua Cambará, 33, Bauru, SP	8555-4321	19	3,75

a. Identifique a informação crítica que falta nos atributos Telefone_local e Telefone_celular. (*Dica:* Como você liga para alguém que mora em um estado diferente?)

b. Você armazenaria essa informação adicional nos atributos Telefone_local e Telefone_celular ou incluiria novos atributos ao esquema para ALUNO?

c. Considere o atributo Nome. Quais são as vantagens e desvantagens de dividir esse campo de um atributo em três atributos (primeiro nome, nome do meio e sobrenome)?

d. Que orientação geral você daria para decidir quando armazenar informações em um único atributo e quando separar a informação?

e. Suponha que o aluno possa ter entre 0 e 5 telefones. Sugira dois projetos diferentes que permitam esse tipo de informação.

3.20. Mudanças recentes nas leis de privacidade dos EUA não permitiram que as organizações usem números de Seguro social (SSN) para identificar indivíduos, a menos que certas restrições sejam satisfeitas. Como resultado, a maioria das universidades nos EUA não pode usar SSNs como chaves primárias (exceto para dados financeiros). Na prática, Cod_aluno, um identificador exclusivo atribuído a cada aluno, provavelmente será usado como chave primária, em vez do Ssn, pois Cod_aluno pode ser usado por todo o sistema.

a. Alguns projetistas de banco de dados preferem projetar chaves geradas (também conhecidas como *chaves substitutas*) para chaves primárias (como Cod_aluno), pois elas são artificiais. Você consegue propor algumas escolhas naturais de chaves que podem ser usadas para identificar o registro do aluno no banco de dados UNIVERSIDADE?

b. Suponha que você consiga garantir a exclusividade de uma chave natural que inclua sobrenome. Você tem garantias de que ele não mudará durante o tempo de vida do banco de dados? Se o sobrenome puder mudar, quais soluções óbvias você pode propor para criar uma chave primária que ainda o inclua, mas permaneça exclusiva?

c. Quais são as vantagens e desvantagens de usar chaves geradas (substitutas)?

Bibliografia selecionada

O modelo relacional foi introduzido por Codd (1970) em um artigo clássico. Codd também introduziu a álgebra relacional e estabeleceu as bases teóricas para o modelo relacional em uma série de artigos (Codd, 1971, 1972, 1972a, 1974); mais tarde, ele recebeu o Turing Award, a honra mais alta da ACM (Association for Computing Machinery) por seu trabalho sobre o modelo relacional. Em um artigo posterior, Codd (1979) discutiu a extensão do modelo relacional para incorporar mais metadados e semântica sobre as relações. Ele também propôs uma lógica de três valores para lidar com a incerteza nas relações e incorporar NULLs na álgebra relacional. O modelo resultante é conhecido como RM/T. Childs (1968) usou inicialmente a teoria de conjunto para modelar bancos de dados. Mais tarde, Codd (1990) publicou um livro examinando mais de 300 recursos do modelo de dados relacional e sistemas de banco de dados. Date (2001) oferece uma crítica e análise retrospectiva do modelo de dados relacional.

Desde o trabalho pioneiro de Codd, muita pesquisa tem sido realizada sobre vários aspectos do modelo relacional. Todd (1976) descreve um SGBD experimental chamado PRTV que implementa diretamente as operações da álgebra relacional. Schmidt e Swenson (1975) apresentam semântica adicional para o modelo relacional, classificando diferentes tipos de relações. O modelo Entidade-Relacionamento de Chen (1976), que será discutido no Capítulo 7, é um meio de comunicar a semântica do mundo real de um banco de dados relacional no nível conceitual. Wiederhold e Elmasri (1979) introduzem diversos tipos de conexões entre relações para aprimorar suas restrições. As extensões do modelo relacional serão discutidas nos capítulos 11 e 26. Notas bibliográficas adicionais para outros aspectos do modelo relacional e suas linguagens, sistemas, extensões e teoria serão apresentados nos capítulos 4 a 6, 9, 11, 13, 15, 16, 24 e 25. Maier (1983) e Atzeni e De Antonellis (1993) oferecem um extenso tratamento teórico do modelo de dados relacional.

capítulo

4 SQL básica

A linguagem SQL pode ser considerada um dos principais motivos para o sucesso dos bancos de dados relacionais comerciais. Como ela se tornou um padrão para esse tipo de bancos de dados, os usuários ficaram menos preocupados com a migração de suas aplicações de outros tipos de sistemas de banco de dados — por exemplo, sistemas de rede e hierárquicos — para sistemas relacionais. Isso aconteceu porque, mesmo que os usuários estivessem insatisfeitos com o produto de SGBD relacional em particular que estavam usando, a conversão para outro produto de SGBD relacional não seria tão cara ou demorada, pois os dois sistemas seguiam os mesmos padrões de linguagem. Na prática, é óbvio, existem muitas diferenças entre diversos pacotes de SGBD relacionais comerciais. Porém, se o usuário for cuidadoso em usar apenas dos recursos que fazem parte do padrão, e se os dois sistemas relacionais admitirem fielmente o padrão, então a conversão entre ambos deverá ser bastante simplificada. Outra vantagem de ter esse padrão é que os usuários podem escrever comandos em um programa de aplicação de banco de dados que pode acessar dados armazenados em dois ou mais SGBDs relacionais sem ter de mudar a sublinguagem de banco de dados (SQL) se os sistemas admitirem o padrão SQL.

Este capítulo apresenta os principais recursos do padrão SQL para SGBDs relacionais *comerciais*, enquanto o Capítulo 3 apresentou os conceitos mais importantes por trás do modelo de dados relacional *formal*. No Capítulo 6 (seções 6.1 a 6.5), discutiremos as operações da *álgebra relacional*, que são muito importantes para entender os tipos de solicitações que podem ser especificadas em um banco de dados relacional. Elas também são importantes para processamento e otimização de consulta em um SGBD relacional, conforme veremos no Capítulo 19. No entanto, as operações da álgebra relacional são consideradas muito técnicas para a maioria dos usuários de SGBD comercial, pois uma consulta em álgebra relacional é escrita como uma sequência de operações que, quando executadas, produz o resultado exigido. Logo, o usuário precisa especificar como — ou seja, *em que ordem* — executar as operações de consulta. Por sua vez, a linguagem SQL oferece uma interface de linguagem *declarativa* de nível mais alto, de modo que o usuário apenas especifica *qual* deve ser o resultado, deixando a otimização real e as decisões sobre como executar a consulta para o SGBD. Embora a SQL inclua alguns recursos da álgebra relacional, ela é baseada em grande parte no *cálculo relacional de tupla*, que descrevemos na Seção 6.6. Porém, a sintaxe SQL é mais fácil de ser utilizada do que qualquer uma das duas linguagens formais.

O nome **SQL** hoje é expandido como Structured Query Language (Linguagem de Consulta Estruturada). Originalmente, SQL era chamada de SEQUEL (Structured English QUEry Language) e foi criada e implementada na IBM Research como a interface para um sistema de banco de dados relacional experimental, chamado SYSTEM R. A SQL agora é a linguagem padrão para SGBDs relacionais comerciais. Um esforço conjunto entre o American National Standards Institute (ANSI) e a International Standards Organization (ISO) levou a uma versão-padrão da SQL (ANSI, 1986), chamada SQL-86 ou SQL1. Um padrão revisado e bastante expandido, denominado SQL-92 (também conhecido como SQL2) foi desenvolvido mais tarde. O próximo padrão reconhecido foi SQL:1999, que começou como SQL3. Duas atualizações posteriores ao padrão são SQL:2003 e SQL:2006, que acrescentaram recursos de XML (ver Capítulo 12) entre outras atualizações para a linguagem. Outra atualização em 2008 incorporou mais

recursos de banco de dados de objeto na SQL (ver Capítulo 11). Tentaremos abordar a última versão da SQL ao máximo possível.

SQL é uma linguagem de banco de dados abrangente: tem instruções para definição de dados, consultas e atualizações. Logo, ela é uma DDL *e* uma DML. Além disso, ela tem facilidades para definir visões sobre o banco de dados, para especificar segurança e autorização, para definir restrições de integridade e para especificar controles de transação. Ela também possui regras para embutir instruções SQL em uma linguagem de programação de uso geral, como Java, COBOL ou C/C++.

Os padrões SQL mais recentes (começando com **SQL:1999**) são divididos em uma especificação **núcleo** mais **extensões** especializadas. O núcleo deve ser implementado por todos os fornecedores de SGBDR que sejam compatíveis com SQL. As extensões podem ser implementadas como módulos opcionais a serem adquiridos independentemente para aplicações de banco de dados específicas, como mineração de dados, dados espaciais, dados temporais, data warehousing, processamento analítico on-line (OLAP), dados de multimídia e assim por diante.

Como a SQL é muito importante (e muito grande), dedicamos dois capítulos para explicar seus recursos. Neste capítulo, a Seção 4.1 descreve os comandos de DDL da SQL para criação de esquemas e tabelas, e oferece uma visão geral dos tipos de dados básicos em SQL. A Seção 4.2 apresenta como restrições básicas, chave e integridade referencial, são especificadas. A Seção 4.3 descreve as construções básicas da SQL para especificar recuperação de consultas, e a Seção 4.4 descreve os comandos SQL para inserção, exclusão e alteração de dados.

No Capítulo 5, descreveremos recuperação de consultas SQL mais complexas, bem como comandos ALTER para alterar o esquema. Também descreveremos a instrução CREATE ASSERTION, que permite a especificação de restrições mais gerais no banco de dados. Também apresentamos o conceito de triggers, que será abordado com mais detalhes no Capítulo 26, e descreveremos a facilidade SQL para definir views no banco de dados no Capítulo 5. As views também são conhecidas como *tabelas virtuais* ou *tabelas derivadas* porque apresentam ao usuário o que parecem ser tabelas; porém, a informação nessas tabelas é derivada de tabelas previamente definidas.

A Seção 4.5 lista alguns dos recursos da SQL que serão apresentados em outros capítulos do livro; entre eles estão o controle de transação no Capítulo 21, segurança/autorização no Capítulo 24, bancos de dados ativos (triggers) no Capítulo 26, recursos orientados a objeto no Capítulo 11 e recursos de processamento analítico on-line (OLAP) no Capítulo 29. No final deste capítulo há um resumo. Os capítulos 13 e 14 discutem as diversas técnicas de programação em banco de dados para programação com SQL.

4.1 Definições e tipos de dados em SQL

A SQL usa os termos **tabela**, **linha** e **coluna** para os termos do modelo relacional formal *relação*, *tupla* e *atributo*, respectivamente. Usaremos os termos correspondentes para indicar a mesma coisa. O principal comando SQL para a definição de dados é o CREATE, que pode ser usado para criar esquemas, tabelas (relações) e domínios (bem como outras construções como views, assertions e triggers). Antes de descrevermos a importância das instruções CREATE, vamos discutir os conceitos de esquema e catálogo na Seção 4.1.1 para contextualizar nossa discussão. A Seção 4.1.2 descreve como as tabelas são criadas, e a Seção 4.1.3, os tipos de dados mais importantes disponíveis para especificação de atributo. Como a especificação SQL é muito grande, oferecemos uma descrição dos recursos mais importantes. Outros detalhes poderão ser encontrados em diversos documentos dos padrões SQL (ver bibliografia selecionada ao final do capítulo).

4.1.1 Conceitos de esquema e catálogo em SQL

As primeiras versões da SQL não incluíam o conceito de um esquema de banco de dados relacional; todas as tabelas (relações) eram consideradas parte do mesmo esquema. O conceito de um esquema SQL foi incorporado inicialmente com SQL2 a fim de agrupar tabelas e outras construções que pertencem à mesma aplicação de banco de dados. Um **esquema SQL** é identificado por um **nome de esquema**, e inclui um **identificador de autorização** para indicar o usuário ou conta proprietário do esquema, bem como **descritores** para *cada elemento*. Esses **elementos** incluem tabelas, restrições, views, domínios e outras construções (como concessões — *grants* — de autorização) que descrevem o esquema que é criado por meio da instrução CREATE SCHEMA. Esta pode incluir todas as definições dos elementos do esquema. Como alternativa, o esquema pode receber um identificador de nome e autorização, e os elementos podem ser definidos mais tarde. Por exemplo, a instrução a seguir cria um esque-

[1] Originalmente, a SQL tinha instruções para criar e remover índices nos arquivos que representam as relações, mas estes foram retirados do padrão SQL por algum tempo.

ma chamado EMPRESA, pertencente ao usuário com identificador de autorização 'Jsilva'. Observe que cada instrução em SQL termina com um ponto e vírgula.

CREATE SCHEMA EMPRESA **AUTHORIZATION** 'Jsilva';

Em geral, nem todos os usuários estão autorizados a criar esquemas e elementos do esquema. O privilégio para criar esquemas, tabelas e outras construções deve ser concedido explicitamente às contas de usuário relevantes pelo administrador do sistema ou DBA.

Além do conceito de um esquema, a SQL usa o conceito de um **catálogo** — uma coleção nomeada de esquemas em um ambiente SQL. Um **ambiente** SQL é basicamente uma instalação de um SGBDR compatível com SQL em um sistema de computador.[2] Um catálogo sempre contém um esquema especial, chamado INFORMATION_SCHEMA, que oferece informações sobre todos os esquemas no catálogo e todos os seus descritores de elemento. As restrições de integridade, como a integridade referencial, podem ser definidas entre as relações somente se existirem nos esquemas dentro do mesmo catálogo. Os esquemas dentro do mesmo catálogo também podem compartilhar certos elementos, como definições de domínio.

4.1.2 O comando CREATE TABLE em SQL

O comando **CREATE TABLE** é usado para especificar uma nova relação, dando-lhe um nome e especificando seus atributos e restrições iniciais. Os atributos são especificados primeiro, e cada um deles recebe um nome, um tipo de dado para especificar seu domínio de valores e quaisquer restrições de atributo, como NOT NULL. As restrições de chave, integridade de entidade e integridade referencial podem ser especificadas na instrução CREATE TABLE, depois que os atributos forem declarados, ou acrescentadas depois, usando o comando ALTER TABLE (ver Capítulo 5). A Figura 4.1 mostra exemplos de instruções de definição de dados em SQL para o esquema de banco de dados relacional EMPRESA da Figura 3.7.

Em geral, o esquema SQL em que as relações são declaradas é especificado implicitamente no ambiente em que as instruções CREATE TABLE são executadas. Como alternativa, podemos conectar explicitamente o nome do esquema ao nome da relação, separados por um ponto. Por exemplo, escrevendo

CREATE TABLE EMPRESA.FUNCIONARIO ...

em vez de

CREATE TABLE FUNCIONARIO ...

como na Figura 4.1, podemos explicitamente (ao invés de implicitamente) tornar a tabela FUNCIONARIO parte do esquema EMPRESA.

As relações declaradas por meio das instruções CREATE TABLE são chamadas de **tabelas da base** (ou relações da base); isso significa que a relação e suas tuplas são realmente criadas e armazenadas como um arquivo pelo SGBD. As relações da base são distintas das **relações virtuais**, criadas por meio da instrução CREATE VIEW (ver Capítulo 5), que podem ou não corresponder a um arquivo físico real. Em SQL, os atributos em uma tabela da base são considerados *ordenados na sequência em que são especificados* no comando CREATE TABLE. No entanto, as linhas (tuplas) não são consideradas ordenadas dentro de uma relação.

É importante observar que, na Figura 4.1, existem algumas *chaves estrangeiras que podem causar erros*, pois são especificadas por referências circulares ou porque dizem respeito a uma tabela que ainda não foi criada. Por exemplo, a chave estrangeira Cpf_supervisor na tabela FUNCIONARIO é uma referência circular, pois se refere à própria tabela. A chave estrangeira Dnr na tabela FUNCIONARIO se refere à tabela DEPARTAMENTO, que ainda não foi criada. Para lidar com esse tipo de problema, essas restrições podem ser omitidas inicialmente do comando CREATE TABLE, e depois acrescentadas usando a instrução ALTER TABLE (ver Capítulo 5). Apresentamos todas as chaves estrangeiras na Figura 4.1 para mostrar o esquema EMPRESA completo em um só lugar.

4.1.3 Tipos de dados de atributo e domínios em SQL

Os **tipos de dados** básicos disponíveis para atributos são numérico, cadeia ou sequência de caracteres, cadeia ou sequência de bits, booleano, data e hora.

- Os tipos de dados **numerico** incluem números inteiros de vários tamanhos (INTEGER ou INT e SMALLINT) e números de ponto flutuante (reais) de várias precisões (FLOAT ou REAL e DOUBLE PRECISION). O formato dos números pode ser declarado usando DECIMAL(*i*, *j*) — ou DEC(*i*, *j*) ou NUMERIC(*i*, *j*) — onde *i*, a *precisão*, é o número total de dígitos decimais e *j*, a *escala*, é o número de dígitos após o ponto decimal. O valor padrão para a escala é zero, e para a precisão, é definido pela implementação.
- Tipos de dados de **cadeia de caracteres** são de tamanho fixo — CHAR(*n*) ou CHARACTER(*n*), onde *n* é o número de caracteres — ou de tamanho variável — VARCHAR(*n*) ou CHAR

[2] A SQL também indui o conceito de um grupo (*cluster*) de catálogos dentro de um ambiente.

```
CREATE TABLE FUNCIONARIO
        (Pnome              VARCHAR(15)          NOT NULL,
        Minicial            CHAR,
        Unome               VARCHAR(15)          NOT NULL,
        Cpf                 CHAR(11),            NOT NULL,
        Datanasc            DATE,
        Endereço            VARCHAR(30),
        Sexo                CHAR,
        Salario             DECIMAL(10,2),
        Cpf_supervisor      CHAR(11),            NOT NULL,
        Dnr                 INT
    PRIMARY KEY (Cpf),
    FOREIGN KEY (Cpf_supervisor) REFERENCES FUNCIONARIO(Cpf),
    FOREIGN KEY (Dnr) REFERENCES DEPARTAMENTO(Dnumero) );
CREATE TABLE DEPARTAMENTO
        (Dnome              VARCHAR(15)          NOT NULL,
        Dnumero             INT                  NOT NULL,
        Cpf_gerente         CHAR(11),            NOT NULL,
        Data_inicio_gerente DATE,
    PRIMARY KEY (Dnumero),
    UNIQUE (Dnome),
    FOREIGN KEY (Cpf_gerente) REFERENCES FUNCIONARIO(Cpf) );
CREATE TABLE LOCALIZACAO_DEP
        (Dnumero            INT                  NOT NULL,
        Dlocal              VARCHAR(15)          NOT NULL,
    PRIMARY KEY (Dnumero, Dlocal),
    FOREIGN KEY (Dnumero) REFERENCES DEPARTAMENTO(Dnumero) );
CREATE TABLE PROJETO
        (Projnome           VARCHAR(15)          NOT NULL,
        Projnumero          INT                  NOT NULL,
        Projlocal           VARCHAR(15),
        Dnum                INT                  NOT NULL,
    PRIMARY KEY (Projnumero),
    UNIQUE (Projnome),
    FOREIGN KEY (Dnum) REFERENCES DEPARTAMENTO(Dnumero) );
CREATE TABLE TRABALHA_EM
        (Fcpf               CHAR(9)              NOT NULL,
        Pnr                 INT                  NOT NULL,
        Horas               DECIMAL(3,1)         NOT NULL,
    PRIMARY KEY (Fcpf, Pnr),
    FOREIGN KEY (Fcpf) REFERENCES FUNCIONARIO(Cpf),
    FOREIGN KEY (Pnr) REFERENCES PROJETO(Projnumero) );
CREATE TABLE DEPENDENTE
        (Fcpf               CHAR(11),            NOT NULL,
        Nome_dependente     VARCHAR(15)          NOT NULL,
        Sexo                CHAR,
        Datanasc            DATE,
        Parentesco          VARCHAR(8),
    PRIMARY KEY (Fcpf, Nome_dependente),
    FOREIGN KEY (Fcpf) REFERENCES FUNCIONARIO(Cpf) );
```

Figura 4.1
Instruções CREATE TABLE da SQL para definição do esquema EMPRESA da Figura 3.7.

VARYING(*n*) ou CHARACTER VARYING(*n*), onde *n* é o número máximo de caracteres. Ao especificar um valor literal de cadeia de caracteres, ele é colocado entre aspas simples (apóstrofos), e é *case sensitive* (diferencia maiúsculas de minúsculas).[3] Para cadeias de caracteres de tamanho fixo, uma cadeia mais curta é preenchida com caracteres em branco à direita. Por exemplo, se o valor 'Silva' for para um atributo do tipo CHAR(10), ele é preenchido com cinco caracteres em branco para se tornar 'Silva ', se necessário. Os espaços preenchidos geralmente são ignorados quando as cadeias são comparadas. Para fins de comparação, as cadeias de caracteres são consideradas ordenadas em ordem alfabética (ou lexicográfica); se uma cadeia *str1* aparecer antes de outra cadeia *str2* em ordem alfabética, então *str1* é considerada menor que *str2*.[4] Há também um operador de concatenação indicado por || (barra vertical dupla), que pode concatenar duas cadeias de caracteres em SQL. Por exemplo, 'abc' || 'XYZ' resulta em uma única cadeia 'abcXYZ'. Outro tipo de dado de cadeia de caracteres de tamanho variável, chamado CHARACTER LARGE OBJECT ou CLOB, também está disponível para especificar colunas que possuem grandes valores de texto, como documentos. O tamanho máximo de CLOB pode ser especificado em kilobytes (K), megabytes (M) ou gigabytes (G). Por exemplo, CLOB(20M) especifica um tamanho máximo de 20 megabytes.

- Tipos de dados de **cadeia de bits** podem ser de tamanho fixo *n* — BIT(*n*) — ou de tamanho variável — BIT VARYING(*n*), onde *n* é o número máximo de bits. O valor padrão para *n*, o tamanho de uma cadeia de caracteres ou cadeia de bits, é 1. Os literais de cadeia de bits literais são colocados entre apóstrofos, mas precedidos por um B para distingui-los das cadeias de caracteres; por exemplo, B'10101'.[5] Outro tipo de dados de cadeia de bits de tamanho variável, chamado BINARY LARGE OBJECT ou BLOB, também está disponível para especificar colunas que possuem grandes valores binários, como imagens. Assim como para CLOB, o tamanho máximo de um BLOB pode ser especificado em kilobits (K), megabits

[3] Isso não acontece com palavras-chave da SQL, como CREATE ou CHAR. Com as palavras-chave, a SQL é *case insensitive*, significando que ela trata letras maiúsculas e minúsculas como equivalentes nessas palavras.

[4] Para caracteres não alfabéticos, existe uma ordem definida.

[5] Cadeias de bits cujo tamanho é um múltiplo de 4 podem ser especificadas em notação *hexadecimal*, em que a cadeia literal é precedida por X e cada caractere hexadecimal representa 4 bits.

(M) ou gigabits (G). Por exemplo, BLOB(30G) especifica um tamanho máximo de 30 gigabits.

- Um tipo de dado **booleano** tem os valores tradicionais TRUE (verdadeiro) ou FALSE (falso). Em SQL, devido à presença de valores NULL (nulos), uma lógica de três valores é utilizada, de modo que um terceiro valor possível para um tipo de dado booleano é UNKNOWN (indefinido). Discutiremos a necessidade de UNKNOWN e a lógica de três valores no Capítulo 5.
- O tipo de dados **DATE** possui dez posições, e seus componentes são DAY (dia), MONTH (mês) e YEAR (ano) na forma DD-MM-YYYY. O tipo de dado TIME (tempo) tem pelo menos oito posições, com os componentes HOUR (hora), MINUTE (minuto) e SECOND (segundo) na forma HH:MM:SS. Somente datas e horas válidas devem ser permitidas pela implementação SQL. Isso implica que os meses devem estar entre 1 e 12 e as datas devem estar entre 1 e 31; além disso, uma data deve ser uma data válida para o mês correspondente. A comparação < (menor que) pode ser usada com datas ou horas — uma data *anterior* é considerada menor que uma data posterior, e da mesma forma com o tempo. Os valores literais são representados por cadeias com apóstrofos precedidos pela palavra-chave DATE ou TIME; por exemplo, DATE '27-09-2008' ou TIME '09:12:47'. Além disso, um tipo de dado TIME(*i*), onde *i* é chamado de *precisão em segundos fracionários de tempo*, especifica *i* + 1 posições adicionais para TIME — uma posição para um caractere separador de período adicional (.), e *i* posições para especificar as frações de um segundo. Um tipo de dados TIME WITH TIME ZONE inclui seis posições adicionais para especificar o *deslocamento* com base no fuso horário universal padrão, que está na faixa de +13:00 a -12:59 em unidades de HOURS:MINUTES. Se WITH TIME ZONE não for incluído, o valor padrão é o fuso horário local para a sessão SQL.

Alguns tipos de dados adicionais são discutidos a seguir. A lista apresentada aqui não está completa; diferentes implementações acrescentaram mais tipos de dados à SQL.

- Um tipo de dado **timestamp** (TIMESTAMP) inclui os campos DATE e TIME, mais um mínimo de seis posições para frações decimais de segundos e um qualificador opcional WITH TIME ZONE. Valores literais são representados por cadeias entre apóstrofos precedidos pela palavra-chave TIMESTAMP, com um espaço em branco entre data e hora; por exemplo, TIMESTAMP '27-09-2008 09:12:47.648302'.
- Outro tipo de dado relacionado a DATE, TIME e TIMESTAMP é o INTERVAL. Este especifica um **intervalo** — um *valor relativo* que pode ser usado para incrementar ou decrementar um valor absoluto de uma data, hora ou timestamp. Os intervalos são qualificados para serem de YEAR/MONTH ou de DAY/TIME.

O formato de DATE, TIME e TIMESTAMP pode ser considerado um tipo especial de cadeia. Logo, eles geralmente podem ser usados em comparações de cadeias sendo **convertidos** (**cast**) em cadeias equivalentes.

É possível especificar o tipo de dado de cada atributo diretamente, como na Figura 4.1; como alternativa, um domínio pode ser declarado e seu nome, usado com a especificação de atributo. Isso torna mais fácil mudar o tipo de dado para um domínio que é usado por diversos atributos em um esquema, e melhora a legibilidade do esquema. Por exemplo, podemos criar um domínio TIPO_CPF com a seguinte instrução:

```
CREATE DOMAIN TIPO_CPF AS CHAR(11);
```

Podemos usar TIPO_CPF no lugar de CHAR(11) na Figura 4.1 para os atributos Cpf e Cpf_supervisor de FUNCIONARIO, Cpf_gerente de DEPARTAMENTO, Fcpf de TRABALHA_EM e Fcpf de DEPENDENTE. Um domínio também pode ter uma especificação DEFAULT, padrão opcional que nos discutiremos mais adiante para os atributos. Observe que os domínios podem não estar disponíveis em algumas implementações da SQL.

4.2 Especificando restrições em SQL

Esta seção descreve as restrições básicas que podem ser especificadas em SQL como parte da criação de tabela. Estas incluem restrições de chave e integridade referencial, restrições sobre domínios de atributos e NULLs e restrições sobre tuplas individuais dentro de uma relação. Discutiremos a especificação de restrições mais gerais, chamadas asserções (ou *assertions*) no Capítulo 5.

4.2.1 Especificando restrições de atributo e defaults de atributo

Como a SQL permite NULLs como valores de atributo, uma *restrição* NOT NULL pode ser especificada se o valor NULL não for permitido para determinado atributo. Isso sempre é especificado de maneira implícita para os atributos que fazem parte da *chave primária* de cada relação, mas pode ser especificado para quaisquer outros atributos cujos valores não podem ser NULL, como mostra a Figura 4.1.

```
CREATE TABLE FUNCIONARIO
    ( . . . ,
    Dnr             INT             NOT NULL     DEFAULT 1,
    CONSTRAINT CHPFUNC
        PRIMARY KEY (Cpf),
    CONSTRAINT CHESUPERFUNC
        FOREIGN KEY (Cpf_supervisor) REFERENCES FUNCIONARIO(Cpf)
        ON DELETE SET NULL    ON UPDATE CASCADE,
    CONSTRAINT CHEDEPFUNC
    FOREIGN KEY (Dnr) REFERENCES DEPARTAMENTO(Dnumero)
            ON DELETE SET DEFAULT ON UPDATE CASCADE);
CREATE TABLE DEPARTAMENTO
    ( . . . ,
    Cpf_gerente     CHAR(11)    NOT NULL     DEFAULT '88866555576',
    . . . ,
    CONSTRAINT CHPDEP
        PRIMARY KEY (Dnumero),
    CONSTRAINT CHSDEP
        UNIQUE (Dnome),
    CONSTRAINT CHEGERDEP
    FOREIGN KEY (Cpf_gerente) REFERENCES FUNCIONARIO(Cpf)
        ON DELETE SET DEFAULT ON UPDATE CASCADE);
CREATE TABLE LOCALIZACAO_DEP
    ( . . . ,
    PRIMARY KEY (Dnumero, Dlocal),
    FOREIGN KEY (Dnumero) REFERENCES DEPARTAMENTO(Dnumero)
        ON DELETE CASCADE    ON UPDATE CASCADE);
```

Figura 4.2
Exemplo ilustrando como os valores de atributo default e as ações disparadas por integridade referencial são especificadas em SQL.

Também é possível definir um *valor padrão* para um atributo anexando a cláusula **DEFAULT** <valor> a uma definição de atributo. O valor padrão está incluído em qualquer nova tupla se um valor explícito não for fornecido para esse atributo. A Figura 4.2 ilustra um exemplo de especificação de um gerente default para um novo departamento e um departamento default para um novo funcionário. Senenhuma cláusula default for especificada, o *valor padrão* será NULL para atributos *que não possuem* a restrição NOT NULL.

Outro tipo de restrição pode limitar valores de atributo ou domínio usando a cláusula **CHECK** (verificação) após uma definição de atributo ou domínio.[6] Por exemplo, suponha que números de departamento sejam restritos a números inteiros entre 1 e 20; então, podemos mudar a declaração de atributo de Dnumero na tabela DEPARTAMENTO (ver Figura 4.1) para o seguinte:

```
Dnumero INT NOT NULL CHECK (Dnumero > 0 AND
Dnumero < 21);
```

A cláusula CHECK também pode ser usada em conjunto com a instrução CREATE DOMAIN. Por exemplo, podemos escrever a seguinte instrução:

```
CREATE DOMAIN D_NUM AS INTEGER
CHECK (D_NUM > 0 AND D_NUM < 21);
```

Depois, podemos usar o domínio criado D_NUM como o tipo de atributo para todos os atributos que se referem aos números de departamento na Figura 4.1, como Dnumero de DEPARTAMENTO, Dnum de PROJETO, Dnr de FUNCIONARIO, e assim por diante.

4.2.2 Especificando restrições de chave e integridade referencial

Como chaves e restrições de integridade referencial são muito importantes, existem cláusulas especiais dentro da instrução CREATE TABLE para especificá-las. Alguns exemplos para ilustrar a especificação de chaves e integridade referencial aparecem na Figura 4.1.[7] A cláusula PRIMARY KEY especifica um ou mais atributos que compõem a chave primária de uma relação. Se uma chave primária tiver um *único* atributo, a cláusula pode acompanhar o atributo diretamente. Por exemplo, a chave primária de DEPARTAMENTO pode ser especificada da seguinte forma (em vez do modo como ela é especificada na Figura 4.1):

```
Dnumero INT PRIMARY KEY;
```

A cláusula **UNIQUE** especifica chaves alternativas (secundárias), conforme ilustramos nas declarações da tabela DEPARTAMENTO e PROJETO na Figura 4.1. A cláusula **UNIQUE** também pode ser especificada diretamente para uma chave secundária se esta for um único atributo, como no exemplo a seguir:

```
Dnome VARCHAR(15) UNIQUE;
```

A integridade referencial é especificada por meio da cláusula **FOREIGN KEY** (chave estrangeira), como mostra a Figura 4.1. Conforme discutimos na Seção 3.2.4, uma restrição de integridade referencial pode ser violada quando tuplas são inseridas ou excluídas, ou quando um valor de atributo de chave estrangeira ou chave primária é modificado. A ação default que a SQL toma para uma violação de integridade é **rejeitar** a operação de atualização que causará uma violação, o que é conhecido como opção RESTRICT. Porém, o projetista do esquema pode especificar uma ação alternativa

[6] A cláusula CHECK também pode ser usada para outras finalidades, conforme veremos.
[7] As restrições de chave e integridade referencial não estavam induídas nas primeiras versões da SQL. Em algumas implementações iniciais, as chaves eram especificadas implicitamente no nível interno por meio do comando CREATE INDEX

para ser tomada conectando uma cláusula de **ação de disparo referencial** a qualquer restrição de chave estrangeira. As opções incluem SET NULL, CASCADE e SET DEFAULT. Uma opção deve ser qualificada com ON DELETE ou ON UPDATE. Ilustramos isso com os exemplos mostrados na Figura 4.2. Aqui, o projetista de banco de dados escolhe ON DELETE SET NULL e ON UPDATE CASCADE para a chave estrangeira Cpf_supervisor de FUNCIONARIO. Isso significa que, se a tupla para um *funcionário supervisor é excluída*, o valor de Cpf_supervisor será automaticamente definido como NULL para todas as tuplas de funcionários que estavam referenciando a tupla do funcionário excluído. Por sua vez, se o valor de Cpf para um funcionário supervisor é *atualizado* (digamos, porque foi inserido incorretamente), o novo valor será *propagado em cascata* de Cpf_supervisor para todas as tuplas de funcionário que referencia a tupla de funcionário atualizada. [8]

Em geral, a ação tomada pelo SGBD para SET NULL ou SET DEFAULT é a mesma para ON DELETE e ON UPDATE: o valor dos atributos de referência afetados é mudado para NULL em caso de SET NULL e para o valor padrão especificado do atributo de referência em caso de SET DEFAULT. A ação para CASCADE ON DELETE é excluir todas as tuplas de referência, enquanto a ação para CASCADE ON UPDATE é mudar o valor do(s) atributo(s) de chave estrangeira de referência para o (novo) valor de chave primária atualizado para todas as tuplas de referência. É responsabilidade do projetista de banco de dados escolher a ação apropriada e especificá-la no esquema do banco de dados. Como uma regra geral, a opção CASCADE é adequada para relações de 'relacionamento' (ver Seção 9.1), como TRABALHA_EM; para relações que representam atributos multivalorados, como LOCALIZACAO_DEP; e para relações que representam tipos de entidades fracas, como DEPENDENTE.

4.2.3 Dando nomes a restrições

A Figura 4.2 também ilustra como uma restrição pode receber um **nome de restrição**, seguindo a palavra-chave **CONSTRAINT**. Os nomes de todas as restrições dentro de um esquema em particular precisam ser exclusivos. Um nome de restrição é usado para identificar uma restrição em particular caso ela deva ser removida mais tarde e substituída por outra, conforme discutiremos no Capítulo 5. Dar nomes a restrições é algo opcional.

4.2.4 Especificando restrições sobre tuplas usando CHECK

Além das restrições de chave e integridade referencial, que são especificadas por palavras-chave especiais, outras *restrições de tabela* podem ser especificadas por meio de cláusula adicional CHECK ao final de uma instrução CREATE TABLE. Estas podem ser chamadas de restrições **baseadas em tupla**, pois se aplicam a cada tupla *individualmente* e são verificadas sempre que uma tupla é inserida ou modificada.

Por exemplo, suponha que a tabela DEPARTAMENTO da Figura 4.1 tivesse um atributo adicional Dep_data_criacao, que armazena a data em que o departamento foi criado. Então, poderíamos acrescentar a seguinte cláusula CHECK ao final da instrução CREATE TABLE para a tabela DEPARTAMENTO para garantir que a data de início de um gerente seja posterior à data de criação do departamento.

```
CHECK (Dep_data_criacao <= Data_inicio_gerente);
```

A cláusula CHECK também pode ser usada para especificar restrições mais gerais usando a instrução CREATE ASSERTION da SQL. Discutiremos isso no Capítulo 5 porque exige o entendimento completo sobre consultas, que são discutidas nas seções 4.3 e 5.1.

4.3

A SQL tem uma instrução básica para recuperar informações de um banco de dados: a instrução **SELECT**. A instrução SELECT *não é o mesmo que* a operação SELECT da álgebra relacional, que discutiremos no Capítulo 6. Existem muitas opções e tipos de instrução SELECT em SQL, de modo que introduziremos seus recursos gradualmente. Usaremos consultas de exemplo especificadas no esquema da Figura 3.5 e vamos nos referir ao exemplo estado de banco de dados mostrado na Figura 3.6 para mostrar os resultados de alguns exemplos de consultas. Nesta seção, apresentamos os recursos da SQL para *consultas de recuperação simples*. Os recursos da SQL para especificar consultas de recuperação mais complexas serão apresentados na Seção 5.1.

Antes de prosseguir, devemos apontar uma *distinção importante* entre a SQL e o modelo relacional formal discutido no Capítulo 3: a SQL permite que uma tabela (relação) tenha duas ou mais tuplas

[8] Observe que a chave estrangeira Cpf_supervisor na tabela FUNCIONARIO é uma referência circular e, portanto, pode ter de ser incluída mais tarde como uma restrição nomeada, usando a instrução ALTER TABLE, conforme discutimos no final da Seção 4.1.2.

que são idênticas em todos os seus valores de atributo. Assim, em geral, uma tabela SQL não é um *conjunto de tuplas*, pois um conjunto não permite dois membros idênticos; em vez disso, ela é um **multiconjunto** (também chamado de *bag*) de tuplas. Algumas relações SQL são *restritas a serem conjuntos* porque uma restrição de chave foi declarada ou porque a opção DISTINCT foi usada com a instrução SELECT (descrita mais adiante nesta seção). Precisamos estar cientes dessa distinção à medida que discutirmos os exemplos.

4.3.1 A estrutura SELECT-FROM-WHERE das consultas SQL básicas

As consultas em SQL podem ser muito complexas. Começaremos com consultas simples, e depois passaremos para as mais complexas de maneira passo a passo. A forma básica do comando SELECT, às vezes chamada de **mapeamento** ou **bloco select-from-where**, é composta pelas três cláusulas SELECT, FROM e WHERE, e tem a seguinte forma: [9]

```
SELECT <lista atributos>
FROM   <lista tabelas>
WHERE  <condição>;
```

onde

- <lista atributos> é uma lista de nomes de atributo cujos valores devem ser recuperados pela consulta.
- <lista tabelas> é uma lista dos nomes de relação exigidos para processar a consulta.
- <condição> é uma expressão condicional (booleana) que identifica as tuplas a serem recuperadas pela consulta.

Em SQL, os operadores básicos de comparação lógicos para comparar valores de atributo entre si e com constantes literais são =, <, <=, >, >= e <>. Estes correspondem aos operadores da álgebra relacional =, <, ≤, >, ≥ e ≠, respectivamente, e aos operadores da linguagem de programação C/C++ =, <, <=, >, >= e !=. A principal diferença sintática é o operador *diferente*. A SQL possui operadores de comparação adicional que apresentaremos de maneira gradual.

Ilustramos a instrução SELECT básica em SQL com alguns exemplos de consultas. As consultas são rotuladas aqui com os mesmos números de consulta usados no Capítulo 6 para facilitar a referência cruzada.

Consulta 0. Recuperar a data de nascimento e o endereço do(s) funcionário(s) cujo nome seja 'João B. Silva'.

C0:
```
SELECT  Datanasc, Endereco
FROM    FUNCIONARIO
WHERE   Pnome='João' AND Minicial='B' AND
        Unome='Silva';
```

Esta consulta envolve apenas a relação FUNCIONARIO listada na cláusula FROM. A consulta *seleciona* as tuplas FUNCIONARIO individuais que satisfazem a condição da cláusula WHERE, depois *projeta* o resultado nos atributos Datanasc e Endereco listados na cláusula SELECT.

A cláusula SELECT da SQL especifica os atributos cujos valores devem ser recuperados, que são chamados **atributos de projeção**, e a cláusula WHERE especifica a condição booleana que deve ser verdadeira para qualquer tupla recuperada, que é conhecida como **condição de seleção**. A Figura 4.3(a) mostra o resultado da consulta C0 sobre o banco de dados da Figura 3.6.

Podemos pensar em uma **variável de tupla** implícita (ou *iterator*) na consulta SQL variando ou *repetindo* sobre cada tupla individual na tabela FUNCIONARIO e avaliando a condição na cláusula WHERE. Somente as tuplas que satisfazem a condição — ou seja, aquelas tuplas para as quais a condição é avaliada como TRUE após substituir seus valores de atributo correspondentes — são selecionadas.

Consulta 1. Recuperar o nome e endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

C1:
```
SELECT  Pnome, Unome, Endereco
FROM    FUNCIONARIO, DEPARTAMENTO
WHERE   Dnome='Pesquisa' AND Dnumero=Dnr;
```

Na cláusula WHERE de C1, a condição Dnome = 'Pesquisa' é uma **condição de seleção** que escolhe a tupla de interesse em particular na tabela DEPARTAMENTO, pois Dnome é um atributo de DEPARTAMENTO. A condição Dnumero = Dnr é chamada **condição de junção**, pois combina duas tuplas: uma de DEPARTAMENTO e uma de FUNCIONARIO, sempre que o valor de Dnumero em DEPARTAMENTO é igual ao valor de Dnr em FUNCIONARIO. O resultado da consulta C1 é mostrado na Figura 4.3(b). Em geral, qualquer número de condições de seleção e junção pode ser especificado em uma única consulta SQL.

[9] As cláusulas SELECT e FROM são necessárias em todas as consultas SQL. O WHERE é opcional (ver Seção 4.3.3).

(a)

Datanasc	Endereco
09-01-1965	Rua das Flores, 751, São Paulo, SP

(b)

Pnome	Unome	Endereco
João	Silva	Rua das Flores, 751, São Paulo, SP
Fernando	Wong	Rua da Lapa, 34, São Paulo, SP
Ronaldo	Lima	Rua Rebouças, 65, Piracicaba, SP
Joice	Leite	Av. Lucas Obes, 74, São Paulo, SP

(c)

Projnumero	Dnum	Unome	Endereco	Datanasc
10	4	Souza	Av. Arturde Lima, 54, Santo André, SP	20-06-1941
30	4	Souza	Av. Arturde Lima, 54, Santo André, SP	20-06-1941

(d)

F.Pnome	F.Unome	S.Pnome	S.Unome
João	Silva	Fernando	Wong
Fernando	Wong	Jorge	Brito
Alice	Zelaya	Jennifer	Souza
Jennifer	Souza	Jorge	Brito
Ronaldo	Lima	Fernando	Wong
Joice	Leite	Fernando	Wong
André	Pereira	Jennifer	Souza

(e)

F.Pnome
12345678966
33344555587
99988777767
98765432168
66688444476
45345345376
98798798733
88866555576

(f)

Cpf	Dnome
12345678966	Pesquisa
33344555587	Pesquisa
99988777767	Pesquisa
98765432168	Pesquisa
66688444476	Pesquisa
45345345376	Pesquisa
98798798733	Pesquisa
88866555576	Pesquisa
12345678966	Administração
33344555587	Administração
99988777767	Administração
98765432168	Administração
66688444476	Administração
45345345376	Administração
98798798733	Administração
88866555576	Administração
12345678966	Matriz
33344555587	Matriz
99988777767	Matriz
98765432168	Matriz
66688444476	Matriz
45345345376	Matriz
98798798733	Matriz
88866555576	Matriz

(g)

Pnome	Minicial	Unome	Cpf	Datanasc	Endereco	Sexo	Salario	Cpf_supervisor	Dnr
João	B	Silva	12345678966	09-01-1965	Rua das Flores, 751, São Paulo, SP	M	30.000	33344555587	5
Fernando	T	Wong	33344555587	08-12-1955	Rua da Lapa, 34, São Paulo, SP	M	40.000	88866555576	5
Ronaldo	K	Lima	66688444476	15-09-1962	Rua Rebouças, 65, Piracicaba, SP	M	38.000	33344555587	5
Joice	A	Leite	45345345376	31-07-1972	Av. Lucas Obes, 74, São Paulo, SP	F	25.000	33344555587	5

Figura 4.3

Resultados das consultas SQL quando aplicados ao estado do banco de dados EMPRESA mostrado na Figura 3.6. (a) C0. (b) C1. (c) C2. (d) C8. (e) C9. (f) C10. (g) C1C.

Uma consulta que envolve apenas condições de seleção e junção mais atributos de projeção é conhecida como uma consulta **seleção-projeção-junção**. O próximo exemplo é uma consulta seleção-projeção-junção com *duas* condições de junção.

Consulta 2. Para cada projeto localizado em 'Mauá', liste o número do projeto, o número do departamento que o controla e o sobrenome, endereço e data de nascimento do gerente do departamento.

```
C2: SELECT Projnumero, Dnum, Unome, Endereco,
           Datanasc
    FROM   PROJETO, DEPARTAMENTO,
           FUNCIONARIO
    WHERE  Dnum=Dnumero AND
           Cpf_gerente=Cpf AND
           Projlocal='Mauá';
```

A condição de junção Dnum = Dnumero relaciona uma tupla de projeto a sua tupla de departamento que o controla, enquanto a condição de junção Cpf_gerente = Cpf relaciona a tupla do departamento que o controla à tupla de funcionário que gerencia esse departamento. Cada tupla no resultado será uma *combinação* de um projeto, um departamento e um funcionário, que satisfaz as condições de junção. Os atributos de projeção são usados para escolher os atributos a serem exibidos. O resultado da consulta C2 aparece na Figura 4.3(c).

4.3.2 Nomes de atributos ambíguos, apelido, renomeação e variáveis de tupla

Em SQL, o mesmo nome pode ser usado para dois (ou mais) atributos, desde que estes estejam em *relações diferentes*. Se isso acontecer, e uma consulta em múltiplas tabelas se referir a dois ou mais atributos com o mesmo nome, é *preciso* **qualificar** o nome do atributo com o nome da relação, para evitar ambiguidade. Isso é feito *prefixando* o nome da relação ao nome do atributo e separando os dois por um ponto. Para ilustrar isso, suponha que, nas figuras 3.5 e 3.6, os atributos Dnr e Unome da relação FUNCIONARIO fossem chamados de Dnumero e Nome, e o atributo Dnome de DEPARTAMENTO também fosse chamado Nome. Então, para evitar ambiguidade, a consulta C1 seria reformulada como mostramos em C1A. Devemos prefixar os nomes de atributo Nome e Dnumero em C1A para especificar a quais estamos nos referindo, pois os mesmos nomes de atributo são usados nas duas relações:

```
C1A: SELECT Pnome, FUNCIONARIO.Nome,
            Endereco
     FROM   FUNCIONARIO, DEPARTAMENTO
     WHERE  DEPARTAMENTO.Nome='Pesquisa' AND
            DEPARTAMENTO.
            Dnumero=FUNCIONARIO.Dnumero;
```

Nomes de atributo totalmente qualificados podem ser usados por clareza mesmo que não haja ambiguidade nos nomes de atributo. C1 aparece dessa maneira como C1' a seguir. Também podemos criar um *apelido* para cada nome de tabela, para evitar a digitação repetida de nomes de tabela longos (ver C8, a seguir).

```
C1': SELECT FUNCIONARIO.Pnome, FUNCIONARIO.
            UNome FUNCIONARIO.Endereco,
     FROM   FUNCIONARIO, DEPARTAMENTO
     WHERE  DEPARTAMENTO.DNome='Pesquisa' AND
            DEPARTAMENTO.Dnumero=
            FUNCIONARIO.Dnr;
```

A ambiguidade dos nomes de atributo também surge no caso de consultas que se referem à mesma relação duas vezes, como no exemplo a seguir.

Consulta 8. Para cada funcionário, recupere o primeiro e o último nome do funcionário e o primeiro e o último nome de seu supervisor imediato.

```
C8: SELECT F.Pnome, F.Unome, S.Pnome, S.Unome
    FROM   FUNCIONARIO AS F, FUNCIONARIO
           AS S
    WHERE  F.Cpf_supervisor=S.Cpf;
```

Neste caso, precisamos declarar nomes de relação alternativos F e S, chamados **apelidos** ou **variáveis de tupla**, para a relação FUNCIONARIO. Um apelido pode vir após a palavra-chave AS, como mostramos em C8, ou pode vir diretamente após o nome da relação — por exemplo, escrevendo FUNCIONARIO F, FUNCIONARIO S na cláusula FROM de C8. Também é possível **renomear** os atributos da relação dentro da consulta em SQL, dando-lhe apelidos. Por exemplo, se escrevermos

FUNCIONARIO **AS** F(Pn, Mi, Un, Cpf, Dn, End, Sexo, Sal, Scpf, Dnr)

na cláusula FROM, Pn torna-se um apelido para Pnome, Mi para Minicial, Un para Unome, e assim por diante.

Em C8, podemos pensar em F e S como duas *cópias diferentes* da relação FUNCIONARIO; a primei-

ra, F, representa funcionários no papel de supervisionados ou subordinados; a segunda,S, representa os funcionários no papel de supervisores. Agora, podemos juntar as duas cópias. Naturalmente, na realidade existe *apenas uma* relação FUNCIONARIO, e a condição de junção serve para juntar a própria relação, combinando as tuplas que satisfazem a condição de junção F.Cpf_supervisor = S.Cpf. Observe que este é um exemplo de uma consulta recursiva de um nível, conforme discutiremos na Seção 6.4.2. Nas versões anteriores da SQL, não era possível especificar uma consulta recursiva geral, com um número desconhecido de níveis, em uma única instrução SQL. Uma construção para especificar consultas recursivas foi incorporada na SQL:1999 (ver Capítulo 5).

O resultado da consultaC8 aparece na Figura 4.3(d). Sempre que um ou mais apelidos são dados a uma relação, podemos usar esses nomes para representar diferentes referências a essa mesma relação. Isso permite múltiplas referências à mesma relação dentro de uma consulta.

Podemos usar esse mecanismo de nomeação de apelidos em qualquer consulta SQL para especificar variáveis de tupla para cada tabela na cláusula WHERE, não importando se a mesma relação precisa ser referenciada mais de uma vez. De fato, essa prática é recomendada, pois resulta em consultas mais fáceis de compreender. Por exemplo, poderíamos especificar a consulta C1 como em C1B:

```
C1B: SELECT F.Pnome, F.Unome, F.Endereco
     FROM   FUNCIONARIO F, DEPARTAMENTO D
     WHERE  D.DNome='Pesquisa' AND
            D.Dnumero=F.Dnr;
```

4.3.3 Cláusula WHERE não especificada e uso do asterisco

Vamos discutir aqui mais dois recursos da SQL. A falta de uma cláusula WHERE indica que não há condição sobre a seleção de tuplas; logo, *todas as tuplas* da relação especificada na cláusula FROM se qualificam e são selecionadas para o resultado da consulta. Se mais de uma relação for especificada na cláusula FROM e não houver uma cláusula WHERE, então o PRODUTO CARTESIANO — *todas as combinações de tuplas possíveis*— dessas relações será selecionado. Por exemplo, a Consulta 9 seleciona todos os Cpfs de FUNCIONARIO (Figura 4.3(e)) e a Consulta 10 seleciona todas as combinações de um Cpf de FUNCIONARIO e um Dnome de DEPARTAMENTO, independentemente de o funcionário trabalhar ou não para o departamento (Figura 4.3(f)).

Consultas 9 e 10. Selecionar todos osCpfs de FUNCIONARIO(C9) e todas as combinações deCpf de FUNCIONARIO e Dnome de DEPARTAMENTO (C10) no banco de dados.

```
C9:  SELECT Cpf
     FROM   FUNCIONARIO;

C10: SELECT Cpf, Dnome
     FROM   FUNCIONARIO, DEPARTAMENTO;
```

É extremamente importante especificar cada condição de junção WHERE ausência dessa condição, o resultado poderá ser relações incorretas e muito grandes. Observe que C10 é semelhante a uma operação de PRODUTO CARTESIANO seguida por uma operação PROJECAO na álgebra relacional (ver Capítulo 6). Se especificarmos todos os atributos de FUNCIONARIO e DEPARTAMENTO em C10, obteremos o PRODUTO CARTESIANO real (exceto pela eliminação de duplicatas, se houver).

Para recuperar todos os valores de atributo das tuplas selecionadas, não precisamos listar os nomes de atributo explicitamente em SQL; basta especificar um *asterisco* (*), que significa *todos os atributos*. Por exemplo, a consulta C1C recupera todos os valores de atributo de qualquer FUNCIONARIO que trabalha no DEPARTAMENTO número 5 (Figura 4.3(g)), C1D recupera todos os atributos de FUNCIONARIO e os atributos de DEPARTAMENTO em que ele ou ela trabalha para todo funcionário no departamento 'Pesquisa', e C10A especifica o PRODUTO CARTESIANO das relações FUNCIONARIO e DEPARTAMENTO.

```
C1C:  SELECT *
      FROM   FUNCIONARIO
      WHERE  Dnr=5;

C1D:  SELECT *
      FROM   FUNCIONARIO, DEPARTAMENTO
      WHERE  Dnome='Pesquisa' AND Dnr=Dnumero;

C10A: SELECT *
      FROM   FUNCIONARIO, DEPARTAMENTO;
```

4.3.4 Tabelas como conjuntos em SQL

Conforme já dissemos, a SQL normalmente trata uma tabela não como um conjunto, mas como um **multiconjunto**; *tuplas duplicadas podem aparecer mais de uma vez* em uma tabela, e no resultado de uma consulta. A SQL não elimina automaticamente tuplas duplicadas nos resultados das consultas, pelos seguintes motivos:

- A eliminação de duplicatas é uma operação dispendiosa. Um modo de implementá-la é classificar as tuplas primeiro e depois eliminar as duplicatas.
- O usuário pode querer ver as tuplas duplicadas no resultado de uma consulta.
- Quando uma função agregada (ver Seção 5.1.7) é aplicada às tuplas, na maioria dos casos não queremos eliminar duplicatas.

Uma tabela SQL com uma chave é restrita a ser distinto em cada tupla[10]. Se *quisermos* eliminar tuplas duplicadas do resultado de uma consulta SQL, usamos a palavra-chave **DISTINCT** na cláusula SELECT, significando que apenas as tuplas distintas deverão permanecer no resultado. Em geral, uma consulta com SELECT DISTINCT elimina duplicatas, enquanto uma consulta com SELECT ALL não elimina. A especificação de SELECT sem ALL ou DISTINCT — como em nossos exemplos anteriores — é equivalente a SELECT ALL. Por exemplo, a C11 recupera o salário de cada funcionário; se vários funcionários tiverem o mesmo salário, esse valor de salário aparecerá muitas vezes no resultado da consulta, como mostra a Figura 4.4(a). Se estivermos interessados apenas em valores de salário distintos, queremos que cada valor apareça apenas uma vez, independentemente de quantos funcionários ganham esse salário. Usando a palavra-chave **DISTINCT**, como em C11A, conseguimos isso, como mostra a Figura 4.4(b).

Consulta 11. Recuperar o salário de cada funcionário (C11) e todos os valores de salário distintos (C11A).

```
C11:  SELECT   ALL Salario
      FROM     FUNCIONARIO;
C11A: SELECT   DISTINCT Salario
      FROM     FUNCIONARIO;
```

A SQL incorporou diretamente algumas das operações de conjunto da *teoria de conjuntos* da matemática, que também fazem parte da álgebra relacional (ver Capítulo 6). Existem operações de união de conjunto (UNION), diferença de conjunto (**EXCEPT**),[11] e interseção de conjunto (**INTERSECT**). As relações resultantes dessas operações de conjunto são conjuntos de tuplas; ou seja, *tuplas duplicadas são eliminadas do resultado*. Essas operações de conjunto se aplicam apenas a *relações compatíveis com união*, de modo que precisamos garantir que as duas relações em que aplicamos a operação tenham os mesmos atributos e que os atributos apareçam na mesma ordem nas duas relações. O próximo exemplo ilustra o uso de UNION.

(a)

Salário
30.000
40.000
25.000
43.000
38.000
25.000
25.000
55.000

(b)

Salário
30.000
40.000
25.000
43.000
38.000
55.000

(c)

Pnome	Unome

(d)

Pnome	Unome
Fernando	Wong

Figura 4.4
Resultados de consultas SQL adicionais, quando aplicadas ao estado de banco de dados EMPRESA, mostrados na Figura 3.6. (a) C11. (b) C11A. (c) C12. (d) C12A.

Consulta 4. Fazer uma lista de todos os números de projeto para aqueles que envolvam um funcionário cujo último nome é 'Silva', seja como um trabalhador ou como um gerente do departamento que controla o projeto.

```
C4A:  ( SELECT   DISTINCT Projnumero
        FROM     PROJETO, DEPARTAMENTO,
                 FUNCIONARIO
        WHERE    Dnum=Dnumero AND
                 Cpf_gerente=Cpf
                 AND Unome='Silva' )
      UNION
      ( SELECT   DISTINCT Projnumero
        FROM     PROJETO, TRABALHA_EM,
                 FUNCIONARIO
        WHERE    Projnumero=Pnr AND Fcpf=Cpf
                 AND Unome='Silva' );
```

A primeira consulta SELECT recupera os projetos que envolvem um 'Silva' como gerente do de-

[10] Em geral, uma tabela SQL não precisa ter uma chave, embora, na maioria dos casos, exista uma.

[11] Em alguns sistemas, a palavra-chave MINUS é usada para a operação de diferença de conjunto, em vez de EXCEPT.

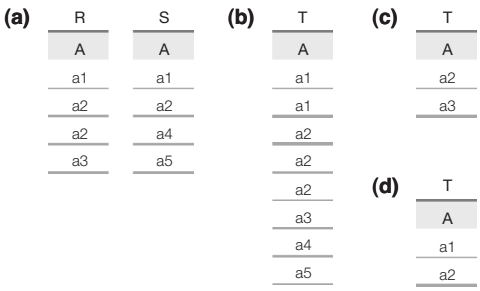


Figura 4.5
Os resultados das operações de multiconjunto da SQL. (a) Duas tabelas, R(A) e S(A). (b) R(A) UNION ALL S(A). (c) R(A) EXCEPT ALL S(A). (d) R(A) INTERSECT ALL S(A).

partamento que controla o projeto, e a segunda, recupera os projetos que envolvem um 'Silva' como um trabalhador no projeto. Observe que, se todos os funcionários tiverem o último nome 'Silva', os nomes de projeto envolvendo qualquer um deles seriam recuperados. A aplicação da operação UNION às duas consultas SELECT gera o resultado desejado.

A SQL também possui operações multiconjuntos correspondentes, que são acompanhadas da palavra-chave ALL (UNION ALL, EXCEPT ALL, INTERSECT ALL). Seus resultados são multiconjuntos (duplicatas não são eliminadas). O comportamento das operações é ilustrado pelos exemplos da Figura 4.5. Basicamente, cada tupla — seja ela uma duplicata ou não — é considerada uma tupla diferente ao aplicar essas operações.

4.3.5 Combinação de padrão de subcadeias e operadores aritméticos

Nesta seção, discutimos vários outros recursos da SQL. O primeiro recurso permite condições de comparação apenas sobre partes de uma cadeia de caracteres, usando o operador de comparação **LIKE**. Isso pode ser usado para **combinação de padrão** de cadeia. Cadeias parciais são especificadas usando dois caracteres reservados: % substitui um número qualquer de zero ou mais caracteres, e o sublinhado (_) substitui um único caractere. Por exemplo, considere a seguinte consulta.

Consulta 12. Recuperar todos os funcionários cujo endereço esteja em São Paulo, SP.

```
C12: SELECT  Pnome, Unome
     FROM    FUNCIONARIO
     WHERE   Endereco LIKE '%SaoPaulo,SP%';
```

Para recuperar todos os funcionários que nasceram durante a década de 1950, podemos usar a Consulta C12A. Aqui, '5' precisa ser o nono caractere da cadeia (de acordo com nosso formato para data), de modo que usamos o valor '_ _ _ _ _ _ _ _ 5 _', com cada sublinhado servindo como um marcador de lugar para um caractere qualquer.

Consulta 12A. Encontrar todos os funcionários que nasceram durante a década de 1950.

```
C12: SELECT  Pnome, Unome
     FROM    FUNCIONARIO
     WHERE   Datanasc LIKE '_ _ _ _ _ _ _ _ 5 _';
```

Se um sublinhado ou % for necessário como um caractere literal na cadeia, este deve ser precedido por um *caractere de escape*, que é especificado após a cadeia usando a palavra-chave ESCAPE. Por exemplo, 'AB_CD\%EF' ESCAPE '\' representa a cadeia literal 'AB_CD%EF', pois \ é especificado como o caractere de escape. Qualquer caractere não usado na cadeia pode ser escolhido como caractere de escape. Além disso, precisamos de uma regra para especificar apóstrofos ou aspas simples (' ') se eles tiverem de ser incluídos em uma cadeia, pois são usados para iniciar e terminar cadeias. Se um apóstrofo (') for necessário, ele será representado como dois apóstrofos consecutivos (''), de modo que não será interpretado como o término da cadeia. Observe que a comparação de subcadeia implica que os valores de atributo não sejam valores atômicos (indivisíveis), conforme assumimos no modelo relacional formal (ver Seção 3.1).

Outro recurso permite o uso de aritmética nas consultas. Os operadores aritméticos padrão para adição (+), subtração (–), multiplicação (*) e divisão (/) podem ser aplicados a valores ou atributos numéricos com domínios numéricos. Por exemplo, suponha que queiramos ver o efeito de dar a todos os funcionários que trabalham no projeto 'ProdutoX' um aumento de 10 por cento; podemos fazer a Consulta 13 para ver quais seriam seus novos salários. Este exemplo também mostra como podemos renomear um atributo no resultado da consulta usando AS na cláusula SELECT.

Consulta 13. Mostrar os salários resultantes se cada funcionário que trabalha no projeto 'ProdutoX' receber um aumento de 10 por cento.

```
C13: SELECT  F.Pnome, F.Unome, 1,1 * F.Salario AS
             Aumento_salario
     FROM    FUNCIONARIO AS F, TRABALHA_EM
             AS T, PROJETO AS P
     WHERE   F.Cpf=T.Fcpf AND T.Pno=P.Projnumero
             AND P.Projnome='ProdutoX';
```

Para os tipos de dados de cadeia, o operador de concatenação || pode ser usado em uma consulta para anexar dois valores de cadeia. Para tipos de dados data, hora, timestamp e intervalo, os operadores incluem incremento (+) ou decremento (–) de uma data, hora ou timestamp por um intervalo. Além disso, um valor de intervalo é o resultado da diferença entre dois valores de data, hora ou timestamp. Outro operador de comparação, que pode ser usado por conveniência, é **BETWEEN**, que está ilustrado na Consulta 14.

Consulta 14. Recuperar todos os funcionários no departamento 5 cujo salário esteja entre R$ 30.000 e R$ 40.000.

```
C14: SELECT   *
     FROM     FUNCIONARIO
     WHERE    (Salario BETWEEN 30.000 AND
              40.000) AND Dnr = 5;
```

A condição (Salario **BETWEEN** 30.000 **AND** 40.000) em C14 é equivalente à condição ((Salario >= 30.000) **AND** (Salario <= 40.000)).

4.3.6 Ordem dos resultados da consulta

A SQL permite que o usuário ordene as tuplas no resultado de uma consulta pelos valores de um ou mais dos atributos que aparecem, usando a cláusula **ORDER BY**. Isso é ilustrado pela Consulta 15.

Consulta 15. Recuperar uma lista dos funcionários e dos projetos em que estão trabalhando, ordenada por departamento e, dentro de cada departamento, ordenada alfabeticamente pelo sobrenome, depois pelo nome.

```
C15: SELECT   D.Dnome, F.Unome, F.Pnome,
              P.Projnome
     FROM     DEPARTAMENTO D, FUNCIONARIO
              F, TRABALHA_EM T, PROJETO P
     WHERE    D.Dnumero= F.Dnr AND F.Cpf=
              T.Fcpf AND T.Pnr= P.Projnumero
     ORDER BY D.Dnome, F.Unome, F.Pnome;
```

A ordem padrão está em ordem crescente de valores. Podemos especificar a palavra-chave **DESC** se quisermos ver o resultado em uma ordem decrescente de valores. A palavra-chave **ASC** pode ser usada para especificar a ordem crescente explicitamente. Por exemplo, se quisermos a ordem alfabética decrescente de Dnome e ordem crescente de Unome, Pnome, a cláusula ORDER BY da C15 pode ser escrita como

```
ORDER BY D.Dnome DESC, F.Unome ASC, F.Pnome
ASC
```

4.3.7 Discussão e resumo das consultas de recuperação da SQL básica

Uma *simples* consulta em SQL pode consistir em até quatro cláusulas, mas apenas as duas primeiras — SELECT e FROM — são obrigatórias. As cláusulas são especificadas na seguinte ordem, com aquelas entre colchetes [...] sendo opcionais:

```
SELECT      <lista atributos>
FROM        <lista tabelas>
[ WHERE     <condição>]
[ ORDER BY  <lista atributos> ];
```

A cláusula SELECT lista os atributos a serem recuperados, e a cláusula FROM especifica todas as relações (tabelas) necessárias na consulta simples. A cláusula WHERE identifica as condições para selecionar as tuplas dessas relações, incluindo condições de junção, se necessário. ORDER BY especifica uma ordem para exibir os resultados de uma consulta. Duas cláusulas adicionais, GROUP BY e HAVING, serão descritas na Seção 5.1.8.

No Capítulo 5, apresentaremos recursos mais complexos das consultas SQL. Estes incluem os seguintes: consultas aninhadas, que permitem que uma consulta seja incluída como parte de outra consulta; funções de agregação, que são usadas para fornecer resumos da informação nas tabelas; duas cláusulas adicionais (GROUP BY e HAVING), que podem ser usadas para fornecer mais poder para as funções agregadas; e vários tipos de junções (*joins*) que podem combinar registros de várias tabelas de diferentes maneiras.

4.4 Instruções INSERT, DELETE e UPDATE em SQL

Em SQL, três comandos podem ser usados para modificar o banco de dados: INSERT, DELETE e UPDATE. Discutiremos cada um deles.

4.4.1 O comando INSERT

Em sua forma mais simples, INSERT é usado para acrescentar uma única tupla a uma relação. Temos de especificar o nome da relação e uma lista de valores para a tupla. Os valores devem ser listados *na mesma ordem* em que os atributos correspondentes foram especificados no comando CREATE TABLE. Por exemplo, para acrescentar uma nova tupla à relação FUNCIONARIO mostrada na Figura 3.5 e especificada no comando CREATE TABLE FUNCIONARIO.. da Figura 4.1, podemos usar U1:

```
U1:  INSERT INTO  FUNCIONARIO
     VALUES       ( 'Ricardo', 'K', 'Marini',
                  '65329865388', '30-12-
                  1962', 'Rua Itapira, 44,
                  Santos, SP', 'M', 37.000,
                  '65329865388', 4 );
```

Uma segunda forma da instrução INSERT permite que o usuário especifique nomes de atributo explícitos que correspondem aos valores fornecidos no comando INSERT. Isso é útil se uma relação tiver muitos atributos, mas apenas alguns deles recebem valores em uma nova tupla. Porém, os valores precisam incluir todos os atributos com a especificação NOT NULL *e* nenhum valor padrão. Os atributos com NULL permitido ou com valores DEFAULT são aqueles que podem ser *omitidos*. Por exemplo, para inserir uma tupla para um novo FUNCIONARIO do qual conhecemos apenas os atributos Pnome, Unome, Dnr e Cpf, podemos usar U1A:

```
U1A: INSERT INTO  FUNCIONARIO (Pnome,
                  Unome, Dnr, Cpf)
     VALUES       ('Ricardo', 'Marini', 4,
                  '65329865388');
```

Os atributos não especificados em U1A são definidos como seu valor DEFAULT ou NULL, e os valores são listados na mesma ordem que os *atributos são listados no próprio comando INSERT* Também é possível inserir em uma relação *múltiplas tuplas* separadas por vírgulas em um único comando INSERT. Os valores de atributo que formam *cada tupla* ficam entre parênteses.

Um SGBD que implementa totalmente a SQL deve aceitar e impor todas as restrições de integridade que podem ser especificadas na DDL. Por exemplo, se emitirmos o comando em U2 sobre o banco de dados mostrado na Figura 3.6, o SGBD deve *rejeitar* a operação, pois não existe uma tupla DEPARTAMENTO no banco de dados com Dnumero = 2. De modo semelhante, U2A seria *rejeitado* porque nenhum valor de Cpf é fornecido e essa é a chave primária, que não pode ser NULL.

```
U2:  INSERT INTO  FUNCIONARIO (Pnome,
                  Unome, Cpf, Dnr)
     VALUES       ('Roberto', 'Gomes',
                  '98076054011', 2);
```

(U2 é rejeitado se a verificação da integridade referencial for oferecida pelo SGBD.)

```
U2A: INSERT INTO  FUNCIONARIO (Pnome,
                  Unome, Dnr)
     VALUES       ('Roberto', 'Gomes', 5);
```

(U2 é rejeitado se a verificação de NOT NULL for oferecida pelo SGBD.)

Uma variação do comando INSERT inclui várias tuplas em uma relação em conjunto com a criação da relação e sua carga com o *resultado de uma consulta* Por exemplo, para criar uma tabela temporária que possui o nome do funcionário, o nome do projeto e as horas por semana para cada funcionário que trabalha em um projeto, podemos escrever as instruções em U3A e U3B:

```
U3A: CREATE TABLE  TRABALHA_EM_INFO
     ( Func_nome       VARCHAR(15),
       Proj_nome       VARCHAR(15),
       Horas_semanal   DECIMAL(3,1) );

U3B: INSERT INTO   TRABALHA_EM_INFO
                   ( Func_nome, Proj_nome,
                   Horas_por_semana)
     SELECT        F.Unome, P.Projnome,
                   T.Horas
     FROM          PROJETO P, TRABALHA_
                   EM T, FUNCIONARIO F
     WHERE         P.Projnumero=T.Pnr AND
                   T.Fcpf=F.Cpf;
```

Uma tabela TRABALHA_EM_INFO é criada por U3A e é carregada com a informação da junção recuperada do banco de dados pela consulta em U3B. Agora, podemos consultar TRABALHA_EM_INFO como faríamos com qualquer outra relação; quando não precisarmos mais dela, poderemos removê-la usando o comando DROP TABLE (ver Capítulo 5). Observe que a tabela TRABALHA_EM_INFO pode não estar atualizada; ou seja, se atualizarmos qualquer uma das relações PROJETO, TRABALHA_EM ou FUNCIONARIO depois de emitir U3B, a informação em TRABALHA_EM_INFO *pode ficar desatualizada.* Temos de criar uma visão, ou view (ver Capítulo 5), para manter essa tabela atualizada.

4.4.2 O comando DELETE

O comando **DELETE** remove tuplas de uma relação. Ele inclui uma cláusula WHERE, semelhante a que é usada em uma consulta SQL, para selecionar as tuplas a serem excluídas. As tuplas são explicitamente excluídas de apenas uma tabela por vez. No entanto, a exclusão pode se propagar para as tuplas

em outras relações, se *ações de disparo referencial* forem especificadas nas restrições de integridade referencial da DDL (ver Seção 4.2.2).[12] Dependendo do número de tuplas selecionadas pela condição na cláusula WHERE, zero, uma ou várias tuplas podem ser excluídas por um único comando DELETE. Uma cláusula WHERE inexistente especifica que todas as tuplas na relação deverão ser excluídas; porém, a tabela permanece no banco de dados como uma tabela vazia. Temos de usar o comando DROP TABLE para remover a definição da tabela (ver Capítulo 5). Os comandos DELETE em U4A a U4D, se aplicados de maneira independente ao banco de dados da Figura 3.6, excluirão zero, uma, quatro e todas as tuplas, respectivamente, da relação FUNCIONARIO:

```
U4A: DELETE FROM  FUNCIONARIO
     WHERE        Unome='Braga';
U4B: DELETE FROM  FUNCIONARIO
     WHERE        Cpf='12345678966';
U4C: DELETE FROM  FUNCIONARIO
     WHERE        Dnr=5;
U4D: DELETE FROM  FUNCIONARIO;
```

4.4.3 O comando UPDATE

O comando **UPDATE** é usado para modificar valores de atributo de uma ou mais tuplas selecionadas. Assim como no comando DELETE, uma cláusula WHERE no comando UPDATE seleciona as tuplas a serem modificadas em uma única relação. No entanto, a atualização de uma chave primária pode ser propagada para os valores de chave estrangeira das tuplas em outras relações se tal *ação de disparo referencial* for especificada nas restrições de integridade referencial da DDL (ver Seção 4.2.2). Uma cláusula **SET** adicional no comando UPDATE especifica os atributos a serem modificados e seus novos valores. Por exemplo, para alterar o local e número de departamento que controla o número de projeto 10 para 'Santo André' e 5, respectivamente, usamos U5:

```
U5: UPDATE  PROJETO
    SET     Projlocal = 'Santo André', Dnum
            = 5
    WHERE   Projnumero=10;
```

Várias tuplas podem ser modificadas com um único comando UPDATE. Um exemplo é dar a todos os funcionários no departamento 'Pesquisa' um aumento de 10 por cento no salário, como mostra U6. Nesta solicitação, o valor de Salario modificado depende do valor de Salario em cada tupla, de modo que duas referências ao atributo Salario são necessárias. Na cláusula SET, a referência ao atributo Salario à direita refere-se ao antigo valor de Salario, *antes da modificação*, e aquele à esquerda refere-se ao novo valor de Salario, *após a modificação*:

```
U6: UPDATE  FUNCIONARIO
    SET     Salario = Salario * 1,1
    WHERE   Dnr = 5;
```

Também é possível especificar NULL ou DEFAULT como o novo valor do atributo. Observe que cada comando UPDATE refere-se explicitamente a apenas uma única relação. Para modificar várias relações, precisamos emitir vários comandos UPDATE.

4.5 Recursos adicionais da SQL

A SQL possui uma série de recursos adicionais que não descrevemos neste capítulo, mas que discutiremos em outras partes do livro. São eles:

- No Capítulo 5, que é uma continuação deste capítulo, apresentaremos os seguintes recursos da SQL: diversas técnicas para especificar consultas de recuperação complexas, incluindo consultas aninhadas, funções de agregação, agrupamento, tabelas com junções (*join*), junções externas (*outer joins*) e consultas recursivas; visões (*views*), gatilhos (*triggers*) e asserções (*assertions*) da SQL; e comandos para modificação de esquema.
- A linguagem SQL possui diversas técnicas para escrever programas em várias linguagens de programação, que incluem instruções SQL para acessar um ou mais bancos de dados. Estas incluem SQL embutida (e dinâmica), SQL/CLI (Call Level Interface) e seu predecessor ODBC (Open Data Base Connectivity) e SQL/PSM (Persistent Stored Modules). Discutiremos essas técnicas no Capítulo 13.

  Também discutiremos como acessar bancos de dados SQL por meio da linguagem de programação Java usando JDBC e SQLJ.
- Cada SGBDR comercial terá, além dos comandos SQL, um conjunto de comandos para especificar parâmetros de projeto do

[12] Outras ações podem ser aplicadas automaticamente por meio de disparos (ver Seção 26.1) e outros mecanismos.

banco de dados físico, estruturas de arquivo para relações e caminhos de acesso, como índices. Chamamos esses comandos de *linguagem de definição de armazenamento* (*SDL*) no Capítulo 2. As primeiras versões da SQL tinham comandos para **criar índices**, mas estes foram removidos da linguagem porque não estavam no nível de esquema conceitual. Muitos sistemas ainda têm comandos CREATE INDEX.

- A SQL possui comandos de controle de transação. Estes são usados para especificar unidades de processamento de banco de dados para fins de controle de concorrência e recuperação. Discutiremos esses comandos no Capítulo 21, depois de discutirmos o conceito de transações com mais detalhes.
- A SQL possui construções da linguagem para especificar a *concessão* e *revogação de privilégios* aos usuários. Os privilégios normalmente correspondem ao direito de usar certos comandos SQL para acessar determinadas relações. Cada relação recebe um owner (proprietário), e este ou o DBA pode conceder a usuários selecionados o privilégio de usar uma instrução SQL — como SELECT, INSERT, DELETE ou UPDATE — para acessar a relação. Além disso, o DBA pode conceder os privilégios para criar esquemas, tabelas ou views a certos usuários. Esses comandos SQL — chamados de **GRANT** e **REVOKE** — serão discutidos no Capítulo 24, onde falaremos sobre segurança e autorização no banco de dados.
- A SQL possui construções de linguagem para a criação de triggers. Estas geralmente são conhecidas como técnicas de **banco de dados ativo**, pois especificam ações que são disparadas automaticamente por eventos, como atualizações no banco de dados. Discutiremos esses recursos na Seção 26.1, na qual abordaremos os conceitos de banco de dados ativo.
- A SQL incorporou muitos recursos dos modelos orientados a objeto para ter capacidades mais poderosas, levando a sistemas relacionais avançados, conhecidos como **objeto-relacional**. Capacidades como criar atributos complexos (também chamadas **relações aninhadas**), especificar tipos de dados abstratos (chamados **UDT**s ou tipos definidos pelo usuário) para atributos e tabelas, criar **identificadores de objeto** para referenciar tuplas e especificar **operações** sobre tipos serão discutidas no Capítulo 11.
- SQL e bancos de dados relacionais podem interagir com novas tecnologias, como XML (ver Capítulo 12) e OLAP (Capítulo 29).

Resumo

Neste capítulo, apresentamos a linguagem de banco de dados SQL. Essa linguagem e suas variações têm sido implementadas como interfaces para muitos SGBDs relacionais comerciais, incluindo Oracle e Rdb, da Oracle;[13] DB2, Informix Dynamic Server e SQL/DS, da IBM; SQL Server e Access, da Microsoft; e INGRES. Alguns sistemas de código aberto também fornecem SQL, como MySQL e PostgreSQL. A versão srcinal da SQL foi implementada no SGBD experimental chamado SYSTEM R, que foi desenvolvido na IBM Research. A SQL foi projetada para ser uma linguagem abrangente, que inclui instruções para definição de dados, consultas, atualizações, especificação de restrição e definição de view. Discutimos os seguintes recursos da SQL neste capítulo: os comandos de definição de dados para criar tabelas, comandos para especificação de restrição, consultas de recuperação simples e comandos de atualização de banco de dados. No próximo capítulo, apresentaremos os seguintes recursos da SQL: consultas de recuperação complexas; views; triggers e assertions; e comandos de modificação de esquema.

Perguntas de revisão

4.1. Como as relações (tabelas) em SQL diferem das relações definidas formalmente no Capítulo 3? Discuta as outras diferenças na terminologia. Por que a SQL permite tuplas duplicadas em uma tabela ou em um resultado de consulta?

4.2. Liste os tipos de dados que são permitidos para atributos SQL.

4.3. Como a SQL permite a implementação das restrições de integridade de entidade e de integridade referencial descritas no Capítulo 3? E as ações de disparo referencial?

4.4. Descreva as quatro cláusulas na sintaxe de uma consulta de recuperação SQL simples. Mostre que tipo de construções pode ser especificado em cada uma das cláusulas. Quais são obrigatórias e quais são opcionais?

[13] O Rdb foi produzido srcinalmente pela Digital Equipment Corporation. Ele foi comprado da Digital pela Oracle em 1994, e está recebendo suporte e sendo melhorado.

Exercícios

4.5. Considere o banco de dados mostrado na Figura 1.2, cujo esquema aparece na Figura 2.1. Quais são as restrições de integridade referencial que devem ser mantidas no esquema? Escreva instruções DDL da SQL apropriadas para definir o banco de dados.

4.6. Repita o Exercício 4.5, mas use o esquema de banco de dados COMPANHIA AEREA da Figura 3.8.

4.7. Considere o esquema de banco de dados relacional BIBLIOTECA mostrado na Figura 4.6. Escolha a ação apropriada (rejeitar, propagar, SET NULL, SET DEFAULT) para cada restrição de integridade referencial, tanto para a *exclusão* de uma tupla referenciada quanto para a *atualização* de um valor de atributo de chave primária em uma tupla referenciada. Justifique suas escolhas.

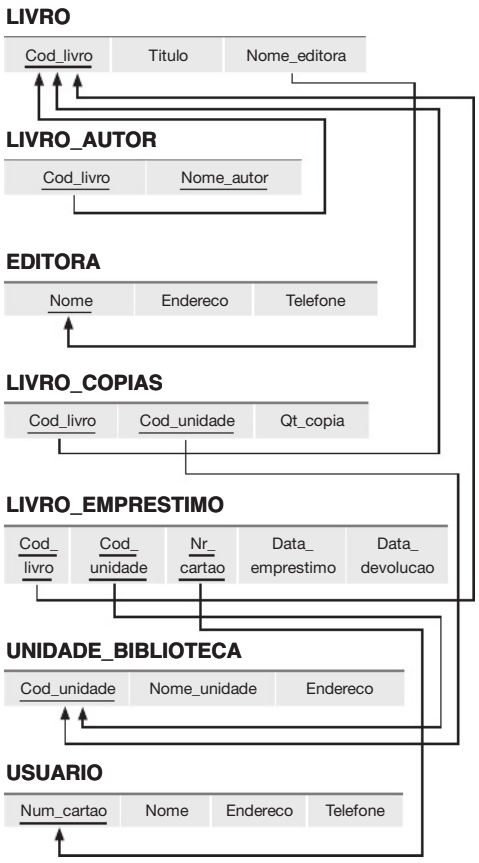


Figura 4.6
Um esquema de banco de dados relacional para um banco de dados BIBLIOTECA.

4.8. Escreva as instruções DDL da SQL apropriadas para declarar o esquema de banco de dados relacional BIBLIOTECA da Figura 4.6. Especifique as chaves e ações de disparo referencial.

4.9. Como as restrições de chave e chave estrangeira podem ser impostas pelo SGBD? A técnica de imposição que você sugere é difícil de implementar? As verificações de restrição podem ser executadas de modo eficiente quando as atualizações são aplicadas ao banco de dados?

4.10. Especifique as seguintes consultas em SQL sobre o esquema de banco de dados relacional EMPRESA mostrado na Figura 3.5. Mostre o resultado de cada consulta se ela for aplicada ao banco de dados EMPRESA na Figura 3.6.

a. Recupere os nomes de todos os funcionários no departamento 5 que trabalham mais de 10 horas por semana no projeto ProdutoX.

b. Liste os nomes de todos os funcionários que possuem um dependente com o mesmo primeiro nome que seu próprio.

c. Ache os nomes de todos os funcionários que são supervisionados diretamente por 'Fernando Wong'.

4.11. Especifique as atualizações do Exercício 3.11 usando comandos de atualização da SQL.

4.12. Especifique as consultas a seguir em SQL no esquema de banco de dados da Figura 1.2.

a. Recupere os nomes de todos os alunos sênior se formando em 'CC' (Ciência da computação).

b. Recupere os nomes de todas as disciplinas lecionadas pelo Professor Kleber em 2007 e 2008.

c. Para cada matéria lecionada pelo Professor Kleber, recupere o número da disciplina, semestre, ano e número de alunos que realizaram a matéria.

d. Recupere o nome e o histórico de cada aluno sênior (Tipo_aluno = 4) formando em CC. Um histórico inclui nome da disciplina, número da disciplina, crédito, semestre, ano e nota para cada disciplina concluída pelo aluno.

4.13. Escreva instruções de atualização SQL para realizar ações sobre o esquema de banco de dados mostrado na Figura 1.2.

a. Inserir um novo aluno <'Alves', 25, 1, 'MAT'>, no banco de dados.

b. Alterar a turma do aluno 'Silva' para 2.

c. Inserir uma nova disciplina, <'Engenharia do conhecimento', 'CC4390', 3, 'CC'>.

d. Excluir o registro para o aluno cujo nome é 'Silva' e cujo número de aluno é 17.

4.14. Crie um esquema de banco de dados relacional para uma aplicação de banco de dados a sua escolha.

a. Declare suas relações, usando a DDL da SQL.

b. Especifique algumas consultas em SQL que sejam necessárias para sua aplicação de banco de dados.

c. Com base em seu uso esperado do banco de dados, escolha alguns atributos que deverão ter índices específicos.

d. Implemente seu banco de dados, se você tiver um SGBD que suporta SQL.

4.15. Considere que a restrição CHESUPERFUNC da tabela FUNCIONARIO, conforme especificado na Figura 4.2, seja mudada para:

```
CONSTRAINT CHESUPERFUNC
  REFERENCES FUNCIONARIO(Cpf)
                ON DELETE CASCADE ON
UPDATE CASCADE,
  FOREIGN KEY (Cpf_supervisor)
```

Responda às seguintes questões:

a. O que acontece quando o comando a seguir é executado no estado de banco de dados mostrado na Figura 3.6?

```
DELETE FUNCIONARIO WHERE Unome = 'Brito'
```

b. É melhor usar CASCADE ou SET NULL no caso da restrição ON DELETE de CHESUPERFUNC?

4.16. Escreva instruções SQL para criar uma tabela FUNCIONARIO_BACKUP para fazer o backup da tabela FUNCIONARIO mostrada na Figura 3.6.

Bibliografia selecionada

A linguagem SQL, srcinalmente chamada SEQUEL, foi baseada na linguagem SQUARE (Specifying Queries as Relational Expressions), descrita por Boyce et al. (1975). A sintaxe da SQUARE foi modificada para a SEQUEL (Chamberlin e Boyce, 1974) e depois para SEQUEL 2 (Chamberlin et al., 1976), na qual a SQL é baseada. A implementação srcinal da SEQUEL foi feita na IBM Research, em San Jose, Califórnia. Mostraremos outras referências aos vários aspectos da SQL ao final do Capítulo 5.

capítulo

5

Mais SQL: Consultas complexas, triggers, views e modificação de esquema

Este capítulo descreve recursos mais avançados da linguagem SQL padrão para bancos de dados relacionais. Começamos na Seção 5.1 apresentando recursos mais complexos das consultas de recuperação SQL, como consultas aninhadas, tabelas com junções, junções externas, funções de agregação e agrupamento. Na Seção 5.2, descrevemos o comando CREATE ASSERTION, que permite a especificação de restrições mais gerais sobre o banco de dados. Também apresentamos o conceito de triggers (gatilhos) e o comando CREATE TRIGGER, que será descrito com mais detalhes na Seção 26.1, quando mostraremos os princípios dos bancos de dados ativos. Depois, na Seção 5.3, descrevemos a facilidade da SQL para definir views (visões) no banco de dados. As views também são chamadas de *tabelas virtuais* ou *derivadas*, pois apresentam ao usuário o que parecem ser tabelas; porém, a informação nessas tabelas é derivada de tabelas previamente definidas. A Seção 5.4 apresenta o comando SQL ALTER TABLE, que é usado para modificar as tabelas e restrições do banco de dados. No final há um resumo do capítulo.

Este capítulo é uma continuação do Capítulo 4. O leitor poderá pular partes deste capítulo se desejar uma introdução menos detalhada à linguagem SQL.

5.1 Consultas de recuperação SQL mais complexas

Na Seção 4.3, descrevemos alguns tipos básicos de consultas de recuperação em SQL. Por causa da generalidade e do poder expressivo da linguagem, existem muitos outros recursos adicionais que permitem que os usuários especifiquem recuperações mais complexas do banco de dados. Discutiremos vários desses recursos nesta seção.

5.1.1 Comparações envolvendo NULL e lógica de três valores

A SQL tem diversas regras para lidar com valores NULL. Lembre-se, da Seção 3.1.2, que NULL é usado para representar um valor que está faltando, mas que em geral tem uma de três interpretações diferentes — valor *desconhecido* (existe, mas não é conhecido), valor *não disponível* (existe, mas é propositadamente retido) ou valor *não aplicável* (o atributo é indefinido para essa tupla). Considere os seguintes exemplos para ilustrar cada um dos significados de NULL.

1. **Valor desconhecido.** A data de nascimento de uma pessoa não é conhecida, e por isso é representada por NULL no banco de dados.
2. **Valor indisponível ou retido.** Uma pessoa tem um telefone residencial, mas não deseja que ele seja listado, por isso ele é retido e representado como NULL no banco de dados.
3. **Atributo não aplicável.** Um atributo Conjuge seria NULL para uma pessoa que não fosse casada, pois ele não se aplica a essa pessoa.

Normalmente, não é possível determinar qual dos significados é intencionado; por exemplo, um NULL para o telefone residencial de uma pessoa pode ter qualquer um dos três significados. Logo, a SQL não distingue entre os diferentes significados de NULL.

Em geral, cada valor NULL individual é considerado diferente de qualquer outro valor NULL nos diversos registros do banco de dados. Quando um

NULL está envolvido em uma operação de comparação, o resultado é considerado UNKNOWN, ou desconhecido (e pode ser TRUE ou FALSE). Assim, a SQL usa uma lógica de três valores com os valores TRUE, FALSE e UNKNOWN em vez da lógica de dois valores (booleana) padrão, com os valores TRUE e FALSF. Portanto, é necessário definir os resultados (ou valores verdadeiros) das expressões lógicas de três valores quando os conectivos lógicos AND, OR e NOT forem usados. A Tabela 5.1 mostra os valores resultantes.

Nas Tabelas 5.1(a) e 5.1(b), as linhas e colunas representam os valores dos resultados das condições de comparação, que normalmente apareceriam na cláusula WHERE de uma consulta SQL. Cada resultado de expressão teria um valor TRUE, FALSE ou UNKNOWN. O resultado da combinação de dois valores usando o conectivo lógico AND é mostrado pelas entradas na Tabela 5.1(a). A Tabela 5.1(b) mostra o resultado do uso do conectivo lógico OR. Por exemplo, o resultado de (FALSE AND UNKNOWN) é FALSE, ao passo que o re sultado de (FALSE OR UNKNOWN) é UNKNOWN. A Tabela 5.1(c) mostra o resultado da operação lógica NOT. Observe que, na lógica booleana padrão, somente valores TRUE e FALSE são permitidos; não existe um valor UNKNOWN.

Nas consultas seleção-projeção-junçã o, a regra geral é que somente as combinações de tuplas que avaliam a expressão lógica na cláusula WHERE da consulta como TRUE são selecionadas. As combinações de tupla que são avaliadas como FALSE ou UNKNOWN não são selecionadas. Porém, existem exceções a essa regra para certas operações, como junções externas (*outer joins*), conforme veremos na Seção 5.1.6.

Tabela 5.1
Conectivos lógicos na lógica de três valores.

(a) **AND**	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

(b) **OR**	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

(c) **NOT**	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

A SQL permite consultas que verificam se o valor de um atributo é **NULL**. Em vez de usar = ou <> para comparar o valor de um atributo com NULL, a SQL usa os operadores de comparação **IS** ou **IS NOT**. Isso porque ela considera cada valor NULL sendo distinto de cada outro valor NULL, de modo que a comparação de igualdade não é apropriada. Acontece que, quando uma condição de junção é especificada, as tuplas com valores NULL para os atributos de junção não são incluídas no resultado (a menos que seja uma OUTER JOIN; ver Seção 5.1.6). A Consulta 18 ilustra isso.

Consulta 18. Recuperar os nomes de todos os funcionários que não possuem supervisores.

```
C18:  SELECT  Pnome, Unome
      FROM    FUNCIONARIO
      WHERE   Cpf_supervisor IS NULL;
```

5.1.2 Consultas aninhadas, tuplas e comparações de conjunto/multiconjunto

Algumas consultas precisam que os valores existentes no banco de dados sejam buscados e depois usados em uma condição de comparação. Essas consultas podem ser formuladas convenientemente usando **consultas aninhadas**, que são blocos select-from-where completos dentro da cláusula WHERE de outra consulta. Essa outra consulta é chamada de **consulta externa**. A Consulta 4 é formulada em C4 sem uma consulta aninhada, mas pode ser reformulada para usar consultas aninhadas, como mostramos em C4A. A C4A introduz o operador de comparação **IN**, que compara um valor *v* com um conjunto (ou multiconjunto) de valores *V* e avalia como **TRUE** se *v* for um dos elementos em *V*.

A primeira consulta aninhada seleciona os números dos projetos que possuem um funcionário com sobrenome 'Silva' envolvido como gerente, enquanto a segunda consulta aninhada seleciona os números dos projetos que possuem um funcionário com o sobrenome 'Silva' envolvido como trabalhador. Na consulta externa, usamos o conectivo lógico **OR** para recuperar uma tupla PROJETO se o valor de PROJNUMERO dessa tupla estiver no resultado de qualquer uma das consultas aninhadas.

```
C4A: SELECT DISTINCT Projnumero
     FROM   PROJETO
     WHERE  Projnumero IN
            ( SELECT    Projnumero
              FROM      PROJETO,
                        DEPARTAMENTO,
                        FUNCIONARIO
              WHERE     Dnum=Dnumero AND
                        Cpf_gerente=Cpf
                        AND Unome='Silva' )
          OR            Projnumero IN
            ( SELECT    Pnr
              FROM      TRABALHA_EM,
                        FUNCIONARIO
              WHERE     Fcpf=Cpf AND
                        Unome='Silva' );
```

Se uma consulta aninhada retornar um único atributo *e* uma única tupla, o resultado da consulta será um único valor (escalar). Nesses casos, é permitido usar = em vez de IN para o operador de comparação. Em geral, a consulta aninhada retornará uma **tabela** (relação), que é um conjunto ou multiconjunto de tuplas.

A SQL permite o uso de **tuplas** de valores em comparações, colocando-os entre parênteses. Para ilustrar isso, considere a seguinte consulta:

```
SELECT DISTINCT Fcpf
FROM   TRABALHA_EM
WHERE  (Pnr, Horas) IN ( SELECT Pnr, Horas
                         FROM   TRABALHA_EM
                         WHERE  Fcpf='12345678966' );
```

Essa consulta selecionará os Fcpfs de todos os funcionários que trabalham na mesma combinação (projeto, horas) em algum projeto que o funcionário 'João Silva' (cujo Cpf = '12345678966') trabalha. Neste exemplo, o operador IN compara a subtupla de valores entre parênteses (Pnr, Horas) dentro de cada tupla em TRABALHA_EM com o conjunto de tuplas com tipos compatíveis produzidas pela consulta aninhada.

Além do operador IN, diversos outros operadores de comparação podem ser usados para comparar um único valor v (em geral, um nome de atributo) com um conjunto ou multiconjunto v (tipicamente, uma consulta aninhada). O operador = ANY (ou = SOME) retorna TRUE se o valor v for igual a *algum valor* no conjunto V e, portanto, é equivalente a IN. As duas palavras-chave ANY e SOME possuem o mesmo efeito. Outros operadores que podem ser combinados com ANY (ou SOME) incluem >, >=, <, <= e <>. A palavra-chave ALL também pode ser combinada com cada um desses operadores. Por exemplo, a condição de comparação (v > ALL V) retorna TRUE se o valor v é maior do que *todos* os valores no conjunto (ou multiconjunto) V. Um exemplo é a consulta a seguir, que retorna os nomes dos funcionários cujo salário é maior do que o salário de todos os funcionários no departamento 5:

```
SELECT Unome, Pnome
FROM   FUNCIONARIO
WHERE  Salario > ALL ( SELECT Salario
                       FROM   FUNCIONARIO
                       WHERE  Dnr=5 );
```

Observe que essa consulta também pode ser especificada usando a função de agregação MAX (ver Seção 5.1.7).

Em geral, podemos ter vários níveis de consultas aninhadas. Podemos mais uma vez lidar com a possível ambiguidade entre nomes de atributo se existirem atributos com o mesmo nome — um em uma relação na cláusula FROM da *consulta exterior* e outro em uma relação na cláusula FROM da *consulta aninhada*. A regra é que uma referência a um *atributo não qualificado* refere-se à relação declarada na **consulta aninhada mais interna**. Por exemplo, nas cláusulas SELECT e WHERE da primeira consulta aninhada de C4A, uma referência a qualquer atributo não qualificado da relação PROJETO refere-se à relação PROJETO especificada na cláusula FROM da consulta aninhada. Para se referir a um atributo da relação PROJETO especificada na consulta externa, especificamos e nos referimos a um *apelido* (variável de tupla) para essa relação. Essas regras são semelhantes às regras de escopo para variáveis de programa na maioria das linguagens de programação que permitem procedimentos e funções aninhadas. Para ilustrar a ambiguidade em potencial dos nomes de atributo nas consultas aninhadas, considere a Consulta 16.

Consulta 16. Recuperar o nome de cada funcionário que tem um dependente com o mesmo nome e com o mesmo sexo do funcionário.

```
C16: SELECT F.Pnome, F.Unome
     FROM   FUNCIONARIO AS F
     WHERE  F.Cpf IN ( SELECT D.Fcpf
                       FROM   DEPENDENTE
                              AS D
                       WHERE  F.Pnome=
                              D.Nome_
                              dependente
                              AND
                              F.Sexo=
                              D.Sexo );
```

Na consulta aninhada de C16, temos de qualificar F.Sexo porque se refere ao atributo Sexo de FUNCIONARIO da consulta externa, e DEPENDENTE também tem um atributo chamado Sexo. Se houvesse quaisquer referências não qualificadas a Sexo na consulta aninhada, elas se refeririam ao atributo Sexo de DEPENDENTF. No entanto, não *teríamos de* qualificar os atributos Pnome e Cpf de FUNCIONARIO se eles aparecessem na consulta aninhada, pois a relação DEPENDENTE não tem atributos chamados Pnome e Cpf, de modo que não existe ambiguidade.

É aconselhável criar variáveis de tupla (*apelidos*) para *todas as tabelas referenciadas em uma consulta SQL*, para evitar erros e ambiguidades em potencial, conforme ilustrado em C16.

5.1.3 Consultas aninhadas correlacionadas

Sempre que uma condição na cláusula WHERE de uma consulta aninhada referencia algum atributo de uma relação declarada na consulta externa, as duas consultas são consideradas **correlacionadas**. Podemos entender melhor uma consulta correlacionada ao considerar que a *consulta aninhada é avaliada uma vez para cada tupla (ou combinação de tuplas) na consulta externa*. Por exemplo, podemos pensar em C16 da seguinte forma: para *cada* tupla FUNCIONARIO, avalie a consulta aninhada, que recupera os valores de Fcpf para todas as tuplas de DEPENDENTE com o mesmo sexo e nome da tupla em FUNCIONARIO; se o valor de Cpf da tupla FUNCIONARIO estiver *no* resultado da consulta aninhada, então selecione essa tupla FUNCIONARIO.

Em geral, uma consulta escrita com blocos aninhados select-from-where e usando os operadores de comparação = ou IN *sempre* pode ser expressa como uma única consulta em bloco. Por exemplo, C16 pode ser escrito como em C16A:

```
C16A: SELECT F.Pnome, F.Unome
      FROM   FUNCIONARIO AS F,
             DEPENDENTE AS D
      WHERE  F.Cpf=D.Fcpf AND F.Sexo=D.Sexo
             AND F.Pnome=D.Nome_dependente;
```

5.1.4 As funções EXISTS e UNIQUE em SQL

A função EXISTS em SQL é usada para verificar se o resultado de uma consulta aninhada correlacionada é *vazio* (não contém tuplas) ou não. O resultado de EXISTS é um valor booleano **TRUE** se o resultado da consulta aninhada tiver pelo menos uma tupla, ou **FALSE**, se o resultado da consulta aninhada não tiver tuplas. Ilustramos o uso de EXISTS — e NOT EXISTS — com alguns exemplos. Primeiro, formulamos a Consulta 16 de uma forma alternativa, que usa EXISTS, como em C16B:

```
C16B: SELECT F.Pnome, F.Unome
      FROM   FUNCIONARIO AS F
      WHERE  EXISTS
             ( SELECT *
             FROM   DEPENDENTE
                    AS D
             WHERE  F.Cpf=D.Fcpf
                    AND F.Sexo=
                    D.Sexo
                    AND F.Pnome=
                    D.Nome_
                    dependente);
```

EXISTS e NOT EXISTS costumam ser usados em conjunto com uma consulta aninhada correlacionada. Em C16B, a consulta aninhada referencia os atributos Cpf, Pnome e Sexo da relação FUNCIONARIO da consulta externa. Podemos pensar em C16B da seguinte forma: para cada tupla FUNCIONARIO, avalie a consulta aninhada, que recupera todas as tuplas DEPENDENTE com os mesmos Fcpf, Sexo e Nome_dependente que a tupla FUNCIONARIO; se houver pelo menos uma tupla EXISTS no resultado da consulta aninhada, então selecionar a tupla FUNCIONARIO. Em geral, EXISTS(C) retorna **TRUE** se existe *pelo menos uma tupla* no resultado da consulta aninhada C, e retorna **FALSE** em caso contrário. Por sua vez, NOT EXISTS(C) retorna **TRUE** se *não houver tuplas* no resultado da consulta aninhada C, e retorna **FALSE** caso contrário. Em seguida, ilustramos o uso de NOT EXISTS.

Consulta 6. Recuperar os nomes de funcionários que não possuem dependentes.

```
C6: SELECT Pnome, Unome
    FROM   FUNCIONARIO
    WHERE  NOT EXISTS ( SELECT*
                        FROM   DEPENDENTE
                        WHERE Cpf=Fcpf );
```

Em C6, a consulta aninhada correlacionada recupera todas as tuplas de DEPENDENTE relacionadas a uma tupla FUNCIONARIO em particular. Se *não existir nenhuma*, a tupla FUNCIONARIO é selecionada, porque a condição da cláusula **WHERE** será avaliada como **TRUE** nesse caso. Podemos explicar C6 da seguinte forma: para *cada* tupla FUNCIONARIO, a

consulta aninhada correlacionada seleciona todas as tuplas DEPENDENTE cujo valor de Fcpf combina com o Cpf de FUNCIONARIO; se o resultado for vazio, nenhum dependente estará relacionado ao funcionário, de modo que selecionamos essa tupla FUNCIONARIO e recuperamos seu Pnome e Unome.

Consulta 7. Listar os nomes dos gerentes que possuem pelo menos um dependente.

```
C7: SELECT Pnome, Unome
    FROM   FUNCIONARIO
    WHERE  EXISTS ( SELECT *
                    FROM   DEPENDENTE
                    WHERE  Cpf=Fcpf )
           AND
           EXISTS ( SELECT *
                    FROM   DEPARTAMENTO
                    WHERE  Cpf=Cpf_gerente );
```

Uma maneira de escrever essa consulta é mostrada em C7, onde especificamos duas consultas aninhadas correlacionadas; a primeira seleciona todas as tuplas de DEPENDENTE relacionadas a um FUNCIONARIO e a segunda seleciona todas as tuplas de DEPARTAMENTO gerenciadas pelo FUNCIONARIO. Se pelo menos uma da primeira e pelo menos uma da segunda existir, selecionamos a tupla FUNCIONARIO. Você consegue reescrever essa consulta usando apenas uma única consulta aninhada ou nenhuma consulta aninhada?

A consulta *C3, recuperar o nome de cada funcionário que trabalha em todos os projetos controlados pelo departamento número 5*, pode ser escrita usando EXISTS e NOT EXISTS nos sistemas SQL. Mostramos duas maneiras de especificar essa consulta C3 em SQL como C3A e C3B. Este é um exemplo de certos tipos de consultas que exigem *quantificação universal*, conforme discutiremos na Seção 6.6.7. Um modo de escrever essa consulta é usar a construção (*S2* EXCEPT *S1*), conforme explicaremos a seguir, e verificar se o resultado é vazio.[1] Essa opção aparece como C3A.

```
C3A: SELECT  Pnome, Unome
     FROM    FUNCIONARIO
     WHERE   NOT EXISTS ( (
             SELECT    Projnumero
             FROM      PROJETO
             WHERE     Dnum=5)
             EXCEPT    (
             SELECT    Pnr
             FROM      TRABALHA_EM
             WHERE     Cpf=Fcpf) );
```

Em C3A, a primeira subconsulta (que não está correlacionada à consulta externa) seleciona todos os projetos controlados pelo departamento 5, e a segunda subconsulta (que está correlacionada) seleciona todos os projetos em que o funcionário em particular trabalha. Se a diferença de conjunto do resultado da primeira subconsulta menos (SUBTRAÇÃO) (EXCEPT) o resultado da segunda subconsulta for vazio, isso significa que o funcionário trabalha em todos os projetos e, portanto, é selecionado.

A segunda opção aparece como C3B. Observe que precisamos de aninhamento de dois níveis em C3B e que essa formulação é muito mais complexa do que C3A, que usa NOT EXISTS e EXCEPT.

```
C3B:  SELECT Unome, Pnome
      FROM   FUNCIONARIO
      WHERE  NOT EXISTS
             ( SELECT *
               FROM   TRABALHA_EM T1
               WHERE  ( T1.Pnr IN
                        ( SELECT Projnumero
                          FROM   PROJETO
                          WHERE  Dnum=5 )
               AND NOT EXISTS
               ( SELECT *
               FROM TRABALHA_EM T2
               WHERE  T2.Fcpf=Cpf
               AND    T2.Pnr=B.Pnr )));
```

Em C3B, a consulta aninhada externa seleciona quaisquer tuplas de TRABALHA_EM (T1) cujo Pnr é de um projeto controlado pelo departamento 5, *se* não houver uma tupla em TRABALHA_EM (T2) com o mesmo Pnr e o mesmo Cpf daquele da tupla FUNCIONARIO em consideração na consulta externa. Se não existir tal tupla, selecionamos a tupla FUNCIONARIO. A forma de C3B combina com a reformulação da Consulta 3 a seguir: selecionar cada funcionário de modo que não exista um projeto controlado pelo departamento 5 em que o funcionário não trabalha. Isso corresponde ao modo como escreveremos essa consulta no cálculo relacional de tupla (ver Seção 6.6.7).

Existe outra função em SQL, UNIQUE(C), que retorna TRUE se não houver tuplas duplicadas no resultado da consulta C; caso contrário, ela retorna FALSE. Isso pode ser usado para testar se o resultado de uma consulta aninhada é um conjunto ou um multiconjunto.

5.1.5 Conjuntos explícitos e renomeação de atributos em SQL

Vimos várias consultas com uma consulta aninhada na cláusula WHERE. Também é possível usar um **conjunto explícito de valores** na cláusula WHERE, em vez de uma consulta aninhada. Esse conjunto é delimitado por parênteses em SQL.

[1] Lembre-se de que EXCEPT é um operador de diferença de conjunto. A palavra-chave MINUS às vezes é usada, por exemplo, no Oracle.

Consulta 17. Recuperar os números do CPF de todos os funcionários que trabalham nos projetos de números 1, 2 ou 3.

```
C17: SELECT   DISTINCT Fcpf
     FROM     TRABALHA_EM
     WHERE    Pnr IN (1, 2, 3);
```

Em SQL, é possível renomear qualquer atributo que apareça no resultado de uma consulta acrescentando o qualificador AS seguido pelo novo nome desejado. Logo, a construção AS pode ser usada para *apelidar* os nomes tanto do atributo quanto da relação, e ele pode ser usado nas cláusulas SELECT e FROM. Por exemplo, a C8A mostra como a consulta C8 da Seção 4.3.2 pode ser ligeiramente alterada para recuperar o último nome de cada funcionário e seu supervisor, enquanto renomeia os atributos resultantes como Nome_funcionario e Nome_supervisor. Os novos nomes aparecerão como cabeçalhos de coluna no resultado da consulta.

```
C8A: SELECT   F.Unome AS Nome_funcionario,
             S.Unome AS Nome_supervisor
     FROM     FUNCIONARIO AS F,
             FUNCIONARIO AS S
     WHERE    F.Cpf_supervisor=S.Cpf;
```

5.1.6 Tabelas de junção em SQL e junções externas (outer joins)

O conceito de uma **tabela de junção** (ou **relação de junção**) foi incorporado na SQL para permitir aos usuários especificar uma tabela resultante de uma operação de junção *na cláusula* FROM de uma consulta. Essa construção pode ser mais fácil de compreender do que misturar todas as condições de seleção e junção na cláusula WHERE. Por exemplo, considere a consulta C1, que recupera o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'. Pode ser mais fácil especificar a junção das relações FUNCIONARIO e DEPARTAMENTO primeiro, e depois selecionar as tuplas e atributos desejados. Isso pode ser escrito em SQL como em C1A:

```
C1A: SELECT   Pnome, Unome, Endereco
     FROM     (FUNCIONARIO JOIN
               DEPARTAMENTO
               ON Dnr=Dnumero)
     WHERE    Dnome='Pesquisa';
```

A cláusula FROM em C1A contém uma única *tabela de junção*. Os atributos dessa tabela são todos os atributos da primeira tabela, FUNCIONARIO, seguidos por todos os atributos da segunda tabela, DEPARTAMENTO. O conceito de uma tabela de junção também permite que o usuário especifique diferentes tipos de junção, como NATURAL JOIN (junção natural), e vários tipos de OUTER JOIN (junção externa). Em uma NATURAL JOIN sobre duas relações *R* e *S*, nenhuma condição de junção é especificada; cria-se uma *condição EQUIJOIN* implícita para *cada par de atributos com o mesmo nome* de *R* e *S*. Cada par de atributos desse tipo é incluído *apenas uma vez* na relação resultante (ver seções 6.3.2 e 6.4.4 para mais detalhes sobre os tipos de operações de junção da álgebra relacional).

Se os nomes dos atributos de junção não forem os mesmos nas relações da base, é possível renomear os atributos de modo que eles combinem, e depois aplicar a NATURAL JOIN. Nesse caso, a construção AS pode ser usada para renomear uma relação e todos os seus atributos na cláusula FROM. Isso é ilustrado em C1B, onde a relação DEPARTAMENTO é renomeada como DEP e seus atributos são renomeados como Dnome, Dnr (para combinar com o nome do atributo de junção desejado Dnr na tabela FUNCIONARIO), Cpf_gerente e Data_incio_gerente. O significado da condição de junção para esse NATURAL JOIN é FUNCIONARIO.Dnr=DEPT.Dnr, porque esse é o único par de atributos com o mesmo nome após a renomeação:

```
C1B: SELECT   Pnome, Unome, Endereco
     FROM     (FUNCIONARIO NATURAL JOIN
              (DEPARTAMENTO AS DEP
              (Dnome, Dnr, Cpf_gerente, Data_inicio_gerente)))
     WHERE    Dnome='Pesquisa';
```

O tipo padrão de junção em uma tabela de junção é chamado de **inner join**, onde uma tupla é incluída no resultado somente se uma tupla combinar na outra relação. Por exemplo, na consulta C8A, somente os funcionários que *possuem um supervisor* são incluídos no resultado; uma tupla FUNCIONARIO cujo valor para Cpf_supervisor é NULL é excluída. Se o usuário exigir que todos os funcionários sejam incluídos, uma OUTER JOIN precisa ser usada explicitamente (veja a definição de OUTER JOIN na Seção 6.4.4). Em SQL, isso é tratado especificando explicitamente a palavra-chave OUTER JOIN em uma tabela de junção, conforme ilustrado em C8B:

```
C8B: SELECT   F.Unome AS Nome_funcionario,
             S.Unome AS Nome_supervisor
     FROM     (FUNCIONARIO AS F LEFT
              OUTER JOIN FUNCIONARIO
              AS S ON F.Cpf_supervisor
              =S.Cpf);
```

Existem diversas operações de junção externa, que discutiremos com mais detalhes na Seção 6.4.4. Em SQL, as opções disponíveis para especificar tabelas de junção incluem INNER JOIN (apenas pares de tuplas que combinam com a condição de junção são recuperadas, o mesmo que JOIN), LEFT OUTER JOIN (toda tupla na tabela da esquerda precisa aparecer no resultado; se ela não tiver uma tupla combinando, ela é preenchida com valores NULL para os atributos da tabela da direita), RIGHT OUTER JOIN (toda tupla na tabela da direita precisa aparecer no resultado; se ela não tiver uma tupla combinando, ela é preenchida com valores NULL para os atributos da tabela da esquerda) e FULL OUTER JOIN. Nas três últimas opções a palavra-chave OUTER pode ser omitida. Se os atributos de junção tiverem o mesmo nome, também é possível especificar a variação de junção natural das junções externas usando a palavra-chave NATURAL antes da operação (por exemplo, NATURAL LEFT OUTER JOIN). A palavra-chave CROSS JOIN é usada para especificar a operação PRODUTO CARTESIANO (ver Seção 6.2.2), embora isso só deva ser feito com o máximo de cuidado, pois gera todas as combinações de tuplas possíveis.

Também é possível *aninhar* especificações de junção; ou seja, uma das tabelas em uma junção pode ela mesma ser uma tabela de junção. Isso permite a especificação da junção de três ou mais tabelas como uma única tabela de junção, o que é chamado de **junção múltipla**. Por exemplo, a C2A é um modo diferente de especificar a consulta C2, da Seção 4.3.1, usando o conceito de uma tabela de junção:

```
C2A: SELECT  Projnumero, Dnum, Unome,
             Endereco, Datanasc
     FROM    ((PROJETO JOIN DEPARTAMENTO
             ON Dnum=Dnumero) JOIN
             FUNCIONARIO ON
             Cpf_gerente =Cpf)
     WHERE   Projlocal='Mauá';
```

Nem todas as implementações de SQL empregaram a nova sintaxe das tabelas de junção. Em alguns sistemas, uma sintaxe diferente foi usada para especificar junções externas usando os operadores de comparação +=, =+ e +=+ para a junção externa esquerda, direta e completa, respectivamente, ao especificar a condição de junção. Por exemplo, essa sintaxe está disponível no Oracle. Para especificar a junção externa esquerda na C8B usando essa sintaxe, poderíamos escrever a consulta C8C, da seguinte forma:

```
C8C: SELECT  F.Unome, S.Unome
     FROM    FUNCIONARIO F,
             FUNCIONARIO S
     WHERE   F.Cpf_supervisor += S.Cpf;
```

5.1.7 Funções de agregação em SQL

Na Seção 6.4.2, apresentaremos o conceito de uma função de agregação como uma operação da álgebra relacional. As **funções de agregação** são usadas para resumir informações de várias tuplas em uma síntese de tupla única. O **agrupamento** é usado para criar subgrupos de tuplas antes do resumo. O agrupamento e a agregação são exigidos em muitas aplicações de banco de dados, e apresentaremos seu uso na SQL por meio de exemplos. Existem diversas funções de agregação embutidas: **COUNT**, **SUM**, **MAX**, **MIN** e **AVG**.[2] A função COUNT retorna o número de tuplas ou valores, conforme especificado em uma consulta. As funções SUM, MAX, MIN e AVG podem ser aplicadas a um conjunto ou multiconjunto de valores numéricos e retornam, respectivamente, a soma, o valor máximo, o valor mínimo e a média desses valores. Essas funções podem ser usadas na cláusula SELECT ou em uma cláusula HAVING (que apresentaremos mais adiante). As funções MAX e MIN também podem ser usadas com atributos que possuem domínios não numéricos, se os valores do domínio tiverem uma *ordenação total* entre si.[3] Vamos ilustrar o uso dessas funções com algumas consultas.

Consulta 19. Achar a soma dos salários de todos os funcionários, o salário máximo, o salário mínimo e a média dos salários.

```
C19: SELECT  SUM (Salario), MAX (Salario),
             MIN (Salario), AVG (Salario)
     FROM    FUNCIONARIO;
```

Se quisermos obter os valores das funções para os funcionários de um departamento específico — digamos, o departamento 'Pesquisa' —, podemos escrever a Consulta 20, na qual as tuplas FUNCIONARIO são restringidas pela cláusula WHERE aos funcionários que trabalham para o departamento 'Pesquisa'.

[2] Funções de agregação adicionais para cálculo estatístico mais avançado foram acrescentadas na SQL-99.

[3] Ordenação total significa que, para dois valores quaisquer no domínio, pode ser determinado que um aparece antes do outro na ordem definida; por exemplo, os domínios DATE, TIME e TIMESTAMP possuem ordenações totais em seus valores, assim como as cadeias alfabéticas.

Consulta 20. Achar a soma dos salários de todos os funcionários do departamento 'Pesquisa', bem como o salário máximo, o salário mínimo e a média dos salários nesse departamento.

```
C20: SELECT  SUM (Salario), MAX (Salario),
             MIN (Salario), AVG (Salario)
     FROM    (FUNCIONARIO JOIN
             DEPARTAMENTO ON Dnr=Dnumero)
     WHERE   Dnome='Pesquisa';
```

Consultas 21 e 22 Recuperar o número total de funcionários na empresa (C21) e o número de funcionários no departamento 'Pesquisa' (C22).

```
C21: SELECT  COUNT (*)
     FROM    FUNCIONARIO;

C22: SELECT  COUNT (*)
     FROM    FUNCIONARIO, DEPARTAMENTO
     WHERE   Dnr=Dnumero
             AND Dnome='Pesquisa';
```

Aqui, o asterisco (*) refere-se às *linhas* (tuplas), de modo que COUNT (*) retorna o número de linhas no resultado da consulta. Também podemos usar a função COUNT para contar os valores em uma coluna, em vez de tuplas, como no exemplo a seguir.

Consulta 23. Contar o número de valores de salário distintos no banco de dados.

```
C23: SELECT  COUNT (DISTINCT Salario)
     FROM    FUNCIONARIO;
```

Se escrevermos COUNT(SALARIO) em vez de COUNT (DISTINCT SALARIO) na C23, então os valores duplicados não serão eliminados. Porém, quaisquer tuplas com NULL para SALARIO não serão contadas. Em geral, valores NULL são **descartados** quando as funções de agregação são aplicadas a determinada coluna (atributo).

Os exemplos anteriores resumem *uma relação inteira* (C19, C21, C23) ou um subconjunto selecionado de tuplas (C20, C22), e, portanto, todos produzem tuplas ou valores isolados. Eles ilustram como as funções são aplicadas para recuperar um valor de resumo ou uma tupla de resumo do banco de dados. Essas funções também podem ser usadas nas condições de seleção envolvendo consultas aninhadas. Podemos especificar uma consulta aninhada correlacionada com uma função de agregação, e depois usar a consulta aninhada na cláusula WHERE de uma consulta externa. Por exemplo, para recuperar os nomes de todos os funcionários que têm dois ou mais dependentes (Consulta 5), podemos escrever o seguinte:

```
C5:  SELECT  Unome, Pnome
     FROM    FUNCIONARIO
     WHERE   ( SELECT COUNT (*)
               FROM DEPENDENTE
               WHERE Cpf=Fcpf ) >= 2;
```

A consulta aninhada correlacionada conta o número de dependentes que cada funcionário tem; se for maior ou igual a dois, a tupla do funcionário é selecionada.

5.1.8 Agrupamento: as cláusulas GROUP BY e HAVING

Em muitos casos, queremos aplicar as funções de agregação *a subgrupos de tuplas em uma relação*, na qual os subgrupos são baseados em alguns valores de atributo. Por exemplo, podemos querer achar o salário médio dos funcionários *em cada departamento* ou o número de funcionários que trabalham *em cada projeto*. Nesses casos, precisamos **particionar** a relação em subconjuntos de tuplas (ou **grupos**) não sobrepostos. Cada grupo (partição) consistirá nas tuplas que possuem o mesmo valor de algum(ns) atributo(s), chamado(s) **atributo(s) de agrupamento**. Podemos, então, aplicar a função a cada grupo desse tipo independentemente, para produzir informações de resumo sobre cada grupo. A SQL tem uma cláusula **GROUP BY** para essa finalidade. A cláusula GROUP BY especifica os atributos de agrupamento, que *também devem aparecer na cláusula* SELECT, de modo que o valor resultante da aplicação de cada função de agregação a um grupo de tuplas apareça junto com o valor do(s) atributo(s) de agrupamento.

Consulta 24. Para cada departamento, recuperar o número do departamento, o número de funcionários no departamento e seu salário médio.

```
C24: SELECT    Dnr, COUNT (*), AVG (Salario)
     FROM      FUNCIONARIO
     GROUP BY  Dnr;
```

Na C24, as tuplas FUNCIONARIO são divididas em grupos — cada grupo tendo o mesmo valor para o atributo de agrupamento Dnr. Logo, cada grupo contém os funcionários que trabalham no mesmo departamento. As funções COUNT e AVG são aplicadas a cada grupo de tuplas. Observe que a cláusula SELECT inclui apenas o atributo de agrupamento e as funções de agregação a serem aplicadas a cada grupo de tuplas. A Figura 5.1(a) ilustra como o agrupamento funciona na C24; ela também mostra o resultado da C24.

(a)

Pnome	Minicial	Unome	Cpf	...	Salario	Cpf_supervisor	Dnr
João	B	Silva	12345678966		30.000	33344555587	5
Fernando	T	Wong	33344555587		40.000	88866555576	5
Ronaldo	K	Lima	66688444476		38.000	33344555587	5
Joice	A	Leite	45345345376	...	25.000	33344555587	5
Alice	J	Zelaya	99988777767		25.000	98765432168	4
Jennifer	S	Souza	98765432168		43.000	88866555576	4
André	V	Pereira	98798798733		25.000	98765432168	4
Jorge	E	Brito	88866555576		55.000	NULL	1

Agrupamento de tuplas FUNCIONARIO pelo valor de Dnr

Dnr	Count(*)	Avg(Salario)
5	4	33.250
4	3	31.000
1	1	55.000

Resultado de C24

(b)

Projnome	Projnumero	-	Fcpf	Pnr	Horas
ProdutoX	1		12345678966	1	32,5
ProdutoX	1		45345345376	1	20,0
ProdutoY	2		12345678966	2	7,5
ProdutoY	2		45345345376	2	20,0
ProdutoY	2		33344555587	2	10,0
ProdutoZ	3		66688444476	3	40,0
ProdutoZ	3		33344555587	3	10,0
Informatização	10	-	33344555587	10	10,0
Informatização	10		99988777767	10	10,0
Informatização	10		98798798733	10	35,0
Reorganização	20		33344555587	20	10,0
Reorganização	20		98765432168	20	15,0
Reorganização	20		88866555576	20	NULL
NovosBenefícios	30		98798798733	30	5,0
NovosBenefícios	30		98765432168	30	20,0
NovosBenefícios	30		99988777767	30	30,0

Estes grupos não são selecionados pela condição de HAVING de C26.

Após aplicar a cláusula WHERE, mas antes de aplicar HAVING

Projnome	Projnumero	-	Fcpf	Pnr	Horas
ProdutoY	2		12345678966	2	7,5
ProdutoY	2		45345345376	2	20,0
ProdutoY	2		33344555587	2	10,0
Informatização	10		33344555587	10	10,0
Informatização	10	-	99988777767	10	10,0
Informatização	10		98798798733	10	35,0
Reorganização	20		33344555587	20	10,0
Reorganização	20		98765432168	20	15,0
Reorganização	20		88866555576	20	NULL
NovosBenefícios	30		98798798733	30	5,0
NovosBenefícios	30		98765432168	30	20,0
NovosBenefícios	30		99988777767	30	30,0

Após aplicar a condição da cláusula HAVING

Projnome	Count(*)
ProdutoY	3
Informatização	3
Reorganização	3
NovosBenefícios	3

Resultado de C26 (Projnumero não mostrado)

Figura 5.1
Resultados de GROUP BY e HAVING. (a) C24. (b) C26.

Se houver NULLs no atributo de agrupamento, então um **grupo separado** é criado para todas as tuplas com um *valor* NULL *no atributo de agrupamento*. Por exemplo, se a tabela FUNCIONARIO tivesse algumas tuplas com NULL para o atributo de agrupamento Dnr, haveria um grupo separado para essas tuplas no resultado da C24.

Consulta 25. Para cada projeto, recuperar o número do projeto, o nome do projeto e o número de funcionários que trabalham nesse projeto.

```
C25: SELECT   Projnumero, Projnome, COUNT (*)
     FROM     PROJETO, TRABALHA_EM
     WHERE    Projnumero=Pnr
     GROUP BY Projnumero, Projnome;
```

A C25 mostra como podemos usar uma condição de junção em conjunto com GROUP BY. Neste caso, o agrupamento e as funções são aplicados *após* a junção das duas relações. Às vezes, queremos recuperar os valores dessas funções somente para *grupos que satisfazem certas condições*. Por exemplo, suponha que queremos modificar a Consulta 25 de modo que apenas projetos com mais de dois funcionários apareçam no resultado. A SQL oferece uma cláusula **HAVING**, que pode aparecer em conjunto com uma cláusula GROUP BY, para essa finalidade. A cláusula HAVING oferece uma condição sobre a informação de resumo referente ao grupo de tuplas associado a cada valor dos atributos de agrupamento. Somente os grupos que satisfazem a condição são recuperados no resultado da consulta. Isso é ilustrado pela Consulta 26.

Consulta 26. Para cada projeto *em que mais de dois funcionários trabalham*, recupere o número e o nome do projeto e o número de funcionários que trabalham no projeto.

```
C26: SELECT   Projnumero, Projnome, COUNT (*)
     FROM     PROJETO, TRABALHA_EM
     WHERE    Projnumero=Pnr
     GROUP BY Projnumero, Projnome
     HAVING   COUNT (*) > 2;
```

Observe que, embora as condições de seleção na cláusula WHERE limitem as *tuplas* às quais as funções são aplicadas, a cláusula HAVING serve para escolher *grupos inteiros*. A Figura 5.1(b) ilustra o uso de HAVING e apresenta o resultado da C26.

Consulta 27. Para cada projeto, recupere o número e o nome do projeto e o número de funcionários do departamento 5 que trabalham no projeto.

```
C27: SELECT   Projnumero, Projnome, COUNT (*)
     FROM     PROJETO, TRABALHA_EM,
              FUNCIONARIO
     WHERE    Projnumero=Pnr AND
              Cpf=Fcpf AND Dnr=5
     GROUP BY Projnumero, Projnome;
```

Aqui, restringimos as tuplas na relação (e, portanto, as tuplas em cada grupo) àquelas que satisfazem a condição especificada na cláusula WHERE — a saber, que eles trabalhem no departamento número 5. Observe que precisamos ter um nome de grupo extra quando duas condições diferentes se aplicam (uma para a função de agregação na cláusula SELECT e outra para a função na cláusula HAVING). Por exemplo, suponha que queremos contar o número *total* de funcionários cujos salários são superiores a R$ 40.000 em cada departamento, mas somente para os departamentos em que há mais de cinco funcionários trabalhando. Aqui, a condição (SALARIO > 40.000) se aplica apenas à função COUNT na cláusula SELECT. Suponha que escrevamos a seguinte consulta *incorreta*:

```
SELECT   Dnome, COUNT (*)
FROM     DEPARTAMENTO, FUNCIONARIO
WHERE    Dnumero=Dnr AND Salario>40.000
GROUP BY Dnome
HAVING   COUNT (*) > 5;
```

Ela está incorreta porque selecionará somente departamentos que tenham mais de cinco funcionários *que ganham cada um mais de R$ 40.000* A regra é que a cláusula WHERE é executada primeiro, para selecionar as tuplas individuais ou tuplas de junção; a cláusula HAVING é aplicada depois, para selecionar grupos individuais de tuplas. Logo, as tuplas já estão restritas a funcionários que ganham mais de R$ 40.000 *antes* que a cláusula HAVING seja aplicada. Um modo de escrever essa consulta corretamente é usar uma consulta aninhada, como mostra a Consulta 28.

Consulta 28. Para cada departamento que tem mais de cinco funcionários, recuperar o número do departamento e o número de seus funcionários que estão ganhando mais de R$ 40.000.

```
C28: SELECT Dnumero, COUNT (*)
     FROM   DEPARTAMENTO, FUNCIONARIO
     WHERE  Dnumero=Dnr AND Salario>40.000 AND
            ( SELECT Dnr IN
              FROM FUNCIONARIO
              GROUP BY Dnr
              HAVING COUNT (*) > 5)
```

5.1.9 Discussão e resumo das consultas em SQL

Uma consulta de recuperação em SQL pode consistir em até seis cláusulas, mas somente as duas primeiras — SELECT e FROM — são obrigatórias. A consulta pode se espalhar por várias linhas, e termina com um sinal de ponto e vírgula. Os termos da consulta são separados por espaços, e parênteses podem ser usados para agrupar partes relevantes de uma consulta na forma padrão. As cláusulas são especificadas na seguinte ordem, sendo que os colchetes [...] são opcionais:

SELECT <lista atributo e função>
FROM <lista tabela>
[**WHERE** <condição>]
[**GROUP BY** <atributo(s) de agrupamento>]
[**HAVING** <condição de grupo>]
[**ORDER BY** <lista atributos>];

A cláusula SELECT lista os atributos ou funções a serem recuperadas. A cláusula FROM especifica todas as relações (tabelas) necessárias na consulta, incluindo as relações, mas não aquelas nas consultas aninhadas. A cláusula WHERE especifica as condições para selecionar as tuplas dessas relações, incluindo as condições de junção, se necessário. A GROUP BY especifica atributos de agrupamento, enquanto a HAVING especifica uma condição sobre os grupos selecionados, em vez das tuplas individuais. As funções de agregação embutidas COUNT, SUM, MIN, MAX e AVG são usadas em conjunto com o agrupamento, mas também podem ser aplicadas a todas as tuplas selecionadas em uma consulta sem uma cláusula GROUP BY. Por fim, ORDER BY especifica uma ordem para exibir o resultado de uma consulta.

Para formular consultas de maneira correta, é útil considerar as etapas que definem o *significado* ou a *semântica* de cada consulta. Uma consulta é avaliada *conceitualmente*[4] aplicando primeiro a cláusula FROM (para identificar todas as tabelas envolvidas na consulta ou materializar quaisquer tabelas de junção), seguida pela cláusula WHERE para selecionar e juntar tuplas, e depois por GROUP BY e HAVING. Conceitualmente, ORDER BY é aplicado no final para classificar o resultado da consulta. Se nenhuma das três últimas cláusulas (GROUP BY, HAVING e ORDER BY) for especificada, podemos *pensar conceitualmente* em uma consulta como sendo executada da seguinte forma: para *cada combinação de tuplas* — uma de cada uma das relações especificadas na cláusula FROM —, avaliar a cláusula WHERE; se ela for avaliada como TRUE, colocar os valores dos atributos especificados na cláusula SELECT dessa combinação de tuplas no resultado da consulta. É óbvio que esse não é um modo eficiente de implementar a consulta em um sistema real, e cada SGBD possui rotinas especiais de otimização de consulta para decidir sobre um plano de execução que seja eficiente para ser executado. Discutiremos sobre o processamento e a otimização da consulta no Capítulo 19.

Em geral, existem várias maneiras de especificar a mesma consulta em SQL. Essa flexibilidade na especificação de consultas possui vantagens e desvantagens. A principal vantagem é que os usuários podem escolher a técnica com a qual estão mais acostumados ao especificar uma consulta. Por exemplo, muitas consultas podem ser especificadas com condições de junção na cláusula WHERE, ou usando relações de junção na cláusula FROM, ou com alguma forma de consultas aninhadas e o operador de comparação IN. Alguns usuários podem se sentir mais confiantes usando uma técnica, enquanto outros podem estar mais acostumados à outra. Do ponto de vista do programador e do sistema com relação à otimização da consulta, é preferível escrever uma consulta com o mínimo de aninhamento e de ordenação possível.

A desvantagem de ter várias maneiras de especificar a mesma consulta é que isso pode confundir o usuário, que pode não saber qual técnica usar para especificar tipos particulares de consultas. Outro problema é que pode ser mais eficiente executar uma consulta especificada de uma maneira do que a mesma consulta especificada de uma maneira alternativa. O ideal é que isso não aconteça: o SGBD deve processar a mesma consulta da mesma maneira, independentemente de como ela é especificada. Mas isso é muito difícil na prática, pois cada SGBD possui diferentes métodos para processar consultas específicas de diversas maneiras. Assim, um fardo adicional sobre o usuário é determinar qual das especificações alternativas é a mais eficiente de se executar. O ideal é que o usuário se preocupe apenas em especificar a consulta corretamente, enquanto o SGBD determinaria como executar a consulta de forma eficiente. Na prática, contudo, ajuda se o usuário souber quais tipos de construções em uma consulta são mais dispendiosas para processar do que outras (ver Capítulo 20).

[4] A ordem real da avaliação de consulta depende da implementação; esse é apenas um modo de visualizar conceitualmente uma consulta a fim de formulá-la de maneira correta.

5.2 Especificando restrições como asserções e ações como triggers

Nesta seção, apresentamos dois recursos adicionais da SQL: o comando **CREATE ASSERTION** e o comando **CREATE TRIGGER**. A Seção 5.2.1 discute o CREATE ASSERTION, que pode ser usado para especificar tipos adicionais de restrições que estão fora do escopo das *restrições embutidas do modelo relacional* (chaves primária e única, integridade de entidade e integridade referencial), que apresentamos na Seção 3.2. Essas restrições embutidas podem ser especificadas dentro do comando **CREATE TABLE** da SQL (ver seções 4.1 e 4.2).

Depois, na Seção 5.2.2, apresentamos CREATE TRIGGER, que pode ser usado para especificar ações automáticas que o sistema de banco de dados realizará quando certos eventos e condições ocorrerem. Esse tipo de funcionalidade costuma ser conhecido como **bancos de dados ativos**. Só apresentamos os fundamentos básicos dos **triggers** neste capítulo, e uma discussão mais completa sobre os bancos de dados ativos pode ser encontrada na Seção 26.1.

5.2.1 Especificando restrições gerais como asserções em SQL

Em SQL, os usuários podem especificar restrições gerais — aquelas que não se encaixam em nenhuma das categorias descritas nas seções 4.1 e 4.2 — por meio de **asserções declarativas**, usando o comando CREATE ASSERTION da DDL. Cada asserção recebe um nome de restrição e é especificada por uma condição semelhante à cláusula WHERE de uma consulta SQL. Por exemplo, para especificar a restrição de que *o salário de um funcionário não pode ser maior que o salário do gerente do departamento para o qual o funcionário trabalha* em SQL, podemos escrever a seguinte asserção:

```
CREATE ASSERTION RESTRICAO_SALARIAL
CHECK ( NOT EXISTS
        ( SELECT     *
        FROM    FUNCIONARIO F,
                FUNCIONARIO G,
                DEPARTAMENTO D
        WHERE   F.Salario>G.Salario
                AND F.Dno=D.Dnumero
                AND D. Cpf_gerente=G.Cpf) );
```

O nome de restrição RESTRICAO_SALARIAL é seguido pela palavra-chave CHECK, que é seguida por uma **condição** entre parênteses que precisa ser verdadeira em cada estado do banco de dados para que a asserção seja satisfeita. O nome da restrição pode ser usado mais tarde para se referir à restrição ou para modificá-la ou excluí-la. O SGBD é responsável por garantir que a condição não seja violada. Qualquer condição de cláusula WHERE pode ser usada, mas muitas restrições podem ser especificadas usando o estilo EXISTS e NOT EXISTS das condições em SQL. Sempre que alguma tupla no banco de dados fizer que a condição de um comando ASSERTION seja avaliada como FALSE, a restrição é **violada**. A restrição é **satisfeita** por um estado do banco de dados se *nenhuma combinação de tuplas* nesse estado do banco de dados violar a restrição.

A técnica de uso comum para escrever essas asserções é especificar uma consulta que seleciona quaisquer tuplas *que violam a condição desejada*. Ao incluir essa consulta em uma cláusula NOT EXISTS, a asserção especificará que o resultado dessa consulta precisa ser vazio para que a condição seja sempre TRUE. Assim, uma asserção é violada se o resultado da consulta não for vazio. No exemplo anterior, a consulta seleciona todos os funcionários cujos salários são maiores que o salário do gerente de seu departamento. Se o resultado da consulta não for vazio, a asserção é violada.

Observe que a cláusula CHECK e a condição de restrição também podem ser utilizadas para especificar restrições sobre atributos e domínios *individuais* (ver Seção 4.2.1) e sobre tuplas *individuais* (ver Seção 4.2.4). A principal diferença entre CREATE ASSERTION e as restrições de domínio e de tupla individuais é que as cláusulas CHECK sobre atributos, domínios e tuplas individuais são verificadas na SQL *somente quando as tuplas são inseridas ou atualizadas*. Logo, a verificação de restrição pode ser implementada de maneira mais eficiente pelo SGBD nesses casos. O projetista do esquema deve usar CHECK sobre atributos, domínios e tuplas apenas quando estiver certo de que a restrição *só pode ser violada pela inserção ou atualização de tuplas*. Além disso, o projetista do esquema deve usar CREATE ASSERTION somente em casos em que não é possível usar CHECK sobre atributos, domínios ou tuplas, de modo que verificações simples são implementadas de modo mais eficiente pelo SGBD.

5.2.2 Introdução às triggers em SQL

Outro comando importante em SQL é o CREATE TRIGGER. Em muitos casos, é conveniente especificar um tipo de ação a ser tomada quando certos eventos ocorrem e quando certas condições são satisfeitas. Por exemplo, pode ser útil especificar uma condição que, se violada, faz que algum usuário seja informado dela. Um gerente pode querer ser informado se as despesas de viagem de um funcionário excederem certo limite, recebendo uma mensagem sempre que isso acontecer. A ação que o SGBD deve tomar nesse caso é enviar uma mensagem apropriada a esse usuário. A condição, portanto, é usada para **monitorar**

o banco de dados. Outras ações podem ser especificadas, como executar um procedimento armazenado (*stored procedure*) específico ou disparar outras atualizações. A instrução CREATE TRIGGER é utilizada para implementar essas ações em SQL. Discutiremos sobre triggers (gatilhos) com detalhes na Seção 26.1, quando descreveremos os *bancos de dados ativos*. Aqui, vamos apenas dar um exemplo simples de como os triggers podem ser usadas.

Suponha que queiramos verificar se o salário de um funcionário é maior que o salário de seu supervisor direto no banco de dados EMPRESA (ver figuras 3.5 e 3.6). Vários eventos podem disparar essa regra: inserir um novo registro de funcionário, alterar o salário de um funcionário ou alterar o supervisor de um funcionário. Suponha que a ação a ser tomada seria chamar o procedimento armazenado,[5] que notificará o supervisor. O trigger poderia então ser escrita como em R5, a seguir. Aqui, estamos usando a sintaxe do sistema de banco de dados Oracle.

```
R5: CREATE TRIGGER VIOLACAO_SALARIAL
    BEFORE INSERT OR UPDATE OF SALARIO,
    CPF_SUPERVISOR ON FUNCIONARIO
FOR EACH ROW
    WHEN ( NEW.SALARIO >
          ( SELECT SALARIO FROM FUNCIONARIO
          WHERE CPF = NEW.CPF_SUPERVISOR ) )

          INFORMAR_SUPERVISOR
          (NEW.Cpf_supervisor,
          NEW.Cpf );
```

O trigger recebe o nome VIOLACAO_SALARIAL, que pode ser usada para remover ou desativar o trigger mais tarde. Um trigger típico tem três componentes:

1. O(s) **evento(s):** estes em geral são operações de atualização no banco de dados, aplicadas explicitamente a ele. Neste exemplo, os eventos são: inserir um novo registro de funcionário, alterar o salário de um funcionário ou alterar o supervisor de um funcionário. A pessoa que escreve o trigger precisa garantir que todos os eventos possíveis sejam considerados. Em alguns casos, pode ser preciso escrever mais de um trigger para cobrir todos os casos possíveis. Esses eventos são especificados após a palavra-chave **BEFORE** em nosso exemplo, o que significa que o trigger deve ser executada antes que a operação de disparo seja executada. Uma alternativa é usar a palavra-chave **AFTER**, que especifica que o trigger deve ser executada após a operação especificada no evento ser concluída.
2. A **condição** que determina se a ação da regra deve ser executada: depois que o evento de disparo tiver ocorrido, uma condição *opcional* pode ser avaliada. Se *nenhuma condição* for especificada, a ação será executada uma vez que o evento ocorra. Se uma condição for especificada, ela primeiro é avaliada e, somente *se for avaliada como verdadeira*, a ação da regra será executada. A condição é especificada na cláusula WHEN do trigger.
3. A **ação** a ser tomada: a ação normalmente é uma sequência de instruções em SQL, mas também poderia ser uma transação de banco de dados ou um programa externo que será executado automaticamente. Neste exemplo, a ação é executar o procedimento armazenado INFORMAR_SUPERVISOR.

Os triggers podem ser usados em várias aplicações, como na manutenção da coerência do banco de dados, no monitoramento de atualizações do banco de dados e na atualização de dados derivados automaticamente. Uma discussão mais completa pode ser vista na Seção 26.1.

5.3 Visões (views) — Tabelas virtuais em SQL

Nesta seção, apresentamos o conceito de uma view (visão) em SQL. Mostraremos como as views são especificadas, depois discutiremos o problema de atualizá-las e como elas podem ser implementadas pelo SGBD.

5.3.1 Conceito de uma view em SQL

Uma **view** em terminologia SQL é uma única tabela que é derivada de outras tabelas.[6] Essas outras tabelas podem ser *tabelas da base* ou views previamente definidas. Uma view não necessariamente existe em forma física; ela é considerada uma **tabela virtual**, ao contrário das **tabelas da base** cujas tuplas sempre estão armazenadas fisicamente no banco de dados. Isso limita as possíveis operações de atualização que podem ser aplicadas às views, mas não oferece quaisquer limitações sobre a consulta de uma view.

[5] Supondo que um procedimento externo tenha sido declarado. Discutiremos os procedimentos armazenados no Capítulo 13.

[6] Conforme usado em SQL, o termo *view* é mais limitado do que o termo *view do usuário* discutido nos capítulos 1 e 2, pois esta última possivelmente incluiria muitas relações.

Pensamos em uma view como um modo de especificar uma tabela que precisamos referenciar com frequência, embora ela possa não existir fisicamente. Por exemplo, em relação ao banco de dados EMPRESA da Figura 3.5, podemos emitir frequentemente consultas que recuperam o nome do funcionário e os nomes dos projetos em que o funcionário trabalha. Em vez de ter que especificar a junção das três tabelas FUNCIONARIO, TRABALHA_EM e PROJETO toda vez que emitirmos essa consulta, podemos definir uma view que é especificada como o resultado dessas junções. Depois, podemos emitir consultas sobre a view, que são especificadas como leituras de uma única tabela, em vez de leituras envolvendo duas junções sobre três tabelas. Chamamos as tabelas FUNCIONARIO, TRABALHA_EM e PROJETO de **tabelas de definição** da view.

5.3.2 Especificação das views em SQL

Em SQL, o comando para especificar uma view é CREATE VIEW. A view recebe um nome de tabela (virtual), ou nome de view, uma lista de nomes de atributo e uma consulta para especificar o conteúdo da view. Se nenhum dos atributos da view resultar da aplicação de funções ou operações aritméticas, não temos de especificar novos nomes de atributo para a view, pois eles seriam iguais aos nomes dos atributos das tabelas de definição no caso default. As views em V1 e V2 criam tabelas virtuais, cujos esquemas são ilustrados na Figura 5.2, quando aplicadas ao esquema de banco de dados da Figura 3.5.

```
V1: CREATE VIEW TRABALHA_EM1
    AS SELECT   Pnome, Unome, Projnome,
                Horas
       FROM     FUNCIONARIO, PROJETO,
                TRABALHA_EM
       WHERE    Cpf=Fcpf AND Pnr=Projnumero;
V2: CREATE VIEW DEP_INFO(Dep_nome, Qtd_
                func, Total_sal)
    AS SELECT   Dnome, COUNT (*), SUM
                (Salario)
       FROM     DEPARTAMENTO, FUNCIONARIO
       WHERE    Dnumero=Dnr
       GROUP BY Dnome;
```

Em V1, não especificamos quaisquer novos nomes de atributo para a view TRABALHA_EM1 (embora pudéssemos tê-lo feito); nesse caso, TRABALHA_EM1 *herda* os nomes dos atributos de view das tabelas de definição FUNCIONARIO, PROJETO e TRABALHA_EM. A view V2 especifica explicitamente novos nomes de atributo para a view DEP_INFO, usando uma correspondência um para um entre os atributos especificados na cláusula CREATE VIEW e aqueles especificados na cláusula SELECT da consulta que define a view.

TRABALHA_EM1

Pnome	Unome	Projnome	Horas

DEP_INFO

Dep_nome	Qtd_func	Total_sal

Figura 5.2
Duas views especificadas sobre o esquema de banco de dados da Figura 3.5.

Agora, podemos especificar consultas SQL em uma view — ou tabela virtual — da mesma forma como fazemos consultas envolvendo tabelas da base. Por exemplo, para recuperar o primeiro e o último nome de todos os funcionários que trabalham no projeto 'ProdutoX', podemos utilizar a view TRABALHA_EM1 e especificar a consulta como na CV1:

```
CV1: SELECT   Pnome, Unome
     FROM     TRABALHA_EM1
     WHERE    Projnome='ProdutoX';
```

A mesma consulta exigiria a especificação de duas junções se fosse realizada sobre as relações da base diretamente, uma das principais vantagens de uma view é simplificar a especificação de certas consultas. As views também são usadas como um mecanismo de segurança e autorização (ver Capítulo 24).

Supõe-se que uma view esteja *sempre atualizada*; se modificarmos as tuplas nas tabelas da base sobre as quais a view é definida, esta precisa refletir automaticamente essas mudanças. Logo, a view não é realizada ou materializada no momento de sua *definição*, mas quando *especificamos uma consulta* na view. É responsabilidade do SGBD, e não do usuário, cuidar para que a view mantenha-se atualizada. Discutiremos várias maneiras como o SGBD pode manter uma view atualizada na próxima subseção.

Se não precisarmos mais de uma view, podemos usar o comando DROP VIEW para descartá-la. Por exemplo, para descartarmos a view V1, podemos usar o comando SQL em V1A:

```
V1A:    DROP VIEW   TRABALHA_EM1;
```

5.3.3 Implementação e atualização de view e views em linha

O problema de implementar uma view de forma eficiente para consulta é muito complexo. Duas

técnicas principais foram sugeridas. Uma estratégia, chamada **modificação de consulta**, envolve modificar ou transformar a consulta da view (submetida pelo usuário) em uma consulta nas tabelas da base. Por exemplo, a consulta CV1 seria automaticamente modificada para a seguinte consulta pelo SGBD:

```
SELECT  Pnome, Unome
FROM    FUNCIONARIO, PROJETO, TRABALHA_EM
WHERE   Cpf=Fcpf AND Pnr=Projnumero
        AND Projnome='ProdutoX';
```

A desvantagem dessa técnica é que ela é ineficaz para views definidas por consultas complexas, que são demoradas de se executar, especialmente se várias delas tiverem de ser aplicadas à mesma view em um curto período. A segunda estratégia, chamada **materialização de view**, envolve criar fisicamente uma tabela de view temporária quando a view for consultada pela primeira vez e manter essa tabela na suposição de que outras consultas a view acontecerão em seguida. Nesse caso, uma estratégia eficiente para atualizar automaticamente a tabela da view quando as tabelas de base forem atualizadas deverá ser desenvolvida para que a view esteja sempre atualizada. As técnicas que usam o conceito de **atualização incremental** têm sido desenvolvidas para essa finalidade, nas quais o SGBD pode determinar quais novas tuplas devem ser inseridas, excluídas ou modificadas em uma *tabela de view materializada* quando uma atualização de banco de dados é aplicada *a uma das tabelas da base definidas*. A view geralmente é mantida como uma tabela materializada (armazenada fisicamente), desde que esteja sendo consultada. Se a view não for consultada por certo período, o sistema pode então remover automaticamente a tabela física e recalculá-la do zero quando consultas futuras referenciarem a view.

A atualização das views é complicada e pode ser ambígua. Em geral, uma atualização em uma view definida sobre uma *única tabela* sem quaisquer *funções de agregação* pode ser mapeada para uma atualização sobre a tabela da base sob certas condições. Para uma view que envolve junções (*joins*), uma operação de atualização pode ser mapeada para operações de atualização sobre as relações da base de *múltiplas maneiras*. Logo, com frequência não é possível que o SGBD determine qual das atualizações é intencionada. Para ilustrar os problemas em potencial com a atualização de uma view definida sobre múltiplas tabelas, considere a view TRABALHA_EM1 e suponha que emitamos o comando para atualizar o atributo PROJNOME de 'João Silva' de 'ProdutoX' para 'ProdutoY'. Essa atualização de view aparece em UV1:

```
UV1:  UPDATE  TRABALHA_EM1
      SET     Projnome = 'ProdutoY'
      WHERE   Unome='Silva' AND Pnome='João'
              AND Projnome='ProdutoX';
```

Essa consulta pode ser mapeada para várias atualizações sobre as relações da base para gerar o efeito de atualização desejado sobre a view. Além disso, algumas das atualizações criarão efeitos colaterais adicionais, que afetam o resultado de outras consultas. Por exemplo, aqui estão duas atualizações possíveis (a) e (b), sobre as relações da base correspondentes à operação de atualização de view em UV1:

```
(a): UPDATE  TRABALHA_EM
     SET     Pnr =   ( SELECT  Projnumero
                       FROM    PROJETO
                       WHERE   Projnome=
                               'ProdutoY' )
     WHERE   Fcpf IN ( SELECT  Cpf
                       FROM    FUNCIONARIO
                       WHERE   Unome='Silva'
                               AND
                               Pnome='João' )
             AND
             Pnr =   ( SELECT  Projnumero
                       FROM    PROJETO
                       WHERE   Projnome=
                               'ProdutoX' );
(b):UPDATE PROJETO   SET     Projnome =
                             'ProdutoY'
     WHERE   Projnome = 'ProdutoX';
```

A atualização (a) relaciona 'João Silva' à tupla 'ProdutoY' de PROJETO em vez da tupla 'ProdutoX' de PROJETO e é a atualização provavelmente mais desejada. Porém, (b) também daria o efeito de atualização desejado sobre a view, mas realiza isso alterando o nome da tupla 'ProdutoX' na relação PROJETO para 'ProdutoY'. É muito pouco provável que o usuário que especificou a atualização de view UV1 queira que ela seja interpretada como em (b), pois isso também tem o efeito colateral de alterar todas as tuplas de view com Projnome = 'ProdutoX'.

Algumas atualizações de view podem não fazer muito sentido; por exemplo, modificar o atributo Total_sal da view DEP_INFO não faz sentido porque Total_sal é definido como sendo a soma dos salários de funcionário individuais. Esta solicitação aparece como em UV2:

```
UV2: UPDATE DEP_INFO
     SET    Total_sal=100.000
     WHERE  Dnome='Pesquisa';
```

Um grande número de atualizações sobre as relações da base pode satisfazer essa atualização de view.

Em geral, uma atualização de view é viável quando somente *uma atualização possível* sobre as relações da base pode realizar seu efeito desejado sobre a view. Sempre que uma atualização sobre a view pode ser mapeada para *mais de uma atualização* sobre as relações da base, precisamos ter um certo procedimento para escolher uma das atualizações possíveis como a mais provável. Alguns pesquisadores desenvolveram métodos para escolher a atualização mais provável, enquanto outros preferem deixar que o usuário escolha o mapeamento de atualização desejado durante a definição da view.

Resumindo, podemos fazer as seguintes observações:

- Uma view com uma única tabela de definição é atualizável se seus atributos tiverem a chave primária da relação da base, bem como todos os atributos com a restrição NOT NULL que *não tem* valor default especificado.
- As views definidas sobre múltiplas tabelas usando junções geralmente não são atualizáveis.
- As views definidas usando funções de agrupamento e agregação não são atualizáveis.

Em SQL, a cláusula **WITH CHECK OPTION** precisa ser acrescentada ao final da definição de view se uma view *tiver de ser atualizada*. Isso permite que o sistema verifique a possibilidade de atualização da view e planeje uma estratégia de execução para ela.

Também é possível definir uma tabela de view na **cláusula FROM** de uma consulta em SQL. Isso é conhecido como uma **view em linha**. Nesse caso, a view é definida na própria consulta.

5.4 Instruções de alteração de esquema em SQL

Nesta seção, oferecemos uma visão geral dos **comandos de evolução de esquema** disponíveis em SQL, que podem ser usados para alterar um esquema, acrescentando ou removendo tabelas, atributos, restrições e outros elementos dele. Isso pode ser feito enquanto o banco de dados está operando e não exige recompilação do esquema. Certas verificações precisam ser feitas pelo SGBD para garantir que as mudanças não afetarão o restante do banco de dados, tornando-o inconsistente.

5.4.1 O comando DROP

O comando DROP pode ser usado para remover elementos *nomeados* do esquema, como tabelas, domínios ou restrições. Também é possível remover um esquema. Por exemplo, se todo um esquema não for mais necessário, o comando DROP SCHEMA pode ser utilizado. Existem duas opções de *comportamento de drop*: CASCADE e RESTRICT. Por exemplo, para remover o esquema de banco de dados EMPRESA e todas as suas tabelas, domínios e outros elementos, a opção CASCADE é usada da seguinte forma:

```
DROP SCHEMA EMPRESA CASCADE;
```

Se a opção RESTRICT for escolhida no lugar da CASCADE, o esquema é removido somente se ele *não tiver elementos*; caso contrário, o comando DROP não será executado. Para usar a opção RESTRICT, o usuário deve primeiro remover individualmente cada elemento no esquema, depois remover o próprio esquema.

Se uma relação da base dentro de um esquema não for mais necessária, a relação e sua definição podem ser excluídas usando o comando DROP TABLF. Por exemplo, se não quisermos mais manter os dependentes dos funcionários no banco de dados EMPRESA da Figura 4.1, podemos descartar a relação DEPENDENTE emitindo o seguinte comando:

```
DROP TABLE DEPENDENTE CASCADE;
```

Se a opção RESTRICT for escolhida em vez da CASCADE, uma tabela é removida somente se ela *não for referenciada* em quaisquer restrições (por exemplo, por definições de chave estrangeira em outra relação) ou views (ver Seção 5.3), ou por quaisquer outros elementos. Com a opção CASCADE, todas essas restrições, views e outros elementos que referenciam a tabela sendo removida também são excluídos automaticamente do esquema, junto com a própria tabela.

Observe que o comando DROP TABLE não apenas exclui todos os registros na tabela se tiver sucesso, mas também remove a *definição de tabela* do catálogo. Se for desejado excluir apenas os registros, mas deixar a definição de tabela para uso futuro, então o comando DELETE (ver Seção 4.4.2) deve ser usado no lugar de DROP TABLF.

O comando DROP também pode ser empregado para descartar outros tipos de elementos de esquema nomeados, como restrições ou domínios.

5.4.2 O comando ALTER

A definição de uma tabela da base ou de outros elementos de esquema nomeados pode ser alterada usando o comando ALTER. Para as tabelas

da base, as possíveis **ações de alteração de tabela** incluem acrescentar ou remover uma coluna (atributo), alterar uma definição de coluna e acrescentar ou remover restrições de tabela. Por exemplo, para incluir um atributo que mantém as tarefas dos funcionários na relação da base FUNCIONARIO do esquema EMPRESA (ver Figura 4.1), podemos usar o comando

ALTER TABLE EMPRESA.FUNCIONARIO **ADD COLUMN** Tarefa VARCHAR(12);

Ainda podemos inserir um valor para o novo atributo Tarefa para cada tupla individual de FUNCIONARIO. Isso pode ser feito especificando uma cláusula default ou usando o comando UPDATE individualmente sobre cada tupla (ver Seção 4.4.3). Se nenhuma cláusula default for especificada, o novo atributo receberá o valor NULL em todas as tuplas da relação imediatamente após o comando ser executado; logo, a restrição NOT NULL *não é permitida* nesse caso.

Para remover uma coluna, temos de escolher CASCADE ou RESTRICT para o comportamento de remoção. Se CASCADE for escolhido, todas as restrições e views que referenciam a coluna são removidas automaticamente do esquema, junto com a coluna. Se RESTRICT for escolhido, o comando só tem sucesso se nenhuma view ou restrição (ou outro elemento do esquema) referenciar a coluna. Por exemplo, o comando a seguir remove o atributo Endereco da tabela FUNCIONARIO:

ALTER TABLE EMPRESA.FUNCIONARIO **DROP COLUMN** Endereco **CASCADE**;

Também é possível alterar uma definição de coluna removendo uma cláusula default existente ou definindo uma nova cláusula default. Os exemplos a seguir ilustram essa cláusula:

ALTER TABLE EMPRESA.DEPARTAMENTO**ALTER COLUMN** Cpf_gerente **DROP DEFAULT**;

ALTER TABLE EMPRESA.DEPARTAMENTO **ALTER COLUMN** Cpf_gerente **SET DEFAULT** '33344555587';

Também é possível alterar as restrições especificadas sobre uma tabela ao acrescentar ou remover uma restrição nomeada. Para ser removida, uma restrição precisa ter recebido um nome quando foi especificada. Por exemplo, para descartar a restrição chamada CHESUPERFUNC da Figura 4.2 da relação FUNCIONARIO, escrevemos:

ALTER TABLE EMPRESA.FUNCIONARIO **DROP CONSTRAINT** CHESUPERFUNC **CASCADE**;

Quando isso é feito, podemos redefinir uma restrição substituída acrescentando uma nova restrição à relação, se necessário. Isso é especificado usando a palavra-chave **ADD** na instrução ALTER TABLE seguida pela nova restrição, que pode ser nomeada ou não, e pode ser de qualquer um dos tipos de restrição de tabela discutidos.

As subseções anteriores deram uma visão geral dos comandos de evolução de esquema da SQL. Também é possível criar tabelas e views em um esquema de banco de dados usando os comandos apropriados. Existem muitos outros detalhes e opções; o leitor interessado deverá consultar os documentos sobre SQL listados na 'Bibliografia selecionada', ao final deste capítulo.

Resumo

Neste capítulo, apresentamos recursos adicionais da linguagem de banco de dados em SQL. Começamos na Seção 5.1 apresentando recursos mais complexos das consultas de atualização de SQL, incluindo consultas aninhadas, tabelas de junção, junções externas, funções agregadas e agrupamento. Na Seção 5.2, descrevemos o comando CREATE ASSERTION, que permite a especificação de restrições mais gerais sobre o banco de dados, e apresentamos o conceito de triggers e o comando CREATE TRIGGER Depois, na Seção 5.3, descrevemos a facilidade da SQL para definir views no banco de dados. As views também são chamadas de *tabelas virtuais* ou *derivadas*, pois apresentam ao usuário o que parecem ser tabelas; no entanto, as informações nessas tabelas são derivadas de outras definidas anteriormente. A Seção 5.4 introduziu o comando ALTER TABLE da SQL, que é usado para modificar as tabelas e restrições do banco de dados.

A Tabela 5.2 resume a sintaxe (ou estrutura) de diversos comandos SQL. Esse resumo não é abrangente, nem descreve cada construção SQL possível; em vez disso, ele serve como uma referência rápida para os principais tipos de construções disponíveis na SQL. Usamos a notação BNF, onde os símbolos não terminais aparecem entre sinais de <...>, as partes opcionais aparecem entre colchetes [...], as repetições aparecem entre chaves {...} e as alternativas aparecem entre parênteses (... | ... | ...). [7]

[7] A sintaxe completa da SQL é descrita em muitos documentos volumosos, com centenas de páginas.

Tabela 5.2
Resumo da sintaxe da SQL.

CREATE TABLE <nome tabela> (<nome coluna> <tipo coluna> [<restrição atributo>] { , <nome coluna> <tipo coluna> [<restrição atributo>] } [<restrição tabela> { , <restrição tabela> }])
DROP TABLE <nome tabela>
ALTER TABLE <nome tabela> ADD <nome coluna> <tipo coluna>
SELECT [DISTINCT] <lista atributos>
FROM (<nome tabela> { <apelido> } | <tabela de junção>) { , (<nome tabela> { <apelido> } | <tabela de junção>) }
[WHERE <condição>]
[GROUP BY <atributos agrupamento> [HAVING <condição seleção grupo>]]
[ORDER BY <nome coluna> [<ordem>] { , <nome coluna> [<ordem>] }]
<lista atributos> ::= (* | (<nome coluna> | <função> (([DISTINCT] <nome coluna> | *))) { , (<nome coluna> | <função> (([DISTINCT] <nome coluna> | *)))))
<atributos agrupamento> ::= <nome coluna> { , <nome coluna> }
<ordem> ::= (ASC | DESC)
INSERT INTO <nome tabela> [(<nome coluna> { , <nome coluna> })]
(VALUES (<valor constante> , { <valor constante> }) { , (<valor constante> { , <valor constante> }) } | <instrução seleção>)
DELETE FROM <nome tabela>
[WHERE <condição seleção>]
UPDATE <nome tabela>
SET <nome coluna> = < expressão valor> { , <nome coluna> = <expressão valor> }
[WHERE <condição seleção>]
CREATE [UNIQUE] INDEX <nome índice>
ON <nome tabela> (<nome coluna> [<ordem>] { , <nome coluna> [<ordem>] })
[CLUSTER]
DROP INDEX <nome índice>
CREATE VIEW <nome view> [(<nome coluna> { , <nome coluna> })]
AS <instrução seleção>
DROP VIEW <nome view>

NOTA: Os comandos para criar e excluir índices não fazem parte do padrão SQL.

Perguntas de revisão

5.1. Descreva as seis cláusulas na sintaxe de uma consulta de recuperação SQL. Mostre que tipos de construções podem ser especificados em cada uma das seis cláusulas. Quais das seis cláusulas são obrigatórias e quais são opcionais?

5.2. Descreva conceitualmente como uma consulta de recuperação SQL será executada, especificando a ordem conceitual de execução de cada uma das seis cláusulas.

5.3. Discuta como os NULLs são tratados nos operadores de comparação em SQL. Como os NULLs são tratados quando funções de agregação são aplicadas em uma consulta SQL? Como os NULLs são tratados quando existem nos atributos de agrupamento?

5.4. Discuta como cada uma das seguintes construções é usada em SQL e quais são as diversas opções para cada construção. Especifique a utilidade de cada construção.

a. Consultas aninhadas.
b. Tabelas de junção e junções externas.
c. Funções de agregação e agrupamento.
d. Triggers.
e. Asserções e como elas diferem dos triggers.
f. Views e suas formas de atualização.
g. Comandos de alteração de esquema.

Exercícios

5.5. Especifique as seguintes consultas no banco de dados da Figura 3.5 em SQL. Mostre os resultados da consulta se cada uma for aplicada ao banco de dados da Figura 3.6.

a. Para cada departamento cujo salário médio do funcionário seja maior do que R$30.000,00, recupere o nome do departamento e o número de funcionários que trabalham nele.
b. Suponha que queiramos o número de funcionários do sexo *masculino* em cada departamento que ganhe mais de R$30.000,00, em vez de todos os funcionários (como no Exercício 5.5a). Podemos especificar essa consulta em SQL? Por quê?

5.6. Especifique as seguintes consultas em SQL sobre o esquema de banco de dados da Figura 1.2.

a. Recupere os nomes e departamentos de todos os alunos com notas A (alunos que têm uma nota A em todos as disciplinas).
b. Recupere os nomes e departamentos de todos os alunos que não têm uma nota A em qualquer uma das disciplinas.

5.7. Em SQL, especifique as seguintes consultas sobre o banco de dados da Figura 3.5 usando o conceito de consultas aninhadas e conceitos descritos neste capítulo.

a. Recupere os nomes de todos os funcionários que trabalham no departamento que tem o funcionário com o maior salário entre todos os funcionários.
b. Recupere os nomes de todos os funcionários cujo supervisor do supervisor tenha como Cpf o número '88866555576'.
c. Recupere os nomes dos funcionários que ganham pelo menos R$10.000,00 a mais que o funcionário que recebe menos na empresa.

5.8. Especifique as seguintes views em SQL no esquema de banco de dados EMPRESA mostrado na Figura 3.5.

a. Uma view que tem o nome do departamento, nome do gerente e salário do gerente para todo departamento.
b. Uma view que tenha o nome do funcionário, nome do supervisor e salário de cada funcionário que trabalha no departamento 'Pesquisa'.
c. Uma view que tenha o nome do projeto, nome do departamento que o controla, número de funcionários e total de horas trabalhadas por semana em cada projeto.
d. Uma view que tenha o nome do projeto, nome do departamento que o controla, número de funcionários e total de horas trabalhadas por semana no projeto para cada projeto *com mais de um funcionário trabalhando nele*.

5.9. Considere a seguinte view, RESUMO_DEPARTAMENTO, definida sobre o banco de dados EMPRESA da Figura 3.6:

```
CREATE VIEW   RESUMO_DEPARTAMENTO (D,
              C, Total_sal, Media_sal)
AS SELECT     Dnr, COUNT (*), SUM (Salario),
              AVG (Salario)
FROM          FUNCIONARIO
GROUP BY      Dnr;
```

Indique quais das seguintes consultas e atualizações seriam permitidas sobre a view. Se uma consulta ou atualização for permitida, mostre como ficaria a consulta ou atualização correspondente nas relações da base e seu resultado quando aplicado ao banco de dados da Figura 3.6.

a.
```
SELECT  *
FROM    RESUMO_DEPARTAMENTO;
```

b.
```
SELECT  D, C
FROM    RESUMO_DEPARTAMENTO
WHERE   TOTAL_SAL > 100.000;
```

c.
```
SELECT  D, MEDIA_SAL
FROM    RESUMO_DEPARTAMENTO
WHERE   C > ( SELECT C FROM
        RESUMO_DEPARTAMENTO
        WHERE D=4);
```

d.
```
UPDATE  RESUMO_DEPARTAMENTO
SET     D=3
WHERE   D=4;
```

e.
```
DELETE  FROM RESUMO_
        DEPARTAMENTO
WHERE   C > 4;
```

Bibliografia selecionada

Reisner (1977) descreve uma avaliação dos fatores humanos da SEQUEL, precursora da SQL, em que descobriu que os usuários possuem alguma dificuldade de

para especificar corretamente as condições de junção e agrupamento. Date (1984) contém uma crítica da linguagem SQL que indica seus portos fortes e fracos. Date e Darwen (1993) descrevem a SQL2. ANSI (1986) esboça o padrão SQL srcinal. Diversos manuais de fabricante descrevem as características da SQL implementadas em DB2, SQL/DS, Oracle, INGRES, Informix e outros produtos de SGBD comerciais. Melton e Simon (1993) oferecem um tratamento abrangente do padrão ANSI 1992, chamado SQL2. Horowitz (1992) discute alguns dos problemas relacionados à integridade referencial e propagação das atualizações em SQL2.

A questão de atualizações de view é abordada por Dayal e Bernstein (1978), Keller (1982) e Langerak (1990), entre outros. A implementação de view é discutida em Blakeley et al. (1989). Negri et al. (1991) descrevem a semântica formal das consultas SQL.

Existem muitos livros que descrevem vários aspectos da SQL. Por exemplo, duas referências que descrevem a SQL-99 são Melton e Simon (2002) e Melton (2003). Outros padrões SQL — SQL 2006 e SQL 2008 — são descritos em diversos relatórios técnicos; mas não existem referências padrão.

capítulo 6

Álgebra e cálculo relacional

Neste capítulo, discutimos as duas *linguagens formais* para o modelo relacional: a álgebra relacional e o cálculo relacional. Ao contrário, os capítulos 4 e 5 descreveram a *linguagem prática* para o modelo relacional, a saber, o padrão SQL. Historicamente, a álgebra e o cálculo relacional foram desenvolvidos antes da linguagem SQL. De fato, de algumas maneiras, a SQL é baseada nos conceitos tanto da álgebra quanto do cálculo, conforme veremos. Como a maioria dos SGBDs relacionais utiliza a SQL como linguagem, nós a apresentamos primeiro.

Lembre-se, do Capítulo 2, de que um modelo de dados precisa incluir um conjunto de operações para manipular o banco de dados, além dos conceitos do modelo de dados para definir a estrutura e as restrições do banco de dados. Apresentamos as estruturas e as restrições do modelo relacional formal no Capítulo 3. O conjunto básico de operações para o modelo relacional é a **álgebra relacional**. Essas operações permitem que um usuário especifique as solicitações de recuperação básicas como *expressões da álgebra relacional*. O resultado de uma recuperação é uma nova relação, que pode ter sido formada de uma ou mais relações. As operações da álgebra, assim, produzem novas relações, que podem ser manipuladas ainda mais usando operações da mesma álgebra. Uma sequência de operações da álgebra relacional forma uma **expressão da álgebra relacional**, cujo resultado também será uma relação que representa o resultado de uma consulta de banco de dados (ou consulta de recuperação).

A álgebra relacional é muito importante por diversos motivos. Primeiro, ela oferece um alicerce formal para as operações do modelo relacional. Segundo, e talvez mais importante, ela é usada como base para a implementação e otimização de consultas nos módulos de otimização e processamento de consulta, que são partes integrais dos sistemas de gerenciamento de banco de dados relacional (SGBDRs), conforme discutiremos no Capítulo 19. Terceiro, alguns de seus conceitos são incorporados na linguagem de consulta padrão SQL para SGBDRs.

Embora a maioria dos SGBDRs comerciais em uso hoje não ofereça interfaces de usuário para consultas da álgebra relacional, as operações e funções essenciais relacionadas nos sistemas de banco de dados são baseadas nas operações da álgebra relacional. Definiremos essas operações com detalhes nas seções 6.1 a 6.4 deste capítulo.

Embora a álgebra defina um conjunto de operações para o modelo relacional, o **cálculo relacional** oferece uma linguagem *declarativa* de nível mais alto para especificar consultas relacionais. Uma expressão do cálculo relacional gera uma nova relação. Em uma expressão do cálculo relacional, *não existe ordem de operações* para especificar como recuperar o resultado da consulta — somente qual informação o resultado deve conter. Esse é o principal fator de distinção entre a álgebra relacional e o cálculo relacional. O cálculo relacional é importante porque tem uma firme base na lógica matemática e porque a linguagem de consulta padrão (SQL) para SGBDRs tem alguns de seus alicerces em uma variação do cálculo relacional conhecida como cálculo relacional de tupla. [1]

A álgebra relacional normalmente é considerada uma parte integral do modelo de dados relacional. Suas operações podem ser divididas em dois grupos. Um grupo inclui conjunto de operações da teoria de

[1] A SQL se baseia no cálculo relacional de tupla, mas também incorpora algumas das operações da álgebra relacional e suas extensões, conforme ilustrado nos capítulos 4, 5 e 9.

conjunto da matemática; estas são aplicáveis porque cada relação é definida como um conjunto de tuplas no modelo relacional *formal* (ver Seção 3.1). As operações de conjunto incluem UNIÃO, INTERSECÇÃO, DIFERENÇA DE CONJUNTO e PRODUTO CARTESIANO (também conhecida como PRODUTO CRUZADO). O outro grupo consiste em operações desenvolvidas especificamente para bancos de dados relacionais — entre elas estão SELEÇÃO, PROJEÇÃO e JUNÇÃO, entre outras. Primeiro, vamos descrever as operações SELEÇÃO e PROJEÇÃO na Seção 6.1, pois elas são **operações unárias** que operam sobre relações isoladas. Depois, discutimos as operações de conjunto na Seção 6.2. Na Seção 6.3, discutimos JUNÇÃO e outras **operações binárias** complexas, que operam sobre duas tabelas combinando tuplas relacionadas (registros) baseadas em *condições de junção*. O banco de dados relacional EMPRESA, mostrado na Figura 3.6, é usado para nossos exemplos.

Algumas solicitações de banco de dados comuns não podem ser realizadas com as operações srcinais da álgebra relacional, de modo que operações adicionais foram criadas para expressá-las. Estas incluem **funções de agregação** , que são operações que podem *resumir* dados das tabelas, bem como tipos adicionais de operações JUNÇÃO e UNIÃO, conhecidas como JUNÇÃO EXTERNA e UNIÃO EXTERNA. Essas operações, que foram acrescentadas à álgebra relacional devido a sua importância para muitas aplicações de banco de dados, são descritas na Seção 6.4. Oferecemos exemplos da especificação de consultas que usam operações relacionais na Seção 6.5. Algumas dessas mesmas consultas foram utilizadas nos capítulos 4 e 5. Ao usar os mesmos números de consulta neste capítulo, o leit or poderá comparar como as mesmas consultas são escritas nas diversas linguagens de consulta.

Nas seções 6.6 e 6.7, descrevemos a outra linguagem formal principal para bancos de dados relacionais, o **cálculo relacional**. Existem duas variações do cálculo relacional. O cálculo relacional de *tupla* é descrito na Seção 6.6, e o cálculo relacional de *domínio* é descrito na Seção 6.7. Algumas das construções SQL discutidas nos capítulos 4 e 5 são baseadas no cálculo relacional de tupla. O cálculo relacional é uma linguagem formal, fundamentada no ramo da lógica matemática chamado de cálculo de predicado.[2] No cálculo relacional de tupla, variáveis estendem-se por *tuplas*, enquanto no cálculo relacional de domínio, variáveis estendem-se por *domínios* (valores) de atributos. No Apêndice C, oferecemos uma visão geral da linguagem Query-By-Example (QBE), que é uma linguagem relacional gráfica de uso facilitado, baseada no cálculo relacional de domínio. No final do capítulo há um resumo.

Para o leitor interessado em uma introdução menos detalhada às linguagens relacionais formais, as seções 6.4, 6.6 e 6.7 podem ser puladas.

6.1 Operações relacionais unárias: SELEÇÃO e PROJEÇÃO

6.1.1 A operação SELEÇÃO

A operação SELEÇÃO é usada para escolher um *subconjunto* das tuplas de uma relação que satisfaça uma **condição de seleção**.[3] Pode-se considerar que a operação SELEÇÃO seja um *filtro* que mantém apenas as tuplas que satisfazem uma condição qualificadora. Como alternativa, podemos considerar que essa operação *restringe* as tuplas em uma relação para apenas aquelas que satisfazem a condição. A operação SELEÇÃO também pode ser visualizada como uma *partição horizontal* da relação em dois conjuntos de tuplas — aquelas que satisfazem a condição e são selecionadas, e aquelas que não satisfazem a condição e são descartadas. Por exemplo, para selecionar a tupla FUNCIONARIO cujo departamento é 4, ou aquelas cujo salário é maior do que R$ 30.000,00, podemos especificar individualmente cada uma dessas duas condições com uma operação SELEÇÃO da seguinte maneira:

$$\sigma_{Dnr=4}(\text{FUNCIONARIO})$$
$$\sigma_{Salario>30.000}(\text{FUNCIONARIO})$$

Em geral, a operação SELEÇÃO é indicada por

$$\sigma_{<\text{condição seleção}>}(R)$$

onde o símbolo σ (sigma) é usado para indicar o operador SELEÇÃO e a condição de seleção é uma expressão booleana (condição) especificada nos atributos da relação *R*. Observe que *R* costuma ser uma *expressão da álgebra relacional* cujo resultado é uma relação — a mais simples expressão desse tipo é apenas o nome de uma relação de banco de dados. A relação resultante da operação SELEÇÃO tem os *mesmos atributos* de *R*.

A expressão booleana especificada em <condição seleção> é composta de uma série de **cláusulas** da forma

[2] Neste capítulo, nenhuma familiaridade com o cálculo de predicado de primeira ordem — que lida com variáveis e valores quantificados — é assumida.

[3] A operação SELEÇÃO é diferente da cláusula SELECT da SQL. A operação SELEÇÃO escolhe tuplas de uma tabela, e às vezes é chamada de operação RESTRINGIR ou FILTRAR.

<nome atributo> <op comparação> <valor constante>

ou

<nome atributo> <op comparação> <nome atributo>

onde <nome atributo> é o nome de um atributo de R, <op comparação> em geral é um dos operadores $\{=, <, \leq, >, \geq, \neq\}$ e <valor constante> é um valor constante do domínio do atributo. As cláusulas podem ser conectadas pelos operadores booleanos padrão *and*, *or* e *not* para formar uma condição de seleção geral. Por exemplo, para selecionar as tuplas para todos os funcionários que ou trabalham no departamento 4 e ganham mais de R$ 25.000,00 por ano, ou trabalham no departamento 5 e ganham mais de R$ 30.000,00, podemos especificar a seguinte operação SELEÇÃO:

$$\sigma_{(\text{Dnr}=4\ \textbf{AND}\ \text{Salario}>25.000)\ \textbf{OR}\ (\text{Dnr}=5\ \textbf{AND}\ \text{Salario}>30.000)}(\text{FUNCIONARIO})$$

O resultado é mostrado na Figura 6.1(a).

Observe que todos os operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$ podem ser aplicados aos atributos cujos domínios são *valores ordenados*, como domínios numéricos ou de data. Os domínios de cadeias de caracteres também são considerados ordenados com base na ordem alfabética dos caracteres. Se o domínio de um atributo for um conjunto de *valores desordenados*, então somente os operadores de comparação no conjunto $\{=, \neq\}$ podem ser usados. Um exemplo de domínio desordenado é o domínio Cor = { 'vermelho', 'azul', 'verde', 'branco', 'amarelo', ...}, onde nenhuma ordem é especificada entre as diversas cores. Alguns domínios permitem tipos adicionais de operadores de comparação; por exemplo, um domínio de cadeias de caracteres pode permitir o operador de comparação SUBSTRING_OF.

Em geral, o resultado de uma operação SELEÇÃO pode ser determinado da seguinte forma. A <condição seleção> é aplicada independentemente para cada *tupla individual t* em R. Isso é feito substituindo cada ocorrência de um atributo A_i na condição de seleção por seu valor na tupla $t[A_i]$. Se a condição for avaliada como TRUE, então a tupla t é **selecionada**. Todas as tuplas selecionadas aparecem no resultado da operação SELEÇÃO. As condições booleanas AND, OR e NOT têm sua interpretação normal, da seguinte forma:

- (cond1 **AND** cond2) é TRUE se (cond1) e (cond2) forem TRUE; caso contrário, é FALSE.
- (cond1 **OR** cond2) é TRUE se (cond1) ou (cond2) ou ambas forem TRUE; caso contrário, é FALSE.
- (**NOT** cond) é TRUE se cond é FALSE; caso contrário, é FALSE.

O operador SELEÇÃO é **unário**; ou seja, ele é aplicado a uma única relação. Além do mais, a operação de seleção é aplicada a *cada tupla individualmente*; logo, as condições de seleção não podem envolver mais de uma tupla. O **grau** da relação resultante de uma operação SELEÇÃO — seu número de atributos — é o mesmo que o grau de R. O número de tuplas na relação resultante é sempre *menor ou igual ao* número de tuplas em R. Ou seja, $|\sigma_c(R)| \leq |R|$ para qualquer condição C. A fração de tuplas selecionada por uma condição de seleção é conhecida como **seletividade** da condição.

Observe que a operação SELECT é **comutativa**; ou seja,

$$\sigma_{<\text{cond1}>}(\sigma_{<\text{cond2}>}(R)) = \sigma_{<\text{cond2}>}(\sigma_{<\text{cond1}>}(R))$$

Portanto, uma sequência de SELEÇÃO pode ser aplicada em qualquer ordem. Além disso, sempre podemos combinar uma **cascata** (ou **sequência**) de operações SELEÇÃO a uma única operação SELEÇÃO, com uma condição conjuntiva (AND); ou seja,

$$\sigma_{<\text{cond1}>}(\sigma_{<\text{cond2}>}(\ldots(\sigma_{<\text{condn}>}(R))\ \ldots)) =$$
$$\sigma_{<\text{cond1}>\ \textbf{AND}\ <\text{cond2}>\ \textbf{AND}\ \ldots\ \textbf{AND}\ <\text{condn}>}(R)$$

Em SQL, a condição SELEÇÃO normalmente é especificada na cláusula WHERE de uma consulta. Por exemplo, a operação a seguir:

$$\sigma_{\text{Dnr}=4\ \textbf{AND}\ \text{Salario}>25.000}(\text{FUNCIONARIO})$$

corresponderia à seguinte consulta SQL:

```
SELECT  *
FROM    FUNCIONARIO
WHERE   Dnr=4 AND Salario>25.000;
```

6.1.2 A operação PROJEÇÃO

Se pensarmos em uma relação como uma tabela, a operação SELEÇÃO escolhe algumas das *linhas* da tabela enquanto descarta outras linhas. A operação **PROJEÇÃO**, por sua vez, seleciona certas *colunas* da tabela e descarta as outras. Se estivermos interessados apenas em certos atributos de uma relação, usamos a operação PROJEÇÃO para *projetar* a relação apenas por esses atributos. Portanto, o resultado da operação PROJEÇÃO pode ser visualizado como uma *partição vertical* da relação em duas relações: uma tem as colunas (atributos) necessárias e contém o resultado da operação, e a outra contém as colunas

descartadas. Por exemplo, para listar último nome e primeiro nome e salário de cada funcionário, podemos usar a operação PROJEÇÃO da seguinte forma:

$$\pi_{\text{Unome, Pnome, Salario}}(\text{FUNCIONARIO})$$

A relação resultante aparece na Figura 6.1(b). A forma geral da operação PROJEÇÃO é

$$\pi_{\text{<lista atributos>}}(R)$$

onde π (pi) é o símbolo usado para representar a operação PROJEÇÃO, e <lista atributos> é a sublista desejada de atributos da relação R. Mais uma vez, observe que R, em geral, é uma *expressão da álgebra relacional* cujo resultado é uma relação, que no caso mais simples é apenas o nome de uma relação do banco de dados. O resultado da operação PROJEÇÃO tem apenas os atributos especificados em <lista atributos> *na mesma ordem em que eles aparecem na lista*. Logo, seu **grau** é igual ao número de atributos em <lista atributos>.

Se a lista de atributos inclui apenas atributos não chave de R, tuplas duplicadas provavelmente ocorrerão. A operação PROJEÇÃO *remove quaisquer tuplas duplicadas*, de modo que o resultado dessa operação é um conjunto de tuplas distintas, e, portanto, uma relação válida. Isso é conhecido como **eliminação de duplicatas**. Por exemplo, considere a seguinte operação PROJEÇÃO:

$$\pi_{\text{Sexo, Salario}}(\text{FUNCIONARIO})$$

O resultado é mostrado na Figura 6.1(c). Observe que a tupla <'F', 25.000> só aparece uma vez na Figura 6.1(c), embora essa combinação de valores apareça duas vezes na relação FUNCIONARIO. A eliminação de duplicatas envolve a classificação ou alguma outra técnica para detectar duplicatas e, portanto, aumenta o processamento. Se as duplicatas não fossem eliminadas, o resultado seria um **multiconjunto** ou **bag** de tuplas, em vez de um conjunto. Isso não era permitido no modelo relacional formal, mas pode ocorrer na SQL (ver Seção 4.3).

O número de tuplas em uma relação resultante de uma operação PROJEÇÃO é sempre menor ou igual ao número de tuplas em R. Se a lista de projeção é uma superchave de R — ou seja, inclui alguma chave de R —, a relação resultante tem o *mesmo número* de tuplas que R. Além do mais,

(a)

Pnome	Minicial	Unome	Cpf	Datanasc	Endereco	Sexo	Salario	Cpf_supervisor	Dnr
Fernando	T	Wong	33344555587	08-12-1955	RuadaLapa,34, São Paulo, SP	M	40.000	88866555576	5
Jennifer	S	Souza	98765432168	20-06-1941	Av.ArthurdeLima, 54, Santo André, SP	F	43.000	88866555576	4
Ronaldo	K	Lima	66688444476	15-09-1962	RuaRebouças,65, Piracicaba, SP	M	38.000	33344555587	5

(b)

Unome	Pnome	Salario
Silva	João	30.000
Wong	Fernando	40.000
Zelaya	Alice	25.000
Souza	Jennifer	43.000
Lima	Ronaldo	38.000
Leite	Joice	25.000
Pereira	André	25.000
Brito	Jorge	55.000

(c)

Sexo	Salario
M	30.000
M	40.000
F	25.000
F	43.000
M	38.000
M	25.000
M	55.000

Figura 6.1

Resultados das operações SELEÇÃO e PROJEÇÃO. (a) $\sigma_{\text{(Dnr=4 AND Salario>25.000) OR (Dnr=5 AND Salario>30.000)}}(\text{FUNCIONARIO})$. (b) $\pi_{\text{Unome, Pnome, Salario}}(\text{FUNCIONARIO})$. (c) $\pi_{\text{Sexo, Salario}}(\text{FUNCIONARIO})$.

$$\pi_{<lista1>}(\pi_{<lista2>}(R)) = \pi_{<lista1>}(R)$$

desde que a <lista2> contenha os atributos em <lista1>; caso contrário, o lado esquerdo é uma expressão incorreta. Também vale a pena notar que a comutatividade *não* é mantida em PROJEÇÃO.

Em SQL, a lista de atributos de PROJEÇÃO é especificada na cláusula SELEÇÃO de uma consulta. Por exemplo, a operação a seguir:

$$\pi_{Sexo, Salario}(FUNCIONARIO)$$

corresponderia à seguinte consulta SQL:

SELECT **DISTINCT** Sexo, Salario
FROM FUNCIONARIO

Observe que, se removermos a palavra-chave **DISTINCT** dessa consulta SQL, então as duplicatas não serão eliminadas. Essa opção não está disponível na álgebra relacional formal.

6.1.3 Sequências de operações e a operação RENOMEAR

As relações mostradas na Figura 6.1, que representam resultados de operação, não possuem nome. Em geral, para a maioria das consultas, precisamos aplicar várias operações da álgebra relacional uma após a outra. Podemos escrevê-las como uma única **expressão da álgebra relacional** aninhando as operações, ou aplicar uma operação de cada vez e criar relações de resultado intermediário. No último caso, temos de dar nomes às relações que mantêm os resultados intermediários. Por exemplo, para recuperar o primeiro nome, sobrenome e salário de todos os funcionários que trabalham no departamento número 5, devemos aplicar uma operação SELEÇÃO e uma PROJEÇÃO. Podemos escrever uma única expressão da álgebra relacional, também conhecida como uma **expressão em linha**, da seguinte forma:

$$\pi_{Pnome, Unome, Salario}(\sigma_{Dnr=5}(FUNCIONARIO))$$

A Figura 6.2(a) mostra o resultado dessa expressão da álgebra relacional em linha. Como alternativa, podemos explicitamente mostrar a sequência de operações, dando um nome a cada relação intermediária, da seguinte forma:

FUNCS_DEPT5 $\leftarrow \sigma_{Dnr=5}$(FUNCIONARIO)
RESULTADO $\leftarrow \pi_{Pnome, Unome, Salario}$(FUNCS_DEP5)

Às vezes, é mais simples desmembrar uma sequência complexa de operações especificando relações de resultado intermediário do que escrever uma única expressão da álgebra relacional. Também podemos usar essa técnica para **renomear** os atributos nas relações intermediárias e de resultado. Isso pode ser útil em conexão com operações mais complexas, como UNIÃO e JUNÇÃO, conforme veremos. Para renomear os atributos em uma relação, simplesmente listamos os novos nomes de atributo entre parênteses, como no exemplo a seguir:

TEMP $\leftarrow \sigma_{Dnr=5}$(FUNCIONARIO)

R(Primeiro_nome, Ultimo_nome, Salario) $\leftarrow$
$\pi_{Pnome, Unome, Salario}$(TEMP)

Essas duas operações são ilustradas na Figura 6.2(b).

Se nenhuma renomeação for aplicada, os nomes dos atributos na relação resultante de uma operação SELEÇÃO serão iguais aos da relação srcinal e na mesma ordem. Para uma operação PROJEÇÃO sem renomeação, a relação resultante tem os mesmos nomes de atributo daqueles na lista de projeção e na mesma ordem em que eles aparecem na lista.

Também podemos definir uma operação **RENOMEAR** formal — que pode renomear o nome da relação ou os nomes de atributo, ou ambos — como um operador unário. A operação RENOMEAR geral, quando aplicada à relação R de grau n, é indicada por qualquer uma das três formas a seguir:

$$\rho_{S(B1, B2, ..., Bn)}(R) \quad \text{ou} \quad \rho_{S}(R) \quad \text{ou} \quad \rho_{(B1, B2, ..., Bn)}(R)$$

onde o símbolo ρ (rho) é usado para indicar o operador RENOMEAR, S é o nome da nova relação, e B_1, B_2, ..., B_n são os novos nomes de atributo. A primeira expressão renomeia tanto a relação quanto seus atributos, a segunda renomeia apenas a relação, e a terceira renomeia apenas os atributos. Se os atributos de R são (A_1, A_2, ..., A_n) nessa ordem, então cada A_i é renomeado como B_i.

Em SQL, uma única consulta costuma representar uma expressão complexa da álgebra relacional. A renomeação em SQL é obtida por apelidos usando **AS**, como no exemplo a seguir:

SELECT F.Pnome **AS** Primeiro_nome, F.Unome
AS Ultimo_nome, F.Salario **AS** Salario
FROM FUNCIONARIO **AS** F
WHERE F.Dnr=5,

(a)

Pnome	Unome	Salario
João	Silva	30.000
Fernando	Wong	40.000
Ronaldo	Lima	38.000
Joice	Leite	25.000

(b)

TEMP

Pnome	Minicial	Unome	Cpf	Datanasc	Endereco	Sexo	Salario	Cpf_supervisor	Dnr
João	B	Silva	12345678966	09-01-1965	RuadasFlores,751, São Paulo, SP	M	30.000	33344555587	5
Fernando	T	Wong	33344555587	08-12-1955	RuadaLapa,34, São Paulo, SP	M	40.000	88866555576	5
Ronaldo	K	Lima	66688444476	15-09-1962	RuaRebouças,65, Piracicaba, SP	M	38.000	33344555587	5
Joice	A	Leite	45345345376	31-07-1972	Av.LucasObes,74, São Paulo, SP	F	25.000	33344555587	5

R

Primeiro_nome	Ultimo_nome	Salario
João	Silva	30.000
Fernando	Wong	40.000
Ronaldo	Lima	38.000
Joice	Leite	25.000

Figura 6.2

Resultados de uma sequência de operações. (a) $\pi_{Pnome,\ Unome,\ Salario}$ ($\sigma_{Dnr=5}$(FUNCIONARIO)). (b) Usando relações intermediárias e renomeando atributos.

6.2 Operações de álgebra relacional com base na teoria dos conjuntos

6.2.1 As operações UNIÃO, INTERSECÇÃO e SUBTRAÇÃO

O próximo grupo de operações da álgebra relacional são as operações matemáticas padrão sobre conjuntos. Por exemplo, para recuperar os números de Cadastro de Pessoa Física de todos os funcionários que ou trabalham no departamento 5 ou supervisionam diretamente um funcionário que trabalha no departamento 5, podemos usar a operação UNIÃO da seguinte forma:[4]

FUNCS_DEP5 ← $\sigma_{Dnr=5}$(FUNCIONARIO)
RESULTADO1 ← π_{Cpf}(FUNCS_DEP5)
RESULTADO2(Cpf) ← $\pi_{Cpf_supervisor}$(FUNCS_DEP5)
RESULTADO ← RESULTADO1 ∪ RESULTADO2

A relação RESULTADO1 tem o Cpf de todos os funcionários que trabalham no departamento 5, enquanto RESULTADO2 tem o Cpf de todos os funcionários que supervisionam diretamente um funcionário que trabalha no departamento 5. A operação UNION produz as tuplas que estão ou no RESULTADO1 ou no RESULTADO2 ou em ambos (ver Figura 6.3), enquanto elimina quaisquer duplicatas. Assim, o valor de Cpf '33344555587' aparece apenas uma vez no resultado.

[4] Como uma única expressão da álgebra relacional, isso se torna Resultado ← π_{Cpf} ($\sigma_{Dnr=5}$ (FUNCIONARIO)) ∪ $\pi_{Cpf_supervisor}$ ($\sigma_{Dnr=5}$(FUNCIONARIO)).

RESULTADO1

Cpf
12345678966
33344555587
66688444476
45345345376

RESULTADO2

Cpf
33344555587
88866555576

RESULTADO

Cpf
12345678966
33344555587
66688444476
45345345376
88866555576

Figura 6.3
Resultado da operação UNIÃO

RESULTADO ← RESULTADO1 ∪ RESULTADO2

Várias operações de teoria de conjunto são usadas para mesclar os elementos de dois conjuntos de diversas maneiras, incluindo **UNIÃO**, **INTERSECÇÃO** e **DIFERENÇA DE CONJUNTO** (também chamada **SUBTRAÇÃO** ou **EXCETO**). Estas são operações **binárias**; ou seja, cada uma é aplicada a dois conjuntos (de tuplas). Quando essas operações são adaptadas aos bancos de dados relacionais, as duas relações sobre as quais qualquer uma dessas três operações são aplicadas precisam ter o mesmo **tipo de tuplas**; essa condição é chamada de *compatibilidade de união* ou *compatibilidade de tipo*. Duas relações $R(A_1, A_2, ..., A_n)$ e $S(B_1, B_2, ..., B_n)$ são consideradas **compatíveis na união** (ou **compatíveis no tipo**) se tiverem o mesmo grau n e se $\text{dom}(A_i) = \text{dom}(B_i)$ para $1 \leq i \leq n$. Isso significa que as duas relações têm o mesmo número de atributos e cada par correspondente de atributos tem o mesmo domínio.

Podemos definir as três operações UNIÃO, INTERSECÇÃO e SUBTRAÇÃO sobre duas relações compatíveis na união, R e S, como se segue:

- UNIÃO: O resultado dessa operação, indicada por $R \cup S$, é uma relação que inclui todas as tuplas que estão em R ou em S ou tanto em R quanto em S. As tuplas duplicadas são eliminadas.
- INTERSECÇÃO: O resultado dessa operação, indicada por $R \cap S$, é uma relação que inclui todas as tuplas que estão tanto em R quanto em S.
- DIFERENÇA DE CONJUNTO (ou SUBTRAÇÃO): O resultado dessa operação, indicada por $R - S$, é uma relação que inclui todas as tuplas que estão em R, mas não em S.

Adotaremos a convenção de que a relação resultante tem os mesmos nomes de atributo da *primeira* relação R. Sempre é possível renomear os atributos no resultado usando o operador de renomeação.

A Figura 6.4 ilustra as três operações. As relações ALUNO e PROFESSOR na Figura 6.4(a) são compatíveis na união e suas tuplas representam os nomes dos alunos e dos professores, respectivamente. O resultado da operação UNIÃO na Figura 6.4(b) mostra os nomes de todos os alunos e professores. Observe que tuplas duplicadas aparecem apenas uma vez no resultado. O resultado da operação INTERSECÇÃO (Figura 6.4(c)) inclui apenas aqueles que são tanto alunos quanto professores.

Observe que tanto UNIÃO quanto INTERSECÇÃO são *operações comutativas*; ou seja,

$$R \cup S = S \cup R \quad \text{e} \quad R \cap S = S \cap R$$

Tanto UNIÃO quanto INTERSECÇÃO podem ser tratadas como operações *n*-árias, aplicáveis a qualquer quantidade de relações, pois ambas também são *operações associativas*; ou seja,

$$R \cup (S \cup T) = (R \cup S) \cup T \text{ e } (R \cap S) \cap T = R \cap (S \cap T)$$

A operação SUBTRAÇÃO *não é comutativa*; ou seja, em geral,

$$R - S \neq S - R$$

A Figura 6.4(d) mostra os nomes dos alunos que não são professores, e a Figura 6.4(e) mostra os nomes dos professores que não são alunos.

Observe que INTERSECÇÃO pode ser expressa em termos de união e diferença de conjunto, da seguinte forma:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

Em SQL, existem três operações — UNION, INTERSECT e EXCEPT — que correspondem às operações de conjunto união, intersecção e subtração. Além disso, existem operações de multiconjunto (UNION ALL, INTERSECT ALL e EXCEPT ALL) que não eliminam duplicatas (ver Seção 4.3.4).

6.2.2 A operação PRODUTO CARTESIANO ou PRODUTO CRUZADO

Em seguida, discutimos a operação **PRODUTO CARTESIANO** **PRODUTO CRUZADO** também chamada **JUNÇÃO CRUZADA** —, que é indicada por ×. Esta também é uma operação de conjunto binária, mas as relações sobre as quais ela é aplicada *não* precisam ser compatíveis na união. Em sua forma binária, esta operação de conjunto produz um novo elemento combinando cada membro (tupla) de uma relação (conjunto) com cada membro (tupla) da outra relação (conjunto). Em geral,

(a) ALUNO

Pn	Un
Susana	Yao
Ronaldo	Lima
José	Gonçalves
Barbara	Pires
Ana	Tavares
Jonas	Wang
Ernesto	Gilberto

PROFESSOR

Pnome	Unome
João	Silva
Ricardo	Braga
Susana	Yao
Francisco	Leme
Ronaldo	Lima

(b)

Pn	Un
Susana	Yao
Ronaldo	Lima
José	Golçalves
Barbara	Pires
Ana	Tavares
Jonas	Wang
Ernesto	Gilberto
João	Silva
Ricardo	Braga
Francisco	Leme

(c)

Pn	Un
Susana	Yao
Ronaldo	Lima

(d)

Pn	Un
José	Gonçalves
Barbara	Pires
Ana	Tavares
Jonas	Wang
Ernesto	Gilberto

(e)

Pnome	Unome
João	Silva
Ricardo	Braga
Francisco	Leme

Figura 6.4
As operações de conjunto UNIÃO, INTERSECÇÃO e SUBTRAÇÃO. (a) Duas relações compatíveis na união. (b) ALUNO ∪ PROFESSOR. (c) ALUNO ∩ PROFESSOR. (d) ALUNO – PROFESSOR. (e) PROFESSOR – ALUNO.

o resultado de $R(A_1, A_2, ..., A_n) \times S(B_1, B_2, ..., B_m)$ é uma relação Q com grau $n + m$ atributos $Q(A_1, A_2, ..., A_n, B_1, B_2, ..., B_m)$, nessa ordem. A relação resultante Q tem uma tupla para cada combinação de tuplas — uma de R e uma de S. Logo, se R tem n_R tuplas (indicado como $|R| = n_R$), e S tem n_S tuplas, então $R \times S$ terá $n_R * n_S$ tuplas.

A operação *n*-ária PRODUTO CARTESIANO é uma extensão desse conceito, que produz novas tuplas ao concatenar todas as combinações de tuplas possíveis de *n* relações básicas.

Com frequência, a operação PRODUTO CARTESIANO aplicada isoladamente não tem significado. Ela é mais útil quando seguida por uma seleção que combina valores de atributos vindos das relações componentes. Por exemplo, suponha que queiramos recuperar uma lista dos nomes dos dependentes de cada funcionária. Podemos fazer isso da seguinte forma:

FUNC_MULHERES ← $\sigma_{Sexo='F'}$(FUNCIONARIO)
FUNCNOMES ← $\pi_{Pnome, Unome, Cpf}$(FUNC_MULHERES)
FUNC_DEPENDENTES ← FUNCNOMES × DEPENDENTE
DEPENDENTE_PARTIC ← $\sigma_{Cpf=Fcpf}$(FUNC_DEPENDENTES)
RESULTADO ← $\pi_{Pnome, Unome, Nome_dependente}$(DEPENDENTE_PARTIC)

As relações resultantes dessa sequência de operações aparecem na Figura 6.5. A relação FUNC_DEPENDENTES é o resultado da aplicação da operação PRODUTO CARTESIANO a FUNCNOMES da Figura 6.5 com DEPENDENTE da Figura 3.6. Em FUNC_DEPENDENTES, cada tupla de FUNCNOMES é combinada com cada tupla de DEPENDENTE, dando um resultado que não é muito significativo (cada dependente é combinado com *cada* funcionária). Queremos combinar uma tupla de funcionária somente com seus dependentes em particular — a saber, as tuplas de DEPENDENTE cujo valor de Fcpf combina com o valor de Cpf da tupla FUNCIONARIO. A relação DEPENDENTES_PARTIC realiza isso. A relação FUNC_DEPENDENTES é um bom exemplo do caso em que a álgebra relacional pode ser corretamente aplicada para gerar resultados que não fazem sentido algum. É responsabilidade do usuário garantir que aplicará apenas operações significativas às relações.

O PRODUTO CARTESIANO cria tuplas com os atributos combinados de duas relações. Podemos selecionar (SELEÇÃO) *tuplas relacionadas somente* das duas relações especificando uma condição de seleção apropriada após o produto Cartesiano, como fizemos no exemplo anterior. Como essa sequência de PRODUTO CARTESIANO seguida por SELEÇÃO é muito uti-

FUNC_MULHERES

Pnome	Minicial	Unome	Cpf	Datanasc	Endereco	Sexo	Salario	Cpf_supervisor	Dnr
Alice	J	Zelaya	99988777767	19-01-1968	Rua Souza Lima, 35, Curitiba, PR	F	25.000	98765432168	4
Jennifer	S	Souza	98765432168	20-06-1941	Av. ArthurdeLima, 54, Santo André, SP	F	43.000	88866555576	4
Joice	A	Leite	45345345376	31-07-1972	Av. LucasObes, 74, São Paulo, SP	F	25.000	33344555587	5

FUNCNOMES

Pnome	Unome	Cpf
Alice	Zelaya	99988777767
Jennifer	Souza	98765432168
Joice	Leite	45345345376

FUNC_DEPENDENTES

Pnome	Unome	Cpf	Fcpf	Nome_dependente	Sexo	Datanasc	.
Alice	Zelaya	99988777767	33344555587	Alicia	F	05-04-1986	. . .
Alice	Zelaya	99988777767	33344555587	Tiago	M	25-10-1983	. . .
Alice	Zelaya	99988777767	33344555587	Janaina	F	03-05-1958	. . .
Alice	Zelaya	99988777767	98765432168	Antonio	M	28-02-1942	. . .
Alice	Zelaya	99988777767	12345678966	Michael	M	04-01-1988	. . .
Alice	Zelaya	99988777767	12345678966	Alicia	F	30-12-1988	. . .
Alice	Zelaya	99988777767	12345678966	Elizabeth	F	05-05-1967	. . .
Jennifer	Souza	98765432168	33344555587	Alicia	F	05-04-1986	.
Jennifer	Souza	98765432168	33344555587	Tiago	M	25-10-1983	.
Jennifer	Souza	98765432168	33344555587	Janaina	F	03-05-1958	.
Jennifer	Souza	98765432168	98765432168	Antonio	M	28-02-1942	.
Jennifer	Souza	98765432168	12345678966	Michael	M	04-01-1988	.
Jennifer	Souza	98765432168	12345678966	Alicia	F	30-12-1988	.
Jennifer	Souza	98765432168	12345678966	Elizabeth	F	05-05-1967	.
Joice	Leite	45345345376	33344555587	Alicia	F	05-04-1986	.
Joice	Leite	45345345376	33344555587	Tiago	M	25-10-1983	.
Joice	Leite	45345345376	33344555587	Janaina	F	03-05-1958	.
Joice	Leite	45345345376	98765432168	Antonio	M	28-02-1942	.
Joice	Leite	45345345376	12345678966	Michael	M	04-01-1988	.
Joice	Leite	45345345376	12345678966	Alicia	F	30-12-1988	.
Joice	Leite	45345345376	12345678966	Elizabeth	F	05-05-1967	.

DEPENDENTES_RESULT

Pnome	Unome	Cpf	Fcpf	Nome_dependente	Sexo	Datanasc	.
Jennifer	Souza	98765432168	98765432168	Antonio	M	28-02-1942	.

RESULTADO

Pnome	Unome	Nome_dependente
Jennifer	Souza	Antonio

Figura 6.5
A operação Produto Cartesiano (Produto Cruzado).

lizada para combinar *tuplas relacionadas* de duas relações, uma operação especial, chamada JUNÇÃO, foi criada para especificar essa sequência como uma única operação. Discutimos a operação JUNÇÃO em seguida.

Em SQL, o PRODUTO CARTESIANO pode ser realizado usando a opção CROSS JOIN nas tabelas juntadas (ver Seção 5.1.6). Como alternativa, se houver duas tabelas na cláusula WHERE e não houver condição de junção correspondente na consulta, o resultado também será o PRODUTO CARTESIANO das duas tabelas (ver C10, na Seção 4.3.3).

6.3 Operações relacionais binárias: JUNÇÃO e DIVISÃO

6.3.1 A operação JUNÇÃO

A operação **JUNÇÃO**, indicada por $\bowtie$, é usada para combinar *tuplas relacionadas* de duas relações em uma única tupla 'maior'. Essa operação é muito importante para qualquer banco de dados relacional com mais de uma relação única, porque nos permite processar relacionamentos entre as relações. Para ilustrar JUNÇÃO, suponha que queiramos recuperar o nome do gerente de cada departamento. Para obter esse nome, precisamos combinar cada tupla de departamento com a tupla de funcionário cujo valor de Cpf (Cadastro de Pessoa Física) combina com o valor de Cpf_gerente na tupla de departamento. Fazemos isso usando a operação JUNÇÃO e depois projetando o resultado nos atributos necessários, da seguinte forma:

DEP_GER ← DEPARTAMENTO $\bowtie_{Cpf_gerente=Cpf}$ FUNCIONARIO

RESULTADO ← $\pi_{Dnome, Unome, Pnome}$(DEP_GER)

A primeira operação é ilustrada na Figura 6.6. Observe que o Cpf_gerente é uma chave estrangeira da relação DEPARTAMENTO que referencia Cpf, a chave primária da relação FUNCIONARIO. Essa restrição de integridade referencial desempenha um papel importante para que haja tuplas combinando na relação referenciada FUNCIONARIO.

A operação JUNÇÃO pode ser especificada como uma operação PRODUTO CARTESIANO seguida por uma operação SELEÇÃO. Porém, a JUNÇÃO é muito importante porque é usada muito frequentemente quando se especifica consultas de banco de dados. Considere o exemplo anterior, ilustrando PRODUTO CARTESIANO que incluiu a seguinte sequência de operações:

FUNC_DEPENDENTES ← FUNCNOMES × DEPENDENTE

DEPENDENTES_PARTIC ← $\sigma_{Cpf=Fcpf}$(FUNC_DEPENDENTES)

Essas duas operações podem ser substituídas por uma única operação JUNÇÃO da seguinte forma:

DEPENDENTES_PARTIC ← FUNCNOMES $\bowtie_{Cpf=Fcpf}$ DEPENDENTE

A forma geral de uma operação JUNÇÃO sobre duas relações[5] $R(A_1, A_2, ..., A_n)$ e $S(B_1, B_2, ..., B_m)$ é

$R \bowtie_{<condição\ junção>} S$

O resultado da JUNÇÃO é uma relação Q com $n + m$ atributos $Q(A_1, A_2, ..., A_n, B_1, B_2, ..., B_m)$ nessa ordem, Q tem uma tupla para cada combinação de tuplas — uma de R e uma de S — *sempre que a combinação satisfaz a condição de junção*. Essa é a principal diferença entre PRODUTO CARTESIANO e JUNÇÃO. Em JUNÇÃO, apenas combinações de tuplas *que satisfaçam a condição de junção* aparecem no resultado, enquanto no PRODUTO CARTESIANO *todas* as combinações de tuplas são incluídas no resultado. A condição de junção é especificada sobre atributos das duas relações R e S e é avaliada para cada combinação de tuplas. Cada combinação de tupla para a qual a condição de junção é avaliada como TRUE é incluída na relação resultante Q *como uma única tupla combinada*.

Uma condição de junção geral tem a forma

<condição> **AND** <condição> **AND**... **AND** <condição>

DEP_GER

Dnome	Dnumero	Cpf_gerente	.	Pnome	Minicial	Unome	Cpf	.
Pesquisa	5	33344555587	...	Fernando	T	Wong	33344555587	...
Administração	4	98765432168	...	Jennifer	S	Souza	98765432168	...
Matriz	1	88866555576	...	Jorge	E	Brito	88866555576	...

Figura 6.6

Resultado da operação JUNÇÃO em DEP_GER ← DEPARTAMENTO $\bowtie_{Cpf_gerente=Cpf}$ FUNCIONARIO.

[5] Novamente, observe que *R* e *S* podem ser quaisquer relações que resultam de *expressões da álgebra relacional* gerais.

onde cada <condição> tem a forma $A_i\ \theta\ B_j$, A_i é um atributo de R, B_j é um atributo de S, A_i e B_j têm o mesmo domínio, e θ (teta) é um dos operadores de comparação $\{=, <, \leq, >, \geq, \neq\}$. Uma operação JUNÇÃO com essa condição de junção geral é chamada de **JUNÇÃO THETA**. As tuplas cujos atributos de junção são NULL ou dos quais a condição de junção é FALSE *não* aparecem no resultado. Nesse sentido, a operação JUNÇÃO *não* necessariamente preserva toda a informação das relações participantes, pois as tuplas que não são combinadas com as correspondentes na outra relação não aparecem no resultado.

6.3.2 Variações de JUNÇÃO: EQUIJUNÇÃO e JUNÇÃO NATURAL

O uso mais comum de JUNÇÃO envolve condições de junção apenas em comparações de igualdade. Esse tipo de JUNÇÃO, em que o único operador de comparação usado é =, é chamado de **EQUIJUNÇÃO**. Os dois exemplos anteriores foram EQUIJUNÇÃO. Observe que, no resultado de um EQUIJUNÇÃO, sempre temos um ou mais pares de atributos que possuem *valores idênticos* em cada tupla. Por exemplo, na Figura 6.6, os valores dos atributos Cpf_gerente e Cpf são idênticos em cada tupla de DEP_GER (o resultado de EQUIJUNÇÃO), porque a condição de junção de igualdade especificada sobre esses dois atributos *requer que os valores sejam idênticos* em cada tupla no resultado. Como um de cada par de atributos com valores idênticos é desnecessário, uma nova operação, chamada **JUNÇÃO NATURAL** — indicada por * — foi criada para eliminar o segundo atributo (desnecessário) na condição de EQUIJUNÇÃO.[6] A definição padrão de JUNÇÃO NATURAL requer que os dois atributos de junção (ou cada par de atributos de junção) tenham o mesmo nome nas duas relações. Se isso não acontecer, uma operação de renomeação é aplicada primeiro.

Suponha que queiramos combinar cada tupla PROJETO com a tupla DEPARTAMENTO que controla o projeto. No exemplo a seguir, primeiro renomeamos o atributo Dnumero de DEPARTAMENTO para Dnum — de modo que ele tenha o mesmo nome do atributo Dnum em PROJETO — e depois aplicamos a JUNÇÃO NATURAL:

PROJETO_DEP ← PROJETO * $\rho_{(Dnome,\ Dnum,\ Cpf_gerente,\ Data_inicio_gerente)}$(DEPARTAMENTO)

A mesma consulta pode ser feita em duas etapas criando uma tabela intermediária DEP da seguinte forma:

DEP ← $\rho_{(Dnome,\ Dnum,\ Cpf_gerente,\ Data_inicio_gerente)}$(DEPARTAMENTO)

PROJETO_DEP ← PROJETO * DEP

O atributo Dnum é chamado de **atributo de junção** para a operação JUNÇÃO NATURAL, pois é o único atributo com o mesmo nome nas duas relações. A relação resultante é ilustrada na Figura 6.7(a). Na relação PROJETO_DEP, cada tupla combina uma tupla PROJETO com a tupla DEPARTAMENTO para o departamento que controla o projeto, mas *somente um valor de atributo de junção* é mantido.

Se os atributos sobre os quais a junção natural é especificada *já tiverem os mesmos nomes nas duas relações*, a renomeação não é necessária. Por exemplo, para aplicar uma junção natural aos atributos Dnumero de DEPARTAMENTO e LOCALIZACAO_DEP, basta escrever

LOCAL_DEP ← DEPARTAMENTO * LOCALIZACAO_DEP

A relação resultante está na Figura 6.7(b), que combina cada departamento com sua localização e tem uma tupla para cada local. Em geral, a condição de junção para JUNÇÃO NATURAL é construída igualando *cada par de atributos de junção* que tem o mesmo nome nas duas relações e combinando essas condições com AND. Para cada par desses atributos, o atributo correspondente deve ter o mesmo nome.

Uma definição mais geral, porém não padrão, para JUNÇÃO NATURAL é

$Q \leftarrow R *_{(<lista1>),\ (<lista2>)} S$

Nesse caso, <lista1> especifica uma lista de i atributos de R, e <lista2> especifica uma lista de i atributos de S. As listas são usadas para formar condições de comparação de igualdade entre pares de atributos correspondentes, e as condições passam então por um AND. Somente a lista correspondente aos atributos da primeira relação R — <lista1> — é mantida no resultado Q.

Observe que, se nenhuma combinação de tuplas satisfizer a condição de junção, o resultado de uma JUNÇÃO é uma relação vazia com zero tuplas. Em geral, se R tiver n_R tuplas e S tiver n_S tuplas, o resultado de uma operação JUNÇÃO $R \bowtie_{<condição\ junção>} S$ terá entre zero e $n_R * n_S$ tuplas. O tamanho esperado do resultado da junção dividido pelo tamanho máximo $n_R * n_S$ leva a uma razão chamada **seletividade de junção**, que é uma propriedade de cada condição de junção.

[6] A JUNÇÃO NATURAL é basicamente uma EQUIJUNÇÃO seguida pela remoção dos atributos desnecessários.

(a)
PROJETO_DEP

Projnome	Projnumero	Projlocal	Dnum	Dnome	Cpf_gerente	Data_inicio_gerente
ProdutoX	1	SantoAndré	5	Pesquisa	33344555587	22-05-1988
ProdutoY	2	Itu	5	Pesquisa	33344555587	22-05-1988
ProdutoZ	3	SãoPaulo	5	Pesquisa	33344555587	22-05-1988
Informatização	10	Mauá	4	Administração	98765432168	01-01-1995
Reorganização	20	SãoPaulo	1	Matriz	88866555576	19-06-1981
Benefícios	30	Mauá	4	Administração	98765432168	01-01-1995

(b)
LOCAL_DEP

Dnome	Dnumero	Cpf_gerente	Data_inicio_gerente	Dlocal
Matriz	1	88866555576	19-06-1981	SãoPaulo
Administração	4	98765432168	01-01-1995	Mauá
Pesquisa	5	33344555587	22-05-1988	SantoAndré
Pesquisa	5	33344555587	22-05-1988	Itu
Pesquisa	5	33344555587	22-05-1988	SãoPaulo

Figura 6.7
Resultados de duas operações JUNÇÃO NATURAL. (a) PROJETO_DEP ← PROJETO * DEP. (b) LOCAL_DEP ← DEPARTAMENTO * LOCALIZACAO_DEP.

Se não houver condição de junção, todas as combinações de tuplas se qualificam e a JUNÇÃO se degenera em um PRODUTO CARTESIANO, chamado PRODUTO CRUZADO ou JUNÇÃO CRUZADA.

Como podemos ver, uma única operação JOIN é usada para combinar dados de duas relações de modo que a informação relacionada possa ser apresentada em uma única tabela. Essas operações também são conhecidas como **junções internas** (*inner joins*), para distingui-las de uma variação de junção diferente chamada *junções externas* (*outer joins*; ver Seção 6.4.4). Informalmente, uma *junção interna* é um tipo de operação de correspondência e combinação definida de maneira formal como uma combinação de PRODUTO CARTESIANO e SELEÇÃO. Observe que, às vezes, uma junção pode ser especificada entre uma relação e si mesma, conforme ilustraremos na Seção 6.4.3. A operação JUNÇÃO NATURAL ou EQUIJUNÇÃO também pode ser especificada entre múltiplas tabelas, levando a uma *junção de n vias*. Por exemplo, considere a junção de três vias a seguir:

$$((\text{PROJETO} \bowtie_{\text{Dnum=Dnumero}} \text{DEPARTAMENTO}) \bowtie_{\text{Cpf_gerente=Cpf}} \text{FUNCIONARIO})$$

Isso combina cada tupla de projeto com sua tupla de departamento de controle em uma única tupla, e depois combina essa tupla com uma tupla de funcionário que é o gerente de departamento. O resultado disso é uma relação consolidada em que cada tupla contém essa informação combinada de projeto-departamento-gerente.

Em SQL, a JUNÇÃO pode ser realizada de diversas maneiras. O primeiro método é especificar as <condições junção> na cláusula WHERE, junto com quaisquer outras condições de seleção. Isso é muito comum, e está ilustrado pelas consultas C1, C1A, C1B, C2 e C8 nas seções 4.3.1 e 4.3.2, bem como por muitos outros exemplos de consulta nos capítulos 4 e 5. A segunda maneira é usar uma relação aninhada, conforme ilustrado pelas consultas C4A e C16 na Seção 5.1.2. Outra maneira é utilizar o conceito de tabelas de junção, conforme ilustrado pelas consultas C1A, C1B, C8B e C2A na Seção 5.1.6. A construção das tabelas de junção foi acrescentada à SQL2 para permitir ao usuário especificar explicitamente todos os diversos tipos de junções, pois os outros métodos eram mais limitados. Isso também permite que o usuário faça a distinção clara das condições de junção com base nas condições de seleção na cláusula WHERE.

6.3.3 Um conjunto completo de operações da álgebra relacional

Já se mostrou que o conjunto de operações da álgebra relacional $\{\sigma, \pi, \cup, \rho, -, \times\}$ é um conjunto

completo; ou seja, qualquer uma das outras operações srcinais da álgebra relacional pode ser expressa como uma *sequência de operações desse conjunto*. Por exemplo, a operação INTERSECÇÃO pode ser expressa usando UNIÃO e SUBTRAÇÃO da seguinte forma:

$$R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$$

Embora, estritamente falando, a INTERSECÇÃO não seja exigida, é inconveniente especificar essa expressão complexa toda vez que quisermos especificar uma interseção. Outro exemplo: uma operação JUNÇÃO pode ser especificada como um PRODUTO CARTESIANO seguido por uma operação SELEÇÃO, conforme discutimos:

$$R \bowtie_{\text{<condição>}} S \equiv \sigma_{\text{<condição>}}(R \times S)$$

De modo semelhante, uma JUNÇÃO NATURAL pode ser especificada como um PRODUTO CARTESIANO precedido por RENOMEAR e seguido por operações SELEÇÃO e PROJEÇÃO. Logo, as diversas operações JUNÇÃO também *não são estritamente necessárias* para o poder expressivo da álgebra relacional. No entanto, elas são importantes para inclusão como operações separadas, pois são convenientes de se usar e bastante utilizadas em aplicações de banco de dados. Outras operações foram inseridas na álgebra relacional básica por conveniência, em vez de necessidade. Discutimos uma delas — a operação DIVISÃO — na próxima seção.

6.3.4 A operação DIVISÃO

A operação DIVISÃO, indicada por ÷, é útil para um tipo especial de consulta que às vezes ocorre nas aplicações de banco de dados. Um exemplo é *Recuperar os nomes dos funcionários que trabalham em* ***todos*** *os projetos em que 'João Silva' trabalha*. Para expressar essa consulta usando a operação DIVISÃO, prossiga da seguinte forma. Primeiro, recupere a lista dos números de projeto em que 'João Silva' trabalha na relação intermediária SILVA_PNRS:

SILVA $\leftarrow \sigma_{\text{Pnome='João' AND Unome='Silva'}}$(FUNCIONARIO)

SILVA_PNRS $\leftarrow \pi_{\text{Pnr}}$(TRABALHA_EM $\bowtie_{\text{Fcpf=Cpf}}$ SILVA)

Em seguida, crie uma relação que inclua uma tupla <Pnr, Fcpf> sempre que um funcionário, cujo Cpf é Fcpf, trabalha no projeto cujo número é Pnr na relação intermediária CPF_PNRS:

CPF_PNRS $\leftarrow \pi_{\text{Fcpf, Pnr}}$(TRABALHA_EM)

Finalmente, aplique a operação DIVISÃO às duas relações, o que gera os números de Cadastro de Pessoa Física dos funcionários desejados:

CPFS(Cpf) $\leftarrow$ CPF_PNRS ÷ SILVA_PNRS

RESULTADO $\leftarrow \pi_{\text{Pnome, Unome}}$(CPFS * FUNCIONARIO)

(a)

CPF_PNRS

Fcpf	Pnr
12345678966	1
12345678966	2
66688444476	3
45345345376	1
45345345376	2
33344555587	2
33344555587	3
33344555587	10
33344555587	20
99988777767	30
99988777767	10
98798798733	10
98798798733	30
98765432168	30
98765432168	20
88866555576	20

SILVA_PNRS

Pnr
1
2

CPFS

Cpf
12345678966
45345345376

(b)

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

Figura 6.8
A operação DIVISÃO. (a) Dividindo CPF_PNRS por SILVA_PNRS. (b) $T \leftarrow R \div S$.

As operações anteriores aparecem na Figura 6.8(a). Em geral, a operação DIVISÃO é aplicada às duas relações $R(Z) \div S(X)$, em que os atributos de R são um subconjunto dos atributos de S; ou seja, $X \subseteq Z$. Considere que Y seja o conjunto de atributos de R que não são atributos de S; ou seja, $Y = Z - X$ (e, portanto, $Z = X \cup Y$). O resultado da DIVISÃO é uma relação $T(Y)$ que inclui uma tupla t se as tuplas t_R aparecerem em R com $t_R[Y] = t$, e com $t_R[X] = t_S$ para *cada* tupla t_S em S. Isso significa que, para uma tupla t aparecer no resultado T da DIVISÃO, os valores em t deverão aparecer em R em combinação com *cada tupla* em S. Observe que, na formulação da operação DIVISÃO, as tuplas na relação do denominador S restringem a relação do numerador R, selecionando aquelas tuplas no resultado que combinam com todos os valores presentes no denominador. Não é necessário saber quais são esses valores, pois eles podem ser calculados por outra operação, conforme ilustrado na relação SILVA_PNRS no exemplo anterior.

A Figura 6.8(b) ilustra uma operação DIVISÃO onde $X = \{A\}$, $Y = \{B\}$ e $Z = \{A, B\}$. Observe que as tuplas (valores) b_1 e b_4 aparecem em R em combinação com todas as três tuplas em S; é por isso que elas aparecem na relação resultante T. Todos os outros valores de B em R não aparecem com todas as tuplas em S, e não são selecionados: b_2 não aparece com a_2, e b_3 não aparece com a_1.

A operação DIVISÃO pode ser expressa como uma sequência de operações π, $\times$ e $-$ da seguinte forma:

$$T1 \leftarrow \pi_Y(R)$$
$$T2 \leftarrow \pi_Y((S \times T1) - R)$$
$$T \leftarrow T1 - T2$$

A operação DIVISÃO é definida por conveniência para lidar com consultas que envolvem *quantificação universal* (ver Seção 6.6.7) ou a condição *all*. A maioria das implementações de SGBDR com SQL como linguagem de consulta primária não implementa a divisão diretamente. A SQL tem um modo indireto de lidar com o tipo de consulta ilustrado anteriormente (ver Seção 5.1.4, consultas C3A e C3B). A Tabela 6.1 lista as diversas operações básicas da álgebra relacional que discutimos.

6.3.5 Notação para árvores de consulta

Nesta seção, descrevemos uma notação que costuma ser usada em sistemas relacionais para representar consultas internamente. A notação é chamada *árvore de consulta* ou, às vezes, é conhecida como *árvore de avaliação de consulta* ou *árvore de execução de consulta*. Ela inclui as operações da álgebra relacional

Tabela 6.1
Operações de álgebra relacional.

OPERAÇÃO	FINALIDADE	NOTAÇÃO
SELEÇÃO	Seleciona todas as tuplas que satisfazem a condição de seleção de uma relação R.	$\sigma_{<\text{condição seleção}>}(R)$
PROJEÇÃO	Produz uma nova relação com apenas alguns dos atributos de R, e remove tuplas duplicadas.	$\pi_{<\text{lista atributos}>}(R)$
JUNÇÃO THETA	Produz todas as combinações de tuplas de R_1 e R_2 que satisfazem a condição de junção.	$R_1 \bowtie_{<\text{condição junção}>} R_2$
EQUIJUNÇÃO	Produz todas as combinações de tuplas de R_1 e R_2 que satisfazem uma condição de junção apenas com comparações de igualdade.	$R_1 \bowtie_{<\text{condição junção}>} R_2$, OR $R_1 \bowtie_{(<\text{atributos junção 1>}),(<\text{atributos junção 2>})} R_2$
JUNÇÃO NATURAL	O mesmo que EQUIJOIN, exceto que atributos de junção de R_2 não são incluídos na relação resultante; se os atributos de junção tiverem os mesmos nomes, eles nem sequer precisam ser especificados.	$R_1 *_{<\text{condição junção}>} R_2$, OR $R_1 *_{(<\text{atributos junção 1>}),(<\text{atributos junção 2>})} R_2$ OR $R_1 * R_2$
UNIÃO	Produz uma relação que inclui todas as tuplas em R_1 ou R_2 ou tanto R_1 quanto R_2; R_1 e R_2 precisam ser compatíveis na união.	$R_1 \cup R_2$
INTERSECÇÃO	Produz uma relação que inclui todas as tuplas em R_1 e R_2; R_1 e R_2 precisam ser compatíveis na união.	$R_1 \cap R_2$
DIFERENÇA	Produz uma relação que inclui todas as tuplas em R_1 que não estão em R_2; R_1 e R_2 precisam ser compatíveis na união.	$R_1 - R_2$
PRODUTO CARTESIANO	Produz uma relação que tem os atributos de R_1 e R_2 e inclui como tuplas todas as possíveis combinações de tuplas de R_1 e R_2.	$R_1 \times R_2$
DIVISÃO	Produz uma relação $R(X)$ que inclui todas as tuplas $t[X]$ em $R_1(Z)$ que aparecem em R_1 em combinação com toda tupla de $R_2(Y)$, onde $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

em execução e é usada como uma estrutura de dados possível para a representação interna da consulta em um SGBDR.

Uma **árvore de consulta** é uma estrutura de dados em árvore que corresponde a uma expressão da álgebra relacional. Ela representa as relações de entrada da consulta como *nós folha* da árvore, e representa as operações da álgebra relacional como nós internos. Uma execução da árvore de consulta consiste em executar uma operação de nó interno sempre que seus operandos (representados por seus nós filhos) estiverem disponíveis, e depois substituir esse nó interno pela relação que resulta da execução da operação. A execução termina quando o nó raiz é executado e produz a relação de resultado para a consulta.

A Figura 6.9 mostra uma árvore de consulta para a Consulta 2 (ver Seção 4.3.1): *para cada projeto localizado em 'Mauá', liste o número do projeto, o número do departamento que o controla e o último nome, endereço e data de nascimento do gerente do departamento*. Essa consulta é especificada no esquema relacional da Figura 3.5 e corresponde à seguinte expressão da álgebra relacional:

$$\pi_{\text{Projnumero, Dnum, Unome, Endereco, Datanasc}}((\sigma_{\text{Projlocal='Mauá'}}(\text{PROJETO})) \bowtie_{\text{Dnum=Dnumero}} (\text{DEPARTAMENTO})) \bowtie_{\text{Cpf_gerente=Cpf}} (\text{FUNCIONARIO}))$$

Na Figura 6.9, os três nós folha P, D e F representam as três relações PROJETO, DEPARTAMENTO e FUNCIONARIO. As operações da álgebra relacional na expressão são representadas pelos nós de árvore internos. A árvore de consulta significa uma ordem de execução explícita no seguinte sentido: para executar C2, o nó marcado com (1) na Figura 6.9 precisa iniciar a execução antes do nó (2), pois algumas tuplas resultantes da operação (1) devem estar disponíveis antes de podermos iniciar a execução da operação (2). De modo semelhante, o nó (2) precisa começar a executar e produzir resultados antes que o nó (3) possa iniciar a execução, e assim por diante. Em geral, uma árvore de consulta oferece uma boa representação visual e compreensão da consulta em relação às operações relacionais que ela usa, e é recomendada como um meio adicional para expressar consultas na álgebra relacional. Retornaremos às árvores de consulta quando discutirmos o processamento e a otimização da consulta no Capítulo 19.

6.4 Outras operações relacionais

Algumas solicitações comuns no banco de dados — que são necessárias em aplicações comerciais para SGBDRs — não podem ser realizadas com as operações da álgebra relacional descritas nas seções 6.1 a 6.3. Nesta seção, definimos operações adicionais para expressar essas solicitações. Essas operações melhoram o poder expressivo da álgebra relacionalsrcinal.

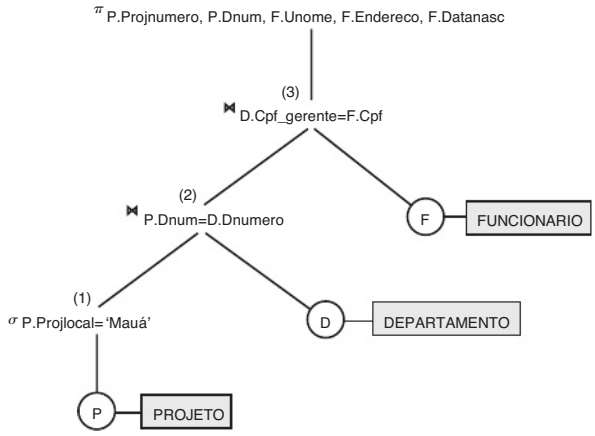


Figura 6.9
Árvore de consulta correspondente à expressão da álgebra relacional para C2.

6.4.1 Projeção generalizada

A operação de projeção generalizada estende a operação de projeção, permitindo que as funções dos atributos sejam incluídas na lista de projeção. A forma generalizada pode ser expressa como:

$$\pi_{F1, F2, ..., Fn}(R)$$

onde F_1, F_2, ..., F_n são funções sobre os atributos na relação R e podem envolver operações aritméticas e valores constantes. Essa operação é útil quando se desenvolvem relatórios em que os valores calculados precisam ser produzidos nas colunas de um resultado da consulta.

Como exemplo, considere a relação

FUNCIONARIO (Cpf, Salario, Deducao, Anos_em_servico)

Um relatório pode ser exigido para mostrar

Salario líquido = Salario − Deducao,
Bonus = 2.000 * Anos_em_servico, e
Imposto = 0,25 * Salario.

Então, uma projeção generalizada combinada com a renomeação pode ser usada da seguinte forma:

RELATORIO ← $\rho_{(Cpf, Salario_liquido, Bonus, Imposto)}$
($\pi_{Cpf, Salario - Deducao, 2.000 * Anos_em_servico, 0,25 * Salario}$(FUNCIONARIO)).

6.4.2 Funçõesde agregaçãoe agrupamento

Outro tipo de solicitação que pode ser expressa na álgebra relacional básica é especificar **funções de agregação** matemáticas sobre coleções de valores do banco de dados. Alguns exemplos dessas funções incluem recuperar a média ou salário total de todos os funcionários ou o número total de tuplas de funcionário. Essas funções são usadas em consultas estatísticas simples que resumem informações das tuplas do banco de dados. Funções comuns aplicadas a coleções de valores numéricos são SOMA, MÉDIA, MÁXIMO e MÍNIMO. A função CONTA é usada para contar tuplas ou valores.

Outro tipo comum de solicitação envolve agrupar as tuplas em uma relação pelo valor de alguns de seus atributos e, depois, aplicar uma função de agregação *independentemente para cada grupo*. Um exemplo seria agrupar tuplas FUNCIONARIO por Dnr, de modo que cada grupo inclua as tuplas para funcionários trabalhando no mesmo departamento. Podemos então listar cada valor de Dnr juntamente com, digamos, o salário médio dos funcionários no departamento, ou o número de funcionários que trabalham no departamento.

Podemos definir uma operação FUNÇÃO AGREGADA, usando o símbolo $\Im$ (pronuncia-se F *script*),[7] para especificar esses tipos de solicitações da seguinte forma:

$$<\text{atributos agrupamento}>\ \Im_{<\text{lista funções}>}(R)$$

onde <atributos agrupamento> é uma lista de atributos da relação especificada em R, e <lista funções> é uma lista de pares (<função> <atributo>). Em cada par desse tipo, <função> é uma das funções permitidas — como SOMA, MÉDIA, MÁXIMO, MÍNIMO, CONTA — e <atributo> é um atributo da relação especificada por R. A relação resultante tem os atributos de agrupamento mais um atributo para cada elemento na lista de funções. Por exemplo, para recuperar cada número de departamento, o número de funcionários no departamento e seu salário médio, enquanto renomeia os atributos resultantes como indicado a seguir, escrevemos:

$$\rho_{R(Dnr, Nr_de_funcionarios, Media_sal)}(_{Dnr}\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO))$$

O resultado dessa operação sobre a relação FUNCIONARIO da Figura 3.6 é exibido na Figura 6.10(a).

No exemplo anterior, especificamos uma lista de nomes de atributo — entre parênteses na operação RENOMEAR — para a relação resultante R. Se não houver renomeação, os atributos da relação resultante que correspondem à lista de funções serão a concatenação do nome da função com o nome do atributo, na forma <função>_<atributo>[8]. Por exemplo, a Figura 6.10(b) mostra o resultado da seguinte operação:

$$_{Dnr}\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO)$$

Se nenhum atributo de agrupamento for especificado, as funções são aplicadas a *todas as tuplas* na relação, de modo que a relação resultante tenha *apenas uma única tupla*. Por exemplo, a Figura 6.10(c) mostra o resultado da seguinte operação:

$$\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO)$$

É importante observar que, em geral, as duplicatas *não são eliminadas* quando uma função de

[7] Não existe uma única notação combinada para especificar funções de agregação. Em alguns casos, é usado um 'script A'.
[8] Observe que estamos sugerindo uma notação arbitrária. Não existe uma notação-padrão.

a. $\rho_{R(Dnr,\ Nr_de_funcionarios,\ Media_sal)}({}_{Dnr}\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO))$.

b. ${}_{Dnr}\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO)$.

c. $\Im_{CONTA\ Cpf,\ MÉDIA\ Salario}(FUNCIONARIO)$.

R

(a)

Dnr	Nr_de_funcionarios	Media_sal
5	4	33.250
4	3	31.000
1	1	55.000

(b)

Dnr	Contador_cpf	Media_salario
5	4	33.250
4	3	31.000
1	1	55.000

(c)

Contador_cpf	Media_salario
8	35.125

Figura 6.10
A operação de função agregada.

agregação é aplicada. Desse modo, a interpretação normal de funções como SOMA e MÉDIA é calculada.[9] Vale a pena enfatizar que o resultado de aplicar uma função de agregação é uma relação, e não um número escalar — mesmo que ele tenha um único valor. Isso torna a álgebra relacional um sistema matemático fechado.

6.4.3 Operações de fechamento recursivo

Outro tipo de operação que, em geral, não pode ser especificada na álgebra relacional srcinal básica é o **fechamento recursivo** Essa operação é aplicada a um **relacionamento recursivo**entre tuplas do mesmo tipo, como o relacionamento entre umfuncionário e um supervisor. Esse relacionamento é descrito pela chave estrangeira Cpf_supervisorda relação FUNCIONARIO nas figuras 3.5 e 3.6, e relaciona cada tupla defuncionário (no papel de supervisionado) a outra tupla de funcionário (no papel de supervisor). Um exemplo de operação recursiva é recuperar todos os supervisionados de um funcionário *f* em todos os níveis — ou seja, todos os funcionários *f'* supervisionados diretamente por*f*, todos os funcionários*f''*supervisionados diretamente por cada funcionário*f'*, todos os funcionários*f'''*, supervisionados diretamente por cada funcionário *f''*, e assim por diante.

É relativamente simples na álgebra relacional especificar todos os funcionários supervisionados por *f em um nível específico* juntando uma ou mais vezes a própria tabela. Porém, é difícil especificar todos os supervisionados em *todos* os níveis. Por exemplo, para especificar os Cpfs de todos os funcionários *f'* supervisionados diretamente — *no nível um* — pelo funcionário *f* cujo nome é 'Jorge Brito' (ver Figura 3.6), podemos aplicar a seguinte operação:

$$CPF_BRITO \leftarrow \pi_{Cpf}(\sigma_{Pnome='Jorge'\ \mathbf{AND}\ Unome='Brito'}(FUNCIONARIO))$$

$$SUPERVISAO(Cpf1, Cpf2) \leftarrow \pi_{Cpf,Cpf_supervisor}(FUNCIONARIO)$$

$$RESULTADO1(Cpf) \leftarrow \pi_{Cpf1}(SUPERVISAO \bowtie_{Cpf2=Cpf} CPF_BRITO)$$

Para recuperar todos os funcionários supervisionados por Brito no nível 2 — ou seja, todos os funcionários *f''* supervisionados por algum funcionário *f'* que é supervisionado diretamente por Brito —, podemos aplicar outra **JUNÇÃO** ao resultado da primeira consulta, da seguinte forma:

$$RESULTADO2(Cpf) \leftarrow \pi_{Cpf1}(SUPERVISAO \bowtie_{Cpf2=Cpf} RESULTADO1)$$

Para obter os dois conjuntos de funcionários supervisionados nos níveis 1 e 2 por 'Jorge Brito', podemos aplicar a operação UNIÃO aos dois resultados, da seguinte forma:

$$RESULTADO \leftarrow RESULTADO2 \cup RESULTADO1$$

[9] Em SQL, a opção de eliminar duplicatas antes de aplicar a função de agregação está disponível ao incluir a palavra-chave DISTINCT (ver Seção 4.4.4).

SUPERVISAO
(Cpf de Brito: 88866555576)

(Cpf)	(Cpf_supervisor)
Cpf1	Cpf2
12345678966	33344555587
33344555587	88866555576
99988777767	98765432168
98765432168	88866555576
66688444476	33344555587
45345345376	33344555587
98798798733	98765432168
88866555576	null

RESULTADO1

Cpf
33344555587
98765432168

(Supervisionado por Brito)

RESULTADO2

Cpf
12345678966
99988777767
66688444476
45345345376
98798798733

(Supervisionado pelos subordinados de Brito)

RESULTADO

Cpf
12345678966
99988777767
66688444476
45345345376
98798798733
33344555587
98765432168

(RESULTADO1 ∪ RESULTADO2)

Figura 6.11
Uma consulta recursiva de dois níveis.

Os resultados dessas consultas são ilustrados na Figura 6.11. Embora seja possível recuperar funcionários em cada nível e depois obter sua UNIÃO, não podemos, em geral, especificar uma consulta desse tipo como 'recuperar os supervisionados de 'Jorge Brito' em todos os níveis' sem utilizar um mecanismo de looping, a menos que saibamos o número máximo de níveis.[10] Uma operação chamada *fechamento transitivo* das relações foi proposta para calcular o relacionamento recursivo à medida que a recursão prossegue.

6.4.4 Operações OUTER JOIN (junção externa)

A seguir, discutimos algumas extensões adicionais à operação JUNÇÃO que são necessárias para especificar certos tipos de consultas. As operações JUNÇÃO descritas anteriormente combinam com tuplas que satisfazem a condição de junção. Por exemplo, para uma operação JUNÇÃO NATURAL $R * S$, somente tuplas de R que possuem tuplas combinando em S — e vice-versa — aparecem no resultado. Logo, as tuplas sem uma tupla *correspondente* (ou *relacionada*) são eliminadas do resultado de JUNÇÃO. As tuplas com valores NULL nos atributos de junção também são eliminadas. Esse tipo de junção, em que as tuplas sem correspondência são eliminadas, é conhecido como **junção interna** (*inner join*). As operações de junção que descrevemos anteriormente na Seção 6.3 são todas internas. Isso equivale à perda de informações se o usuário quiser que o resultado da JUNÇÃO inclua todas as tuplas em uma ou mais relações componentes.

Um conjunto de operações, chamadas **junções externas** (*outer joins*), foi desenvolvido para o caso em que o usuário deseja manter todas as tuplas em R, ou todas em S, ou todas aquelas nas duas relações no resultado da JUNÇÃO, independentemente delas possuírem ou não tuplas correspondentes na outra relação. Isso satisfaz a necessidade de consultas em que as tuplas das duas tabelas devem ser combi-

[10] O padrão SQL3 inclui sintaxe para fechamento recursivo.

nadas por linhas correspondentes, mas sem perda de quaisquer tuplas por falta de valores correspondentes. Por exemplo, suponha que queiramos uma lista de todos os nomes de funcionário, bem como o nome dos departamentos que eles gerenciam, *se eles gerenciarem um departamento* . Caso não o façam, podemos indicar isso com um valor NULL. Podemos aplicar uma operação **JUNÇÃO EXTERNA À ESQUERDA**, indicada por ⟕, para recuperar o resultado da seguinte forma:

TEMP ← (FUNCIONARIO ⟕ $_{Cpf=Cpf_gerente}$ DEPARTAMENTO)
RESULTADO ← $\pi_{Pnome, Minicial, Unome, Dnome}$(TEMP)

A operação JUNÇÃO EXTERNA À ESQUERDA mantém cada tupla na *primeira* relação, *R*, ou da *esquerda*, em *R* ⟕ *S*; se nenhuma tupla correspondente for encontrada em *S*, então os atributos de *S* no resultado da junção são *preenchidos* com valores NULL. O resultado dessas operações é mostrado na Figura 6.12.

Uma operação semelhante, **JUNÇÃO EXTERNA À DIREITA**, indicada por ⟖, mantém cada tupla na *segunda* relação, *S*, ou da *direita*, no resultado de *R* ⟖ *S*. Uma terceira operação, **JUNÇÃO EXTERNA COMPLETA**, indicada por ⟗, mantém todas as tuplas nas relações da esquerda e da direita quando nenhuma tupla correspondente for encontrada, preenchendo-as com valores NULL conforme a necessidade. As três operações de junção externa fazem parte do padrão SQL2 (ver Seção 5.1.6). Essas operações foram fornecidas mais tarde, como uma extensão da álgebra relacional em resposta à necessidade típica nas aplicações de negócios para mostrar informações relacionadas de várias tabelas exaustivamente. Às vezes, um relatório completo dos dados de várias tabelas é exigido havendo ou não valores correspondentes.

RESULTADO

Pnome	Minicial	Unome	Dnome
João	B	Silva	NULL
Fernando	T	Wong	Pesquisa
Alice	J	Zelaya	NULL
Jennifer	S	Souza	Administração
Ronaldo	K	Lima	NULL
Joice	A	Leite	NULL
André	V	Pereira	NULL
Jorge	E	Brito	Matriz

Figura 6.12
O resultado de uma operação JUNÇÃO EXTERNA À ESQUERDA.

6.4.5 A operação UNIÃO EXTERNA

A operação **UNIÃO EXTERNA** foi desenvolvida para fazer a união de tuplas de duas relações que possuem alguns atributos comuns, mas *não são compatíveis na união (tipo)*. Essa operação fará a UNIÃO de tuplas nas relações *R*(*X*, *Y*) e *S*(*X*, *Z*) que são **parcialmente compatíveis**, significando que somente alguns de seus atributos, digamos *X*, são compatíveis na união. Os atributos compatíveis na união são representados apenas uma vez no resultado, e aqueles atributos que não são compatíveis na união de qualquer uma das relações também são mantidos na relação de resultado *T*(*X*, *Y*, *Z*). Portanto, é o mesmo que uma JUNÇÃO EXTERNA COMPLETA sobre os atributos comuns.

Duas tuplas t_1 em *R* e t_2 em *S* **são ditas combinar** se $t_1[X]=t_2[X]$. Estas serão combinadas (unidas) em uma única tupla em *t*. As tuplas em qualquer relação que não tiverem uma tupla correspondente na outra relação são preenchidas com valores NULL. Por exemplo, uma UNIÃO EXTERNA pode ser aplicada a duas relações cujos esquemas são ALUNO (Nome, Cpf, Departamento, Orientador) e PROFESSOR (Nome, Cpf, Departamento, Classificacao). As tuplas das duas relações são combinadas com base na existência da mesma combinação de valores dos atributos compartilhados — Nome, Cpf, Departamento. A relação resultante, ALUNO_OU_PROFESSOR, terá os seguintes atributos:

ALUNO_OU_PROFESSOR(Nome, Cpf, Departamento, Orientador, Classificacao)

Todas as tuplas das duas relações são incluídas no resultado, mas as tuplas com a mesma combinação (Nome, Cpf, Departamento) aparecerão apenas uma vez no resultado. As tuplas que aparecem apenas em ALUNO terão um NULL para o atributo Classificacao, enquanto as tuplas que aparecem apenas em PROFESSOR terão um NULL para o atributo Orientador. Uma tupla que existe nas duas relações, que representa um aluno que também é um professor, terá valores para todos os seus atributos.[11]

Observe que a mesma pessoa pode ainda aparecer duas vezes no resultado. Por exemplo, pode-se ter um aluno formado no departamento de Matemática que seja um professor no departamento de Ciência da Computação. Embora as duas tuplas representando essa pessoa em ALUNO e PROFESSOR tenham os mesmos valores (Nome, Cpf), elas não combinarão no valor de Departamento, e, por isso, não serão combinadas. Isso porque Departamento tem dois significados diferentes em ALUNO (o de-

[11] Observe que UNIÃO EXTERNA é equivalente a uma JUNÇÃO EXTERNA COMPLETA se os atributos de junção forem *todos* os atributos comuns das duas relações.

partamento onde a pessoa estuda) e PROFESSOR (o departamento onde a pessoa está empregada como professor). Se quiséssemos aplicar a UNIÃO EXTERNA com base apenas na mesma combinação (Nome, Cpf), deveríamos trocar o nome do atributo Departamento em cada tabela para refletir que eles possuem significados diferentes e designá-los como não fazendo parte dos atributos compatíveis na união. Por exemplo, poderíamos renomear os atributos como DeptPrincipal em ALUNO e DeptTrabalho em PROFESSOR.

6.5 Exemplos de consultas na álgebra relacional

A seguir temos exemplos adicionais para ilustrar o uso das operações da álgebra relacional. Todos os exemplos referem-se ao banco de dados da Figura 3.6. Em geral, a mesma consulta pode ser indicada de várias maneiras usando as diversas operações. Declararemos cada consulta de uma maneira e deixaremos que o leitor apresente formulações equivalentes.

Consulta 1. Recuperar o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

$$\text{DEP_PESQUISA} \leftarrow \sigma_{\text{Dnome='Pesquisa'}}(\text{DEPARTAMENTO})$$

$$\text{FUNCS_PESQUISA} \leftarrow (\text{DEP_PESQUISA} \bowtie_{\text{Dnumero=Dnr}} \text{FUNCIONARIO})$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Pnome, Unome, Endereco}}(\text{FUNCS_PESQUISA})$$

Como uma única expressão em linha, esta consulta se torna:

$$\pi_{\text{Pnome, Unome, Endereco}}(\sigma_{\text{Dnome='Pesquisa'}}(\text{DEPARTAMENTO} \bowtie_{\text{Dnumero=Dnr}} (\text{FUNCIONARIO}))$$

Esta consulta poderia ser especificada de outras maneiras. Por exemplo, a ordem das operações JUNÇÃO e SELEÇÃO poderia ser revertida, ou então a JUNÇÃO poderia ser substituída por uma JUNÇÃO NATURAL após renomear um dos atributos de junção para corresponder ao nome do outro atributo de junção.

Consulta 2. Para cada projeto localizado em 'Mauá', liste o número do projeto, o número do departamento que o controla e o último nome, endereço e data de nascimento do gerente do departamento.

$$\text{PROJ_MAUA} \leftarrow \sigma_{\text{Projlocal='Mauá'}}(\text{PROJETO})$$

$$\text{DEP_CONTROLE} \leftarrow (\text{PROJ_MAUA} \bowtie_{\text{Dnum=Dnumero}} \text{DEPARTAMENTO})$$

$$\text{GER_DEP_PROJ} \leftarrow (\text{DEP_CONTROLE} \bowtie_{\text{Cpf_gerente=Cpf}} \text{FUNCIONARIO})$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Projnumero, Dnum, Unome, Endereco, Datanasc}}(\text{GER_DEP_PROJ})$$

Neste exemplo, primeiro selecionamos os projetos localizados em Mauá, depois os juntamos a seus departamentos de controle, em seguida juntamos o resultado com os gerentes de departamento. Finalmente, aplicamos uma operação de projeto sobre os atributos desejados.

Consulta 3. Descobrir os nomes dos funcionários que trabalham em *todos* os projetos controlados pelo departamento número 5.

$$\text{PROJ_DEP5} \leftarrow \rho_{(\text{Pnr})}(\pi_{\text{Projnumero}}(\sigma_{\text{Dnum=5}}(\text{PROJETO})))$$

$$\text{FUNC_PROJ} \leftarrow \rho_{(\text{Cpf, Pnr})}(\pi_{\text{Fcpf, Pnr}}(\text{TRABALHA_EM}))$$

$$\text{RESULTADO_CPF_FUNC} \leftarrow \text{FUNC_PROJ} \div \text{PROJ_DEP5}$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Unome, Pnome}}(\text{RESULTADO_CPF_FUNC} * \text{FUNCIONARIO})$$

Nesta consulta, primeiro criamos uma tabela PROJ_DEP5 que contém os números de projeto de todos aqueles controlados pelo departamento 5. Depois, criamos uma tabela FUNC_PROJ que mantém tuplas (Cpf, Pnr), e aplicamos a operação de divisão. Observe que renomeamos os atributos de modo que eles sejam usados corretamente na operação de divisão. Por fim, acrescentamos o resultado da divisão, que mantém apenas valores Cpf, com a tabela FUNCIONARIO para recuperar os atributos desejados de FUNCIONARIO.

Consulta 4. Fazer uma lista dos números de projeto para aqueles que envolvem um funcionário cujo último nome é 'Silva', seja como um trabalhador ou como um gerente do departamento que controla o projeto.

$$\text{SILVA(Fcpf)} \leftarrow \pi_{\text{Cpf}}(\sigma_{\text{Unome='Silva'}}(\text{FUNCIONARIO}))$$

$$\text{PROJ_SILVA_TRAB} \leftarrow \pi_{\text{Pnr}}(\text{TRABALHA_EM} * \text{SILVA})$$

$$\text{GERENTES} \leftarrow \pi_{\text{Unome, Dnumero}}(\text{FUNCIONARIO} \bowtie_{\text{Cpf=Cpf_gerente}} \text{DEPARTAMENTO})$$

$$\text{DEPS_GERENCIADOS_SILVA(Dnum)} \leftarrow \pi_{\text{Dnumero}}(\sigma_{\text{Unome='Silva'}}(\text{GERENTES}))$$

$$\text{PROJS_SILVA_GER(Pnr)} \leftarrow \pi_{\text{Projnumero}}(\text{DEPS_GERENCIADOS_SILVA} * \text{PROJETO})$$

$$\text{RESULTADO} \leftarrow (\text{PROJS_SILVA_TRAB} \cup \text{PROJS_SILVA_GER})$$

Nesta consulta, recuperamos os números de projeto que envolvem um funcionário chamado Silva como um trabalhador em PROJS_SILVA_TRAB. Depois, recuperamos os números de projeto que envolvem um funcionário chamado Silva como gerente do departamento que controla o projeto em PROJS_SILVA_GER. Por último, aplicamos a operação **UNIÃO** sobre PROJS_SILVA_TRAB e PROJS_SILVA_GER. Como uma única expressão em linha, esta consulta torna-se:

$$\text{RESULTADO} \leftarrow \pi_{\text{Pnome, Unome}}(\text{FUNCIONARIO} \bowtie_{\text{Cpf=Fcpf}} (\pi_{\text{Pnr}}(\text{PROJETO} \bowtie_{\text{Dnum=Dnumero}} (\pi_{\text{Dnumero, Cpf}}(\text{DEPARTAMENTO} \bowtie_{\text{Cpf_gerente=Cpf}} (\pi_{\text{Unome, Dnumero}}(\text{FUNCIONARIO}) \bowtie_{\text{Unome='Silva'}} \pi_{\text{Cpf}}(\sigma_{\text{Unome='Silva'}}(\text{FUNCIONARIO}))))))) \cup (\pi_{\text{Pnr}}(\text{PROJETO} \bowtie_{\text{Dnum=Dnumero}} (\pi_{\text{Dnumero, Cpf}}(\text{DEPARTAMENTO} \bowtie_{\text{Cpf_gerente=Cpf}} (\pi_{\text{Unome, Dnumero}}(\text{FUNCIONARIO}) \bowtie_{\text{Unome='Silva'}} \pi_{\text{Cpf}}(\sigma_{\text{Unome='Silva'}}(\text{FUNCIONARIO})))))))$$

Consulta 5. Listar os nomes de todos os funcionários com dois ou mais dependentes.

Estritamente falando, esta consulta não pode ser feita na *álgebra relacional básica* (*srcinal*). Temos de usar a operação FUNÇÃO DE AGREGAÇÃO com a função de agregação CONTA. Consideramos que os dependentes do *mesmo* funcionário têm valores *distintos* de Nome_dependente.

$$\rho_{T1(\text{Cpf, Nr_de_dependentes})} \leftarrow {}_{\text{Fcpf}}\Im_{\text{COUNT Nome_dependente}}(\text{DEPENDENTE})$$

$$T2 \leftarrow \sigma_{\text{Nr_de_dependentes}>2}(T1)$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Unome, Pnome}}(T2 * \text{FUNCIONARIO})$$

Consulta 6. Recuperar os nomes dos funcionários que não possuem dependentes.

Este é um exemplo do tipo de consulta que usa a operação SUBTRAÇÃO (DIFERENÇA DE CONJUNTO).

$$\text{TODOS_FUNCS} \leftarrow \pi_{\text{Cpf}}(\text{FUNCIONARIO})$$

$$\text{FUNCS_COM_DEPEND}(\text{Cpf}) \leftarrow \pi_{\text{Fcpf}}(\text{DEPENDENTE})$$

$$\text{FUNCS_SEM_DEPEND} \leftarrow (\text{TODOS_FUNCS} - \text{FUNCS_COM_DEPEND})$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Unome, Pnome}}(\text{FUNCS_SEM_DEPEND} * \text{FUNCIONARIO})$$

Primeiro, recuperamos uma relação com todos os Cpfs de funcionários em TODOS_FUNCS. Depois, criamos uma tabela com os Cpfs dos funcionários que possuem pelo menos um dependente em FUNCS_COM_DEPEND. Então, aplicamos a operação DIFERENÇA DE CONJUNTO para recuperar os Cpfs de funcionários sem dependentes em FUNCS_SEM_DEPEND, e finalmente juntamos isso com FUNCIONARIO para recuperar os atributos desejados. Como uma única expressão em linha, esta consulta se torna:

$$\pi_{\text{Unome, Pnome}}((\pi_{\text{Cpf}}(\text{FUNCIONARIO}) - \rho_{\text{Cpf}}(\pi_{\text{Fcpf}}(\text{DEPENDENTE}))) * \text{FUNCIONARIO})$$

Consulta 7. Listar os nomes dos gerentes que têm pelo menos um dependente.

$$\text{GERS}(\text{Cpf}) \leftarrow \pi_{\text{Cpf_gerente}}(\text{DEPARTAMENTO})$$

$$\text{FUNCS_COM_DEPEND}(\text{Cpf}) \leftarrow \pi_{\text{Fcpf}}(\text{DEPENDENTE})$$

$$\text{GER_COM_DEPEND} \leftarrow (\text{GERS} \cap \text{FUNCS_COM_DEPEND})$$

$$\text{RESULTADO} \leftarrow \pi_{\text{Unome, Pnome}}(\text{GERS_COM_DEPEND} * \text{FUNCIONARIO})$$

Nessa consulta, recuperamos os Cpfs dos gerentes em GERS, e os Cpfs dos funcionários com pelo menos um dependente em FUNCS_COM_DEPEND, depois aplicamos a operação INTERSECÇÃO para obter os Cpfs dos gerentes que têm pelo menos um dependente.

Conforme mencionamos anteriormente, a mesma consulta pode ser especificada de muitas maneiras diferentes na álgebra relacional. Em particular, as operações podem ser aplicadas com frequência em diversas ordens. Além disso, algumas delas podem ser usadas para substituir outras. Por exemplo, a operação INTERSECÇÃO em C7 pode ser substituída por uma JUNÇÃO NATURAL. Como exercício, tente fazer cada um desses exemplos de consulta usando diferentes operações. [12] Mostramos como escrever consultas como expressões isoladas da álgebra relacional para as consultas C1, C4 e C6. Tente escrever as consultas restantes como expressões isoladas. Nos capítulos 4 e 5 e nas seções 6.6 e 6.7, mostramos como essas consultas são escritas em outras linguagens relacionais.

6.6 O cálculo relacional de tupla

Nesta e na próxima seção, apresentamos outra linguagem de consulta formal para o modelo relacional, chamada **cálculo relacional**. Esta seção apresenta a linguagem conhecida como **cálculo relacional de tupla**, e a Seção 6.7 apresenta uma variação chamada **cálculo relacional de domínio**. Nas duas variações do cálculo relacional, escrevemos uma expressão **declarativa** para especificar uma solicitação de recuperação; logo, não existe descrição

[12] Quando as consultas são otimizadas (ver Capítulo 19), o sistema escolherá uma sequência de operações em particular que corresponde a uma estratégia de execução que pode ser executada de modo eficiente.

de como, ou *em que ordem*, avaliar uma consulta. Uma expressão de cálculo especifica *o que* deve ser recuperado em vez de *como* recuperá-lo. Portanto, o cálculo relacional é considerado uma linguagem **não procedimental**. Isso difere da álgebra relacional, na qual precisamos escrever uma *sequência de operações* para especificar uma solicitação de recuperação *em uma ordem particular* da aplicação das operações. Assim, ela pode ser considerada um modo **procedimental** de indicar uma consulta. É possível aninhar operações da álgebra para formar uma única expressão; entretanto, certa ordem entre as operações é sempre explicitamente especificada em uma expressão da álgebra relacional. Essa ordem também influencia a estratégia para avaliar a consulta. Uma expressão de cálculo pode ser escrita de maneiras diferentes, mas o modo como ela é escrita não tem relação com o modo como uma consulta deve ser avaliada.

Foi mostrado que qualquer recuperação que possa ser especificada na álgebra relacional básica também pode ser especificada no cálculo relacional e vice-versa. Em outras palavras, o **poder expressivo** das linguagens é *idêntico*. Isso levou à definição do conceito de uma linguagem *relacionalmente completa*. Uma linguagem de consulta relacional L é considerada **relacionalmente completa** se pudermos expressar em L qualquer consulta que possa ser expressa no cálculo relacional. A integralidade relacional se tornou uma base importante para comparar o poder expressivo das linguagens de consulta de alto nível. No entanto, como vimos na Seção 6.4, certas consultas frequentemente exigidas nas aplicações de banco de dados não podem ser expressas na álgebra ou no cálculo relacional básico. A maioria das linguagens de consulta relacional é relacionalmente completa, mas possui *mais poder expressivo* do que a álgebra relacional ou o cálculo relacional, por causa de operações adicionais como funções de agregação, agrupamento e ordenação. Conforme mencionamos na introdução deste capítulo, o cálculo relacional é importante por dois motivos. Primeiro, ele tem uma base firme na lógica matemática. Segundo, a linguagem de consulta padrão (SQL) para SGBDRs tem alguns de seus alicerces no cálculo relacional de tupla.

Nossos exemplos se referem ao banco de dados mostrado nas figuras 3.6 e 3.7. Usaremos as mesmas consultas da Seção 6.5. As seções 6.6.6, 6.6.7 e 6.6.8 discutem o tratamento com quantificadores universais e as questões de segurança de expressão. (Os alunos interessados em uma introdução básica ao cálculo relacional de tupla podem pular essas seções.)

6.6.1 Variáveis de tupla e relações de intervalo

O cálculo relacional de tupla é baseado na especificação de uma série de **variáveis de tupla**. Cada variável de tupla costuma *percorrer* determinada relação do banco de dados, significando que pode tomar como seu valor qualquer tupla individual dessa relação. Um cálculo relacional de tupla simples tem a forma:

$$\{t \mid \text{COND}(t)\}$$

onde t é uma variável de tupla e COND(t) é uma expressão condicional (booleana) que envolve t e que é avaliada como TRUE ou FALSE para diferentes atribuições de tuplas à variável t. O resultado dessa consulta é o conjunto de todas as tuplas t que avaliam COND(t) como TRUE. Diz-se que essas tuplas **satisfazem** COND(t). Por exemplo, para encontrar todos os funcionários cujo salário é maior que R$50.000,00, podemos escrever a seguinte expressão de cálculo de tupla:

$$\{t \mid \text{FUNCIONARIO}(t) \textbf{ AND } t.\text{Salario} > 50.000\}$$

A condição FUNCIONARIO(t) especifica que a **relação de intervalo** da variável de tupla t é FUNCIONARIO. Cada tupla de FUNCIONARIO t que satisfaz a condição t.Salario>50.000 será recuperada. Observe que t.Salario referencia o atributo Salario da variável de tupla t. Tal notação é semelhante ao modo como os nomes de atributo são qualificados com nomes de relação ou apelidos em SQL, como vimos no Capítulo 4. Na notação do Capítulo 3, t.Salario é o mesmo que escrever t[Salario].

A consulta anterior recupera todos os valores de atributo para cada tupla t de FUNCIONARIO selecionada. Para recuperar apenas *alguns* dos atributos — digamos, o nome e o sobrenome —, escrevemos:

$$\{t.\text{Pnome}, t.\text{Unome} \mid \text{FUNCIONARIO}(t) \textbf{ AND } t.\text{Salario} > 50.000\}$$

Informalmente, precisamos especificar a seguinte informação em uma expressão de cálculo relacional de tupla:

- Para cada variável de tupla t, a **relação de intervalo** R de t. Esse valor é especificado por uma condição na forma $R(t)$. Se não especificarmos uma relação de intervalo, então a variável t oscilará por todas as tuplas possíveis 'no universo', pois ela não é restrita a nenhuma relação isolada.
- Uma condição para selecionar combinações de tuplas em particular. À medida que as va-

riáveis de tupla oscilam em suas respectivas relações de intervalo, a condição é avaliada em cada combinação possível de tuplas, a fim de identificar as **combinações selecionadas** para as quais a condição é avaliada com TRUE.

- Um conjunto de atributos a serem recuperados, os **atributos solicitados**. Os valores desses atributos são recuperados para cada combinação de tuplas selecionada.

Antes de discutirmos a sintaxe formal do cálculo relacional de tupla, considere outra consulta.

Consulta 0. Recuperar a data de nascimento e o endereço do funcionário (ou funcionários) cujo nome é João B. Silva.

C0: {*t*.Datanasc, *t*.Endereco | FUNCIONARIO(*t*) **AND** *t*.Pnome='João' **AND** *t*.Minicial='B' **AND** *t*.Unome='Silva'}

No cálculo relacional de tupla, primeiro especificamos os atributos solicitados *t*.Datanasc e *t*.Endereco para cada tupla *t* selecionada. Depois, especificamos a condição para selecionar uma tupla após a barra (|) — a saber, que *t* seja uma tupla da relação FUNCIONARIO cujos valores de atributo Pnome, Minicial e Unome são 'João', 'B' e 'Silva', respectivamente.

6.6.2 Expressões e fórmulas no cálculo relacional de tupla

Uma **expressão** geral do cálculo relacional de tupla tem a forma

$$\{t_1.A_j, t_2.A_k, \ldots, t_n.A_m \mid \text{COND}(t_1, t_2, \ldots, t_n, t_{n+1}, t_{n+2}, \ldots, t_{n+m})\}$$

onde $t_1, t_2, \ldots, t_n, t_{n+1}, \ldots, t_{n+m}$ são variáveis de tupla, cada A_i é um atributo da relação em que t_i varia e COND é uma **condição** ou **fórmula**[13] do cálculo relacional de tupla. Uma fórmula é composta de **átomos** de cálculo de predicado, que podem ser um dos seguintes:

1. Um átomo da forma $R(t_i)$, onde R é um nome de relação e t_i é uma variável de tupla. Esse átomo identifica o intervalo da variável de tupla t_i como a relação cujo nome é R. Ele é avaliado como TRUE se t_i for uma tupla na relação R, e como FALSE em caso contrário.
2. Um átomo da forma $t_i.A$ **op** $t_j.B$, onde **op** é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$, t_i e t_j são variáveis de tupla, A é um atributo da relação em que t_i varia, e B é um atributo da relação em que t_j varia.
3. Um átomo da forma $t_i.A$ **op** c ou c **op** $t_j.B$, onde **op** é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$, t_i e t_j são variáveis de tupla, A é um atributo da relação sobre a qual t_i varia, B é um atributo da relação em que t_j varia e c é um valor constante.

Cada um dos átomos anteriores é avaliado como TRUE ou FALSE para uma combinação específica de tuplas, o que é chamado de **valor verdade** do átomo. Em geral, uma variável de tupla t ocorre em todas as tuplas possíveis *no universo*. Para átomos da forma $R(t)$, se t for atribuído a uma tupla que é um *membro da relação especificada* R, o átomo é TRUE; caso contrário, ele é FALSE. Em átomos dos tipos 2 e 3, se as variáveis de tupla forem atribuídas a tuplas de modo que os valores dos atributos especificados delas satisfaçam a condição, então o átomo é TRUE.

Uma **fórmula** (condição booleana) é composta de um ou mais átomos conectados por meio dos operadores lógicos AND, OR e NOT e é definida recursivamente pelas regras 1 e 2 da seguinte forma:

- *Regra 1*: todo átomo é uma fórmula.
- *Regra 2*: se F_1 e F_2 são fórmulas, então o mesmo ocorre para (F_1 **AND** F_2), (F_1 **OR** F_2), **NOT** (F_1) e **NOT** (F_2). Os valores verdade dessas fórmulas são derivados de suas fórmulas componentes F_1 e F_2, como se segue:

a. (F_1 **AND** F_2) é TRUE se tanto F_1 quanto F_2 forem TRUE; caso contrário, é FALSE.
b. (F_1 **OR** F_2) é FALSE se tanto F_1 quanto F_2 forem FALSE; caso contrário, é TRUE.
c. **NOT** (F_1) é TRUE se F_1 for FALSE; é FALSE se F_1 for TRUE.
d. **NOT** (F_2) é TRUE se F_2 for FALSE; é FALSE se F_2 for TRUE.

6.6.3 Os quantificadores existenciais e universais

Além disso, dois símbolos especiais, chamados **quantificadores**, podem aparecem nas fórmulas: o **quantificador universal** ($\forall$) e o **quantificador existencial** ($\exists$). Os valores verdade para fórmulas com quantificadores são descritos nas regras 3 e 4 a seguir. Primeiro, definir os conceitos de variáveis de tupla livre e ligada em uma fórmula. Informalmente, uma variável de tupla t é ligada se for quantificada, significando que ela aparece em uma cláusula

[13] Também chamada de **fórmula bem formada**, ou **WFF** (**well-formed formula**) em lógica matemática.

$(\exists t)$ ou $(\forall t)$; caso contrário, ela é livre. De maneira formal, definimos uma variável de tupla em uma fórmula como **livre** ou **ligada** de acordo com as seguintes regras:

- Uma ocorrência de uma variável de tupla em uma fórmula F que *é um átomo* é livre em F.
- Uma ocorrência de uma variável de tupla t é livre ou vinculada em uma fórmula composta de conectivos lógicos — (F_1 **AND** F_2), (F_1 **OR** F_2), **NOT**(F_1) e **NOT**(F_2) — dependendo dela ser livre ou ligada em F_1 ou F_2 (se ocorrer em uma delas). Observe que, em uma fórmula com a forma $F = (F_1$ **AND** $F_2)$ ou $F = (F_1$ **OR** $F_2)$, uma variável de tupla pode ser livre em F_1 e ligada em F_2, ou vice-versa. Nesse caso, uma ocorrência da variável de tupla é ligada e a outra é livre em F.
- Todas as ocorrências *livres* de uma variável de tupla t em F são **ligadas** em uma fórmula F' da forma $F' = (\exists t)(F)$ ou $F' = (\forall t)(F)$. A variável de tupla é ligada ao quantificador especificado em F'. Por exemplo, considere as seguintes fórmulas:

  F_1: d.Dnome='Pesquisa'

  F_2: $(\exists t)(d$.Dnumero=t.Dnr)

  F_3: $(\forall d)(d$.Cpf_gerente='33344555587')

A variável de tupla d é livre em F_1 e F_2, a não ser que seja ligada a um quantificador ($\forall$) em F_3. A variável t é ligada ao quantificador ($\exists$) em F_2.

Agora, podemos dar as regras 3 e 4 para a definição de uma fórmula que iniciamos anteriormente:

- *Regra 3*: se F é uma fórmula, então o mesmo vale para $(\exists t)(F)$, onde t é uma variável de tupla. A fórmula $(\exists t)(F)$ é TRUE se a fórmula F é avaliada como TRUE para *alguma* (pelo menos uma) tupla atribuída a ocorrências livres de t em F; caso contrário, $(\exists t)(F)$ é FALSE.
- *Regra 4*: se F é uma fórmula, então o mesmo vale para $(\forall t)(F)$, onde t é uma variável de tupla. A fórmula $(\forall t)(F)$ é TRUE se a fórmula F é avaliada como TRUE para *cada tupla* (no universo) atribuída a ocorrências livres de t em F; caso contrário, $(\forall t)(F)$ é FALSE.

O quantificador $(\exists)$ é chamado de quantificador existencial porque uma fórmula $(\exists t)(F)$ é TRUE se *existir* alguma tupla que torne F TRUE. Para o quantificador universal, $(\forall t)(F)$ é TRUE se cada tupla possível que pode ser atribuída a ocorrências livres de t em F for substituída por t, e F é TRUE para *cada substituição desse tipo*. Ele é chamado quantificador universal ou *para cada* porque cada tupla no *universo de* tuplas precisa tornar F TRUE para tornar a fórmula quantificada TRUE.

6.6.4 Exemplo de consultas no cálculo relacional de tupla

Usaremos algumas das mesmas consultas da Seção 6.5 para dar uma ideia de como elas são especificadas na álgebra relacional e no cálculo relacional. Observe que algumas consultas são mais fáceis de especificar na álgebra relacional do que no cálculo relacional, e vice-versa.

Consulta 1. Listar o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

C1: {t.Pnome, t.Unome, t.Endereco| FUNCIONARIO(t) **AND** $(\exists d)$(DEPARTAMENTO(d) **AND** d.Dnome='Pesquisa'**AND** d.Dnumero=t.Dnr)}

As *únicas variáveis de tupla livres* em uma expressão de cálculo relacional de tupla devem ser aquelas que aparecem à esquerda da barra (|). Em C1, t é a única variável livre; depois, ela é *ligada sucessivamente* a cada tupla. Se uma tupla *satisfaz as condições* especificadas após a barra em C1, os atributos Pnome, Unome e Endereco são recuperados para cada tupla desse tipo. As condições FUNCIONARIO(t) e DEPARTAMENTO(d) especificam as relações de intervalo para t e d. A condição d.Dnome = 'Pesquisa' é uma **condição de seleção** e corresponde a uma operação SELEÇÃO na álgebra relacional, ao passo que a condição d.Dnumero = t.Dnr é uma **condição de junção** e semelhante em finalidade à operação JUNÇÃO INTERNA (ver Seção 6.3).

Consulta 2. Para cada projeto localizado em 'Mauá', listar o número do projeto, o número do departamento de controle e o sobrenome, data de nascimento e endereço do gerente do departamento.

C2: {p.Projnumero, p.Dnum, g.Unome, g.Datanasc, g.Endereco | PROJETO(p) **AND** FUNCIONARIO(g) **AND** p.Projlocalizacao='Mauá' **AND** ($\exists d$) (DEPARTAMENTO(d) **AND** p.Dnum=d.Dnumero **AND** d.Cpf_gerente=g.Cpf)}

Em C2, existem duas variáveis de tupla livres, p e g. A variável de tupla d é ligada ao quantificador existencial. A condição de consulta é avaliada para cada combinação de tuplas atribuídas a p e g, e de todas as combinações possíveis de tuplas às quais p e g estão ligadas, somente as combinações que satisfazem a condição são selecionadas.

Diversas variáveis de tupla em uma consulta podem oscilar na mesma relação. Por exemplo, para especificar C8 — para cada funcionário, recupere o nome e último nome do funcionário e o nome e sobrenome de seu supervisor imediato —, determinamos duas variáveis de tupla *f* e *s* que percorrem a relação FUNCIONARIO:

C8: {*f*.Pnome, *f*.Unome, *s*.Pnome, *s*.Unome | FUNCIONARIO(*f*) **AND** FUNCIONARIO(*s*) **AND** *f*.Cpf_supervisor=s.Cpf}

Consulta 3'. Listar o nome do funcionário que trabalha em *algum* projeto controlado pelo departamento número 5. Essa é uma variação de C3 em que *todos* é mudado para *algum*. Neste caso, precisamos de duas condições de junção e dois quantificadores existenciais.

C0': {*f*.Unome, *f*.Pnome | FUNCIONARIO(*f*) **AND** ((∃*p*)(∃*t*)(PROJETO(*p*) **AND** TRABALHA_EM(*t*) **AND** *p*.Dnum=5 **AND** *t*.Fcpf=*f*.Cpf **AND** *p*.Projnumero=*t*.Pnr))}

Consulta 4. Fazer uma lista dos números de projeto que envolvem um funcionário cujo último nome é 'Silva', seja como um trabalhador ou como um gerente do departamento de controle para o projeto.

C4: {*p*.Projnumero | PROJETO(*p*) **AND** (((∃*f*)(∃*t*)(FUNCIONARIO(*f*) **AND** TRABALHA_EM(*t*) **AND** *t*.Pnr=*p*.Projnumero **AND** *f*.Unome='Silva' **AND** *f*.Cpf=*t*.Fcpf))

OR

((∃*g*)(∃*d*)(FUNCIONARIO(*g*) **AND** DEPARTAMENTO(*d*) **AND** *p*.Dnum=*d*.Dnumero **AND** *d*.Cpf_gerente=*g*.Cpf **AND** *g*.Unome='Silva'))))}

Compare isso com a versão da álgebra relacional dessa consulta na Seção 6.5. A operação UNIÃO na álgebra relacional normalmente pode ser substituída por um conectivo OR no cálculo relacional.

6.6.5 Notação para grafos de consulta

Nesta seção, descrevemos uma notação que foi proposta para representar as consultas do cálculo relacional que não envolvem quantificação complexa em uma forma gráfica. Esses tipos de consultas são conhecidos como **consultas seleção-projeção-junção**, pois envolvem essas três operações da álgebra relacional. A notação pode ser expandida para consultas mais gerais, mas não vamos discutir essas extensões aqui. Essa representação gráfica de uma consulta é chamada de **grafo de consulta**. A Figura 6.13 mostra o grafo de consulta para C2. As relações na consulta são representadas por **nós de relação**, que aparecem como círculos isolados. Valores constantes, normalmente das condições de seleção de consulta, são representados por **nós de constante**, que aparecem como círculos ou ovais duplas. As condições de seleção e junção são representadas pelas **arestas** do grafo (as linhas que conectam os nós), como mostra a Figura 6.13. Finalmente, os atributos a serem recuperados de cada relação são exibidos entre colchetes acima de cada uma delas.

A representação do grafo de consulta não indica uma ordem em particular para especificar quais operações realizar primeiro e, portanto, é uma representação mais neutra de uma consulta seleção-projeção-junção do que a representação na árvore de consulta (ver Seção 6.3.5), onde a ordem de execução é especificada de maneira implícita. Existe apenas um grafo de consulta correspondente a cada consulta. Embora algumas técnicas de otimização de consulta fossem baseadas em grafos de consulta, agora as árvores de consulta são preferíveis porque, na prática, o otimizador de consulta precisa mostrar a ordem das operações para a execução da consulta, o que não é possível nos grafos de consulta.

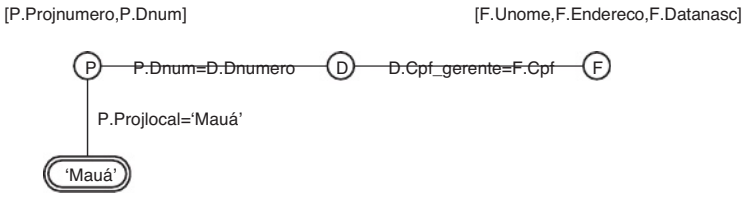


Figura 6.13
Grafo de consulta para C2.

Na próxima seção, discutiremos o relacionamento entre os quantificadores universais e existenciais e mostraremos como um pode ser transformado no outro.

6.6.6 Transformando os quantificadores universais e existenciais

Agora, vamos apresentar algumas transformações bem conhecidas da lógica matemática que se relacionam aos quantificadores universais e existenciais. É possível transformar um quantificador universal em um quantificador existencial, e vice-versa, para obter uma expressão equivalente. Uma transformação geral pode ser descrita informalmente da seguinte forma: transformar um tipo de quantificador no outro com a negação (precedida por **NOT**); **AND** e **OR** substituem um ao outro; uma fórmula negada torna-se não negada; e uma fórmula não negada torna-se negada. Alguns casos especiais dessa transformação podem ser declarados da seguinte forma, onde o símbolo ≡ significa **equivale a:**

$(\forall x)\ (P(x)) \equiv$ **NOT** $(\exists x)\ ($**NOT** $(P(x)))$

$(\exists x)\ (P(x)) \equiv$ **NOT** $(\forall x)\ ($**NOT** $(P(x)))$

$(\forall x)\ (P(x)$ **AND** $Q(x)) \equiv$ **NOT** $(\exists x)\ ($**NOT** $(P(x))$ **OR** **NOT** $(Q(x)))$

$(\forall x)\ (P(x)$ **OR** $Q(x)) \equiv$ **NOT** $(\exists x)\ ($**NOT** $(P(x))$ **AND** **NOT** $(Q(x)))$

$(\exists x)\ (P(x)$ **OR** $Q(x)) \equiv$ **NOT** $(\forall x)\ ($**NOT** $(P(x))$ **AND** **NOT** $(Q(x)))$

$(\exists x)\ (P(x)$ **AND** $Q(x)) \equiv$ **NOT** $(\forall x)\ ($**NOT** $(P(x))$ **OR** **NOT** $(Q(x)))$

Observe também que o seguinte é TRUE, onde o símbolo ⇒ significa **implica**:

$(\forall x)(P(x)) \Rightarrow (\exists x)(P(x))$

NOT $(\exists x)(P(x)) \Rightarrow$ **NOT** $(\forall x)(P(x))$

6.6.7 Usando o quantificador universal nas consultas

Sempre que usamos um quantificador universal, é prudente seguir algumas regras para garantir que nossa expressão faça sentido. Discutimos essas regras com relação à consulta C3.

Consulta 3. Listar os nomes dos funcionários que trabalham em *todos* os projetos controlados pelo departamento número 5. Um modo de especificar essa consulta é usar o quantificador universal, conforme mostrado:

C3: {*f*.Unome, *f*.Pnome | FUNCIONARIO(*f*) **AND** ((∀*p*)(**NOT**(PROJETO(*p*)) **OR NOT** (*x*.Dnum=5) OR ((∃*t*)(TRABALHA_EM(*t*) **AND** *t*.Fcpf=*f*.Cpf **AND** *p*.Projnumero=*t*.Pnr)))))}

Podemos desmembrar C3 em seus componentes básicos como se segue:

C3: {*f*.Unome, *f*.Pnome | FUNCIONARIO(*f*) **AND** *F*'}
F' = ((∀*p*)(**NOT**(PROJETO(*p*)) **OR** F_1))
F_1 = **NOT**(*p*.Dnum=5) **OR** F_2
F_2 = ((∃*t*)(TRABALHA_EM(*t*) **AND** *t*.Fcpf=*f*.Cpf **AND** *p*.Projnumero=*t*.Pnr))

Queremos garantir que um funcionário selecionado *f* trabalha em *todos os projetos* controlados pelo departamento 5, mas a *definição de quantificador universal* diz que, para tornar a fórmula quantificada TRUE, *a fórmula interna* precisa ser TRUE *para todas as tuplas no universo*. O truque é excluir da quantificação universal todas as tuplas em que não estamos interessados, tornando a condição TRUE *para todas essas tuplas*. Isso é necessário porque uma variável de tupla universalmente quantificada, como *p* em C3, precisa ser avaliada como TRUE *para cada tupla possível* atribuída a ela, para tornar a fórmula quantificada TRUE.

As primeiras tuplas a excluir (fazendo que sejam avaliadas automaticamente como TRUE) são aquelas que não estão na relação *R* de interesse. Em C3, o uso da expressão **NOT**(PROJETO(*p*)) na fórmula quantificada universalmente avalia como TRUE todas as tuplas *x* que não estão na relação PROJETO. Depois, excluímos as tuplas em que não estamos interessados da própria *R*. Em C3, o uso da expressão **NOT**(*p*.Dnum=5) avalia como TRUE todas as tuplas *p* que estão na relação PROJETO, mas não são controladas pelo departamento 5. Por fim, especificamos uma condição F_2 que precisa ser mantida sobre todas as tuplas restantes em *R*. Logo, podemos explicar C3 da seguinte forma:

1. Para a fórmula *F*' = (∀*p*)(*F*) ser TRUE, precisamos fazer que a fórmula *F* seja TRUE *para todas as tuplas no universo que possam ser atribuídas a p*. Porém, em C3 só estamos interessados em *F* ser TRUE para todas as tuplas da relação PROJEÇÃO que são controladas pelo departamento 5. Logo, a fórmula *F* tem a forma (**NOT**(PROJETO(*p*)) **OR** F_1). A condição '**NOT** (PROJETO(*p*)) **OR** ...' é TRUE para todas as tuplas que *não estão na relação* PROJETO e tem o efeito de eliminar essas tuplas da consideração no valor verdade de F_1. Para cada tupla na relação PROJETO, F_1 precisa ser TRUE se *F*' tiver de ser TRUE.

2. Usando a mesma linha de raciocínio, não queremos considerar tuplas na relação PROJETO que não sejam controladas pelo departamento número 5, pois só estamos interessados em tuplas de PROJETO cujo Dnum=5. Portanto, podemos escrever:

   IF (*p*.Dnum=5) **THEN** F_2

   que é equivalente a

   (**NOT** (*p*.Dnum=5) **OR** F_2)

3. Logo, a fórmula F_1 tem a forma **NOT** (*p*.Dnum=5) **OR** F_2. No contexto de C3, isso significa que, para uma tupla *p* na relação PROJETO, seu Dnum≠5 ou ela precisa satisfazer F_2.
4. Finalmente, F_2 dá a condição que queremos manter para uma tupla selecionada FUNCIONARIO: que o funcionário trabalhe em *cada tupla de* PROJETO *que ainda não tenha sido excluída*. Essas tuplas de funcionários são selecionadas pela consulta.

Em português, C3 gera a seguinte condição para selecionar uma tupla de FUNCIONARIO *f*: para cada tupla *p* na relação PROJETO com *p*.Dnum=5, é preciso que haja uma tupla *t* em TRABALHA_EM tal que *t*.Fcpf=*f*.Cpf e *t*.Pnr=*p*.Projnumero. Isso equivale a dizer que FUNCIONARIO *f* trabalha em cada PROJETO *p* no DEPARTAMENTO número 5.

Usando a transformação geral dos quantificadores universal para existencial, dada na Seção 6.6.6, podemos reformular a consulta em C3 como mostramos em C3A, que usa um quantificador existencial negado em vez do quantificador universal:

C3A: {*f*.Unome, *f*.Pnome | FUNCIONARIO(*f*) **AND** (**NOT** (∃*p*) (PROJETO(*p*) **AND** (*p*.Dnum=5) **AND** (**NOT** (∃*t*)(TRABALHA_EM(*t*) **AND** *t*.Fcpf=*f*.Cpf **AND** *p*.Projnumero=*t*.Pnr)))))}

Agora, mostramos alguns exemplos adicionais de consultas que usam quantificadores.

Consulta 6. Listar os nomes de funcionários que não possuem dependentes.

C6: {*f*.Pnome, *f*.Unome | FUNCIONARIO(*f*) **AND** (**NOT** (∃*d*)(DEPENDENTE(*d*) **AND** *f*.Cpf=*d*.Fcpf))}

Usando a regra da transformação geral, podemos reformular C6 da seguinte forma:

C6A: {*f*.Pnome, *f*.Unome | FUNCIONARIO(*f*) **AND** ((∀*d*)(**NOT**(DEPENDENTE(*d*)) **OR NOT**(*f*.Cpf=*d*.Fcpf)))}

Consulta 7. Listar os nomes dos gerentes que possuem pelo menos um dependente.

C7: {*f*.Pnome, *f*.Unome | FUNCIONARIO(*f*) **AND** ((∃*d*)(∃*p*)(DEPARTAMENTO(*d*) **AND** DEPENDENTE(p) **AND** *f*.Cpf=*d*.Cpf_gerente **AND** *p*.Fcpf=*f*.Cpf))}

Essa consulta é tratada interpretando os *gerentes que possuem pelo menos um dependente como gerentes para os quais existe algum dependente.*

6.6.8 Expressões seguras

Sempre que usamos quantificadores universais, quantificadores existenciais ou negação de predicados em uma expressão de cálculo, temos de garantir que a expressão resultante faça sentido. Uma **expressão segura** em cálculo relacional é aquela que garante a geração de um *número finito de tuplas* como resultado; caso contrário, a expressão é chamada de **insegura**. Por exemplo, a expressão

{*t* | **NOT** (FUNCIONARIO(*t*))}

é *insegura*, pois gera todas as tuplas no universo que *não são* tuplas de FUNCIONARIO, e que são infinitamente numerosas. Se seguirmos as regras para C3, discutidas anteriormente, obteremos uma expressão segura ao usar quantificadores universais. Podemos definir expressões seguras com mais precisão introduzindo o conceito do *domínio de uma expressão de cálculo relacional de tupla*: este é o conjunto de todos os valores que aparecem como valores constantes na expressão ou existem em qualquer tupla nas relações referenciadas na expressão. Por exemplo, o domínio de {*t* | **NOT**(FUNCIONARIO(*t*))} é o conjunto de todos os valores de atributo que aparecem em alguma tupla da relação FUNCIONARIO (para qualquer atributo). O domínio da expressão C3A incluiria todos os valores que aparecem em FUNCIONARIO, PROJETO e TRABALHA_EM (unidos com o valor 5 aparecendo na própria consulta).

Uma expressão é considerada **segura** se todos os valores em seu resultado forem do domínio da expressão. Observe que o resultado de {*t* | **NOT**(FUNCIONARIO(*t*))} é inseguro, pois, em geral, incluirá tuplas (e, portanto, valores) de fora da relação FUNCIONARIO. Esses valores não estão no domínio da expressão. Todos os outros exemplos são expressões seguras.

6.7 O cálculo relacional de domínio

Existe outro tipo de cálculo relacional chamado cálculo relacional de domínio, ou apenas **cálculo de domínio**. Historicamente, enquanto a SQL (ver capítulos 4 e 5), que foi baseada no cálculo relacional de tupla, estava sendo desenvolvida pela IBM Research em San Jose, Califórnia, outra linguagem, chamada QBE (Query-By-Example), que está relacionada ao cálculo de domínio, estava sendo desenvolvida quase simultaneamente no IBM T. J. Watson Research Center em Yorktown Heights, Nova York. A especificação formal do cálculo de domínio foi proposta após o desenvolvimento da linguagem e sistema em QBE.

O cálculo de domínio difere do cálculo de tupla no *tipo das variáveis* usadas nas fórmulas: em vez de ter variáveis percorrendo as tuplas, elas o fazem por valores isolados dos domínios de atributos. Para formar uma relação de grau n para um resultado de consulta, precisamos ter n dessas **variáveis de domínio** — uma para cada atributo. Uma expressão do cálculo de domínio tem a forma

$$\{x_1, x_2, ..., x_n \mid \text{COND}(x_1, x_2, ..., x_n, x_{n+1}, x_{n+2}, ..., x_{n+m})\}$$

onde $x_1, x_2, ..., x_n, x_{n+1}, x_{n+2}, ..., x_{n+m}$ são variáveis de domínio que percorrem os domínios (dos atributos) e COND é uma **condição** ou **fórmula** do cálculo relacional do domínio.

Uma fórmula é composta de **átomos**. Os átomos de uma fórmula são ligeiramente diferentes daqueles para o cálculo de tupla e podem ser um dos seguintes:

1. Um átomo da forma $R(x_1, x_2, ..., x_j)$, onde R é o nome de uma relação de grau j e cada x_i, $1 \leq i \leq j$, é uma variável de domínio. Esse átomo declara que uma lista de valores de $<x_1, x_2, ..., x_j>$ deve ser uma tupla na relação cujo nome é R, onde x_i é o valor do i-ésimo valor de atributo da tupla. Para tornar uma expressão de cálculo de domínio mais concisa, podemos *remover as vírgulas* em uma lista de variáveis, escrevendo assim:

   $$\{x_1, x_2, ..., x_n \mid R(x_1\ x_2\ x_3) \text{ AND } ...\}$$

   no lugar de

   $$\{x^1, x^2, ..., x^n \mid R(x^1, x^2, x^3) \text{ AND } ...\}$$

2. Um átomo da forma x_i **op** x_j, onde **op** é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$ e x_i e x_j são variáveis de domínio.

3. Um átomo da forma x_i **op** c ou c **op** x_j, onde **op** é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$, x_i e x_j são variáveis de domínio, e c é um valor constante.

Assim como no cálculo de tupla, os átomos são avaliados como TRUE ou FALSE para um conjunto de valores específico, chamados **valores verdade** dos átomos. No caso 1, se as variáveis de domínio receberem valores correspondentes a uma tupla da relação especificada R, então o átomo é TRUE. Nos casos 2 e 3, se as variáveis de domínio receberem valores que satisfazem a condição, então o átomo é TRUE.

De um modo semelhante ao cálculo relacional de tupla, as fórmulas são compostas de átomos, variáveis e quantificadores, de modo que não repetiremos as especificações de fórmulas aqui. A seguir, apresentamos alguns exemplos de consultas especificadas no cálculo de domínio. Usaremos as letras minúsculas l, m, n, ..., x, y, z para as variáveis de domínio.

Consulta 0. Listar a data de nascimento e o endereço do funcionário cujo nome é 'João B. Silva'.

C0: $\{u, v \mid (\exists q)\ (\exists r)\ (\exists s)\ (\exists t)\ (\exists w)\ (\exists x)\ (\exists y)\ (\exists z)$ (FUNCIONARIO($qrstuvwxyz$) **AND** q='João' **AND** r='B' **AND** s='Silva')}

Precisamos de dez variáveis para a relação FUNCIONARIO, uma para percorrer cada um dos domínios dos atributos de FUNCIONARIO em ordem. Das dez variáveis q, r, s, ..., z, somente u e v são livres, pois aparecem à esquerda da barra e, portanto, não devem estar ligadas a um quantificador. Primeiro, especificamos os *atributos solicitados*, Datanasc e Endereco, pelas variáveis de domínio livres u para DATANASC e v para ENDERECO. Depois, especificamos a condição para selecionar uma tupla após a barra (|) — a saber, que a sequência de valores atribuídos às variáveis *qrstuvwxyz* seja uma tupla da relação FUNCIONARIO e que os valores para q (Pnome), r (Minicial) e s (Unome) sejam iguais a 'João', 'B' e 'Silva', respectivamente. Por conveniência, quantificaremos apenas as variáveis que *realmente aparecem em uma condição* (estas seriam q, r e s em C0) no restante de nossos exemplos.[14]

Uma notação abreviada alternativa, usada em QBE, para escrever essa consulta, é atribuir as constantes 'João', 'B' e 'Silva' diretamente como mostramos em C0A. Aqui, todas as variáveis que não aparecem à esquerda da barra são implicitamente quantificadas de maneira existencial:[15]

[14] Observe que a notação de quantificar apenas as variáveis de domínio realmente usadas nas condições e de mostrar um predicado como FUNCIONARIO (*qrstuvwxyz*) sem separar as variáveis de domínio com vírgulas é uma notação abreviada, utilizada por conveniência; essa não é a notação formal correta.

[15] Mais uma vez, esta não é uma notação formalmente precisa.

C0A: {u, v | FUNCIONARIO('João','B','Silva',*t,u, v,w,x,y,z*) }

Consulta 1. Recuperar o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

C1: {*q, s, v* | (∃*z*) (∃*l*) (∃*m*) (FUNCIONARIO (*qrstuvwxyz*) **AND** DEPARTAMENTO(*lmno*) **AND** *l*='Pesquisa' **AND** *m*=*z*)}

Uma condição relacionando duas variáveis de domínio que percorra os atributos de duas relações, como $m = z$ em C1, é uma **condição de junção**, enquanto uma condição que relaciona uma variável de domínio a uma constante, como l = 'Pesquisa', é uma **condição de seleção**.

Consulta 2. Para cada projeto localizado em 'Mauá', listar o número do projeto, o número do departamento de controle e último nome, data de nascimento e endereço do gerente do departamento.

C2: {*i, k, s, u, v* | (∃*j*)(∃*m*)(∃*n*)(∃*t*)(PROJETO(*hijk*) **AND** FUNCIONARIO(*qrstuvwxyz*) **AND** DEPARTAMENTO(*lmno*) **AND** *k*=*m* **AND** *n*=*t* **AND** *j*='Mauá')}

Consulta 6. Listar os nomes dos funcionários que não têm dependentes.

C6: {*q, s* | (∃*t*)(FUNCIONARIO(*qrstuvwxyz*) **AND** (**NOT**(∃*l*)(DEPENDENTE(*lmnop*) **AND** *t*=*l*))}

C6 pode ser reformulada usando quantificadores universais no lugar dos quantificadores existenciais, como mostramos em C6A:

C6A: {*q, s* | (∃*t*)(FUNCIONARIO(*qrstuvwxyz*) **AND** ((∀*l*)(**NOT**(DEPENDENTE(*lmnop*)) **OR** **NOT**(*t*=*l*))))}

Consulta 7. Listar os nomes dos gerentes que têm pelo menos um dependente.

C7: {*s, q* | (∃*t*)(∃*j*)(∃*l*)(FUNCIONARIO(*qrstuvwxyz*) **AND** DEPARTAMENTO(*hijk*) **AND** DEPENDENTE(*lmnop*) **AND** *t*=*j* **AND** *l*=*t*)}

Como dissemos anteriormente, pode-se mostrar que qualquer consulta que pode ser expressa na álgebra relacional básica também pode ser expressa no cálculo relacional de domínio ou de tupla. Além disso, qualquer *expressão segura* no cálculo relacional de domínio ou de tupla pode ser expressa na álgebra relacional básica.

A linguagem QBE foi baseada no cálculo relacional de domínio, embora isso fosse observado mais tarde, depois que o cálculo de domínio foi formalizado. A QBE foi uma das primeiras linguagens de consulta gráficas com sintaxe mínima desenvolvidas para sistemas de banco de dados. Ela foi desenvolvida na IBM Research e está disponível como um produto comercial da empresa como parte da opção de interface Query Management Facility (QMF) para DB2. As ideias básicas usadas na QBE têm sido aplicadas em vários outros produtos comerciais. Devido a seu lugar importante na história das linguagens relacionais, incluímos uma visão geral da QBE no Apêndice C.

Resumo

Neste capítulo, apresentamos duas linguagens formais para o modelo de dados relacional. Elas são usadas para manipular relações e produzir novas relações como respostas às consultas. Discutimos a álgebra relacional e suas operações, que são usadas para especificar uma sequência de operações para determinar uma consulta. Depois, apresentamos dois tipos de cálculos relacionais, chamados cálculo de tupla e cálculo de domínio.

Nas seções de 6.1 a 6.3, apresentamos as operações básicas da álgebra relacional e ilustramos os tipos de consultas para as quais cada uma é usada. Primeiro, discutimos os operadores relacionais unários SELEÇÃO e PROJEÇÃO, bem como a operação RENOMEAR. Depois, discutimos as operações teóricas do conjunto binário exigindo que as relações sobre as quais são aplicadas sejam compatíveis na união (ou tipo); estas incluem UNIÃO, INTERSECÇÃO e DIFERENÇA DE CONJUNTO. A operação PRODUTO CARTESIANO é uma operação de conjunto que pode ser usada para combinar tuplas de duas relações, produzindo todas as combinações possíveis. Ela é raramente usada na prática. Porém, mostramos como o PRODUTO CARTESIANO seguido por SELEÇÃO pode ser usado para definir tuplas combinadas de duas relações e levar à operação JUNÇÃO. Diferentes operações JUNÇÃO, chamadas JUNÇÃO THETA, EQUIJUNÇÃO e JUNÇÃO NATURAL, foram apresentadas. Árvores de consulta foram apresentadas como uma representação gráfica das consultas da álgebra relacional, as quais também podem ser usadas como base para estruturas de dados internas, que o SGBD pode utilizar para representar uma consulta.

Discutimos alguns tipos importantes de consultas que *não podem* ser declaradas com as operações básicas da álgebra relacional, mas são importantes para situações práticas. Apresentamos a PROJEÇÃO GENERALIZADA para usar funções de atributos na lista de projeção e a operação FUNÇÃO DE AGREGAÇÃO para lidar com tipos de agregação das solicitações estatísticas, que resumem as informações nas tabelas. Discutimos sobre

as consultas recursivas, para as quais não existe suporte direto na álgebra, mas que podem ser tratadas de uma maneira passo a passo, conforme demonstramos. Depois, apresentamos as operações JUNÇÃO EXTERNA e UNIÃO EXTERNA, que estendem JUNÇÃO e UNIÃO e permitem que todas as informações nas relações de origem sejam preservadas no resultado.

As duas últimas seções descreveram os conceitos básicos por trás do cálculo relacional, que é baseado no ramo da lógica matemática chamado cálculo de predicado. Existem dois tipos de cálculo relacional: (1) o cálculo relacional de tupla, que usa variáveis de tupla que percorrem as tuplas (linhas) das relações e (2) cálculo relacional de domínio, que usa variáveis de domínio que percorrem os domínios (colunas das relações). No cálculo relacional, uma consulta é especificada em um único comando declarativo, sem determinar qualquer ordem ou método para recuperar o resultado da consulta. Logo, o cálculo relacional normalmente é considerado como uma linguagem *declarativa* de nível mais alto do que a álgebra relacional, pois uma expressão do cálculo relacional declara *o que* queremos recuperar, independentemente de *como* a consulta pode ser executada.

Discutimos a sintaxe das consultas do cálculo relacional usando variáveis de tupla e de domínio. Apresentamos os grafos de consulta como uma representação interna para as consultas no cálculo relacional. Também discutimos o quantificador existencial (∃) e o quantificador universal (∀). Vimos que as variáveis do cálculo relacional são ligadas por esses quantificadores. Descrevemos, com detalhes, como as consultas com quantificação universal são escritas, e discutimos o problema de especificar consultas seguras, cujos resultados são finitos. Também discutimos regras para transformar quantificadores universais em existenciais, e vice-versa. São os quantificadores que dão poder expressivo ao cálculo relacional, tornando-o equivalente à álgebra relacional básica. Não existe algo semelhante para funções de agrupamento e agregação no cálculo relacional básico, embora algumas extensões tenham sido sugeridas.

Perguntas de revisão

6.1. Liste as operações da álgebra relacional e a finalidade de cada uma.

6.2. O que é compatibilidade na união? Por que as operações UNIÃO, INTERSECÇÃO e DIFERENÇA exigem que as relações sobre as quais são aplicadas sejam compatíveis na união?

6.3. Discuta alguns tipos de consultas para as quais a renomeação de atributos é necessária para especificar a consulta de forma não ambígua.

6.4. Discuta os vários tipos de operações de *junção interna* (JUNÇÃO INTERNA). Por que a JUNÇÃO THETA é exigida?

6.5. Que papel é desempenhado pela *chave estrangeira* ao especificar os tipos mais comuns de operações de junção significativas?

6.6. O que é a operação de FUNÇÃO? Para que é usada?

6.7. Como as operações JUNÇÃO EXTERNA diferem das operações JUNÇÃO INTERNA? Como a operação UNIÃO EXTERNA é diferente de UNIÃO?

6.8. Em que sentido o cálculo relacional difere da álgebra relacional, e em que sentido eles são semelhantes?

6.9. Como o cálculo relacional de tupla difere do cálculo relacional de domínio?

6.10. Discuta os significados do quantificador existencial (∃) e do quantificador universal (∀).

6.11. Defina os seguintes termos com relação ao cálculo de tupla: *tupla variável*, *relação de intervalo*, *átomo*, *fórmula* e *expressão*.

6.12. Defina os seguintes termos com relação ao cálculo de domínio: *variável de domínio*, *relação de intervalo*, *átomo*, *fórmula* e *expressão*.

6.13. O que significa uma *expressão segura* no cálculo relacional?

6.14. Quando uma linguagem de consulta é chamada de relacionalmente completa?

Exercícios

6.15. Mostre o resultado de cada um dos exemplos da consulta na Seção 6.5 se fosse aplicado ao estado de banco de dados da Figura 3.6.

6.16. Especifique as seguintes consultas no esquema de banco de dados relacional EMPRESA mostrado na Figura 5.5, usando os operadores relacionais discutidos neste capítulo. Mostre também o resultado de cada consulta como ele se aplica ao estado de banco de dados da Figura 3.6.

a. Recupere o nome de todos os funcionários no departamento 5 que trabalham mais de 10 horas por semana no projeto ProdutoX.

b. Liste o nome de todos os funcionários que têm um dependente com o primeiro nome igual ao seu.

c. Liste o nome de todos os funcionários que são supervisionados diretamente por 'Fernando Wong'.

d. Para cada projeto, liste o nome do projeto e o total de horas por semana (por todos os funcionários) gastas nesse projeto.

e. Recupere o nome de todos os funcionários que trabalham em cada projeto.

f. Recupere o nome de todos os funcionários que não trabalham em projeto algum.

g. Para cada departamento, recupere o nome do departamento e o salário médio de todos os seus funcionários.

h. Recupere o salário médio de todos os funcionários do sexo feminino.

i. Encontre o nome e endereço de todos os funcionários que trabalham em pelo menos um projeto localizado em São Paulo, mas cujo departamento não está localizado lá.

j. Liste o último nome de todos os gerentes de departamento que não possuem dependentes.

6.17. Considere o esquema de banco de dados relacional COMPANHIA AEREA mostrado na Figura 3.8, que foi descrito no Exercício 3.12. Especifique as seguintes consultas em álgebra relacional:

a. Para cada voo, liste o número do voo, o aeroporto de partida para o primeiro trecho e o aeroporto de chegada para o último trecho.

b. Liste os números de voo e dias da semana de todos os voos ou de seus trechos que saem do Aeroporto Internacional de Guarulhos em São Paulo (código de aeroporto 'GRU') e chegam ao Aeroporto Internacional Salgado Filho de Porto Alegre (código de aeroporto 'POA').

c. Liste o número do voo, o código do aeroporto de partida, a hora de partida programada, o código do aeroporto de chegada, a hora de chegada programada e os dias da semana de todos os voos ou trechos que saem de algum aeroporto na cidade de São Paulo e chegam a algum aeroporto na cidade de Porto Alegre.

d. Liste todas as informações de tarifa para o voo número 'CO197'.

e. Recupere o número de assentos disponíveis para o voo número 'CO197' em '09-10-2010'.

6.18. Considere o esquema de banco de dados relacional BIBLIOTECA mostrado na Figura 6.14, que é usado para registrar livros, leitores e empréstimos de livro. As restrições de integridade referencial aparecem como arcos diretos na Figura 6.14, como na notação da Figura 3.7. Escreva as expressões relacionais para as seguintes consultas:

a. Quantas cópias do livro intitulado *A Tribo Perdida* existem na unidade da biblioteca cujo nome é 'Central'?

b. Quantas cópias do livro intitulado *A Tribo Perdida* existem em cada unidade da biblioteca?

c. Recupere o nome de todos os usuários que não possuem livros registrados em seu nome.

d. Para cada livro que é emprestado da unidade de Central e cuja Data_devolucao é hoje, recupere o título do livro, o nome e o endereço do usuário.

e. Para cada unidade da biblioteca, recupere o nome da unidade e o número total de livros retirados de lá.

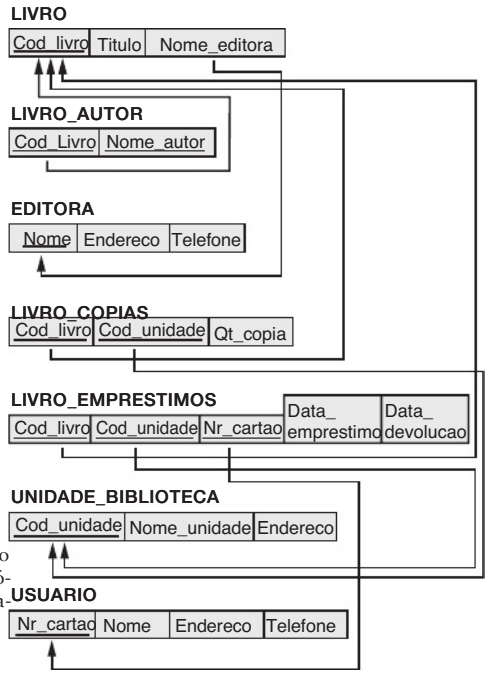


Figura 6.14

Um esquema de banco de dados relacional para um banco de dados BIBLIOTECA.

f. Recupere o nome, endereço e número de livros emprestados para todos os usuários que possuem mais de cinco livros emprestados.

g. Para cada livro cujo autor (ou coautor) é Stephen King, recupere o título e o número de cópias pertencentes à unidade da biblioteca cujo nome é Central.

6.19. Especifique as seguintes consultas na álgebra relacional sobre o esquema de banco de dados mostrado no Exercício 3.14:

a. Liste o Num_Pedido e Data_envio para todos os pedidos enviados do Num_deposito com número 2.

b. Liste a informação do DEPOSITO do qual o CLIENTE chamado Jose Lopez recebeu seus pedidos. Produza uma listagem: Num_pedido, Num_deposito.

c. Produza uma listagem com Nome_cliente, Nr_pedidos, Media_valor_pedido, em que a coluna do meio seja o número total de pedidos feitos pelo cliente e a última coluna seja o valor médio do pedido para esse cliente.

d. Liste os pedidos que foram entregues em até 30 dias após sua solicitação.

e. Liste o Num_pedido para os pedidos que foram entregues de *todos* os depósitos que a empresa tem em Curitiba.

6.20. Especifique as seguintes consultas em álgebra relacional sobre o esquema de banco de dados mostrado no Exercício 3.15:

a. Dê os detalhes (todos os atributos da relação de viagem) para as viagens que excederam R$ 2.000 nas despesas.

b. Imprima o Cpf dos vendedores que realizaram viagens para Pernambuco.

c. Imprima o total de despesas de viagem contratadas pelo vendedor com Cpf = '23456789011'.

6.21. Especifique as seguintes consultas na álgebra relacional sobre o esquema de banco de dados dado no Exercício 3.16:

a. Liste o número de cursos realizados por todos os alunos chamados 'João Silva' no segundo semestre de 2009 (ou seja, Semestre=S09).

b. Produza uma lista dos livros-texto (inclua Num_disciplina, Isbn_livro, Titulo_livro) para os cursos oferecidos pelo departamento 'CC' que usaram mais de dois livros.

c. Liste qualquer departamento que tenha todos os livros adotados publicados pela 'Editora Pearson'.

6.22. Considere as duas tabelas $T1$ e $T2$ mostradas na Figura 6.15. Mostre os resultados das seguintes operações:

a. $T1 \bowtie_{T1.P = T2.A} T2$

b. $T1 \bowtie_{T1.Q = T2.B} T2$

c. $T1 ⟕_{T1.P = T2.A} T2$

d. $T1 ⟖_{T1.Q = T2.B} T2$

e. $T1 \cup T2$

f. $T1 \bowtie_{(T1.P = T2.A \textbf{ AND } T1.R = T2.C)} T2$

TABELA T1

P	Q	R
10	a	5
15	b	8
25	a	6

TABELA T2

A	B	C
10	b	6
25	c	3
10	b	5

Figura 6.15
Um estado do banco de dados para as relações $T1$ e $T2$.

6.23. Especifique as seguintes consultas na álgebra relacional sobre o esquema de banco de dados do Exercício 3.17:

a. Para a vendedora chamada 'Jane Dolores', liste as seguintes informações para todos os carros que ela vendeu: Num_chassi, Fabricante, Preco_venda

b. Liste o Num_chassi e Modelo dos carros que não possuem opcionais.

c. Considere a operação JUNÇÃO NATURAL entre VENDEDOR e VENDA. Qual é o significado de uma junção externa esquerda para essas tabelas (não mude a ordem das relações)? Explique com um exemplo.

d. Escreva uma consulta na álgebra relacional envolvendo seleção e uma operação de conjunto e diga, em palavras, o que a consulta faz.

6.24. Especifique as consultas a, b, c, e, f, i e j do Exercício 6.16 no cálculo relacional de tupla e de domínio.

6.25. Especifique as consultas a, b, c e d do Exercício 6.17 no cálculo relacional de tupla e de domínio.

6.26. Especifique as consultas c, d e f do Exercício 6.18 no cálculo relacional de tupla e de domínio.

6.27. Em uma consulta de cálculo relacional de tupla com n variáveis de tupla, qual seria o número mínimo típico de condições de junção? Por quê? Qual é o efeito de ter um número menor de condições de junção?

6.28. Reescreva as consultas do cálculo relacional de domínio que seguiram C0 na Seção 6.7 no estilo da notação abreviada de C0A, em que o objetivo é minimizar o número de variáveis de domínio escrevendo constantes no lugar de variáveis sempre que possível.

6.29. Considere esta consulta: recupere o Cpf dos funcionários que trabalham pelo menos nos projetos em que o funcionário com Cpf=12345678966 trabalha. Isso pode ser declarado como (**FORALL** p) (**IF** P **THEN** Q), onde

- p é uma variável de tupla que percorre a relação PROJETO.
- $P \equiv$ FUNCIONARIO com Cpf=12345678966 trabalha no PROJETO x.
- $Q \equiv$ FUNCIONARIO f trabalha no PROJETO p.

Expresse a consulta no cálculo relacional de tupla, usando as regras

- $(\forall p)(P(p)) \equiv$ **NOT**$(\exists p)$(**NOT**$(P(p)))$.
- (**IF** P **THEN** Q) $\equiv$ (**NOT**(P) **OR** Q).

6.30. Mostre como você pode especificar as seguintes operações da álgebra relacional no cálculo relacional de tupla e de domínio.

a. $\sigma_{A=C}(R(A, B, C))$

b. $\pi_{<A, B>}(R(A, B, C))$

c. $R(A, B, C) * S(C, D, E)$

d. $R(A, B, C) \cup S(A, B, C)$

e. $R(A, B, C) \cap S(A, B, C)$

f. $R(A, B, C) = S(A, B, C)$

g. $R(A, B, C) \times S(D, E, F)$

h. $R(A, B) \div S(A)$

6.31. Sugira extensões ao cálculo relacional, de modo que ele possa expressar os seguintes tipos de operações que foram discutidas na Seção 6.4: (a) funções de agregação e agrupamento; (b) operações JUNÇÃO EXTERNA; (c) consultas de fechamento recursivo.

6.32. Uma consulta aninhada é uma consulta dentro de outra. Mais especificamente, trata-se de uma consulta entre parênteses cujo resultado pode ser usado como um valor em diversos lugares, como no lugar de uma relação. Especifique as seguintes consultas sobre o banco de dados mostrado na Figura 3.5 usando o conceito de consultas aninhadas e os operadores relacionais discutidos neste capítulo. Mostre também o resultado de cada consulta se fosse aplicado ao estado do banco de dados na Figura 3.6.

a. Liste o nome de todos os funcionários que trabalham no departamento que tem o funcionário com o maior salário entre todos os funcionários.

b. Liste o nome de todos os funcionários cujo supervisor do supervisor tem como Cpf o numero '88866555576'.

c. Liste o nome dos funcionários que ganham pelo menos R$10.000 a mais do que o funcionário que menos recebe na empresa.

6.33. Indique se as seguintes conclusões são verdadeiras ou falsas.

a. **NOT** $(P(x)$ **OR** $Q(x)) \rightarrow$ (**NOT** $(P(x))$ **AND** (**NOT** $(Q(x)))$

b. **NOT** $(\exists x)\ (P(x)) \rightarrow \forall\ x$ (**NOT** $(P(x))$

c. $(\exists x)\ (P(x)) \rightarrow \forall\ x\ ((P(x))$

Exercícios de laboratório

6.34. Especifique e execute as seguintes consultas em álgebra relacional (RA — *relational algebra*) usando o interpretador RA sobre o esquema de banco de dados EMPRESA da Figura 3.5.

a. Liste o nome de todos os funcionários no departamento 5 que trabalham mais de 10 horas por semana no projeto ProdutoX.

b. Liste o nome de todos os funcionários que têm um dependente com o primeiro nome igual.

c. Liste o nome dos funcionários que são supervisionados diretamente por Fernando Wong.

d. Liste o nome dos funcionários que trabalham em cada projeto.

e. Liste o nome dos funcionários que não trabalham em projeto algum.

f. Liste o nome e endereço dos funcionários que trabalham em pelo menos um projeto localizado em São Paulo, mas cujo departamento não está localizado lá.

g. Liste o nome de gerentes de departamento que não têm dependentes.

6.35. Considere o seguinte esquema relacional PEDIDO_CORREIO descrevendo os dados para uma empresa de vendas por catálogo.

PECAS(Pnr, Pnome, Qtddisp, Preco, Pnivel)
CLIENTES(Cnr, Cnome, Rua, Cep, Telefone)
FUNCIONARIOS(Fnr, Fname, Cep, Datacontrat
CEPS(Cep, Cidade)
PEDIDOS(PEDnr, Cnr, Fnr, Recebimento, Envio)
DETALHES(PEDnr, Pnr, Qtd)

Qtddisp significa quantidade disponível: os outros nomes de atributo são relativamente autoexplicativos. Especifique e execute as seguintes consultas usando o interpretador RA sobre o esquema de banco de dados PEDIDO_CORREIO.

a. Recupere o nome das peças que custam menos de R$20,00.

b. Recupere o nome e cidade dos funcionários que receberam pedidos por peças custando mais de R$50,00.

c. Recupere os pares de valores de número de cliente dos que moram no mesmo CEP.

d. Recupere o nome dos clientes que pediram peças de funcionários que moram em Brasília.

e. Recupere o nome dos clientes que pediram peças que custam menos de R$20,00.

f. Recupere o nome de clientes que não fizeram um pedido.

g. Recupere o nome de clientes que fizeram exatamente dois pedidos.

6.36. Considere o seguinte esquema relacional DIARIONOTAS descrevendo os dados para um diário de um professor em particular. (*Nota:* Os atributos A, B, C e D de DISCIPLINAS armazenam os limites das notas.)

CATALOGO(Cnr, Ctitulo)
ALUNOS(Cod_aluno, Pnome, Unome, Minicial)
DISCIPLINAS(Sigla, Num_semestre, Cnr, A, B, C, D)
MATRICULA(Cod_aluno, Sigla, Num_semestre)

Especifique e execute as seguintes consultas usando o interpretador RA sobre o esquema de banco de dados DIARIONOTAS.

a. Recupere o nome dos alunos matriculados na turma de Autômato durante o primeiro semestre de 2009.

b. Recupere os valores de Cod_aluno daqueles que se matricularam em CCc226 e CCc227.

c. Recupere os valores de Cod_aluno daqueles que se matricularam em CCc226 ou CCc227.

d. Recupere o nome dos alunos que não se matricularam em nenhuma turma.

e. Recupere o nome dos alunos que se matricularam em todos os cursos da tabela CATALOGO.

6.37. Considere um banco de dados que consiste nas relações a seguir.

FORNECEDOR(Fnr, Fnome)

PECA(Pnr, Pnome)
PROJETO(PBnr, PRnome)
FORNECIMENTO(Fnr, Pnr, PBnr)

O banco de dados registra informações sobre fornecedores, peças e projetos, e inclui um relacionamento ternário entre fornecedores, peças e projetos. Este é um relacionamento muitos-para-muitos. Especifique e execute as seguintes consultas usando o interpretador RA.

a. Recupere os números de peça que são fornecidos para exatamente dois projetos.

b. Recupere o nome dos fornecedores que fornecem mais de duas peças ao projeto 'P1'.

c. Recupere os números de peça que são fornecidos por cada fornecedor.

d. Recupere os nomes de projeto que são fornecidos pelo fornecedor 'F1' apenas.

e. Recupere o nome dos fornecedores que fornecem pelo menos duas peças diferentes cada a pelo menos dois projetos diferentes.

6.38. Especifique e execute as seguintes consultas para o banco de dados do Exercício 3.16 usando o interpretador RA.

a. Recupere o nome dos alunos que se matricularam em uma disciplina que usa um livro-texto publicado pela Editora Pearson.

b. Recupere o nome das disciplinas em que o livro-texto foi mudado pelo menos uma vez.

c. Recupere o nome dos departamentos que adotam somente livros-texto publicados pela Editora Pearson.

d. Recupere o nome dos departamentos que adotam livros-texto escritos por Navathe e publicados pela Editora Pearson.

e. Recupere o nome dos alunos que nunca usaram um livro (em um curso) escrito por Navathe e publicado pela Editora Pearson.

6.39. Repita os Exercícios de Laboratório 6.34a 6.38 no cálculo relacional de domínio (DRC—*Domain Relational Calculus*) usando o interpretador DRC.

Bibliografia selecionada

Codd (1970) definiu a álgebra relacional básica. Date (1983a) discute as junções externas. O trabalho sobre a extensão das operações relacionais é discutido por Carlis (1986) e Ozsoyoglu et al. (1985). Cammarata et al. (1989) estendem as restrições de integridade e junções do modelo relacional.

Codd (1971) introduziu a linguagem Alpha, que é baseada nos conceitos do cálculo relacional de tupla. Alpha também inclui a noção de funções de agregação, que vai além do cálculo relacional. A definição formal srcinal do cálculo relacional foi dada por Codd (1972), que também forneceu um algoritmo que transforma qualquer expressão do cálculo relacional de tupla em álgebra relacional. A QUEL (Stonebraker et al., 1976) é baseada no cálculo relacional de tupla, com quantificadores existenciais implícitos, mas sem quantificadores universais, e foi implementada no sistema INGRES como uma linguagem disponível comercialmente. Codd definiu a totalidade relacional de uma linguagem de consulta como sendo pelo menos tão poderosa quanto o cálculo relacional. Ullman (1988) descreve uma prova formal da equivalência da álgebra relacional com expressões seguras de cálculo relacional de tupla e domínio. Abiteboul et al. (1995) e Atzeni e deAntonellis (1993) fazem um tratamento detalhado das linguagens relacionais formais.

Embora as ideias do cálculo relacional de domínio tenham sido propostas inicialmente na linguagem QBE (Zloof, 1975), o conceito foi definido de maneira formal por Lacroix e Pirotte (1977a). A versão experimental do sistema Query-By-Example é descrita em Zloof (1975). A ILL (Lacroix e Pirotte, 1977b) é baseada no cálculo relacional de domínio. Whang et al. (1990) estendem a QBE com quantificadores universais. As linguagens de consulta visuais, dentre as quais a QBE é um exemplo, estão sendo propostas como um meio de consultar bancos de dados. Conferências como a Visual Database Systems Working Conference [por exemplo, Arisawa e Catarci (2000) ou Zhou e Pu (2002)] apresentam uma série de propostas para tais linguagens.

parte 3

Modelagem conceitual e projeto de banco de dados

capítulo 7

Modelagem de dados usando o modelo Entidade-Relacionamento (ER)

A modelagem conceitual é uma fase muito importante no projeto de uma aplicação de banco de dados bem-sucedida. Geralmente, o termo **aplicação de banco de dados** refere-se a um banco de dados em particular e aos programas associados que implementam as consultas e atualizações dele. Por exemplo, uma aplicação de banco de dados para um BANCO que controla contas de clientes incluiria programas que implementam atualizações ao banco de dados correspondentes a depósitos e saques de clientes. Esses programas oferecem interfaces gráficas com o usuário (GUIs) de fácil utilização, com formulários e menus para os usuários finais da aplicação — os caixas de banco, neste exemplo. Logo, uma parte importante da aplicação de banco de dados exigirá o projeto, a implementação e o teste desses programas de aplicação. Tradicionalmente, o projeto e o teste dos **programas de aplicação** têm sido considerados parte da *engenharia de software*, em vez do *projeto de banco de dados*. Em muitas ferramentas de projeto de software, as metodologias de projeto de banco de dados e de engenharia de software são interligadas, pois essas atividades estão fortemente relacionadas.

Neste capítulo, abordamos a técnica tradicional de concentrar nas estruturas e restrições de banco de dados durante seu projeto conceitual. O projeto de programas de aplicação normalmente é abordado em cursos de engenharia de software. Apresentamos os conceitos de modelagem do **modelo Entidade-Relacionamento (ER)** , que é um modelo de dados conceitual popular de alto nível. Esse modelo e suas variações costumam ser utilizados para o projeto conceitual de aplicações de banco de dados, e muitas ferramentas de projeto de banco de dados empregam seus conceitos. Descrevemos os conceitos e restrições básicas de estruturação de dados do modelo ER e discutimos seu uso no projeto de esquemas conceituais para aplicações de banco de dados. Também apresentamos a notação diagramática associada ao modelo ER, conhecido como **diagramas ER**

As metodologias de modelagem de objeto, como a **Unified Modeling Language (UML)**, estão se tornando cada vez mais populares no projeto de software de banco de dados. Essas metodologias vão além do projeto de banco de dados para especificar o projeto detalhado dos módulos de software e suas interações usando vários tipos de diagramas. Uma parte importante dessas metodologias — a saber, os *diagramas de classe*[1] — é semelhante, de muitas maneiras, aos diagramas ER. Nos diagramas de classe são especificadas *operações* sobre objetos, além da especificação da estrutura do esquema do banco de dados. As operações podem ser usadas para especificar os *requisitos funcionais* durante o projeto de banco de dados, conforme discutiremos na Seção 7.1. Apresentamos parte da notação e conceitos em UML para diagramas de classe que são particularmente relevantes ao projeto de banco de dados da Seção 7.8, e comparamos rapidamente estes com a notação e conceitos de ER. Notação e conceitos de UML adicionais serão apresentados na Seção 8.6 e no Capítulo 10.

Este capítulo está organizado da seguinte forma: a Seção 7.1 discute o papel dos modelos de dados conceituais de alto nível no projeto de banco de dados. Apresentamos os requisitos para uma aplicação de banco de dados de exemplo na Seção 7.2, para

[1] Uma **classe** é semelhante a um *tipo de entidade* de várias maneiras.

ilustrar o uso dos conceitos do modelo ER. Esse banco de dados de exemplo também é usado no decorrer do livro. Na Seção 7.3, apresentamos os conceitos de entidades e atributos, e gradualmente introduzimos a técnica diagramática para exibir um esquema ER. Na Seção 7.4, apresentamos os conceitos de relacionamentos binários e suas funções e restrições estruturais. A Seção 7.5 apresenta os tipos de entidade fraca. A Seção 7.6 mostra como um projeto de esquema é refinado para incluir relacionamentos. A Seção 7.7 analisa a notação para diagramas ER, resume os problemas e armadilhas comuns que ocorrem no projeto de esquema e discute como escolher os nomes para construções de esquema de banco de dados. A Seção 7.8 apresenta alguns conceitos de diagrama de classe UML, compara-os com os conceitos do modelo ER e os aplica ao mesmo banco de dados de exemplo. A Seção 7.9 discute tipos de relacionamentos mais complexos. No final há um resumo do capítulo.

O material nas seções 7.8 e 7.9 pode ser excluído de um curso introdutório. Se uma cobertura mais completa dos conceitos de modelagem de dados e projeto de banco de dados conceitual for desejada, deitor deverá continuar até o Capítulo 8, onde descrevemos as extensões ao modelo ER que levamao modelo EER (ER Estendido), o qual inclui conceitos como especialização, generalização, herança e tipos (categorias) de união. Também apresentaremos alguns conceitos adicionais e a notação de UML no Capítulo 8.

7.1 Usando modelos de dados conceituais de alto nível para o projeto do banco de dados

A Figura 7.1 mostra uma visão geral simplificada do processo de projeto de banco de dados. A primeira etapa mostrada é o **levantamento e análise de requisitos**. Durante essa etapa, os projetistas de banco de dados entrevistam os usuários esperados para entenderem e documentarem seus **requisitos de dados**. O resultado dessa etapa é um conjunto de requisitos dos usuários escrito de forma concisa. Esses requisitos devem ser especificados da forma mais detalhada e completa possível. Em paralelo com a especificação dos requisitos de dados, é útil determinar os conhecidos **requisitos funcionais** da aplicação. Estes consistem em **operações** (ou **transações**) definidas pelo usuário, que serão aplicadas ao banco de dados, incluindo recuperações e atualizações. No projeto de software, é comum usar *diagramas de fluxo de dados*, *diagramas de sequência*, *cenários* e outras técnicas para especificar requisitos funcionais. Não discutiremos essas técnicas aqui; elas normalmente são descritas em detalhes nos textos de engenharia de software. Daremos uma visão geral dessas técnicas no Capítulo 10.

Assim que os requisitos tiverem sido levantados e analisados, a próxima etapa é criar um **esquema conceitual** para o banco de dados, usando um modelo de dados conceitual de alto nível. Essa etapa é chamada de **projeto conceitual**. O esquema conceitual é uma descrição concisa dos requisitos de dados dos usuários e inclui detalhes dos tipos de entidade, relacionamentos e restrições; estes são expressos usando os conceitos fornecidos pelo modelo de dados de alto nível. Como não incluem descrições detalhadas de implementação, esses conceitos normalmente são mais fáceis de entender e podem ser usados para a comunicação com usuários não técnicos. O esquema conceitual de alto nível também pode ser utilizado como uma referência para garantir que todos os requisitos de dados dos usuários sejam atendidos e que não estejam em conflito. Essa técnica permite que os projetistas de banco de dados se concentrem em especificar as propriedades dos dados, sem se preocuparem com detalhes de armazenamento e implementação. Isso torna mais fácil criar um bom projeto de banco de dados conceitual.

Durante ou após o projeto do esquema conceitual, as operações básicas do modelo de dados podem ser usadas para especificar as consultas e operações do usuário de alto nível, identificadas durante a análise funcional. Isso também serve para confirmar se o esquema conceitual atende a todos os requisitos funcionais identificados. Modificações no esquema conceitual podem ser introduzidas se alguns requisitos funcionais não puderem ser especificados usando o esquema inicial.

A próxima etapa no projeto de banco de dados é a implementação real do próprio banco de dados, usando um SGBD comercial. A maioria dos SGBDs comerciais utiliza um modelo de dados de implementação — como o modelo de banco de dados relacional ou objeto-relacional —, de modo que o esquema conceitual é transformado do modelo de dados de alto nível para o modelo de dados da implementação. Essa etapa é chamada de **projeto lógico** ou **mapeamento do modelo de dados** Seu resultado é um esquema de banco de dados no modelo de dados da implementação do SGBD. O mapeamento do modelo de dados normalmente é automatizado ou semiautomatizado nas ferramentas de projeto do banco de dados.

A última etapa é a fase do **projeto físico**, durante a qual as estruturas de armazenamento internas, organizações de arquivo, índices, caminhos de acesso e parâmetros físicos do projeto para os arquivos do banco de dados são especificados. Em paralelo com essas atividades, os programas de aplicação são projetados e implementados como transações de banco de dados correspondentes às especificações da transação de alto nível. Discutiremos o processo de projeto do banco de dados com mais detalhes no Capítulo 10.

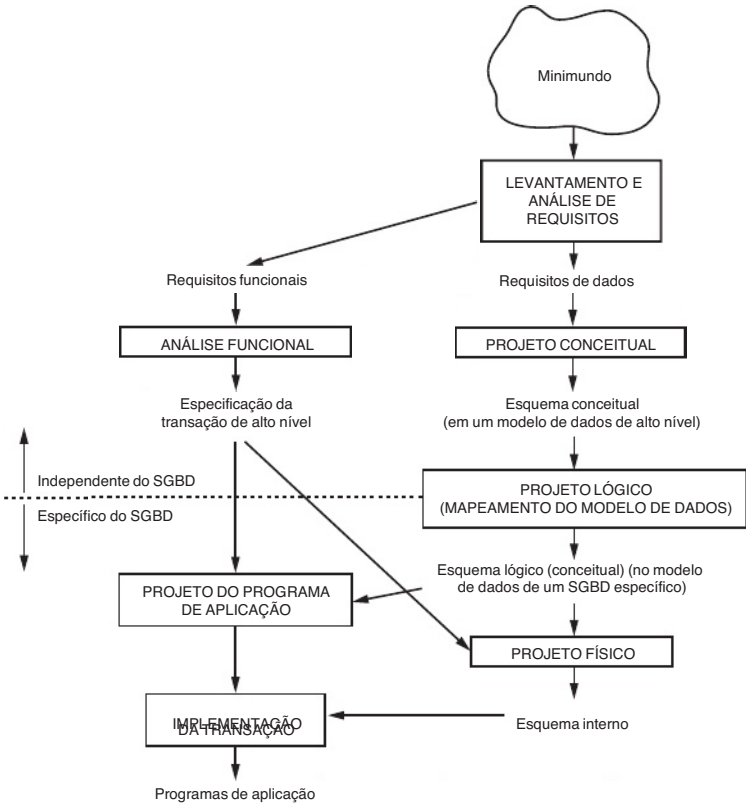


Figura 7.1
Um diagrama simplificado para ilustrar as principais fases do projeto de banco de dados.

Neste capítulo, apresentamos apenas os conceitos básicos do modelo ER para o projeto do esquema conceitual. Outros conceitos de modelagem serão discutidos no Capítulo 8, quando apresentaremos o modelo EER.

7.2 Exemplo de aplicação de banco de dados

Nesta seção, descrevemos um exemplo de aplicação de banco de dados, chamado EMPRESA, que serve para ilustrar os conceitos básicos do modelo ER e seu uso no projeto do esquema. Listamos os requisitos de dados para o banco de dados aqui, e depois criaremos seu esquema conceitual passo a passo, quando introduzirmos os conceitos de modelagem do modelo ER. O banco de dados EMPRESA registra os funcionários, departamentos e projetos de uma empresa. Suponha que, depois da fase de levantamento e análise de requisitos, os projetistas de banco de dados ofereçam a seguinte descrição do *minimundo* — a parte da empresa que será representada no banco de dados:

- A empresa é organizada em departamentos. Cada departamento tem um nome exclusivo, um número exclusivo e um funcionário em particular que o gerencia. Registramos a data inicial em que esse funcionário começou a gerenciar o departamento. Um departamento pode ter vários locais.
- Um departamento controla uma série de projetos, cada um deles com um nome exclusivo, um número exclusivo e um local exclusivo.

- Armazenamos o nome, número do Cadastro de Pessoa Física, [2] endereço, salário, sexo (gênero) e data de nascimento de cada funcionário. Um funcionário é designado para um departamento, mas pode trabalhar em vários projetos, que não necessariamente são controlados pelo mesmo departamento. Registramos o número atual de horas por semana que um funcionário trabalha em cada projeto. Também registramos o supervisor direto de cada funcionário (que é outro funcionário).
- Queremos registrar os dependentes de cada funcionário para fins de seguro. Para cada dependente, mantemos o nome, sexo, data de nascimento e parentesco com o funcionário.

A Figura 7.2 mostra como o esquema para essa aplicação de banco de dados pode ser exibido por meio da notação gráfica conhecida como **diagramas ER**. Essa figura será explicada gradualmente à medida que os conceitos do modelo ER forem apresentados. Descrevemos o processo passo a passo da derivação desse esquema com base nos requisitos declarados — e explicamos a notação diagramática ER — à medida que introduzirmos os conceitos do modelo ER.

7.3 Tipos de entidade, conjuntos de entidades, atributos e chaves

O modelo ER descreve os dados como *entidades*, *relacionamentos* e *atributos*. Na Seção 7.3.1, apresentamos os conceitos de entidades e seus atributos. Discutimos os tipos de entidade e os principais atributos na Seção 7.3.2. Depois, na Seção 7.3.3, especificamos o projeto conceitual inicial dos tipos de entidade para o banco de dados EMPRESA. Os relacionamentos são descritos na Seção 7.4.

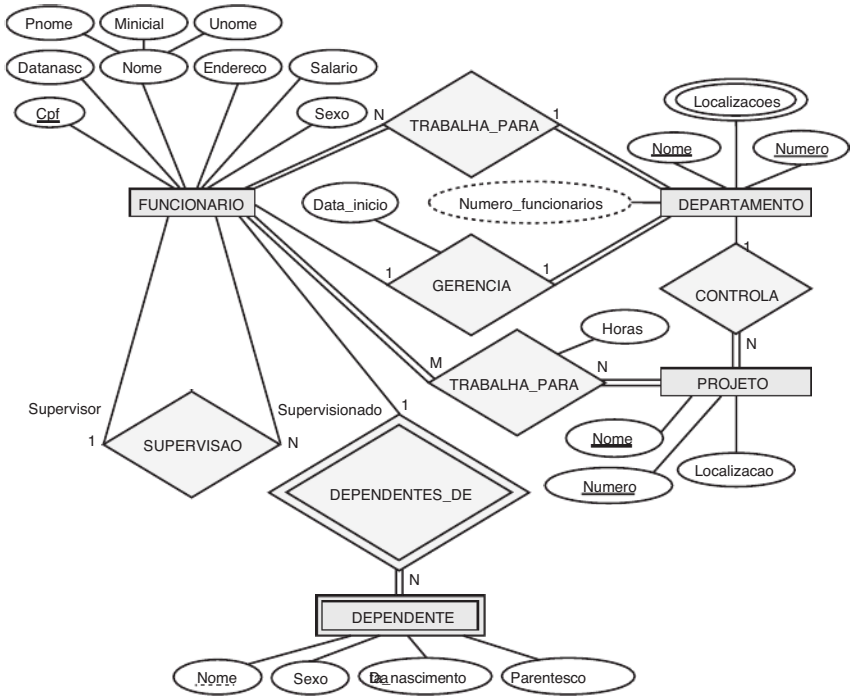


Figura 7.2
Um diagrama de esquema ER para o banco de dados EMPRESA. A notação diagramática é apresentada gradualmente no decorrer do capítulo e resumida na Figura 7.14.

[2] O número do Cadastro de Pessoa Física (CPF) é um identificador exclusivo de onze dígitos, atribuído a cada indivíduo no Brasil, para registrar seu emprego, benefícios e impostos. Outros países possuem esquemas de identificação semelhantes, como números do Seguro Social (SSN) e identificação civil.

7.3.1 Entidades e atributos

Entidades e seus atributos. O objeto básico que o modelo ER representa é uma **entidade**, que é algo no mundo real com uma existência independente. Uma entidade pode ser um objeto com uma existência física (por exemplo, uma pessoa em particular, um carro, uma casa ou um funcionário), ou pode ser um objeto com uma existência conceitual (por exemplo, uma empresa, um cargo ou um curso universitário). Cada entidade possui **atributos** — as propriedades específicas que a descrevem. Por exemplo, uma entidade FUNCIONARIO pode ser descrita pelo nome, idade, endereço, salário e cargo do funcionário. Uma entidade em particular terá um valor para cada um de seus atributos. Os valores de atributo que descrevem cada entidade tornam-se uma parte importante dos dados armazenados no banco de dados.

A Figura 7.3 mostra duas entidades e os valores de seus atributos. A entidade FUNCIONARIO f_1 tem quatro atributos: Nome, Endereco, Idade e Telefone_residencial; seus valores são 'João Silva,' 'Rua das Flores, 751, São Paulo, SP, 07700110', '55' e '13-4749-2630', respectivamente. A entidade EMPRESA e_1 tem três atributos: Nome, Matriz e Presidente; seus valores são 'Companhia Modelo', 'São Paulo' e 'João Silva', respectivamente.

Vários tipos de atributos ocorrem no modelo ER: *simples* versus *composto*, *valor único* versus *multivalorado*, e *armazenado* versus *derivado*. Primeiro, vamos definir esses tipos de atributo e ilustrar seu uso por meio de exemplos. Depois, discutiremos o conceito de um *valor* NULL para um atributo.

Atributos compostos *versus* simples (atômicos). **Atributos compostos** podem ser divididos em subpartes menores, que representam atributos mais básicos, com significados independentes. Por exemplo, o atributo Logradouro da entidade FUNCIONARIO mostrada na Figura 7.3 pode ser subdividido em Logradouro, Cidade, Estado e Cep,[3] com os valores 'Rua das Flores, 751', 'São Paulo', 'SP' e '07700110.' Os atributos não divisíveis são chamados **atributos simples** ou **atômicos**. Os atributos compostos podem formar uma hierarquia; por exemplo, Logradouro pode ser subdividido em três atributos simples: Numero, Rua e Numero_apartamento, como mostra a Figura 7.4. O valor de um atributo composto é a concatenação dos valores de seus componentes atributos simples.

Atributos compostos são úteis para modelar situações em que um usuário às vezes se refere ao atributo composto como uma unidade, mas outras vezes se refere especificamente a seus componentes. Se o atributo composto for referenciado apenas como um todo, não é necessário subdividi-lo em atributos componentes. Por exemplo, se não for preciso referenciar os componentes individuais de um endereço (CEP, rua etc.), então o endereço inteiro pode ser designado como um atributo simples.

Atributos de valor único *versus* multivalorados. A maioria dos atributos possui um valor único para uma entidade em particular; tais atributos são chamados de **valor único**. Por exemplo, Idade é um atributo de valor único de uma pessoa. Em alguns casos, um atributo pode ter um conjunto de valores para a mesma entidade — por exemplo, um atributo Cores para um carro, ou um atributo Formacao_academica para uma pessoa. Os carros com uma cor têm um único valor, enquanto os carros com duas cores possuem dois valores. De modo semelhante, uma pessoa pode não ter formação acadêmica, outra pessoa pode ter, e uma terceira pode ter duas ou mais formações; portanto, diferentes pessoas podem ter distintos *números de valores* para o atributo Formacao_academica. Esses atributos são chamados de **multivalorados**. Um atributo multivalorado pode ter um limite mínimo e um máximo para restringir o *número de valores* permitidos para cada entidade individual. Por exemplo, o atributo Cores de

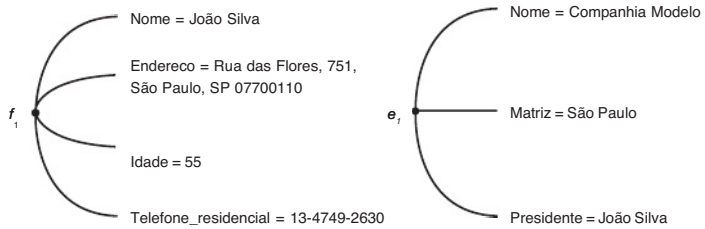


Figura 7.3
Duas entidades, FUNCIONARIO f_1 e EMPRESA e_1, e seus atributos.

[3] CEP (Código de Endereçamento Postal) é o nome usado no Brasil para um código postal com oito dígitos, como 07601-090.

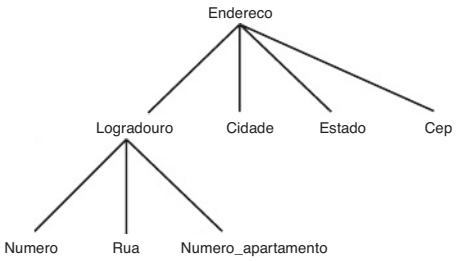


Figura 7.4
Uma hierarquia de atributos compostos.

um carro pode ser restrito a ter entre um e três valores, se considerarmos que um carro pode ter no máximo três cores.

Atributos armazenados *versus* derivados. Em alguns casos, dois (ou mais) valores de atributo estão relacionados — por exemplo, os atributos Idade e Data_nascimento de uma pessoa. Para uma entidade de pessoa em particular, o valor de Idade pode ser determinado pela data atual (hoje) e o valor da Data_nascimento dessa pessoa. O atributo Idade, portanto, é chamado de **atributo derivado** e considerado **derivável** do atributo Datanasc, que é chamado, por sua vez, de **atributo armazenado**. Alguns valores de atributo podem ser derivados de *entidades relacionadas*; por exemplo, um atributo Numero_funcionarios de uma entidade DEPARTAMENTO pode ser derivado contando-se o número de funcionários relacionados a (trabalhando para) esse departamento

Valores NULL. Em alguns casos, uma entidade em particular pode não ter um valor aplicável para um atributo. Por exemplo, o atributo Numero_apartamento de um endereço só se aplica a endereços que estão em prédios de apartamento, e não a outros tipos de residências, como casas. De modo semelhante, um atributo Formacao_academica só se aplica a pessoas com esse tipo de formação. Para tais situações, foi criado um valor especial, chamado NULL. Um endereço de uma casa teria NULL para seu atributo Numero_apartamento, e uma pessoa sem formação acadêmica teria NULL para Formacao_academica. NULL também pode ser usado quando não conhecemos o valor de um atributo para determinada entidade — por exemplo, se não soubermos o número do telefone residencial de 'João Silva' na Figura 7.3. O significado do primeiro tipo de NULL é *não aplicável*, enquanto o significado do segundo é *desconhecido*. A categoria *desconhecido* de NULL pode ser classificada

{Endereço_telefone({Telefone(Codigo_area, Numero_telefone)},Endereco(Logradouro (Numero,Rua,Numero_apartamento),Cidade,Estado,Cep))}

Figura 7.5
Um atributo complexo: Endereço_telefone.

ainda em mais dois casos. O primeiro caso acontece quando se sabe que o valor do atributo existe, mas está *faltando* — por exemplo, se o atributo Altura de uma pessoa for listado como NULL. O segundo caso surge quando *não se sabe* se o valor do atributo existe — por exemplo, se o atributo Telefone_residencial de uma pessoa for NULL.

Atributos complexos. Observe que, em geral, os atributos compostos e multivalorados podem ser aninhados arbitrariamente. Podemos representar o aninhamento arbitrário ao agrupar componentes de um atributo composto entre parênteses ( ) e separá-los com vírgulas, e ao exibir os atributos multivalorados entre chaves { }. Esses atributos são chamados de **atributos complexos**. Por exemplo, se uma pessoa pode ter mais de uma residência e cada residência pode ter um único endereço e vários telefones, um atributo Endereço_telefone para uma pessoa pode ser especificado como na Figura 7.5.[4] Tanto Telefone quanto Endereco são atributos compostos.

7.3.2 Tipos de entidade, conjuntos de entidade, chaves e conjuntos de valores

Tipos de entidade e conjuntos de entidade Um banco de dados em geral contém grupos de entidades que são semelhantes. Por exemplo, uma empresa que emprega centenas de funcionários pode querer armazenar informações semelhantes com relação a cada um dos funcionários. Essas entidades de funcionário compartilham os mesmos atributos, mas cada uma tem *o(s) próprio(s) valor(es)* para cada atributo. Um **tipo de entidade** define uma *coleção* (ou *conjunto*) de entidades que têm os mesmos atributos. Cada tipo de entidade no banco de dados é descrito por seu nome e atributos. A Figura 7.6 mostra dois tipos de entidade: FUNCIONARIO e EMPRESA, e uma lista de alguns dos atributos para cada um. Algumas entidades individuais de cada tipo também são ilustradas, junto com os valores de seus atributos. A coleção de todas as entidades de determinado tipo de entidade no banco de dados, em qualquer ponto no tempo, é chamada de **conjunto de entidades** Normalmente, refere-se ao conjunto de entidades para usar o mesmo nome do

[4] Para aqueles acostumados com XML, devemos observar que os atributos complexos são semelhantes aos elementos complexos em XML (ver Capítulo 12).

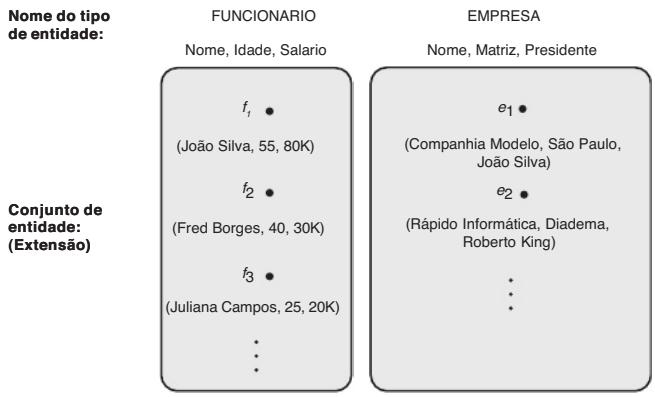


Figura 7.6
Dois tipos de entidade, FUNCIONARIO e EMPRESA, e algumas entidades membro de cada uma.

tipo de entidade. Por exemplo, FUNCIONARIO refere-se ao *tipo de entidade* e também ao conjunto atual *de todas as entidades de funcionário* no banco de dados.

Um tipo de entidade é representado nos diagramas ER[5] (ver Figura 7.2) como uma caixa retangular delimitando seu nome. Os nomes de atributo são delimitados em ovais, sendo ligados a seu tipo de entidade por linhas retas. Os atributos compostos são ligados aos seus atributos componentes por linhas retas. Os atributos multivalorados aparecem em ovais duplas. A Figura 7.7(a) mostra um tipo de entidade CARRO nessa notação.

Um tipo de entidade descreve o **esquema** ou **conotação** para um *conjunto de entidades* que compartilham a mesma estrutura. A coleção de entidades de determinado tipo é agrupada em um conjunto de entidades, que também é chamado de **extensão** do tipo de entidade.

Atributos-chave de um tipo de entidade. Uma restrição importante das entidades de um tipo de entidade é a **chave** ou **restrição de exclusividade** sobre os atributos. Um tipo de entidade normalmente tem um ou mais atributos cujos valores são distintos para cada entidade individual no conjunto de entidades. Esse atributo é denominado **atributo-chave**, e seus valores podem ser usados para identificar cada entidade de maneira exclusiva. Por exemplo, o atributo Nome é uma chave do tipo de entidade EMPRESA na Figura 7.6, pois duas empresas não podem ter o mesmo nome. Para o tipo de entidade PESSOA, um atributo-chave típico é o Cpf (Cadastro de Pessoa Física). Às vezes, vários atributos juntos formam uma chave, significando que a *combinação* dos valores de atributo deve ser distinta para cada entidade. Se um conjunto de atributos possui essa propriedade, o modo correto de representar isso no modelo ER que descrevemos aqui é definir um *atributo composto* e designá-lo como um atributo-chave do tipo de entidade. Observe que essa chave composta precisa ser *mínima*, ou seja, todos os atributos componentes precisam estar incluídos no atributo composto para ter a propriedade de exclusividade. Atributos supérfluos não devem ser incluídos em uma chave. Na notação diagramática ER, cada atributo-chave tem seu nome **sublinhado** dentro da oval, conforme ilustrado na Figura 7.7(a).

Especificar que um atributo é uma chave de um tipo de entidade significa que a propriedade anterior da exclusividade precisa ser mantida para *cada conjunto de entidades* do tipo de entidade. Logo, essa é uma restrição que proíbe que duas entidades tenham o mesmo valor para o atributo-chave ao mesmo tempo. Essa não é a propriedade de um conjunto de entidades em particular; em vez disso, é uma restrição sobre *qualquer conjunto de entidades* do tipo de entidade em qualquer ponto no tempo. Essa restrição-chave (e outras restrições que discutiremos mais adiante) é derivada das restrições do mundo real que o banco de dados representa.

Alguns tipos de entidade possuem *mais de um* atributo-chave. Por exemplo, cada um dos atributos Cod_veiculo e Registro do tipo de entidade CAR-

[5] Usamos a notação para diagramas ER, a qual é próxima da notação da proposta srcinal (Chen, 1976). Muitas outras notações estão em uso; ilustraremos algumas delas mais adiante neste capítulo, quando apresentarmos os diagramas de classe UML, e no Apêndice A.

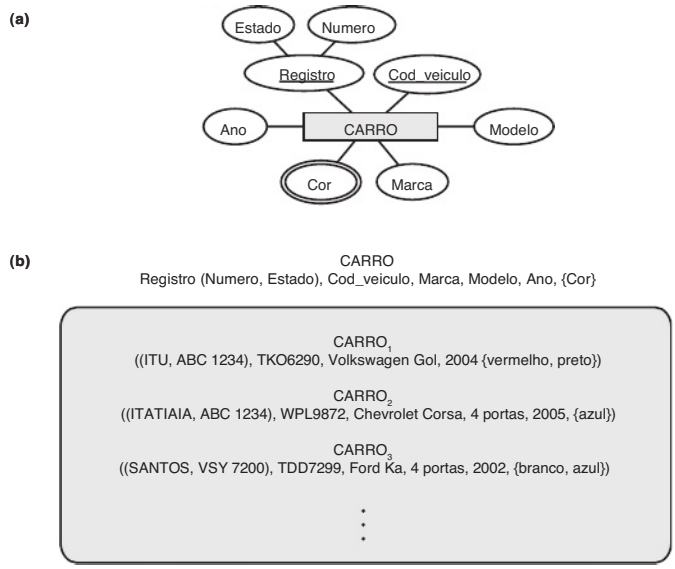


Figura 7.7
O tipo de entidade CARRO com dois atributos-chave, Registro e Cod_veiculo. (a) Notação do diagrama ER. (b) Conjunto de entidade com três entidades.

RO (Figura 7.7) é uma chave por si só. O atributo Registro é um exemplo de uma chave composta formada por dois atributos componentes simples, Estado e Numero, nenhum deles sendo uma chave por si só. Um tipo de entidade também *pode não ter chave*; nesse caso, ele é chamado de *tipo de entidade fraca* (ver Seção 7.5).

Em nossa notação diagramática, se dois atributos forem sublinhados separadamente, então *cada um é uma chave por si só*. Diferentemente do modelo relacional (ver Seção 3.2.2), não existe o conceito de chave primária no modelo ER que apresentamos aqui; a chave primária será escolhida durante o mapeamento para um esquema relacional (ver Capítulo 9).

Conjuntos (domínios) de valores dos atributos Cada atributo simples de um tipo de entidade é associado a um **conjunto de valores** (ou **domínio** de valores), o qual especifica o conjunto de valores que podem ser designados a esse atributo para cada entidade individual. Na Figura 7.6, se o intervalo de idades permitidas para os funcionários estiver entre 16 e 70, podemos especificar o conjunto de valores do atributo Idade de FUNCIONARIO como sendo o conjunto de números inteiros entre 16 e 70. De modo semelhante, podemos especificar o conjunto de valores para o atributo Nome como sendo o conjunto de cadeias de caracteres alfabéticos separados por caracteres de espaço, e assim por diante. Os conjuntos de valores não são exibidos nos diagramas ER, e são com frequência especificados usando os **tipos de dados** básicos disponíveis na maioria das linguagens de programação, como inteiro, cadeia de caracteres, booleano, real, tipo enumerado, intervalo de valores, e assim por diante. Outros tipos de dados que representam tipos comuns no banco de dados, como data, hora e outros conceitos, também são empregados. Matematicamente, um atributo A do conjunto de entidades E, cujo conjunto de valores é V, pode ser definido como uma **função** de E ao conjunto de potência[6] $P(V)$ de V:

$$A : E \rightarrow P(V)$$

Referimo-nos ao valor do atributo A para a entidade e como $A(e)$. A definição anterior cobre tanto atributos de único valor quanto atributos multivalorados, bem como NULLs. Um valor NULL é representado pelo *conjunto vazio*. Para atributos de único valor, $A(e)$ é restrito a ser um *conjunto*

[6] O **conjunto de potência** $P(V)$ de um conjunto V é o conjunto de todos os subconjuntos de V.

singular[7] para cada entidade *e* em *E*, ao passo que não existe restrição sobre atributos multivalorados. Para um atributo composto *A*, o conjunto de valores *V* é o conjunto de potência do produto Cartesiano de $P(V_1)$, $P(V_2)$, ..., $P(V_n)$, onde V_1, V_2, ..., V_n são os conjuntos de valores dos atributos componentes simples que formam *A*:

$$V = P(P(V_1) \times P(V_2) \times ... \times P(V_n))$$

O conjunto de valores oferece todos os valores possíveis. Em geral, apenas um pequeno número desses valores existe no banco de dados em determinado momento. Esses valores representam os dados do estado atual do minimundo. Eles correspondem aos dados conforme realmente existem no minimundo.

7.3.3 Projeto conceitual inicial do banco de dados EMPRESA

Agora, podemos definir os tipos de entidade para o banco de dados EMPRESA, com base nos requisitos descritos na Seção 7.2. Após definir aqui vários tipos de entidade e seus atributos, refinamos nosso projeto na Seção 7.4, depois de introduzir o conceito de um relacionamento. De acordo com os requisitos listados na Seção 7.2, podemos identificar quatro tipos de entidade — uma correspondente a cada um dos quatro itens na especificação (ver Figura 7.8):

1. Um tipo de entidade DEPARTAMENTO com atributos Nome, Numero, Localizacoes, Gerente e Data_inicio_gerente. Localizacoes é o único atributo multivalorado. Podemos especificar que tanto Nome quanto Numero são atributos-chave (separados), pois cada um foi especificado como sendo exclusivo.
2. Um tipo de entidade PROJETO com atributos Nome, Numero, Localizacao e Departamento_gerenciador. Tanto Nome quanto Numero são atributos-chave (separados).
3. Um tipo de entidade FUNCIONARIO com atributos Nome, Cpf, Sexo, Endereco, Salario, Data_nascimento, Departamento e Supervisor. Tanto Nome quanto Endereco podem ser atributos compostos; no entanto, isso não foi especificado nos requisitos. Temos de voltar aos usuários para ver se algum deles irá se referir aos componentes individuais de Nome — Primeiro_nome, Inicial_meio, Ultimo_nome — ou de Endereco.
4. Um tipo de entidade DEPENDENTE com atributos Funcionario, Nome_dependente, Sexo, Data_nascimento e Parentesco (para o funcionário).

Até aqui, não representamos o fato de que um funcionário pode trabalhar em vários projetos nem o número de horas por semana que um funcionário trabalha em cada projeto. Essa característica é listada como parte do terceiro requisito na Seção 7.2, e pode ser representada por um atributo composto multivalorado de FUNCIONARIO, chamado Trabalha_em, com os componentes simples (Projeto, Horas). Como alternativa, ela pode ser representada como um atributo composto multivalorado de PROJETO, chamado Trabalhadores, com os componentes simples (Funcionario, Horas). Escolhemos a primeira alternativa na Figura 7.8, que mostra cada um dos tipos de entidade descritos. O atributo Nome de FUNCIONARIO aparece como um atributo composto, aparentemente após a consulta com os usuários.

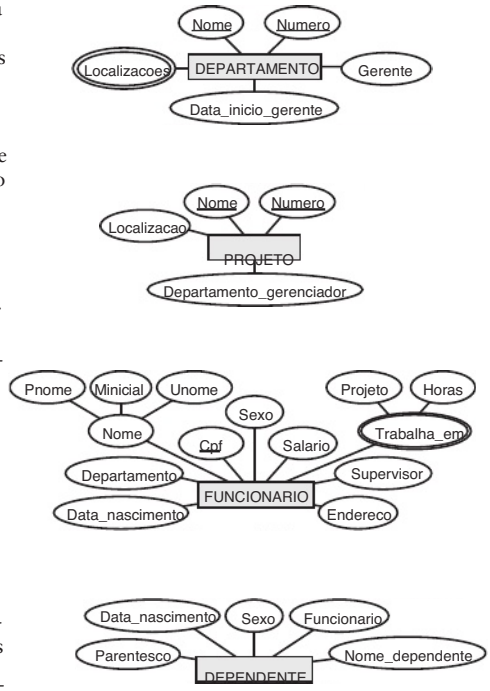


Figura 7.8
Projeto preliminar de tipos de entidade para o banco de dados EMPRESA. Alguns dos atributos mostrados serão refinados nos relacionamentos.

[7] Um conjunto **singular** (ou *singleton*) é um conjunto com apenas um elemento (valor).

7.4 Tipos e conjuntos de relacionamentos, papéis e restrições estruturais

Na Figura 7.8 existem vários *relacionamentos implícitos* entre os diversos tipos de entidade. De fato, sempre que um atributo de um tipo de entidade se refere a outro tipo de entidade, existe algum relacionamento. Por exemplo, o atributo Gerente de DEPARTAMENTO refere-se a um funcionário que gerencia o departamento; o atributo Departamento_gerenciador de PROJETO refere-se ao departamento que controla o projeto; o atributo Supervisor de FUNCIONARIO refere-se a outro funcionário (aquele que supervisiona esse funcionário); o atributo Departamento de FUNCIONARIO refere-se ao departamento para o qual o funcionário trabalha, e assim por diante. No modelo ER, essas referências não devem ser representadas como atributos, mas como **relacionamentos**, que são discutidos nesta seção. O esquema do banco de dados EMPRESA será refinado na Seção 7.6 para representar relacionamentos de maneira explícita. No projeto inicial dos tipos de entidade, os relacionamentos normalmente são capturados na forma de atributos. À medida que o projeto é refinado, esses atributos são convertidos em relacionamentos entre os tipos de entidade.

Este tópico é organizado da seguinte forma: a Seção 7.4.1 apresenta os conceitos dos tipos, conjuntos e instâncias de relacionamento. Definimos os conceitos de grau de relacionamento, nomes de função e relacionamentos recursivos na Seção 7.4.2, e depois discutimos as restrições estruturais sobre os relacionamentos — como as razões de cardinalidade e dependências de existência — na Seção 7.4.3. A Seção 7.4.4 mostra como os tipos de relacionamento também podem ter atributos.

7.4.1 Tipos, conjuntos e instâncias de relacionamento

Um **tipo de relacionamento** R entre n tipos de entidade $E_1, E_2, ..., E_n$ define um conjunto de associações — ou um **conjunto de relacionamento** — entre as entidades desses tipos de entidade. Assim como no caso dos tipos de entidade e conjuntos de entidade, um tipo de relacionamento e seu conjunto de relacionamento correspondente em geral são referenciados pelo *mesmo nome*, R. Matematicamente, o conjunto de relacionamento R é um conjunto de **instâncias de relacionamento** r_i, onde cada r_i associa-se a n entidades individuais $(e_1, e_2, ..., e_n)$, e cada entidade e_j em r_i é um membro do conjunto de entidades E_j, $1 \leq j \leq n$. Logo, um conjunto de relacionamento é uma relação matemática sobre $E_1, E_2, ..., E_n$. Como alternativa, ele pode ser definido como um subconjunto do produto cartesiano dos conjuntos de entidades $E_1 \times E_2 \times ... \times E_n$. Cada um dos tipos de entidade $E_1, E_2, ..., E_n$ é dito participar no tipo de relacionamento R; de modo semelhante, cada uma das entidades individuais $e_1, e_2, ..., e_n$ é dito **participar** na instância de relacionamento $r_i = (e_1, e_2, ..., e_n)$.

Informalmente, cada instância de relacionamento r_i em R é uma associação de entidades, onde a associação inclui exatamente uma entidade de cada tipo de entidade participante. Cada instância de relacionamento r_i desse tipo representa o fato de que as entidades participantes em r estão relacionadas de alguma maneira na situação i do minimundo correspondente. Por exemplo, considere um tipo de relacionamento TRABALHA_PARA entre os dois tipos de entidade FUNCIONARIO e DEPARTAMENTO, que associa cada funcionário ao departamento para o qual o funcionário trabalha no conjunto de entidades correspondente. Cada instância de relacionamento no conjunto de relacionamentos TRABALHA_PARA associa uma entidade FUNCIONARIO a uma entidade DEPARTAMENTO. A Figura 7.9 ilustra esse exemplo, onde cada instância de relacionamento r_i aparece conectada às entidades FUNCIONARIO e DEPARTAMENTO que participam em r_i. No minimundo representado pela Figura 7.9, os funcionários f_1, f_3 e f_6 trabalham para o departamento d_1; os funcionários f_2 e f_4 trabalham para o departamento d_2; e os funcionários f_5 e f_7 trabalham para o departamento d_3.

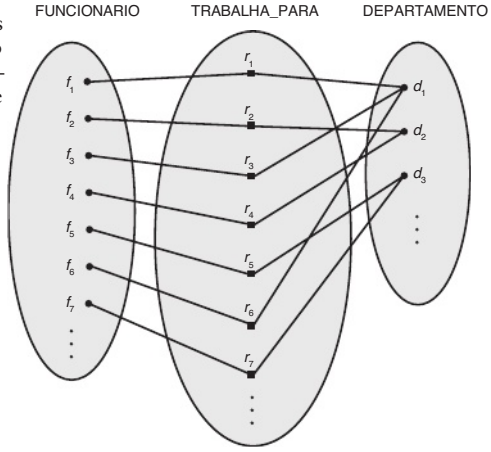


Figura 7.9
Algumas instâncias no conjunto de relacionamentos TRABALHA_PARA, que representa um tipo de relacionamento TRABALHA_PARA entre FUNCIONARIO e DEPARTAMENTO.

Nos diagramas ER, os tipos de relacionamento são exibidos como caixas em forma de losango, que são conectadas por linhas retas às caixas retangulares que representam os tipos de entidade participantes. O nome do relacionamento é exibido na caixa em forma de losango (ver Figura 7.2).

7.4.2 Grau de relacionamento, nomes de função e relacionamentos recursivos

Grau de um tipo de relacionamento. O **grau** de um tipo de relacionamento é o número dos tipos de entidade participantes. Logo, o relacionamento TRABALHA_PARA tem grau dois. Um tipo de relacionamento de grau dois é chamado de **binário**, e um tipo de grau três é chamado de **ternário**. Um exemplo de relacionamento ternário é FORNECE, mostrado na Figura 7.10, em que cada instância de relacionamento r_i associa três entidades — um fornecedor f, uma peça p e um projeto j — sempre que f fornece a peça p ao projeto j. Os relacionamentos geralmente podem ser de qualquer grau, mas os mais comuns são os relacionamentos binários. Relacionamentos de grau mais alto geralmente são mais complexos do que os binários. Nós os caracterizaremos mais adiante, na Seção 7.9.

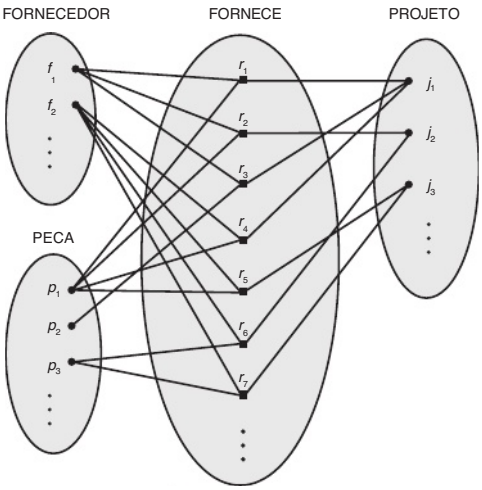


Figura 7.10
Algumas instâncias de relacionamento no conjunto de relacionamento ternário FORNECE.

Relacionamentos como atributos. Às vezes, é conveniente pensar em um tipo de relacionamento binário em termos de atributos, conforme discutimos na Seção 7.3.3. Considere o tipo de relacionamento TRABALHA_PARA da Figura 7.9. Pode-se pensar em um atributo chamado Departamento do tipo de entidade FUNCIONARIO, em que o valor do Departamento para cada entidade FUNCIONARIO é a (uma referência à) entidade DEPARTAMENTO para a qual esse funcionário trabalha. Logo, o conjunto de valores para esse atributo Departamento é o conjunto de *todas* as entidades DEPARTAMENTO, que é o conjunto de entidades DEPARTAMENTO. Foi isso que fizemos na Figura 7.8, quando especificamos o projeto inicial do tipo de entidade FUNCIONARIO para o banco de dados EMPRESA. Porém, quando pensamos em um relacionamento binário como um atributo, sempre temos duas opções. Neste exemplo, a alternativa é pensar em um atributo multivalorado Funcionario do tipo de entidade DEPARTAMENTO, cujos valores para cada entidade DEPARTAMENTO é o conjunto de entidades de FUNCIONARIO que trabalham para esse departamento. O conjunto de valores desse atributo Funcionario é o conjunto de potência do conjunto de entidades FUNCIONARIO. Qualquer um desses dois atributos — Departamento de FUNCIONARIO ou Funcionario de DEPARTAMENTO — pode representar o tipo de relacionamento TRABALHA_PARA. Se os dois forem representados, eles são restringidos a serem o inverso um do outro.[8]

Nomes de função e relacionamentos recursivos. Cada tipo de entidade que participa de um tipo de relacionamento desempenha nele uma função em particular. O **nome da função** significa a função que uma entidade participante do tipo de entidade desempenha em cada instância de relacionamento, e ajuda a explicar o que o relacionamento significa. Por exemplo, no tipo de relacionamento TRABALHA_PARA, FUNCIONARIO desempenha a função de *funcionário* ou *trabalhador*, e DEPARTAMENTO desempenha a função de *departamento* ou *empregador*.

Os nomes de função não são tecnicamente necessários nos tipos de relacionamento em que todos os tipos de entidade participantes são distintos, pois o nome de cada tipo de entidade participante pode ser usado como nome de função. Contudo, em algumas ocasiões, o mesmo tipo de entidade participa mais de uma vez em um tipo de relacionamento em *funções diferentes*. Nesses casos, o nome da função torna-se essencial para distinguir o significado da função que cada entidade

[8] Esse conceito de representar os tipos de relacionamento como atributos é usado em uma classe de modelos de dados chamada **modelos de dados funcionais**. Nos bancos de dados de objeto (ver Capítulo 11), os relacionamentos podem ser representados por atributos de referência, seja em uma direção ou nas duas direções como inversos. Em bancos de dados relacionais (ver Capítulo 3), as chaves estrangeiras são um tipo de atributo de referência usado para representar relacionamentos.

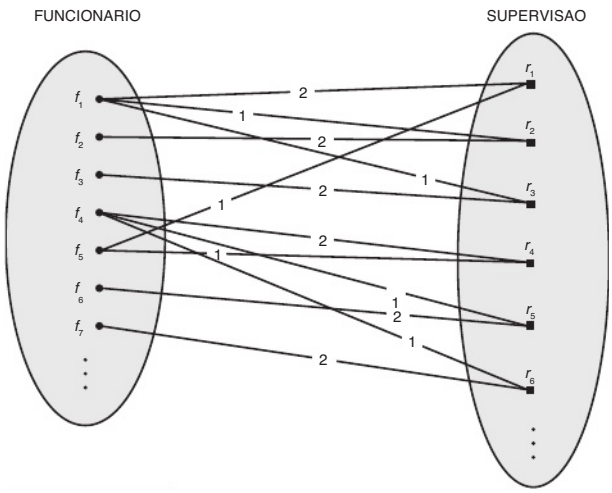


Figura 7.11
Um relacionamento recursivo SUPERVISAO entre FUNCIONARIO no papel de *supervisor* (1) e FUNCIONARIO no papel de *subordinado* (2).

[9] N significa *qualquer número* de entidades relacionadas (zero ou mais).

participante desempenha. Esses tipos de relacionamentos são chamados de **relacionamentos recursivos.** A Figura 7.11 mostra um exemplo. O tipo de relacionamento SUPERVISAO relaciona um funcionário a um supervisor, no qual as entidades funcionário e supervisor são membros do mesmo conjunto entidade FUNCIONARIO. Logo, o tipo de entidade FUNCIONARIO *participa duas vezes* na SUPERVISAO: uma vez no papel de *supervisor* (ou *chefe*) e outra no papel de *supervisionado* (ou *subordinado*). Cada instância de relacionamento r_i na SUPERVISAO associa duas entidades de funcionário f_j e f_k, uma das quais desempenha a função de supervisor e a outra, a função de supervisionado. Na Figura 7.11, as linhas marcadas com '1' representam a função de supervisor, e aquelas marcadas com '2' representam a função de supervisionado; assim, f_1 supervisiona f_2 e f_3, f_4 supervisiona f_6 e f_7, e f_5 supervisiona f_1 e f_4. Neste exemplo, cada instância de relacionamento precisa ser conectada com duas linhas, uma marcada com '1' (supervisor) e a outra com '2' (supervisionado).

7.4.3 Restrições sobre tipos de relacionamento binários

Os tipos de relacionamento costumam ter certas restrições que limitam as combinações de entidades que podem participar no conjunto de relacionamentos correspondente. Essas restrições são determinadas com base na situação do mundo que os relacionamentos representam. Por exemplo, na Figura 7.9, se a empresa tem uma regra de que cada funcionário precisa trabalhar para exatamente um departamento, então gostaríamos de descrever essa restrição no esquema. Podemos distinguir dois tipos principais de restrições de relacionamento binário: *razão de cardinalidade* e *participação*.

Razões de cardinalidade para relacionamentos binários. A **razão de cardinalidade** para um relacionamento binário especifica o número *máximo* de instâncias de relacionamento em que uma entidade pode participar. Por exemplo, no tipo de relacionamento binário TRABALHA_PARA, DEPARTAMENTO:FUNCIONARIO tem razão de cardinalidade 1:N, significando que cada departamento pode estar relacionado a (ou seja, emprega) qualquer número de funcionários,[9] mas um funcionário só pode estar relacionado a (trabalha para) um departamento. Isso significa que, para esse relacionamento TRABALHA_PARA em particular, uma entidade de departamento em particular pode estar relacionada a qualquer número de funcionários (N indica que não existe um número máximo). Por sua vez, um funcionário pode estar relacionado no máximo a um único departamento. As razões de cardinalidade possíveis para tipos de relacionamento binários são 1:1, 1:N, N:1 e M:N.

Um exemplo de relacionamento binário 1:1 é GERENCIA (Figura 7.12), o qual relaciona uma entidade de departamento ao funcionário que gerencia esse departamento. Isso representa as restrições do minimundo que — em qualquer ponto no tempo — um funcionário pode gerenciar apenas um departamento e um departamento pode ter apenas um gerente. O tipo de relacionamento TRABALHA_EM (Figura 7.13) é de razão de cardinalidade M:N, porque a regra do minimundo é a de que um funcionário pode trabalhar em vários projetos e um projeto pode ter vários funcionários.

As razões de cardinalidade para relacionamentos binários são representadas nos diagramas ER exibidos do 1, M e N nos losangos, como mostra a Figura 7.2. Observe que, nessa notação, podemos especificar nenhum máximo (N) ou um máximo de um (1) na participação. Uma notação alternativa (ver Seção 7.7.4) permite que o projetista especifique um *número máximo* na participação, como 4 ou 5.

Restrições de participação e dependências de existência. A **restrição de participação** especifica se a existência de uma entidade depende dela estar relacionada a outra entidade por meio do tipo de relacionamento. Essa restrição especifica o número *mínimo* de instâncias de relacionamento em que cada entidade pode participar, e às vezes é chamada de **restrição de cardinalidade mínima**. Existem dois tipos de restrições de participação — total e parcial — que vamos ilustrar. Se a política de uma empresa afirma que *todo* funcionário precisa trabalhar para um departamento, então uma entidade de funcionário só pode existir se participar em, pelo menos, uma instância de relacionamento TRABALHA_PARA (Figura 7.9). Assim, a participação de FUNCIONARIO em TRABALHA_PARA é chamada de **participação total**, significando que cada entidade *no conjunto total* de entidades de funcionário deve estar relacionada a uma entidade de departamento por meio de TRABALHA_PARA. A participação total também é conhecida como **dependência de existência**. Na Figura 7.12, não esperamos que cada funcionário gerencie um departamento, de modo que a participação de FUNCIONARIO no tipo de relacionamento GERENCIA é **parcial**, significando que *uma parte do conjunto de* entidades de funcionário está relacionada a alguma entidade de departamento por meio de GERENCIA, mas não necessariamente todas. Vamos nos referir à razão de cardinalidade e restrições de participação, juntas, como as **restrições estruturais** de um tipo de relacionamento.

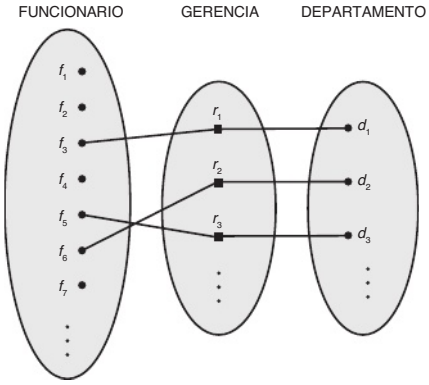


Figura 7.12
Um relacionamento 1:1, GERENCIA.

Em diagramas ER, a participação total (ou dependência de existência) é exibida como uma *linha dupla* que conecta o tipo de entidade participante ao relacionamento, enquanto a participação parcial é representada por uma *linha simples* (ver Figura 7.2). Observe que, nessa notação, podemos especificar nenhum mínimo (participação parcial) ou um mínimo de um (participação total). A notação alternativa (ver Seção 7.7.4) permite que o projetista indique um *número mínimo* específico da participação no relacionamento, como 4 ou 5.

Discutiremos as restrições sobre os relacionamentos de grau mais alto na Seção 7.9.

7.4.4 Atributos de tipos de relacionamento

Os tipos de relacionamento também podem ter atributos, semelhantes àqueles dos tipos de entidade. Por exemplo, para registrar o número de horas por semana que um funcionário trabalha em determinado projeto, podemos incluir um atributo Horas para o tipo de relacionamento TRABALHA_EM na Figura 7.13. Outro exemplo é incluir a data em que um gerente começou a chefiar um departamento por meio de um atributo Data_inicio para o tipo de relacionamento GERENCIA na Figura 7.12.

Observe que os atributos dos tipos de relacionamento 1:1 ou 1:N podem ser migrados para um dos tipos de entidade participantes. Por exemplo, o atributo Data_inicio para o relacionamento GERENCIA pode ser um atributo de FUNCIONARIO ou de DEPARTAMENTO, embora conceitualmente ele pertença a GERENCIA. Isso porque GERENCIA é um relacionamento 1:1, de modo que cada entidade de departamento ou funcionário participa de *no máximo uma* instância de relacionamento. Logo, o valor do atributo Data_inicio pode ser determinado separadamente,

pela entidade do departamento participante ou pela entidade do funcionário participante (gerente).

Para um tipo de relacionamento 1:N, um atributo de relacionamento pode ser migrado *somente* para o tipo de entidade no lado N do relacionamento. Por exemplo, na Figura 7.9, se o relacionamento TRABALHA_PARA também tiver um atributo Data_inicio que indica quando um funcionário começou a trabalhar para um departamento, esse atributo pode ser incluído como um atributo de FUNCIONARIO. Isso porque cada funcionário trabalha para somente um departamento, e por isso participa de, no máximo, uma instância de relacionamento em TRABALHA_PARA. Nos tipos de relacionamento 1:1 e 1:N, a decisão de onde colocar um atributo de relacionamento — como um atributo de tipo de relacionamento ou como um atributo de um tipo de entidade participante — é determinada de maneira subjetiva pelo projetista do esquema.

Para tipos de relacionamento M:N, alguns atributos podem ser determinados pela *combinação de entidades participantes* em uma instância de relacionamento, e não por qualquer entidade isolada. Esses atributos *precisam ser especificados como atributos de relacionamento*. Um exemplo é o atributo Horas do relacionamento M:N de TRABALHA_EM (Figura 7.13). O número de horas por semana que um funcionário trabalha atualmente em um projeto é determinado por uma combinação funcionário-projeto, e não de maneira separada por qualquer entidade.

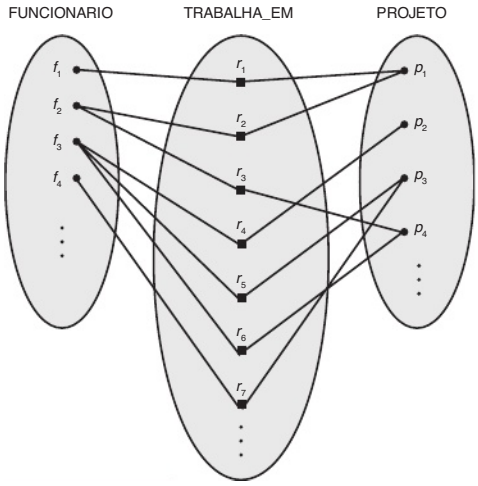


Figura 7.13
Um relacionamento M:N, TRABALHA_EM.

7.5 Tipos de entidade fraca

Tipos de entidade que não possuem atributos-chave próprios são chamados **tipos de entidade fraca.** Ao contrário, os **tipos de entidade regulares** que não têm um atributo-chave — o que inclui todos os exemplos discutidos até aqui — são chamados **tipos de entidade fortes**. As entidades pertencentes a um tipo de entidade fraca são identificadas por estarem relacionadas a entidades específicas de outro tipo em combinação com um de seus valores de atributo. Chamamos esse outro tipo de entidade de **tipo de entidade de identificação** ou **proprietário,**[10] e chamamos o tipo de relacionamento que relaciona um tipo de entidade fraca a seu proprietário de **relacionamento de identificação** do tipo de entidade fraca.[11] Um tipo de entidade fraca sempre tem uma *restrição de participação total* (dependência de existência) com relação a seu relacionamento de identificação, porque a entidade fraca não pode ser identificada sem uma entidade proprietária. Porém, nem toda dependência de existência resulta em um tipo de entidade fraca. Por exemplo, uma entidade CARTEIRA_MOTORISTA não pode existir a menos que esteja relacionada a uma entidade PESSOA, embora tenha a própria chave (Numero_habilitacao) e, portanto, não seja uma entidade fraca.

Considere o tipo de entidade DEPENDENTE, relacionado a FUNCIONARIO, que é usado para registrar os dependentes de cada funcionário por meio de um relacionamento 1:N (Figura 7.2). Em nosso exemplo, os atributos de DEPENDENTE são Nome (o primeiro nome do dependente), Data_nascimento, Sexo e Parentesco (com o funcionário). Dois dependentes de *dois funcionários distintos* podem, por coincidência, ter os mesmos valores para Nome, Data_nascimento, Sexo e Parentesco, mas, ainda assim, eles são entidades distintas. Eles são identificados como entidades distintas apenas depois de determinar a *entidade de funcionário em particular* à qual cada dependente está relacionado. Considera-se que cada entidade de funcionário *possui* as entidades dependentes que estão relacionadas a ele.

Um tipo de entidade fraca normalmente tem uma **chave parcial**, que é o atributo que pode iden-

[10] O tipo de entidade de identificação é também chamado de **tipo de entidade pai** ou **tipo de entidade dominante**
[11] O tipo de entidade fraca também é chamado de **tipo de entidade filho** ou **tipo de entidade subordinado**

tificar exclusivamente as entidades fracas que estão *relacionadas à mesma entidade proprietária*.[12] Em nosso exemplo, se considerarmos que dois dependentes do mesmo funcionário não poderão ter o mesmo nome, o atributo Nome de DEPENDENTE é a chave parcial. No pior dos casos, um atributo composto de *todos os atributos da entidade fraca* será a chave parcial.

Em diagramas ER, tanto um tipo de entidade fraca quanto seu relacionamento de identificação são distinguidos ao delimitar suas caixas e losangos com linhas duplas (ver Figura 7.2). O atributo de chave parcial é sublinhado com uma linha tracejada ou pontilhada.

Os tipos de entidade fraca às vezes podem ser representados como atributos complexos (compostos, multivalorados). No exemplo anterior, poderíamos especificar um atributo multivalorado Dependentes para FUNCIONARIO, que é um atributo composto com os atributos componentes Nome, Data_nascimento, Sexo e Parentesco. A escolha de qual representação usar é feita pelo projetista de banco de dados. Um critério que pode ser usado é escolher a representação do tipo de entidade fraca se houver muitos atributos. Se a entidade fraca participar independentemente nos tipos de relacionamento além de seu tipo de relacionamento de identificação, então ela não deverá ser modelada como um atributo complexo.

Em geral, podemos definir qualquer quantidade de níveis de tipos de entidade fraca; um tipo de entidade proprietário pode ele mesmo ser um tipo de entidade fraca. Além disso, um tipo de entidade fraca pode ter mais de um tipo de entidade de identificação e um tipo de relacionamento de identificação de grau maior que dois, conforme ilustraremos na Seção 7.9.

7.6 Refinando o projeto ER para o banco de dados EMPRESA

Agora, podemos refinar o projeto de banco de dados da Figura 7.8 alterando os atributos que representam relacionamentos para tipos de relacionamento. A razão de cardinalidade e a restrição de participação de cada tipo de relacionamento são determinadas com base nos requisitos listados na Seção 7.2. Se alguma delas não puder ser especificada dessa maneira, os usuários terão de ser questionados ainda mais para determinar essas restrições estruturais.

Em nosso exemplo, especificamos os seguintes tipos de relacionamento:

- GERENCIA, um tipo de relacionamento 1:1 entre FUNCIONARIO e DEPARTAMENTO. A participação de FUNCIONARIO é parcial. A participação de DEPARTAMENTO não é clara pelos requisitos. Questionamos os usuários, que dizem que um departamento precisa ter um gerente o tempo todo, o que implica participação total.[13] O atributo Data_inicio é atribuído a esse tipo de relacionamento.
- TRABALHA_PARA, um tipo de relacionamento 1:N entre DEPARTAMENTO e FUNCIONARIO. As duas participações são totais.
- CONTROLA, um tipo de relacionamento 1:N entre DEPARTAMENTO e PROJETO. A participação de PROJETO é total, enquanto a de DEPARTAMENTO é determinada para ser parcial, depois que os usuários indicaram que alguns departamentos podem não controlar projeto algum.
- SUPERVISAO, um tipo de relacionamento 1:N entre FUNCIONARIO (no papel de supervisor) e FUNCIONARIO (no papel de supervisionado). As duas participações são determinadas como sendo parciais, depois que os usuários indicaram que nem todo funcionário é um supervisor e nem todo funcionário tem um supervisor.
- TRABALHA_EM, determinado como sendo um tipo de relacionamento M:N com atributo Horas, depois que os usuários indicaram que um projeto pode ter vários funcionários trabalhando nele. As duas participações são determinadas como totais.
- DEPENDENTES_DE, um tipo de relacionamento 1:N entre FUNCIONARIO e DEPENDENTE, que também é o relacionamento de identificação para o tipo de entidade fraca DEPENDENTE. A participação de FUNCIONARIO é parcial, enquanto a de DEPENDENTE é total.

Depois de especificar os seis tipos de relacionamento citados, removemos dos tipos de entidade da Figura 7.8 todos os atributos que foram refinados para relacionamentos. Estes incluem Gerente e Data_inicio_gerente de DEPARTAMENTO, Departamento_gerenciador de PROJETO; Departamento, Supervisor e Trabalha_em de FUNCIONARIO; e Funcionario de DEPENDENTE. É importante ter o mínimo possível de redundância quando projetamos o esquema concei-

[12] A chave parcial às vezes é chamada de **discriminadora**

[13] As regras no minimundo que determinam as restrições também são chamadas de *regras de negócio*, pois elas são determinadas pelo *negócio* ou pela organização que utilizará o banco de dados.

tual de um banco de dados. Se alguma redundância for desejada no nível de armazenamento ou no nível de visão do usuário, ela pode ser introduzida mais tarde, conforme discutimos na Seção 1.6.1.

7.7 Diagramas ER, convenções de nomes e questões de projeto

7.7.1 Resumo da notação para diagramas ER

As figuras 7.9 a 7.13 ilustram exemplos da participação dos tipos de entidade nos tipos de relacionamento ao exibir seus conjuntos ou extensões — as instâncias de entidade individuais em um conjunto de entidades e as instâncias de relacionamento individuais em um conjunto de relacionamentos. Nos diagramas ER, a ênfase está em representar os esquemas em vez das instâncias. Isso é mais útil no projeto de banco de dados porque um esquema de banco de dados muda raramente, enquanto o conteúdo dos conjuntos de entidades muda com frequência. Além disso, o esquema é obviamente mais fácil de exibir, pois é muito menor.

A Figura 7.2 exibe o **esquema de banco de dados** EMPRESA como um **diagrama ER**. Agora, revisamos a notação completa do diagrama ER. Tipos de entidade como FUNCIONARIO, DEPARTAMENTO e PROJETO são mostrados nas caixas retangulares. Tipos de relacionamento como TRABALHA_PARA, GERENCIA, CONTROLA e TRABALHA_EM são mostrados em caixas em forma de losango, conectadas aos tipos de entidade participantes com linhas retas. Os atributos são mostrados em ovais, e cada atributo é conectado por uma linha reta a seu tipo de entidade ou tipo de relacionamento. Os atributos componentes de um atributo composto são conectados à oval que representa o atributo composto, conforme ilustrado pelo atributo Nome de FUNCIONARIO. Os atributos multivalorados aparecem em ovais duplas, conforme ilustrado pelo atributo Localizacoes de DEPARTAMENTO. Os atributos-chave têm seus nomes sublinhados. Os atributos derivados aparecem em ovais pontilhadas, conforme ilustrado pelo atributo Numero_funcionarios de DEPARTAMENTO.

Os tipos de entidade fraca são distinguidos ao serem colocados em retângulos duplos e terem seu relacionamento de identificação colocado em losangos duplos, conforme ilustrado pelo tipo de entidade DEPENDENTE e DEPENDENTES_DE identificando o tipo de relacionamento. A chave parcial do tipo de entidade fraca é sublinhada com uma linha tracejada.

Na Figura 7.2, a razão de cardinalidade de cada tipo de relacionamento *binário* é especificada pela conecção de um 1, M ou N em cada aresta participante. A razão de cardinalidade de DEPARTAMENTO:FUNCIONARIO em GERENCIA é 1:1, enquanto é 1:N para DEPARTAMENTO: FUNCIONARIO em TRABALHA_PARA, e M:N para TRABALHA_EM. A restrição de participação é especificada por uma linha simples para participação parcial e por linhas duplas para a participação total (dependência de existência).

Na Figura 7.2, mostramos os nomes de papel para o tipo de relacionamento SUPERVISAO, pois o mesmo tipo de entidade FUNCIONARIO desempenha dois papéis distintos nesse relacionamento. Observe que a razão de cardinalidade é 1:N de supervisor para supervisionado porque cada funcionário no papel de supervisionado tem, no máximo, um supervisor direto, ao passo que um funcionário no papel de supervisor pode controlar zero ou mais funcionários.

A Figura 7.14 resume as convenções para diagramas ER. É importante observar que existem muitas outras notações diagramáticas alternativas (ver Seção 7.7.4 e Apêndice A).

7.7.2 Nomeação apropriada de construções de esquema

Ao projetar um esquema de banco de dados, a escolha de nomes para tipos de entidade, atributos, tipos de relacionamento e (particularmente) funções nem sempre é simples. É preciso escolher nomes que transmitam, tanto quanto possível, os significados conectados às diferentes construções no esquema. Escolhemos usar *nomes no singular* para os tipos de entidade, em vez de nomes no plural, porque o nome se aplica a cada entidade individual pertencente a esse tipo de entidade. Em nossos diagramas ER, usaremos a convenção de que os nomes do tipo de entidade e tipo de relacionamento são escritos em letras maiúsculas, os nomes de atributo têm apenas a letra inicial em maiúscula e os nomes de papel são escritos em letras minúsculas. Usamos essa convenção na Figura 7.2.

Como uma prática geral, dada uma descrição narrativa dos requisitos do banco de dados, os *nomes* que aparecem na narrativa tendem a gerar nomes de tipo de entidade, e os *verbos* tendem a indicar nomes de tipos de relacionamento. Os nomes de atributo costumam surgir de nomes adicionais que descrevem os nomes correspondentes para tipos de entidade.

Outra consideração de nomeação envolve a escolha de nomes de relacionamento binário para tornar o diagrama ER do esquema legível da esquerda para a direita e de cima para baixo. Seguimos essa orientação de modo geral na Figura 7.2. Para explicar essa convenção de nomeação ainda mais, temos uma exceção para a Figura 7.2 — o tipo de relaciona-

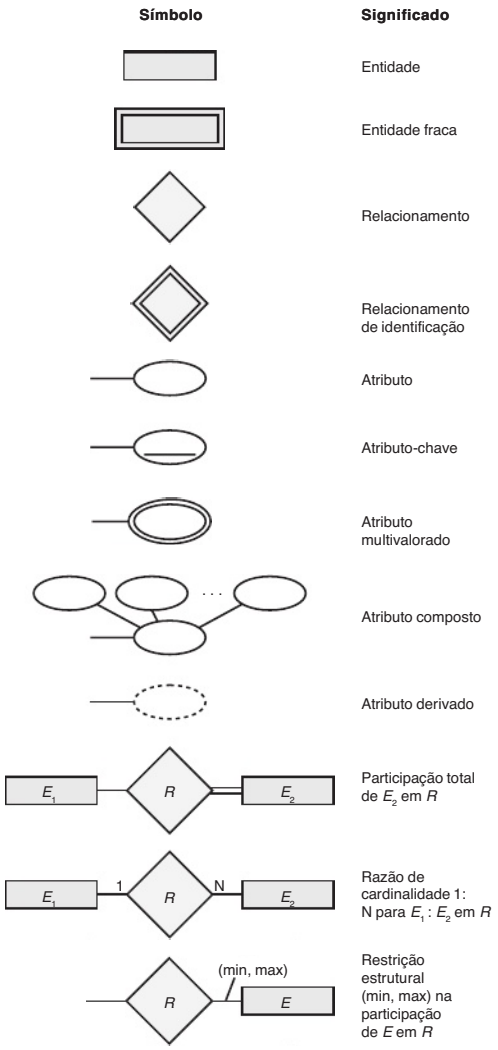


Figura 7.14
Resumo da notação para diagramas ER.

mento DEPENDENTES_DE, lido de baixo para cima. Quando descrevemos esse relacionamento, podemos dizer que as entidades DEPENDENTE (tipo de entidade inferior) são DEPENDENTES_DE (nome de relacionamento) um FUNCIONARIO (tipo de entidade superior). Para mudar isso e ler de cima para baixo, poderíamos renomear o tipo de relacionamento para POSSUI_DEPENDENTES, que então seria lido da seguinte forma: uma entidade FUNCIONARIO (tipo de entidade superior) POSSUI_DEPENDENTES (nome de relacionamento) do tipo DEPENDENTE (tipo de entidade inferior). Observe que esse problema surge porque cada relacionamento binário pode ser descrito começando de qualquer um dos dois tipos de entidades participantes, conforme discutido no início da Seção 7.4.

7.7.3 Escolhas de projeto para o projeto conceitual ER

Às vezes, é difícil decidir se um conceito em particular no minimundo deve ser modelado como um tipo de entidade, um atributo ou um tipo de relacionamento. Nesta seção, oferecemos algumas orientações rápidas sobre qual construção deve ser escolhida em situações específicas.

Em geral, o processo de projeto de esquema deve ser considerado um processo de refinamento iterativo, no qual um projeto inicial é criado e depois refinado iterativamente até que seja alcançado o mais adequado. Alguns dos refinamentos frequentemente utilizados incluem o seguinte:

- Um conceito pode ser modelado como um atributo e depois refinado para um relacionamento, pois é determinado que o atributo é uma referência para outro tipo de entidade. Como exemplo, considere um atributo Gerente_projeto do tipo de entidade PROJETO. Isso pode ser refinado em um relacionamento GERENCIA_PROJETO entre FUNCIONARIO e PROJETO. Discutimos esse tipo de refinamento com detalhes na Seção 7.6. É importante observar que, em nossa notação, quando um atributo é substituído por um relacionamento, o próprio atributo deve ser removido do tipo de entidade para evitar duplicação e redundância.
- De modo semelhante, um atributo que existe em vários tipos de entidade pode ser elevado ou promovido para um tipo de entidade independente. Por exemplo, suponha que cada tipo de entidade em um banco de dados UNIVERSIDADE, como ALUNO, PROFESSOR e DISCIPLINA, tenha um atributo Departamento no projeto inicial. O projetista pode então escolher criar um tipo de entidade DEPARTAMENTO com um único atributo Dept_nome e relacioná-lo aos três tipos de entidade (ALUNO, PROFESSOR e DISCIPLINA) por meio de relacionamentos apropriados. Outros atributos/relacionamentos de DEPARTAMENTO podem ser descobertos mais tarde.

- Um refinamento inverso para o caso anterior pode ser aplicado — por exemplo, se um tipo de entidade DEPARTAMENTO existir no projeto inicial com um atributo isolado Dept_nome e estiver relacionado a somente outro tipo de entidade, ALUNO. Nesse caso, DEPARTAMENTO pode ser reduzido ou rebaixado para um atributo de ALUNO.
- A Seção 7.9 vai discutir as escolhas referentes ao grau de um relacionamento. No Capítulo 8, discutiremos outros refinamentos referentes à especialização/generalização. O Capítulo 10 abordará refinamentos top-down e bottom-up adicionais que são comuns no projeto de esquema conceitual em grande escala.

7.7.4 Notações alternativas para diagramas ER

Existem muitas notações diagramáticas alternativas para exibir diagramas ER. O Apêndice A mostra algumas das mais populares. Na Seção 7.8, vamos introduzir a notação Unified Modeling Language (UML) para diagramas de classe, que foi proposta como um padrão para a modelagem conceitual de objeto.

Nesta seção, descrevemos uma notação ER alternativa para especificar restrições estruturais sobre os relacionamentos, que substitui a razão de cardinalidade (1:1, 1:N, M:N) e a notação de linha simples/dupla para as restrições de participação. Essa notação envolve associar um par de números inteiros (min, max) a cada *participação* de um tipo de entidade E em um tipo de relacionamento R, onde $0 \leq \text{min} \leq \text{max}$ e $\text{max} \geq 1$. Os números significam que, para cada entidade e em E, e precisa participar de pelo menos min e no máximo max instâncias de relacionamento em R *em qualquer ponto no tempo*. Nesse método, min = 0 implica participação parcial, enquanto min > 0 implica participação total.

A Figura 7.15 mostra o esquema de banco de dados EMPRESA usando a notação (min, max).[14] Em geral, usa-se ou a notação de razão de cardinalidade/linha, simples/linha dupla *ou* a notação (min, max). A notação (min, max) é mais precisa, e podemos usá-la para especificar algumas restrições estruturais para os tipos de relacionamento de *maior grau*. Porém, isso não é suficiente para especificar algumas restrições de chave nos relacionamentos de maior grau, conforme discutiremos na Seção 7.9.

A Figura 7.15 também apresenta todos os nomes de papel para o esquema de banco de dados EMPRESA.

7.8 Exemplo de outra notação: diagramas de classes UML

A metodologia UML está sendo bastante utilizada no projeto de software e tem muitos tipos de diagramas para diversas finalidades do projeto de software. Aqui, apresentamos rapidamente os fundamentos dos **diagramas de classes UML** e os comparamos com os diagramas ER. De algumas maneiras, os diagramas de classes podem ser considerados uma notação alternativa aos diagramas ER. A notação e os conceitos adicionais da UML serão apresentados na Seção 8.6 e no Capítulo 10. A Figura 7.16 mostra como o esquema de banco de dados ER EMPRESA da Figura 7.15 pode ser exibido usando a notação de diagrama de classes UML. Os *tipos de entidade* na Figura 7.15 são modelados como *classes* na Figura 7.16. Uma *entidade* em ER corresponde a um *objeto* em UML.

Nos diagramas de classes UML, uma **classe** (semelhante a um tipo de entidade em ER) é exibida como uma caixa (ver Figura 7.16) que inclui três seções: a seção superior mostra o **nome da classe** (semelhante ao nome do tipo de entidade); a seção do meio inclui os **atributos**; e a última seção inclui as **operações** que podem ser aplicadas aos objetos individuais (semelhante às entidades individuais em um conjunto de entidades) da classe. As operações *não são* especificadas em diagramas ER. Considere a classe FUNCIONARIO na Figura 7.16. Seus atributos são Nome, Cpf, Datanasc, Sexo, Endereco e Salario. O projetista pode, opcionalmente, especificar o **domínio** de um atributo, se desejar, colocando um sinal de dois-pontos (:) seguido pelo nome ou descrição do domínio, conforme ilustrado pelos atributos Nome, Sexo e Datanasc de FUNCIONARIO na Figura 7.16. Um atributo composto é modelado como um **domínio estruturado**, conforme ilustrado pelo atributo Nome de FUNCIONARIO. Um atributo multivalorado geralmente será modelado como uma classe separada, conforme ilustrado pela classe LOCALIZACAO na Figura 7.16.

Os tipos de relacionamento são chamados de **associações** em terminologia UML, e as instâncias de relacionamento são chamadas de **ligações**. Uma **associação binária** (tipo de relacionamento binário) é representada como uma linha que conecta as classes participantes (tipos de entidade) e pode, de maneira opcional, ter um nome. Um atributo de relacionamento, chamado **atributo de ligação**, é colocado em uma caixa que está conectada à linha da associa-

[14] Em algumas notações, particularmente aquelas usadas nas metodologias de modelagem de objeto, como UML, o (min, max) é colocado *nos lados opostos* aos que mostramos. Por exemplo, para o relacionamento TRABALHA_PARA da Figura 7.15, o (1,1) estaria no lado DEPARTAMENTO, e o (4,N) estaria no lado FUNCIONARIO. Aqui, usamos a notação srcinal de Abrial (1974).

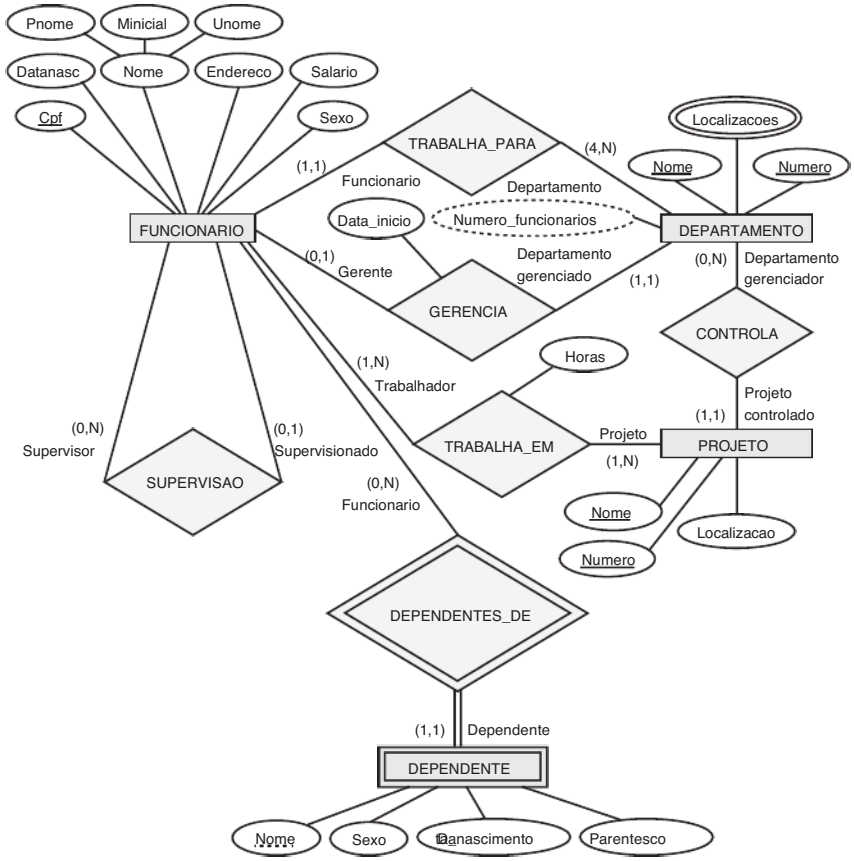


Figura 7.15
Diagramas ER para o esquema EMPRESA, com restrições estruturais especificadas usando a notação (min, max) e nomes de função.

ção por uma linha tracejada. A notação (min, max) descrita na Seção 7.7.4 é usada para especificar restrições de relacionamento, que são chamadas **multiplicidades** em terminologia UML. As multiplicidades são especificadas na forma *min..max*, e um asterisco (*) indica nenhum limite máximo na participação. Contudo, as multiplicidades são colocadas *nos lados opostos do relacionamento* quando comparadas com a notação discutida na Seção 7.7.4 (compare as figuras 7.15 e 7.16). Em UML, um único asterisco indica uma multiplicidade de 0..*, e um único 1 indica uma multiplicidade de 1..1. Um relacionamento recursivo (ver Seção 7.4.2) é chamado de **associação reflexiva** em UML, e os nomes de função — como as multiplicidades — são colocados nos cantos opostos de uma associação quando comparados com o posicionamento dos nomes de função na Figura 7.15.

Em UML, existem dois tipos de relacionamentos: associação e agregação. A **agregação** serve para representar um relacionamento entre um objeto inteiro e suas partes componentes, e possui uma notação diagramática distinta. Na Figura 7.16, modelamos os locais de um departamento e o local isolado de um projeto como agregações. Porém, agregação

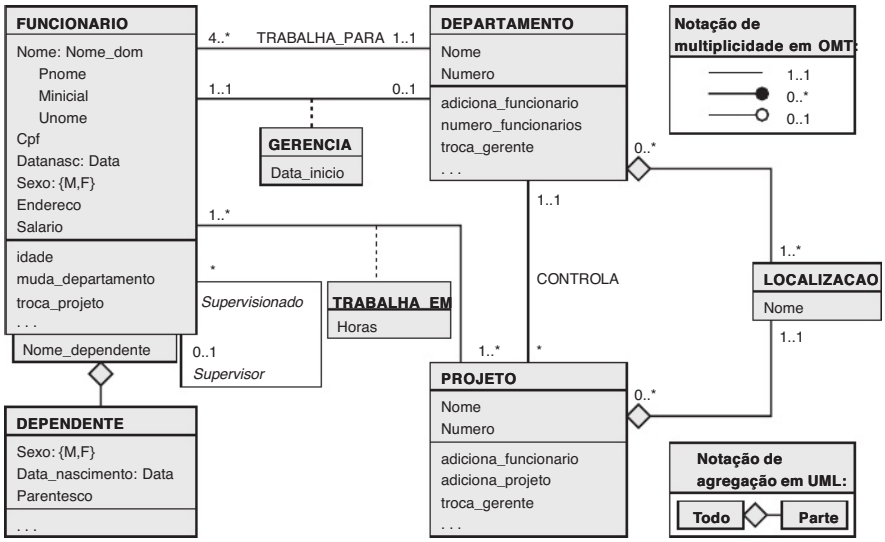


Figura 7.16
O esquema conceitual EMPRESA na notação do diagrama de classes UML.

e associação não possuem propriedades estruturais diferentes, e a escolha quanto a qual tipo de relacionamento usar é um tanto subjetiva. No modelo ER, ambas são representadas como relacionamentos.

A UML também distingue entre associações (ou agregações) **unidirecionais** e **bidirecionais**. No caso unidirecional, a linha que conecta as classes é exibida com uma seta para indicar que apenas uma direção para acessar objetos relacionados é necessária. Se nenhuma seta for exibida, o caso bidirecional é assumido, que é o padrão. Por exemplo, se sempre esperamos acessar o gerente de um departamento começando por um objeto DEPARTAMENTO, podemos desenhar a linha de associação representando a associação GERENCIA com uma seta de DEPARTAMENTO para FUNCIONARIO. Além disso, as instâncias de relacionamento podem ser especificadas para serem **ordenadas**. Por exemplo, poderíamos especificar que os objetos do funcionário relacionados a cada departamento por meio da associação (relacionamento) TRABALHA_PARA devem ser ordenados por seu valor de atributo Salario. Os nomes de associação (relacionamento) são *opcionais* em UML, e os atributos do relacionamento são exibidos em uma caixa conectada com uma linha tracejada à linha que representa a associação/agregação (ver Data_inicio e Horas na Figura 7.16).

As operações dadas em cada classe são derivadas dos requisitos funcionais da aplicação, conforme discutimos na Seção 7.1. Em geral, basta especificar os nomes de operação inicialmente para as operações lógicas que deverão ser aplicadas a objetos individuais de uma classe, conforme mostra a Figura 7.16. À medida que o projeto é refinado, mais detalhes são acrescentados, como os tipos de argumento (parâmetros) exatos para cada operação, mais uma descrição funcional de cada operação. A UML tem *descrições de função* e *diagramas de sequência* para especificar alguns dos detalhes da operação, mas estes estão fora do escopo de nossa discussão. O Capítulo 10 apresentará alguns desses diagramas.

Entidades fracas podem ser modeladas usando a construção chamada de **associação qualificada** (ou **agregação qualificada**) em UML. Esta pode representar tanto o relacionamento de identificação quanto a chave parcial, que é colocada em uma caixa ligada à classe proprietária. Isso é ilustrado pela classe DEPENDENTE e sua agregação qualificada a FUNCIONARIO na Figura 7.16. A chave parcial Nome_dependente é chamada de **discriminador** em terminologia UML, pois seu valor distingue os objetos associados (relacionados) ao mesmo FUNCIONARIO. As associações qualificadas não são restritas à modelagem de

entidades fracas e podem ser usadas para modelar outras situações em UML.

Esta seção não pretende oferecer uma descrição completa dos diagramas de classe UML, mas sim ilustrar um tipo popular de notação diagramática alternativa que pode ser utilizada para representar os conceitos de modelagem ER.

7.9 Tipos de relacionamento de grau maior que dois

Na Seção 7.4.2, definimos o **grau** de um tipo de relacionamento como o número de tipos de entidade participantes e chamamos um tipo de relacionamento de grau dois de *binário*, e de grau três de *ternário*. Nesta seção, explicamos melhor as diferenças entre os relacionamentos binário e de grau maior, quando escolher relacionamentos de grau maior *versus* binário, além de como especificar as restrições sobre relacionamentos de grau maior.

7.9.1 Escolhendo entre relacionamentos binário e ternário (ou de grau maior)

A notação de diagrama ER para um tipo de relacionamento ternário aparece na Figura 7.17(a), a qual mostra o esquema para o tipo de relacionamento FORNECE que foi mostrado no nível de conjunto de entidades/conjunto de relacionamentos ou de instância na Figura 7.10. Lembre-se de que o conjunto de relacionamentos de FORNECE é um conjunto de instâncias

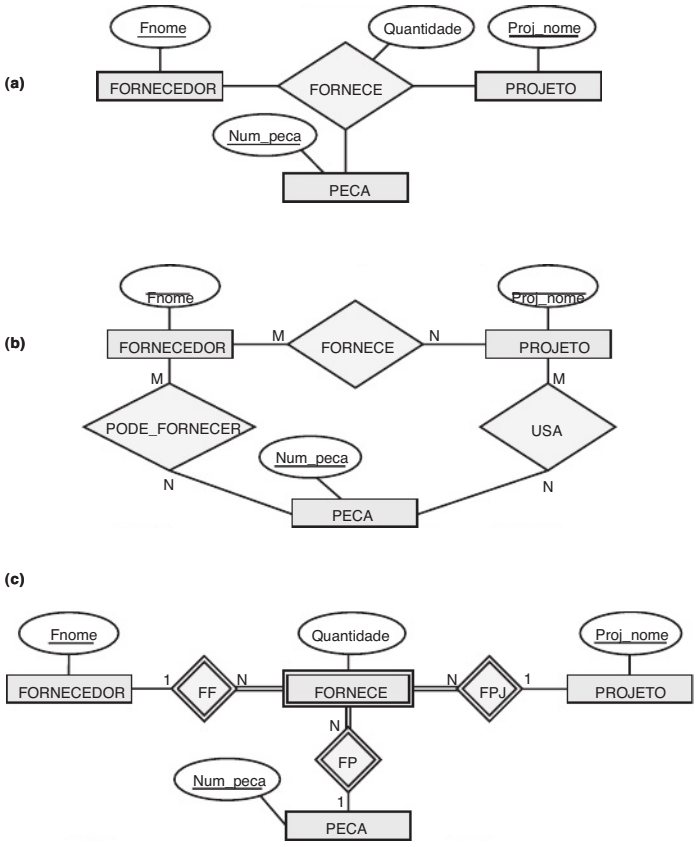


Figura 7.17
Tipos de relacionamento ternário. (a) O relacionamento FORNECE. (b) Três relacionamentos binários não equivalentes a FORNECE. (c) FORNECE representado como um tipo de entidade fraca.

de relacionamento (*f*, *j*, *p*), onde *f* é um FORNECEDOR que atualmente está abastecendo um PROJETO *j* com uma PECA *p*. Em geral, um tipo de relacionamento *R* de grau *n* terá *n* arestas em um diagrama ER, uma conectando *R* a cada tipo de entidade participante.

A Figura 7.17(b) mostra um diagrama ER para os três tipos de relacionamento binário PODE_FORNECER, USA e FORNECE. Em geral, um tipo de relacionamento ternário representa informações diferentes dos três tipos de relacionamento binário. Considere os três tipos de relacionamento binário PODE_FORNECER, USA e FORNECE. Suponha que PODE_FORNECER, entre FORNECEDOR e PECA, inclua uma instância (*f*, *p*) sempre que o fornecedor *f puder fornecer* a peça *p* (a qualquer projeto); USA, entre PROJETO e PECA, inclui uma instância (*j*, *p*) sempre que o projeto *j* usa a peça *p*; e FORNECE, entre FORNECEDOR e PROJETO, inclui uma instância (*f*, *j*) sempre que o fornecedor *f* fornece *alguma peça* ao projeto *j*. A existência de três instâncias de relacionamento (*f*, *p*), (*j*, *p*) e (*f*, *j*) em PODE_FORNECER, USA e FORNECE, respectivamente, não implica que existe uma instância (*f*, *j*, *p*) no relacionamento ternário FORNECE, pois o *significado é diferente*. Com frequência, é complicado decidir se um relacionamento em particular deve ser representado como um tipo de relacionamento de grau *n* ou se deve ser desmembrado em vários tipos de relacionamento de graus menores. O projetista deverá basear essa decisão na semântica ou significado da situação em particular que está sendo representada. A solução típica é incluir ao relacionamento ternário *com* um ou mais dos relacionamentos binários, se eles representarem significados diferentes e se todos forem necessários à aplicação.

Algumas ferramentas de projeto de banco de dados são baseadas em variações do modelo ER que permitem apenas relacionamentos binários. Nesse caso, um relacionamento ternário como FORNECE deve ser representado como um tipo de entidade fraca, sem chave parcial e com três relacionamentos de identificação. Os três tipos de entidade participantes FORNECEDOR, PECA e PROJETO são, juntos, os tipos de entidade proprietária (ver Figura 7.17(c)). Logo, uma entidade no tipo de entidade fraca FORNECE na Figura 7.17(c) é identificada pela combinação de suas três entidades proprietárias de FORNECEDOR, PECA e PROJETO.

Também é possível representar o relacionamento ternário como um tipo de entidade regular introduzindo uma chave artificial ou substituta. Neste exemplo, um atributo-chave Cod_fornecimento poderia ser usado para o tipo de entidade FORNECE, convertendo-o em um tipo de entidade regular. Três relacionamentos binários N:1 relacionam FORNECE aos três tipos de entidade participantes.

Outro exemplo é mostrado na Figura 7.18. O tipo de relacionamento ternário OFERECE representa informações sobre professores que oferecem cursos durante determinados semestres. Logo, ele inclui uma instância de relacionamento (*p*, *s*, *d*) sempre que o PROFESSOR *p* oferece a DISCIPLINA *d* durante o SEMESTRE *s*. Os três tipos de relacionamento binário mostrados na Figura 7.18 têm os seguintes significados: PODE_LECIONAR relaciona uma disciplina aos professores que *podem lecionar* esta disciplina, LECIONOU_DURANTE relaciona um semestre aos professores que *lecionaram* durante esse semestre, e OFERECIDA_DURANTE relaciona um semestre às disciplinas oferecidas durante esse semestre *por qualquer professor*. Esses relacionamentos ternários e binários representam informações diferentes, mas certas restrições deverão ser mantidas entre os relacionamentos. Por exemplo, uma instância de relacionamento (*p*, *s*, *d*) não deve existir em OFERECE *a menos que* exista uma instância (*p*, *s*) em LECIONOU_DURANTE , uma instância (*s*, *d*) existe em OFERECIDA_DURANTE e uma instância (*p*, *d*) existe em PODE_LECIONAR. Contudo, a recíproca nem sempre é verdadeira: podemos ter instâncias (*p*, *s*), (*s*, *d*) e (*p*, *d*) nos três tipos de relacionamento binário sem a instância correspondente (*p*, *s*, *d*) em OFERECE. Observe que, neste exemplo, com base no significado dos relacionamentos, podemos deduzir as instâncias de LECIONOU_DURANTE e OFERECIDA_DURANTE com base nas instâncias em OFERECE, mas não podemos deduzir as instâncias de PODE_LECIONAR. Portanto, LECIONOU_DURANTE e OFERECIDA_DURANTE são redundantes e podem ser omitidas.

Embora em geral três relacionamentos binários *não possam* substituir um relacionamento ternário, eles podem fazer isso sob certas *restrições adicionais*. Em nosso exemplo, se o relacionamento PODE_LE-

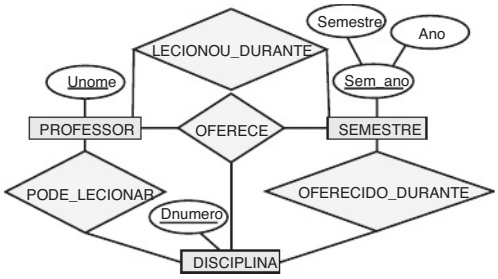


Figura 7.18
Outro exemplo de tipos de relacionamento ternário *versus* binário.

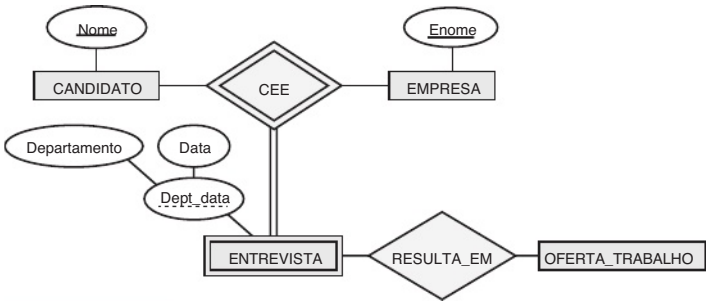


Figura 7.19
Um tipo de entidade fraca ENTREVISTA com um tipo de relacionamento de identificação ternário.

[15] Esta notação nos permite determinar a chave da *relação do relacionamento*, conforme discutiremos no Capítulo 9.
[16] Isso também é verdadeiro para razões de cardinalidade dos relacionamentos binários.

CIONAR for 1:1 (um professor pode lecionar uma disciplina, e uma disciplina pode ser lecionada por apenas um professor), então o relacionamento ternário OFERECE pode ser omitido porque pode ser deduzido pelos três relacionamentos binários PODE_LECIONAR, LECIONOU_DURANTE e OFERECIDA_DURANTE. O projetista do esquema precisa analisar o significado de cada situação específica para decidir quais dos tipos de relacionamento binário e ternário são necessários.

Observe que é possível ter um tipo de entidade fraca com um ternário (*n*-ário) identificando o tipo de relacionamento. Nesse caso, o tipo de entidade fraca pode ter *vários* tipos de entidade proprietários. Um exemplo é mostrado na Figura 7.19. Ele mostra parte de um banco de dados que registra candidatos para entrevistas de emprego em diversas empresas, e pode fazer parte de um banco de dados de agência de emprego, por exemplo. Nos requisitos, um candidato pode ter várias entrevistas com a mesma empresa (por exemplo, com diferentes departamentos dela ou em datas separadas), mas uma oferta de emprego é feita com base em uma das entrevistas. Aqui, ENTREVISTA é representada como uma entidade fraca com dois proprietários CANDIDATO e EMPRESA, e com a chave parcial Dept_data. Uma entidade ENTREVISTA é identificada exclusivamente por um candidato, uma empresa e a combinação da data e departamento da entrevista.

7.9.2 Restrições sobre relacionamentos ternários (ou de grau mais alto)

Existem duas notações para especificar restrições estruturais sobre relacionamentos *n*-ários, e elas especificam restrições diferentes. Assim, *ambas devem ser usadas* se for importante determinar totalmente as restrições estruturais sobre um relacionamento ternário ou de grau maior. A primeira notação é baseada na notação de razão de cardinalidade dos relacionamentos binários exibidos na Figura 7.2. Aqui, um 1, M ou N é especificado em cada arco de participação (os símbolos M e N significam *muitos* ou *qualquer número*).[15] Vamos ilustrar essa restrição usando o relacionamento FORNECE da Figura 7.17.

Lembre-se de que o conjunto de relacionamento de FORNECE é um conjunto de instâncias de relacionamento (*f*, *j*, *p*), em que *f* é um FORNECEDOR, *j* é um PROJETO e *p* é uma PECA. Suponha que exista a restrição de que, para determinada combinação de projeto-peça, somente um fornecedor será usado (somente um fornecedor abastece determinado projeto com determinada peça). Nesse caso, colocamos 1 na participação de FORNECEDOR, e M, N nas participações de PROJETO, PECA na Figura 7.17. Isso especifica a restrição de que uma combinação em particular (*j*, *p*) pode aparecer no máximo uma vez no conjunto de relacionamento, pois cada combinação (PROJETO, PECA) desse tipo determina de maneira exclusiva um único fornecedor. Logo, qualquer instância de relacionamento (*f*, *j*, *p*) é identificada exclusivamente no conjunto de relacionamentos por sua combinação (*j*, *p*), que torna (*j*, *p*) uma chave para o conjunto de relacionamentos. Nessa notação, as participações que têm 1 especificado nelas não precisam fazer parte da chave de identificação para o conjunto de relacionamentos.[16] Se todas as três cardinalidades forem M ou N, então a chave será a combinação de todos os três participantes.

A segunda notação é baseada na notação (min, max) exibida na Figura 7.15 para relacionamentos

binários. Um (min, max) em uma participação aqui especifica que cada entidade está relacionada a pelo menos *min* e no máximo *max instâncias de relacionamento* no conjunto de relacionamentos. Essas restrições não têm influência na determinação da chave de um relacionamento *n*-ário, no qual $n > 2$,[17] mas especificam um tipo diferente de restrição, que faz restrições sobre o número de instâncias de relacionamento de que cada entidade participa.

Resumo

Neste capítulo, apresentamos os conceitos de modelagem de um modelo de dados conceitual de alto nível, o modelo Entidade-Relacionamento (ER). Começamos discutindo o papel que um modelo de dados de alto nível desempenha no processo de projeto de banco de dados, e depois apresentamos um exemplo de conjunto de requisitos para o banco de dados EMPRESA, que é um dos exemplos usados no decorrer deste livro. Definimos os conceitos básicos do modelo ER de entidades e seus atributos. Depois, discutimos os valores NULL e apresentamos os diversos tipos de atributos, que podem ser aninhados arbitrariamente para produzir atributos complexos:

- Simples ou atômicos.
- Compostos.
- Multivalorados.

Também discutimos rapidamente atributos armazenados *versus* derivados. Depois, abordamos os conceitos do modelo ER no nível de esquema ou 'conotação':

- Tipos de entidade e seus conjuntos de entidades correspondentes.
- Atributos-chave dos tipos de entidade.
- Conjuntos de valores (domínios) dos atributos.
- Tipos de relacionamento e seus conjuntos de relacionamentos correspondentes.
- Funções de participação dos tipos de entidade nos tipos de relacionamento.

Apresentamos dois métodos para especificar as restrições estruturais sobre tipos de relacionamento. O primeiro método distinguiu dois tipos de restrições estruturais:

- Razões de cardinalidade (1:1, 1:N, M:N para relacionamentos binários).
- Restrições de participação (total, parcial).

Observamos que, como alternativa, outro método de especificação de restrições estruturais é fazê-lo com números mínimo e máximo (min, max) sobre a participação de cada tipo de entidade em um tipo de relacionamento. Discutimos sobre tipos de entidade fraca e os conceitos relacionados de tipos de entidade proprietária, tipos de relacionamento de identificação e atributos-chave parciais.

Os esquemas Entidade-Relacionamento podem ser representados de maneira diagramática como diagramas ER. Mostramos como projetar um esquema ER para o banco de dados EMPRESA ao definir, primeiro, os tipos de entidade e seus atributos e depois refinando o projeto para incluir tipos de relacionamento. Apresentamos o diagrama ER para o esquema de banco de dados EMPRESA. Discutimos alguns dos conceitos básicos dos diagramas de classe UML e como eles se relacionam aos conceitos de modelagem ER. Também descrevemos os tipos de relacionamento ternário e de grau maior com mais detalhes, e discutimos as circunstâncias sob as quais eles são distinguidos dos relacionamentos binários.

Os conceitos de modelagem ER que apresentamos até aqui — tipos de entidade, tipos de relacionamento, atributos, chaves e restrições estruturais — podem modelar muitas aplicações de banco de dados. Contudo, aplicações mais complexas — como projeto de engenharia, sistemas de informações médicas e telecomunicações — exigem conceitos adicionais se quisermos modelá-las com maior precisão. Discutiremos alguns conceitos de modelagem avançados no Capítulo 8 e analisaremos técnicas de modelagem de dados ainda mais avançadas no Capítulo 26.

Perguntas de revisão

7.1. Discuta o papel de um modelo de dados de alto nível no processo de projeto de banco de dados.

7.2. Liste os diversos casos em que o uso de um valor NULL seria apropriado.

7.3. Defina os seguintes termos: *entidade*, *atributo*, *valor de atributo*, *instância de relacionamento*, *atributo composto*, *atributo multivalorado*, *atributo derivado*, *atributo complexo*, *atributo-chave* e *conjunto de valores* (*domínio*).

7.4. O que é um tipo de entidade? O que é um conjunto de entidades? Explique as diferenças entre uma entidade, um tipo de entidade e um conjunto de entidades.

7.5. Explique a diferença entre um atributo e um conjunto de valores.

7.6. Qual é a diferença entre uma instância de relacionamento, um tipo de relacionamento e um conjunto de relacionamentos?

7.7. O que é uma função de participação? Quando é necessário usar nomes de função na descrição dos tipos de relacionamento?

[17] No entanto, as restrições (min, max) podem determinar as chaves para relacionamentos binários.

7.8. Descreva as duas alternativas para especificar restrições estruturais sobre tipos de relacionamento. Quais são as vantagens e desvantagens de cada um?

7.9. Sob que condições um atributo de um tipo de relacionamento binário pode ser migrado para se tornar um atributo de um dos tipos de entidade participantes?

7.10. Quando pensamos nos relacionamentos como atributos, quais são os conjuntos de valores desses atributos? Que classe de modelos de dados é baseada nesse conceito?

7.11. O que queremos dizer com um tipo de relacionamento recursivo? Dê alguns exemplos.

7.12. Quando o conceito de uma entidade fraca é usado na modelagem de dados? Defina os termos *tipo de entidade proprietária*, *tipo de entidade fraca*, *tipo de relacionamento de identificação* e *chave parcial*.

7.13. Um relacionamento de identificação de um tipo de entidade fraca pode ser de um grau maior que dois? Dê exemplos para ilustrar sua resposta.

7.14. Discuta as convenções para exibir um esquema ER como um diagrama ER.

7.15. Discuta as convenções de nomeação usadas para os diagramas de esquema ER.

Exercícios

7.16. Considere o seguinte conjunto de requisitos para um banco de dados UNIVERSIDADE, que é usado para registrar os históricos dos alunos. Este é semelhante, mas não idêntico, ao banco de dados mostrado na Figura 1.2:

a. A universidade registrar o nome, número de aluno, número do CPF, endereço atual e com seu número de telefone fixo, endereço permanente com seu número de telefone fixo, data de nascimento, sexo, turma (novato, segundo ano, ..., formado), departamento principal, departamento secundário (se houver) e programa de formação (graduação, mestrado, ..., doutorado) de cada aluno. Algumas aplicações do usuário precisam se referir à cidade, estado e CEP do endereço permanente do aluno e ao sobrenome do aluno. O número do CPF e o número de aluno possuem valores exclusivos para cada um deles.

b. Cada departamento é descrito por um nome, código de departamento, número de escritório, número de telefone comercial e faculdade. Nome e código possuem valores exclusivos para cada departamento.

c. Cada disciplina tem um nome, descrição, número de disciplina, número de horas por semestre, nível e departamento que oferece. O valor do número da disciplina é exclusivo para cada uma delas.

d. Cada turma tem um professor, semestre, ano, disciplina e número de turma. O número de turma distingue as turmas da mesma disciplina que são lecionadas durante o mesmo semestre/ano; seus valores são 1, 2, 3, ..., até o número de turmas lecionadas durante cada semestre.

e. Um relatório de notas tem um aluno, turma, nota com letra e nota numérica (0 A 10).

Projete um esquema ER para essa aplicação e desenhe um diagrama ER para o esquema. Especifique os atributos de chave de cada tipo de entidade, e as restrições estruturais sobre cada tipo de relacionamento. Observe quaisquer requisitos não especificados e faça suposições apropriadas para tornar a especificação completa.

7.17. Atributos compostos e multivalorados podem ser aninhados para qualquer número de níveis. Suponha que queiramos projetar um atributo para um tipo de entidade ALUNO a fim de registrar a formação acadêmica anterior. Esse atributo terá uma entrada para cada faculdade frequentada anteriormente, e cada entrada desse tipo será composta de um nome de faculdade, datas de início e término, entradas de título (títulos concedidos nessa faculdade, se houver) e entradas de histórico (disciplinas completadas nessa faculdade, se houver). Cada entrada de título contém o nome do título, o mês e o ano em que o título foi conferido, e cada entrada de histórico contém um nome de disciplina, semestre, ano e turma. Crie um atributo para manter essa informação. Use as convenções da Figura 7.5.

7.18. Mostre um projeto alternativo para o atributo descrito no Exercício 7.17 que use apenas tipos de entidade (incluindo tipos de entidade fraca, se for preciso) e tipos de relacionamento.

7.19. Considere o diagrama ER da Figura 7.20, que mostra um esquema simplificado para um sistema de reserva aérea. Extraia do diagrama ER os requisitos e restrições que produziram esse esquema. Tente ser o mais preciso possível em sua especificação de requisitos e restrições.

7.20. Nos capítulos 1 e 2, discutimos o ambiente e os usuários de banco de dados. Podemos considerar muitos tipos de entidade para descrever tal ambiente, como SGBD, banco de dados armazenado, DBA e catálogo/dicionário de dados. Tente especificar todos os tipos de entidade que podem descre-

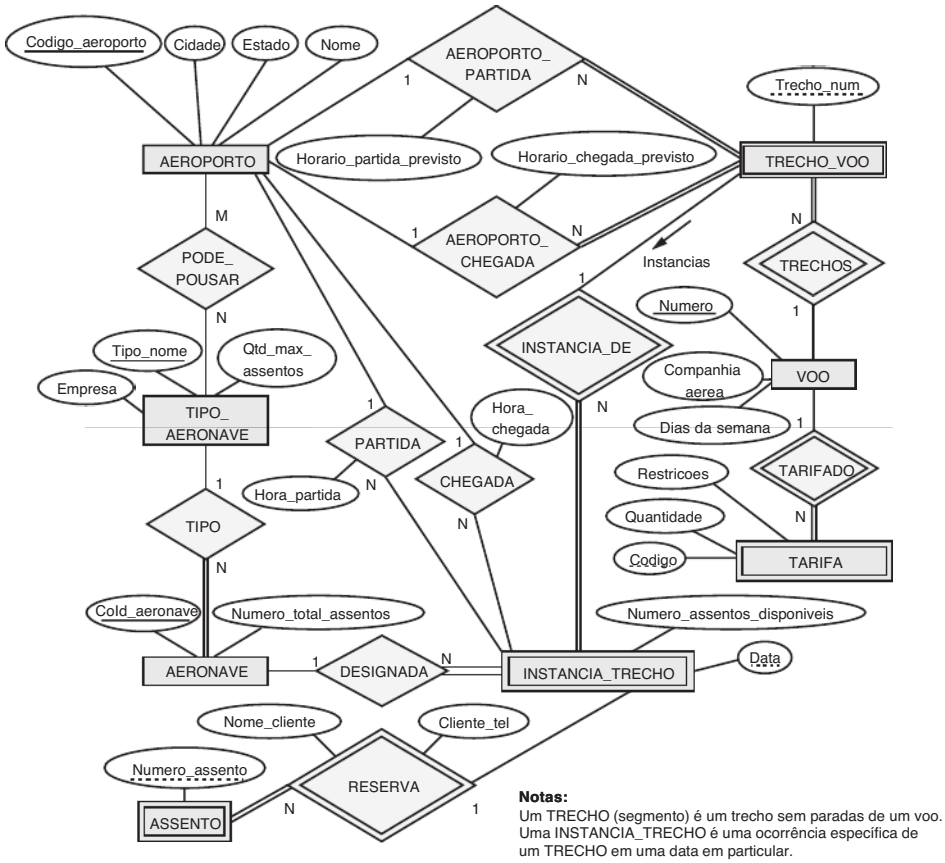


Figura 7.20
Um diagrama ER para um esquema de banco de dados COMPANHIA AEREA

ver totalmente um sistema de banco de dados e seu ambiente; depois, especifique os tipos de relacionamento entre eles e desenhe um diagrama ER para descrever tal ambiente de banco de dados geral.

7.21. Projete um esquema ER para registrar informações sobre votos realizados no Congresso Nacional durante a sessão atual de dois anos no congresso. O banco de dados precisa registrar cada nome de ESTADO do Brasil (por exemplo, 'São Paulo', 'Rio de Janeiro', 'Porto Alegre') e incluir a Regiao do estado (cujo domínio é {'Nordeste', 'Centro-Oeste', 'Sudeste', 'Sul', 'Norte'}). Cada CONGRESSISTA no Congresso Nacional é descrito por seu Nome, mais o Estado representado, a Data_inicio em que o congressista foi eleito pela primeira vez e o Partido político ao qual ele ou ela pertence (cujo domínio é {'Oposição', 'Aliados', 'Independente', 'Outro'}). O banco de dados registra cada PROJETO_LEI (ou seja, lei proposta), incluindo a Nome_proj, a Data_votacao sobre a lei, se a lei passou_ou_falhou (cujo domínio é {'Sim', 'Não'}) e o Proponente (o congressista que patrocinou — ou seja, propôs — a lei). O banco de dados também registra como cada congressista votou em cada lei (domínio do atributo Voto é {'Sim', 'Não', 'Abstenção', 'Nulo'}). Desenhe um diagrama de esquema ER para essa aplicação. Indique claramente quaisquer suposições que você fizer.

7.22. Um banco de dados está sendo construído para registrar os times e jogos de uma liga esportiva. Um time tem uma série de jogadores, nem todos participando em todos os jogos. Deseja-se registrar os jogadores que participam em cada jogo para cada time, as posições em que eles jogaram e o resultado do jogo. Crie um diagrama de esquema ER para essa aplicação, indicando quaisquer suposições que você fizer. Escolha seu esporte favorito (por exemplo, futebol, basquete, voleibol).

7.23. Considere o diagrama ER mostrado na Figura 7.21 para parte de um banco de dados BANCO.

Cada banco pode ter várias filiais, e cada filial pode ter várias contas e empréstimos.

a. Liste os tipos de entidade forte (não fraca) no diagrama ER.

b. Existe um tipo de entidade fraca? Se houver, diga seu nome, chave parcial e relacionamento de identificação.

c. Quais restrições a chave parcial e o relacionamento de identificação do tipo de entidade fraca especificam nesse diagrama?

d. Liste os nomes de todos os tipos de relacionamento e especifique a restrição (min, max) sobre cada participação de um tipo de entidade em um tipo de relacionamento. Justifique suas escolhas.

e. Liste resumidamente os requisitos do usuário que levaram a esse projeto de esquema ER.

f. Suponha que cada cliente deva ter pelo menos uma conta, mas esteja restrito a no máximo dois empréstimos de cada vez, e que uma filial de banco não pode ter mais de 1.000 empréstimos. Como isso é exposto nas restrições (min, max)?

7.24. Considere o diagrama ER da Figura 7.22. Suponha que um funcionário possa trabalhar em até dois departamentos ou não possa ser atribuído a qualquer departamento. Suponha que cada departamento deva ter um e possa ter até três números de telefone. Forneça restrições (min, max) sobre esse diagrama. *Indique claramente quaisquer suposições adicionais que estiver fazendo*. Sob que condições o relacionamento POSSUI_TELEFONE seria redundante neste exemplo?

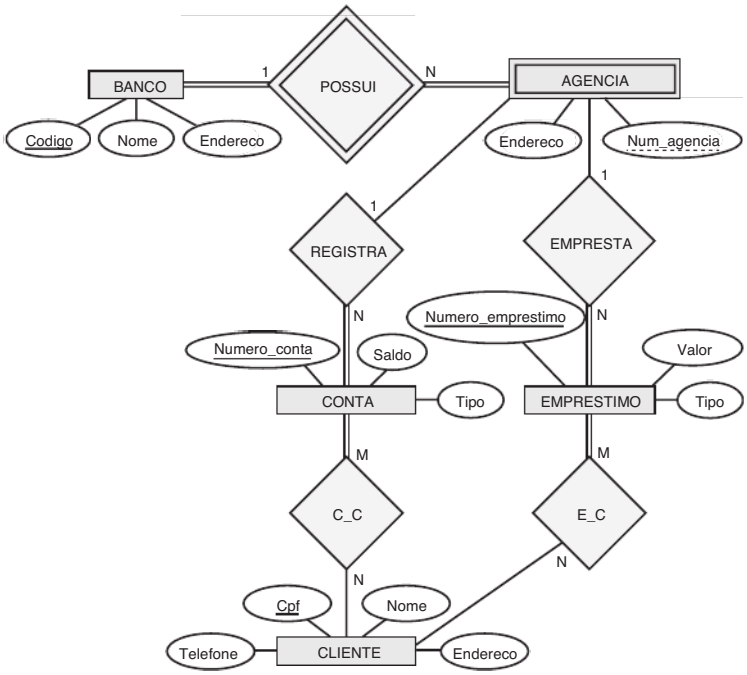


Figura 7.21
Um diagrama ER para um esquema de banco de dados BANCO.

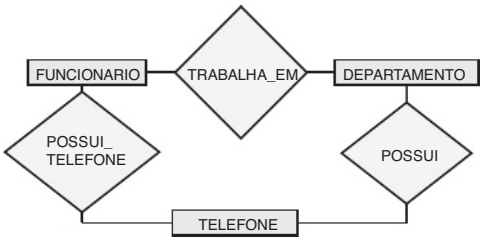


Figura 7.22
Parte de um diagrama ER para um banco de dados EMPRESA.

7.25. Considere o diagrama ER da Figura 7.23. Suponha que uma disciplina possa ou não usar um livro-texto, mas que um texto por definição é um livro que é usado em alguma disciplina. Uma disciplina não pode usar mais de cinco livros. Os professores lecionam de duas a quatro disciplinas. Forneça restrições (min, max) sobre esse diagrama. *Indique claramente quaisquer suposições adicionais que estiver fazendo*. Se acrescentarmos o relacionamento ADOTADO, para indicar o(s) livros(s)-texto que um professor utiliza para uma disciplina, ele deverá ser um relacionamento binário entre PROFESSOR e LIVRO_TEXTO, ou um relacionamento ternário entre todos os três tipos de entidade? Que restrições (min, max) você incluiria? Por quê?

7.26. Considere um tipo de entidade TURMA em um banco de dados UNIVERSIDADE, que descreve as ofertas de turmas das disciplinas. Os atributos da TURMA são Numero_turma, Semestre, Ano, Numero_disciplina, Professor, Num_sala (em que a turma é realizada), Predio (onde a turma é realizada), Dias_da_semana (domínio são as combinações possíveis de dias da semana em que a turma pode ser oferecida {'SQS', 'SQ', 'TQ' e assim por diante}) e Horas (domínio são todos os períodos possíveis durante os quais as turmas são oferecidas {'9–9:50', '10–10:50', ..., '15:30–16:50', '17:30–18:20', e assim por diante}). Suponha que Numero_turma seja exclusivo para cada disciplina em determinada combinação de semestre/ano (ou seja, se uma disciplina for oferecida várias vezes durante um semestre em particular, suas ofertas são numeradas com 1, 2, 3, e assim por diante). Existem várias chaves compostas por turma, e alguns atributos são componentes de mais de uma chave. Identifique três chaves compostas e mostre como elas podem ser representadas em um diagrama de esquema ER.

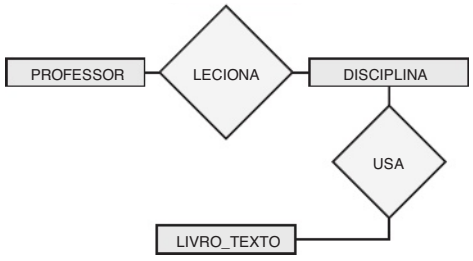


Figura 7.23
Parte de um diagrama ER para um banco de dados DISCIPLINAS.

7.27. Razões de cardinalidade normalmente ditam o projeto detalhado de um banco de dados. A razão de cardinalidade depende do significado no mundo real dos tipos de entidade envolvidos e é definida pela aplicação específica. Para os seguintes relacionamentos binários, sugira razões de cardinalidade com base no significado comum dos tipos de entidade. Indique claramente quaisquer suposições que você fizer.

Entidade 1	Razão de cardinalidade	Entidade 2
1. ALUNO	_____	CADASTRO_ PESSOA_FISICA
2. ALUNO	_____	PROFESSOR
3. SALA_AULA	_____	PAREDE
4. PAIS	_____	PRESIDENTE_ATUAL
5. DISCIPLINA	_____	LIVRO_TEXTO
6. ITEM (que pode ser encontrado em um pedido)	_____	PEDIDO
7. ALUNO	_____	AULA
8. AULA	_____	PROFESSOR
9. PROFESSOR	_____	ESCRITORIO
10. ITEM_LEILOADO	_____	COD_LEILAO

7.28. Considere o esquema ER para o banco de dados FILMES mostrado na Figura 7.24.

Suponha que FILMES seja um banco de dados preenchido. ATOR é usado como um termo genérico e inclui atrizes. Dadas as restrições mostradas no esquema ER, responda às seguintes afirmações com *Verdadeira*, *Falsa* ou *Talvez*. Atribua uma resposta *Talvez* a declarações que, embora não mostradas explicitamente como sendo *Verdadeiras*, não se pode provar que sejam *Falsas* com base no esquema mostrado. Justifique cada resposta.

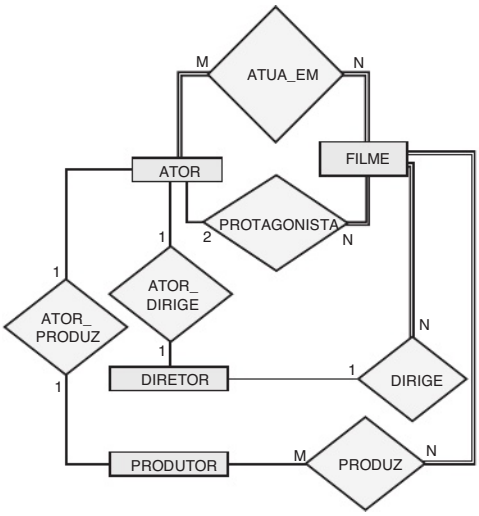


Figura 7.24
Um diagrama ER para um esquema de banco de dados FILMES.

a. Não existem atores neste banco de dados que não estiveram em nenhum filme.
b. Existem alguns atores que atuaram em mais de dez filmes.
c. Alguns atores foram protagonistas em vários filmes.
d. Um filme só pode ter um máximo de dois atores protagonistas.
e. Cada diretor foi ator em algum filme.
f. Nenhum produtor já foi um ator.
g. Um produtor não pode ser ator em outro filme.
h. Existem filmes com mais de doze atores.
i. Alguns produtores também já foram diretores.
j. A maioria dos filmes tem um diretor e um produtor.
k. Alguns filmes têm um diretor, mas vários produtores.
l. Existem alguns atores que foram protagonistas, dirigiram um filme e produziram algum filme.
m. Nenhum filme tem um diretor que também atuou nesse filme.

7.29. Dado o esquema ER para o banco de dados FILMES da Figura 7.24, desenhe um diagrama de instância usando três filmes que foram lançados recentemente. Desenhe instâncias de cada tipo de entidade: FILMES, ATORES, PRODUTORES, DIRETORES envolvidos; crie instâncias dos relacionamentos conforme existem na realidade para esses filmes.

7.30. Ilustre o Diagrama UML para o Exercício 7.16. Seu projeto UML deverá observar os seguintes requisitos:

a. Um aluno deverá ter a capacidade de calcular sua média e acrescentar ou retirar disciplinas obrigatórias e optativas.
b. Cada departamento deverá ser capaz de acrescentar ou retirar disciplinas e contratar ou demitir o corpo docente.
c. Cada professor deverá ser capaz de atribuir ou alterar a nota de um aluno para uma disciplina.

Nota: algumas dessas funções podem se espalhar por várias turmas.

Exercícios de laboratório

7.31. Considere o banco de dados UNIVERSIDADE descrito no Exercício 7.16. Crie o esquema ER para esse banco de dados usando uma ferramenta de modelagem de dados como ERwin ou Rational Rose.

7.32. Considere um banco de dados PEDIDO_CORREIO em que os funcionários fazem pedidos de peças dos clientes. Os requisitos de dados são resumidos da seguinte forma:

- A empresa que vende por catálogo tem funcionários, cada um identificado por um número de funcionário exclusivo, nome e sobrenome, e CEP.
- Cada cliente da empresa é identificado por um número de cliente exclusivo, nome e sobrenome, e CEP.
- Cada peça vendida pela empresa é identificada por um número de peça exclusivo, um nome de peça, preço e quantidade em estoque.
- Cada pedido feito por um cliente é recuperado por um funcionário e recebe um número de pedido exclusivo. Cada pedido contém quantidades especificadas de uma ou mais peças. Cada pedido tem uma data de recebimento bem como uma data de entrega esperada. A data de entrega real também é registrada.

Crie um diagrama Entidade-Relacionamento para o banco de dados de compras por catálogo e construa o projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

7.33. Considere um banco de dados FILME em que os dados são registrados sobre a indústria do cinema. Os requisitos de dados são resumidos a seguir:

- Cada filme é identificado por um título e ano de lançamento. Cada filme tem uma duração em minutos. Cada um tem uma companhia produtora, e

é classificado sob um ou mais gêneros (como terror, ação, drama etc.). Cada filme tem um ou mais diretores e um ou mais atores participando dele. Também tem um resumo da trama. Finalmente, cada filme tem zero ou mais falas, cada uma delas dita por um ator em particular que aparece no filme.

- Os atores são identificados por nome e data de nascimento e aparecem em um ou mais filmes. Cada ator tem um papel no filme.
- Os diretores também são identificados por nome e data de nascimento e dirigem um ou mais filmes. É possível que um diretor atue em um filme (incluindo aquele que ele ou ela também pode dirigir).
- As empresas produtoras são identificadas por nome e cada uma tem um endereço. Uma produtora produz um ou mais filmes.

Crie um diagrama Entidade-Relacionamento para o banco de dados de filmes e construa o projeto usando uma ferramenta de modelagem de dados, como ERwin ou Rational Rose.

7.34. Considere um banco de dados REVISAO_CONFERENCIA em que os pesquisadores submetem seus artigos de pesquisa para avaliação. As análises dos revisores são registradas para uso no processo de seleção de artigo. O sistema de banco de dados atende principalmente a revisores que registram respostas a perguntas de avaliação para cada artigo que eles revisam e fazem recomendações com relação a se aceitar ou rejeitar artigo. Os requisitos de dados são resumidos da seguinte forma:

- Autores de artigos são identificados exclusivamente pelo correio eletrônico. Os nomes e sobrenomes também são registrados.
- Cada artigo recebe um identificador exclusivo pelo sistema e é descrito por um título, resumo e o nome do arquivo eletrônico que contém o artigo.
- Um artigo pode ter vários autores, mas um deles é designado como o autor de contato.
- Os revisores dos artigos são identificados exclusivamente pelo endereço de correio eletrônico. Nome, sobrenome, número de telefone, afiliação e tópicos de interesse de cada revisor também são registrados.
- Cada artigo é atribuído a dois a quatro revisores. Um revisor avalia cada artigo atribuído a ele ou ela em uma escala de 1 a 10, em quatro categorias: mérito técnico, legibilidade, srcinalidade e relevância à conferência. Por fim, cada revisor oferece uma recomendação geral com relação a cada artigo.
- Cada revisão contém dois tipos de comentários escritos: um a ser visto pelo comitê de revisão apenas e o outro como retorno ao(s) autor(es).

Crie um diagrama Entidade-Relacionamento para o banco de dados REVISAO_CONFERENCIA e construa o projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

7.35. Considere o diagrama ER para o banco de dados COMPANHIA AEREA mostrado na Figura 7.20. Construa esse projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

Bibliografia selecionada

O modelo Entidade-Relacionamento foi introduzido por Chen (1976), e um trabalho relacionado aparece em Schmidt e Swenson (1975), Wiederhold e Elmasri (1979) e Senko (1975). Desde então, diversas modificações foram sugeridas no modelo ER. Incorporamos algumas delas em nossa apresentação. Restrições estruturais sobre os relacionamentos são discutidas em Abrial (1974), Elmasri e Wiederhold (1980), e Lenzerini e Santucci (1983). Atributos multivalorados e compostos são incorporados ao modelo ER em Elmasri et al. (1985). Embora não tenhamos discutido as linguagens para o modelo ER e suas extensões, há várias propostas para tais linguagens. Elmasri e Wiederhold (1981) propuseram a linguagem de consulta GORDAS para o modelo ER. Outra linguagem de consulta ER foi proposta por Markowitz e Raz (1983). Senko (1980) apresentou uma linguagem de consulta para o modelo DIAM de Senko. Um conjunto formal de operações chamado álgebra ER foi proposto por Parent e Spaccapietra (1985). Gogolla e Hohenstein (1991) apresentaram outra linguagem formal para o modelo ER. Campbell et al. (1985) apresentaram um conjunto de operações ER e mostraram que elas são completas no sentido relacional. Uma reunião para a disseminação dos resultados de pesquisa relacionada ao modelo ER tem sido mantida regularmente desde 1979. A conferência, agora conhecida como International Conference on Conceptual Modeling, foi realizada em Los Angeles (ER 1979, ER 1983, ER 1997), Washington, D.C. (ER 1981), Chicago (ER 1985), Dijon, na França (ER 1986), Nova York (ER 1987), Roma (ER 1988), Toronto (ER 1989), Lausanne, na Suíça (ER 1990), San Mateo, na Cali fórnia (ER 1991), Karlsruhe, na Alemanha (ER 1992), Arlington, no Texas (ER 1993), Manchester, na Inglaterra (ER 1994), Brisbane, na Austrália (ER 1995), Cottbus, na Alemanha (ER 1996), Cingapura (ER 1998), Paris (ER 1999), Salt Lake City, em Utah, (ER 2000), Yokohama, no Japão (ER 2001), Tampere, na Finlândia (ER 2002), Chicago, em Illinois (ER 2003), Shanghai , na China (ER 2004), Klagenfurt, na Áustria (ER 2005), Tucson, no Arizona (ER 2006), Auckland, na Nova Zelândia (ER 2007), Barcelona, Catalunha, na Espanha (ER 2008) e Gramado, no Rio Grande do Sul, Brasil (ER 2009). A conferência de 2010 será realizada em Vancouver, BC, no Canadá.

capítulo 8

O modelo Entidade-Relacionamento Estendido (EER)

Os conceitos de modelagem ER discutidos no Capítulo 7 são suficientes para representar muitos esquemas de banco de dados para aplicações *tradicionais*, que incluem diversas aplicações de processamento de dados no comércio e na indústria. Desde o final da década de 1970, porém, os projetistas de aplicações de banco de dados têm tentado projetar esquemas de banco de dados mais precisos, que refletem as propriedades de dados e restrições com mais precisão. Isso foi particularmente importante para aplicações mais novas da tecnologia de banco de dados, como aqueles para projeto de engenharia e manufatura (CAD/CAM),[1] telecomunicações, sistemas de software complexos e sistemas de informações geográficas (**GIS — Geographic Information Systems**), entre muitas outras aplicações. Esses tipos de bancos de dados possuem requisitos mais complexos do que as aplicações mais tradicionais. Isso levou ao desenvolvimento de conceitos adicionais de *modelagem semântica de dados*, que foram incorporados em modelos de dados conceituais, como o modelo ER. Vários modelos de dados semânticos têm sido propostos na literatura. Muitos desses conceitos também foram desenvolvidos independentemente nas áreas relacionadas de ciência da computação, como a área de representação do conhecimento da inteligência artificial e a área de modelagem de objeto na engenharia de software.

Neste capítulo, descrevemos recursos que foram propostos para modelos de dados semânticos, e mostramos como o modelo ER pode ser melhorado para incluir esses conceitos, levando ao modelo **ER Estendido** (**EER**).[2] Começamos na Seção 8.1 incorporando os conceitos de *relacionamentos de classe/subclasse* e *herança de tipo* ao modelo ER. Depois, na Seção 8.2, acrescentamos os conceitos de *especialização* e *generalização*. A Seção 8.3 discute os diversos tipos de *restrições* sobre especialização/generalização, e a Seção 8.4 mostra como a construção UNION pode ser modelada ao incluir o conceito de *categoria* no modelo EER. A Seção 8.5 oferece um esquema de banco de dados de exemplo UNIVERSIDADE no modelo EER e resume os conceitos do modelo EER oferecendo definições formais. Usaremos os termos *objeto* e *entidade* com o mesmo significado neste capítulo, pois muitos desses conceitos são comumente usados nos modelos orientados a objeto.

Apresentamos a notação do diagrama de classes UML para representar a especialização e a generalização na Seção 8.6, e as comparamos resumidamente com a notação e os conceitos de EER. Isso serve como um exemplo de notação alternativa, e é uma continuação da Seção 7.8, a qual apresentou a notação básica do diagrama de classes UML que corresponde ao modelo ER básico. Na Seção 8.7, discutimos as abstrações fundamentais que são usadas como base de muitos modelos de dados semânticos. No final do capítulo há um resumo.

Para uma introdução detalhada à modelagem conceitual, o Capítulo 8 deve ser considerado uma continuação do Capítulo 7. Contudo, se apenas uma introdução básica à modelagem ER for desejada, este

[1] CAD/CAM significa Computer-Aided Design/Computer-Aided Manufacturing (projeto auxiliado por computador/fabricação auxiliada por computador).

[2] EER também tem sido usado para indicar o modelo ER Aprimorado.

capítulo poderá ser omitido. Como alternativa, o leitor pode decidir pular algumas ou todas as seções posteriores deste capítulo (seções 8.4 a 8.8).

8.1 Subclasses, superclasses e herança

O modelo EER inclui *todos os conceitos de modelagem do modelo ER* que foram apresentados no Capítulo 7. Além disso, inclui os conceitos de **subclasse** e **superclasse** e os conceitos relacionados de **especialização** e **generalização** (ver seções 8.2 e 8.3). Outro conceito incluído no modelo EER é o de uma **categoria** ou **tipo de união** (ver Seção 8.4) usado para representar uma coleção de objetos (entidades), que é a *união* de objetos de diferentes tipos de entidade. Associado a esses conceitos está o importante mecanismo de **herança de atributo e relacionamento**. Infelizmente, não existe uma terminologia-padrão para esses conceitos, de modo que usamos a terminologia mais comum. A terminologia alternativa é dada nas notas de rodapé. Também descrevemos uma técnica diagramática para exibir esses conceitos quando eles surgem em um esquema EER. Chamamos os diagramas de esquema resultantes de **diagramas ER estendidos**, ou **diagramas EER**.

O primeiro conceito do modelo ER Estendidos (EER) ao qual nos dedicamos é o de um **subtipo** ou **subclasse** de um tipo de entidade. Conforme discutimos no Capítulo 7, um tipo de entidade é usado para representar um *tipo de entidade* e o *conjunto de entidades* ou *coleção de entidades desse tipo* que existem no banco de dados. Por exemplo, o tipo de entidade FUNCIONARIO descreve o tipo (ou seja, os atributos e relacionamentos) de cada entidade de funcionário, e também se refere ao conjunto atual de entidades FUNCIONARIO no banco de dados EMPRESA. Em muitos casos, um tipo de entidade tem diversos subagrupamentos ou subtipos de suas entidades que são significativos e precisam ser representados explicitamente, por causa de seu significado para a aplicação de banco de dados. Por exemplo, as entidades que são membros do tipo de entidade FUNCIONARIO podem ser distinguidas ainda mais em SECRETARIA, ENGENHEIRO, GERENTE, TECNICO, FUNCIONARIO_MENSAL, FUNCIONARIO_HORISTA, e assim por diante. O conjunto de entidades em cada um desses agrupamentos é um subconjunto das entidades que pertencem ao conjunto de entidades FUNCIONARIO, significando que cada entidade que é membro de um desses subagrupamentos também é um funcionário. Chamamos cada um desses subagrupamentos de **subclasse** ou **subtipo** do tipo de entidade FUNCIONARIO, e o tipo de entidade FUNCIONARIO é chamado de **superclasse** ou **supertipo** para cada uma dessas subclasses. A Figura 8.1 mostra como representar esses conceitos nos diagramas EER. (A notação de círculo na Figura 8.1 será explicada na Seção 8.2.)

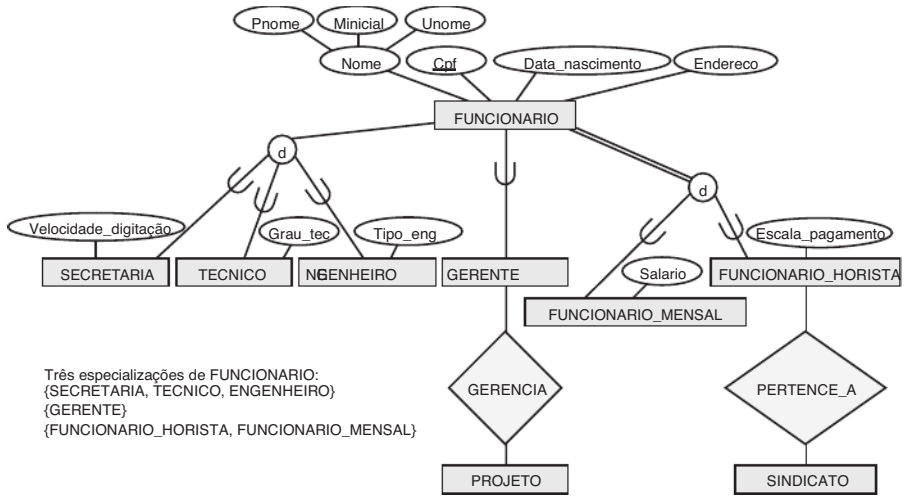


Figura 8.1
Notação do diagrama EER para representar subclasses e especialização.

Chamamos o relacionamento entre uma superclasse e qualquer uma de suas subclasses de **relacionamento superclasse/subclasse** ou **supertipo/subtipo**, ou simplesmente **classe/subclasse**.[3] Em nosso exemplo anterior, FUNCIONARIO/SECRETARIA e FUNCIONARIO/TECNICO são dois relacionamentos de classe/subclasse. Observe que uma entidade-membro da subclasse representa a *mesma entidade do mundo real* de algum membro da superclasse. Por exemplo, uma entidade SECRETARIA 'Joana Logano' também é a entidade FUNCIONARIO 'Joana Logano'. Logo, o membro da subclasse é, de certo modo, o mesmo que a entidade na superclasse, mas *em um papel específico*. Assim, quando implementamos um relacionamento de superclasse/subclasse no sistema de banco de dados, podemos representar um membro da subclasse como um objeto de banco de dados distinto — digamos, um registro distinto que é relacionado por meio do atributo-chave a sua entidade de superclasse. Na Seção 9.2, discutiremos diversas opções para representar os relacionamentos de superclasse/subclasse nos bancos de dados relacionais.

Uma entidade não pode existir no banco de dados simplesmente por ser um membro de uma subclasse; ela também precisa ser um membro da superclasse. Essa entidade pode ser incluída opcionalmente como um membro de qualquer número de subclasses. Por exemplo, um funcionário assalariado que também é um engenheiro pertence às duas subclasses ENGENHEIRO e FUNCIONARIO_MENSAL do tipo de entidade FUNCIONARIO. Contudo, não é necessário que toda entidade em uma superclasse seja um membro de alguma subclasse.

Um conceito importante associado às subclasses (subtipos) é o de **herança de tipo**. Lembre-se de que o *tipo* de uma entidade é definido pelos atributos que ela possui e os tipos de relacionamento de que participa. Como uma entidade na subclasse representa a mesma entidade do mundo real da superclasse, ela deve possuir valores para seus atributos específicos *bem como* valores de seus atributos como um membro da superclasse. Dizemos que uma entidade que é um membro de uma subclasse **herda** todos os atributos da entidade como um membro da superclasse. A entidade também herda todos os relacionamentos de que a superclasse participa. Observe que uma subclasse, com os próprios atributos específicos (ou locais) e relacionamentos, junto com todos os atributos e relacionamentos que herda da superclasse, pode ser considerada um *tipo de entidade* por si própria.[4]

8.2 Especialização e generalização

8.2.1 Especialização

Especialização é o processo de definir um *conjunto de subclasses* de um tipo de entidade. Esse tipo de entidade é chamado de **superclasse** da especialização. O conjunto de subclasses que forma uma especialização é definido com base em alguma característica distinta das entidades na superclasse. Por exemplo, o conjunto de subclasses {SECRETARIA, ENGENHEIRO, TECNICO} é uma especialização da superclasse FUNCIONARIO, que distingue as entidades do funcionário com base no *tipo de cargo* de cada uma. Podemos ter várias especializações do mesmo tipo de entidade com base em características distintas. Por exemplo, outra especialização do tipo de entidade FUNCIONARIO pode gerar o conjunto de subclasses {FUNCIONARIO_MENSAL, FUNCIONARIO_HORISTA}. Tal especialização distingue os funcionários baseando-se no *método de pagamento*.

A Figura 8.1 mostra como representamos uma especialização na forma de um diagrama EER. As subclasses que definem uma especialização são conectadas por linhas a um círculo que representa a especialização, o qual está conectado, por sua vez, à superclasse. O *símbolo de subconjunto* em cada linha que conecta uma subclasse ao círculo indica a direção do relacionamento superclasse/subclasse.[5] Os atributos que se aplicam apenas a entidades de uma subclasse em particular — como Velocidade_digitacao de SECRETARIA — são conectados ao retângulo que representa essa subclasse. Estes são chamados de **atributos específicos** (ou **atributos locais**) da subclasse. De modo semelhante, uma subclasse pode participar de **tipos de relacionamento específicos**, como a subclasse FUNCIONARIO_HORISTA que participa do relacionamento PERTENCE_A da Figura 8.1. Explicaremos o símbolo **d** nos círculos da Figura 8.1 e a notação adicional do diagrama EER em breve.

A Figura 8.2 mostra algumas instâncias de entidade que pertencem às subclasses da especialização {SECRETARIA, ENGENHEIRO, TECNICO}. Novamente, observe que uma entidade que pertence a uma subclasse representa *a mesma entidade do mundo real* que é conectada a ela na superclasse FUNCIONARIO, embora uma mesma entidade apareça duas vezes; por exemplo, f_1 aparece em FUNCIONARIO e SECRETARIA na Figura 8.2. Como a figura sugere,

[3] Um relacionamento de classe/subclasse com frequência é chamado de **relacionamento É_UM** (**IS-A** — ou seja, é um(a)), devido ao modo como nos referimos ao conceito. Dizemos que uma SECRETARIA *é uma* FUNCIONARIA, um TECNICO *é um* FUNCIONARIO, e assim por diante.

[4] Em algumas linguagens de programação orientadas a objeto, uma restrição comum é que uma entidade (ou objeto) tenha *apenas um tipo*. Isso geralmente é muito restritivo para a modelagem conceitual do banco de dados.

[5] Existem muitas notações alternativas para a especialização; apresentamos a notação UML na Seção 8.6 e outras notações propostas no Apêndice A.

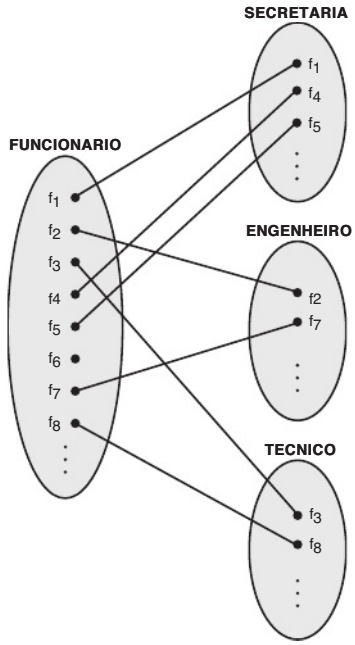


Figura 8.2
Instâncias de uma especialização.

um relacionamento de superclasse/subclasse, como FUNCIONARIO/SECRETARIA, assemelha-se a um relacionamento 1:1 *no nível de instância* (ver Figura 7.12). A principal diferença é que, em um relacionamento 1:1, duas *entidades distintas* estão relacionadas, enquanto em um relacionamento superclasse/subclasse a entidade na subclasse é a mesma entidade do mundo real que a entidade na superclasse, mas está desempenhando um *papel especializado* — por exemplo, um FUNCIONARIO especializado no papel de SECRETARIA, ou um FUNCIONARIO especializado no papel de TECNICO.

Existem dois motivos principais para incluir relacionamentos de classe/subclasse e especializações em um modelo de dados. O primeiro é que certos atributos podem se aplicar a algumas, mas não a todas as entidades da superclasse. Uma subclasse é definida a fim de agrupar as entidades às quais esses atributos se aplicam. Os membros da subclasse ainda podem compartilhar a maioria de seus atributos com os outros membros da superclasse. Por exemplo, na Figura 8.1, a subclasse SECRETARIA tem o atributo específico Velocidade_digitação, enquanto a subclasse ENGENHEIRO tem o atributo específico Tipo_eng, mas SECRETARIA e ENGENHEIRO compartilham seus outros atributos herdados do tipo de entidade FUNCIONARIO.

O segundo motivo para usar subclasses é que alguns tipos de relacionamento podem participar apenas de entidades que são membros da subclasse. Por exemplo, se apenas FUNCIONARIOS_HORISTAS puderem pertencer a um sindicato, podemos representar esse fato criando a subclasse FUNCIONARIO_HORISTA de FUNCIONARIO e relacionando a subclasse a um tipo de entidade SINDICATO por meio do tipo de relacionamento PERTENCE_A, conforme ilustrado na Figura 8.1.

Resumindo, o processo de especialização nos permite fazer o seguinte:

- Definir um conjunto de subclasses de um tipo de entidade.
- Estabelecer atributos específicos adicionais com cada subclasse.
- Estabelecer tipos de relacionamento específico adicionais entre cada subclasse e outros tipos de entidade ou outras subclasses.

8.2.2 Generalização

Podemos pensar em um *processo reverso* da abstração em que suprimimos as diferenças entre vários tipos de entidade, identificamos suas características comuns e as **generalizamos** em uma única **superclasse** da qual os tipos de entidade srcinais são **subclasses** especiais. Por exemplo, considere os tipos de entidade CARRO e CAMINHAO mostrados na Figura 8.3(a). Como eles têm vários atributos comuns, podem ser generalizados no tipo de entidade VEICULO, como mostra a Figura 8.3(b). Tanto CARRO quanto CAMINHAO agora são subclasses da **superclasse generalizada** VEICULO. Usamos o termo **generalização** para nos referir ao processo de definição de um tipo de entidade generalizado com base nos tipos de entidade dados.

Observe que o processo de generalização pode ser visto como sendo funcionalmente o inverso do processo de especialização. Assim, na Figura 8.3, podemos ver {CARRO, CAMINHAO} como uma especialização de VEICULO, em vez de VEICULO como uma generalização de CARRO e CAMINHAO. De modo semelhante, na Figura 8.1, podemos ver FUNCIONARIO como uma generalização de SECRETARIA, TECNICO e ENGENHEIRO. Uma notação diagramática para distinguir generalização de especialização é usada em algumas metodologias de projeto. Uma seta apontando para a superclasse ge-

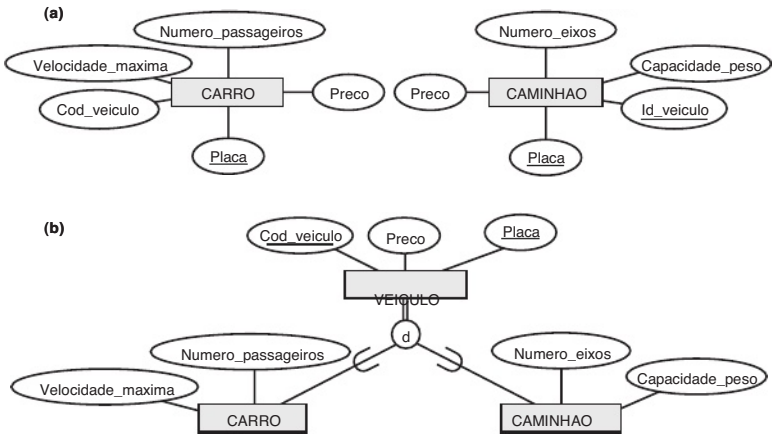


Figura 8.3
Generalização. (a) Dois tipos de entidade, CARRO e CAMINHAO. (b) Generalizando CARRO e CAMINHAO na superclasse VEICULO.

neralizada representa uma generalização, ao passo que setas apontando para subclasses especializadas representam uma especialização. *Não* usaremos essa notação porque a decisão sobre qual processo é seguido em determinada situação costuma ser subjetiva. O Apêndice A oferece algumas notações diagramáticas alternativas sugeridas para diagramas de esquema e diagramas de classe.

Até aqui, apresentamos os conceitos de subclasses e relacionamentos de superclasse/subclasse, bem como os processos de especialização e generalização. Em geral, uma superclasse ou subclasse representa uma coleção de entidades do mesmo tipo e, portanto, também descreve um *tipo de entidade*; é por isso que as superclasses e subclasses são todas mostradas em retângulos nos diagramas EER, como os tipos de entidade. Em seguida, discutimos as propriedades das especializações e generalizações com mais detalhes.

8.3 Restrições e características das hierarquias de especialização e generalização

Primeiro, vamos discutir as restrições que se aplicam a uma única especialização ou a uma única generalização. Para abreviar, nossa discussão refere-se apenas à *especialização*, embora se aplique também à generalização. Depois, vamos discutir as diferenças entre *reticulado* (*herança múltipla*) e *hierarquias* (*herança simples*) de especialização/generalização, e detalhar as diferenças entre os processos de especialização e generalização durante o projeto de esquema de banco de dados conceitual.

8.3.1 Restrições sobre especialização e generalização

Em geral, podemos ter várias especializações definidas no mesmo tipo de entidade (ou superclasse), como mostra a Figura 8.1. Nesse caso, as entidades podem pertencer a subclasses em cada uma das especializações. Contudo, uma especialização também pode consistir em uma *única* subclasse apenas, como a especialização {GERENTE} na Figura 8.1; em tal caso, não usamos a notação de círculo.

Em algumas especializações, podemos determinar exatamente as entidades que se tornarão membros de cada subclasse ao colocar uma condição sobre o valor de algum atributo da superclasse. Essas subclasses são chamadas **subclasses definidas por predicado** (ou **definidas por condição**). Por exemplo, se o tipo de entidade FUNCIONARIO tiver um atributo Tipo_emprego, como mostra a Figura 8.4, podemos especificar a condição de membro na subclasse SECRETARIA pela condição (Tipo_emprego = 'Secretaria'), que chamamos de **predicado de definição** da subclasse. Essa condição é uma *restrição* que especifica exatamente que aquelas entidades do tipo de entidade FUNCIONARIO, cujo valor de atributo para Tipo_emprego é 'Secretaria', pertencem à subclasse. Apresentamos uma subclasse definida pelo predicado escrevendo a condição de predicado ao lado da linha que conecta a subclasse ao círculo de especialização.

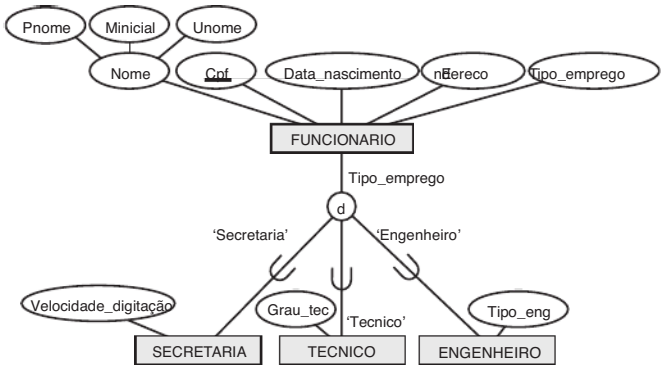


Figura 8.4
Notação do diagrama EER para uma especialização definida por atributo sobre Tipo_emprego.

Se *todas* as subclasses em uma especialização tiverem sua condição de membro no *mesmo* atributo da superclasse, a própria especialização é chamada de **especialização definida por atributo**, e o atributo é chamado de **atributo de definição** da especialização.[6] Nesse caso, todas as entidades com o mesmo valor para o atributo pertencem à mesma subclasse. Apresentamos uma especialização definida por atributo colocando o nome do atributo de definição ao lado do arco que vai do círculo à superclasse, como mostra a Figura 8.4.

Quando não temos uma condição para determinar os membros em uma subclasse, diz-se que esta é **definida pelo usuário**. A condição de membro nessa subclasse é determinada pelos usuários do banco de dados quando eles aplicam a operação para incluir uma entidade à subclasse; logo, a condição de membro é *especificada individualmente para cada entidade pelo usuário,* e não por qualquer condição que possa ser avaliada automaticamente.

Duas outras restrições podem se aplicar a uma especialização. A primeira é a **restrição de disjunção** (ou **desconexão**), que especifica que as subclasses da especialização devem ser disjuntas. Isso significa que uma entidade pode ser um membro de *no máximo* uma das subclasses da especialização. Uma especialização que é definida por atributo implica a restrição de disjunção (se o atributo usado para definir o predicado de membro for de valor único). A Figura 8.4 ilustra esse caso, onde o **d** no círculo significa *disjunção*. A notação **d** também se aplica a subclasses definidas pelo usuário de uma especialização que precisa ser disjunta, conforme ilustrado pela especialização {FUNCIONARIO_HORISTA, FUNCIONARIO_MENSAL} na Figura 8.1. Se as subclasses não forem restringidas a serem disjuntas, seus conjuntos de entidades podem ser **sobrepostos**; ou seja, a mesma entidade (mundo real) pode ser um membro de mais de uma subclasse da especialização. Esse caso, que é o padrão, é exibido colocando-se um **o** (de **overlapping**) no círculo, como mostra a Figura 8.5.

A segunda restrição sobre a especialização é chamada de **restrição de completude** (ou **totalidade**), que pode ser total ou parcial. Uma restrição de **especialização total** especifica que *toda* entidade na superclasse precisa ser um membro de pelo menos uma subclasse na especialização. Por exemplo, se todo FUNCIONARIO tiver de ser um FUNCIONARIO_HORISTA ou um FUNCIONARIO_MENSAL, então a especialização {FUNCIONARIO_HORISTA, FUNCIONARIO_MENSAL} da Figura 8.1 é uma especialização total de FUNCIONARIO. Isso é mostrado nos diagramas EER usando uma linha dupla para conectar a superclasse ao círculo. Uma linha simples é utilizada para exibir uma **especialização parcial**, que permite que uma entidade não pertença a qualquer uma das subclasses. Por exemplo, se algumas entidades FUNCIONARIO não pertencerem a nenhuma das subclasses {SECRETARIA, ENGENHEIRO, TECNICO} nas figuras 8.1 e 8.4, então essa especialização será parcial.[7]

Observe que as restrições de disjunção e completude são *independentes*. Logo, temos quatro restrições possíveis na especialização:

[6] Tal atributo é chamado de *discriminador* em terminologia UML.

[7] A notação de usar linhas simples ou duplas é semelhante à da participação parcial ou total de um tipo de entidade em um tipo de relacionamento, conforme descrito no Capítulo 7.

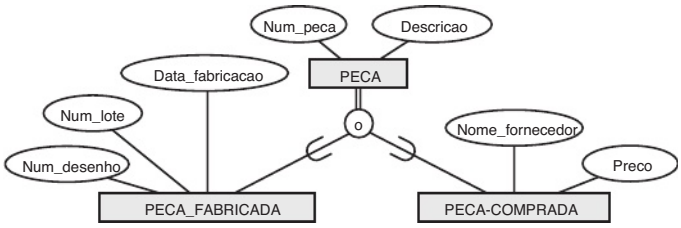


Figura 8.5

Notação de diagrama EER para uma especialização sobreposta (não disjunta).

- Disjunção, total.
- Disjunção, parcial.
- Sobreposição, total.
- Sobreposição, parcial.

Naturalmente, a restrição correta é determinada com base no significado do mundo real que se aplica a cada especialização. Em geral, uma superclasse que foi identificada por meio do processo de *generalização* costuma ser **total**, pois a superclasse é *derivada das* subclasses e, portanto, contém apenas as entidades que estão nas subclasses.

Certas regras de inserção e exclusão se aplicam à especialização (e generalização) como uma consequência das restrições especificadas anteriormente. Algumas dessas regras são as seguintes:

- Excluir uma entidade de uma superclasse implica que ela seja automaticamente excluída de todas as subclasses às quais pertence.
- Inserir uma entidade em uma superclasse implica que a entidade seja obrigatoriamente inserida em todas as subclasses *definidas por predicado* (ou *definidas por atributo*) para as quais a entidade satisfaz o predicado de definição.
- Inserir uma entidade em uma superclasse de uma *especialização total* implica que a entidade é obrigatoriamente inserida em, pelo menos, uma das subclasses da especialização.

O leitor é encorajado a fazer uma lista completa das regras para inserções e exclusões para os vários tipos de especializações.

8.3.2 Hierarquias e reticulado da especialização e generalização

A própria subclasse pode ter mais subclasses especificadas nela, formando uma hierarquia ou um reticulado de especializações. Por exemplo, na Figura 8.6, ENGENHEIRO é uma subclasse de FUNCIONARIO e também uma superclasse de GERENTE_ENGENHEIRO. Isso representa a restrição do mundo real de que cada gerente engenheiro precisa ser um engenheiro. Uma **hierarquia de especialização** tem a restrição de que cada subclasse participa *como uma subclasse em apenas um* relacionamento de classe/subclasse; ou seja, cada subclasse tem apenas um pai, que resulta em uma **estrutura de árvore** ou **hierarquia estrita**. Ao contrário, para um **reticulado de especialização**, uma subclasse pode ser uma subclasse em *mais de um* relacionamento de classe/subclasse. Assim, a Figura 8.6 é um reticulado.

A Figura 8.7 mostra outro reticulado de especialização de mais de um nível. Isso pode ser parte de um esquema conceitual para um banco de dados UNIVERSIDADE. Observe que essa organização teria sido uma hierarquia, exceto para a subclasse ALUNO_COLABORADOR, que é uma subclasse em dois relacionamentos de classe/subclasse distintos.

Os requisitos para a parte do banco de dados UNIVERSIDADE mostrados na Figura 8.7 são os seguintes:

1. O banco de dados registra três tipos de pessoas: funcionários, ex-alunos e alunos. Uma pessoa pode pertencer a um, dois ou a todos esses três tipos. Cada pessoa tem um nome, CPF, sexo, endereço e data de nascimento.
2. Cada funcionário tem um salário, e existem três tipos de funcionários: docente, administrativo e aluno colaborador. Cada funcionário pertence a exatamente um desses tipos. Para cada ex-aluno, é mantido um registro do título ou dos títulos que ele obteve na universidade, incluindo o nome do título, o ano em que o título foi concedido e o curso em que se formou. Cada aluno tem um departamento principal.
3. Cada membro do corpo docente tem um nível, enquanto cada membro administrativo tem um cargo administrativo. Os alunos colaboradores são classificados ainda como colabora-

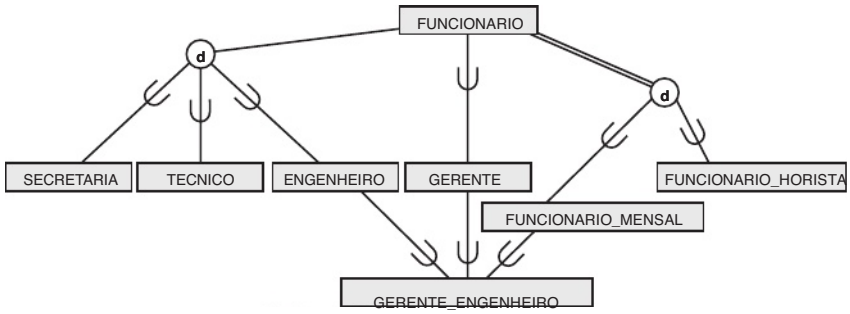


Figura 8.6
Um reticulado de especialização com a subclasse compartilhada GERENTE_ENGENHEIRO.

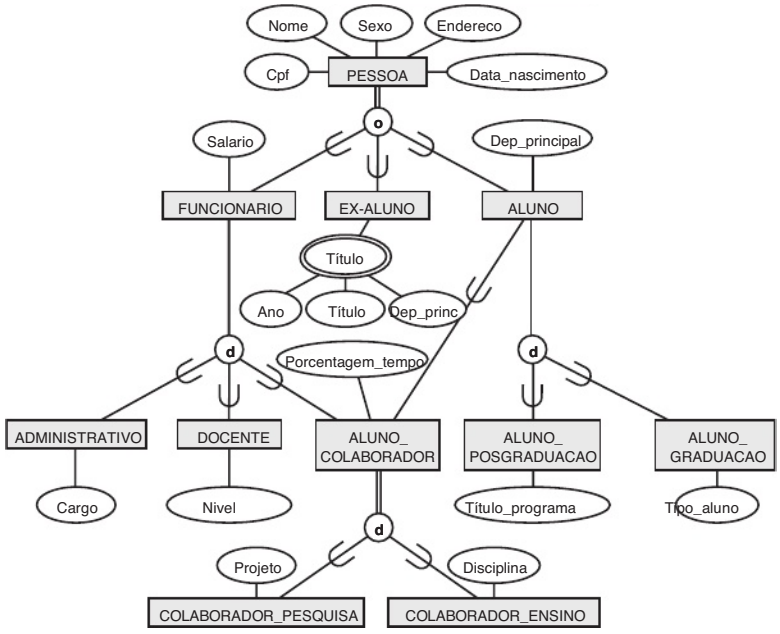


Figura 8.7
Um reticulado de especialização com herança múltipla para um banco de dados UNIVERSIDADE.

dores de pesquisa ou colaboradores de ensino, e a porcentagem de tempo em que eles trabalham é registrada no banco de dados. Os colaboradores de pesquisa têm seu projeto de pesquisa armazenado, ao passo que os colaboradores de ensino têm a disciplina atual em que trabalham.

4. Os alunos são classificados ainda como pós-graduação ou graduação, com os atributos específicos de título do programa (mestrado, doutorado, M.B.A. etc.) para alunos de pós-graduação formados e tipo de aluno (novato, segundo ano etc.) para os alunos de graduação.

Na Figura 8.7, todas as entidades de pessoa representadas no banco de dados são membros do tipo de entidade PESSOA, que é especializado nas subclasses

{FUNCIONARIO, EX-ALUNO, ALUNO}. Essa especialização é sobreposta; por exemplo, um ex-aluno também pode ser um funcionário e ainda pode ser um aluno buscando um título avançado. A subclasse ALUNO é a superclasse para a especialização {ALUNO_POSGRADUACAO, ALUNO_GRADUACAO}, enquanto FUNCIONARIO é a superclasse para a especialização {ALUNO_COLABORADOR, DOCENTE, ADMINISTRATIVO}. Observe que ALUNO_COLABORADOR também é uma subclasse de ALUNO. Finalmente, ALUNO_COLABORADOR é a superclasse para a especialização em {COLABORADOR_PESQUISA, COLABORADOR_ENSINO}.

Em tal reticulado ou hierarquia de especialização, uma subclasse herda os atributos não apenas de sua superclasse direta, mas também de todas as suas superclasses predecessoras *até chegar à raiz* da hierarquia ou reticulado, se necessário. Por exemplo, uma entidade em ALUNO_POSGRADUACAO herda todos os atributos dessa entidade como um ALUNO *e* como uma PESSOA. Observe que uma entidade pode existir em vários *nós folha* da hierarquia, em que um **nó folha** é uma classe que *não tem subclasses próprias*. Por exemplo, um membro de ALUNO_GRADUACAO também pode ser um membro de COLABORADOR_PESQUISA.

Uma subclasse com *mais de uma* superclasse é chamada de **subclasse compartilhada**, como GERENTE_ENGENHEIRO na Figura 8.6. Isso leva ao conceito conhecido como **herança múltipla**, na qual a subclasse compartilhada GERENTE_ENGENHEIRO herda diretamente atributos e relacionamentos de múltiplas classes. Observe que a existência de pelo menos uma subclasse compartilhada leva a um reticulado (e, portanto, à *herança múltipla*). Se não existisse qualquer subclasse compartilhada, teríamos uma hierarquia em vez de um reticulado, e somente a **herança simples** existiria. Uma regra importante relacionada à herança múltipla pode ser ilustrada pelo exemplo da subclasse compartilhada ALUNO_COLABORADOR na Figura 8.7, que herda atributos de FUNCIONARIO e ALUNO. Aqui, tanto FUNCIONARIO quanto ALUNO herdam *os mesmos atributos* de PESSOA. A regra declara que, se um atributo (ou relacionamento) originado na *mesma superclasse* (PESSOA) é herdado mais de uma vez por caminhos diferentes (FUNCIONARIO e ALUNO) no reticulado, então ele deverá ser incluído apenas uma vez na subclasse compartilhada (ALUNO_COLABORADOR). Logo, os atributos de PESSOA são herdados *apenas uma vez* na subclasse ALUNO_COLABORADOR da Figura 8.7.

É importante observar aqui que alguns modelos e linguagens são limitados à **herança simples** e *não permitem* a herança múltipla (subclasses compartilhadas). Também é importante observar que alguns modelos não permitem que uma entidade tenha tipos múltiplos, e, portanto, uma entidade pode ser membro *de apenas uma classe folha*.[8] Em tal modelo, é necessário criar subclasses adicionais como nós folha para cobrir todas as combinações possíveis de classes que podem ter alguma entidade que pertença a todas essas classes simultaneamente. Por exemplo, em uma especialização de sobreposição de PESSOA para {FUNCIONARIO, EX-ALUNO, ALUNO} (ou {F, E, A} para abreviar), seria necessário criar sete subclasses de PESSOA a fim de cobrir todos os tipos de entidades possíveis: F, E, A, F_E, F_A, E_A e F_E_A. Obviamente, isso pode gerar uma complexidade extra.

Embora tenhamos usado a especialização para ilustrar nossa discussão, conceitos semelhantes *se aplicam igualmente* à generalização, conforme mencionamos no início desta seção. Logo, também podemos falar de **hierarquias de generalização** e **reticulados de generalização.**

8.3.3 Utilizando especialização e generalização no refinamento de esquemas conceituais

Agora, vamos detalhar as diferenças entre os processos de especialização e generalização, e como eles são usados para refinar os esquemas conceituais durante o projeto conceitual do banco de dados. No processo de especialização, normalmente começamos com um tipo de entidade e depois definimos subclasses do tipo de entidade pela especialização sucessiva; ou seja, definimos repetidamente agrupamentos mais específicos do tipo de entidade. Por exemplo, ao projetar o reticulado de especialização da Figura 8.7, podemos primeiro especificar um tipo de entidade PESSOA para um banco de dados de universidade. Depois, descobriremos que três tipos de pessoas serão representados no banco de dados: funcionários da universidade, ex-alunos e alunos. Criamos a especialização {FUNCIONARIO, EX-ALUNO, ALUNO} para essa finalidade e escolhemos a restrição de sobreposição, pois uma pessoa pode pertencer a mais de uma das subclasses. Especializamos FUNCIONARIO ainda mais em {DOCENTE, ADMINISTRATIVO, ALUNO_COLABORADOR} e especializamos ALUNO em {ALUNO_GRADUACAO, ALUNO_POSGRADUACAO}. Por fim, especializamos ALUNO_COLABORADOR em {COLABORADOR_PESQUISA, COLABORADOR_ENSINO}. Essa especialização sucessiva corresponde a um **processo de refinamento conceitual de cima para baixo (top-down)** durante o projeto do esquema conceitual. Até aqui, temos uma hierarquia; depois, obser-

[8] Em alguns modelos, a classe é restringida ainda mais para ser um *nó folha* na hierarquia ou no reticulado.

vamos que ALUNO_COLABORADOR é uma subclasse compartilhada, pois ela também é uma subclasse de ALUNO, o que leva ao reticulado.

É possível chegar à mesma hierarquia ou reticulado vindo de outra direção. Nesse caso, o processo envolve a generalização, em vez da especialização, e corresponde a uma **síntese conceitual de baixo para cima (bottom-up)**. Por exemplo, os projetistas de banco de dados podem primeiro descobrir tipos de entidade como ADMINISTRATIVO, DOCENTE, EX-ALUNO, ALUNO_GRADUACAO, ALUNO_POSGRADUACAO, COLABORADOR_PESQUISA, COLABORADOR_ENSINO, e assim por diante; depois, eles podem generalizar {ALUNO_GRADUACAO, ALUNO_POSGRADUACAO} para ALUNO; daí generalizar {COLABORADOR_PESQUISA, COLABORADOR_ENSINO} para ALUNO_COLABORADOR; então generalizar {ADMINISTRATIVO, DOCENTE, ALUNO_COLABORADOR} para FUNCIONARIO; e, finalmente, generalizar {FUNCIONARIO, EX-ALUNO, ALUNO} para PESSOA.

Em termos estruturais, hierarquias ou reticulados resultantes de qualquer processo podem ser idênticas. A única diferença relaciona-se à maneira ou ordem em que as superclasses e subclasses do esquema foram criadas durante o processo de projeto. Na prática, é provável que nem o processo de generalização nem de especialização seja seguido estritamente, mas que seja empregada uma combinação dos dois processos. Novas classes são incorporadas de maneira contínua em uma hierarquia ou reticulado à medida que elas se tornam aparentes aos usuários e projetistas. Observe que a noção de representar dados e conhecimento usando hierarquias e reticulados de superclasse/subclasse é muito comum em sistemas baseados em conhecimento e sistemas especialistas, que combinam a tecnologia de banco de dados com técnicas de inteligência artificial. Por exemplo, esquemas de representação do conhecimento baseada em quadro são muito semelhantes às hierarquias de classes. A especialização também é comum nas metodologias de projeto de engenharia de softwa re que são baseadas no paradigma orientado a objeto.

8.4 Modelagem dos tipos UNIAO usando categorias

Todos os relacionamentos de superclasse/subclasse que vimos até aqui têm uma *única superclasse*. Uma subclasse compartilhada como GERENTE_ENGENHEIRO no reticulado da Figura 8.6 é a subclasse em três relacionamentos de superclasse/subclasse *distintos*, em que cada um dos três relacionamentos tem uma *única* superclasse. Porém, às vezes é necessário representar um único relacionamento de superclasse/subclasse com *mais de uma* superclasse, no qual as superclasses representam diferentes tipos de entidade. Nesse caso, a subclasse representará uma coleção de objetos que é um subconjunto da UNIAO de tipos de entidade distintos. Chamamos essa *subclasse* de **tipo de união** ou **categoria**.[9]

Por exemplo, suponha que tenhamos três tipos de entidade: PESSOA, BANCO e EMPRESA. Em um banco de dados para registro de veículo a motor, o proprietário de um veículo pode ser uma pessoa, um banco (mantendo uma alienação sobre um veículo) ou uma empresa. Precisamos criar uma classe (coleção de entidades) que inclui entidades de todos os três tipos para desempenhar o papel de *proprietário de veículo*. Uma categoria (tipo de união) PROPRIETARIO, que é uma *subclasse da UNIÃO* dos três conjuntos de entidades de EMPRESA, BANCO e PESSOA, pode ser criada para essa finalidade. Exibimos categorias no diagrama EER como mostra a Figura 8.8. As superclasses EMPRESA, BANCO e PESSOA são conectadas ao círculo com o símbolo ∪, que significa *operação de união de conjuntos*. Um arco com o símbolo de subconjunto conecta o círculo à categoria (subclasse) PROPRIETARIO. Se um predicado de definição for necessário, ele é exibido ao lado da linha da superclasse à qual o predicado se aplica. Na Figura 8.8, temos duas categorias: PROPRIETARIO, que é uma subclasse da união de PESSOA, BANCO e EMPRESA; e VEICULO_REGISTRADO, que é uma subclasse da união de CARRO e CAMINHAO.

Uma categoria tem duas ou mais superclasses que podem representar *tipos de entidade distintos*, enquanto outros relacionamentos de superclasse/subclasse sempre têm uma única superclasse. Para entender melhor a diferença, podemos comparar uma categoria, como PROPRIETARIO da Figura 8.8, com a subclasse compartilhada GERENTE_ENGENHEIRO da Figura 8.6. Esta última é uma subclasse de *cada uma* das três superclasses ENGENHEIRO, GERENTE e FUNCIONARIO_MENSAL, de modo que uma entidade que é um membro de GERENTE_ENGENHEIRO deva existir em *todas as três*. Isso representa a restrição de que um gerente de engenharia precisa ser um ENGENHEIRO, um GERENTE *e* um FUNCIONARIO_MENSAL; ou seja, GERENTE_ENGENHEIRO é um subconjunto da *interseção* das três classes (conjuntos de entidades). Por sua vez, uma categoria é um subconjunto da *união* de suas su-

[9] Nosso uso do termo *categoria* é baseado no modelo ECR (Entity-Category-Relationship, ou entidade-categoria-relacionamento) (Elmasri et al., 1985).

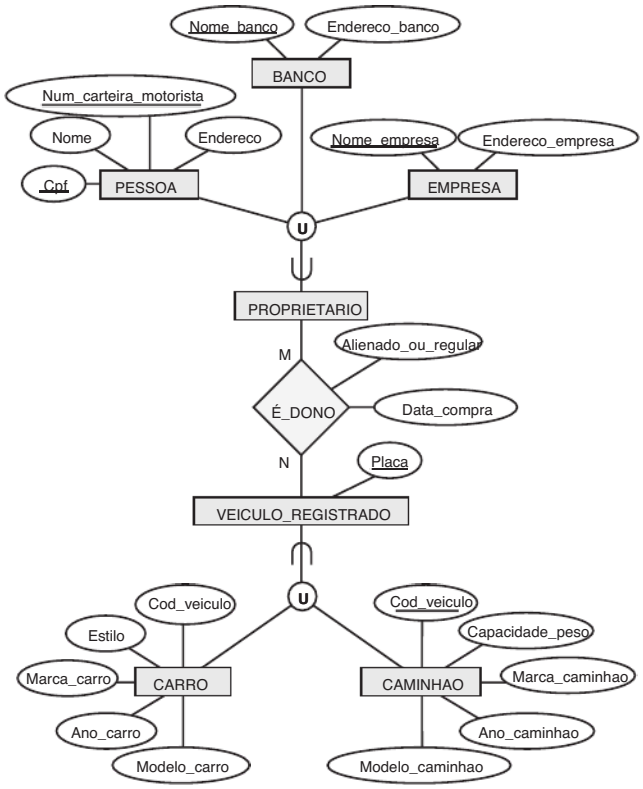


Figura 8.8
Duas categorias (tipos de união): PROPRIETARIO e VEICULO_REGISTRADO.

perclasses. Logo, uma entidade que é um membro de PROPRIETARIO deve existir em *apenas uma* das superclasses. Isso representa a restrição de que um PROPRIETARIO pode ser uma EMPRESA, um BANCO *ou* uma PESSOA na Figura 8.8.

A herança de atributo funciona de maneira mais seletiva no caso de categorias. Por exemplo, na Figura 8.8, cada entidade PROPRIETARIO herda os atributos de uma EMPRESA, uma PESSOA ou um BANCO, dependendo da superclasse à qual a entidade pertence. Por sua vez, uma subclasse compartilhada como GERENTE_ENGENHEIRO (Figura 8.6) herda *todos* os atributos de suas superclasses FUNCIONARIO_MENSAL, ENGENHEIRO e GERENTE.

É interessante observar a diferença entre a categoria VEICULO_REGISTRADO (Figura 8.8) e a superclasse generalizada VEICULO (Figura 8.3(b)). Na Figura 8.3(b), todo carro e todo caminhão é um VEICULO; mas, na Figura 8.8, a categoria VEICULO_REGISTRADO inclui alguns carros e alguns caminhões, mas não necessariamente todos eles (por exemplo, alguns carros ou caminhões podem não ser registrados). Em geral, uma especialização ou generalização como a da Figura 8.3(b), se fosse *parcial*, não impediria VEICULO de conter outros tipos de entidades, como motocicletas. Porém, uma categoria como VEICULO_REGISTRADO na Figura 8.8 implica que somente carros e caminhões, mas não outros tipos de entidades, possam ser membros de VEICULO_REGISTRADO.

Uma categoria pode ser **total** ou **parcial**. A total mantém a *união* de todas as entidades em suas superclasses, enquanto a parcial pode manter um *subconjunto da união*. Uma categoria total é represen-

tada em diagrama por uma linha dupla que conecta a categoria e o círculo, ao passo que uma categoria parcial é indicada por uma linha simples.

As superclasses de uma categoria podem ter diferentes atributos de chave, conforme demonstrado pela categoria PROPRIETARIO da Figura 8.8, ou podem ter o mesmo atributo de chave, conforme demonstrado pela categoria VEICULO_REGISTRADO. Observe que, se uma categoria é total (não parcial), ela pode ser representada alternativamente como uma especialização total (ou uma generalização total). Nesse caso, a escolha de qual representação usar é subjetiva. Se as duas classes representam o mesmo tipo de entidades e compartilham diversos atributos, incluindo os mesmos atributos-chave, a especialização/generalização é preferida; caso contrário, a categorização (tipo de união) é mais apropriada.

É importante observar que algumas metodologias de modelagem não possuem tipos de união. Nesses modelos, um tipo de união precisa ser representado de uma maneira indireta (ver Seção 9.2).

8.5 Um exemplo de esquema UNIVERSIDADE de EER, opções de projeto e definições formais

Nesta seção, primeiro vamos dar um exemplo de esquema de banco de dados no modelo EER para ilustrar o uso dos diversos conceitos discutidos aqui e no Capítulo 7. Depois, vamos discutir as escolhas de projeto para esquemas conceituais e, por fim, resumir os conceitos do modelo EER e defini-los formalmente da mesma maneira que fizemos com os conceitos do modelo ER básico, no Capítulo 7.

8.5.1 O exemplo do banco de dados UNIVERSIDADE

Para nosso exemplo de aplicação de banco de dados, considere um banco de dados UNIVERSIDADE que registra alunos e suas disciplinas, históricos e registro, bem como as ofertas de curso da universidade. O banco de dados também registra os projetos de pesquisa patrocinados do corpo docente e dos alunos de pós-graduação. Esse esquema aparece na Figura 8.9. Uma discussão dos requisitos que levaram a esse esquema pode ser vista em seguida.

Para cada pessoa, o banco de dados mantém informações sobre o nome dela [Nome], número do Cadastro de Pessoa Física [Cpf], endereço [Endereco], sexo [Sexo] e data de nascimento [Data_nasc]. Duas subclasses do tipo de entidade PESSOA são identificadas: DOCENTE e ALUNO. Atributos específicos de DOCENTE são a classificação [Classificacao] (assistente, associado, adjunto, pesquisador, visitante etc.), escritório [Doc_escritorio], telefone do escritório [Doc_telefone] e salário [Salario]. Todos os membros do corpo docente estão relacionados a departamento(s) acadêmico(s) ao(s) qual(is) eles estão afiliados [PERTENCE] (um membro do corpo docente pode ser associado a vários departamentos, de modo que o relacionamento é M:N). Um atributo específico de ALUNO é [Tipo_aluno] (novato = 1, segundo ano = 2, ..., aluno formado = 5). Cada ALUNO também está relacionado a seus departamentos principal e secundário (se forem conhecidos) [PRINCIPAL] e [SECUNDARIO], às turmas da disciplina que está frequentando atualmente e às disciplinas completadas [HISTORICO_ESCOLAR]. Cada instância de HISTORICO_ESCOLAR inclui a nota que o aluno recebeu [Nota] em uma turma de um curso.

ALUNO_POSGRADUACAO é uma subclasse de ALUNO, com o predicado de definição Tipo_aluno = 5. Para cada aluno de pós-graduação, mantemos uma lista dos títulos anteriores em um atributo composto, multivalorado [Titulos]. Também relacionamos o aluno formado a um orientador acadêmico [ORIENTADOR] e a um comitê de tese [BANCA], se existir.

Um departamento acadêmico tem os atributos nome [Dnome], telefone [Dtelefone] e número de escritório [Escritorio], e está relacionado ao membro acadêmico que é seu DIRETOR e à faculdade à qual pertence [DF]. Cada faculdade tem como atributos o nome da faculdade [Fnome], número do escritório [Fescritorio] e o nome de seu reitor [Reitor].

Uma disciplina tem os atributos núme ro da disciplina [Num_discip], nome da disciplina [Dnome] e descrição da disciplina [Ddesc]. São oferecidas várias turmas de cada disciplina, com cada uma tendo os atributos número da turma [Num_turma], o ano e trimestre em que foi oferecida ([Ano] e [Trim]).[10] Os números de turma identificam cada uma de maneira exclusiva. As turmas oferecidas durante o trimestre atual estão em uma subclasse TURMA_ATUAL de TURMA, com o predicado de definição Trim = Atual_trim e Ano = Atual_ano. Cada turma está relacionada ao professor que lecionou ou está lecionando ([ENSINA]), se ele estiver no banco de dados.

A categoria PROFESSOR_PESQUISADOR é um subconjunto da união de DOCENTE e ALUNO_POSGRADUACAO e inclui todos os docentes, bem como

[10] Consideramos que o sistema de *trimestre*, em vez de *semestre*, seja utilizado na universidade do exemplo.

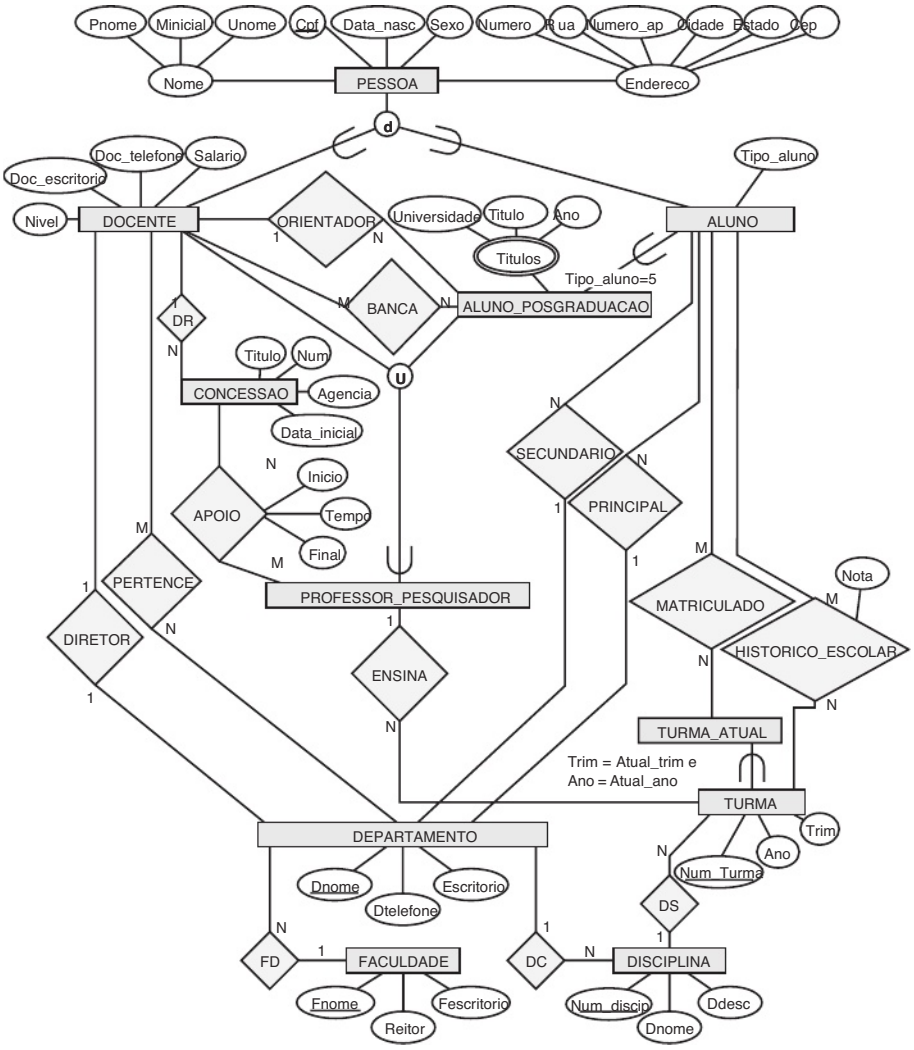


Figura 8.9
O esquema conceitual EER para um banco de dados UNIVERSIDADE.

alunos formados que recebem apoio por ensino ou pesquisa. Finalmente, o tipo de entidade CONCESSAO registra concessões e contratos de pesquisa outorgados à universidade. Cada concessão tem os atributos de título da concessão [Titulo], número da concessão [Num], agência de fomento [Agencia] e a data inicial [Data_inicial]. Uma concessão está relacionada a um docente responsável [DR] e a todos os pesquisadores a que ele dá apoio [APOIO]. Cada instância de apoio tem como atributos a data inicial do apoio [Inicio], a data final do apoio (se for conhecida) [Final] e a porcentagem do tempo gasto no projeto [Tempo] pelo pesquisador que recebe o apoio.

8.5.2 Escolhas de projeto para especialização/generalização

Nem sempre é fácil escolher o projeto conceitual mais apropriado para uma aplicação de banco de dados. Na Seção 7.7.3, apresentamos algumas das questões típicas enfrentadas por um projetista de banco de dados ao escolher entre os conceitos de tipos de entidade, tipos de relacionamento e atributos para representar uma situação em particular do minimundo como um esquema ER. Nesta seção, vamos discutir as diretrizes e escolhas de projeto para os conceitos EER de especialização/generalização e categorias (tipos de união).

Conforme mencionamos na Seção 7.7.3, o projeto conceitual do banco de dados deve ser considerado um processo de refinamento iterativo, até que o projeto mais adequado seja alcançado. As orientações a seguir ajudam a guiar o processo de projeto para conceitos de EER:

- Em geral, muitas especializações e subclasses podem ser definidas para tornar o modelo conceitual preciso. No entanto, a desvantagem é que o projeto se torna muito confuso. É importante representar apenas as subclasses que se julgue necessárias para evitar uma aglomeração extrema do esquema conceitual.
- Se uma subclasse possui poucos atributos específicos (locais) e nenhum relacionamento específico, ela pode ser mesclada à superclasse. Os atributos específicos manteriam valores NULL para entidades que não são membros da subclasse. Um atributo de *tipo* poderia especificar se uma entidade é um membro da subclasse.
- De modo semelhante, se todas as subclasses da especialização/generalização tiverem alguns atributos específicos e nenhum relacionamento específico, elas podem ser mescladas à superclasse e substituídas por um ou mais atributos de *tipo* que especificam a subclasse ou subclasses a que cada entidade pertence (ver Seção 9.2 para saber como esse critério se aplica aos bancos de dados relacionais).
- Os tipos de união e categorias geralmente devem ser evitados, a menos que a situação definitivamente justifique esse tipo de construção, o que ocorre em algumas situações práticas. Se possível, tentamos modelar usando a especialização/generalização conforme discutimos no final da Seção 8.4.
- A escolha de restrições disjuntas/sobrepostas e totais/parciais sobre a especialização/generalização é controlada pelas regras no minimundo que está sendo modelado. Se os requisitos não indicarem quaisquer restrições em particular, o padrão geralmente seria sobreposição e parcial, pois isso não especifica quaisquer restrições sobre a condição de membro da subclasse.

Como um exemplo da aplicação dessas orientações, considere a Figura 8.6, na qual nenhum atributo específico (local) aparece. Poderíamos mesclar todas as subclasses no tipo de entidade FUNCIONARIO e acrescentar os seguintes atributos a ele:

- Um atributo Tipo_emprego cujo conjunto de valores {'Secretária', 'Engenheiro', 'Técnico'} indicaria a qual subclasse cada funcionário pertence na primeira especialização.
- Um atributo Forma_pagto cujo conjunto de valores {'Mensal', 'Horista'} indicaria a qual subclasse cada funcionário pertence na segunda especialização.
- Um atributo É_gerente cujo conjunto de valores {'Sim', 'Não'} indicaria se uma entidade de funcionário individual é um gerente ou não.

8.5.3 Definições formais para os conceitos do modelo EER

Agora, vamos resumir os conceitos do modelo EER e mostrar as definições formais. Uma **classe**[11] é um conjunto ou coleção de entidades, incluindo qualquer uma das construções de esquema EER das entidades de grupo, como tipos de entidade, subclasses, superclasses e categorias. Uma **subclasse** *S* é uma classe cujas entidades sempre precisam ser um subconjunto das entidades em outra classe, chamada **superclasse** *C* do **relacionamento superclasse/subclasse** (ou **É_UM**). Indicamos tal relacionamento com *C*/*S*. Para tal relacionamento superclasse/subclasse, sempre devemos ter

$$S \subseteq C$$

Uma **especialização** $Z = \{S_1, S_2, ..., S_n\}$ é um conjunto de subclasses que têm a mesma superclasse *G*;

[11] O uso da palavra *classe* aqui difere de sua utilização mais comum nas linguagens de programação orientadas a objeto, como C++. Em C++, uma classe é uma definição de tipo estruturada com suas funções (operações) aplicáveis.

ou seja, G/S_i é um relacionamento superclasse/subclasse para $i = 1, 2, ..., n$. G é chamado de **tipo de entidade generalizada** (ou a **superclasse** da especialização, ou uma **generalização** das subclasses $\{S_1, S_2, ..., S_n\}$). Z é considerada **total** se sempre (em qualquer ponto no tempo) tivermos

$$\bigcup_{i=1}^{n} S_i = G$$

Caso contrário, Z é considerada **parcial**. Z é considerada **disjunta** se sempre tivermos

$$S_i \cap S_j = \varnothing \text{ (conjunto vazio) para } i \neq j$$

Caso contrário, Z é considerada **sobreposta**.

Uma subclasse S de C é considerada **definida por predicado** se um predicado p sobre os atributos de C for usado para especificar quais entidades em C são membros de S; ou seja, $S = C[p]$, que é o conjunto de entidades em C que satisfazem a p. Uma subclasse que não é definida por um predicado é chamada de **definida pelo usuário**.

Uma especialização Z (ou generalização G) é considerada **definida por atributo** se um predicado $(A = c_i)$, onde A é um atributo de G e c_i é um valor constante do domínio de A, for usado para especificar a condição de membro em cada subclasse S_i em Z. Observe que, se $c_i \neq c_j$ para $i \neq j$, e A for um atributo de único valor, então a especialização será disjunta.

A **categoria** T é uma classe que é um subconjunto da união de n que define as superclasses D_1, D_2, ..., D_n, $n > 1$, podendo ser especificada formalmente da seguinte maneira:

$$T \subseteq (D_1 \cup D_2 \cup \cdots \cup D_n)$$

Um predicado p_i sobre os atributos de D_i pode ser usado para especificar os membros de cada D_i que são membros de T. Se um predicado for especificado sobre cada D_i, obtemos

$$T = (D_1[p_1] \cup D_2[p_2] \cup \cdots \cup D_n[p_n])$$

Agora, devemos estender a definição de **tipo de relacionamento** dada no Capítulo 7, permitindo que qualquer classe — não apenas um tipo de entidade — participe de um relacionamento. Logo, devemos substituir as palavras *tipo de entidade* por *classe* naquela definição. A notação gráfica de EER é coerente com ER porque todas as classes são representadas por retângulos.

8.6 Exemplo de outra notação: representando especialização e generalização em diagramas de classes em UML

Agora, vamos discutir as notações UML para especialização/generalização e herança. Já apresentamos a notação e terminologia básica do diagrama de classes UML na Seção 7.8. A Figura 8.10 ilustra um possível diagrama de classes UML correspondente ao diagrama EER da Figura 8.7.

A notação básica para especialização/generalização (ver Figura 8.10) é conectar as subclasses por linhas verticais a uma linha horizontal, que tem um triângulo conectando a linha horizontal por meio de outra linha vertical até a superclasse. Um triângulo preto indica uma especialização/generalização com a restrição *disjunta*, e um triângulo contornado indica uma restrição de *sobreposição*. A superclasse raiz é chamada de **classe base**, e as subclasses (nós folhas) são chamadas de **classes de folha**.

A discussão citada e o exemplo da Figura 8.10, junto com a apresentação da Seção 7.8, oferecem uma rápida introdução aos diagramas de classes UML e sua terminologia. Focalizamos os conceitos que são relevantes à modelagem de banco de dados ER e EER, em vez dos conceitos que são mais relevantes à engenharia de software. Em UML, existem muitos detalhes que não discutimos, pois estão fora do escopo deste livro e são relevantes principalmente para a engenharia de software. Por exemplo, as classes podem ser de vários tipos:

- Classes abstratas definem atributos e operações, mas não têm objetos correspondentes a essas classes. Elas são usadas principalmente para especificar um conjunto de atributos e operações que podem ser herdados.
- Classes concretas podem ter objetos (entidades) instanciados para pertencer à classe.
- Classes genéricas (ou *template*) especificam um modelo que pode ser usado também para definir outras classes.

No projeto de banco de dados estamos preocupados principalmente com a especificação de classes concretas, cujas coleções de objetos são permanentemente (ou persistentemente) armazenadas no banco de dados. A bibliografia selecionada ao final deste capítulo oferece algumas referências a livros que descrevem os detalhes completos de UML. Há um material adicional relacionado à UML incluído no Capítulo 10.

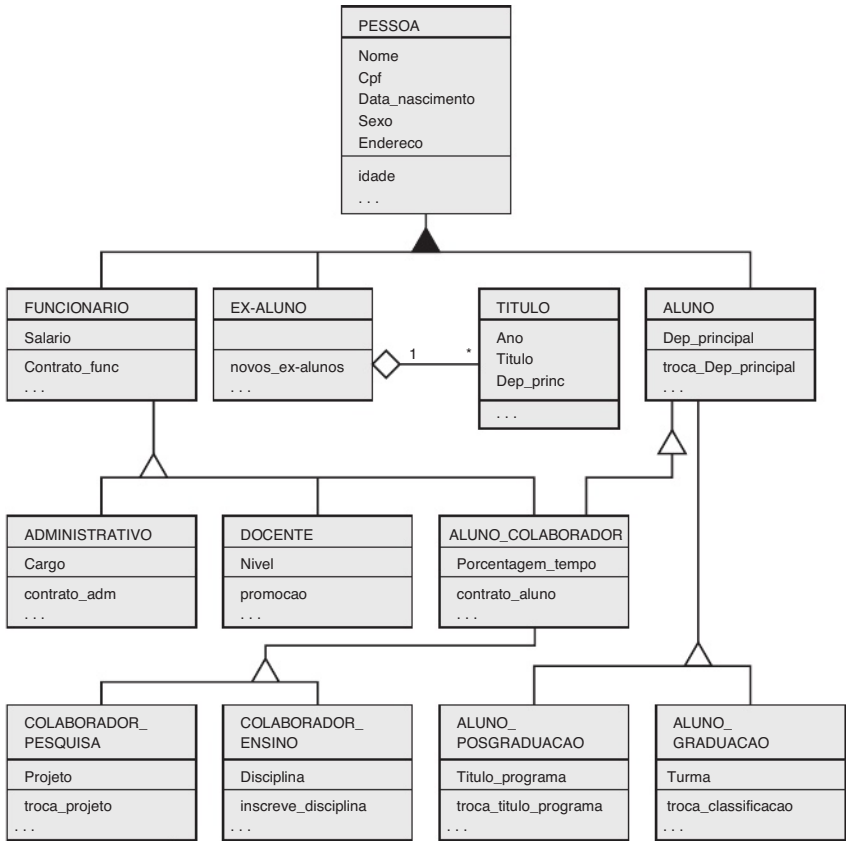


Figura 8.10
Um diagrama de classes UML correspondente ao diagrama EER da Figura 8.7, ilustrando a notação UML para especialização/generalização.

8.7 Conceitos de abstração de dados, representação do conhecimento e ontologia

Nesta seção, discutimos em termos gerais alguns dos conceitos de modelagem que descrevemos bem especificamente em nossa apresentação dos modelos ER e EER no Capítulo 7 e anteriormente neste capítulo. Essa terminologia não é usada apenas na modelagem de dados conceituais, mas também na literatura de inteligência artificial quando se discute a **representação do conhecimento** (ou **RC**, *knowledge representation*). Esta seção discute as semelhanças e diferenças entre a modelagem conceitual e a representação do conhecimento, além de introduzir uma terminologia alternativa e de alguns conceitos adicionais.

O objetivo das técnicas de RC é desenvolver conceitos para modelar com precisão algum **domínio de conhecimento**, criando uma **ontologia**[12] que descreve os conceitos do domínio e como esses conceitos estão inter-relacionados. Tal ontologia é usada para arma-

[12] Uma *ontologia* é algo semelhante a um esquema conceitual, mas com mais conhecimento, regras e exceções.

zenar e manipular o conhecimento para tirar conclusões, tomar decisões ou responder a perguntas. Os objetivos da RC são semelhantes aos dos modelos de dados semânticos, mas existem algumas semelhanças e diferenças importantes entre as duas disciplinas:

- Ambas as disciplinas usam um processo de abstração para identificar propriedades comuns e aspectos importantes de objetos no minimundo (também conhecido como *domínio de discurso* em RC), enquanto suprimem diferenças insignificantes e detalhes sem importância.
- As duas disciplinas oferecem conceitos, relacionamentos, restrições, operações e linguagens para definir dados e representar conhecimento.
- A RC geralmente é mais ampla em escopo do que os modelos de dados semânticos. Diferentes formas de conhecimento, como regras (usadas na inferência, dedução e pesquisa), conhecimento incompleto e padrão, e conhecimento temporal e espacial, são representados em esquemas RC. Os modelos de banco de dados estão sendo expandidos para incluir alguns desses conceitos (ver Capítulo 26).
- Esquemas RC incluem **mecanismos de raciocínio** que deduzem fatos adicionais dos fatos armazenados em um banco de dados. Logo, embora a maioria dos sistemas de banco de dados atuais seja limitada a responder a consultas diretas, os sistemas baseados em conhecimento que usam esquemas RC podem responder a consultas que envolvem **inferências** sobre os dados armazenados. A tecnologia de banco de dados está sendo estendida com mecanismos de inferência (ver Seção 26.5).
- Embora a maioria dos modelos de dados se concentre na representação dos esquemas de banco de dados, ou metaconhecimento, os esquemas RC costumam misturar os esquemas com as próprias instâncias, a fim de oferecer flexibilidade na representação de exceções. Isso normalmente resulta em ineficiências quando esses esquemas RC são implementados, especialmente quando comparados com bancos de dados e quando uma grande quantidade de dados (fatos) precisa ser armazenada.

Agora, vamos discutir sobre quatro **conceitos de abstração** que são usados em modelos de dados semânticos, como o modelo EER, bem como em esquemas RC: (1) classificação e instanciação, (2) identificação, (3) especialização e generalização e (4) agregação e associação. Os conceitos emparelhados de classificação e instanciação são inversos um do outro, assim como a generalização e a especialização. Os conceitos de agregação e associação também são relacionados. Discutimos esses conceitos abstratos e sua relação com as representações concretas usadas no modelo EER para esclarecer o processo de abstração de dados e melhorar nosso conhecimento do processo relacionado de projeto de esquema conceitual. Fechamos a seção com uma rápida discussão sobre *ontologia*, que está sendo bastante usada na pesquisa recente sobre representação do conhecimento.

8.7.1 Classificação e instanciação

O processo de **classificação** envolve atribuir sistematicamente objetos/entidades semelhantes aos tipos classe/entidade do objeto. Agora, podemos descrever (em BD) ou raciocinar sobre (em RC) as classes em vez dos objetos individuais. Coleções de objetos que compartilham os mesmos tipos de atributos, relacionamentos e restrições são classificadas em classes, a fim de simplificar o processo de descoberta de suas propriedades. A **instanciação** é o inverso da classificação, e refere-se à geração e exame específico de objetos distintos de uma classe. Uma instância de objeto está relacionada à sua classe de objeto por um relacionamento **É_UMA_INSTÂNCIA_DE** ou **É_UM_MEMBRO_DE**. Embora os diagramas EER não apresentem instâncias, os diagramas UML permitem uma forma de instanciação ao possibilitar a exibição de objetos individuais. *Não* vamos descrever esse recurso em nossa introdução aos diagramas de classes UML.

Em geral, os objetos de uma classe devem ter uma estrutura de tipo semelhante. Contudo, alguns objetos podem exibir propriedades que diferem em alguns aspectos dos outros objetos da classe. Esses **objetos de exceção** também precisam ser modelados, e os esquemas RC permitem exceções mais variadas do que os modelos de banco de dados. Além disso, certas propriedades se aplicam à classe como um todo, e não aos objetos individuais; os esquemas RC permitem tais **propriedades de classe**. Os diagramas UML também permitem a especificação de propriedades de classe.

No modelo EER, as entidades são classificadas em tipos de entidade de acordo com seus atributos e relacionamentos básicos. As entidades são classificadas ainda em subclasses e categorias, com base nas semelhanças e diferenças adicionais (exceções) entre elas. As instâncias de relacionamento são classificadas em tipos de relacionamento. Logo, os tipos de entidade, subclasses, categorias e tipos de relacionamento são os diferentes conceitos usados para

classificação no modelo EER. O modelo EER não provê explicitamente propriedades de classificação, mas pode ser estendido para fazer isso. Em UML, os objetos são classificados, e é possível exibir tanto propriedades de classe quanto objetos individuais.

Os modelos de representação do conhecimento permitem múltiplos esquemas de classificação, em que uma classe é uma *instância* de outra classe (chamada **metaclasse**). Observe que isso *não pode* ser representado diretamente no modelo EER, pois temos apenas dois níveis — classes e instâncias. O único relacionamento entre classes no modelo EER é um relacionamento de superclasse/subclasse, ao passo que em alguns esquemas RC um relacionamento adicional de classe/instância pode ser representado diretamente em uma hierarquia de classes. Uma instância pode, por si só, ser outra classe, permitindo esquemas de classificação multiníveis.

8.7.2 Identificação

Identificação é o processo de abstração pelo qual classes e objetos se tornam exclusivamente identificáveis por meio de algum **identificador**. Por exemplo, um nome de classe identifica de maneira exclusiva uma classe inteira dentro de um esquema. É necessário que haja um mecanismo adicional para distinguir instâncias de objeto distintas por meio de identificadores de objeto. Além disso, é necessário identificar múltiplas manifestações no banco de dados do mesmo objeto no mundo real. Por exemplo, podemos ter uma tupla <'Mauro Campos', '610618', '3376-9821'> em uma relação PESSOA e outra tupla <'301-540-836-51', 'CC', 3,8> em uma relação ALUNO que representem a mesma entidade do mundo real. Não há como identificar o fato de que esses dois objetos de banco de dados (tuplas) representam a mesma entidade do mundo real, a menos que tomemos uma providência *em tempo de projeto* para a referência cruzada apropriada, que fornece essa identificação. Logo, a identificação é necessária em dois níveis:

- Para distinguir objetos de classes de banco de dados.
- Para identificar objetos de banco de dados e relacioná-los a seus equivalentes no mundo real.

No modelo EER, a identificação das construções de esquema é baseada em um sistema de nomes exclusivos para tais construções. Por exemplo, cada classe em um esquema EER — seja um tipo de entidade, uma subclasse, uma categoria ou um tipo de relacionamento — precisa ter um nome distinto. Os nomes de atributos de determinada classe também precisam ser distintos. As regras para identificar referências de nome de atributo sem ambiguidade em um reticulado ou hierarquia de especialização ou generalização também são necessárias.

No nível de objeto, os valores dos atributos-chave são usados para distinguir entre entidades de um tipo em particular. Para tipos de entidade fraca, as entidades são identificadas por uma combinação de valores próprios de chave parcial e aquelas às quais estão relacionadas, dependendo da razão de cardinalidade especificada.

8.7.3 Especialização e generalização

Especialização é o processo de classificar uma classe de objetos em subclasses mais especializadas. **Generalização** é o processo inverso de generalizar várias classes em uma classe abstrata de mais alto nível, que inclua os objetos em todas essas classes. A especialização é o refinamento conceitual, enquanto a generalização é a síntese conceitual. Subclasses são usadas no modelo EER para representar a especialização e a generalização. Chamamos o relacionamento entre uma subclasse e suas superclasses de relacionamento **É_UMA_SUBCLASSE_DE**, ou simplesmente um relacionamento **É_UM**. Trata-se do mesmo relacionamento **É_UM** discutido anteriormente, na Seção 8.5.3.

8.7.4 Agregação e associação

Agregação é um conceito de abstração para a criação de objetos compostos com base em seus objetos componentes. Existem três casos em que esse conceito pode estar relacionado ao modelo EER. O primeiro caso é a situação em que agregamos valores de atributo de um objeto para formar o objeto total. O segundo caso é quando representamos um relacionamento de agregação como um relacionamento comum. O terceiro caso, que o modelo EER não determina explicitamente, envolve a possibilidade de combinar objetos que são relacionados por uma instância de relacionamento em particular a um *objeto agregado de nível superior*. Isso às vezes é útil quando o próprio objeto de agregação de nível mais alto tem de estar relacionado a outro objeto. Chamamos o relacionamento entre os objetos primitivos e seu objeto de agregação **É_UMA_PARTE_DE** e o inverso é chamado de **É_UM_COMPONENTE_DE**. A UML possibilita todos os três tipos de agregação.

A abstração da **associação** é usada para associar objetos de várias *classes independentes*. Assim, às vezes ela é semelhante ao segundo uso da agregação, representada no modelo EER por tipos de relacionamento, e em UML, por associações. Esse relacionamento abstrato é chamado de **É_ASSOCIADO_A**.

Para entender melhor os diferentes usos da agregação, considere o esquema ER mostrado na Figura 8.11(a), que armazena informações sobre entrevistas por candidatos a emprego para várias empresas. A classe EMPRESA é uma agregação dos atributos (ou objetos componentes) Enome (nome de empresa) e Eendereco (endereço da empresa), enquanto CANDIDATO é uma agregação de Cpf, Nome, Endereco e Telefone. Os atributos de relacionamento Nome_resp e Telefone_resp representam o nome e o número de telefone da pessoa na empresa que é responsável pela entrevista. Suponha que algumas entrevistas sejam ofertas de emprego, ao passo que outras não. Gostaríamos de tratar ENTREVISTA como uma classe para associá-la a OFERTA_EMPREGO. O esquema mostrado na Figura 8.11(b) está *incorreto* porque requer que cada instância de relacionamento de entrevista tenha uma oferta de emprego. O esquema mostrado na Figura 8.11(c) *não é permitido* porque o modelo ER não permite relacionamentos entre relacionamentos.

Uma forma de representar essa situação é criar uma classe agregada de nível mais alto, composta de EMPRESA, CANDIDATO e ENTREVISTA, e relacioná-la a OFERTA_EMPREGO, como mostra a Figura 8.11(d). Embora o modelo EER, conforme descrito neste livro, não tenha essa facilidade, alguns modelos de dados semânticos o permitem, e chamam o objeto resultante de **objeto composto** ou **molecular**. Outros modelos tratam tipos de entidade e tipos de relacionamento de maneira uniforme e, portanto, permitem relacionamentos entre relacionamentos, conforme ilustrado pela Figura 8.11(c).

Para representar essa situação corretamente no modelo ER descrito aqui, precisamos criar um novo tipo de entidade fraca ENTREVISTA, como mostra a Figura 8.11(e), e relacioná-lo a OFERTA_EMPREGO. Logo, sempre podemos representar essas situações de modo correto no modelo ER criando tipos de entidade adicionais, embora possa ser conceitualmente mais desejável permitir a representação direta da agregação, como na Figura 8.11(d), ou permitir relacionamentos entre relacionamentos, como na Figura 8.11(c).

A distinção estrutural principal entre agregação e associação é que, quando uma instância de associação é excluída, os objetos participantes podem continuar a existir. Porém, se definirmos suporte à noção de um objeto de agregação — por exemplo, um CARRO que é composto dos objetos MOTOR, CHASSI e PNEUS —, então a exclusão do objeto de agregação CARRO corresponde a excluir todos os seus objetos componentes.

8.7.5 Ontologias e a Web semântica

Nos últimos anos, a quantidade de dados e informações computadorizadas disponíveis na Web se tornou algo fora de controle. Muitos modelos e formatos diferentes são utilizados. Além dos modelos de banco de dados que apresentamos neste livro, muita informação é armazenada na forma de **documentos**, que possuem consideravelmente muito menos estrutura do que a informação do banco de dados. Um projeto em andamento, que está tentando permitir a troca de informações entre computadores na Web, é chamado de **Web semântica**, que tenta criar modelos de representação do conhecimento que sejam bastante genéricos, a fim de permitir a troca e a pesquisa de informações significativas entre máquinas. O conceito de *ontologia* é considerado a base mais promissora para alcançar os objetivos da Web semântica e está bastante relacionado à representação do conhecimento. Nesta seção, oferecemos uma rápida introdução ao que é a ontologia e como ela pode ser usada como uma base para automatizar o conhecimento, a busca e troca de informações.

O estudo das ontologias tenta descrever as estruturas e os relacionamentos possíveis na realidade por meio de um vocabulário comum. Portanto, ele pode ser considerado um meio para descrever o conhecimento de certa comunidade sobre a realidade. A ontologia srcinou-se nas áreas de filosofia e metafísica. Uma definição muito usada de **ontologia** é *uma especificação* de uma *conceitualização*.[13]

Nessa definição, uma **conceitualização** é o conjunto de conceitos usados para representar a parte da realidade ou conhecimento que é de interesse de uma comunidade de usuários. **Especificação** refere-se à linguagem e termos do vocabulário usados para especificar a conceitualização. A ontologia inclui tanto a *especificação* quanto a *conceitualização*. Por exemplo, a mesma conceitualização pode ser especificada em duas linguagens diferentes, gerando duas ontologias separadas. Com base nessa definição bastante geral, não existe consenso sobre o que é exatamente uma ontologia. Algumas maneiras possíveis de descrever as ontologias são as seguintes:

- Um ***thesaurus*** (ou ainda um **dicionário** ou um **glossário** de termos) descreve os relacionamentos entre palavras (vocabulário) que representam diversos conceitos.
- Uma **taxonomia** descreve como os conceitos de determinada área do conhecimento são relacionados usando estruturas semelhantes àquelas utilizadas em uma especialização ou generalização.

[13] Essa definição é dada em Gruber (1995).

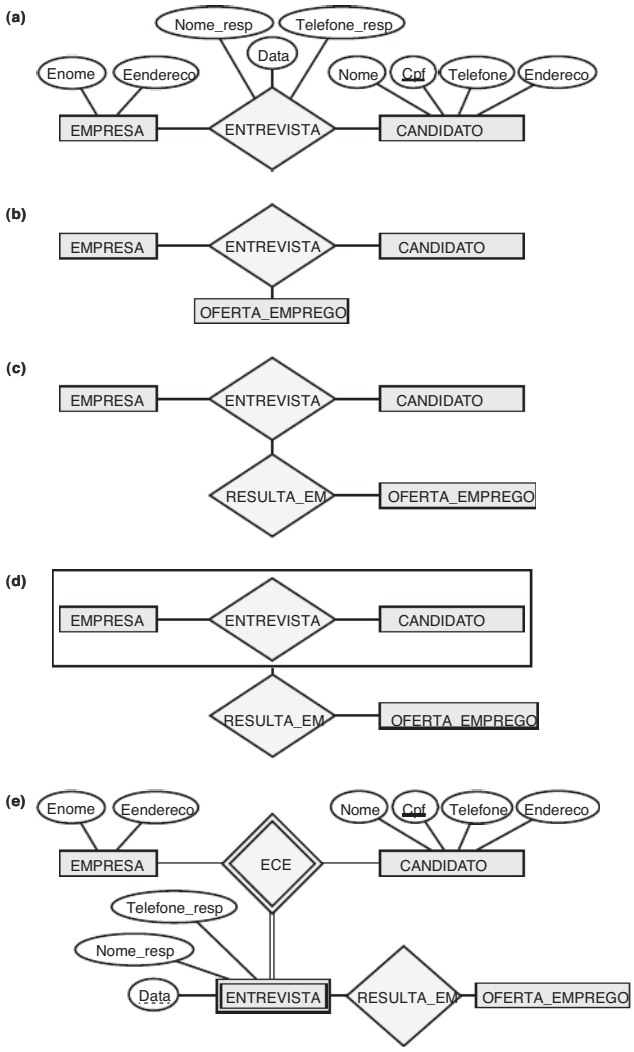


Figura 8.11

Agregação. (a) O tipo de relacionamento ENTREVISTA. (b) Incluindo OFERTA_EMPREGO em um tipo de relacionamento ternário (incorreto). (c) Fazendo o relacionamento RESULTA_EM participar de outros relacionamentos (não permitido em ER). (d) Usando agregação e um objeto composto (molecular) (geralmente, não permitido em ER, mas permitido por algumas ferramentas de modelagem). (e) Representação correta em ER.

- Um **esquema de banco de dados** detalhado é considerado por alguns uma ontologia que descreve os conceitos (entidades e atributos) e relacionamentos de um minimundo da realidade.
- Uma **teoria lógica** usa conceitos da lógica matemática para tentar definir conceitos e seus inter-relacionamentos.

Normalmente, os conceitos usados para descrever ontologias são muito semelhantes aos conceitos que discutimos na modelagem conceitual, como entidades, atributos, relacionamentos, especializações, e assim por diante. A principal diferença entre uma ontologia e, digamos, um esquema de banco de dados é que o esquema em geral está limitado a descrever um pequeno subconjunto de um minimundo da realidade a fim de armazenar e gerenciar dados. Uma ontologia costuma ser considerada mais geral no sentido de tentar descrever uma parte da realidade ou um domínio de interesse (por exemplo, termos médicos, aplicações de comércio, por exemplo, esportes etc.) o mais completamente possível.

Resumo

Neste capítulo, discutimos as extensões ao modelo ER que melhoram suas capacidades de representação. Chamamos o modelo resultante de modelo ER estendido ou EER. Apresentamos o conceito de uma subclasse e sua superclasse e o mecanismo relacionado de herança de atributo/relacionamento. Vimos como às vezes é necessário criar classes de entidades adicionais, seja por causa dos atributos específicos adicionais ou tipos de relacionamento específicos. Discutimos dois processos principais para definir hierarquias e reticulados de superclasse/subclasse: especialização e generalização.

Em seguida, mostramos como apresentar essas novas construções em um diagrama EER. Também discutimos os diversos tipos de restrições que podem se aplicar à especialização ou generalização. As duas principais restrições são total/parcial e disjunta/sobreposta. Além disso, um predicado de definição para uma subclasse ou um atributo de definição para uma especialização podem ser especificados. Abordamos as diferenças entre subclasses definidas pelo usuário, por predicado e entre especializações definidas pelo usuário e por atributo. Por fim, discutimos o conceito de uma categoria ou tipo de união, que é um subconjunto da união de duas ou mais classes, e mostramos as definições formais de todos os conceitos apresentados.

Apresentamos parte da notação e terminologia da UML para representar a especialização e a generalização. Na Seção 8.7, discutimos rapidamente a disciplina de representação do conhecimento (RC) e como ela está relacionada à modelagem de dados semântica. Também demos uma visão geral e um caso de dois sistemas RC baseados em lógica: classificação e instanciação, identificação, especialização e generalização, e agregação e associação. Vimos como os conceitos de EER e UML estão relacionados a cada um deles.

Perguntas de revisão

8.1. O que é uma subclasse? Quando uma subclasse é necessária na modelagem de dados?

8.2. Defina os seguintes termos: *superclasse de uma subclasse*, *relacionamento de superclasse/subclasse*, *relacionamento É_UM*, *especialização*, *generalização*, *categoria*, *atributos específicos (locais)* e *relacionamentos específicos*.

8.3. Discuta o mecanismo de herança de atributo/relacionamento. De que forma ele é útil?

8.4. Discuta as subclasses definidas pelo usuário e definidas por predicado, e identifique as diferenças entre as duas.

8.5. Discuta as especializações definidas pelo usuário e definidas por atributo, e identifique as diferenças entre as duas.

8.6. Discuta os dois tipos principais de restrições sobre especializações e generalizações.

8.7. Qual é a diferença entre uma hierarquia de especialização e um reticulado de especialização?

8.8. Qual é a diferença entre especialização e generalização? Por que não exibimos essa diferença nos diagramas de esquema?

8.9. Como uma categoria difere de uma subclasse compartilhada regular? Para que uma categoria é usada? Ilustre sua resposta com exemplos.

8.10. Para cada um dos seguintes termos da UML (ver seções 7.8 e 8.6), discuta o termo correspondente no modelo EER, se houver: *objeto*, *classe*, *associação*, *agregação*, *generalização*, *multiplicidade*, *atributos*, *discriminador*, *ligação*, *atributo de ligação*, *associação reflexiva* e *associação qualificada*.

8.11. Discuta as principais diferenças entre a notação para diagramas de esquema EER e diagramas de classe UML comparando como os conceitos comuns são representados em cada um.

8.12. Liste os diversos conceitos de abstração de dados e os conceitos de modelagem correspondentes no modelo EER.

8.13. Que recurso de agregação está faltando do modelo EER? Como o modelo EER pode ser melhorado para dar suporte a esse recurso?

8.14. Quais são as principais semelhanças e diferenças entre as técnicas conceituais de modelagem de banco de dados e as técnicas de representação do conhecimento?

8.15. Discuta as semelhanças e diferenças entre uma ontologia e um esquema de banco de dados.

Exercícios

8.16. Projete um esquema EER para uma aplicação de banco de dados em que você está interessado. Especifique todas as restrições que devem ser mantidas no banco de dados. Cuide para que o esquema tenha pelo menos cinco tipos de entidade, quatro tipos de relacionamento, um tipo de entidade fraca, um relacionamento de superclas-

se/subclasse, uma categoria e um tipo de relacionamento n-ário ($n > 2$).

8.17. Considere o esquema ER BANCO da Figura 7.21 e suponha que seja necessário registrar diferentes tipos de CONTAS (CONTA_POUPANCA, CONTA_CORRENTE, ...) e EMPRESTIMOS (EMPRESTIMO_CARRO, EMPRESTIMO_HABITACAO, ...). Suponha que também se deseje registrar cada uma das TRANSACOES de CONTA (depósitos, saques, cheques, ...) e os PAGAMENTOS de EMPRESTIMO; ambos incluem o valor, data e hora. Modifique o esquema BANCO, usando os conceitos de ER e EER de especialização e generalização. Indique quaisquer suposições que você fizer sobre os requisitos adicionais.

8.18. A narrativa a seguir descreve uma versão simplificada da organização das instalações olímpicas planejadas para os Jogos Olímpicos de verão. Desenhe um diagrama EER que mostre os tipos de entidade, atributos, relacionamentos e especializações para essa aplicação. Indique quaisquer suposições que você fizer. As instalações olímpicas são divididas em complexos esportivos. Os complexos esportivos são divididos em tipos de *um esporte* e *poliesportivo*. Os complexos poliesportivos possuem áreas designadas para cada esporte com um indicador de localização (por exemplo, centro, canto NE, e assim por diante). Um complexo tem um local, organizador, chefe, área total ocupada, e assim por diante. Cada complexo mantém uma série de eventos (por exemplo, o estádio com raias pode englobar muitas corridas diferentes). Para cada evento, existe uma data planejada, duração, número de participantes, número de oficiais e assim por diante. Uma relação de todos os oficiais será mantida junto com a lista dos eventos em que cada oficial estará envolvido. Diferentes equipamentos são necessários para os eventos (por exemplo, balizas, postes, barras paralelas), bem como para a manutenção. Os dois tipos de instalações (um esporte e poliesportivo) terão diferentes tipos de informação. Para cada tipo, o número de instalações necessárias é mantido, junto com um orçamento aproximado.

8.19. Identifique todos os conceitos importantes representados no estudo de caso do banco de dados de biblioteca descrito a seguir. Em particular, identifique as abstrações de classificação (tipos de entidade e tipos de relacionamento), agregação, identificação e especialização/generalização. Especifique as restrições de cardinalidade (min, max) sempre que possível. Liste detalhes que afetarão o eventual projeto, mas que não têm significado sobre o projeto conceitual. Liste as restrições semânticas separadamente. Desenhe um diagrama EER do banco de dados de biblioteca.

Estudo de caso: A Georgia Tech Library (GTL) tem aproximadamente 16 mil usuários, 100 mil títulos e 250 mil volumes (uma média de 2,5 cópias por livro). Cerca de 10 por cento dos volumes estão emprestados a qualquer momento. Os bibliotecários garantem que os livros estejam disponíveis quando os usuários querem apanhá-los emprestado. Além disso, os bibliotecários precisam saber quantas cópias de cada livro existem na biblioteca ou emprestadas a qualquer momento. Um catálogo de livros está disponível on-line, listando livros por autor, título e assunto. Para cada título da biblioteca, uma descrição do livro é mantida no catálogo, que varia de parágrafos a várias páginas. Os bibliotecários de referência desejam poder acessar essa descrição quando os usuários solicitarem informações sobre um livro. O pessoal da biblioteca inclui o bibliotecário chefe, bibliotecários associados ao departamento, bibliotecários de referência, pessoal de despacho e assistentes de bibliotecário.

Os livros podem ser emprestados por 21 dias. Os usuários têm permissão para pegar apenas cinco livros de uma só vez. Os usuários normalmente retornam os livros dentro de três a quatro semanas. A maioria dos usuários sabe que têm uma semana de tolerância antes que uma nota seja enviada para eles, e por isso tentam retornar os livros antes que o período de tolerância termine. Cerca de 5 por cento dos usuários precisa receber lembretes para devolver os livros. A maioria dos livros atrasados é retornada dentro de um mês da data de vencimento. Aproximadamente 5 por cento dos livros atrasados são mantidos ou nunca são retornados. Os membros mais ativos da biblioteca são definidos como aqueles que pegam livros emprestados pelo menos dez vezes durante o ano. Um por cento dos usuários que mais utilizam empréstimos realizam 15 por cento dos empréstimos, e os maiores 10 por cento dos usuários realizam 40 por cento dos empréstimos. Cerca de 20 por cento dos usuários são totalmente inativos por nunca terem emprestado livros.

Para tornar-se um usuário da biblioteca, os candidatos preenchem um formulário incluindo seu CPF, endereço de correspondência do campus e da residência, e números de telefone. Os bibliotecários emitem um cartão numerado, legível à máquina, com a foto do usuário. Esse cartão tem validade de quatro anos. Um mês antes de um cartão expirar, uma nota é enviada ao usuário para fazer a renovação. Os professores do instituto são considerados usuários automáticos. Quando um novo usuário do corpo docente entra para o instituto, sua informação é puxada

dos registros de funcionário e um cartão da biblioteca é remetido ao seu endereço no campus. Os professores têm permissão para retirar livros por intervalos de três meses, e possuem um período de tolerância de duas semanas. As notas de renovação para os professores são enviadas para seu endereço no campus.

A biblioteca não empresta alguns livros, como livros de referência, livros raros e mapas. Os bibliotecários precisam diferenciar livros que podem ser emprestados daqueles que não podem. Além disso, eles possuem uma lista de alguns livros em que estão interessados em adquirir, mas não conseguem obter, como livros raros ou que estão esgotados, e livros que foram perdidos ou destruídos, mas não substituídos. Os bibliotecários precisam ter um sistema que registre os livros que não podem ser emprestados bem como os livros que eles estão interessados em adquirir. Alguns livros podem ter o mesmo título; portanto, o título não pode ser usado como um meio de identificação. Cada livro é identificado por seu International Standard Book Number (ISBN), um código internacional exclusivo atribuído a todos os livros. Dois livros com o mesmo título podem ter diferentes ISBNs se estiverem em diferentes idiomas ou tiverem diferentes encadernações (capa dura ou brochura). As edições de um mesmo livro possuem ISBNs diferentes.

O sistema de banco de dados proposto precisa ser projetado para registrar os usuários, os livros, o catálogo e a atividade de empréstimo.

8.20. Projete um banco de dados para registrar informações para um museu de arte. Suponha que os seguintes requisitos foram coletados:

- O museu tem uma coleção de OBJETOS_ARTE. Cada OBJETO_ARTE tem um numero_id exclusivo, um Artista (se conhecido), um Ano (quando foi criado, se conhecido), um Titulo e uma Descricao. Os objetos de arte são categorizados de várias maneiras, conforme discutido a seguir.
- OBJETOS_ARTE são categorizados com base em seu tipo. Existem três tipos principais: PINTURA, ESCULTURA e ESTATUA, mais um tipo chamado OUTRO para acomodar objetos que não se encaixam em nenhum dos três tipos principais.
- Uma PINTURA tem um Tipo_pintura (óleo, aquarela etc.), material em que é desenhada Desenhado_em (papel, tela, madeira etc.) e Estilo (moderno, abstrato etc.).
- Uma ESCULTURA ou uma estátua tem um Material com a qual foi criada (madeira, pedra etc.), Altura, Peso e Estilo.
- Um objeto de arte na categoria OUTRO tem um Tipo (impressão, foto etc.) e Estilo.
- OBJETOS_ARTE são categorizados como COLECAO_PERMANENTE (objetos que pertencem ao museu) e EMPRESTADOS. As informações capturadas sobre os objetos na COLECAO_PERMANENTE incluem Data_aquisicao, Status (em exibição, emprestado ou guardado) e Custo. A informação capturada sobre objetos EMPRESTADOS inclui a Colecao da qual foi emprestado, Data_emprestimo e Data_retorno.
- A informação descrevendo o país ou cultura da Origem (italiano, egípcio, norte-americano, indiano etc.) e Epoca (Renascença, Moderno, Antiguidade, e assim por diante) é capturada para cada OBJETO_ARTE.
- O museu registra a informação deARTISTA, se for conhecida:Nome, Data_nascimento (se conhecida), Data_morte (se não estiver vivo), Pais_de_srcem, Epoca, Estilo_principal e Descricao. O Nome é considerado exclusivo.
- Ocorrem diferentes EXPOSICOES, cada uma com um Nome, Data_inicio e Data_final. As EXPOSICOES são relacionadas a todos os objetos de arte que estavam em amostra durante a exposição.
- A informação é mantida em outras COLECOES com as quais o museu interage, incluindo Nome (exclusivo), Tipo (museu, pessoal etc.), Descricao, Endereco, Telefone e Pessoa_contato atual.

Desenhe um diagrama de esquema EER para essa aplicação. Discuta quaisquer suposições que você fizer e que justifiquem suas escolhas de projeto EER.

8.21. A Figura 8.12 mostra um exemplo de diagrama EER para o banco de dados de um pequeno aeroporto particular, que é usado para registrar aeronaves, seus proprietários, funcionários do aeroporto e pilotos. Com base nos requisitos para esse banco de dados, a informação a seguir foi coletada: cada AERONAVE tem um número de registro [Num_reg], é de um tipo de avião em particular [DO_TIPO] e é mantido em um hangar em particular [GUARDADO_EM]. Cada TIPO_AVIAO tem um número de modelo [Modelo], uma capacidade [Capacidade] e um peso [Peso]. Cada HANGAR tem um número [Numero], uma capacidade [Capacidade] e um local [Localizacao]. O banco de dados também registra os PROPRIETARIOS de cada avião [PERTENCE] e os FUNCIONARIOS que fazem a manutenção do avião [MANUTENCAO]. Cada instância de relacionamento em PERTENCE relaciona uma AERONAVE a um PROPRIETARIO e inclui a data de compra [Data_comp]. Cada instância de relacionamento em MANUTENCAO relaciona um FUNCIONARIO a um registro de serviço [SERVICO].

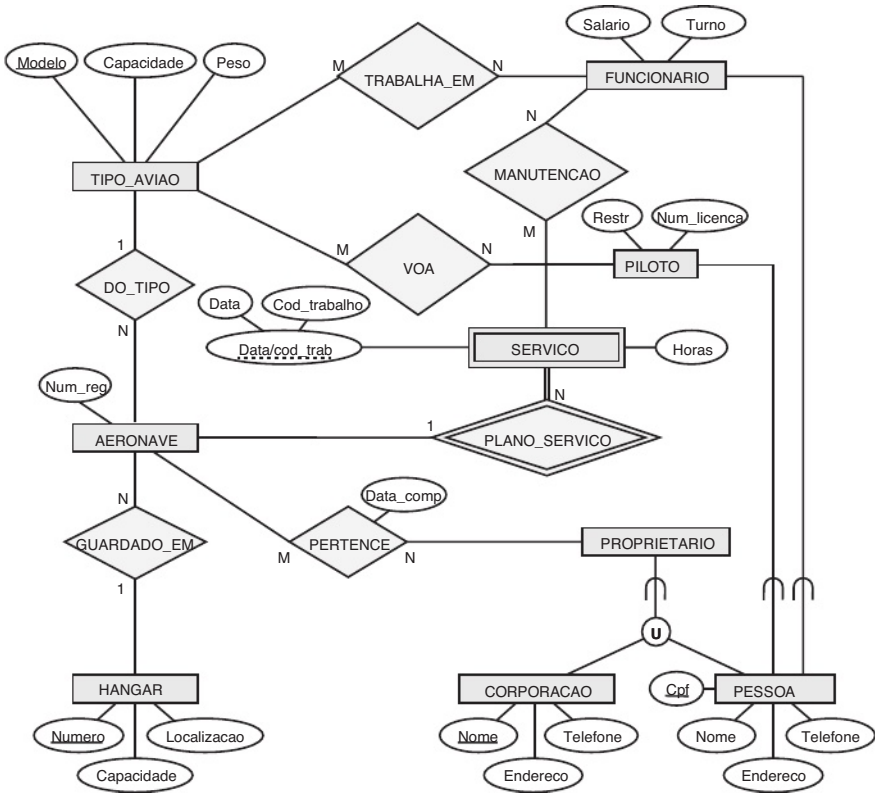


Figura 8.12
Esquema EER para um banco de dados PEQUENO_AEROPORTO.

Cada avião passa por manutenção muitas vezes; logo, ela é relacionada por [PLANO_SERVICO] a uma série de registros de SERVICO. Um registro de SERVICO inclui como atributos a data da manutenção [Data], o número de horas gastas no trabalho [Horas] e o tipo de trabalho realizado [Cod_trab]. Usamos um tipo de entidade fraca [SERVICO] para representar o serviço na aeronave, pois o número de registro da aeronave é usado para identificar um registro de manutenção. Um PROPRIETARIO é uma pessoa ou uma corporação. Assim, usamos um tipo de união (categoria) [PROPRIETARIO] que é um subconjunto da união dos tipos de entidade corporação [CORPORACAO] e pessoa [PESSOA]. Tanto pilotos [PILOTO] quanto funcionários [FUNCIONARIO] são subclasses de PESSOA. Cada PILOTO tem atributos específicos de número de licença [Num_licenca] e restrições [Restricoes]; cada FUNCIONARIO tem atributos específicos de salário [Salario] e turno trabalhado [Turno]. Todas as entidades PESSOA no banco de dados possuem dados mantidos sobre seu número de Cadastro de Pessoa Física [Cpf], nome [Nome], endereço [Endereco] e número de telefone [Telefone]. Para entidades CORPORACAO, os dados mantidos incluem nome [Nome], endereço [Endereco] e número de telefone [Telefone]. O banco de dados também registra os tipos de aviões que cada piloto é autorizado a voar [VOA] e os tipos de aviões em que cada funcionário pode realizar o trabalho de manutenção [TRABALHA_EM]. Mostre como o esquema EER PEQUENO_AEROPORTO da Figura 8.12 pode ser representado em notação UML. (*Nota:* não discutimos como representar categorias (tipos de união) em UML, de modo que você não precisa mapear as categorias nesta e na próxima questão.)

	Conjunto de entidades	(a) Tem relacionamento com	(b) Tem um atributo que é	(c) É uma especialização de	(d) É uma generalização de	Conjunto de entidades ou atributo
1.				PESSOA		
2.				MÃE		
3.				PESSOA		
4.	ALUNO					Cod_aluno
5.				ALUNO		
6.				SALA_AULA		
7.				CAVALO		
8.	CAVALO					Raça
9.	CAVALO					Idade
10.				CPF		
11.				CADEIRA		
12.	CADEIRA					Peso
13.				MULHER		
14.				PESSOA		
15.				PESSOA		

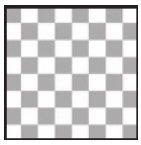
8.22. Mostre como o esquema EER UNIVERSIDADE da Figura 8.9 pode ser representado em notação UML.

8.23. Considere os conjuntos de entidades e atributos mostrados na tabela desta página. Coloque uma marcação em uma coluna de cada linha, para indicar o relacionamento entre as colunas mais à esquerda e à direita.

- **a.** O lado esquerdo tem um relacionamento com o lado direito.
- **b.** O lado direito é um atributo do lado esquerdo.
- **c.** O lado esquerdo é uma especialização do lado direito.
- **d.** O lado esquerdo é uma generalização do lado direito.

8.24. Desenhe um diagrama UML para armazenar um jogo de xadrez em um banco de dados. Você pode examinar em <http://www.chessgames.com> como fazer uma aplicação semelhante à que você está projetando. Indique claramente quaisquer suposições que você fizer em seu diagrama UML. Uma amostra das suposições que você pode fazer sobre o escopo é a seguinte:

1. O jogo de xadrez é realizado por dois jogadores.
2. O jogo é realizado em um tabuleiro de 8 × 8, como o que aparece a seguir:



3. Os jogadores recebem uma cor preta ou branca no início do jogo.
4. Cada jogador começa com as seguintes peças:
   - **a.** rei
   - **b.** rainha
   - **c.** 2 torres
   - **d.** 2 bispos
   - **e.** 2 cavalos
   - **f.** 8 peões
5. Cada peça tem a própria posição inicial.
6. Cada peça tem o próprio conjunto de jogadas válidas com base no estado do jogo. Você não precisa se preocupar com quais jogadas são válidas ou não, exceto pelas seguintes questões:
   - **a.** Uma peça pode se mover para um quadrado vazio ou capturar uma peça do oponente.
   - **b.** Se uma peça for capturada, ela é removida do tabuleiro.
   - **c.** Se um peão se mover para a última fileira, ele é 'promovido', sendo convertido para outra peça (rainha, torre, bispo ou cavalo).

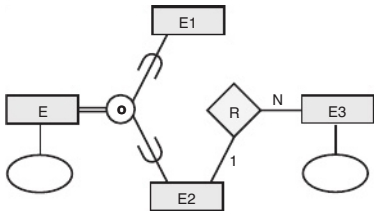
Nota: algumas dessas funções podem se espalhar por várias classes.

8.25. Desenhe um diagrama EER para um jogo de xadrez conforme descrito no Exercício 8.24. Focalize nos aspectos de armazenamento persistente do sistema. Por exemplo, o sistema precisaria recuperar todas as jogadas de cada jogo realizado em ordem sequencial.

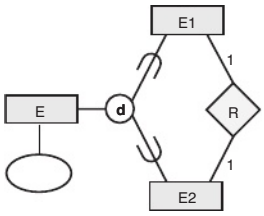
8.26. Quais dos seguintes diagramas EER são incorretos e por quê? Indique claramente quaisquer suposições que você fizer.

8.27. Considere o seguinte diagrama EER que descreve os sistemas de computador em uma empresa. Forneça os próprios atributos e chave para cada tipo de entidade. Forneça restrições de cardinalidade max justificando sua escolha. Escreva uma descrição narrativa completa do que esse diagrama EER representa.

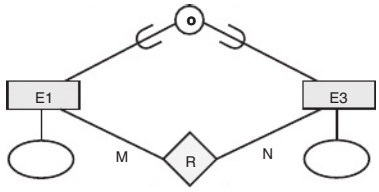
a.

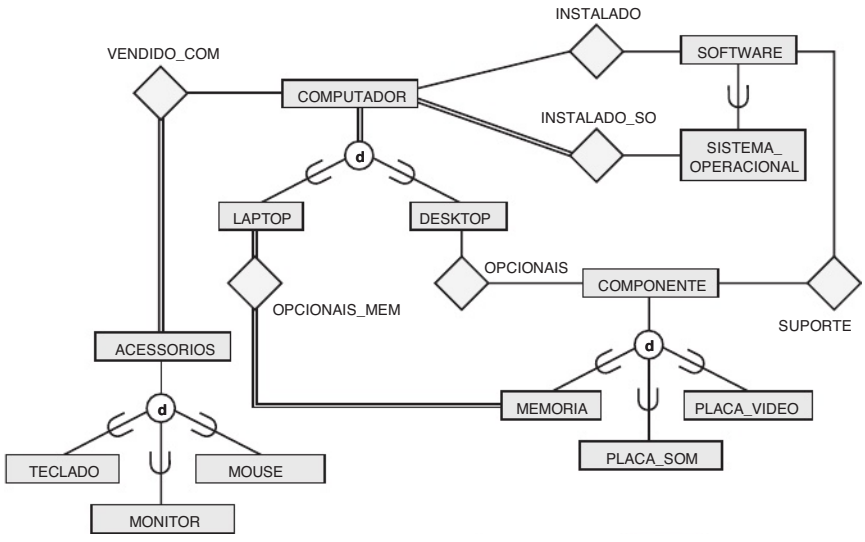


b.



c.





Exercícios de laboratório

8.28. Considere um banco de dados DIARIO_NOTAS em que os professores de um departamento acadêmico registram pontos ganhos por alunos individuais em suas aulas. Os requisitos de dados são resumidos da seguinte forma:

- Cada aluno é identificado por um identificador exclusivo, nome e sobrenome, e por um endereço de e-mail.
- Cada professor leciona certas disciplinas a cada período. Cada disciplina é identificada por um número, um número de seção e o período em que ela é realizada. Para cada disciplina, o professor especifica o número mínimo de pontos necessários para ganhar notas A, B, C, D e F. Por exemplo, 90 pontos para um A, 80 pontos para um B, 70 pontos para um C, e assim por diante.

- Os alunos são matriculados em cada disciplina lecionada pelo professor.
- Cada disciplina tem uma série de componentes de avaliação (como exame do meio do período, exame final, projeto, e assim por diante). Cada componente de avaliação tem um número máximo de pontos (como 100 ou 50) e um peso (como 20 por cento ou 10 por cento). Os pesos de todos os componentes de avaliação de um curso em geral totalizam 100.
- Finalmente, o professor registra os pontos ganhos por aluno em cada um dos componentes de avaliação em cada uma das disciplinas. Por exemplo, o aluno 1234 ganha 84 pontos para o componente de avaliação do meio do período da disciplina CCc2310 da seção 2 no período do segundo semestre de 2009. O componente de avaliação de exame do meio do período pode ter sido definido para ter um máximo de 100 pontos e um peso de 20 por cento da nota da disciplina.

Crie um diagrama Entidade-Relacionamento estendido para o banco de dados do diário e monte o projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

8.29. Considere um sistema de banco de dados LEILAO_ON-LINE em que os membros (compradores e vendedores) participam na venda de itens. Os requisitos de dados para esse sistema são resumidos a seguir:

- O site on-line tem membros, e cada um é identificado por um número de membro exclusivo e descrito por um endereço de e-mail, nome, senha, endereço residencial e número de telefone.
- Um membro pode ser um comprador ou um vendedor. Um comprador tem um endereço de entrega registrado no banco de dados. Um vendedor tem um número de conta bancária e um número de encaminhamento registrados no banco de dados.
- Os itens são colocados à venda por um vendedor e identificados por um número de item exclusivo, atribuído pelo sistema. Os itens também são descritos por um título de item, uma descrição, um preço de lance inicial, um incremento de lance, a data inicial do leilão e a data final do leilão.
- Os itens também são classificados com base em uma hierarquia de classificação fixa (por exemplo, um modem pode ser classificado como COMPUTADOR→HARDWARE→MODEM).
- Os compradores fazem lances para os itens em que estão interessados. O preço do lance e a hora do lance são registrados. O comprador ao final do leilão com o maior preço de lance é declarado o vencedor e uma transação entre comprador e vendedor pode então prosseguir.
- O comprador e o vendedor podem registrar uma nota em relação às transações completadas. A nota contém uma pontuação da outra parte na transação (1-10) e um comentário.

Crie um diagrama Entidade-Relacionamento estendido para o banco de dados LEILAO_ON-LINE e monte o projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

8.30. Considere um sistema de banco de dados para uma organização de beisebol como as principais ligas nacionais. Os requisitos de dados são resumidos a seguir:

- O pessoal envolvido na liga inclui jogadores, técnicos, dirigentes e árbitros. Cada um tem uma identificação pessoal exclusiva. Eles também são descritos por seu nome e sobrenome, junto com a data e local de nascimento.
- Os jogadores são descritos ainda por outros atributos, como sua orientação de batida (esquerda, direita ou ambas) e têm uma média de batidas (MB) por toda a vida.
- Dentro do grupo de jogadores existe um subgrupo de jogadores chamados lançadores. Os lançadores têm uma média de corrida ganha (MCG) por toda a vida associada a eles.
- As equipes são identificadas exclusivamente por seus nomes. As equipes também são descritas pela cidade em que estão localizadas e pela divisão e liga em que jogam (como a divisão Central da Liga Norte-americana).
- As equipes possuem um dirigente, uma série de técnicos e uma série de jogadores.
- Os jogos são realizados entre dois times com um designado como o time da casa e o outro como o time visitante em determinada data. A pontuação (corridas, batidas e erros) é registrado para cada time. O time com a maioria das corridas é declarado o vencedor do jogo.
- A cada jogo terminado, um lançador vencedor e um lançador perdedor são registrados. Caso seja concedido um salvamento, o lançador salvo também é registrado.
- A cada jogo, o *número* de arcos (simples, duplos, triplos e *home runs*) obtidos por jogador também é registrado.

Crie um diagrama Entidade-Relacionamento estendido para o banco de dados BEISEBOL e monte o projeto usando uma ferramenta de modelagem como ERwin ou Rational Rose.

8.31. Considere o diagrama EER para o banco de dados UNIVERSIDADE mostrado na Figura 8.9.

Entre com seu projeto usando uma ferramenta de modelagem de dados como ERwin ou Rational Rose. Faça uma lista das diferenças na notação entre o diagrama no texto e a notação diagramática equivalente que você acabou usando com a ferramenta.

8.32. Considere o diagrama EER para o pequeno banco de dados AEROPORTO mostrado na Figura 8.12. Monte esse projeto usando uma ferramenta de modelagem de dados como ERwin ou Rational Rose. Tenha cuidado ao modelar a categoria PROPRIETARIO nesse diagrama. (*Dica:* considere o uso de CORPORACAO_É_PROPRIETARIA e PESSOA_É_PROPRIETARIA como tipos de relacionamento distintos.)

8.33. Considere o banco de dados UNIVERSIDADE descrito no Exercício 7.16. Você já desenvolveu um esquema ER para esse banco de dados usando uma ferramenta de modelagem de dados como ERwin ou Rational Rose no Exercício de Laboratório 7.31. Modifique esse diagrama classificando DISCIPLINAS como DISCIPLINA_GRADUACAO ou DISCIPLINA_POSGRADUACAO e PROFESSORES como PROFESSORES_JUNIOR ou PROFESSORES_SENIOR. Inclua atributos apropriados para esses novos tipos de entidade. Depois, estabeleça relacionamentos indicando que os professores júnior lecionam disciplinas para alunos em graduação enquanto os professores sênior lecionam disciplinas para alunos de pós-graduação.

Bibliografia selecionada

Muitos artigos propuseram modelos de dados conceituais ou semânticos. Aqui, oferecemos uma lista representativa. Um grupo de artigos, incluindo Abrial (1974), modelo DIAM de Senko (1975), o método NIAM (Verheijen e VanBekkum, 1982) e Bracchi et al. (1976), apresenta modelos semânticos que são baseados no conceito de relacionamentos binários. Outro grupo de artigos antigos discute métodos para estender o modelo relacional para melhorar suas capacidades de modelagem. Isso inclui os artigos de Schmid e Swenson (1975), Navathe e Schkolnick (1978), modelo RM/T de Codd (1979), Furtado (1978) e o modelo estrutural de Wiederhold e Elmasri (1979).

O modelo ER foi proposto srcinalmente por Chen (1976) e é formalizado em Ng (1981). Desde então, diversas extensões de suas capacidades de modelagem foram propostas, como em Scheuermann et al. (1979), Dos Santos et al. (1979), Teorey et al. (1986), Gogolla e Hohenstein (1991) e o modelo Entidade-Categoria-Relacionamento (ECR) de Elmasri et al. (1985). Smith e Smith (1977) apresentam os conceitos de generalização e agregação. O modelo de dados semântico de Hammer e McLeod (1981) introduziu os conceitos de reticulados de classe/subclasse, bem como outros conceitos de modelagem avançados.

Um estudo da modelagem semântica de dados aparece em Hull e King (1987). Eick (1991) discute projeto e transformações dos esquemas conceituais. A análise de restrições para relacionamentos *n*-ários é dada em Soutou (1998). A UML é descrita detalhadamente em Booch, Rumbaugh e Jacobson (1999). Fowler e Scott (2000) e Stevens e Pooley (2000) oferecem introduções concisas aos conceitos de UML.

Fensel (2000, 2003) discute a Web semântica e a aplicação de ontologias. Uschold e Gruninger (1996) e Gruber (1995) discutem sobre ontologias. A edição de junho de 2002 de *Communications of the ACM* é dedicada a conceitos e aplicações da ontologia. Fensel (2003) é um livro que discute as ontologias e o comércio eletrônico.

capítulo 9

Projeto de banco de dados relacional por mapeamento ER e EER para relacional

Este capítulo discute como **projetar um esquema de banco de dados relacional** com base em um projeto de esquema conceitual. A Figura 7.1 apresentou uma visão de alto nível do processo de projeto de banco de dados, e neste capítulo focamos a etapa de projeto lógico de banco de dados lógico ou mapeamento de modelo de dados do projeto de banco de dados. Apresentamos os procedimentos para criar um esquema relacional com base em um esquema Entidade-Relacionamento (ER) ou ER estendido (EER). Nossa discussão relaciona as construções dos modelos ER e EER, apresentadas nos capítulos 7 e 8, às construções do modelo relacional, apresentadas nos capítulos 3 a 6. Muitas ferramentas de engenharia de software auxiliada por computador (CASE) são baseadas nos modelos ER e EER, ou outros modelos semelhantes, conforme discutimos nos capítulos 7 e 8. Muitas ferramentas utilizam diagramas ER ou EER ou variações para desenvolver um esquema graficamente, e depois o convertem de maneira automática em um esquema de banco de dados relacional na DDL de um SGBD relacional específico, empregando algoritmos semelhantes aos apresentados neste capítulo.

Esboçamos um algoritmo de sete etapas na Seção 9.1 para converter os dados (tipos básicos, relacionamentos binários (com várias restrições estruturais), relacionamentos *n*-ários e atributos (simples, compostos e multivalorados) — em relações. Depois, na Seção 9.2, continuamos o algoritmo de mapeamento ao descrever como mapear as construções do modelo EER — especialização/generalização e tipos de união (categorias) — em relações. No final do capítulo há um resumo.

9.1 Projeto de banco de dados relacional usando o mapeamento ER para relacional

9.1.1 Algoritmo de mapeamento ER para relacional

Nesta seção, vamos descrever as etapas de um algoritmo para mapeamento ER para relacional. Usamos o exemplo de banco de dados EMPRESA para ilustrar o procedimento de mapeamento. O esquema ER EMPRESA aparece novamente na Figura 9.1, e o esquema de banco de dados relacional EMPRESA correspondente aparece na Figura 9.2 para ilustrar as etapas de mapeamento. Assumimos que o mapeamento criará tabelas com atributos simples de único valor. As restrições do modelo relacional definidas no Capítulo 3, que incluem chaves primárias, chaves únicas (se houver) e restrições de integridade referencial sobre as relações, também serão especificadas nos resultados do mapeamento.

Etapa 1: Mapeamento de tipos de entidade regular. Para cada tipo de entidade regular (forte) E no esquema ER, crie uma relação R que inclua todos os atributos simples de E. Inclua apenas os atributos de componente simples de um atributo composto. Escolha um dos atributos-chave de E como chave primária para R. Se a chave escolhida de E for composta, então o conjunto de atributos simples que a compõem juntos formarão a chave primária de R.

Se várias chaves fossem identificadas para E durante o projeto conceitual, a informação que descreve os atributos que formam cada chave adicional é

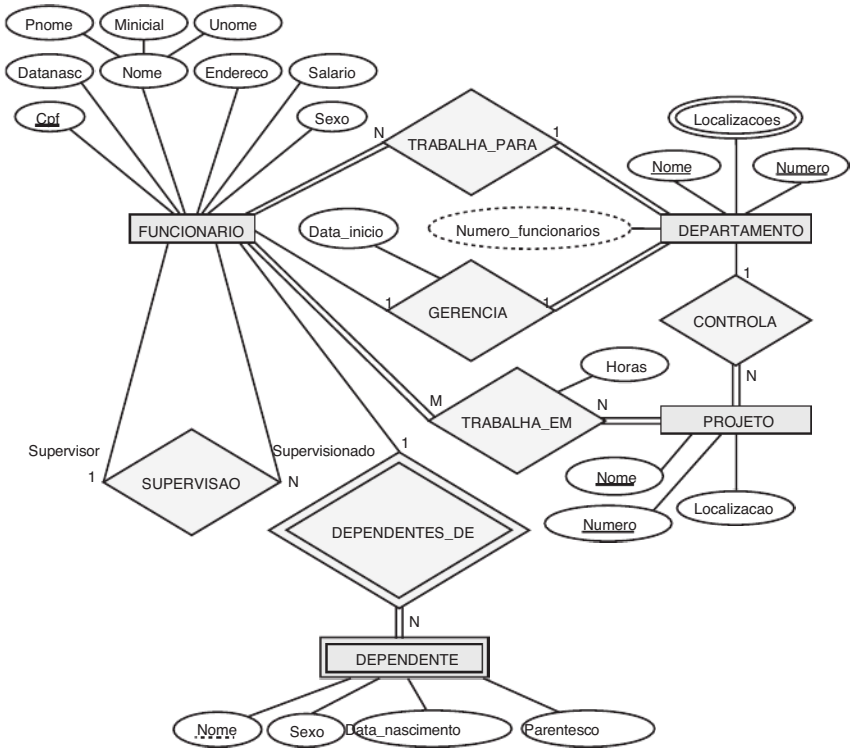


Figura 9.1
O diagrama do esquema conceitual ER para o banco de dados EMPRESA.

mantida a fim de especificar chaves secundárias (únicas) da relação *R*. O conhecimento sobre as chaves também é mantido para fins de indexação e outros tipos de análises.

Em nosso exemplo, criamos as relações FUNCIONARIO, DEPARTAMENTO e PROJETO na Figura 9.2 para corresponder aos tipos de entidade regular FUNCIONARIO, DEPARTAMENTO e PROJETO da Figura 9.1. Os atributos de chave estrangeira e relacionamento, se houver, ainda não estão incluídos; eles serão acrescentados durante as etapas seguintes. Estes incluem os atributos Cpf_supervisor e Dnr de FUNCIONARIO, Cpf_gerente e Data_inicio_gerente de DEPARTAMENTO, e Dnum de PROJETO. Em nosso exemplo, escolhemos Cpf, Dnumero e Projnumero como chaves primárias para as relações FUNCIONARIO, DEPARTAMENTO e PROJETO, respectivamente. O conhecimento de que o Dnome de DEPARTAMENTO e o Projnome de PROJETO são chaves secundárias é mantido para possível uso posterior no projeto.

As relações que são criadas com base no mapeamento dos tipos de entidade às vezes são chamadas **relações de entidade**, pois cada tupla representa uma instância de entidade. O resultado após essa etapa de mapeamento aparece na Figura 9.3(a).

Etapa 2: Mapeamento de tipos de entidade fraca. Para cada tipo de entidade fraca *F* no esquema ER com tipo de entidade proprietária *E*, crie uma relação *R* e inclua todos os atributos simples (ou componentes simples dos atributos compostos) de *F* como atributos de *R*. Além disso, inclua como atributos de chave estrangeira de *R* os atributos de chave primária da(s) relação(ões) que corresponde(m) aos tipos de entidade proprietária. Isso consegue mapear o tipo de relacionamento de identificação de *F*. A chave pri-

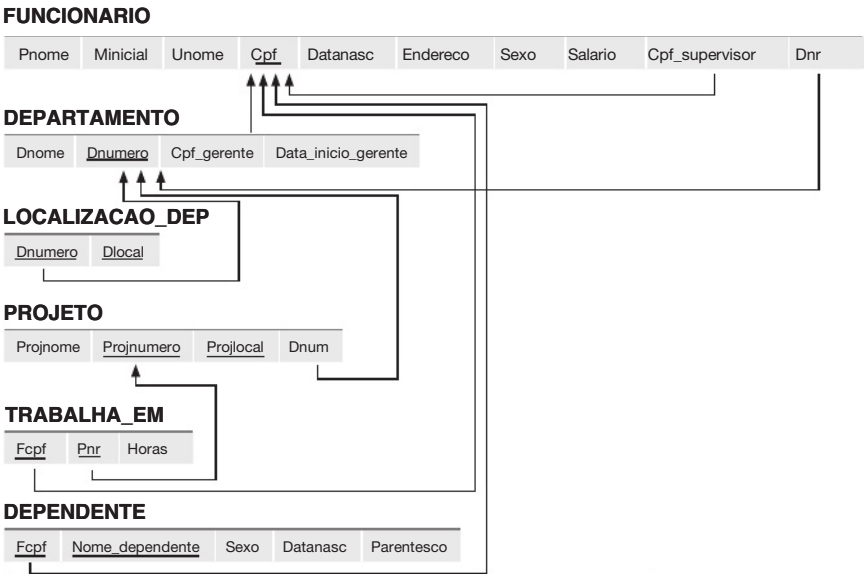


Figura 9.2
Resultado do mapeamento do esquema ER EMPRESA para um esquema de banco de dados relacional.

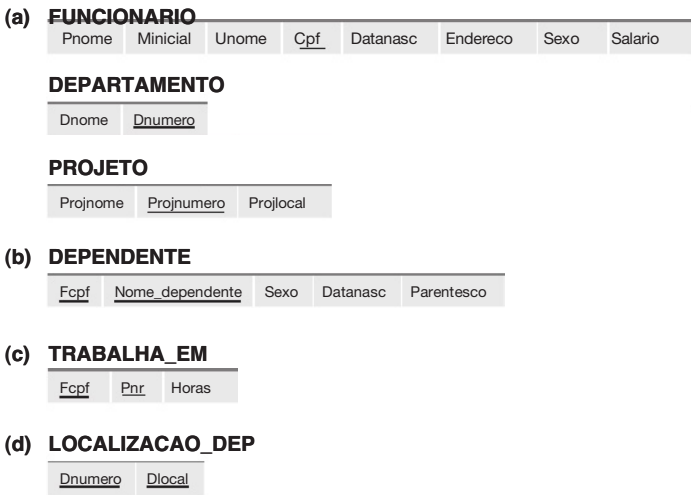


Figura 9.3
Ilustração de algumas etapas de mapeamento. (a) Relações de entidade após a etapa 1. (b) Relação de entidade fraca após a etapa 2. (c) Relação de relacionamento após a etapa 5. (d) Relação representando atributo multivalorado após a etapa 6.

mária de *R* é a combinação das chaves primárias dos proprietários e a chave parcial do tipo de entidade fraca *F*, se houver.

Se houver um tipo de entidade fraca E_2, cujo proprietário também é um tipo de entidade E_1, então E_1 deve ser mapeado antes de E_2 para determinar primeiro sua chave primária.

Em nosso exemplo, criamos a relação DEPENDENTE nesta etapa para corresponder ao tipo de entidade fraca DEPENDENTE (ver Figura 9.3(b)). Incluímos a chave primária Cpf da relação FUNCIONARIO — que corresponde ao tipo de entidade proprietária — como um atributo de chave estrangeira de DEPENDENTE, renomeamos para Fcpf, embora isso não seja necessário. A chave primária da relação DEPENDENTE é a combinação {Fcpf, Nome_dependente}, pois Nome_dependente (também renomeado de Nome na Figura 9.1) é a chave parcial de DEPENDENTE.

É comum escolher a opção de propagação (CASCADE) para a ação de disparo referencial (ver Seção 4.2) na chave estrangeira na relação correspondente ao tipo de entidade fraca, pois uma entidade fraca tem uma dependência de existência em sua entidade proprietária. Isso pode ser usado para ON UPDATE e ON DELETE.

Etapa 3: Mapeamento dos tipos de relacionamento binários 1:1. Para cada tipo de relacionamento binário 1:1 *R* no esquema ER, identifique as relações *S* e *T* que correspondem aos tipos de entidade participantes em *R*. Existem três técnicas possíveis: (1) a técnica de chave estrangeira, (2) a técnica de relacionamento mesclado e (3) a técnica de relação de referência cruzada ou relacionamento. A primeira técnica é a mais útil e deve ser seguida a menos que haja condições especiais, conforme discutimos a seguir.

1. **Técnica de chave estrangeira:** escolha uma das relações — digamos, *S* — e inclua como chave estrangeira em *S* a chave primária de *T*. É melhor escolher um tipo de entidade com *participação total* em *R* no papel de *S*. Inclua todos os atributos simples (ou componentes simples dos atributos compostos) do tipo de relacionamento 1:1 *R* como atributos de *S*.
   Em nosso exemplo, mapeamos o tipo de relacionamento 1:1 GERENCIA da Figura 9.1 ao escolher o tipo de entidade de participação DEPARTAMENTO para servir ao papel de *S*, pois sua participação no tipo de relacionamento GERENCIA é total (cada departamento tem um gerente). Incluímos a chave primária da relação FUNCIONARIO como chave estrangeira na relação DEPARTAMENTO e a renomeamos como Cpf_gerente. Também incluímos o atributo simples Data_inicio do tipo de relacionamento GERENCIA na relação DEPARTAMENTO e o renomeamos como Data_inicio_gerente (ver Figura 9.2).
   Observe que é possível incluir a chave primária de *S* como uma chave estrangeira em *T* em vez disso. Em nosso exemplo, isso significa ter um atributo de chave estrangeira, digamos, Depart_gerenciado na relação FUNCIONARIO, mas terá um valor NULL para as tuplas de funcionários que não gerenciam um departamento. Se apenas 2 por cento dos funcionários gerenciam um departamento, então 98 por cento das chaves estrangeiras seriam NULL nesse caso. Outra possibilidade é ter chaves estrangeiras nas relações *S* e *T* de maneira redundante, mas isso cria redundância e agrega uma penalidade para a manutenção da consistência.
2. **Técnica de relação mesclada:** um mapeamento alternativo de um tipo de relacionamento 1:1 é mesclar os dois tipos de entidade e o relacionamento em uma única relação. Isso é possível quando *ambas as participações são totais*, pois indicaria que as duas tabelas terão exatamente o mesmo número de tuplas o tempo inteiro.
3. **Técnica de relação de referência cruzada ou relacionamento:** a terceira opção é configurar uma terceira relação *R* para a finalidade de referência cruzada das chaves primárias das duas relações *S* e *T* representando os tipos de entidade. Conforme veremos, essa técnica é exigida para relacionamentos M:N binários. A relação *R* é chamada de **relação de relacionamento** (ou, às vezes, de **tabela de pesquisa**), porque cada tupla em *R* representa uma instância de relacionamento que relaciona uma tupla de *S* a uma tupla de *T*. A relação *R* incluirá os atributos de chave primária de *S* e *T* como chaves estrangeiras para *S* e *T*. A chave primária de *R* será uma das duas chaves estrangeiras, e a outra chave estrangeira será uma chave unique de *R*. A desvantagem é ter uma relação extra e exigir uma operação de junção extra ao combinar tuplas relacionadas das tabelas.

Etapa 4: Mapeamento de tipos de relacionamento binário 1:N. Para cada tipo de relacionamento *R* binário regular 1:N, identifique a relação *S* que representa o tipo de entidade participante no *lado N* do tipo de relacionamento. Inclua como chave estrangeira em *S* a chave primária da relação *T* que representa

o outro tipo de entidade participante em *R*; fazemos isso porque cada instância de entidade no lado N está relacionada a, no máximo, uma instância de entidade no lado 1 do tipo de relacionamento. Inclua quaisquer atributos simples (ou componentes simples dos atributos compostos) do tipo de relacionamento 1:N como atributos de *S*.

Em nosso exemplo, agora mapeamos os tipos de relacionamento 1:N TRABALHA_PARA, CONTROLA e SUPERVISAO da Figura 9.1. Para TRABALHA_PARA, incluímos a chave primária Dnumero da relação DEPARTAMENTO como chave estrangeira na relação FUNCIONARIO e a chamamos de Dnr. Para SUPERVISAO, incluímos a chave primária da relação FUNCIONARIO como chave estrangeira na própria relação FUNCIONARIO — pois o relacionamento é recursivo — e a chamamos de Cpf_supervisor. O relacionamento CONTROLA é mapeado para o atributo de chave estrangeira Dnum de PROJETO, que referencia a chave primária Dnumero da relação DEPARTAMENTO. Essas chaves estrangeiras são mostradas na Figura 9.2.

Uma técnica alternativa é usar a opção de **relação de relacionamento** (referência cruzada) como na terceira opção para os relacionamentos binários 1:1. Criamos uma relação separada *R* cujos atributos são chaves primárias de *S* e *T*, que também serão chaves estrangeiras para *S* e *T*. A chave primária de *R* é igual à chave primária de *S*. Essa opção pode ser usada se algumas tuplas em *S* participarem do relacionamento para evitar valores NULL excessivos na chave estrangeira.

Etapa 5: Mapeamento de tipos de relacionamento binário M:N. Para cada tipo de relacionamento *R* binário M:N, crie uma nova relação *S* para representar *R*. Inclua como atributos de chave estrangeira em *S* as chaves primárias das relações que representam os tipos de entidade participantes; sua *combinação* formará a chave primária de *S*. Inclua também quaisquer atributos simples do tipo de relacionamento M:N (ou componentes simples dos atributos compostos) como atributos de *S*. Observe que não podemos representar um tipo de relacionamento M:N por um único atributo de chave estrangeira em uma das relações participantes (como fizemos para os tipos de relacionamento 1:1 ou 1:N) devido à razão de cardinalidade M:N; temos de criar uma *relação de relacionamento* *S* separada.

Em nosso exemplo, mapeamos o tipo de relacionamento M:N TRABALHA_EM da Figura 9.1 criando a relação TRABALHA_EM na Figura 9.2. Incluímos as chaves primárias das relações PROJETO e FUNCIONARIO como chaves estrangeiras em TRABALHA_EM e as renomeamos como Pnr e Fcpf, respectivamente. Também incluímos um atributo Horas em TRABALHA_EM para representar o atributo Horas do tipo de relacionamento. A chave primária da relação TRABALHA_EM é a combinação dos atributos de chave estrangeira {Fcpf, Pnr}. Essa **relação de relacionamento** aparece na Figura 9.3(c).

A opção de propagação (CASCADE) para a ação de disparo referencial (ver Seção 4.2) deve ser especificada sobre as chaves estrangeiras na relação correspondente ao relacionamento *R*, pois cada instância de relacionamento tem uma dependência de existência sobre cada uma das entidades a que ela se relaciona. Isso pode ser usado tanto para ON UPDATE quanto para ON DELETE.

Observe que sempre podemos mapear relacionamentos 1:1 ou 1:N de uma maneira semelhante aos relacionamentos M:N usando a técnica de referência cruzada (relação de relacionamento), conforme discutimos anteriormente. Essa alternativa é particularmente útil quando existem poucas instâncias de relacionamentos, a fim de evitar valores NULL em chaves estrangeiras. Nesse caso, a chave primária da relação de relacionamento será *apenas uma* das chaves estrangeiras que referenciam as relações da entidade participante. Para um relacionamento 1:N, a chave primária da relação de relacionamento será a chave estrangeira que referencia a relação de entidade no lado N. Para um relacionamento 1:1, qualquer chave estrangeira pode ser usada como chave primária da relação de relacionamento.

Etapa 6: Mapeamento de atributos multivalorados. Para cada atributo multivalorado *A*, crie uma relação *R*. Essa relação *R* incluirá um atributo correspondente a *A*, mais o atributo da chave primária *Ch* — como uma chave estrangeira em *R* — da relação que representa o tipo de entidade ou tipo de relacionamento que tem *A* como atributo multivalorado. A chave primária de *R* é a combinação de *A* e *Ch*. Se o atributo multivalorado for composto, incluímos seus componentes simples.

Em nosso exemplo, criamos uma relação LOCALIZACAO_DEP (ver Figura 9.3(d)). O atributo Dlocalizacao representa o atributo multivalorado LOCALIZACOES de DEPARTAMENTO, enquanto Dnumero — como chave estrangeira — representa a chave primária da relação DEPARTAMENTO. A chave primária de LOCALIZACAO_DEP é a combinação de {Dnumero, Dlocalizacao}. Uma tupla separada existirá em LOCALIZACAO_DEP para cada local que tenha um departamento.

A opção de propagação (CASCADE) para a ação de disparo referencial (ver Seção 4.2) deve ser especificada na chave estrangeira da relação *R* correspondente ao atributo multivalorado para ON UPDATE e

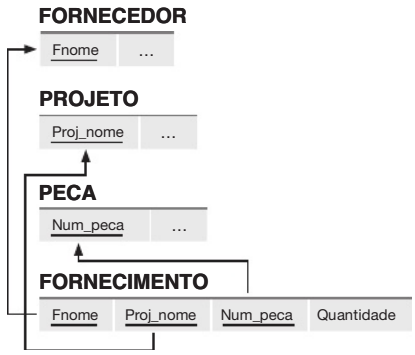


Figura 9.4
Mapeando o tipo de relacionamento *n*-ário FORNECIMENTO da Figura 7.17(a).

Tabela 9.1
Correspondência entre os modelos ER e relacional.

MODELO ER	MODELO RELACIONAL
Tipo de entidade	Relação de *entidade*
Tipo de relacionamento 1:1 ou 1:N	Chave estrangeira (ou relação de *relacionamento*)
Tipo de relacionamento M:N	Relação de *relacionamento* e *duas* chaves estrangeiras
Tipo de relacionamento *n*-ário	Relação de *relacionamento* e *n* chaves estrangeiras
Atributo simples Atributo composto	Atributo Conjunto de atributos componentes simples
Atributo multivalorado	Relação e chave estrangeira
Conjuntodevalores	Domínio
Atributo-chave	Chave primária (ou secundária)

ON DELETE. Também devemos observar que a chave de *R*, ao mapear um atributo composto, multivalorado, requer alguma análise do significado dos atributos componentes. Em alguns casos, quando um atributo multivalorado é composto, somente alguns dos atributos componentes são exigidos para fazer parte da chave de *R*. Esses atributos são semelhantes à chave parcial de um tipo de entidade fraca que corresponde ao atributo multivalorado (ver Seção 7.5).

A Figura 9.2 mostra o esquema de banco de dados relacional EMPRESA obtido com as etapas 1 a 6, e a Figura 3.6 mostra um exemplo de estado de banco de dados. Observe que ainda não discutimos o mapeamento de tipos de relacionamento *n*-ário ($n > 2$), pois ele não existe na Figura 9.1. Estes são mapeados de um modo semelhante aos tipos de relacionamento M:N, incluindo a etapa adicional a seguir no algoritmo de mapeamento.

Etapa 7: Mapeamento de tipos de relacionamento *n*-ário. Para cada tipo de relacionamento *n*-ário *R*, onde $n > 2$, crie uma relação *S* para representar *R*. Inclua como atributos de chave estrangeira em *S* as chaves primárias das relações que representam os tipos de entidade participantes. Inclua também quaisquer atributos simples do tipo de relacionamento *n*-ário (ou componentes simples de atributos compostos) como atributos de *S*. A chave primária de *S* normalmente é uma combinação de todas as chaves estrangeiras que referenciam as relações representando os tipos de entidade participantes. Porém, se as restrições de cardinalidade sobre qualquer um dos tipos de entidade *E* participantes em *R* for 1, então a chave primária de *S* não deve incluir o atributo de chave estrangeira que referencia a relação *E*' correspondente a *E* (ver discussão na Seção 7.9.2, referente a restrições sobre relacionamentos *n*-ários).

Por exemplo, considere o tipo de relacionamento FORNECIMENTO da Figura 7.17. Este pode ser mapeado para a relação FORNECIMENTO mostrada na Figura 9.4, cuja chave primária é a combinação das três chaves estrangeiras {Fnome, Num_peca, Proj_nome}.

9.1.2 Discussão e resumo do mapeamento para construções no modelo ER

A Tabela 9.1 resume as correspondências entre as construções e restrições do modelo ER e relacional.

Um dos principais pontos a observar em um esquema relacional, ao contrário de um esquema ER, é que os tipos de relacionamento não são representados explicitamente. Em vez disso, eles são representados com dois atributos *A* e *B*, um é uma chave primária e o outro é uma chave estrangeira (no mesmo domínio) incluída em duas relações *S* e *T*. Duas tuplas em *S* e *T* são relacionadas quando têm o mesmo valor para *A* e *B*. Usando a operação EQUIJUNÇÃO (ou JUNÇÃO NATURAL, se os dois atributos de junção tiverem o mesmo nome) em *S.A* e *T.B*, podemos combinar todos os pares de tuplas relacionadas de *S* e *T* e materializar o relacionamento. Quando um tipo de relacionamento binário 1:1 ou 1:N é envolvido, uma única operação de junção costuma ser necessária. Para um tipo de relacionamento binário M:N, duas operações de junção são necessárias, enquanto para tipos de relacionamento *n*-ários, *n* junções são necessárias para materializar totalmente as instâncias de relacionamento.

Por exemplo, para formar uma relação que inclui o nome do funcionário, nome do projeto e horas

que o funcionário trabalha em cada projeto, precisamos conectar cada tupla FUNCIONARIO às tuplas PROJETO relacionadas por meio da relação TRABALHA_EM na Figura 9.2. Logo, precisamos aplicar a operação EQUIJUNÇÃO às relações FUNCIONARIO e TRABALHA_EM com a condição de junção Cpf = Fcpf, e depois aplicar outra operação EQUIJUNÇÃO à relação resultante e a relação PROJETO com a condição de junção Pnr = Projnumero. Em geral, quando vários relacionamentos precisam ser examinados, diversas operações de junção precisam ser especificadas. Um usuário de banco de dados relacional sempre precisa estar ciente dos atributos de chave estrangeira para poder usá-los corretamente na combinação de tuplas relacionadas de duas ou mais relações. Isso às vezes é considerado uma desvantagem do modelo de dados relacional, porque as correspondências de chave estrangeira/chave primária nem sempre são óbvias pela inspeção dos esquemas relacionais. Se uma EQUIJUNÇÃO for realizada entre atributos de duas relações que não representam um relacionamento de chave estrangeira/chave primária, o resultado pode com frequência ser sem sentido e levar a dados falsos (espúrios). Por exemplo, o leitor pode tentar juntar as relações PROJETO e LOCALIZACAO_DEP na condição Dlocal = Projlocal e examinar o resultado (veja a discussão sobre tuplas espúrias na Seção 15.1.4).

No esquema relacional, criamos uma relação separada para *cada* atributo multivalorado. Para uma entidade em particular com um conjunto de valores para o atributo multivalorado, o valor do atributo-chave da entidade é repetido uma vez para cada valor do atributo multivalorado em uma tupla separada, pois o modelo relacional básico *não* permite valores múltiplos (uma lista, ou um conjunto de valores) para um atributo em uma única tupla. Por exemplo, como o departamento 5 tem três locais, existem três tuplas na relação LOCALIZACAO_DEP da Figura 3.6; cada tupla especifica um dos locais. Em nosso exemplo, aplicamos EQUIJUNÇÃO a LOCALIZACAO_DEP e DEPARTAMENTO no atributo Dnumero para obter os valores de todas as localizações junto com outros atributos de DEPARTAMENTO. Na relação resultante, os valores dos outros atributos de DEPARTAMENTO são repetidos em tuplas separadas para cada local que tenha um departamento.

A álgebra relacional básica não tem uma operação ANINHAR ou COMPRIMIR que produziria um conjunto de tuplas na forma {<'1', 'São Paulo'>, <'4', 'Mauá'>, <'5', {'Santo André', 'Itu', 'São Paulo'}>} com base na relação LOCALIZACAO_DEP da Figura 3.6. Essa é uma desvantagem séria da versão básica normalizada ou do modelo relacional. O modelo de dados de objeto e os sistemas objeto-relacional (ver Capítulo 11) permitem atributos multivalorados.

9.2 Mapeando construções do modelo EER para relações

A seguir, vamos discutir o mapeamento das construções do modelo EER para relações, estendendo o algoritmo de mapeamento ER para relacional que foi apresentado na Seção 9.1.1.

9.2.1 Mapeamento da especialização ou generalização

Existem várias opções para mapear uma série de subclasses que juntas formam uma especialização (ou, como alternativa, que são generalizadas para uma superclasse), como as subclasses {SECRETARIA, TECNICO, ENGENHEIRO} de FUNCIONARIO na Figura 8.4. Podemos acrescentar outro passo ao nosso algoritmo de mapeamento da Seção 9.1.1, que tem sete etapas, para lidar com o mapeamento da especialização. A etapa 8, que vem a seguir, oferece as opções mais comuns; outros mapeamentos também são possíveis. Discutimos as condições sob as quais cada opção deve ser usada. Usamos Atrs(R) para indicar *os atributos da relação R* e ChP(R) para indicar a *chave primária de R*. Primeiro, vamos descrever o mapeamento formalmente e depois ilustrar com exemplos.

Etapa 8: Opções para mapeamento da especialização ou generalização. Converta cada especialização com m subclasses $\{S_1, S_2, ..., S_m\}$ e superclasse (generalizada) C, em que os atributos de C são $\{ch, a_1, ...a_n\}$ e ch é a chave (primária) para os esquemas da relação usando uma das seguintes opções:

- **Opção 8A: Múltiplas relações — superclasse e subclasses.** Crie uma relação L para C com atributos Atrs(L) = $\{ch, a_1, ..., a_n\}$ e ChP(L_i) = ch. Crie uma relação L_i para cada subclasse S_i, $1 \le i \le m$, com os atributos Atrs(L_i) = $\{ch\} \cup$ {atributos de S_i} e ChP(L_i) = ch. Essa opção funciona para qualquer especialização (total ou parcial, disjunta ou sobreposta).
- **Opção 8B: Múltiplas relações — apenas relações de subclasse.** Crie uma relação L_i para cada subclasse S_i, $1 \le i \le m$, com os atributos Atrs(L_i) = {atributos de S_i} $\cup$ $\{ch, a_1, ..., a_n\}$ e ChP(L_i) = ch. Essa opção só funciona para uma especialização cujas subclasses são *totais* (cada entidade na superclasse deve pertencer a (pelo menos) uma das subclas-

ses). Além disso, isso só é recomendado se a especialização tiver a *restrição de disjunção* (ver Seção 8.3.1). Se a especialização for *sobreposta*, a mesma entidade pode ser duplicada em várias relações.

- **Opção 8C: Relação única com um atributo de tipo.** Crie uma única relação L com atributos $Atrs(L) = \{ch, a_1, ..., a_n\} \cup \{\text{atributos de } S_1\} \cup ... \cup \{\text{atributos de } S_m\} \cup \{t\}$ e $ChP(L) = ch$. O atributo t é chamado de atributo de **tipo** (ou **discriminador**), cujo valor indica a subclasse à qual cada tupla pertence, se houver alguma. Essa opção funciona somente para uma especialização cujas subclasses são *disjuntas*, e tem o potencial para gerar muitos valores NULL se diversos atributos específicos existirem nas subclasses.
- **Opção 8D: Relação isolada com atributos de múltiplos tipos.** Crie um único esquema de relação L com atributos $Atrs(L) = \{ch, a_1, ..., a_n\} \cup \{\text{atributos de } S_1\} \cup ... \cup \{\text{atributos de } S_m\} \cup \{t_1, t_2, ..., t_m\}$ e $ChP(L) = ch$. Cada t_i, $1 \leq i \leq m$, é um **atributo de tipo booleano** indicando se uma tupla pertence à subclasse S_i. Essa opção é usada para uma especialização cujas subclasses são *sobrepostas* (mas também funcionará para uma especialização disjunta).

As opções 8A e 8B podem ser chamadas de **opções de relação múltipla**, enquanto as opções 8C e 8D podem ser chamadas de **opções de relação única**. A opção 8A cria uma relação L para a superclasse C e seus atributos, mais uma relação L_i para cada subclasse S_i; cada L_i inclui os atributos específicos (ou locais) de S_i, mais a chave primária da superclasse C, que é propagada para L_i e torna-se sua chave primária. Ela também se torna uma chave estrangeira para a relação da superclasse. Uma operação EQUIJUNÇÃO na chave primária entre qualquer L_i e L produz todos os atributos específicos e herdados das entidades em S_i. Essa opção é ilustrada na Figura 9.5(a) para o esquema EER da Figura 8.4. A opção 8A funciona para quaisquer restrições sobre a especialização: disjuntas ou sobrepostas, totais ou parciais. Observe que a restrição

$$\pi_{<ch>}(L_i) \subseteq \pi_{<ch>}(L)$$

precisa ser mantida para cada L_i. Esta especifica uma chave estrangeira de cada L_i para L, bem como uma *dependência de inclusão* $L_i.ch < L.ch$ (ver Seção 16.5).

Na opção 8B, a operação EQUIJUNÇÃO entre cada subclasse e a superclasse é *embutida* no esquema e a relação L é abolida, conforme ilustra a Figura 9.5(b) para a especialização EER na Figura 8.3(b). Essa opção funciona bem somente quando *ambas* as restrições de disjunção e total são mantidas. Se a especialização não

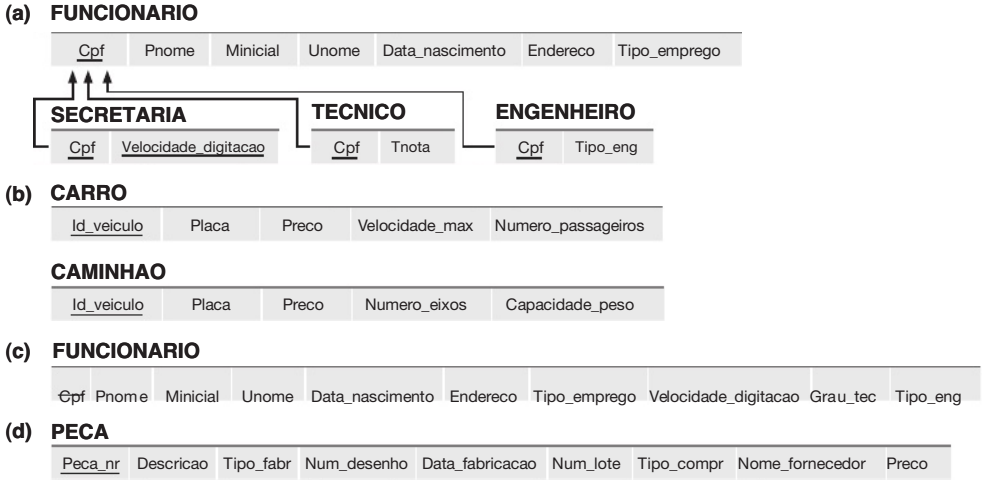


Figura 9.5

Opções para mapeamento de especialização ou generalização. (a) Mapeando o esquema EER na Figura 8.4 ao usar a opção 8A. (b) Mapeando o esquema EER na Figura 8.3(b) ao usar a opção 8B. (c) Mapeando o esquema EER na Figura 8.4 ao usar a opção 8C. (d) Mapeando a Figura 8.5 ao usar a opção 8D com campos de tipo booleano Tipo_fabr e Tipo_compr.

for total, uma entidade que não pertence a qualquer uma das subclasses S_i é perdida. Se a especialização não for disjunta, uma entidade pertencente a mais de uma subclasse terá seus atributos herdados da superclasse C e armazenada de maneira redundante em mais de uma L_i. Com a opção 8B, nenhuma relação mantém todas as entidades na superclasse C; consequentemente, temos de aplicar uma operação UNIÃO EXTERNA (ou JUNÇÃO EXTERNA COMPLETA) (ver Seção 6.4) às relações L_i para recuperar todas as entidades em C. O resultado da união externa será semelhante às relações sob as opções 8C e 8D, exceto que os campos de tipo estarão faltando. Sempre que procurarmos uma entidade arbitrária em C, devemos procurar todas as m relações L_i.

As opções 8C e 8D criam uma única relação para representar a superclasse C e todas as suas subclasses. Uma entidade que não pertence a nenhuma das subclasses terá valores NULL para os atributos específicos dessas subclasses. Essas opções não são recomendadas se muitos atributos específicos forem definidos para as subclasses. Contudo, se houver poucos atributos de subclasse, esses mapeamentos são preferíveis às opções 8A e 8B porque dispensam a necessidade de especificar operações EQUIJUNÇÃO e UNIÃO EXTERNA. Portanto, eles podem produzir uma implementação mais eficiente.

A opção 8C é utilizada para lidar com subclasses disjuntas, incluindo um único **atributo de tipo** (ou de **imagem** ou **discriminador**) t para indicar a qual das m subclasses cada tupla pertence; logo, o domínio de t poderia ser {1, 2, ..., m}. Se a especialização for parcial, t pode ter valores NULL em tuplas que não pertencem a nenhuma subclasse. Se a especialização for definida por atributo, esse atributo que serve à finalidade de t e t não é necessário. Tal opção é ilustrada na Figura 9.5(c) para a especialização EER da Figura 8.4.

A opção 8D é projetada para lidar com subclasses sobrepostas incluindo m campos de **tipo** (ou **flag**) *booleano*, um para *cada* subclasse. Ela também pode ser usada para subclasses disjuntas. Cada campo de tipo t_i pode ter um domínio {sim, não}, em que um valor sim indica que a tupla é um membro da subclasse S_i. Se usarmos essa opção para a especialização EER na Figura 8.4, incluiríamos três atributos de tipo — tipo_secretaria, tipo_engenheiro e tipo_tecnico — no lugar do atributo Tipo_emprego da Figura 9.5(c). Observe que também é possível criar um único atributo de tipo de m *bits* em vez dos m campos de tipo. A Figura 9.5(d) mostra o mapeamento da especialização da Figura 8.5 usando a opção 8D.

Quando temos uma hierarquia ou reticulado de especialização (ou generalização) multinível, não precisamos seguir a mesma opção de mapeamento para todas as especializações. Em vez disso, podemos utilizar uma opção de mapeamento para parte da hierarquia ou reticulado e outras opções para outras partes. A Figura 9.6 mostra um mapeamento possível para as relações do reticulado EER da Figura 8.7. Aqui, usamos a opção 8A para PESSOA/{FUNCIONARIO, EX_ALUNO, ALUNO}, a opção 8C para FUNCIONARIO/{ADMINISTRATIVO, DOCENTE, ALUNO_COLABORADOR}, incluindo o atributo de tipo Tipo_funcionario, e a opção 8D para ALUNO_COLABORADOR/{COLABORADOR_PESQUISA, COLABORADOR_ENSINO} ao incluir os atributos de tipo Tipo_col_ensino e Tipo_col_pesq em FUNCIONARIO, ALUNO/ALUNO_COLABORADOR incluindo os atributos de tipo Tipo_aluno_col em ALUNO, e ALUNO/{ALUNO_POSGRADUACAO, ALUNO_GRADUACAO} incluindo os atributos de tipo tipo_pos e tipo_grad em



Figura 9.6
Mapeando o reticulado de especialização EER da Figura 8.7 usando opções múltiplas.

ALUNO. Na Figura 9.6, todos os atributos cujos nomes iniciam com *tipo* são campos de tipo.

9.2.2 Mapeamento de subclasses compartilhadas (herança múltipla)

Uma subclasse compartilhada, como GERENTE_ENGENHEIRO da Figura 8.6, é uma subclasse de várias superclasses, indicando a herança múltipla. Todas essas classes precisam ter o mesmo atributo-chave; caso contrário, a subclasse compartilhada será modelada como uma categoria (tipo de união), conforme discutimos na Seção 8.4. Podemos aplicar qualquer uma das opções discutidas na etapa 8 a uma subclasse compartilhada, sujeita às restrições discutidas na etapa 8 do algoritmo de mapeamento. Na Figura 9.6, as opções 8C e 8D são utilizadas para a subclasse compartilhada ALUNO_COLABORADOR. A opção 8C é usada na relação FUNCIONARIO (atributo Tipo_funcionario) e a opção 8D, na relação ALUNO (atributo Tipo_aluno_col).

9.2.3 Mapeamento de categorias (tipos de união)

Acrescentamos outra etapa ao procedimento de mapeamento — etapa 9 — para lidar com categorias. Uma categoria (ou tipo de união) é uma subclasse da *união* de duas ou mais superclasses que podem ter diferentes chaves porque podem ser de diferentes tipos de entidade (ver Seção 8.4). Um exemplo é a categoria PROPRIETARIO mostrada na Figura 8.8, que é um subconjunto da união de três tipos de entidade PESSOA, BANCO e EMPRESA. A outra categoria nessa figura, VEICULO_REGISTRADO, tem duas superclasses que possuem o mesmo atributo de chave.

Etapa 9: Mapeamento de tipos de união (categorias). Para o mapeamento de uma categoria cuja definição de superclasses tem chaves diferentes, é comum especificar um novo atributo-chave, chamado **chave substituta**, ao criar uma relação para corresponder à categoria. As chaves das classes de definição são diferentes e, portanto, não podemos usar nenhuma delas exclusivamente para identificar todas as entidades na categoria. Em nosso exemplo da Figura 8.8, criamos uma relação PROPRIETARIO para corresponder à categoria PROPRIETARIO, conforme ilustrado na Figura 9.7, e incluímos alguns atributos da categoria nessa relação. A chave primária da relação PROPRIETARIO é a chave substituta, que chamamos de Id_proprietario. Também incluímos o atributo de chave substituta Id_proprietario como uma chave estrangeira em cada relação correspondente a uma superclasse da categoria,

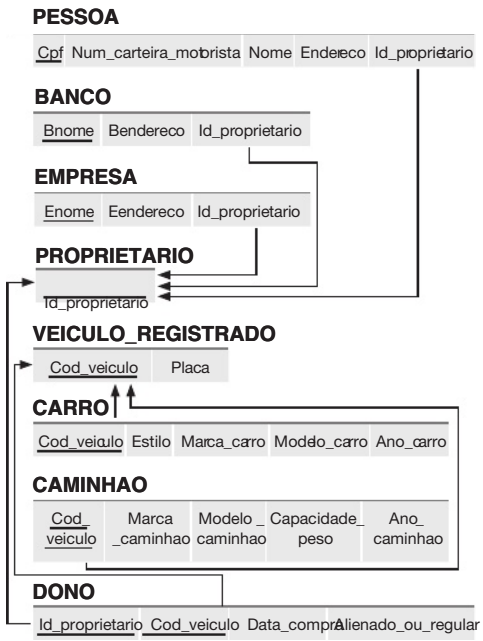


Figura 9.7
Mapeamento das categorias EER (tipos de união) na Figura 8.8 para relações.

para especificar a correspondência nos valores entre a chave substituta e a chave de cada superclasse. Observe que, se determinada entidade PESSOA (ou BANCO ou EMPRESA) não for um membro de PROPRIETARIO, ela teria um valor NULL para seu atributo Id_proprietario em sua tupla correspondente na relação PESSOA (ou BANCO ou EMPRESA), e não teria uma tupla na relação PROPRIETARIO. Também é recomendado acrescentar um atributo de tipo (não mostrado na Figura 9.7) à relação PROPRIETARIO para indicar o tipo de entidade em particular ao qual cada tupla pertence (PESSOA, BANCO ou EMPRESA).

Para uma categoria cujas superclasses têm a mesma chave, como VEICULO na Figura 8.8, não há necessidade de uma chave substituta. O mapeamento da categoria VEICULO_REGISTRADO, que ilustra esse caso, também aparece na Figura 9.7.

Resumo

Na Seção 9.1, mostramos como um projeto de esquema conceitual no modelo ER pode ser mapeado para um esquema de banco de dados relacional. Um algorit-

mo para mapeamento ER para relacional foi dado e ilustrado por meio de exemplos do banco de dados EMPRESA. A Tabela 9.1 resumiu as correspondências entre as construções e restrições do modelo ER e relacional. Em seguida, acrescentamos as etapas adicionais ao algoritmo da Seção 9.2 para mapear as construções do modelo EER para o modelo relacional. Algoritmos semelhantes são incorporados nas ferramentas gráficas de banco de dados para criar um esquema relacional com base em um projeto de esquema conceitual automaticamente.

Perguntas de revisão

9.1. Discuta as correspondências entre as construções do modelo ER e as construções do modelo relacional. Mostre como cada construção do modelo ER pode ser mapeada pa ra o modelo relacional e discuta quaisquer mapeamentos alternativos.

9.2. Discuta as opções para mapear as construções do modelo EER para relações.

Exercícios

9.3. Tente mapear o esquema relacional da Figura 6.14 em um esquema ER. Isso faz parte de um processo conhecido como *engenharia reversa*, em que um esquema conceitual é criado para um banco de dados implementado existente. Indique quaisquer suposições que você fizer.

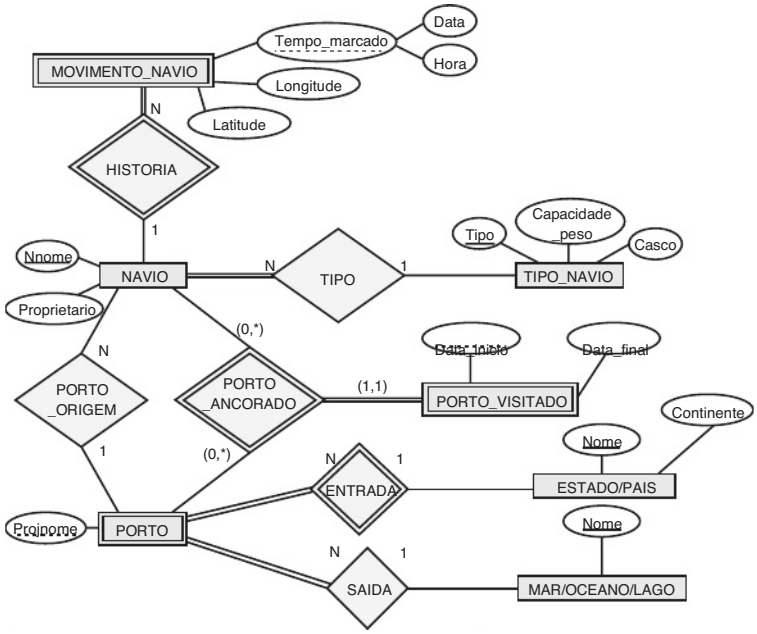


Figura 9.8
Um esquema ER para um banco de dados MOVIMENTO_NAVIO.

9.4. A Figura 9.8 mostra um esquema ER para um banco de dados que pode ser usado para registrar navios de transporte e seus locais para autoridades marítimas. Mapeie esse esquema para um esquema relacional e especifique todas as chaves primárias e estrangeiras.

9.5. Mapeie o esquema ER BANCO do Exercício 7.23 (mostrado na Figura 7.21) em um esquema relacional. Especifique todas as chaves primárias e estrangeiras. Repita para o esquemaCOMPANHIA AEREA (Figura 7.20) do Exercício 7.19 e para os outros esquemas dos exercícios 7.16 a 7.24.

9.6. Mapeie os diagramas EER das figuras 8.9 e 8.12 para esquemas relacionais. Justifique sua escolha de opções de mapeamento.

9.7. É possível mapear com sucesso um tipo de relacionamento binário M:N sem exigir uma nova relação? Por quê?

9.8. Considere o diagrama EER da Figura 9.9 para um revendedor de automóveis.

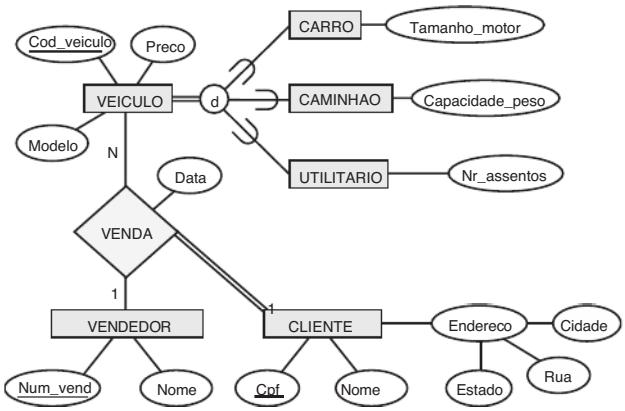


Figura 9.9
Diagrama EER para um revendedor de automóveis.

Mapeie o esquema EER para um conjunto de relações. Para a generalização de VEICULO para CARRO, CAMINHAO, UTILITARIO, considere as quatro opções apresentadas na Seção 9.2.1 e mostre o projeto de esquema relacional sob cada uma dessas opções.

9.9. Usando os atributos que você forneceu para o diagrama EER do Exercício 8.27, mapeie o esquema completo para um conjunto de relações. Escolha uma opção apropriada de 8A até 8D, da Seção 9.2.1, fazendo o mapeamento de generalizações, e defenda sua escolha.

Exercícios de laboratório

9.10. Considere o projeto ER para o banco de dados UNIVERSIDADE que foi modelado usando uma ferramenta como ERwin ou Rational Rose no Exercício de Laboratório 7.31. Utilizando o recurso de geração de esquema SQL da ferramenta de modelagem, gere o esquema SQL para um banco de dados Oracle.

9.11. Considere o projeto ER para o banco de dados PEDIDO_CORREIO que foi modelado usando uma ferramenta como ERwin ou Rational Rose no Exercício de Laboratório 7.32. Utilizando o recurso de geração de esquema SQL da ferramenta de modelagem, gere o esquema SQL para um banco de dados Oracle.

9.12. Considere o projeto ER para o banco de dados REVISAO_CONFERENCIA que foi modelado usando uma ferramenta como ERwin ou Rational Rose no Exercício de Laboratório 7.34. Utilizando o recurso de geração de esquema SQL da ferramenta de modelagem, gere o esquema SQL para um banco de dados Oracle.

9.13. Considere o projeto EER para o banco de dados DIARIO_NOTAS que foi modelado usando uma ferramenta como ERwin ou Rational Rose no Exercício de Laboratório 8.28. Utilizando o recurso de geração de esquema SQL da ferramenta de modelagem, gere o esquema SQL para um banco de dados Oracle.

9.14. Considere o projeto EER para o banco de dados LEILAO_ONLINE que foi modelado usando uma ferramenta como ERwin ou Rational Rose no Exercício de Laboratório 8.29. Utilizando o recurso de geração de esquema SQL da ferramenta de modelagem, gere o esquema SQL para um banco de dados Oracle.

Bibliografia selecionada

O algoritmo de mapeamento ER para relacional original foi descrito no artigo clássico de Chen (1976), que apresentou o modelo ER original. Batini et al. (1992) discutem uma série de algoritmos de mapeamento de modelos ER e EER para modelos legados, e vice-versa.

capítulo

10

Metodologia prática de projeto de banco de dados e uso de diagramas UML

Neste capítulo, saímos dos princípios de projeto de banco de dados apresentados nos capítulos 7 a 9 para examinar alguns de seus aspectos mais práticos. Já descrevemos o material relevante ao projeto dos bancos de dados atuais para aplicações práticas do mundo real. Esse material inclui os capítulos 7 e 8, sobre modelagem conceitual do banco de dados; os capítulos 3 a 6, sobre o modelo relacional, a linguagem SQL e álgebra e cálculo relacional; e o Capítulo 9, sobre mapeamento de um esquema ER ou EER conceitual de alto nível para um esquema relacional. Apresentaremos outros materiais relevantes em capítulos posteriores, incluindo uma visão geral das técnicas de programação para os sistemas relacionais (SGBDRs) nos capítulos 13 e 14, e a teoria de dependência de dados e algoritmos de normalização relacional nos capítulos 15 e 16.

A atividade geral de projeto de banco de dados precisa passar por um processo sistemático chamado **metodologia de projeto**, seja o banco de dados alvo gerenciado por um SGBDR, um sistema de gerenciamento de banco de dados de objeto (SGBDO, ver Capítulo 11), um sistema de gerenciamento de banco de dados objeto-relacional (SGBDOR, ver Capítulo 11) ou algum outro tipo de sistema de gerenciamento de banco de dados. Diversas metodologias de projeto são fornecidas nas ferramentas de projeto de banco de dados atualmente fornecidas pelos vendedores. Ferramentas populares incluem o Oracle Designer e produtos relacionados no Oracle Developer Suite, da Oracle; ERwin e produtos relacionados de CA, PowerBuilder e PowerDesigner, da Sybase; e ER/Studio e produtos relacionados da Embarcadero Technologies, entre muitos outros. Nosso objetivo neste capítulo não é discutir uma metodologia específica, mas sim o projeto de banco de dados em um contexto mais amplo, conforme realizado em grandes organizações para o projeto e implementação de aplicações que atendem a centenas ou milhares de usuários.

Geralmente, o projeto de pequenos bancos de dados com 20 usuários ou menos pode ser relativamente simples. Contudo, para bancos de dados de tamanho médio ou grande, que servem a vários grupos de aplicações diversificadas, cada uma com dezenas ou centenas de usuários, um enfoque sistemático para a atividade geral de projeto de banco de dados torna-se necessário. O simples tamanho de um banco de dados preenchido não reflete a complexidade do projeto; é o esquema dele que é o foco mais importante do projeto. Qualquer banco de dados com um esquema que inclua mais de 20 tipos de entidade e um número semelhante de tipos de relacionamento requer uma metodologia de projeto cuidadosa.

Ao usar o termo **banco de dados grande** para bancos de dados com várias dezenas de gigabytes de dados e um esquema com mais de 30 ou 40 tipos de entidade distintos, podemos cobrir uma grande gama de bancos de dados usados no governo, na indústria e em instituições financeiras e comerciais. Indústrias do setor de serviços, incluindo bancos, hotéis, linhas aéreas, seguro, concessionárias de serviços públicos e comunicações, utilizam bancos de dados para suas operações diárias, 24 horas por dia, 7 dias por semana — conhecido na indústria como operações *24 por 7*. Os sistemas de aplicação para esses bancos de dados são chamados de *sistemas de processamento*

de transação, por causa do grande volume e velocidade de transação exigidos. Neste capítulo, vamos nos concentrar no projeto para bancos de dados em escala média e grande, em que o processamento de transação domina.

Este capítulo tem uma série de objetivos. A Seção 10.1 discute o ciclo de vida do sistema de informação dentro das organizações com uma ênfase particular no sistema de banco de dados. A Seção 10.2 destaca as fases de uma metodologia de projeto de banco de dados dentro do contexto organizacional. A Seção 10.3 introduz alguns tipos de diagramas UML e oferece detalhes sobre as notações que são particularmente úteis na coleta de requisitos e realização do projeto conceitual e lógico dos bancos de dados. Apresentamos um exemplo parcial ilustrando o projeto de um banco de dados de universidade. A Seção 10.4 introduz a ferramenta de desenvolvimento de software popular, chamada Rational Rose, que usa diagramas UML como a principal técnica de especificação. Os recursos da Rational Rose específicos à modelagem de requisitos e projeto de esquema de banco de dados são destacados. A Seção 10.5 discute rapidamente as ferramentas de projeto de banco de dados automatizadas. No final do capítulo há um resumo.

10.1 O papel dos sistemas de informação nas organizações

10.1.1 O contexto organizacional para o uso de sistemas de banco de dados

Os sistemas de banco de dados têm se tornado uma parte dos sistemas de informação de muitas organizações. Historicamente, os sistemas de informação eram dominados por sistemas de arquivos na década de 1960, mas, desde o início dos anos 1970 as organizações passaram para sistemas de gerenciamento de banco de dados (SGBDs) de maneira gradual. Para acomodar os SGBDs, muitas organizações criaram o cargo de administrador de banco de dados (DBA, do inglês *database administrator*) e departamentos de administração de banco de dados para supervisionar e controlar as atividades de seu ciclo de vida. De modo semelhante, os departamentos de tecnologia da informação (TI) e gestão de recursos de informação (GRI) têm sido reconhecidos por grandes organizações como sendo fundamentais para o gerenciamento comercial bem-sucedido pelos seguintes motivos:

- Os dados são considerados um recurso corporativo, e seu gerenciamento e controle, são considerados centrais para o trabalho eficaz da organização.
- Mais funções nas organizações são computadorizadas, aumentando a necessidade de manter grande volume de dados disponíveis em um estado atualizado a cada minuto.
- À medida que a complexidade dos dados e aplicações cresce, relacionamentos complexos entre os dados precisam ser modelados e mantidos.
- Existe uma tendência para a consolidação de recursos de informação em muitas organizações.
- Muitas organizações permitem que seus usuários finais realizem transações de negócios. Isso é evidente em serviços de viagem, serviços financeiros, educação superior, governo e muitos outros tipos de serviços. Essa tendência foi observada desde cedo por pontos de venda de varejo on-line e comércio eletrônico da empresa ao cliente, como eBay e Amazon.com. Nessas organizações, um banco de dados operacional publicamente acessível e atualizável precisa ser projetado e se tornar disponível para as transações do cliente.

Muitas capacidades fornecidas pelos sistemas de banco de dados as tornaram componentes integrais nos sistemas de informação baseados em computador. A seguir estão alguns dos principais recursos que eles oferecem:

- Integração de dados em várias aplicações a um único banco de dados.
- Suporte para o desenvolvimento de novas aplicações em pouco tempo, usando linguagens de alto nível, como a SQL.
- Fornecimento de suporte para acesso casual para navegação e consulta por gerentes enquanto oferece suporte para o processamento principal de transação em nível de produção para os clientes.

Desde o início da década de 1970 até meados dos anos 1980, houve uma mudança na criação de grandes repositórios centralizados de dados gerenciados por um único SGBD. Desde então, a tendência tem sido a utilização de sistemas distribuídos, devido aos seguintes desenvolvimentos:

1. Computadores pessoais e produtos de software para sistema de banco de dados, como Excel, Visual FoxPro, Access (da Microsoft) e SQL Anywhere (da Sybase), e produtos de domínio público, como MySQL e PostgreSQL, estão sendo bastante utilizados por usuários que anteriormente pertenciam à categoria de usuários de banco de dados ca-

suais e ocasionais. Muitos administradores, secretárias, engenheiros, cientistas, arquitetos e alunos pertencem a essa categoria. Como resultado, a prática de criação de **bancos de dados pessoais** está ganhando popularidade. Às vezes, é possível conter uma cópia de parte de um banco de dados grande de um computador mainframe ou um servidor de banco de dados, trabalhar nela de uma estação de trabalho pessoal e depois restaurá-la no mainframe. De modo semelhante, os usuários podem projetar e criar os próprios bancos de dados e depois mesclá-los em um maior.

2. O advento dos SGBDs distribuídos e cliente-servidor (ver Capítulo 25) está abrindo a opção de distribuir o banco de dados por vários sistemas de computação, para melhorar o controle local e agilizar o processamento local. Ao mesmo tempo, os usuários locais podem acessar dados remotos usando as facilidades fornecidas pelo SGBD como um cliente, ou pela Web. Ferramentas de desenvolvimento de aplicação, como PowerBuilder e PowerDesigner (da Sybase) e OracleDesigner e Oracle Developer Suite (da Oracle), estão sendo usadas com facilidades embutidas para ligar aplicações a vários servidores de banco de dados de back-end.

3. Muitas organizações agora utilizam sistemas de **dicionário de dados** ou **repositórios de informações**, que são miniSGBDs que gerenciam **metadados** — ou seja, dados que descrevem a estrutura, as restrições, as aplicações, as autorizações, os usuários do banco de dados, e assim por diante. Estes normalmente são usados como uma ferramenta essencial para gerenciamento de recursos de informação. Um sistema útil de dicionário de dados deve armazenar e gerenciar os seguintes tipos de informação:
   a. Descrições dos esquemas do sistema de banco de dados.
   b. Informações detalhadas sobre o projeto físico do banco de dados, como estruturas de armazenamento, caminhos de acesso e tamanhos de arquivo e registro.
   c. Descrições dos tipos de usuários do banco de dados, suas responsabilidades e seus direitos de acesso.
   d. Descrições de alto nível das transações e aplicações de banco de dados e dos relacionamentos dos usuários com as transações.
   e. O relacionamento entre as transações de banco de dados e os itens de dados referenciados por elas. Isso é útil para determinar quais transações são afetadas quando certas definições de dados são alteradas.
   f. Uso estatístico como frequências de consultas, transações e contadores de acesso para diferentes partes do banco de dados.
   g. A história de quaisquer mudanças feitas no banco de dados e nas aplicações, e a documentação que descreve os motivos para essas mudanças. Isso às vezes é conhecido como **procedência dos dados**.

Esses metadados estão disponíveis aos DBAs, projetistas e usuários autorizados como documentação on-line do sistema. Isso melhora o controle dos DBAs sobre o sistema de informação, bem como seu entendimento e uso pelos usuários. O advento da tecnologia de data warehousing (ver Capítulo 29) destacou a importância dos metadados.

Ao projetar **sistemas de processamento de transação** de alto desempenho, que exigem operação contínua 24 horas por dia, o desempenho torna-se crítico. Esses bancos de dados com frequência são acessados por centenas ou milhares de transações por minuto, de computadores remotos e terminais locais. O desempenho da transação, em relação ao número médio de transações por minuto e o tempo de resposta médio e máximo da transação, é essencial. Um projeto físico de banco de dados cuidadoso, que atende às necessidades de processamento de transação da organização, é uma obrigação em tais sistemas.

Algumas organizações têm comprometido seu gerenciamento de recursos de informação a certos produtos de SGBD e dicionário de dados. Seu investimento no projeto e implementação de sistemas grandes e complexos torna difícil para elas mudarem para produtos de SGBD mais novos, o que significa que as organizações tornaram-se presas a seu sistema atual. Com relação a tais bancos de dados grandes e complexos, não podemos enfatizar de maneira excessiva a importância de um projeto cuidadoso, que leve em conta a necessidade de possíveis modificações do sistema — chamadas de *ajuste* — para responder à mudança de requisitos. Discutiremos o ajuste, juntamente com a otimização da consulta, no Capítulo 21. O custo pode ser muito alto se um sistema grande e complexo não puder evoluir, e torna-se necessário migrar para outros produtos de SGBD e reprojetar o sistema inteiro.

10.1.2 O ciclo de vida do sistema de informação

Em uma grande organização, o sistema de banco de dados costuma fazer parte de um **sistema de informação** (**SI**), que inclui todos os recursos que estão

envolvidos na coleção, gerenciamento, uso e disseminação dos recursos de informação da organização. Em um ambiente computadorizado, esses recursos incluem os próprios dados, o software de SGBD, o hardware do sistema de computação e o meio de armazenamento, o pessoal que usa e gerencia os dados (DBA, usuários finais, e assim por diante), os programas de aplicação (software) que acessam e atualizam os dados, e os programadores que desenvolvem essas aplicações. Assim, o sistema de banco de dados é parte de um sistema de informação organizacional muito maior.

Nesta seção, examinamos o ciclo de vida típico de um sistema de informação e como o sistema de banco de dados se encaixa nele. O ciclo de vida do sistema de informação tem sido chamado **ciclo de vida macro**, enquanto o ciclo de vida do sistema de banco de dados tem sido chamado de **ciclo de vida micro**. A distinção entre eles está se tornando menos pronunciada para os sistemas de informação em que os bancos de dados são um componente essencialmente importante. O *ciclo de vida macro* normalmente inclui as seguintes fases:

1. **Análise de viabilidade.** Essa fase refere-se a analisar áreas de aplicação em potencial, identificar economias da coleta e disseminação de informações, realizar estudos preliminares de custo-benefício, determinar a complexidade de dados e processos, e estabelecer prioridades entre as aplicações.
2. **Levantamento e análise de requisitos.** Os requisitos detalhados são levantados pela interação com os usuários em potencial e grupos de usuários, para identificar seus problemas e necessidades em particular. Procedimentos de dependências entre aplicações, comunicação e relatório são identificados.
3. **Projeto.** Essa fase tem dois aspectos: o projeto do sistema de banco de dados e o projeto dos sistemas de aplicação (programas) que usam e processam o banco de dados por meio de recuperações e atualizações.
4. **Implementação.** O sistema de informação é implementado, o banco de dados é carregado e as transações deste são implementadas e testadas.
5. **Validação e teste de aceitação.** A aceitabilidade do sistema em atender aos requisitos dos usuários e critérios de desempenho é validada. O sistema é testado contra os critérios de desempenho e especificações de comportamento.
6. **Implantação, operação e manutenção.** Isso pode ser precedido pela conversão de usuários de um sistema mais antigo, bem como pelo treinamento do usuário. A fase operacional começa quando todas as funções do sistema estão em funcionamento e foram validadas. À medida que surgem novos requisitos ou aplicações, eles passam pelas fases anteriores até que sejam validados e incorporados ao sistema. O monitoramento do desempenho do sistema e sua manutenção são atividades importantes durante a fase operacional.

10.1.3 O ciclo de vida do sistema de aplicação de banco de dados

As atividades relacionadas ao *ciclo de vida micro*, que focalizam o sistema de aplicação de banco de dados, incluem:

1. **Definição do sistema.** O escopo do sistema de banco de dados, seus usuários e suas aplicações são definidos. As interfaces para diversas categorias de usuários, as restrições do tempo de resposta e as necessidades de armazenamento e processamento são identificadas.
2. **Projeto do banco de dados.** Um projeto lógico e físico completo do sistema de banco de dados no SGBD escolhido é preparado.
3. **Implementação do banco de dados.** Isso compreende o processo de especificar as definições de banco de dados conceituais, externas e internas, criar os arquivos de banco de dados (vazios) e implementar as aplicações de software.
4. **Carga ou conversão de dados.** O banco de dados é preenchido ou pela carga dos dados diretamente ou pela conversão de arquivos existentes para o formato do sistema de banco de dados.
5. **Conversão de aplicação.** Quaisquer aplicações de software de um sistema anterior são convertidas para o novo sistema.
6. **Teste e validação.** O novo sistema é testado e validado. O teste e a validação dos programas de aplicação podem ser um processo bastante complicado, e as técnicas empregadas normalmente são abordadas em cursos de engenharia de software. Existem ferramentas automatizadas que auxiliam nesse processo, mas uma discussão acerca delas está fora do escopo deste livro-texto.

7. **Operação.** O sistema de banco de dados e suas aplicações são colocados em operação. Normalmente, os sistemas antigos e os novos são operados em paralelo por um período de tempo.
8. **Monitoramento e manutenção.** Durante a fase operacional, o sistema é constantemente monitorado e mantido. O crescimento e a expansão podem ocorrer no conteúdo de dados e nas aplicações de software. Importantes modificações e reorganizações podem ser necessárias de tempos em tempos.

As atividades 2, 3 e 4 fazem parte das fases de projeto e implementação do ciclo de vida macro do sistema de informação maior. Nossa ênfase na Seção 10.2 está nas atividades 2 e 3, que cobrem as fases de projeto e implementação de banco de dados. A maioria dos bancos de dados nas organizações passa por todas as atividades anteriores do ciclo de vida. As atividades de conversão (4 e 5) não se aplicam quando o banco de dados e as aplicações são novos. Quando uma organização passa de um sistema estabelecido para um novo, as atividades 4 e 5 tendem a ser muito demoradas e o esforço para realizá-las costuma ser subestimado. Com frequência, existe um retorno entre as várias etapas, pois novos requisitos surgem constantemente a cada estágio. A Figura 10.1 mostra o ciclo de retorno que afeta as fases de projeto conceitual e lógico como resultado da implementação e do ajuste do sistema.

10.2 O projeto de banco de dados e o processo de implementação

Agora, focalizamos as atividades 2 e 3 do ciclo de vida do sistema de aplicação de banco de dados, que são seu projeto e implementação. O problema do projeto de banco de dados pode ser declarado da seguinte forma:

Projetar a estrutura lógica e física de um ou mais bancos de dados para acomodar as informações necessárias dos usuários em uma organização para um conjunto definido de aplicações.

Os objetivos do projeto de banco de dados são múltiplos:

- Satisfazer os requisitos de conteúdo de informação dos usuários e aplicações especificadas.

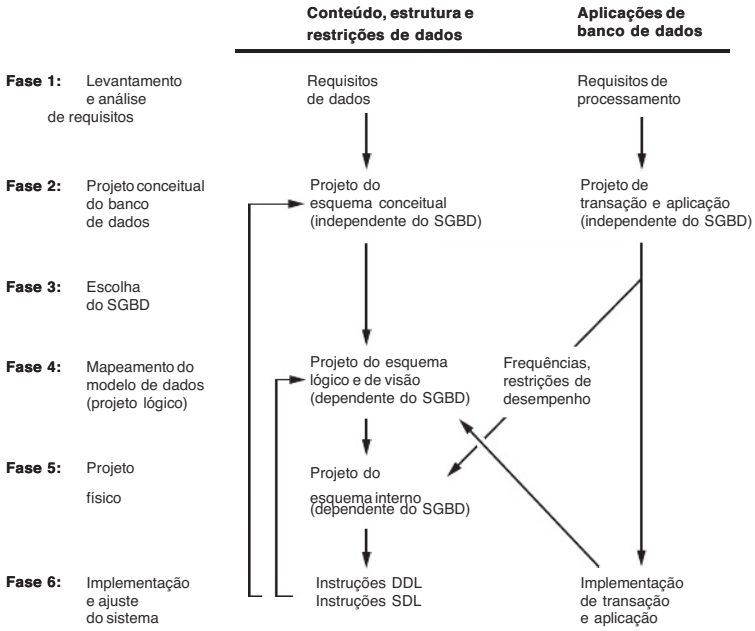


Figura 10.1
Fases de projeto e implementação para grandes bancos de dados.

- Oferecer uma estruturação da informação que seja natural e fácil de entender.
- Dar suporte aos requisitos de processamento e quaisquer objetivos de desempenho, como tempo de resposta, tempo de processamento e espaço de armazenamento.

Esses objetivos são muito difíceis de realizar e medir, e envolvem uma escolha inerente: se alguém tentar obter mais *naturalidade* e *compreensibilidade* do modelo, isso pode custar o desempenho. O problema é agravado porque o processo de projeto de banco de dados normalmente começa com requisitos informais e incompletos. Ao contrário, o resultado da atividade de projeto é um esquema rigidamente definido, que não pode ser facilmente modificado quando o banco de dados é implementado. Podemos identificar seis fases principais do processo geral de projeto e implementação do banco de dados:

1. Levantamento e análise de requisitos.
2. Projeto conceitual do banco de dados.
3. Escolha de um SGBD.
4. Mapeamento do modelo de dados (também chamado de *projeto lógico do banco de dados*).
5. Projeto físico do banco de dados.
6. Implementação e ajuste do sistema de banco de dados.

O processo de projeto consiste em duas atividades paralelas, conforme ilustra a Figura 10.1. A primeira atividade envolve o projeto do **conteúdo de dados, estrutura e restrições** do banco de dados; a segunda relaciona-se ao projeto das **aplicações de banco de dados**. Para manter a figura simples, evitamos mostrar a maioria das interações entre esses lados, mas as duas atividades estão intimamente interligadas. Por exemplo, analisando as aplicações de banco de dados, podemos identificar itens de dados que serão armazenados nele. Além disso, a fase de projeto físico do banco de dados, durante a qual escolhemos as estruturas de armazenamento e os caminhos de acesso dos arquivos de banco de dados, depende das aplicações que usarão esses arquivos para consulta e atualização. Por outro lado, costumamos especificar o projeto das aplicações de banco de dados referindo-nos às construções do seu esquema, as quais são especificadas durante a primeira atividade. Claramente, essas duas atividades influenciam bastante uma à outra. De maneira tradicional, as metodologias de projeto de banco de dados têm focalizado principalmente a primeira dessas atividades, enquanto o projeto de software tem focalizado a segunda; isso pode ser chamado de **projeto controlado por dados** *versus* **controlado por processo**. Agora, é reconhecido por projetistas de banco de dados e engenheiros de software que as duas atividades devem prosseguir lado a lado, e as ferramentas de projeto as combinam cada vez mais.

As seis fases já mencionadas em geral não prosseguem estritamente em sequência. Em muitos casos podemos ter de modificar o projeto de uma fase mais antiga durante outra mais recente. Esses ciclos de retorno entre as fases — e também dentro delas — são comuns. Mostramos apenas alguns dos ciclos de retorno na Figura 10.1, mas existem muitos outros entre várias fases. Também mostramos alguma interação entre os lados dos dados e do processo da figura; na realidade, há mais interações. A Fase 1 na Figura 10.1 envolve o levantamento de informações sobre o uso intencionado do banco de dados, e a Fase 6 trata da implementação e do reprojeto do banco de dados. A parte central do processo de projeto de banco de dados compreende as fases 2, 4 e 5. Vamos resumir essas fases:

- **Projeto conceitual do banco de dados (Fase 2).** O objetivo dessa fase é produzir um esquema conceitual para o banco de dados que seja independente de um SGBD específico. Normalmente, usamos um modelo de dados de alto nível, como o modelo ER ou EER (ver capítulos 7 e 8), durante essa fase. Além disso, especificamos o máximo possível das aplicações ou transações de banco de dados conhecidas, usando uma notação que seja independente de qualquer SGBD específico. Em geral, a escolha do SGBD já é feita para a organização; a intenção do projeto conceitual ainda é mantê-lo o mais livre possível das considerações de implementação.
- **Mapeamento do modelo de dados (Fase 4).** Durante essa fase, que também é chamada **projeto lógico do banco de dados, mapeamos** (ou **transformamos**) o esquema conceitual do modelo de dados de alto nível usado na Fase 2 para o modelo de dados do SGBD escolhido. Podemos começar essa fase após escolhermos um tipo específico de SGBD — por exemplo, se decidirmos usar algum SGBD relacional, mas ainda não tivermos decidido sobre qual deles em particular. Chamamos este de projeto lógico *independente do sistema* (mas *dependente do modelo de dados*). Em relação à arquitetura de SGBD em três níveis, discutida no Capítulo 2, o resultado dessa fase é um *esquema conceitual* no modelo de dados escolhido. Além disso, o projeto dos *esquemas externos* (visões) para aplicações específicas normalmente é elaborado durante essa fase.

- **Projeto físico do banco de dados (Fase 5).** Durante essa fase, projetamos as especificações para o banco de dados armazenado em matéria das estruturas físicas de armazenamento de arquivo, posicionamento de registros e índices. Isso corresponde ao projeto do *esquema interno* na terminologia da arquitetura de SGBD em três níveis.
- **Implementação e ajuste do sistema de banco de dados (Fase 6).** Durante essa fase, o banco de dados e os programas de aplicação são implementados, testados e, por fim, implantados para serviço. Diversas transações e aplicações são testadas individualmente e, depois, em conjunto umas com as outras. Isso normalmente revela oportunidades para as mudanças físicas no projeto, indexação de dados, reorganização e diferente posicionamento de dados — uma atividade conhecida como **ajuste do banco de dados**. O ajuste é uma atividade contínua — uma parte da manutenção do sistema que continua pelo ciclo de vida de um banco de dados, enquanto o banco de dados e as aplicações continuam evoluindo e os problemas de desempenho são detectados.

Vamos discutir cada uma das seis fases do projeto de banco de dados com mais detalhes nas próximas subseções.

10.2.1 Fase 1: levantamento e análise de requisitos[1]

Antes de podermos efetivamente projetar um banco de dados, devemos conhecer e analisar as expectativas dos usuários e os usos intencionados do banco de dados com o máximo de detalhe possível. Esse processo é chamado de **levantamento e análise de requisitos**. Para especificar os requisitos, primeiro identificamos as outras partes do sistema de informação que interagirão com o sistema de banco de dados. Estas incluem usuários e aplicações novas e existentes, cujos requisitos são então coletados e analisados. Normalmente, as seguintes atividades fazem parte dessa fase:

1. As principais áreas de aplicação e grupos de usuário que utilizarão o banco de dados ou cujo trabalho será afetado por ele são identificados. Os principais indivíduos e comitês dentro de cada grupo são escolhidos para executar etapas subsequentes do levantamento e especificação de requisitos.
2. A documentação existente referente às aplicações é estudada e analisada. Outra documentação — manuais de política, formulários, relatórios e gráficos de organização — é revista para determinar se tem qualquer influência sobre o processo de levantamento e especificação de requisitos.
3. O ambiente operacional atual e o uso planejado da informação são estudados. Isso inclui a análise dos tipos de transações e sua frequência, bem como o fluxo de informações dentro do sistema. Características geográficas com relação a usuários, srcem de transações, destino de relatórios, e assim por diante, são estudados. Os dados de entrada e saída para as transações são especificados.
4. Respostas escritas aos conjuntos de perguntas às vezes são coletadas de usuários de banco de dados em potencial ou grupos de usuários. Essas perguntas envolvem as prioridades dos usuários e a importância que eles dão a várias aplicações. Os principais indivíduos podem ser entrevistados para ajudar na avaliação do valor da informação e no estabelecimento de prioridades.

A análise de requisitos é executada para os usuários finais, ou *clientes*, do sistema de banco de dados, por uma equipe de analistas de sistemas ou especialistas em requisitos. É provável que os requisitos iniciais sejam informais, incompletos, inconsistentes e parcialmente incorretos. Portanto, muito trabalho precisa ser feito para transformar esses requisitos iniciais em uma especificação da aplicação, que pode ser usada por desenvolvedores e testadores como ponto de partida para escrever a implementação e os casos de teste. Como os requisitos refletem o conhecimento inicial de um sistema que ainda não existe, eles inevitavelmente mudarão. Portanto, é importante usar técnicas que ajudem os clientes a convergir rapidamente nos requisitos da implementação.

Existe evidência de que a participação do cliente no processo de desenvolvimento aumenta sua satisfação com o sistema entregue. Por esse motivo, muitos profissionais usam reuniões e workshops envolvendo todos os participantes. Uma metodologia para detalhar os requisitos iniciais do sistema é chamada de Joint Application Design (JAD). Mais recentemente, foram desenvolvidas técnicas, como o Projeto Contextual (Contextual Design), que envolvem projetistas inseridos no local de trabalho em que a aplicação deverá

[1] Uma parte desta seção teve a contribuição de Colin Potts.

ser usada. Para ajudar os representantes do cliente a entenderem melhor o sistema proposto, é comum percorrer o fluxo de trabalho, os cenários de transação ou criar um protótipo rápido da aplicação.

Os modos anteriores ajudam a estruturar e detalhar requisitos, mas ainda os deixam em um estado informal. Para transformar os requisitos em uma representação mais bem estruturada, as **técnicas de especificação de requisitos** são usadas. Estes incluem análise orientada a objeto (AOO), diagramas de fluxo de dados (DFD) e o detalhamento dos objetivos da aplicação. Esses métodos usam técnicas diagramáticas para organizar e apresentar requisitos de processamento de informação. A documentação adicional, na forma de texto, tabelas, gráficos e requisitos de decisão, costuma acompanhar os diagramas. Existem técnicas que produzem uma especificação formal que pode ser verificada matematicamente pela consistência e análise simbólica *hipotética*. Esses métodos podem se tornar padrão no futuro para as partes dos sistemas de informação que atendem às funções de missão crítica e que, assim, precisam atuar conforme o planejado. Os métodos de especificação formal baseados em modelo, dos quais a notação e metodologia *Z* é um exemplo proeminente, podem ser considerados extensões do modelo ER e, portanto, são os mais aplicáveis ao projeto do sistema de informação.

Algumas técnicas auxiliadas por computador — chamadas de ferramentas *Upper CASE* — foram propostas para ajudar a verificar a consistência e a integralidade das especificações, que normalmente são armazenadas em um único repositório e podem ser exibidas e atualizadas enquanto o projeto prossegue. Outras ferramentas são usadas para rastrear as ligações entre requisitos e outras entidades de projeto, como módulos de código e casos de teste. Esses *bancos de dados de rastreabilidade* são especialmente importantes em conjunto com os procedimentos impostos de gerenciamento de mudança para sistemas onde os requisitos mudam com frequência. Eles também são usados em projetos contratuais onde a organização de desenvolvimento precisa fornecer evidência documental ao cliente de que todos os requisitos foram implementados.

A fase de levantamento e análise de requisitos pode ser bastante demorada, mas é crucial para o sucesso do sistema de informação. A correção de um erro de requisitos é mais dispendiosa do que a correção de um erro cometido durante a implementação, porque os efeitos de um erro de requisito normalmente são difundidos e, como resultado, é preciso reimplementar muito mais trabalho adiante. Não corrigir um erro significativo quer dizer que o sistema não satisfará o cliente e pode nem sequer ser utilizado. O levantamento e a análise de requisitos são assunto de livros inteiros.

10.2.2 Fase 2: projeto conceitual do banco de dados

A segunda fase do projeto de banco de dados envolve duas atividades paralelas.[2] A primeira atividade, o **projeto do esquema conceitual**, examina os requisitos de dados resultantes da Fase 1 e produz um esquema conceitual do banco de dados. A segunda atividade, o **projeto de transação e aplicação**, examina as aplicações de banco de dados analisadas na Fase 1 e produz especificações de alto nível para essas aplicações.

Fase 2a: Projeto do esquema conceitual. O esquema conceitual produzido por essa fase normalmente é contido em um modelo de dados de alto nível independente do SGBD pelas seguintes razões:

1. O objetivo do projeto do esquema conceitual é um conhecimento completo da estrutura do banco de dados, do significado (semântica), dos inter-relacionamentos e das restrições. Isso é mais bem alcançado independentemente de um SGBD específico, porque cada SGBD com frequência possui particularidades e restrições que não deverão influenciar o projeto do esquema conceitual.
2. O esquema conceitual é valioso como uma *descrição estável* do conteúdo de banco de dados. A escolha do SGBD e, mais tarde, as decisões de projeto podem mudar sem alterar o esquema conceitual independente do SGBD.
3. Um bom conhecimento do esquema conceitual é crucial para os usuários de banco dedados e projetistas de aplicação. O uso de um modelo de dados de alto nível que é mais expressivo e geral do que os modelos de dados dos SGBDs individuais é, portanto, muito importante.
4. A descrição diagramática do esquema conceitual pode servir como um veículo de comunicação entre os usuários do banco de dados, projetistas e analistas. Como os modelos de dados de alto nível normalmente contam com os conceitos que são mais fáceis de entender do que os modelos de dados específicos do SGBD em nível mais baixo, ou definições sintáticas dos dados, qualquer comunicação referente ao projeto de esquema torna-se mais exata e mais direta.

[2] Essa fase do projeto é discutida com mais detalhes nos sete primeiros capítulos de Batini et al. (1992). Resumimos essa discussão aqui.

Nessa fase do projeto de banco de dados, é importante usar um modelo de dados conceitual de alto nível com as seguintes características:

1. **Expressividade.** O modelo de dados deve ser expressivo o suficiente para distinguir diferentes tipos de dados, relacionamentos e restrições.
2. **Simplicidade e compreensão.** O modelo deve ser simples o suficiente para que usuários típicos não especialistas compreendam e usem seus conceitos.
3. **Minimalismo.** O modelo deve ter um número pequeno de conceitos básicos, que são distintos e não sobrepostos no significado.
4. **Representação diagramática.** O modelo deverá ter uma notação diagramática para exibir um esquema conceitual que seja fácil de interpretar.
5. **Formalidade.** Um esquema conceitual expresso no modelo de dados deve representar uma especificação não ambígua formal dos dados. Logo, os conceitos do modelo devem ser definidos com precisão e sem ambiguidade.

Alguns desses requisitos — o primeiro, em particular — às vezes entram em conflito com os outros requisitos. Muitos modelos conceituais de alto nível foram propostos para o projeto de banco de dados (ver a bibliografia selecionada no Capítulo 8).

No Capítulo 8, usamos o modelo EER apresentado no Capítulo 8 e assumiremos que ele está sendo usado nesta fase. O projeto do esquema conceitual, incluindo a modelagem de dados, está se tornando uma parte integral das metodologias de análise e projeto orientadas a objeto. A UML possui diagramas de classes que, em grande parte, são baseados em extensões do modelo EER.

Técnicas para o projeto de esquema conceitual. Para o projeto de esquema conceitual, devemos identificar os componentes básicos (ou construções) do esquema: os tipos de entidade, tipos de relacionamento e atributos. Também devemos especificar atributos-chave, cardinalidade e restrições de participação nos relacionamentos, tipos de entidade fraca e hierarquias/reticulados de especialização/generalização.

Existem duas técnicas para designar o esquema conceitual, que é derivado dos requisitos coletados durante a Fase 1.

A primeira técnica é a **técnica de projeto de esquema centralizado** (ou **única tentativa**), em que os requisitos das diferentes aplicações e grupos de usuários da Fase 1 são mesclados em um único conjunto de requisitos antes que o projeto do esquema comece. Um único esquema correspondente ao conjunto mesclado de requisitos é então projetado. Quando existem muitos usuários e aplicações, mesclar todos os requisitos pode ser uma tarefa árdua e demorada. Supõe-se que uma autoridade centralizada, o DBA, seja responsável por decidir como mesclar os requisitos e projetar o esquema conceitual para o banco de dados inteiro. Uma vez que o esquema conceitual esteja projetado e finalizado, os esquemas externos para diversos grupos de usuários e aplicações podem ser especificados pelo DBA.

A segunda técnica é a **técnica de integração de visão**, em que os requisitos não são mesclados. Em vez disso, um esquema (ou visão) é projetado para cada grupo de usuários ou aplicação com base apenas nos próprios requisitos. Assim, desenvolvemos um esquema de alto nível (visão) para cada grupo de usuários ou aplicação desse tipo. Durante a fase de **integração de visão** subsequente, esses esquemas são mesclados ou integrados em um **esquema conceitual global** para o banco de dados inteiro. As visões individuais podem ser reconstruídas como esquemas externos após a integração da visão.

A principal diferença entre as duas técnicas está na maneira e no estágio em que várias visões ou requisitos dos muitos usuários e aplicações são reconciliados e mesclados. Na técnica centralizada, a reconciliação é feita manualmente pelo DBA antes do projeto de quaisquer esquemas, e aplicada diretamente aos requisitos coletados na Fase 1. Isso coloca o peso para reconciliar as diferenças e conflitos entre os grupos de usuários no DBA. O problema costuma ser tratado usando consultores/especialistas em projeto externos, que aplicam seus métodos específicos para resolver esses conflitos. Devido às dificuldades de gerenciar essa tarefa, a técnica de integração de visão tem sido proposta como uma alternativa.

Na técnica de integração de visão, cada grupo de usuários ou aplicação realmente projeta o próprio esquema conceitual (EER) com base em seus requisitos, com assistência do DBA. Depois, um processo de integração é aplicado a esses esquemas (visões) pelo DBA para formar um esquema integrado global. Embora a integração de visão possa ser feita manualmente, sua aplicação em um grande banco de dados, envolvendo dezenas de grupos de usuários, requer uma metodologia e o uso de ferramentas automatizadas. As correspondências entre os atributos, tipos de entidade e tipos de relacionamento nas diversas visões precisam ser especificadas antes que a integração possa ser aplicada. Adicionalmente, problemas como a integração de visões em conflito e a verificação da consistência das correspondências especificadas entre esquemas precisam ser tratados.

Estratégias para projeto de esquema. Dado um conjunto de requisitos, seja para um único usuário ou para uma grande comunidade de usuários, temos de criar um esquema conceitual que satisfaça tais requisitos. Existem diversas estratégias para projetar tal esquema. A maioria das estratégias segue uma técnica incremental — ou seja, elas começam com algumas construções de esquema importantes derivadas dos requisitos e depois modificam, detalham e constroem incrementalmente sobre elas. Agora, vamos discutir algumas dessas estratégias:

1. **Estratégia de cima para baixo (top-down).** Começamos com um esquema contendo abstrações de alto nível e depois aplicamos sucessivos detalhamentos de cima para baixo. Por exemplo, podemos especificar apenas alguns tipos de entidade de alto nível e, depois, à medida que especificarmos seus atributos, dividi-los em tipos de entidade de nível inferior e especificar os relacionamentos. O processo de especialização para detalhar um tipo de entidade nas subclasses que ilustramos nas seções 8.2 e 8.3 (ver figuras 8.1, 8.4 e 8.5) é outra atividade durante a estratégia de projeto de cima para baixo.
2. **Estratégia de baixo para cima (bottom-up).** Comece com um esquema contendo abstrações básicas e depois combine ou acrescente algo a essas abstrações. Por exemplo, podemos começar com os atributos de banco de dados e agrupá-los em tipos de entidade e relacionamentos. Podemos acrescentar novos relacionamentos entre os tipos de entidade à medida que o projeto prossegue. O processo de generalizar tipos de entidades em superclasses generalizadas de nível mais alto (ver seções 8.2 e 8.3 e Figura 8.3) é outra atividade durante a estratégia de projeto de baixo para cima.
3. **Estratégia de dentro para fora (inside-out).** Esse é um caso especial de uma estratégia de cima para baixo, em que a atenção é focada em um conjunto central de conceitos que são mais evidentes. A modelagem então se espalha para fora, considerando novos conceitos nas vizinhanças dos existentes. Poderíamos especificar alguns tipos de entidade claramente evidentes no esquema e continuar acrescentando outros tipos de entidade e relacionamentos que estão ligados a cada um.
4. **Estratégia mista.** Em vez de seguir qualquer estratégia particular por todo o projeto, os requisitos são particionados de acordo com uma estratégia de cima para baixo, e parte do esquema é projetado para cada partição de acordo com uma estratégia de baixo para cima. As diversas partes do esquema são então combinadas.

As figuras 10.2 e 10.3 ilustram alguns exemplos simples de detalhamento de cima para baixo e de baixo para cima, respectivamente. Um exemplo de detalhamento de cima para baixo primitivo é a decomposição de um tipo de entidade em vários tipos de entidade. A Figura 10.2(a) mostra uma DISCIPLINA sendo detalhada para DISCIPLINA e SEMINARIO, e o relacionamento ENSINA é dividido, da mesma forma, em ENSINA e OFERECE. A Figura 10.2(b) mostra um tipo de entidade OFERTA_DISCIPLINA sendo detalhado para dois tipos de entidade (DISCIPLINA e PROFESSOR) e um relacionamento entre eles. O detalhamento normalmente força um projetista a fazer mais perguntas e extrair mais restrições e detalhes: por exemplo, as razões de cardinalidade (min, max) entre DISCIPLINA e PROFESSOR são obtidas durante o detalhamento. A Figura 10.3(a) mostra o primitivo de detalhamento de baixo pra cima da geração de relacionamentos entre os tipos de entidade DOCENTE e ALUNO. Dois relacionamentos são identificados: ACONSELHA e TUTOR_RESPONSAVEL. O detalhamento de baixo para cima que usa a categorização (tipo de união) é ilustrado na Figura 10.3(b), onde o novo conceito de PROPRIETARIO_VEICULO é descoberto com base nos tipos de entidade existentes DOCENTE, ADMINISTRATIVO e ALUNO. Esse processo de criação de uma categoria e a notação diagramática relacionada seguem o que apresentamos na Seção 8.4.

Integração de esquema (visão). Para grandes bancos de dados, com muitos usuários e aplicações esperadas, pode ser utilizada a técnica de integração de visão do projeto de esquemas individuais para depois mesclá-los. Como as visões individuais podem ser mantidas relativamente pequenas, o projeto dos esquemas é simplificado. Porém, uma metodologia para integrar as visões em um esquema de banco de dados global é necessário. A integração de esquema pode ser dividida nas seguintes subtarefas:

1. **Identificar correspondências e conflitos entre os esquemas.** Como os esquemas são projetados individualmente, é necessário especificar construções nos esquemas que representam o mesmo conceito do mundo real. Essas correspondências devem ser identificadas antes que a integração possa prosseguir. Durante esse processo, vários tipos de conflitos entre os esquemas podem ser descobertos:
   a. **Conflitos de nomes.** Estes são de dois tipos: sinônimos e homônimos. Um **sinônimo** ocorre quando dois esquemas utilizam diferentes nomes para descrever o mesmo conceito. Por exemplo, um tipo de entidade CONSUMIDOR em um esquema pode descrever o mesmo conceito de um tipo de entidade CLIENTE em outro

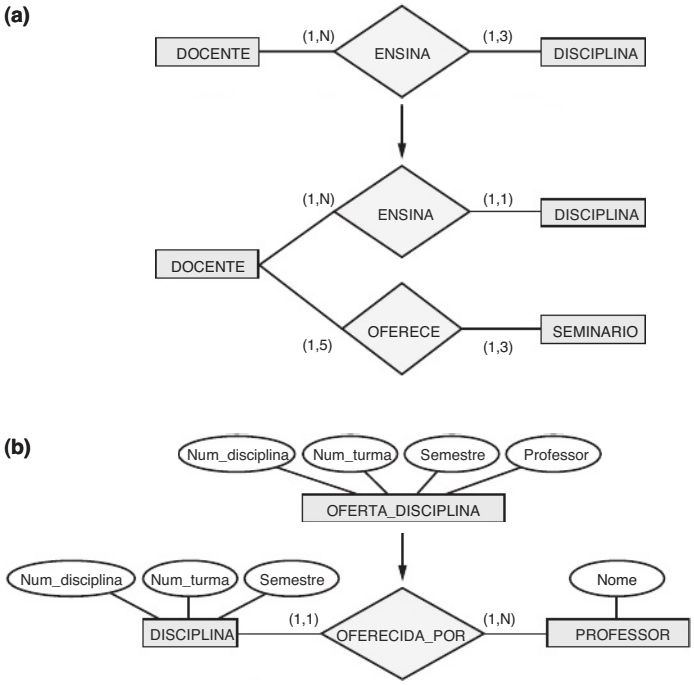


Figura 10.2
Exemplos de detalhamento de cima para baixo. (a) Gerando um novo tipo de entidade. (b) Decompondo um tipo de entidade em dois tipos de entidade e um tipo de relacionamento.

esquema. Um **homônimo** ocorre quando dois esquemas usam o mesmo nome para descrever diferentes conceitos. Por exemplo, um tipo de entidade PECA pode representar peças de computador em um esquema e peças de mobília em outro esquema, por exemplo.

b. **Conflitos de tipo.** O mesmo conceito pode ser representado em dois esquemas por diferentes construções de modelagem. Por exemplo, o conceito de um DEPARTAMENTO pode ser um tipo de entidade em um esquema e um atributo em outro esquema.

c. **Conflitos de domínio (conjunto de valores).** Um atributo pode ter diferentes domínios em dois esquemas. Por exemplo, Cpf pode ser declarado como um inteiro em um esquema e como uma cadeia de caracteres em outro. Um conflito da unidade de medida poderia ocorrer se um esquema representasse Peso em libras e o outro usasse quilogramas.

d. **Conflitos entre restrições.** Dois esquemas podem impor diferentes restrições; por exemplo, uma chave de um tipo de entidade pode ser diferente em cada esquema. Outro exemplo envolve diferentes restrições estruturais em um relacionamento como ENSINA. Um esquema pode representá-lo como 1:N (uma disciplina tem um professor), enquanto outro esquema o representa como M:N (uma disciplina pode ter mais de um professor).

2. **Modificar visões para que se tornem semelhantes.** Alguns esquemas são modificados de modo que sejam ajustados a outros esquemas mais parecidos. Alguns dos conflitos identificados na primeira subtarefa são resolvidos durante essa etapa.

3. **Mesclar visões.** O esquema global é criado pela mescla dos esquemas individuais. Con-

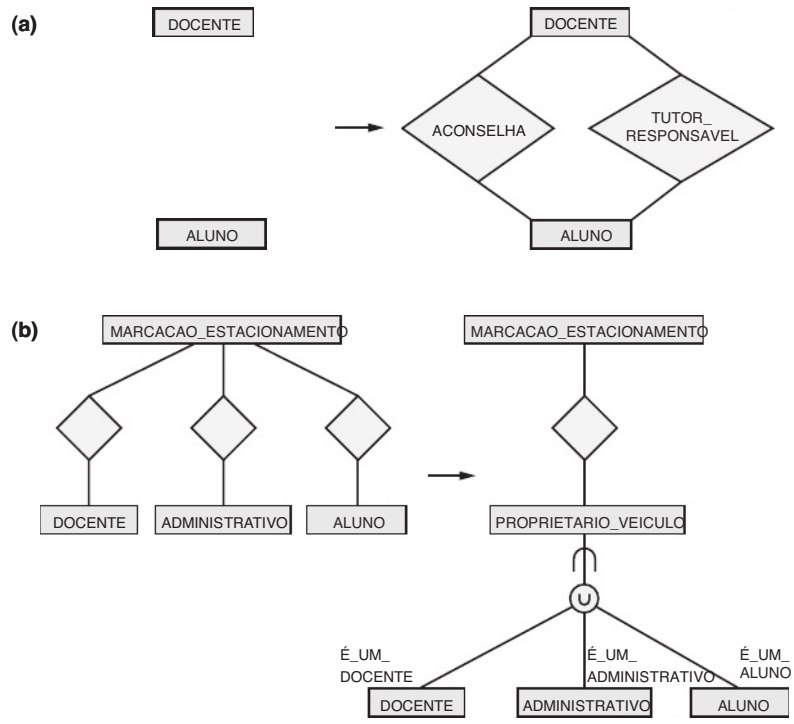


Figura 10.3
Exemplos de detalhamento de baixo para cima. (a) Descobrindo e acrescentando novos relacionamentos. (b) Descobrindo uma nova categoria (tipo de unidade) e relacionando-a.

ceitos correspondentes são representados apenas uma vez no esquema global, e os mapeamentos entre as visões e o esquema global são especificados. Essa é a etapa mais difícil de alcançar nos bancos de dados da vida real envolvendo dezenas ou centenas de entidades e relacionamentos. Ela envolve uma quantidade considerável de intervenção humana e negociação para resolver conflitos e decidir sobre as soluções mais razoáveis e aceitáveis para um esquema global.

4. **Reestruturar.** Como uma última etapa opcional, o esquema global pode ser analisado e reestruturado para remover quaisquer redundâncias ou complexidade desnecessária.

Algumas dessas ideias são ilustradas pelo exemplo relativamente simples apresentado nas figuras 10.4 e 10.5. Na Figura 10.4, duas visões são mescladas para criar um banco de dados bibliográfico. Durante a identificação das correspondências entre as duas visões, descobrimos que PESQUISADOR e AUTOR são sinônimos (em se tratando desse banco de dados), assim como CONTRIBUIDO_POR e ESCRITO_POR. Além disso, decidimos modificar a VISAO 1 a fim de incluir um ASSUNTO para ARTIGO, como mostra a Figura 10.4, *para adequar-se à* VISAO 2. A Figura 10.5 mostra o resultado da mescla de VISAO MODIFICADA 1 com a VISAO 2. Generalizamos os tipos de entidade ARTIGO e LIVRO no tipo de entidade PUBLICACAO, com seu atributo comum Titulo. Os relacionamentos CONTRIBUIDO_POR e ESCRITO_POR são mesclados, assim como os tipos de entidade PESQUISADOR e AUTOR. O atributo Editora só se aplica ao tipo de entidade LIVRO, enquanto o atributo Tamanho e o tipo de relacionamento PUBLICADO_EM só se aplicam a ARTIGO.

Esse exemplo simples ilustra a complexidade do processo de mescla e como o significado dos diversos conceitos precisa ser considerado na simplificação do

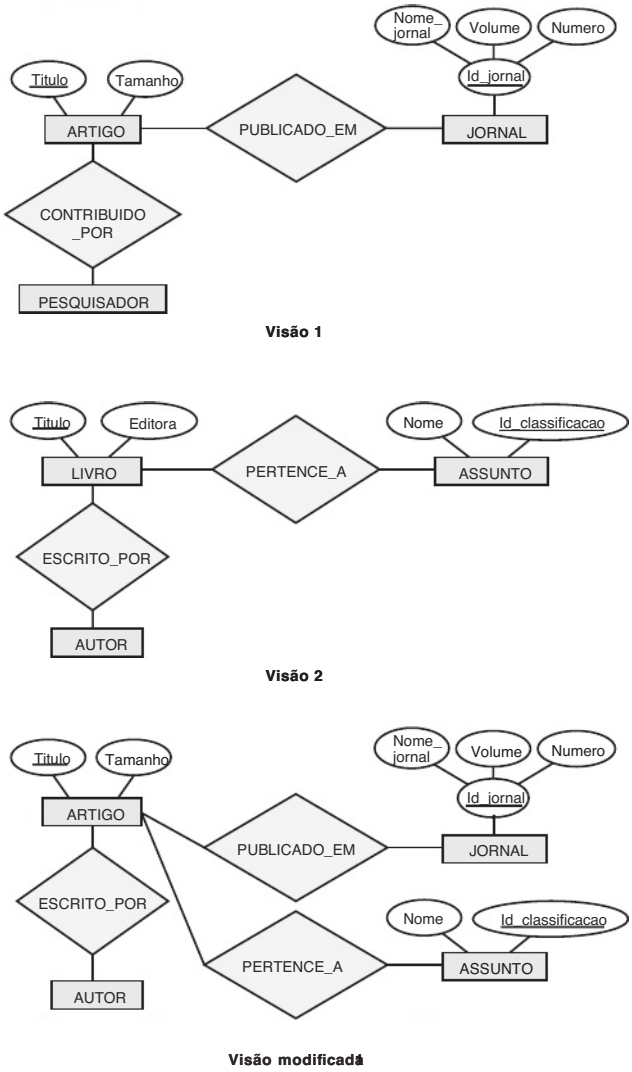


Figura 10.4
Modificando visões para conformidade antes da integração.

projeto de esquema resultante. Para projetos da vida real, o processo de integração de esquema requer uma técnica mais disciplinada e sistemática. Várias estratégias foram propostas para o processo de integração de visão (ver Figura 10.6):

1. **Integração em escala binária.** Dois esquemas muito semelhantes são integrados primeiro. O esquema resultante é então integrado a outro esquema, e o processo é repetido até que todos os esquemas sejam integrados. A orde-

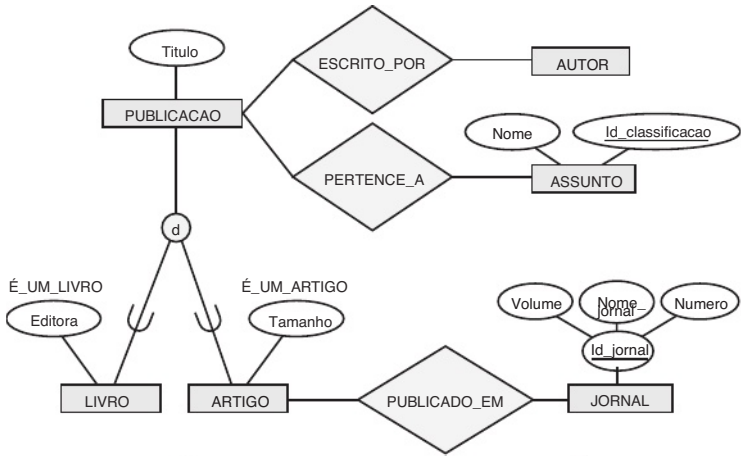


Figura 10.5
Esquema integrado após mesclar visões 1 e 2.

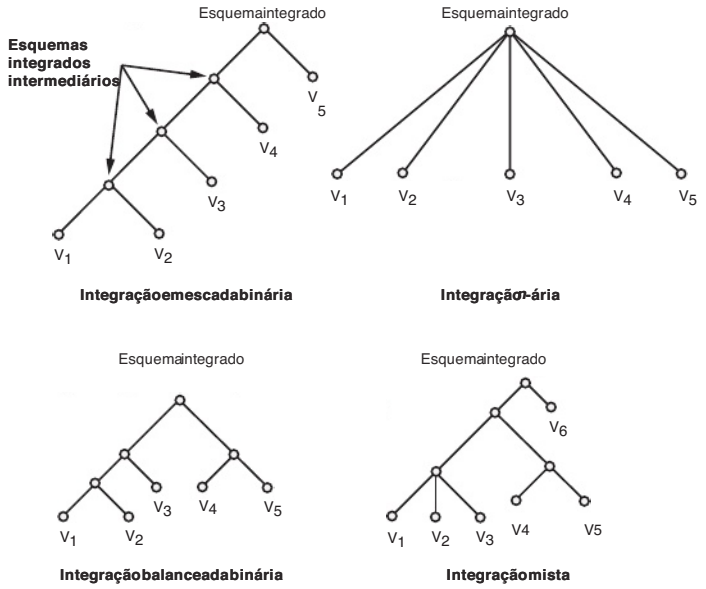


Figura 10.6
Diferentes estratégias para o processo de integração de visão.

nação dos esquemas para integração pode ser baseada em alguma medida de semelhança do esquema. Essa estratégia é adequada para a integração manual, devido a sua técnica passo a passo.

2. **Integração *n*-ária.** Todas as visões são integradas em um procedimento após uma análise e especificação de suas correspondências. Essa estratégia requer ferramentas computadorizadas para grandes problemas de projeto. Tais ferramentas foram montadas como protótipos de pesquisa, mas ainda não estão disponíveis para comercialização.
3. **Estratégia balanceada binária.** Pares de esquemas são integrados primeiro, e depois os esquemas resultantes são emparelhados para maior integração; esse procedimento é repetido até que o resultado seja um esquema global final.
4. **Estratégia mista.** Inicialmente, os esquemas são particionados em grupos com base em sua semelhança, e cada grupo é integrado de maneira separada. Os esquemas intermediários são agrupados de novo e integrados, e assim por diante.

Fase 2b: projeto de transação. A finalidade da Fase 2b, que prossegue em paralelo com a Fase 2a, é projetar as características das transações (aplicações) de banco de dados conhecidas de uma maneira independente do SGBD. Quando um sistema de banco de dados está sendo projetado, os projetistas estão cientes de muitas aplicações (ou **transações**) conhecidas, as quais serão executadas no banco de dados uma vez implementado. Uma parte importante do projeto do banco de dados é especificar as características funcionais dessas transações desde cedo no processo de projeto. Isso garante que o esquema de banco de dados incluirá todas as informações exigidas por essas transações. Além disso, conhecer a importância relativa das diversas transações e as frequências esperadas de suas chamadas desempenha um papel fundamental durante o projeto físico do banco de dados (Fase 5). Normalmente, nem todas as transações do banco de dados são conhecidas durante o projeto. Depois que o sistema de banco de dados estiver implementado, novas transações são continuamente identificadas e implementadas. Porém, as transações mais importantes costumam ser conhecidas antes da implementação do sistema, e devem ser especificadas em um estágio inicial. A regra informal dos *80-20* em geral se aplica nesse contexto: 80 por cento da carga de trabalho é representada por 20 por cento das transações usadas com mais frequência, que controlam o projeto físico do banco de dados. Em aplicações que são de consultas ocasionais ou da variedade de processamento de lote, consultas e aplicações que processam uma quantidade substancial de dados devem ser identificadas.

Uma técnica comum para especificar transações em um nível conceitual é identificar sua **entrada/saída** e o **comportamento funcional**. Ao determinar os parâmetros de entrada e saída (argumentos) e o fluxo de controle funcional interno, os projetistas podem especificar uma transação de uma maneira conceitual e independente do sistema. As transações normalmente podem ser agrupadas em três categorias: (1) **transações de recuperação**, que são usadas para recuperar dados para exibição em uma tela ou imprimir um relatório; (2) **transações de atualização**, que são utilizadas para a inserção de novos dados ou modificação de dados existentes no banco de dados; e (3) **transações mistas**, que são usadas para aplicações mais complexas, que realizam alguma recuperação e alguma atualização. Por exemplo, considere um banco de dados de reservas aéreas. Uma transação de recuperação poderia, primeiro, listar todos os voos matutinos em determinada data entre duas cidades. Uma transação de atualização poderia ser uma reserva de um assento em determinado voo. Uma transação mista poderia, em primeiro lugar, exibir alguns dados, como mostrar uma reserva do cliente em algum voo, e depois atualizar o banco de dados, como ao cancelar a reserva excluindo-a, ou acrescentando um trecho de voo a uma reserva existente. As transações (aplicações) podem ser srcinadas em uma ferramenta de front-end, como o PowerBuilder (da Sybase), que coleta parâmetros on-line e depois envia uma transação ao SGBD, como back-end.[3]

Várias técnicas para especificação de requisitos incluem uma notação para especificar **processos**, os quais nesse contexto são operações mais complexas, que podem consistir em várias transações. Ferramentas de modelagem de processo, como BPwin, bem como ferramentas de modelagem de fluxo de trabalho estão se tornando populares para identificar fluxos de informação nas organizações. A linguagem UML, que provê a modelagem de dados por meio de diagramas de classes e objetos, possui uma série de diagramas de modelagem de processo, incluindo diagramas de transição de estado, de atividades, de sequência e de colaboração. Todos estes se referem a atividades,

[3] Essa filosofia é seguida há mais de 20 anos em produtos populares como CICS, que serve como uma ferramenta para gerar transações para SGBDs legados, como IMS.

eventos e operações dentro do sistema de informação, as entradas e saídas dos processos, os requisitos de sequência ou sincronismo, e outras condições. É possível detalhar essas especificações e extrair transações individuais delas. Outras propostas para especificar transações incluem TAXIS, GALILEO e GORDAS (veja na bibliografia selecionada deste capítulo). Algumas destas foram implementadas em sistemas e ferramentas de protótipo. A modelagem de processos continua sendo uma área de pesquisa ativa.

O projeto da transação é tão importante quanto o projeto do esquema, mas com frequência é considerado parte da engenharia de software, em vez do projeto de banco de dados. Muitas metodologias de projeto atuais enfatizam um sobre o outro. Deve-se passar pelas fases 2a e 2b em paralelo, usando ciclos de retorno para o detalhamento, até que seja alcançado um projeto estável do esquema e das transações.[4]

10.2.3 Fase 3: escolha de um SGBD

A escolha de um SGBD é controlada por uma série de fatores — alguns técnicos, outros econômicos e ainda outros referentes à política da organização. Os fatores técnicos focalizam a adequação do SGBD para a tarefa disponível. As questões a considerar são o tipo de SGBD (relacional, objeto-relacional, objeto, outro), as estruturas de armazenamento e os caminhos de acesso que o SGBD admite, as interfaces de usuário e programador disponíveis, os tipos de linguagens de consulta de alto nível, a disponibilidade de ferramentas de desenvolvimento, a capacidade de interagir com outros SGBDs por meio de interfaces-padrão, as opções arquiteturais relacionadas à operação cliente-servidor, e assim por diante. Os fatores não técnicos incluem o status financeiro e a organização de suporte do vendedor. Nesta seção, concentramo-nos na discussão dos fatores econômicos e organizacionais que afetam a escolha do SGBD. Os seguintes custos devem ser considerados:

1. **Custo de aquisição do software.** Esse é o custo inicial da compra do software, incluindo opções de linguagem de programação, diferentes opções de interface (formulários, menus e ferramentas de interface gráfica com o usuário (GUI) baseadas na Web), opções de recuperação/backup, métodos de acesso especiais e documentação. É preciso selecionar a versão de SGBD correta para um sistema operacional específico. Normalmente, as ferramentas de desenvolvimento, ferramentas de projeto e suporte adicional da linguagem não estão incluídos no preço básico.
2. **Custo de manutenção.** Este é o custo recorrente do recebimento de serviço de manutenção-padrão do vendedor e para manter a versão do SGBD atualizada.
3. **Custo de aquisição de hardware.** Pode ser preciso adquirir novo hardware, como memória adicional, terminais, unidades de disco e controladores, ou dispositivos especializados em armazenamento e arquivamento do SGBD.
4. **Custo de criação ou conversão do banco de dados.** Este é o custo da criação do sistema de banco de dados do zero ou da conversão de um sistema existente para o novo software de SGBD. Neste último caso, é comum operar o sistema existente em paralelo ao novo sistema até que todas as novas aplicações sejam totalmente implementadas e testadas. Esse custo é difícil de projetar e costuma ser subestimado.
5. **Custo de pessoal.** A aquisição de software de SGBD pela primeira vez por uma organização com frequência é acompanhada por uma reorganização do departamento de processamento de dados. Cargos de DBA e seu pessoal existem na maioria das empresas que adotaram SGBDs.
6. **Custo de treinamento.** Como os SGBDs em geral são sistemas complexos, o pessoal deve ser treinado com frequência para usar e programar o SGBD. O treinamento é exigido em todos os níveis, incluindo programação e desenvolvimento de aplicações, projeto físico e administração de banco de dados.
7. **Custo operacional.** O custo da operação continuada do sistema de banco de dados normalmente não é calculado com base em uma avaliação das alternativas porque é contraído independentemente do SGBD selecionado.

Os benefícios da aquisição de um SGBD não são tão fáceis de medir e quantificar. Um SGBD tem diversas vantagens intangíveis em relação aos sistemas de arquivo tradicionais, como facilidade de uso, consolidação de informações de toda a empresa, maior disponibilidade de dados e acesso mais rápido à informação. Com o acesso baseado na Web, certas partes dos dados podem se tornar globalmente acessíveis a funcionários e também a usuários externos. Benefícios mais tangí-

[4] A modelagem de transações de alto nível é abordada em Batini et al. (1992, capítulos 8, 9 e 11). A filosofia de análise funcional e de dados é defendida no decorrer deste livro.

veis incluem redução nos custos de desenvolvimento de aplicação, redução na redundância de dados e melhor controle e segurança. Embora os bancos de dados estejam fortemente estabelecidos na maioria das organizações, a decisão de passar uma aplicação de um enfoque baseado em arquivo para um centralizado no banco de dados ainda precisa ser tomada. Essa mudança geralmente é baseada nos seguintes fatores:

1. **Complexidade dos dados.** À medida que os relacionamentos dos dados se tornam mais complexos, a necessidade de um SGBD é maior.
2. **Compartilhamento entre aplicações.** A necessidade de um SGBD é maior quando as aplicações compartilham dados comuns armazenados de forma redundante em vários arquivos.
3. **Evolução ou crescimento dinâmico dos dados.** Se os dados mudam constantemente, é mais fácil lidar com essas mudanças usando um SGBD do que um sistema de arquivo.
4. **Frequência das solicitações casuais dos dados.** Os sistemas de arquivo não são adequados de forma alguma para a recuperação casual de dados.
5. **Volume e necessidade de controle dos dados.** O grande volume de dados e a necessidade de controlá-los às vezes exige um SGBD.

É difícil desenvolver um conjunto genérico de orientações para adotar uma única técnica para o gerenciamento de dados em uma organização — seja ela relacional, orientada a objeto ou objeto-relacional. Se os dados a serem armazenados no banco de dados tiverem um alto nível de complexidade e lidarem com vários tipos de dados, a técnica típica pode ser considerar um SGBD orientado a objeto ou objeto-relacional.[5] Além disso, os benefícios da herança entre as classes e a consequente vantagem da reutilização favorecem essas técnicas. Por fim, diversos fatores econômicos e organizacionais afetam a escolha de um SGBD em relação a outro:

1. **Adoção em toda a organização de certa filosofia.** Esse costuma ser um fator dominante que afeta a aceitação de determinado modelo de dados (por exemplo, relacional *versus* objeto), de certo vendedor ou certa metodologia e ferramentas de desenvolvimento (por exemplo, o uso de ferramentas e metodologia de análise e projeto orientados a objeto pode ser exigido para todas as novas aplicações).
2. **Familiaridade do pessoal com o sistema.** Se o pessoal de programação da organização estiver familiarizado com determinado SGBD, este pode ser favorecido, para reduzir o custo de treinamento e o tempo de aprendizagem.
3. **Disponibilidade de serviços pelo vendedor.** A disponibilidade de assistência do vendedor na solução de problemas com o sistema é importante, uma vez que passar de um ambiente não SGBD para SGBD em geral é uma grande incumbência, exigindo muita assistência do vendedor no início.

Outro fator a considerar é a portabilidade do SGBD entre diferentes tipos de hardware. Muitos SGBDs comerciais agora possuem versões que rodam em muitas configurações (ou **plataformas**) de hardware/software. A necessidade de aplicações para backup, recuperação, desempenho, integridade e segurança também precisa ser considerada. Muitos SGBDs atualmente estão sendo projetados como *soluções totais* para as necessidades de processamento de informação e gerenciamento de recursos de informação nas organizações. A maioria dos vendedores de SGBD está combinando seus produtos com as seguintes opções ou recursos embutidos:

- Editores de texto e navegadores.
- Geradores de relatórios e utilitários de listagem.
- Softwares de comunicação (em geral chamados de *monitores de teleprocessamento*).
- Recursos de entrada e exibição de dados, como formulários, telas e menus com recursos de edição automáticos.
- Ferramentas de consulta e acesso que podem ser usadas na World Wide Web (ferramentas habilitadas para a Web).
- Ferramentas gráficas para projeto de banco de dados.

Uma grande quantidade de software de *terceiros* está disponível e oferece funcionalidade adicional a um SGBD em cada uma dessas áreas. Em casos raros, pode ser preferível desenvolver software interno em vez de usar um SGBD — por exemplo, se as aplicações forem muito bem definidas e *todas* conhecidas de antemão. Sob tais circunstâncias, um sistema personalizado, criado internamente, pode ser apropriado para implementar as aplicações conhecidas da forma mais eficiente. Porém, na maioria dos casos, novas

[5] Ver, no Capítulo 11, uma discussão a respeito dessa questão.

aplicações que não foram previstas no momento do projeto aparecem *após* a implantação do sistema. É exatamente por isso que os SGBDs se tornaram muito populares: eles facilitam a incorporação de novas aplicações apenas com modificações incrementais ao projeto existente de um banco de dados. Essa evolução de projeto — ou **evolução de esquema** — é um recurso presente em vários graus nos SGBDs comerciais.

10.2.4 Fase 4: mapeamento do modelo de dados (projeto lógico do banco de dados)

A próxima fase do projeto de banco de dados é criar um esquema conceitual e esquemas externos no modelo de dados do SGBD selecionado, mapeando os esquemas produzidos na Fase 2a. O mapeamento pode prosseguir em dois estágios:

1. **Mapeamento independente do sistema.** Nesse estágio, o mapeamento não considera quaisquer características específicas ou casos especiais que se aplicam à implementação do SGBD específico do modelo dedados. Discutimos o mapeamento independente do SGBD de um esquema ER para um esquema relacional na Seção 9.1, e das construções do esquema EER para esquemas relacionais na Seção 9.2.
2. **Ajuste dos esquemas para um SGBD específi-**

   co. Diferentes SGBDs implementam dele modelo de dados usando recursos de modelagem e restrições específicas. Pode ser preciso ajustar os esquemas obtidos na etapa 1 para que corresponda aos recursos de implementação específicos de um modelo de dados conforme usado no SGBD selecionado.

O resultado dessa fase deverá ser comandos DDL (Data Definition Language) na linguagem do SGBD escolhido, que especificam os esquemas no nível conceitual e externo do sistema de banco de dados. Mas, se os comandos DDL incluírem alguns parâmetros do projeto físico, uma especificação DDL completa deve esperar até depois de a fase de projeto de banco de dados estar completada. Muitas ferramentas de projeto CASE (Computer-Aided Software Engineering) (ver Seção 10.5) podem gerar DDL para sistemas comerciais com base em um projeto do esquema conceitual.

10.2.5 Fase 5: projeto físico do banco de dados

O projeto físico do banco de dados é o processo de escolher estruturas específicas de armazenamento de arquivo e caminhos de acesso para os arquivos para alcançar um bom desempenho nas diversas aplicações de banco de dados. Cada SGBD oferece uma série de opções para as organizações de arquivo e caminhos de acesso. Estas normalmente incluem diversos tipos de indexação, agrupamento de registros relacionados em blocos de disco, ligação de registros relacionados por meio de ponteiros e vários tipos de técnicas de hashing (ver capítulos 17 e 18). Quando um SGBD específico é escolhido, o processo de projeto físico do banco de dados fica restrito a escolher as estruturas mais apropriadas para os arquivos de banco de dados dentre as opções oferecidas por esse SGBD. Nesta seção, damos orientações genéricas, mantidas para qualquer tipo de SGBD para decisões do projeto físico. Os critérios a seguir com frequência são usados para orientar a escolha das opções de projeto físico do banco de dados:

1. **Tempo de resposta.** Este é o tempo decorrido entre a submissão de uma transação do banco de dados para execução e o recebimento de uma resposta. Uma influência importante sobre o tempo de resposta que está sob o controle do SGBD é o tempo de acesso ao banco de dados para os itens de dados referenciados pela transação. O tempo de resposta também é influenciado por fatores que não estão sob o controle do SGBD, como carga do sistema, escalonamento do sistema operacional ou atrasos de comunicação.
2. **Utilização de espaço.** Essa é a quantidade de espaço de armazenamento usada pelos arquivos de banco de dados e suas estruturas de caminho de acesso no disco, incluindo índices e outros caminhos de acesso.
3. **Throughput da transação.** Esse é o número médio de transações que podem ser processadas por minuto. É um parâmetro crítico dos sistemas de transação, como aqueles usados para reservas aéreas ou operações bancárias. O throughput da transação precisa ser medido sob condições de pico no sistema.

Em geral, limites médios e comprometidos sobre os parâmetros anteriores são especificados como parte dos requisitos de desempenho do sistema. Técnicas analíticas ou experimentais, que podem incluir protótipo e simulação, são usadas para estimar os valores médio e pior caso sob diferentes decisões de projeto físico, para determinar se elas atendem aos requisitos de desempenho especificados.

O desempenho depende do tamanho e do número de registros no arquivo. Logo, precisamos estimar esses parâmetros para cada arquivo. Além disso, devemos estimar os padrões de atualização e recuperação para o arquivo cumulativamente de todas as transações. Os atributos usados para procurar registros específicos devem ter caminhos de acesso principais e índices secundários construídos para eles. Esti-

mativas do crescimento de arquivo, seja no tamanho do registro, devido a novos atributos, ou no número de registros, também devem ser levadas em consideração durante o projeto físico do banco de dados.

O resultado da fase de projeto físico do banco de dados é uma determinação *inicial* das estruturas de armazenamento e caminhos de acesso para os arquivos. Quase sempre é necessário modificar o projeto com base em seu desempenho observado após o sistema de banco de dados ser implementado. Incluímos essa atividade, **ajuste de banco de dados**, na próxima fase e a abordaremos no contexto da otimização de consulta no Capítulo 20.

10.2.6 Fase 6: implementação e ajuste do sistema de banco de dados

Depois que os projetos lógico e físico forem concluídos, podemos implementar o sistema de banco de dados. Isso normalmente é responsabilidade do DBA e é executado em conjunto com os projetistas de banco de dados. Os comandos da linguagem na DDL, incluindo a SDL (*storage definition language*) do SGBD selecionado, são compilados e usados para criar os esquemas e arquivos de banco de dados (vazios). O banco de dados pode então ser **carregado** (preenchido) com os dados. Caso estes tiverem de ser convertidos de um sistema computadorizado mais antigo, **rotinas de conversão** podem ser necessárias para reformatá-los para a carga no novo banco de dados.

Os programas do banco de dados são implementados pelos programadores de aplicação, consultando as especificações conceituais das transações e depois escrevendo e testando o código do programa com comandos embutidos na DML (*Data Manipulation Language*). Quando as transações estiverem prontas e os dados forem carregados no banco de dados, a fase de projeto e implementação termina e começa a fase operacional do sistema de banco de dados.

A maioria dos sistemas inclui um utilitário de monitoramento para reunir estatísticas de desempenho, que são mantidas no catálogo do sistema ou no dicionário de dados para análise posterior. Estas incluem estatísticas sobre o número de chamadas de transações ou consultas predefinidas, atividade de entrada/saída ao invés de arquivos, contagens de páginas de disco do arquivo ou registros de índice, e frequência de uso do índice. À medida que os requisitos do sistema de banco de dados mudam, com frequência torna-se necessário acrescentar ou remover tabelas existentes e reorganizar alguns arquivos, alterando os métodos de acesso principais ou removendo índices antigos e construindo novos. Algumas consultas ou transações podem ser reescritas para melhorar o desempenho. O ajuste continua enquanto o banco de dados existir, enquanto problemas de desempenho forem descobertos e conforme os requisitos continuarem mudando (ver Capítulo 20).

10.3 Uso de diagramas UML como recurso para especificação de projeto de banco de dados[6]

10.3.1 UML como um padrão de especificação de projeto

Existe uma necessidade de alguma técnica-padrão para cobrir todo o espectro de análise de requisitos, modelagem, projeto, implementação e implantação de bancos de dados e suas aplicações. Uma delas, que está recebendo grande atenção e que também é proposta como um padrão pelo Object Management Group (OMG), é a técnica da linguagem de modelagem unificada (**Unified Modeling Language— UML**). Ela oferece um mecanismo na forma de notação diagramática e sintaxe de linguagem associada para cobrir todo o ciclo de vida. Atualmente, a UML pode ser usada por desenvolvedores de software, modeladores de dados, projetistas de banco de dados, dentre outros, para definir a especificação detalhada de uma aplicação. Eles também a utilizam para determinar o ambiente que consiste em usuários, software, comunicações e hardware para implementar e instalar a aplicação.

A UML combina conceitos comumente aceitos de muitos métodos e metodologias orientados a objeto (O-O) (veja, na bibliografia selecionada deste capítulo, as metodologias que contribuíram para a UML). Ela é genérica, independente de linguagem e independente de plataforma. Com a UML, os arquitetos de software podem modelar qualquer tipo de aplicação, rodando em qualquer sistema operacional, linguagem de programação ou rede. Isso tem tornado a técnica bastante aceitável. Ferramentas como Rational Rose atualmente são populares para desenhar diagramas UML — elas permitem que os desenvolvedores de software façam modelos claros e fáceis de entender para especificar, visualizar, construir e documentar componentes de sistemas de software. Como o escopo da UML se estende para o desenvolvimento de software e aplicação em geral, não abordaremos todos os aspectos dessa linguagem aqui. Nosso objetivo é mostrar algumas notações UML relevantes, que normalmente são usadas na fase de levantamento e análise de requisitos do projeto de banco de dados, bem como na fase de projeto conceitual (ver as

[6] Agradecemos a colaboração de Abrar Ul-Haque às seções de UML e Rational Rose.

fases 1 e 2 na Figura 10.1). A metodologia de desenvolvimento de aplicação detalhada usando UML está fora do escopo deste livro, e pode ser encontrada em diversos livros-texto dedicados ao projeto orientado a objeto, à engenharia de software e UML (veja na bibliografia selecionada ao final deste capítulo).

A UML tem muitos tipos de diagramas. **Diagramas de classes**, que podem representar o resultado final do projeto conceitual do banco de dados, foram discutidos nas seções 7.8 e 8.6. Para chegar aos diagramas de classes, os requisitos da aplicação podem ser coletados e especificados usando **diagramas de casos de uso**, **diagramas de sequência** e **diagramas de estados**. No restante desta seção, vamos apresentar rapidamente os diferentes tipos de diagramas UML, para que o leitor tenha uma ideia do escopo da UML. Depois, vamos descrever um exemplo de uma pequena aplicação para ilustrar o uso de alguns desses diagramas e mostrar como eles levam ao eventual diagrama de classes para o projeto conceitual final do banco de dados. Os diagramas apresentados nesta seção pertencem à notação UML padrão e foram desenhados usando Rational Rose. A Seção 10.4 é dedicada a uma discussão geral do uso da Rational Rose no projeto de aplicações de banco de dados.

10.3.2 UML para o projeto de aplicação de banco de dados

A UML foi desenvolvida como uma metodologia de engenharia de software. Conforme mencionamos anteriormente na Seção 7.8, a maioria dos sistemas de software possui componentes de banco de dados muito grandes. A comunidade de banco de dados começou a aceitar a UML e, agora, alguns projetistas e desenvolvedores a estão usando para modelagem de dados e também para as fases subsequentes do projeto de banco de dados. A vantagem da UML é que, embora seus conceitos sejam baseados nas técnicas orientadas a objeto, os modelos resultantes de estrutura e comportamento podem ser usados para projetar bancos de dados relacionais, orientados a objeto ou objeto-relacionais (veja no Capítulo 11 as definições de bancos de dados orientados a objeto e objeto-relacional).

Uma das principais contribuições da abordagem da UML foi reunir os modeladores, analistas e projetistas de banco de dados tradicionais aos desenvolvedores de aplicação de software. Na Figura 10.1, mostramos as fases do projeto e implementação de banco de dados e como elas se aplicam a esses dois grupos. A UML também nos permite realizar a modelagem comportamental, funcional e dinâmica com a introdução de vários tipos de diagramas. Isso resulta em uma especificação/descrição mais completa da aplicação geral de banco de dados. Nas próximas seções, vamos resumir os diferentes tipos de diagramas UML e depois oferecer um exemplo dos diagramas de casos de uso, sequência e estados em um exemplo de aplicação.

10.3.3 Diferentes tipos de diagramas em UML

A UML define nove tipos de diagramas, divididos em duas categorias:

- **Diagramas estruturais.** Estes descrevem os relacionamentos estruturais ou estáticos entre objetos de esquema, objetos de dados e componentes de software. Incluem de classes, de objetos, de componentes e diagramas de implantação.
- **Diagramas comportamentais.** Sua finalidade é descrever o comportamento ou os relacionamentos dinâmicos entre os componentes. Incluem diagramas de casos de uso, de sequência, de colaboração, de estados e de atividades.

A seguir, apresentamos os nove tipos rapidamente. Os **diagramas estruturais** incluem:

A. Diagramas de classes. Os diagramas de classes capturam a estrutura estática do sistema e atuam como alicerce para outros modelos. Mostram classes, interfaces, colaborações, dependências, generalizações, associações e outros relacionamentos. Eles são um modo muito útil de modelar o esquema conceitual do banco de dados. Mostramos exemplos de diagramas de classes para o banco de dados EMPRESA na Figura 7.16 e para uma hierarquia de generalização na Figura 8.10.

Diagramas de pacotes. Os diagramas de pacotes são um subconjunto dos diagramas de classes. Eles organizam elementos do sistema em grupos relacionados, chamados de **pacotes**. Um pacote pode ser uma coleção de classes relacionadas e os relacionamentos entre elas. Os diagramas de pacotes ajudam a minimizar as dependências em um sistema.

B. Diagramas de objetos. Os diagramas de objetos mostram um conjunto de objetos individuais e seus relacionamentos, e às vezes são referenciados como *diagramas de instâncias*. Eles dão uma visão estática de um sistema em determinado momento e normalmente são usados para testar a precisão dos diagramas de classes.

C. Diagramas de componentes. Os diagramas de componentes ilustram as organizações e as dependências entre os componentes de software. Um diagrama de componentes em geral é composto de componentes, interfaces e relacionamentos de dependência. Um componente pode ser um componente do código fonte, um componente em tempo de execução ou um executável. Ele é um bloco de construção físico

no sistema, sendo representado como um retângulo com dois pequenos retângulos ou abas sobrepostas em seu lado esquerdo. Uma **interface** é um grupo de operações usadas ou criadas por um componente, e normalmente é representada por um pequeno círculo. O relacionamento de dependência é usado para modelar o relacionamento entre dois componentes e é representado por uma setapontilhada que aponta de um componente para outro do qual ele depende. Para bancos de dados, os diagramas de componentes correspondem a dados armazenados, como tablespaces ou partições. As interfaces referem-se a aplicações que utilizam os dados armazenados.

D. Diagramas de implantação. Diagramas de implantação representam a distribuição de componentes (executáveis, bibliotecas, tabelas, arquivos) pela topologia de hardware. Eles representam os recursos físicos em um sistema, incluindo nós, componentes e conexões, e basicamente são usados para mostrar a configuração dos elementos de processamento em tempo de execução (os nós) e os processos de software que residem neles (os threads).

A seguir, descrevemos rapidamente os diversos tipos de **diagramas comportamentais** e expandimos aqueles que são de interesse particular.

E. Diagramas de casos de uso. Os diagramas de casos de uso são utilizados para modelar as interações funcionais entre os usuários e o sistema. Um **cenário** é uma sequência de etapas que descrevem uma interação entre um usuário e um sistema. Um **caso de uso** é um conjunto de cenários que possuem um objetivo comum. O diagrama de casos de uso foi introduzido por Jacobson[7] para visualizar tais casos. Um **diagrama de casos de uso** mostra atores interagindo com os casos de uso e pode ser entendido facilmente sem o conhecimento de qualquer notação. Um caso de uso individual aparece como uma oval e corresponde a uma tarefa específica realizada pelo sistema. Um **ator**, mostrado com um desenho simbólico de uma pessoa, representa um usuário externo, que pode ser um usuário humano, um grupo representativo de usuários, uma função determinada de uma pessoa na organização ou algo externo ao sistema (ver Figura 10.7). O diagrama de casos de uso mostra as possíveis interações do sistema (em nosso caso, um sistema de banco de dados) e descreve as tarefas específicas que o sistema realiza como casos de uso. Como não especificam qualquer detalhe de implementação e supostamente são fáceis de entender, eles são usados como um veículo para a comunicação entre os usuários finais e desenvolvedores para facilitar a validação do usuário em um estágio inicial. Os planos de teste também podem ser descritos com diagramas desse tipo. A Figura 10.7 mostra a notação do diagrama de caso de uso. O relacionamento **include** é utilizado para chegar a algum comportamento comum de dois ou mais dos casos de uso srcinais — ele é uma forma de reutilização. Por exemplo, em um ambiente de universidade mostrado na Figura 10.8, os casos de uso *Registrar para disciplina* e *Inserir notas*, em que os atores aluno e professor estão envolvidos, incluem um caso de uso comum chamado *Validar usuário*. Se um caso de uso incorporar dois ou mais cenários significativamente diferentes, com base em circunstâncias ou condições variáveis, o relacionamento **extend** é usado para mostrar os subcasos conectados ao caso básico.

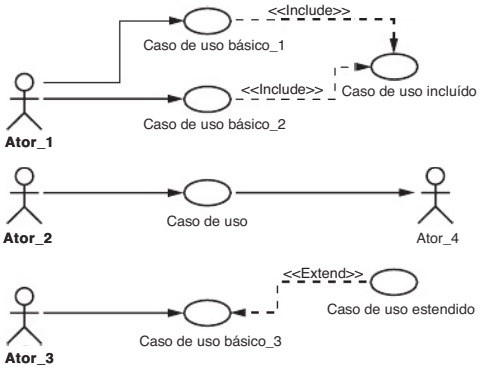


Figura 10.7
A notação do diagrama de casos de uso.

Diagramas de interação. Os dois tipos seguintes de diagramas comportamentais em UML, **diagramas de interação**, são usados para modelar os aspectos dinâ-

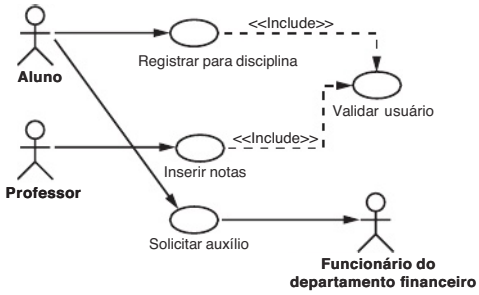


Figura 10.8
Um exemplo de diagrama de casos de uso para um banco de dados UNIVERSIDADE.

[7] Ver Jacobson et al. (1992).

micos de um sistema. Eles consistem em um conjunto de mensagens trocadas entre um conjunto de objetos. Existem dois tipos de diagramas de interação, sequência e colaboração.

F. Diagramas de sequência. Os diagramas de sequência descrevem as interações entre diversos objetos com o tempo. Eles basicamente oferecem uma visão dinâmica do sistema, mostrando o fluxo de mensagens entre objetos. No diagrama de sequência, um objeto ou um ator é mostrado como uma caixa no topo de uma linha vertical tracejada, que é chamada de **linha da vida do objeto**. Para um banco de dados, esse objeto em geral é algo físico — um livro em um depósito que seria representado no banco de dados, um documento ou formulário externo, como um formulário de pedido, ou uma tela visual externa — que pode fazer parte de uma interface com o usuário. A linha da vida representa a existência de um objeto no tempo. A **ativação**, que indica quando um objeto está realizando uma ação, é representada como uma caixa retangular em uma linha da vida. Cada mensagem é representada como uma seta entre as linhas da vida de dois objetos. Uma mensagem tem um nome e pode ter argumentos e informações de controle para explicar a natureza da interação. A ordem das mensagens é lida de cima para baixo. Um diagrama de sequência também dá a opção de *autochamada*, que é basicamente uma mensagem de um objeto para si mesmo. **Marcadores de condição** e **iteração** também podem ser mostrados nos diagramas de sequência para especificar quando a mensagem deve ser enviada e para determinar a condição para envio de múltiplos marcadores. Uma linha tracejada de retorno mostra um retorno da mensagem e é opcional, a menos que carregue um significado especial. A exclusão do objeto é mostrada com um X grande. A Figura 10.9 explica parte da notação usada nos diagramas de sequência.

G. Diagramas de colaboração. Os diagramas de colaboração representam interações entre objetos como uma série de mensagens em sequência. Neles, a ênfase é na organização estrutural dos objetos que enviam e recebem mensagens, ao passo que nos diagramas de sequência a ênfase é na ordenação das mensagens com o tempo. Os diagramas de colaboração mostram objetos como ícones e numeram as mensagens; as mensagens numeradas representam uma ordenação. O layout espacial dos diagramas de colaboração permite ligações entre objetos que mostram seus relacionamentos estruturais. O uso de diagramas de colaboração e sequência para representar interações é uma questão de escolha, visto que eles podem ser utilizados para propósitos semelhantes. Daqui por diante, usaremos apenas diagramas de sequência.

H. Diagramas de estados. Os diagramas de estados descrevem como o estado de um objeto muda em resposta a eventos externos.

Para descrever o comportamento de um objeto, é comum, na maioria das técnicas orientadas a objeto, desenhar um diagrama de estados para mostrar todos os estados possíveis em que um objeto pode entrar em seu tempo de vida. Os diagramas de estados em UML são baseados nos de David Harel.[8] Eles mostram uma máquina de estado que consiste em estados, transições, eventos e ações, e são muito úteis

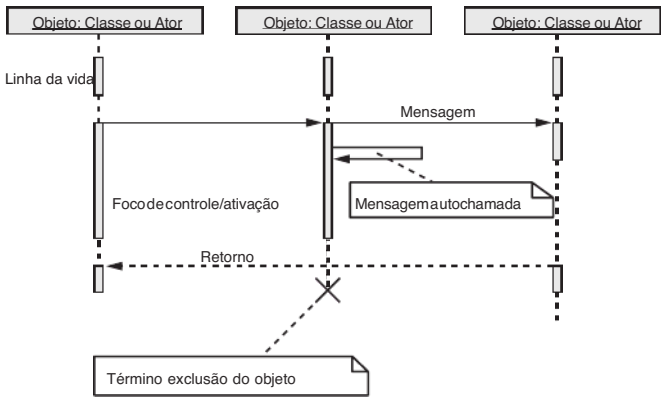


Figura 10.9
A notação do diagrama de sequência.

[8] Ver Harel (1987).

no projeto conceitual da aplicação, que atua contra um banco de dados de objetos armazenados.

Os elementos importantes de um diagrama de estados que aparecem na Figura 10.10 são os seguintes:

- **Estados.** Mostrados como caixas com cantos arredondados, eles representam situações no tempo de vida de um objeto.
- **Transições.** Mostradas como setas sólidas entre os estados, elas representam os caminhos entre diferentes estados de um objeto. Elas são rotuladas pelo nome do evento [condição] / ação; o evento dispara a transição e a ação resulta dele. A guarda é uma condição adicional e opcional, que especifica uma condição sob a qual a mudança de estado pode não ocorrer.
- **Estado inicial.** Mostrado por um círculo sólido com uma seta de saída para um estado.
- **Estado final.** Mostrado com um duplo círculo recebendo uma seta vinda de um estado.

Diagramas de estados são úteis na especificação de como a reação de um objeto a uma mensagem depende de seu estado. Um *evento* é algo feito a um objeto, como receber uma mensagem; uma *ação* é algo que um objeto faz, como enviar uma mensagem.

I. Diagramas de atividades Os diagramas de atividades apresentam uma visão dinâmica do sistema, modelando o fluxo de controle de umaatividade para outra. Eles podem ser considerados fluxogramas com estados. Uma *atividade* é um estado de fazer algo, que poderia ser um processo do mundo real ou uma operação sobre algum objeto ou classe no banco de dados. Normalmente, os diagramas de atividades são usados para modelar o fluxo de trabalho e as operações de negócios internas para uma aplicação.

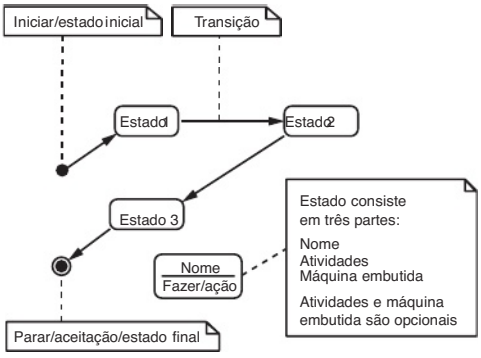


Figura 10.10
A notação do diagrama de estado.

10.3.4 Um exemplo de modelagem e projeto: banco de dados UNIVERSIDADE

Nesta seção, vamos ilustrar rapidamente o uso de alguns dos diagramas UML que já apresentamos para projetar um banco de dados simples em um ambiente de universidade. Muitos detalhes são omitidos para economizar espaço. Ilustramos apenas um uso gradativo desses diagramas, que leva a um projeto conceitual e ao projeto de componentes do programa. Conforme indicamos, o SGBD eventual em que esse banco de dados por fim será implementado pode ser relacional, orientado a objeto ou objeto-relacional. Isso não mudará a análise e modelagem gradual da aplicação usando os diagramas UML.

Imagine um cenário com alunos se matriculando em disciplinas que são oferecidas por professores. O departamento de registro acadêmico é encarregado de manter um programa de disciplinas em um catálogo. Eles têm autoridade para acrescentar e excluir disciplinas, além de realizar mudanças no programa. Também definem limites de matrícula nas disciplinas. O departamento financeiro é encarregado de processar pedidos de auxílio realizados pelos alunos. Suponha que tenhamos de projetar um banco de dados que mantém os dados sobre alunos, professores, disciplinas, auxílio financeiro, e assim por diante. Também queremos projetar algumas das aplicações que nos permitam realizar matrícula na disciplina, processamento de pedido de auxílio financeiro e manutenção do catálogo de disciplinas da universidade pelo departamento de registro acadêmico. Os requisitos anteriores podem ser representados por uma série de diagramas UML.

Conforme mencionamos anteriormente, um dos primeiros passos envolvidos no projeto de um banco de dados é levantar requisitos do cliente utilizando *diagramas de caso de uso*. Suponha que um dos requisitos no banco de dados UNIVERSIDADE seja permitir que os professores incluam notas para as disciplinas que estão lecionando e que os alunos possam se matricular nelas e solicitar auxílio financeiro. O diagrama de caso de uso correspondente a esses casos de uso pode ser desenhado como mostra a Figura 10.8.

Outro elemento útil ao projetar um sistema é representar graficamente alguns dos estados em que o sistema pode estar, para visualizar os diversos estados do sistema no decorrer de uma aplicação. Por exemplo, em nosso banco de dados UNIVERSIDADE, os diversos estados pelos quais o sistema passa quando o registro para uma disciplina com 50 vagas é aberto podem ser representados pelo *diagrama de estados* da Figura 10.11. Este mostra os estados de uma disciplina enquanto a matrícula está em andamento. O

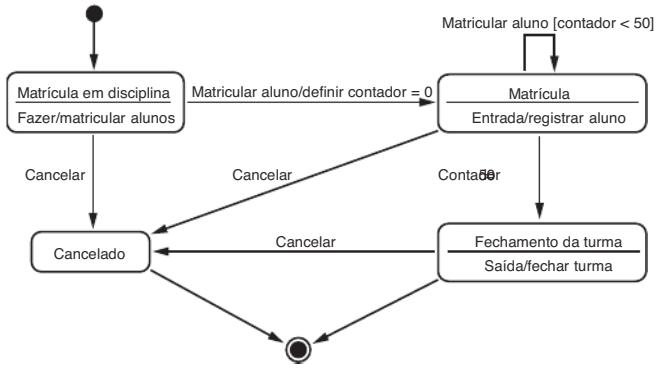


Figura 10.11
Um exemplo de diagrama de estados para o banco de dados UNIVERSIDADE.

primeiro estado define a quantidade de alunos matriculados como zero. Durante o estado de matrícula, a transição *Matricular aluno* continua enquanto a contagem dos alunos matriculados for menor que 50. Quando a contagem atingir 50, o estado para fechar a seção é iniciado. Em um sistema real, estados e/ou transições adicionais poderiam ser acrescentados para permitir que um aluno deixe uma turma e quaisquer outras ações necessárias.

Em seguida, podemos projetar um diagrama de sequência para visualizar a execução dos casos de uso. Para o banco de dados da univers idade, o diagrama de sequência corresponde ao caso de uso: *aluno solicita matrícula e seleciona uma disciplina específica para se matricular* é mostrado na Figura 10.12. O catálogo é pesquisado inicialmente para obter a lista de disciplinas. Depois, quando o aluno seleciona uma disciplina para se matricular, os pré-requisitos e a capacidade da disciplina são verificados, e esta é então acrescentada aos horários do aluno se os pré-requisitos forem atendidos e houver vaga.

Esses diagramas UML *não são* a especificação completa do banco de dados UNIVERSIDADE. Ha-

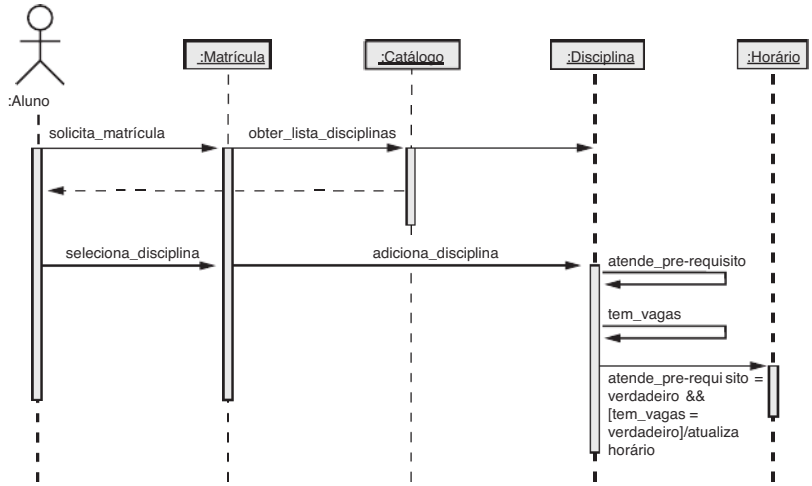


Figura 10.12
Um diagrama de sequência para o banco de dados UNIVERSIDADE.

verá outros casos de uso para as diversas aplicações dos atores, incluindo registro, aluno, professor, e assim por diante. Uma metodologia completa de como chegar aos diagramas de classes para os diversos diagramas que ilustramos nesta seção está fora de nosso escopo. As metodologias de projeto continuam sendo uma questão de bom senso e preferência pessoal. Porém, o projetista deve garantir que o diagrama de classes considerará todas as especificações que foram dadas na forma dos diagramas de casos de uso, estados e sequência. A Figura 10.13 mostra um diagrama de classes possível para essa aplicação, com os relacionamentos estruturais e as operações dentro das classes. Essas classes precisarão ser implementadas para que se desenvolva o banco de dados UNIVERSIDADE e, junto com as operações, elas implementarão a aplicação completa de horário/matrícula/auxílio. Somente alguns dos atributos e métodos (operações) são mostrados na Figura 10.13. É provável que esses diagramas de classes sejam modificados à medida que mais detalhes sejam especificados e mais funções evoluam na aplicação UNIVERSIDADE.

10.4 Rational Rose: uma ferramenta de projeto baseada em UML

10.4.1 Rational Rose para projeto de banco de dados

Rational Rose é uma das ferramentas de modelagem usadas na indústria para desenvolver sistemas de informação. Ela foi adquirida pela IBM em 2003. Conforme indicamos nas duas primeiras seções deste capítulo, um banco de dados é um componente central da maioria dos sistemas de informação. O Rational Rose oferece a especificação inicial em UML que eventualmente leva ao desenvolvimento do banco de dados. Muitas extensões foram feitas nas versões mais recentes do Rose para modelagem de dados, e

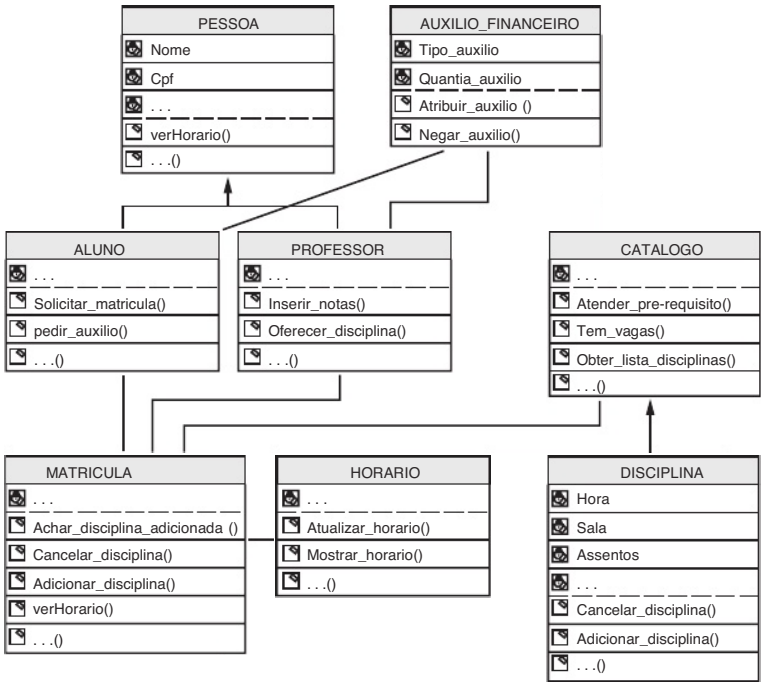


Figura 10.13
O projeto do banco de dados UNIVERSIDADE como um diagrama de classes.

agora ele oferece suporte para modelagem e projeto conceitual, lógico e físico do banco de dados.

10.4.2 Rational Rose Data Modeler

Rational Rose Data Modeler é uma ferramenta de modelagem visual para projetar bancos de dados. Por ser baseado na UML, oferece uma ferramenta e linguagem comuns para unir a lacuna entre projetistas de banco de dados e desenvolvedores de aplicação. Isso permite que projetistas de banco de dados, desenvolvedores e analistas trabalhem juntos, capturem e compartilhem requisitos de negócios, e os acompanhem enquanto mudam no decorrer do processo. Além disso, ao permitir que os projetistas modelem e projetem todas as especificações na mesma plataforma, usando a mesma notação, ele melhora o processo de projeto e reduz o risco de erros.

As capacidades do processo de modelagem no Rational Rose permitem a modelagem do comportamento de aplicações de banco de dados, como vimos no pequeno exemplo anterior, na forma de casos de uso (Figura 10.8), diagramas de sequência (Figura 10.12) e diagramas de estados (Figura 10.11). Existe um mecanismo adicional dos diagramas de colaboração, para mostrar as interações entre objetos, e diagramas de atividades, para modelar o fluxo de controle, que não mostramos em nosso exemplo. O objetivo final é gerar a especificação do banco de dados e o código da aplicação ao máximo possível. O Rose Data Modeler também pode capturar triggers, stored procedures (procedimentos armazenados) e outros conceitos de modelagem explicitamente no diagrama, em vez de representá-los com valores marcados ocultos, nos bastidores (ver Capítulo 26, que discute bancos de dados ativos e triggers). O Rose Data Modeler também oferece a capacidade de realizar *engenharia direta* de um banco de dados, em relação à alteração constante dos requisitos, e *engenharia reversa*, passando de um banco de dados implementado, já existente, para seu projeto conceitual.

10.4.3 Modelagem de dados usando o Rational Rose Data Modeler

Existem muitas ferramentas e opções disponíveis no Rose Data Modeler para modelagem de dados.

Engenharia reversa. A engenharia reversa de um banco de dados permite que o usuário crie um modelo de dados conceitual baseado em um esquema de dados existente, especificado em um arquivo DDL. Podemos usar o assistente de engenharia reversa no Rational Rose Data Modeler para essa finalidade. Tal assistente basicamente lê o esquema no banco de dados ou arquivo DDL e o recria como um modelo de dados. Enquanto isso é feito, ele também inclui o nome de todas as entidades identificadoras citadas.

Engenharia direta e geração de DDL. Também podemos criar um modelo de dados diretamente, a partir do zero, no Rose. Após criar o modelo de dados[9], também podemos usá-lo para gerar a DDL para um SGBD específico. Existe um assistente de engenharia direta no Rose Data Modeler que lê o esquema no modelo de dados ou lê tanto o esquema no modelo de dados como os locais de armazenamento (tablespaces) no modelo de armazenamento de dados, gerando o código DDL apropriado em um arquivo DDL. O assistente também oferece a opção de gerar um banco de dados ao executar o arquivo DDLgerado.

Projeto conceitual em notação UML. O Rational Rose permite a modelagem do banco de dados usando a notação UML. Diagramas ER em geral são usados no projeto conceitual de bancos de dados e pode facilmente ser embutido utilizando a notação UML como diagramas de classes no Rational Rose. Por exemplo, o esquema ER de nosso banco de dados EMPRESA do Capítulo 7 pode ser redesenhado no Rose usando a notação UML, como mostra a Figura 10.14. A especificação textual na Figura 10.14 pode ser convertida para a representação gráfica mostrada na Figura 10.15, com a opção do diagrama de modelo de dados no Rose.

A Figura 10.15 é semelhante à Figura 7.16, com a exceção de estar usando a notação fornecida pelo Rational Rose. Logo, pode ser considerada um diagrama ER usando a notação UML, com a inclusão de métodos e outros detalhes. **Relacionamentos de identificação** especificam que um objeto em uma classe filha (DEPENDENTE na Figura 10.15) não pode existir sem um objeto pai correspondente na classe pai (FUNCIONARIO na Figura 10.15), enquanto **relacionamentos sem identificação** especificam uma associação regular (relacionamento) entre duas classes independentes. É possível atualizar os esquemas diretamente em sua forma textual ou gráfica. Por exemplo, se o relacionamento entre FUNCIONARIO e PROJETO, chamado TRABALHA_EM, fosse excluído, o Rose automaticamente atualizaria ou excluiria todas as chaves estrangeiras nas tabelas relevantes.

Uma diferença importante entre a Figura 10.15 e nossa notação ER anterior, mostrada nos capítulos 7 e 8, é que os atributos de chave estrangeira realmen-

[9] O termo *modelo de dados* utilizado pelo Rational Rose Data Modeler corresponde a nossa noção de um modelo de aplicação ou esquema conceitual.

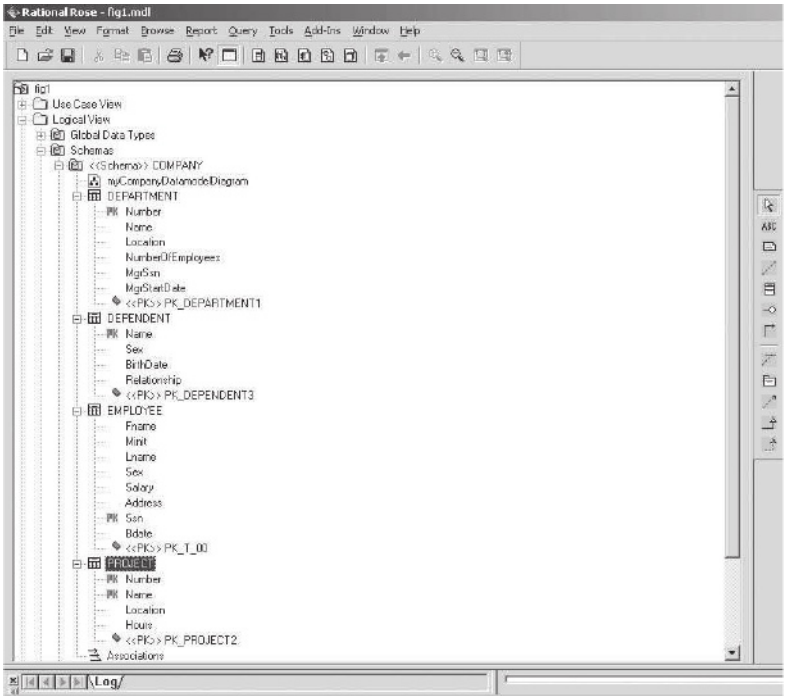


Figura 10.14
Uma definição do diagrama de modelo de dados lógico em Rational Rose.

te aparecem nos diagramas de classes em Rational Rose. Isso é comum em diversas notações diagramáticas, para tornar o projeto conceitual mais próximo da forma como ele é realizado na implementação do modelo relacional. Nos capítulos 7 e 8, os diagramas ER e EER conceituais e os diagramas de classes UML não incluíram os atributos de chave estrangeira, que foram acrescentamos ao esquema relacional durante o processo de mapeamento (ver Capítulo 9).

Convertendo o modelo de dados lógico para o modelo de objeto, e vice-versa. O Rational Rose Data Modeler também oferece a opção de converter um projeto lógico de banco de dados (esquema relacional) para um projeto do modelo de objeto (esquema de objeto), e vice-versa. Por exemplo, o modelo lógico de dados mostrado na Figura 10.14 pode ser convertido para um modelo de objeto. Esse tipo de mapeamento permite um conhecimento profundo dos relacionamentos entre o modelo conceitual e o modelo de implementação, e ajuda a mantê-los atualizados quando forem feitas mudanças em qualquer modelo durante o processo de desenvolvimento. A Figura 10.16 mostra a tabela Funcionario depois de sua conversão para uma classe em um modelo de objeto. As diversas abas na janela podem então ser usadas para inserir/exibir diferentes tipos de informação. Elas incluem operações, atributos e relacionamentos para essa classe.

Sincronismo entre o projeto conceitual e o banco de dados real. O Rose Data Modeler permite que o modelo de dados e a implementação do banco de dados se mantenham sincronizados. Ele permite visualizar tanto o modelo de dados quanto o banco de dados e, depois, com base nas diferenças, oferece a opção de atualizar o modelo ou alterar o banco de dados.

Suporte de domínio extensivo. O Rose Data Modeler permite que os projetistas de banco de dados criem um conjunto-padrão de tipos de dados definidos pelo usuário (estes são semelhantes aos domínios

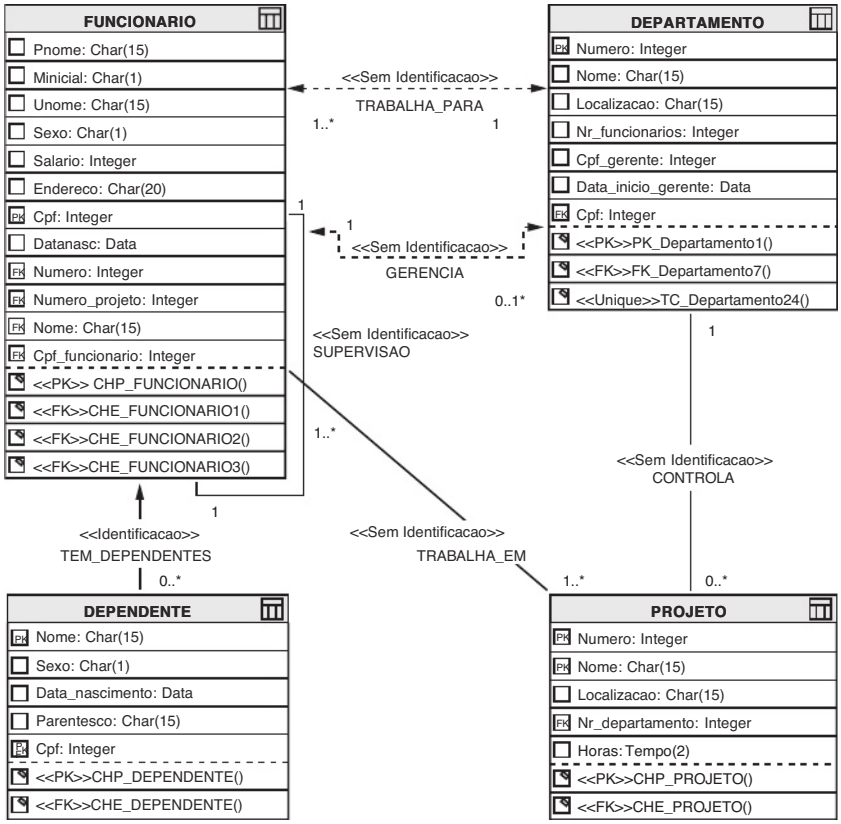


Figura 10.15
Um diagrama gráfico do modelo de dados em Rational Rose para o banco de dados EMPRESA.

em SQL; ver Capítulo 4) e os atribuem a qualquer coluna no modelo de dados. As propriedades do domínio são então propagadas e recebem colunas. Esses domínios podem então ser mantidos por um grupo de padrões e implantados a todos os modeladores quando começam a criar modelos usando o framework do Rational Rose.

Comunicação fácil entre as equipes de projeto . Conforme dissemos, o uso de uma ferramenta comum permite a fácil comunicação entre as equipes. No Rose Data Modeler, um desenvolvedor de aplicação pode acessar tanto o objeto quanto os modelos de dados e ver como eles são relacionados, para então poder tomar decisões melhores e mais inteligentes a respeito de como montar os métodos de acesso aos dados. Há também a opção de usar o *Rational Rose Web Publisher* para permitir que os modelos e os metadados debaixo desses modelos estejam disponíveis a qualquer um na equipe.

O que mostramos aqui é uma descrição parcial das capacidades da ferramenta Rational Rose relacionadas às fases de projeto conceitual e lógico da Figura 10.1. A faixa completa de diagramas UML que descrevemos na Seção 10.3 pode ser desenvolvida e mantida no Rose. Para obter mais detalhes, o leitor deve consultar a documentação do produto. A Figura 10.17 oferece outra versão do diagrama de classes na Figura 7.16, desenhada usando o Rational Rose. A Figura 10.17 difere da Figura 10.15 porque os atributos de chave estrangeira não aparecem de

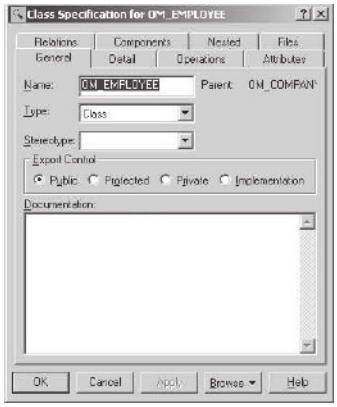


Figura 10.16
A classe OM_EMPLOYEE correspondente à tabela Funcionario da Figura 10.14.

maneira explícita. Logo, a Figura 10.17 está utilizando as notações apresentadas nos capítulos 7 e 8. O Rational Rose permite o uso de qualquer uma dessas opções, dependendo da preferência dos projetistas.

10.5 Ferramentas de projeto automatizado de banco de dados

A atividade de projeto de banco de dados predominantemente se espalha pela Fase 2 (projeto conceitual), Fase 4 (mapeamento do modelo de dados, ou projeto lógico) e Fase 5 (projeto físico do banco de dados) no processo que discutimos na Seção 10.2. A discussão da Fase 5 fica para o Capítulo 20, depois que apresentarmos as técnicas de armazenamento e indexação, e a otimização de consulta. Discutimos as fases 2 e 4 em detalhes com o uso da notação UML na Seção 10.3, e indicamos os recursos da ferramenta Rational Rose, que oferece suporte para essas fases, na Seção 10.4. Como dissemos, o Rational Rose é mais do que apenas uma ferramenta de projeto de banco de dados. É uma ferramenta de desenvolvimento de software e realiza modelagem de banco de dados e projeto de esquema na forma de diagramas de classes como parte de sua metodologia de desenvolvimento de aplicação orientada a objeto. Nesta seção, vamos resumir os recursos e deficiências do conjunto de ferramentas comerciais que focalizam a automação do processo de projeto conceitual, lógico e físico dos bancos de dados.

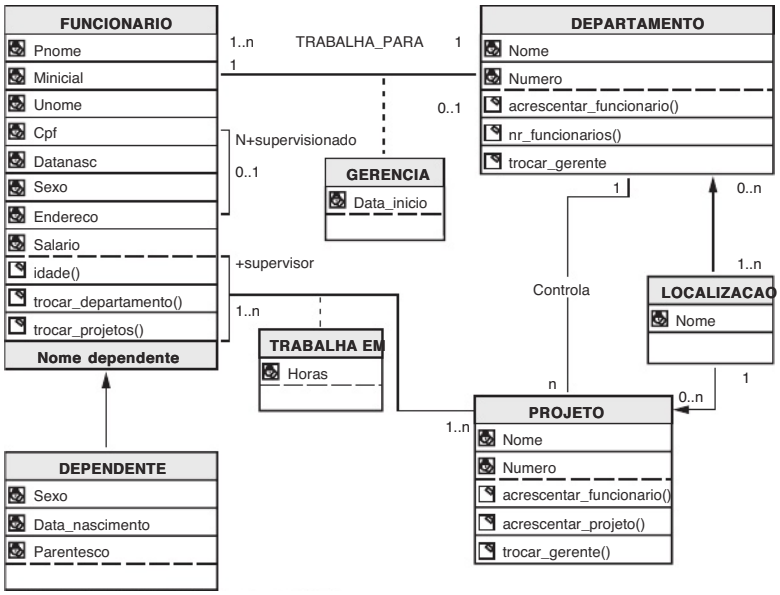


Figura 10.17
O diagrama de classes do banco de dados EMPRESA (Figura 7.16) desenhado no Rational Rose.

Quando a tecnologia de banco de dados foi introduzida, a maior parte do projeto de banco de dados era executada manualmente por projetistas especializados, que usavam sua experiência e conhecimento no processo. Porém, pelo menos dois fatores indicaram que alguma forma de automação teria de ser utilizada, se possível:

1. À medida que uma aplicação envolve mais e mais complexidade de dados em matéria de relacionamentos e restrições, o número de opções ou projetos diferentes para modelar a mesma informação continua aumentando rapidamente. Torna-se difícil lidar com essa complexidade e as consequentes alternativas de projeto de maneira manual.
2. O grande tamanho de alguns bancos de dados depara com centenas de tipos de entidades e de relacionamentos, tornando a tarefa de gerenciar esses projetos manualmente quase impossível. A metainformação relacionada ao processo de projeto, que descrevemos na Seção 10.2, produz outro banco de dados, que deve ser criado, mantido e consultado como um banco de dados por si só.

Esses fatores deram srcem a muitas ferramentas que apareceram sob a categoria geral de ferramenta de engenharia de software auxiliada por computador (**CASE** — *Computer-Aided Software Engineering*) para o projeto de banco de dados. O Rational Rose é um bom exemplo de ferramenta CASE moderna. Normalmente, essas ferramentas consistem em uma combinação das seguintes facilidades:

1. **Diagramação.** Esta permite que o projetista desenhe um diagrama de esquema conceitual em alguma notação específica da ferramenta. A maioria das notações inclui tipos de entidade (classes) e tipos de relacionamento (associações) que são mostrados como caixas separadas ou simplesmente como linhas direcionadas ou não, restrições de cardinalidade, mostradas ao longo das linhas ou em relação aos diferentes tipos de pontas de seta, ou restrições min/max, atributos, chaves, e assim por diante.[10] Algumas ferramentas exigem hierarquias de herança e usam notação adicional para mostrar a natureza parcial-*versus*-total e disjunta-*versus*-sobreposta da especialização/generalização. Os diagramas são armazenados internamente como projetos conceituais e estão disponíveis para modificação, bem como para geração de relatórios, listagens de referência cruzada e outros usos.
2. **Mapeamento do modelo.** Este implementa algoritmos de mapeamento semelhantes aos que apresentamos nas seções 9.1 e 9.2. O mapeamento é específico do sistema — a maioria das ferramentas gera esquemas em SQL DDL para Oracle, DB2, Informix, Sybase e outros SGBDRs. Essa parte da ferramenta é mais receptiva à automação. Se for preciso, o projetista pode editar ainda mais os arquivos DDL produzidos.
3. **Normalização do projeto.** Esta utiliza um conjunto de dependências funcionais que são fornecidas no projeto conceitual ou após os esquemas relacionais serem produzidos durante o projeto lógico. Depois, os algoritmos de decomposição de projeto (ver Capítulo 16) são aplicados para decompor as relações existentes em relações com forma normal mais alta. Geralmente, muitas dessas ferramentas não possuem o enfoque de gerar projetos 3NF ou BCNF alternativos (descritos no Capítulo 15) e permitir que o projetista selecione entre eles com base em alguns critérios, como o número mínimo de relações ou a quantidade mínima de armazenamento.

A maioria das ferramentas incorpora alguma forma de projeto físico, incluindo as escolhas de índices. Há uma faixa inteira de ferramentas separadas para monitoramento e medição do desempenho. O problema de ajustar um projeto ou a implementação do banco de dados ainda é tratadoprincipalmente como uma atividade de tomada de decisão humana. Além das fases do projeto descritas neste capítulo, uma área em que mal existe algum suporte de ferramenta comercial é a integração de visão (ver Seção 10.2.2).

Não vamos analisar aqui as ferramentas de projeto de banco de dados, mas apenas mencionar as características que uma boa ferramenta de projeto deve possuir:

1. **Uma interface fácil de usar.** Isso é essencial, porque permite que os projetistas focalizem a tarefa em mãos, e não o conhecimento da ferramenta. Interfaces gráficas e do tipo apontar-e-clicar costumam ser usadas. Algumas ferramentas, como a de projeto SECSI, utilizam entrada em linguagem natural. Diferentes interfaces podem ser ajustadas para projetistas iniciantes ou especialistas.

[10] Mostramos as notações ER, EER e diagramas de classes UML nos capítulos 7 e 8. Veja, no Apêndice A, uma ideia dos diferentes tipos de notações diagramáticas utilizadas.

2. **Componentes analíticos.** As ferramentas devem oferecer componentes analíticos para tarefas que são difíceis de realizar manualmente, como a avaliação das alternativas de projeto físicas ou a detecção das restrições de conflito entre as visões. Essa área é fraca na maioria das ferramentas atuais.
3. **Componentes heurísticos.** Os aspectos do projeto que não podem ser quantificados com precisão podem ser automatizados pela entrada de regras heurísticas na ferramenta para avaliar projeto alternativo.
4. **Análises de trade-off.** Uma ferramenta deve apresentar ao projetista uma análise comparativa adequada sempre que apresentar várias alternativas para escolha. O ideal é que as ferramentas incorporem uma análise de uma mudança de projeto em nível conceitual, até o projeto físico. Devido às muitas alternativas possíveis para o projeto físico em determinado sistema, tal análise de trade-off é difícil de ser executada, e a maioria das ferramentas atuais a evita.
5. **Exibição dos resultados do projeto.** Os resultados do projeto, como os esquemas, com frequência são exibidos de forma diagramática. Diagramas esteticamente atraentes e bem dispostos não são fáceis de gerar de maneira automática. Layouts de projeto em múltiplas páginas, que sejam fáceis de ler, são outro desafio. Outros tipos de resultados de projeto podem ser mostrados como tabelas, listas ou relatórios, os quais devem ser fáceis de interpretar.
6. **Verificação do projeto.** Esse é um recurso altamente desejável. Sua finalidade é verificar se o projeto resultante satisfaz os requisitos iniciais. A menos que os requisitos sejam capturados e representados internamente de alguma forma que possa ser analisada, não é possível tentar fazer a verificação.

Atualmente, há cada vez mais consciência do valor das ferramentas de projeto, e elas estão se tornando essenciais para lidar com grandes problemas de projeto de banco de dados. Há também uma consciência crescente de que o projeto do esquema e o projeto da aplicação devem seguir lado a lado, e a tendência atual entre as ferramentas CASE é focalizar as duas áreas. A popularidade de ferramentas como o Rational Rose deve-se ao fato de que resolve os dois braços do processo de projeto mostrados na Figura 10.1 simultaneamente, aproximando o projeto do banco de dados e o projeto da aplicação como uma atividade unificada. Após a aquisição da Rational pela IBM em 2003, o pacote de ferramentas Rational tem sido melhorado com ferramentas XDE (*eXtended Development Environment* ou ambiente de desenvolvimento estendido). Alguns vendedores como Platinum (CA) oferecem uma ferramenta para modelagem de dados e projeto de esquema (ERwin) e outra para modelagem de processo e projeto funcional (BPwin). Outras ferramentas (por exemplo, SECSI) utilizam tecnologia de sistema especialista para orientar o processo de projeto, incluindo a habilidade de projeto na forma de regras. A tecnologia de sistema especialista também é útil na fase de levantamento e análise de requisitos, que normalmente é um processo trabalhoso e frustrante. A tendência é usar repositórios de metadados e ferramentas de projeto para obter projetos melhores para bancos de dados complexos. Sem a pretensão de ser completa, a Tabela 10.1 lista algumas ferramentas populares de projeto de banco de dados e modelagem de aplicação. As empresas da tabela são listadas na ordem alfabética.

Resumo

Começamos este capítulo discutindo o papel dos sistemas de informação nas organizações. Os sistemas de banco de dados são considerados uma parte dos sistemas de informação nas aplicações de grande escala. Discutimos como os bancos de dados se encaixam em um sistema de informação para o gerenciamento de recursos de informação em uma organização e o ciclo de vida pelo qual passam.

Depois, discutimos as seis fases do processo de projeto. As três fases normalmente incluídas como parte do projeto de banco de dados são o projeto conceitual, o projeto lógico (mapeamento do modelo de dados) e o projeto físico. Também discutimos a fase inicial do levantamento e análise de requisitos, que costuma ser considerada uma *fase de pré-projeto*. Além disso, em algum ponto durante o projeto, um pacote de SGBD específico precisa ser escolhido. Discutimos alguns dos critérios organizacionais que entram em cena na seleção de um SGBD. À medida que são detectados problemas de desempenho, e quando novas aplicações são acrescentadas, os projetos precisam ser modificados. A importância de projetar o esquema e as aplicações (ou transações) foi destacada. Discutimos diferentes técnicas para o projeto do esquema conceitual e a diferença entre o projeto de esquema centralizado e a técnica de integração de visão.

Introduzimos os diagramas UML como um auxílio para a especificação de modelos e projetos de banco de dados. Apresentamos todos os tipos de diagramas estruturais e comportamentais e depois descrevemos os detalhes de notação sobre os seguintes tipos de diagramas: casos de uso, sequência e estados. (Os diagramas de classes já haviam sido discutidos nas seções 7.8 e 8.6, respectivamente.) Mostramos como alguns dos requisitos para o banco de dados UNIVERSIDADE são especificados usando esses diagramas e podem ser utilizados para desenvolver o projeto conceitual do banco de dados.

Tabela 10.1
Algumas das ferramentas de projeto automatizado de banco de dados atualmente disponíveis.

Empresa	Ferramenta	Funcionalidade
Embarcadero Technologies	ER/Studio DBArtisan	Modelagem de banco de dados em ER e IDEF1x Administração de banco de dados e gerenciamento de espaço e segurança
Oracle	Developer 2000 e Designer 2000	Modelagem de banco de dados, desenvolvimento de aplicação
PersistenceInc.	PowerTier	Mapeamento de modelo O-O para modelo relacional
Platinum Technology (Computer Associates)	Platinum ModelMart, ERwin, BPwin, AllFusion Component Modeler	Modelagem de componentes de dados, processo e negócios
Popkin Software	Telelogic System Architect	Modelagem de dados, modelagem de objetos, modelagem de processos, análise/projeto estruturado
Rational (IBM)	Rational Rose XDE Developer Plus	Modelagem em UML e geração de aplicação em C++ e Java
Resolution Ltd.	XCase	Modelagem conceitual até manutenção de código
Sybase	Enterprise Application Suite	Modelagem de dados, modelagem de lógica de negócios
Visio	Visio Enterprise	Modelagem de dados, projeto e reengenharia Visual Basic e Visual C++

Fornecemos apenas detalhes ilustrativos, e não a especificação completa. Depois, discutimos uma ferramenta de desenvolvimento de software específica — Rational Rose e Rose Data Modeler —, que oferece suporte para as fases de projeto conceitual e projeto lógico do banco de dados. O Rose é uma ferramenta muito mais ampla para o projeto de sistemas de informação em geral. Por fim, discutimos rapidamente a funcionalidade e os recursos desejáveis das ferramentas automatizadas comerciais de projeto de banco de dados, que são mais voltadas para o projeto do banco de dados, ao contrário do Rational Rose. Ao final, mostramos um resumo em forma de tabela de recursos.

Perguntas de revisão

10.1. Quais são as seis fases do projeto de banco de dados? Discuta cada fase.

10.2. Quais das seis fases são consideradas as atividades principais no processo de projeto de banco de dados em si? Por quê?

10.3. Por que é importante projetar os esquemas e as aplicações em paralelo?

10.4. Por que é importante usar um modelo de dados independente da implementação durante o projeto do esquema conceitual? Que modelos são usados nas ferramentas de projeto atuais? Por quê?

10.5. Discuta a importância do levantamento e análise de requisitos.

10.6. Considere a aplicação real de um sistema de banco de dados de seu interesse. Defina os requisitos dos diferentes níveis de usuários em matérias de dados necessários, tipos de consulta e transações a serem processadas.

10.7. Discuta as características que um modelo de dados para o projeto do esquema conceitual deve possuir.

10.8. Compare as duas principais técnicas para o projeto do esquema conceitual.

10.9. Discuta as estratégias para projetar um único esquema conceitual com base em seus requisitos.

10.10. Quais são as etapas da técnica de integração de visão para o projeto do esquema conceitual? Quais são as dificuldades durante cada etapa?

10.11. Como funcionaria uma ferramenta de integração de visão? Crie uma arquitetura modular de exemplo para tal ferramenta.

10.12. Quais são as diferentes estratégias para a integração de visão?

10.13. Discuta os fatores que influenciam a escolha de um pacote de SGBD para o sistema de informação de uma organização.

10.14. O que é o mapeamento do modelo de dados independente do sistema? Como ele se diferencia do mapeamento do modelo de dados dependente do sistema?

10.15. Quais são os fatores importantes que influenciam o projeto físico do banco de dados?

10.16. Discuta as decisões tomadas durante o projeto físico do banco de dados.

10.17. Discuta os ciclos de vida macro e micro de um sistema de informação.

10.18. Discuta as orientações para o projeto físico do banco de dados nos SGBDRs.

10.19. Discuta os tipos de modificações que podem ser aplicadas ao projeto lógico de um banco de dados relacional.

10.20. Quais funções são fornecidas pelas ferramentas típicas de projeto de banco de dados?

10.21. Que tipo de funcionalidade seria desejável nas ferramentas automatizadas para dar suporte ao projeto ideal de grandes bancos de dados?

10.22. Quais são os SGBDs relacionais que dominam o mercado atualmente? Escolha um com que você esteja acostumado e mostre como ele é avaliado com base nos critérios dispostos na Seção 10.2.3.

10.23. Uma DDL possível, correspondente à Figura 3.1, é a seguinte:

```
CREATE TABLE ALUNO (
Nome                VARCHAR(30)    NOT NULL,
Cpf                 CHAR(9)        PRIMARY KEY,
Telefone_residencial VARCHAR(14),
Endereco            VARCHAR(40),
Telefone_comercial  VARCHAR(14),
Idade               INT,
Media               DECIMAL(4,3)
);
```

Discuta as seguintes decisões de projeto:

- **a.** A escolha de exigir que Nome seja NOT NULL.
- **b.** A seleção de Cpf como a PRIMARY KEY.
- **c.** A escolha de tamanhos e precisão dos campos.
- **d.** Qualquer modificação dos campos definidos neste banco de dados.
- **e.** Quaisquer restrições sobre campos individuais.

10.24. Que convenções de nomeação você poderia desenvolver para ajudar a identificar as chaves estrangeiras com mais eficiência?

10.25. Que funções são oferecidas pelas ferramentas de projeto de banco de dados típicas?

Bibliografia selecionada

Existe muita literatura sobre projeto de banco de dados. Primeiro, listamos alguns dos livros que tratam desse assunto. Batini et al. (1992) fazem um tratamento abrangente do projeto conceitual e lógico do banco de dados. Wiederhold (1987) aborda todas as fases do projeto de banco de dados, com ênfase no projeto físico. O'Neil (1994) tem uma discussão detalhada sobre o projeto físico e as questões de transação em referência a SGBDRs comerciais. Um grande acervo de trabalho sobre modelagem e projeto conceitual foi feito na década de 1980. Brodie et al. (1984) oferecem uma coleção de capítulos sobre modelagem conceitual, especificação e análise de restrição e projeto de transação. Yao (1985) apresenta uma coleção de trabalhos que variam de técnicas de especificação de requisitos à reestruturação de esquema. Teorey (1998) enfatiza a modelagem EER e discute diversos aspectos do projeto conceitual e lógico do banco de dados. Hoffer et al. (2009) trazem uma boa introdução às questões de aplicações de negócios do gerenciamento de banco de dados.

Navathe e Kerschberg (1986) discutem todas as fases do projeto de banco de dados e apontam o papel de dicionário de dados. Goldfine e Konig (1988) e ANSI (1989) discutem o papel dos dicionários de dados no projeto de banco de dados. Rozen e Shasha (1991) e Carlis e March (1984) apresentam diferentes modelos para o problema de projeto físico de banco de dados. A análise e o projeto orientados a objeto são discutidos em Schlaer e Mellor (1988), Rumbaugh et al. (1991), Martin e Odell (1991) e Jacobson et al. (1992). Livros recentes de Blaha e Rumbaugh (2005) e de Martin e Odell (2008) consolidam as técnicas existentes na análise e projeto orientados a objeto usando UML. Fowler e Scott (2000) fazem uma introdução rápida à UML. Para ver um tratamento abrangente da UML e seu uso no processo de desenvolvimento de software, consulte Jacobson et al. (1999) e Rumbaugh et al. (1999).

O levantamento e análise de requisitos é um tópico bastante pesquisado. Chatzoglu et al. (1997) e Lubars et al. (1993) apresentam estudos das práticas atuais em levantamento, modelagem e análise de requisitos. Carroll (1995) oferece um conjunto de leituras sobre o uso de cenários para levantamento de requisitos nos estágios iniciais do desenvolvimento do sistema. Wood e Silver (1989) oferecem uma boa visão geral do processo oficial de Joint Application Design (JAD). Potter et al. (1991) descrevem a notação Z e a metodologia para a especificação formal do software. Zave (1997) classificou os esforços de pesquisa na engenharia de requisitos.

Um grande acervo de trabalho tem sido produzido sobre os problemas de integração de esquema e visão, que está se tornando particularmente relevante agora por causa da necessidade de integrar uma série de bancos de dados existentes. Navathe e Gadgil (1982) definiram técnicas para a integração de visão. As metodologias de integração de esquema são comparadas em Batini et al. (1987). O trabalho detalhado sobre integração de visão binária pode ser encontrado em Navathe et al. (1986), Elmasri et al. (1986) e Larson et al. (1989). Uma ferramenta de integração baseada em Elmasri et al. (1986) é descrita em Sheth et al. (1988). Outro sistema de integração de visão é discutido em Hayne e Ram (1990). Casanova et al. (1991) descrevem uma ferramenta para o projeto modular de banco de dados. Motro (1987) discute a integração com relação aos bancos de dados pré-existentes. A estratégia balanceada binária para a integração de visão é discutida em Teorey e Fry (1982). Uma abordagem

formal para a integração de visão, que usa dependências de inclusão, é dada em Casanova e Vidal (1982). Ramesh e Ram (1997) descrevem uma metodologia para a integração de relacionamentos nos esquemas utilizando o conhecimento das restrições de integridade; isso estende o trabalho anterior de Navathe et al. (1984a). Sheth et al. (1993) descrevem as questões de montagem de esquemas globais pelo raciocínio a respeito dos relacionamentos de atributo e equivalências de entidade. Navathe e Savasere (1996) descrevem uma abordagem prática para a montagem de esquemas globais com base nos operadores aplicados aos componentes do esquema. Santucci (1998) oferece um tratamento detalhado dos esquemas ER para integração. Castano et al. (1998) apresentam um estudo abrangente das técnicas de análise do esquema conceitual.

O projeto de transação é um tópico com pesquisa relativamente menos minucioso. Mylopoulos et al. (1980) propuseram a linguagem TAXIS, e Albano et al. (1985) desenvolveram o sistema GALILEO, ambos abrangentes para especificar transações. A linguagem GORDAS para o modelo ECR (Elmasri et al., 1985) contém uma capacidade de especificação de transação. Navathe e Balaraman (1991) e Ngu (1989) discutem a modelagem de transação em geral para modelos de dados semânticos. Elmagarmid (1992) discute os modelos de transação para aplicações avançadas. Batini et al. (1992, capítulos 8, 9 e 11) discutem o projeto de transação de alto nível e a análise conjunta de dados e funções. Shasha (1992) é uma excelente fonte sobre ajuste de banco de dados.

Informações sobre algumas ferramentas comerciais de banco de dados bem conhecidas podem ser encontradas nos sites dos vendedores (veja o nome das empresas na Tabela 10.1). Os princípios por trás das ferramentas de projeto automatizadas são discutidos em Batini et al. (1992, Capítulo 15). A ferramenta SECSI é descrita em Metais et al. (1998). DKE (1997) é uma edição especial sobre questões de linguagem natural nos bancos de dados.

parte 4

Objeto, objeto-relacional e XML: conceitos, modelos, linguagens e padrões

capítulo 11

Bancos de dados de objeto e objeto-relacional

Neste capítulo, vamos discutir os recursos dos modelos de dados orientados a objeto e mostrar como alguns desses recursos foram incorporados nos sistemas de bancos de dados relacionais. Os bancos de dados orientados a objeto agora são conhecidos omo **bancos de dados de objeto (BDO)** (anteriormente chamados BDOO), e os sistemas de bancos de dados são conhecidos como **sistemas de gerenciamento de dados de objeto** (SGDO) (anteriormente conhecidos como SGBDO ou SGBDOO). Sistemas e modelos de dados tradicionais, como relacionais, de rede e hierárquicos, têm tido muito sucesso no desenvolvimento das tecnologias de banco de dados exigidas para muitas aplicações de banco de dados de negócios tradicionais. Porém, eles têm certas deficiências quando aplicações de banco de dados mais complexas precisam ser projetadas e implementadas — por exemplo, bancos de dados para projeto de engenharia e manufatura (CAD/CAM e CIM[1]), experimentos científicos, telecomunicações, sistemas de informações geográficas e multimídia.[2] Essas aplicações mais recentes possuem requisitos e características que diferem daqueles das aplicações de negócios tradicionais, como estruturas mais complexas para objetos armazenados; a necessidade de novos tipos de dados para armazenar imagens, vídeos ou itens de texto grandes; transações de maior duração; e a necessidade de definir operações fora do padrão específicas da aplicação.

Os bancos de dados de objeto foram propostos para atender a algumas das necessidades dessas aplicações mais complexas. Um recurso chave dos bancos de dados de objeto é o poder que eles dão ao projetista para especificar tanto a *estrutura* dos objetos complexos quanto as *operações* que podem ser aplicadas a esses objetos.

Outro motivo para a criação de bancos de dados orientados a objeto é o grande aumento no uso de linguagens de programação orientadas a objeto para o desenvolvimento de aplicações de software. Os bancos de dados são componentes fundamentais em muitos sistemas de software, e os bancos de dados tradicionais às vezes são difíceis de usar com aplicações de software que são desenvolvidas em uma linguagem de programação orientada a objeto como C++ ou Java. Os bancos de dados de objeto são projetados de modo que possam ser integrados diretamente — ou *transparentemente* — ao software desenvolvido usando linguagens de programação orientadas a objeto.

Vendedores de SGBD relacional (SGBDR) também reconheceram a necessidade de incorporar recursos que foram propostos para bancos de dados de objeto, e versões mais novas de sistemas relacionais incorporaram muitos desses recursos. Isso levou a sistemas de banco de dados que são caracterizados como *objeto-relacional* ou SGBDORs. A versão mais recente do padrão SQL (2008) para SGBDRs inclui muitos desses recursos, que eram conhecidos srcinalmente como SQL/Object e agora têm sido incorporados na especificação SQL principal, conhecida como SQL/Foundation.

Embora muitos protótipos experimentais e sistemas de banco de dados comerciais orientados a objeto tenham sido criados, eles não tiveram uso generalizado por causa da popularidade dos sistemas

[1] Computer-Aided Design/Computer-Aided Manufacturing (projeto auxiliado por computador/fabricação auxiliada por computador) e Computer-Integrated Manufacturing (fabricação integrada ao computador).

[2] Bancos de dados de multimídia precisam armazenar vários tipos de objetos de multimídia, como vídeo, áudio, imagens, gráficos e documentos (ver Capítulo 26).

relacionais e objeto-relacional. Os protótipos experimentais incluíram o sistema Orion, desenvolvido na MCC;[3] o OpenOODB, na Texas Instruments; o sistema Iris, nos laboratórios da Hewlett-Packard; o sistema Ode, na AT&T Bell Labs;[4] e o projeto ENCORE/ObServer, na Brown University. Sistemas disponíveis comercialmente incluíram GemStone Object Server da GemStone Systems, ONTOS DB da Ontos, Objectivity/DB da Objectivity Inc., Versant Object Database e FastObjects da Versant Corporation (e Poet), ObjectStore da Object Design e Ardent Database da Ardent.[5] Estes representam apenas uma lista parcial dos protótipos experimentais e sistemas de banco de dados orientados a objeto comerciais que foram criados.

À medida que os SGBDs de objeto comerciais se tornaram disponíveis, reconheceu-se a necessidade de um modelo e de uma linguagem padrão. Como o procedimento formal para aprovar padrões costuma levar alguns anos, um consórcio de vendedores e usuários de SGBD de objeto, chamado ODMG,[6] propôs um padrão cuja especificação atual é conhecida como padrão ODMG 3.0.

Os bancos de dados orientados a objeto adotaram muitos dos conceitos que foram desenvolvidos srcinalmente para as linguagens de programação orientadas a objeto.[7] Na Seção 11.1, descrevemos os principais conceitos utilizados em muitos sistemas de banco de dados de objeto e que foram, mais tarde, incorporados em sistemas objeto-relacional e no padrão SQL. Estes incluem *identidade de objeto*, *estrutura de objeto* e *construtores de tipo*, *encapsulamento de operações* e a definição de *métodos* como parte das declarações de classe, mecanismos para armazenar objetos em um banco de dados, tornando-os *persistentes*, e *hierarquias e herança de tipo e classe* Depois, na Seção 11.2, vemos como esses conceitos foram incorporados nos padrões SQL mais recentes, levando a bancos de dados objeto-relacional. Os recursos de objeto foram introduzidos srcinalmente na SQL:1999, e depois atualizados na versão mais recente (SQL:2008) do padrão. Na Seção 11.3, voltamos nossa atenção para os padrões de banco de dados de objeto 'puros', apresentando recursos do padrão de banco de dados de objeto ODMG 3.0 e a linguagem de definição de objeto ODL. A Seção 11.4 apresenta uma visão geral do processo de projeto para bancos de dados de objeto. A Seção 11.5 discute a linguagem de consulta de objeto (OQL), que faz parte do padrão ODMG 3.0. Na Seção 11.6, discutimos os bindings da linguagem de programação, que especificam como estender as linguagens de programação orientadas a objeto para incluir recursos do padrão de banco de dados de objeto. No final há um resumo do capítulo. As seções 11.5 e 11.6 podem ser omitadas se for desejada uma introdução menos completa aos bancos de dados de objeto.

11.1 Visão geral dos conceitos de banco de dados de objeto

11.1.1 Introdução aos conceitos e recursos orientados a objeto

O termo *orientado a objeto* — abreviado como *OO* ou *O-O* — tem suas srcens nas linguagens de programação OO, ou LPOO. Hoje, os conceitos de OO são aplicados nas áreas de bancos de dados, engenharia de software, bases de conhecimento, inteligência artificial e sistemas de computação em geral. LPOOs têm suas raízes na linguagem SIMULA, que foi proposta no final da década de 1960. A linguagem de programação Smalltalk, desenvolvida na Xerox PARC[8] nos anos 1970, foi uma das primeiras linguagens a incorporar explicitamente conceitos OO adicionais, como passagem de mensagens e herança. Ela é conhecida como uma linguagem de programação OO *pura*, significando que foi projetada de maneira explicitamente para ser orientada a objeto. Isso é diferente das linguagens de programação OO *híbridas*, que incorporam conceitos de OO a uma linguagem já existente. Um exemplo desse segundo tipo é a C++, que incorpora conceitos de OO à popular linguagem de programação C.

Um **objeto** normalmente possui dois componentes: estado (valor) e comportamento (operações). Ele pode ter uma *estrutura de dados complexa*, bem como *operações específicas* definidas pelo programador.[9] Os objetos em uma LPOO existem apenas durante a execução do programa; assim, eles são chamados de *objetos transientes*. Um banco de dados OO pode estender a existência de objetos de modo que eles sejam armazenados permanentemente em um banco de dados e, portanto, os objetos se tornam *objetos persistentes*,

[3] Microelectronics and Computer Technology Corporation, Austin, Texas.
[4] Agora chamada Lucent Technologies.
[5] Anteriormente, O2 da O2 Technology.
[6] Object Data Management Group.
[7] Conceitos semelhantes também foram desenvolvidos nos campos de modelagem de dados semântica e representação do conhecimento.
[8] Palo Alto Research Center, Palo Alto, Califórnia.
[9] Os objetos possuem muitas outras características, conforme discutiremos adiante neste capítulo.

que existem além do término do programa e podem ser recuperados mais tarde e compartilhados por outros programas. Em outras palavras, os bancos de dados OO armazenam objetos persistentes de maneira permanente no armazenamento secundário, e permitem o compartilhamento desses objetos entre vários programas e aplicações. Isso requer a incorporação de outros recursos bem conhecidos dos sistemas de gerenciamento de banco de dados, como os mecanismos de indexação, para localizar os objetos com eficiência, o controle de concorrência, para permitir o compartilhamento de objetos entre programas concorrentes, e a recuperação de falhas. Um sistema de banco de dados OO costuma interagir com uma ou mais linguagens de programação OO para oferecer capacidades de objeto persistentes e compartilhadas.

A estrutura interna de um objeto na LPOOs inclui a especificação de **variáveis de instância**, as quais mantêm os valores que definem o estado interno do objeto. Uma variável de instância é semelhante ao conceito de um *atributo* no modelo relacional, exceto que as variáveis de instância podem ser encapsuladas dentro do objeto e, portanto, não são necessariamente visíveis aos usuários externos. As variáveis de instância também podem ter tipos de dados de qualquer complexidade. Os sistemas orientados a objeto permitem a definição das operações ou funções (comportamento) que podem ser aplicadas a objetos de determinado tipo. De fato, alguns modelos OO insistem para que todas as operações que um usuário pode aplicar a um objeto sejam predefinidas. Isso força um *encapsulamento completo* dos objetos. Essa abordagem rígida tem sido relaxada na maioria dos modelos de dados OO por dois motivos. Primeiro, os usuários do banco de dados normalmente precisam saber os nomes de atributo para poder especificar condições de seleção sobre os atributos a fim de recuperar objetos específicos. Segundo, o encapsulamento completo implica que qualquer recuperação simples exija uma operação predefinida, tornando assim as consultas casuais difíceis de especificar no ato.

Para encorajar o encapsulamento, uma operação é definida em duas partes. A primeira parte, chamada *assinatura* ou *interface* da operação, especifica o nome desta e os argumentos (ou parâmetros). A segunda parte, chamada de *método* ou *corpo*, especifica a *implementação* da operação, com frequência escrita em alguma linguagem de programação de uso geral. As operações podem ser invocadas passando uma *mensagem* a um objeto, que inclui o nome da operação e os parâmetros. O objeto, então, executa o método para essa operação. Esse encapsulamento permite a modificação da estrutura interna de um objeto, além da implementação de suas operações, sem a necessidade de interromper os programas externos que chamam essas operações. Logo, o encapsulamento oferece uma forma de independência entre dados e operação (ver Capítulo 2).

Outro conceito fundamental nos sistemas OO é o de *herança* e hierarquias de tipo e classe. Isso permite a especificação de novos tipos ou classes que herdam grande parte de sua estrutura e/ou operações de tipos ou classes previamente definidas. Isso torna mais fácil desenvolver os tipos de dados de um sistema de forma incremental e *reutilizar* definições de tipo existentes em novos tipos de objetos.

Um problema nos primeiros sistemas de banco de dados OO envolvia a representação de *relacionamentos* entre objetos. A insistência sobre encapsulamento completo nos primeiros modelos de dados OO levaram ao argumento de que os relacionamentos não devem ser representados explicitamente, mas, em vez disso, devem ser descritos definindo métodos apropriados que localizam objetos relacionados. No entanto, essa abordagem não funciona muito bem para bancos de dados complexos, com diversos relacionamentos, porque é útil identificar esses relacionamentos e torná-los visíveis aos usuários. O padrão de banco de dados de objeto ODMG reconheceu essa necessidade e representa explicitamente os relacionamentos binários por meio de um par de *referências inversas*, conforme descreveremos na Seção 11.3.

Outro conceito de OO é a *sobrecarga de operador*, que se refere à capacidade de uma operação de ser aplicada a diferentes tipos de objetos. Em tal situação, um *nome de operação* pode se referir a várias *implementações* distintas, dependendo do tipo de objeto ao qual é aplicado. Esse recurso também é chamado *polimorfismo de operador*. Por exemplo, uma operação para calcular a área de um objeto geométrico pode diferir em seu método (implementação), dependendo de o objeto ser do tipo triângulo, círculo ou retângulo. Isso pode exigir o uso da *ligação tardia* do nome da operação ao método apropriado em tempo de execução, quando o tipo de objeto ao qual a operação é aplicada se torna conhecido.

Nas próximas seções, vamos discutir com certos detalhes as principais características dos bancos de dados de objeto. A Seção 11.1.2 discute a identidade do objeto; a Seção 11.1.3 mostra como os tipos para objetos estruturados complexos são especificados por meio de construtores de tipo; a Seção 11.1.4 discute o encapsulamento e a persistência; e a Seção 11.1.5 apresenta conceitos de herança. A Seção 11.1.6 discute alguns conceitos adicionais de OO, e a Seção 11.1.7 oferece um resumo de todos os conceitos de OO que serão apresentados. Na Seção 11.2, mostra-

mos como alguns desses conceitos foram incorporados ao padrão SQL:2008 para bancos de dados relacionais. Depois, na Seção 11.3, mostramos como esses conceitos são realizados no padrão de banco de dados de objeto ODMG 3.0.

11.1.2 Identidade de objeto e objetos *versus* literais

Um dos objetivos de um SGDO (Sistema de Gerenciamento de Dados de Objeto) é manter uma correspondência direta entre objetos do mundo real e do banco de dados, de modo que os objetos não percam sua integridade e identidade e possam facilmente ser identificados e operados. Assim, um SGDO oferece uma **identidade única** a cada objeto independente armazenado no banco de dados. Essa identidade única normalmente é implementada por meio de um **identificador de objeto** (**OID**) único, gerado pelo sistema. O valor de um OID não é visível ao usuário externo, mas é utilizado internamente pelo sistema para identificar cada objeto de maneira exclusiva, criar e gerenciar referências entre objetos. O OID pode ser atribuído a variáveis de programa do tipo apropriado, quando necessário.

A principal propriedade exigida de um OID é que ele seja **imutável**; ou seja, o valor do OID de um objeto em particular não deve mudar. Isso preserva a identidade do objeto do mundo real que está sendo representado. Logo, um SGDO precisa ter algum mecanismo para gerar OIDs e preservar a propriedade de imutabilidade. Também é desejável que cada OID seja usado apenas uma vez; isto é, mesmo que um objeto seja removido do banco de dados, seu OID não deverá ser atribuído a outro objeto. Essas duas propriedades implicam que o OID não deve depender de quaisquer valores de atributo do objeto, pois o valor de um atributo pode ser alterado ou corrigido. Podemos comparar isso com o modelo relacional, no qual cada relação precisa ter um atributo de chave primária cujo valor identifica cada tupla de maneira exclusiva. No modelo relacional, se o valor da chave primária for alterado, a tupla terá uma nova identidade, embora ela ainda possa representar o mesmo objeto do mundo real. Como alternativa, um objeto do mundo real pode ter diferentes nomes para atributos chave em relações distintas, tornando difícil garantir que as chaves representem o mesmo objeto do mundo real (por exemplo, o identificador de objeto pode ser representado como `Emp_id` em uma relação e como `Cpf` em outra).

Não é apropriado basear o OID no endereço físico do objeto em seu local de armazenamento, pois esse endereço pode mudar após uma reorganização física do banco de dados. Contudo, alguns dos primeiros SGDOs usaram o endereço físico como OID, para aumentar a eficiência da recuperação do objeto. Se o endereço físico do objeto muda, um *ponteiro indireto* pode ser colocado no endereço anterior, dando o novo local físico do objeto. É mais comum usar inteiros longos como OIDs e depois usar alguma forma de tabela de hash para mapear o valor do OID ao endereço físico atual do objeto no local de armazenamento.

Alguns dos primeiros modelos de dados OO exigiam que tudo — desde um simples valor a um objeto complexo — fosse representado como um objeto; logo, cada valor básico, como um inteiro, cadeia ou valor booleano, tem um OID. Isso permite que dois valores básicos idênticos tenham OIDs diferentes, o que pode ser útil em alguns casos. Por exemplo, o valor inteiro 50 às vezes pode ser usado para significar um peso em quilogramas e, em outras ocasiões, para significar a idade de uma pessoa. Então, dois objetos básicos com OIDs distintos poderiam ser criados, mas ambos representariam o valor inteiro 50. Embora útil como um modelo teórico, ele não é muito prático, pois leva à geração de muitos OIDs. Assim, a maioria dos sistemas de banco de dados OO permite a representação de objetos e **literais** (ou valores). Cada objeto precisa ter um OID imutável, enquanto um valor literal não tem OID e seu valor simplesmente corresponde a si mesmo. Dessa forma, um valor literal normalmente é armazenado dentro de um objeto e *não pode ser referenciado* de outros objetos. Em muitos sistemas, valores literais estruturados complexos também podem ser criados sem ter um OID correspondente, se for preciso.

11.1.3 Estruturas de tipo complexas para objetos e literais

Outro recurso de um SGDO (e BDOs em geral) é que os objetos e literais podem ter uma *estrutura de tipo* de *complexidade arbitrária*, a fim de conter todas as informações necessárias que descrevem o objeto ou literal. Ao contrário, nos sistemas de banco de dados tradicionais, a informação sobre um objeto complexo com frequência é *espalhada* por muitas relações ou registros, levando à perda de correspondência direta entre um objeto do mundo real e sua representação no banco de dados. Nos BDOs, um tipo complexo pode ser construído com base em outros tipos pelo *aninhamento* de **construtores de tipo**. Os três construtores mais básicos são átomo, struct (ou tupla) e coleção.

1. Um construtor de tipo é chamado de construtor de **átomo**, embora esse termo não seja usado no padrão de objeto mais recente. Ele inclui os tipos de dados embutidos básicos do modelo de objeto, que são semelhantes aos tipos básicos em muitas linguagens de programação: inteiros,

cadeias de caracteres, números de ponto flutuante, tipos enumerados, booleanos, e assim por diante. Eles são chamados tipos de **valor único** ou **atômicos**, pois cada valor do tipo é considerado um único valor atômico (indivisível).

2. Um segundo construtor de tipo é chamado de construtor **struct** (ou **tuple**). Ele pode criar tipos estruturados padrão, como tuplas (tipos de registro) no modelo relacional básico. Um tipo estruturado é composto de vários componentes, e às vezes também é chamado de tipo *composto*. Mais precisamente, o construtor struct não é considerado um tipo, mas sim um **gerador de tipo**, pois muitos tipos estruturados diferentes podem ser criados. Por exemplo, dois tipos estruturados diferentes que podem ser criados são: struct Nome<PrimeiroNome: string, InicialMeio: char, Sobrenome: string> e struct TituloAcademico<Principal: string, Titulo: string, Ano: date>. Para criar estruturas de tipo aninhado complexas no modelo de objeto, são necessários construtores do tipo de *coleção*, que vamos discutimos a seguir. Observe que os construtores de tipo *átomo* e *struct* são os únicos disponíveis no modelo relacional srcinal (básico).
3. Construtores de tipo de **coleção** (ou *multivalorados*) incluem os construtores de tipo **set(T)**, **list(T)**, **bag(T)**, **array(T)** e **dictionary(K,T)**. Estes permitem que parte de um objeto ou valor literal inclua uma coleção de outros objetos ou valores quando necessário. Esses construtores também são considerados **geradores de tipos**, pois muitos tipos diferentes podem ser criados. Por exemplo, set(*string*), set(*integer*) e set(*Funcionario*) são três tipos diferentes que podem ser criados com base no construtor de tipo *set*. Todos os elementos em um valor de coleção em particular precisam ser do mesmo tipo. Por exemplo, todos os valores em uma coleção do tipo set(*string*) precisam ser valores de string.

O *construtor de átomo* é usado para representar todos os valores atômicos básicos, como inteiros, números reais, cadeias de caracteres, booleanos e qualquer outro tipo de dado básico que o sistema aceite diretamente. O *construtor tuple* pode criar valores estruturados e objetos no formato $<a_1:i_1, a_2:i_2, ..., a_n:i_n>$, onde cada a_j é um nome de atributo [10] e cada i_j é um valor ou um OID.

Os outros construtores bastante usados são conhecidos coletivamente como tipos de coleção, mas possuem diferenças individuais entre eles. O **construtor set** (conjunto) criará objetos ou literais que são um conjunto de elementos *distintos* $\{i_1, i_2, ..., i_n\}$, todos do mesmo tipo. O **construtor bag** (às vezes chamado de *multiconjunto*) é semelhante a um conjunto, exceto que os elementos em uma bag *não precisam ser distintos*. O **construtor list** criará uma *lista ordenada* $[i_1, i_2, ..., i_n]$ de OIDs ou valores do mesmo tipo. Uma lista é semelhante a uma **bag**, com exceção de os elementos em uma lista serem *ordenados*, e, portanto, podemos nos referir ao primeiro, segundo ou enésimo elemento. O **construtor array** cria um vetor unidimensional de elementos do mesmo tipo. A principal diferença entre array e lista é que uma lista pode ter um número arbitrário de elementos, enquanto um array normalmente tem um tamanho máximo. Por fim, o **constructor dictionary** cria uma coleção de duas tuplas (K, V), onde o valor de uma chave K pode ser usado para recuperar o valor correspondente V.

A principal característica de um tipo de coleção é que seus objetos ou valores serão uma *coleção de objetos ou valores do mesmo tipo*, que podem ser desordenados (como um set ou uma bag) ou ordenados (como uma list ou um array). O tipo de construtor **tuple** normalmente é chamado de um **tipo estruturado**, pois corresponde à construção **struct** nas linguagens de programação C e C++.

Uma **linguagem de definição de objeto** (ODL — *Object Definition Language*) [11] que incorpora os construtores de tipo anteriores pode ser usada para definir os tipos de objeto para determinada aplicação de banco de dados. Na Seção 11.3, descreveremos a ODL padrão do ODMG, mas primeiro vamos apresentar os conceitos gradualmente nesta seção usando uma notação mais simples. Os construtores de tipo podem ser usados para definir as *estruturas de dados* para um *esquema de banco de dados* OO. A Figura 11.1 mostra como podemos declarar os tipos FUNCIONARIO e DEPARTAMENTO.

Na Figura 11.1, os atributos que se referem a outros objetos — como Dep de FUNCIONARIO ou Projetos de DEPARTAMENTO — são basicamente OIDs que servem como **referências** para outros objetos ao representar *relacionamentos* entre os objetos. Por exemplo, o atributo Dep de FUNCIONARIO é do tipo DEPARTAMENTO e, portanto, é usado como referência a um objeto DEPARTAMENTO específico (o objeto DEPARTAMENTO onde o funcionário trabalha). O valor desse atributo seria um OID para um objeto DEPARTAMENTO específico. Um relacionamento binário pode ser representado em uma direção, ou pode ter uma *referência inversa*. Essa última representação facilita a travessia pelo relacionamento nas duas direções. Por exemplo, na Figura 11.1, o atributo Funcionarios de DEPARTAMENTO tem como seu valor

[10] Também chamado de *nome de variável de instância* em terminologia OO.

[11] Esta corresponde à DDL (linguagem de definição de dados) do sistema de banco de dados (ver Capítulo 2).

```
define type FUNCIONARIO
tuple (
    Pnome:              string;
    Minicial:           char;
    Unome:              string;
    Cpf:                string;
    Data_nascimento:    DATE;
    Endereco:           string;
    Sexo:               char;
    Salario:            float;
    Supervisor:         FUNCIONARIO;
    Dep:                DEPARTAMENTO;

define type DATA
tuple (
    Ano:                integer;
    Mes:                integer;
    Dia:                integer; );

define type DEPARTAMENTO
tuple (
    Dnome:              string;
    Dnumero:            integer;
    Ger:                tuple (
                        Gerente: FUNCIONARIO;
                        Data_inicio: DATE; );
    Localizacoes:       set(string);
    Funcionarios:       set(FUNCIONARIO);
    Projetos:           set(PROJETO); );
```

Figura 11.1
Especificando os tipos de objeto FUNCIONARIO, DATA e DEPARTAMENTO usando construtores de tipo.

um *conjunto de referências* (ou seja, um conjunto de OIDs) para objetos do tipo FUNCIONARIO. Estes são os funcionários que trabalham para o DEPARTAMENTO. O inverso é o atributo de referência Dep de FUNCIONARIO. Veremos, na Seção 11.3, como o padrão ODMG permite que os inversos sejam declarados explicitamente como atributos de relacionamento para garantir que as referências inversas sejam consistentes.

11.1.4 Encapsulamento de operações e persistência de objetos

Encapsulamento de operações. O conceito de *encapsulamento* é uma das principais características das linguagens e sistemas de OO. Ele também está relacionado aos conceitos de *tipos de dados abstratos* e *ocultação de informações* nas linguagens de programação. Em modelos e sistemas de banco de dados tradicionais, esse conceito não foi aplicado, já que é comum tornar a estrutura dos objetos do banco de dados visível aos usuários e programas externos. Nesses modelos tradicionais, diversas operações do banco de dados são aplicáveis a objetos *de todos os tipos*. Por exemplo, no modelo relacional, as operações para selecionar, inserir, excluir e modificar tuplas são genéricas e podem ser aplicadas a *qualquer relação* no banco de dados. A relação e seus atributos são visíveis para usuários e programas externos que acessam a relação usando essas operações. Os conceitos de encapsulamento são aplicados a objetos de banco de dados nos BDOs ao definir o **comportamento** de um tipo de objeto com base nas **operações** que podem ser aplicadas externamente a objetos desse tipo. Algumas operações podem ser usadas para criar (inserir) ou destruir (excluir) objetos; outras operações podem atualizar o estado do objeto; e outras podem ser utilizadas para recuperar partes do estado do objeto ou para aplicar alguns cálculos. Ainda, outras operações podem realizar uma combinação de recuperação, cálculo e atualização. Em geral, a **implementação** de uma operação pode ser especificada em uma *linguagem de programação de uso geral* que oferece flexibilidade e poder na definição das operações.

Os usuários externos do objeto só se tornam cientes da **interface** das operações, que define o nome e os argumentos (parâmetros) de cada operação. A implementação é escondida desses usuários — ela inclui a definição de quaisquer estruturas de dados internas ocultas do objeto e a implementação das operações que acessam essas estruturas. A parte de interface de uma operação às vezes é chamada de **assinatura**, e a implementação da operação pode ser chamada de **método**.

Para aplicações de banco de dados, o requisito de que todos os objetos sejam completamente encapsulados é muito rigoroso. Um modo de relaxar esse requisito é dividir a estrutura de um objeto em atributos **visíveis** e **ocultos** (variáveis de instância). Os atributos visíveis podem ser vistos e são acessíveis diretamente pelos usuários e programadores de banco de dados por meio da linguagem de consulta. Os atributos ocultos de um objeto são completamente encapsulados e só podem ser acessados por meio de operações predefinidas. A maioria dos SGDOs emprega linguagens de consulta de alto nível para acessar atributos visíveis. Na Seção 11.5, descreveremos a linguagem de consulta OQL, que é proposta como uma linguagem de consulta padrão para BDOs.

O termo **classe** é frequentemente utilizado para se referir a uma definição de tipo, junto com as defi-

nições das operações para esse tipo.[12] A Figura 11.2 mostra como as definições de tipo na Figura 11.1 podem ser estendidas com operações para definir classes. Diversas operações são declaradas para cada classe, e a assinatura (interface) de cada operação é incluída na definição da classe. Um método (implementação) para cada operação deve ser definido em outro lugar usando uma linguagem de programação. As operações típicas incluem a operação de **construtor de objeto** (normalmente chamada de *new*), que é utilizada para criar um objeto, e a operação de **destruidor**, que serve para destruir (excluir) um objeto. Diversas operações **modificadoras de objeto** também podem ser declaradas para modificar os estados (valores) de vários atributos de um objeto. Operações adicionais podem **recuperar** informações sobre o objeto.

Uma operação costuma ser aplicada a um objeto usando a **notação de ponto**. Por exemplo, se d é uma referência a um objeto de DEPARTAMENTO, podemos chamar uma operação como nr_funcs escrevendo d.nr_funcs. De modo semelhante, ao escrever d.destroi_dep, o objeto referenciado por d é destruído (excluído). A única exceção é a operação construtora, que retorna uma referência a um novo objeto DEPARTAMENTO. Logo, em alguns modelos OO, é comum ter um nome padronizado para a operação construtora, que é o nome da própria classe, embora isso não tenha sido usado na Figura 11.2.[13] A notação de ponto também é utilizada para se referir aos atributos de um objeto — por exemplo, ao escrever d.Dnumero ou d.Data_inicio.

Especificando a persistência do objeto por meio de nomeação e acessibilidade Um SBDO normalmente está bastante ligado a uma linguagem de programação orientada a objeto (LPOO). A LPOO é usada para especificar as implementações de método (operação), bem como outro código de aplicação. Nem todos os objetos visam ser armazenados permanentemente no banco de dados. **Objetos transientes** existem no programa em execução e desaparecem quando o programa termina. **Objetos persistentes** são armazenados no banco de dados e persistem após o término do programa. Os mecanismos típicos para tornar um objeto persistente são a *nomeação* e *acessibilidade*.

O **mecanismo de nomeação** envolve dar a um objeto um nome persistente único dentro de determinado banco de dados. Esse **nome de objeto** persistente pode receber uma instrução ou operação específica no programa, como mostra a Figura 11.3. Os objetos

```
define class FUNCIONARIO
 type tuple (
     Pnome:              string;
     Minicial:           char;
     Unome:              string;
     Cpf:                string;
     Data_nascimento:    DATE;
     Endereco:           string;
     Sexo:               char;
     Salario:            float;
     Supervisor:         FUNCIONARIO;
     Dep:                DEPARTAMENTO;  );
 operations
     idade:              integer;
     criar_func:         FUNCIONARIO;
     destroi_func:       boolean;
end FUNCIONARIO;

define class DEPARTAMENTO
 type tuple (
     Dnome:              string;
     Dnumero:            integer;
     Ger:                tuple (
                         Gerente: FUNCIONARIO;
                         Data_inicio: DATE; );
     Localizacoes:       set (string);
     Funcionarios:       set (FUNCIONARIO);
     Projetos            set(PROJETO); );
 operations
     nr_funcs:           integer;
     criar_dep:          DEPARTAMENTO;
     destroi_dep:        boolean;
     aloca_func          (e: FUNCIONARIO):  boolean;

     (* acrescenta um funcionário ao departamento *)
     remove_func(e: FUNCIONARIO):
     boolean;
     (* remove um funcionário do departamento *)
end DEPARTAMENTO;
```

Figura 11.2
Acrescentando operações às definições de FUNCIONARIO e DEPARTAMENTO.

[12] Essa definição de *classe* é semelhante ao modo como é usada na popular linguagem de programação C++. O padrão ODMG usa a palavra *interface* além de *classe* (ver Seção 11.3). No modelo EER, o termo *classe* foi usado para se referir a um tipo de objeto, junto com o conjunto de todos os objetos desse tipo (ver Capítulo 8).

[13] Existem nomes padronizados para as operações construtoras e destruidoras na linguagem de programação C++. Por exemplo, para a classe FUNCIONARIO, o *nome do construtor default* é FUNCIONARIO e o *nome do destruidor default* é ~FUNCIONARIO. Também é comum usar a operação *new* para criar novos objetos.

persistentes nomeados são utilizados como **pontos de entrada** para o banco de dados, por meio dos quais os usuários e as aplicações podem iniciar seu acesso ao banco de dados. Obviamente, não é prático dar nomes a todos os objetos em um banco de dados grande, que inclui milhares de objetos, de modo que a maioria deles se torna persistente pelo uso do segundo mecanismo, chamado **acessibilidade**. O mecanismo de acessibilidade funciona tornando o objeto alcançável a partir de algum outro objeto persistente. Um objeto *B* é considerado **alcançável** com base no objeto *A* se uma sequência de referências no banco de dados levar do objeto *A* até o objeto *B*.

Se primeiro criarmos um objeto persistente nomeado *N*, cujo estado é um *conjunto* (ou possivelmente uma *bag*) de objetos de alguma classe *C*, podemos tornar objetos de *C* persistentes *acrescentando-os* ao conjunto, tornando-os assim alcançáveis de *N*. Logo, *N* é um objeto nomeado que define uma **coleção persistente** de objetos de classe *C*. No padrão do modelo de objeto, *N* é chamado de **extensão** de *C* (ver Seção 11.3).

Por exemplo, podemos definir uma classe SET_DEPARTAMENTO (ver Figura 11.3), cujos objetos são do tipo set (DEPARTAMENTO).[14] Podemos criar um objeto do tipo SET_DEPARTAMENTO, dando-lhe um nome persistente TODOS_DEPARTAMENTOS, como mostra a Figura 11.3. Qualquer objeto DEPARTAMENTO que seja acrescentado ao conjunto de TODOS_DEPARTAMENTOS usando a operação adiciona_dep torna-se persistente em virtude de estar sendo alcançável de TODOS_DEPARTAMENTOS. Conforme veremos na Seção 11.3, o padrão ODL do ODMG oferece ao projetista de esquema a opção de nomear uma extensão como parte da definição da classe.

Observe a diferença entre os modelos de banco de dados tradicionais e os BDOs com relação a isso. Nos modelos de banco de dados tradicionais, como o modelo relacional, *todos* os objetos são considerados persistentes. Logo, quando uma tabela como FUNCIONARIO é criada em um banco de dados relacional, ela representa tanto a *declaração de tipo* para FUNCIONARIO quanto um *conjunto persistente* de *todos* os registros (tuplas) de FUNCIONARIO. Na abordagem OO, uma declaração de classe de FUNCIONARIO especifica apenas o tipo e as operações para uma classe de objetos. O usuário precisa definir separadamente um objeto persistente do tipo set(FUNCIONARIO) ou bag(FUNCIONARIO), cujo valor é a *coleção de referências* (OIDs) a todos os objetos FUNCIONARIO persistentes, se isso for desejado, conforme mostra a Figura 11.3.[15] Isso permite que objetos transientes e persistentes sigam as mesmas declarações de tipo e classe da ODL e da LPOO. Em geral, é possível definir várias coleções persistentes para a mesma definição de classe, caso seja desejado.

```
define class SET_DEPARTAMENTO
    type set (DEPARTAMENTO);
    operations  adiciona_dep(d: DEPARTAMENTO):    boolean;
            (* acrescenta um departamento ao objeto SET_DEPARTAMENTO *)
                remove_dep(d:  DEPARTAMENTO):boolean;
            (* remove um departamento do objeto SET_DEPARTAMENTO *)
                criar_set_dep: SET_DEPARTAMENTO;
                destroi_set_dep: boolean;
end SET_DEPARTAMENTO;
...
persistent name TODOS_DEPARTAMENTOS: SET_DEPARTAMENTO;
(* TODOS_DEPARTAMENTOS é um objeto persistente nomeado do tipo SET_DEPARTA-
MENTO *)
...
d:= cria_dep;

(* cria um objeto DEPARTAMENTO na variável d *)
b:= TODOS_DEPARTAMENTOS.adiciona_dep(d);
(* torna d persistente incluindo-o no conjunto persistente TODOS_DEPARTAMENTOS *)
```

Figura 11.3
Criando objetos persistentes por nomeação e acessibilidade.

[14] Como veremos na Seção 11.3, a sintaxe ODL do ODMG usa **set**<DEPARTAMENTO> em vez de **set**(DEPARTAMENTO).

[15] Alguns sistemas, como POET, criam automaticamente a extensão para uma classe.

11.1.5 Hierarquias de tipo e herança

Modelo simplificado para herança Outra característica principal dos BDOs é que eles permitem hierarquias de tipo e herança. Usamos um modelo OO simples nesta seção — um modelo em que os atributos e as operações são tratados de maneira uniforme — visto que tanto atributos quanto operações podem ser herdadas. Na Seção 11.3, discutiremos o modelo de herança do padrão ODMG, que difere do modelo discutido aqui porque distingue entre *dois tipos de herança*. A herança permite a definição de novos tipos com base em outros predefinidos, levando a uma **hierarquia de tipo** (ou **classe**).

Um tipo é definido atribuindo-lhe um nome e depois estabelecendo uma série de atributos (variáveis de instância) e operações (métodos) para ele.[16] No modelo simplificado que usamos nesta seção, os atributos e operações são chamados de *funções*, pois os primeiros são semelhantes a funções com zero argumentos. Um nome de função pode ser usado para se referir ao valor de um atributo ou para se referir ao valor resultante de uma operação (método). Usamos o termo **função** para nos referirmos a atributos *e* operações, pois eles são tratados de modo semelhante em uma introdução básica à herança.[17]

Um tipo, em sua forma mais simples, tem um **nome de tipo** e uma lista de **funções** visíveis (*públicas*). Ao especificar um tipo nesta seção, usamos o formato a seguir, que não especifica argumentos de funções, para simplificar a discussão:

NOME_TIPO: função, função, ..., função

Por exemplo, um tipo que descreve as características de uma PESSOA pode ser definido da seguinte forma:

PESSOA: Nome, Endereco, Data_nascimento, Idade, Cpf

No tipo PESSOA, as funções Nome, Endereco, Cpf e Data_nascimento podem ser implementadas como atributos armazenados, enquanto a função Idade pode ser implementada como uma operação que calcula a Idade do valor do atributo Data_nascimento e a data atual.

O conceito de **subtipo** é útil quando o projetista ou usuário precisa criar um tipo que é semelhante, mas não idêntico, a um tipo já definido. O subtipo, então, herda todas as funções do tipo predefinido, que é conhecido como **supertipo**. Por exemplo, suponha que queiramos definir dois novos tipos FUNCIONARIO e ALUNO da seguinte forma:

FUNCIONARIO: Nome, Endereco, Data_nascimento, Idade, Cpf, Salario, Data_contratacao, Nivel

ALUNO: Nome, Endereco, Data_nascimento, Idade, Cpf, Curso, Coeficiente

Visto que tanto ALUNO quanto FUNCIONARIO incluem todas as funções definidas para PESSOA mais algumas funções adicionais próprias, podemos declará-los **subtipos** de PESSOA. Cada um herdará as funções previamente definidas de PESSOA — a saber, Nome, Endereco, Data_nascimento, Idade e Cpf. Para ALUNO, só é necessário definir as novas funções (locais) Curso e Coeficiente, que não são herdadas. Presume-se que Curso possa ser definido como um atributo armazenado, enquanto Coeficiente pode ser implementada como uma operação que calcula a média de pontos da nota do aluno ao acessar os valores de Nota que são armazenados internamente (ocultos) dentro de cada objeto ALUNO como *atributos ocultos*. Para FUNCIONARIO, as funções Salario e Data_contratacao podem ser atributos armazenados, ao passo que Nivel pode ser uma operação que calcula Nivel de experiencia baseando-se no valor de Data_contratacao.

Portanto, podemos declarar FUNCIONARIO e ALUNO da seguinte forma:

FUNCIONARIO **subtype-of** PESSOA: Salario, Data_contratacao, Nivel

ALUNO **subtype-of** PESSOA: Curso, Coeficiente

Em geral, um subtipo inclui *todas* as funções que são definidas para seu supertipo mais algumas funções adicionais que são *específicas* apenas ao subtipo. Logo, é possível gerar uma **hierarquia de tipos** para mostrar os relacionamentos de supertipo/subtipo entre todos os tipos declarados no sistema.

Como outro exemplo, considere um tipo que descreve objetos na geometria plana, e que podem ser definidos da seguinte forma:

GEOMETRIA_OBJETO: Formato, Area, Ponto_referencia

Para o tipo GEOMETRIA_OBJETO, Formato é implementado como um atributo (seu domínio pode ser um tipo enumerado com valores 'triangulo', 'retangulo', 'circulo', e assim por diante), e Area é um método aplicado para calcular a área. Ponto_referencia especifica as coordenadas de um ponto que determina a localização do objeto. Agora, suponha que queiramos definir uma série de subtipos para o tipo GEOMETRIA_OBJETO, da seguinte forma:

[16] Nesta seção, usaremos os termos *tipo* e *classe* para indicar a mesma coisa — a saber, os atributos e operações de algum tipo de objeto.

[17] Veremos, na Seção 11.3, que os tipos com funções são semelhantes ao conceito de interfaces usado na ODL do ODMG.

RETANGULO **subtype-of** GEOMETRIA_OBJETO: Largura, Altura

TRIANGULO **subtype-of** GEOMETRIA_OBJETO: Lado1, Lado2, Angulo

CIRCULO **subtype-of** GEOMETRIA_OBJETO: Raio

Observe que a operação Area pode ser implementada por um método diferente para cada subtipo, pois o procedimento para cálculo de área é distinto para retângulos, triângulos e círculos. De modo semelhante, o atributo Ponto_referencia pode ter um significado diferente para cada subtipo; ele poderia ser o ponto central para objetos RETANGULO e CIRCULO, e o ponto de vértice entre os dois lados dados para um objeto TRIANGULO.

Observe que as definições de tipo descrevem objetos, mas *não* geram objetos por si sós. Quando um objeto é criado, em geral ele pertence a um ou mais desses tipos que foram declarados. Por exemplo, um objeto círculo é do tipo CIRCULO e GEOMETRIA_OBJETO (por herança). Cada objeto também se torna um membro de uma ou mais coleções persistentes de objetos (ou extensões), que são usadas para agrupar coleções de objetos que são armazenadas persistentemente no banco de dados.

Restrições sobre extensões correspondentes a uma hierarquia de tipos. Na maioria dos BDOs, uma **extensão** é definida para armazenar a coleção de objetos persistentes para cada tipo ou subtipo. Nesse caso, a restrição é que todo objeto em uma extensão que corresponda a um subtipo também deve ser um membro da *extensão* que corresponde a seu supertipo. Alguns sistemas de banco de dados OO têm um tipo do sistema predefinido (chamado de classe ROOT ou classe OBJETO), cuja extensão contém todos os objetos do sistema.[18]

A classificação, então, prossegue atribuindo objetos a subtipos adicionais que são significativos à aplicação, criando uma **hierarquia de tipos** (ou **hierarquia de classes**) para o sistema. Todas as extensões para classes definidas pelo sistema e pelo usuário são subconjuntos da extensão correspondente à classe OBJETO, direta ou indiretamente. No modelo ODMG (ver Seção 11.3), o usuário pode ou não especificar uma extensão para cada classe (tipo), dependendo da aplicação.

Uma extensão é um objeto persistente nomeado cujo valor é uma **coleção persistente** que mantém uma coleção de objetos do mesmo tipo que, por sua vez, são armazenados permanentemente no banco de dados. Os objetos podem ser acessados e compartilhados por vários programas. Também é possível criar uma **coleção transiente**, que existe por certo tempo durante a execução de um programa, mas não é mantida quando este termina. Por exemplo, uma coleção transiente pode ser criada em um programa para manter o resultado de uma consulta que seleciona alguns objetos de uma coleção persistente e os copia para a coleção transiente. O programa pode, então, manipular os objetos na coleção transiente e, quando o programa terminar, a coleção transiente deixa de existir. Em geral, diversas coleções — transientes ou persistentes — podem conter objetos do mesmo tipo.

O modelo de herança discutido nesta seção é muito simples. Conforme veremos na Seção 11.3, o modelo ODMG distingue a herança de tipo — chamada de *herança de interface* e indicada por um sinal de dois pontos (:) — da restrição de *herança de extensão*, indicada pela palavra-chave EXTEND.

11.1.6 Outros conceitos de orientação a objeto

Polimorfismo de operações (sobrecarga de operador). Outra característica dos sistemas OO em geral é que eles oferecem o **polimorfismo** de operações, conhecido também como **sobrecarga de operador**. Esse conceito permite que o mesmo *nome de operador* ou *símbolo* estejam ligados a duas ou mais *implementações* diferentes do operador, dependendo do tipo de objetos aos quais o operador é aplicado. Um exemplo simples das linguagens de programação pode ilustrar esse conceito. Em algumas linguagens, o símbolo de operador '+' pode significar coisas distintas quando aplicado a operandos (objetos) de tipos diferentes. Se os operandos de '+' forem do tipo *inteiro*, a operação chamada é a adição de inteiros; se forem do tipo *ponto flutuante*, a operação chamada é a adição de ponto flutuante; e se forem do tipo *set*, a operação chamada é a união de conjunto. O compilador pode determinar qual operação executar com base nos tipos dos operandos fornecidos.

Em bancos de dados OO, pode ocorrer uma situação semelhante. Podemos usar o exemplo GEOMETRIA_OBJETO da Seção 11.1.5 para ilustrar o polimorfismo de operações[19] no BDO.

Nesse exemplo, a função Area é declarada para todos os objetos do tipo GEOMETRIA_OBJETO. No entanto, a implementação do método para Area pode ser diferente para cada subtipo de GEOMETRIA_OBJETO. Uma possibilidade é ter uma implementação geral

[18] Isso é chamado de OBJETO no modelo ODMG (ver Seção 11.3).

[19] Em linguagens de programação há diversos tipos de polimorfismo. O leitor interessado deve consultar a bibliografia selecionada ao final deste capítulo, os trabalhos que incluem uma discussão mais aprofundada.

para calcular a área de GEOMETRIA_OBJETO generalizado (por exemplo, ao escrever um algoritmo geral para calcular a área de um polígono) e depois para reescrever algoritmos mais eficientes para calcular as áreas de tipos específicos de objetos geométricos, como um círculo, um retângulo, um triângulo, e assim por diante. Nesse caso, a função Area é *sobrecarregada* por diferentes implementações.

O SGDO agora precisa selecionar o método apropriado para a função Area com base no tipo de objeto geométrico ao qual ele é aplicado. Em sistemas fortemente tipados, isso pode ser feito em tempo à compilação, pois os tipos de objeto precisam ser conhecidos. Isso é chamado de **vínculo antecipado** (ou **estático**). Porém, em sistemas com tipagem fraca ou sem tipagem (como Smalltalk e LISP), o tipo do objeto ao qual uma função é aplicada pode não ser conhecido antes da execução. Nesse caso, a função precisa verificar o tipo do objeto durante a execução e depois chamar o método apropriado. Isso normalmente é conhecido como **vínculo tardio** (ou **dinâmico**).

Herança múltipla e herança seletiva. A **herança múltipla** ocorre quando certo subtipo *T* é um subtipo de dois (ou mais) tipos e, portanto, herda as funções (atributos e métodos) dos dois supertipos. Por exemplo, podemos criar um subtipo GERENTE_ENGENHEIRO, que é um subtipo tanto de GERENTE quanto de ENGENHEIRO. Isso leva à criação de um **reticulado de tipos**, em vez de uma hierarquia deles. Um problema que pode ocorrer com a herança múltipla é que os supertipos dos quais o subtipo herda podem ter funções distintas do mesmo nome, criando uma ambiguidade. Por exemplo, tanto GERENTE quanto ENGENHEIRO podem ter uma função chamada Salario. Se a função Salario for implementada por diferentes métodos nos supertipos GERENTE e ENGENHEIRO, existe uma ambiguidade quanto a qual dos dois é herdado pelo subtipo GERENTE_ENGENHEIRO. Contudo, é possível que tanto ENGENHEIRO quanto GERENTE herdem Salario do mesmo supertipo (como FUNCIONARIO) mais acima no reticulado. A regra geral é que, se uma função é herdada de algum *supertipo comum*, então ela é herdada apenas uma vez. Nesse caso, não existe ambiguidade; o problema só aparece se as funções forem distintas nos dois supertipos.

Existem várias técnicas para lidar com a ambiguidade na herança múltipla. Uma solução é fazer que o sistema verifique a ambiguidade quando o subtipo for criado, e deixar que o usuário escolha explicitamente qual função deve ser herdada nesse momento. Uma segunda solução é usar alguma padronização do sistema. Uma terceira solução é não permitir a herança múltipla completamente se houver ambiguidade de nomes, em vez de forçar o usuário a mudar o nome de uma das funções em um dos supertipos. Na realidade, alguns sistemas OO não permitem herança múltipla alguma. No padrão de banco de dados de objeto (ver Seção 11.3), a herança múltipla é permitida para a operação das interfaces, mas não para a herança EXTENDS de classes.

A **herança seletiva** ocorre quando um subtipo herda apenas algumas das funções de um supertipo. Outras funções não são herdadas. Nesse caso, uma cláusula EXCEPT pode ser usada para listar as funções em um supertipo que *não* devem ser herdadas pelo subtipo. O mecanismo de herança seletiva não costuma ser fornecido nos BDOs, mas é usado com mais frequência nas aplicações de inteligência artificial.[20]

11.1.7 Resumo dos conceitos de banco de dados de objeto

Para concluir esta seção, oferecemos um resumo dos principais conceitos usados nos BDOs e sistemas objeto-relacional:

- **Identidade de objeto.** Os objetos possuem identidades únicas, que são independentes de seus valores de atributo e geradas pelo SGDO.
- **Construtores de tipos.** Estruturas de objeto complexas podem ser construídas ao aplicar de uma maneira aninhada um conjunto de construtores básicos, como tuple, set, list, array e bag.
- **Encapsulamento de operações.** Tanto a estrutura do objeto quanto as operações que podem ser aplicadas aos objetos individuais são incluídas nas definições de tipo.
- **Compatibilidade da linguagem de programação.** Objetos persistentes e transientes são tratados de maneira transparente. Os objetos se tornam persistentes ao serem alcançáveis de uma coleção persistente (extensão) ou pela nomeação explícita.
- **Hierarquias de tipos e herança.** Os tipos de objetos podem ser especificados usando uma hierarquia de tipos, que permite a herança de atributos e métodos (operações) de tipos previamente definidos. A herança múltipla é permitida em alguns modelos.
- **Extensões.** Todos os objetos persistentes de determinado tipo podem ser armazenados em

[20] No modelo ODMG, a herança de tipos refere-se apenas à herança de operações, e não de atributos (ver Seção 11.3).

uma extensão. As extensões correspondentes a uma hierarquia de tipo possuem restrições de conjunto/subconjunto em suas coleções de objetos persistentes.

- **Polimorfismo e sobrecarga de operador.** As operações e nomes de método podem ser sobrecarregadas para que se apliquem a diferentes tipos de objeto com diversas implementações.

Nas seções a seguir, mostramos como esses conceitos são realizados no padrão SQL (Seção 11.2) e no padrão ODMG (Seção 11.3).

11.2 Recursos objeto-relacional: extensões do banco de dados de objeto para SQL

Apresentamos a SQL como a linguagem-padrão para SGBDRs nos capítulos 4 e 5. Conforme discutimos, a SQL foi especificada inicialmente por Chamberlin e Boyce (1974) e passou por melhorias e padronização em 1989 e 1992. A linguagem continuou sua evolução com um novo padrão, inicialmente chamado SQL3 enquanto estava sendo desenvolvido, e mais tarde ficou conhecido como SQL:99 para as partes da SQL3 que foram aprovadas no padrão. Começando com a versão da SQL conhecida como SQL3, recursos dos bancos de dados de objeto foram incorporados ao padrão SQL. A princípio, essas extensões foram conhecidas como SQL/Object, mas depois foram incorporadas na parte principal da SQL, conhecida como SQL/Foundation. Usaremos esse padrão mais recente, SQL:2008, em nossa apresentação dos recursos de objeto da SQL, embora isso possa ainda não ter sido realizado nos SGBDs comerciais que seguem a SQL. Também discutiremos como os recursos de objeto da SQL evoluíram até sua manifestação mais recente na SQL:2008.

O modelo relacional com melhorias de banco de dados de objeto às vezes é conhecido como **modelo objeto-relacional**. Revisões adicionais foram feitas à SQL em 2003 e 2006 para acrescentar recursos relacionados à XML (ver Capítulo 12).

A seguir estão alguns dos recursos do banco de dados de objeto que foram incluídos na SQL:

- Alguns **construtores de tipo** foram acrescentados para especificar objetos complexos. Estes incluem o *tipo de linha*, que corresponde ao construtor de tupla (ou struct). Um *tipo de array* para especificar coleções também é fornecido. Outros construtores de tipo de coleção, como *set*, *list* e *bag*, não fizeram parte das especificações SQL/Object srcinais, mas foram incluídos mais tarde ao padrão.
- Foi incluído um mecanismo para especificar a **identidade de objeto** por meio do uso de *tipo de referência*.
- O **encapsulamento de operações** é fornecido por meio do mecanismo de **tipos definidos pelo usuário** (**UDTs** — *User-Defined Types*), que podem incluir operações como parte de sua declaração. Estes são um pouco semelhantes ao conceito de *tipos de dados abstratos*, que foram desenvolvidos nas linguagens de programação. Além disso, o conceito de **rotinas definidas pelo usuário** (**UDRs** — *User-Defined Routines*) permite a definição de métodos (operações) gerais.
- Mecanismos de **herança** são fornecidos usando a palavra-chave UNDER.

Agora, vamos discutir cada um desses conceitos com mais detalhes. Em nossa discussão, vamos nos referir ao exemplo da Figura 11.4.

11.2.1 Tipos definidos pelo usuário e estruturas complexas para objetos

Para permitir a criação de objetos estruturados complexos, e para separar a declaração de um tipo da criação de uma tabela, a SQL agora oferece **tipos definidos pelo usuário** (**UDTs**). Além disso, quatro tipos de coleção foram incluídos para permitir tipos e atributos multivalorados, a fim de especificar objetos com estruturas complexas, em vez de apenas registros simples (planos). O usuário criará os UDTs para determinada aplicação como parte do esquema do banco de dados. Um **UDT** pode ser especificado em sua forma mais simples usando a seguinte sintaxe:

```
CREATE TYPE NOME_TIPO AS
(<declarações de componentes>);
```

A Figura 11.4 ilustra alguns dos conceitos de objeto na SQL. Explicaremos os exemplos dessa figura gradualmente, à medida que explicarmos os conceitos. Primeiro, um UDT pode ser usado como tipo para um atributo ou como tipo para uma tabela. Ao usar um UDT como tipo para um atributo dentro de outro UDT, podemos criar uma estrutura complexa para objetos (tuplas) em uma tabela, como aquela obtida pelo aninhamento de construtores de tipos. Isso é semelhante a usar o construtor de tipo *struct* da Seção 11.1.3. Por exemplo, na Figura 11.4(a), o UDT TIPO_END_RUA é utilizado como tipo para o atributo END_RUA no UDT TIPO_END_BRASIL. De modo semelhante, o UDT TIPO_END_BRASIL, por sua vez, é usado como tipo para o atributo END no

(a)
```
CREATE TYPE TIPO_END_RUA AS (
        NUMERO          VARCHAR(5),
        NOME_RUA        VARCHAR(25),
        NR_APTO         VARCHAR(5),
        NR_BLOCO        VARCHAR(5)
    );
    CREATE TYPE TIPO_END_BRASIL AS (
        END_RUA         TIPO_END_RUA,
        CIDADE          VARCHAR(25),
        CEP             VARCHAR(10)
    );
    CREATE TYPE TIPO_TELEFONE_BRASIL AS (
        TIPO_TELEFONE   VARCHAR (5),
        CODIGO_AREA     CHAR(3),
        NUM_TELEFONE    CHAR(7)
    );
```

(b)
```
CREATE TYPE TIPO_PESSOA AS (
        NOME            VARCHAR(35),
        SEXO            CHAR,
        DATA_NASCIMENTO DATE,
        TELEFONES       TIPO_TELEFONE_BRASIL ARRAY [4],
        END             TIPO_END_BRASIL
    INSTANTIABLE
    NOT FINAL
    REF IS SYSTEM GENERATED

    INSTANCE METHOD IDADE( ) RETURNS INTEGER;
    CREATE INSTANCE METHOD IDADE( ) RETURNS INTEGER
        FOR TIPO_PESSOA
        BEGIN
            RETURN /* CÓDIGO PARA CALCULAR A IDADE DE UMA PESSOA COM BASE NA
                      DATA DE HOJE E SUA DATA_NASCIMENTO */
        END;
    );
```

(c)
```
CREATE TYPE TIPO_NOTA AS (
        DISCIPLINA      CHAR(8),
        SEMESTRE        VARCHAR(8),
        ANO             CHAR (4),
        NOTA            CHAR
    );
    CREATE TYPE TIPO_ALUNO UNDER TIPO_PESSOA AS (
        CODIGO_CURSO    CHAR(4),
        COD_ALUNO       CHAR(12),
        GRAU            VARCHAR(5),
        HISTORICO_ESCOLAR TIPO_NOTA ARRAY [100]
```

(*continua*)

Figura 11.4
Ilustrando alguns dos recursos de objeto da SQL. (a) Usando UDTs como tipos para atributos como Endereco e Telefone. (b) Especificando UDT para TIPO_PESSOA. (c) Especificando UDTs para TIPO_ALUNO e TIPO_FUNCIONARIO como dois subtipos de TIPO_PESSOA.

```
    INSTANTIABLE
    NOT FINAL
    INSTANCE METHOD COEFICIENTE( ) RETURNS FLOAT;
    CREATE INSTANCE METHOD COEFICIENTE( ) RETURNS FLOAT
        FOR TIPO_ALUNO
        BEGIN
            RETURN /* CÓDIGO PARA CALCULAR COEFICIENTE MEDIO DE UM ALUNO COM BASE EM
                      SEU HISTORICO ESCOLAR */
        END;
);
CREATE TYPE TIPO_FUNCIONARIO UNDER TIPO_PESSOA AS (
    CODIGO_EMPREGO      CHAR(4),
    SALARIO             FLOAT,
    CPF                 CHAR(11)
INSTANTIABLE
NOT FINAL
);
CREATE TYPE TIPO_GERENTE UNDER TIPO_FUNCIONARIO AS (
    DEP_GERENCIADO      CHAR(20)
INSTANTIABLE
);

(d) CREATE TABLE PESSOA OF TIPO_PESSOA
        REF IS ID_PESSOA SYSTEM GENERATED;
    CREATE TABLE FUNCIONARIO OF TIPO_FUNCIONARIO
        UNDER PESSOA;
    CREATE TABLE GERENTE OF TIPO_GERENTE
        UNDER FUNCIONARIO;
    CREATE TABLE ALUNO OF TIPO_ALUNO
        UNDER PESSOA;

(e) CREATE TYPE TIPO_EMPRESA AS (
        NOME_EMP            VARCHAR(20),
        LOCALIZACAO         VARCHAR(20));
    CREATE TYPE TIPO_EMPREGO AS (
        Funcionario REF (TIPO_FUNCIONARIO) SCOPE (FUNCIONARIO),
        Empresa REF (TIPO_EMPRESA) SCOPE (EMPRESA) );
    CREATE TABLE EMPRESA OF TIPO_EMPRESA (
        REF IS COD_EMP SYSTEM GENERATED,
        PRIMARY KEY (NOME_EMP) );
    CREATE TABLE EMPREGO OF TIPO_EMPREGO;
```

Figura 11.4 (continuação)
Ilustrando alguns dos recursos de objeto da SQL. (c) (continuação) Especificando UDTs para TIPO_ALUNO e TIPO_FUNCIONARIO como dois subtipos de TIPO_PESSOA. (d) Criando tabelas com base em alguns dos UDTs, e ilustrando a herança de tabela. (e) Especificando relacionamentos com REF e SCOPE.

UDT TIPO_PESSOA da Figura 11.4(b). Se um UDT não tiver nenhuma operação, como nos exemplos da Figura 11.4(a), é possível usar o conceito de **ROW TYPE** para criar diretamente um atributo estruturado usando a palavra-chave **ROW**. Por exemplo, poderíamos usar o seguinte em vez de declarar TIPO_END_RUA como um tipo separado, como na Figura 11.4(a):

```
CREATE TYPE TIPO_END_BRASIL AS (
  END_RUA    ROW( NUMERO       VARCHAR (5),
                  NOME_RUA     VARCHAR (25),
                  NR_AP_TO     VARCHAR (5),
                  NR_BLOCO     VARCHAR(5) ),
  CIDADE     VARCHAR (25),
  CEP        VARCHAR (10)
);
```

Para permitir tipos de coleção a fim de criar objetos estruturados complexos, quatro construtores agora estão incluídos na SQL: ARRAY, MULTISET, LIST e SET. Estes são semelhantes aos construtores de tipo discutidos na Seção 11.1.3. Na especificação inicial da SQL/Object, apenas o tipo ARRAY foi estabelecido, pois ele pode ser usado para simular os outros tipos, mas os três tipos de coleção adicionais foram incluídos na versão mais recente do padrão SQL. Na Figura 11.4(b), o atributo TELEFONES de TIPO_PESSOA tem como tipo um array cujos elementos são do UDT previamente definido TIPO_TELEFONE_BRASIL. Esse array tem um máximo de quatro elementos, significando que podemos armazenar até quatro números de telefone por pessoa. Um array também pode não ter um número máximo de elementos, se isso for desejado.

Um tipo de array pode ter seus elementos referenciados usando a notação comum dos colchetes. Por exemplo, TELEFONES[1] refere-se ao valor do primeiro local em um atributo TELEFONES (ver Figura 11.4(b)). Uma função embutida **CARDINALITY** pode retornar o número atual de elementos em um array (ou qualquer outro tipo de coleção). Por exemplo, TELEFONES[**CARDINALITY** (TELEFONES)] refere-se ao último elemento no array.

A notação de ponto comumente utilizada serve para se referir aos componentes de um **ROW TYPE** ou um UDT. Por exemplo, END.CIDADE refere-se ao componente CIDADE de um atributo END (ver Figura 11.4(b)).

11.2.2 Identificadores de objeto usando tipos de referência

Identificadores de objeto gerados pelo sistema podem ser criados por meio do **tipo de referência** na versão mais recente da SQL. Por exemplo, na Figura 11.4(b), a frase:

REF IS SYSTEM GENERATED

indica que, sempre que um objeto TIPO_PESSOA for criado, o sistema lhe atribuirá um identificador único, gerado pelo sistema. Também é possível não ter um identificador de objeto gerado pelo sistema e usar as chaves tradicionais do modelo relacional básico, se isso for desejado.

Em geral, o usuário pode especificar que devem ser criados identificadores de objeto gerados pelo sistema para linhas individuais em uma tabela. Ao usar a sintaxe:

REF IS <OID_ATRIBUTO>
<METODO_GERACAO_VALOR> ;

o usuário declara que o atributo chamado <OID_ATRIBUTO> será usado para identificar tuplas individuais na tabela. As opções para <METODO_GERACAO_VALOR> são SYSTEM GENERATED ou DERIVED. No primeiro caso, o sistema gerará automaticamente um identificador único (exclusivo) para cada tupla. No segundo caso, é aplicado o método tradicional de uso do valor de chave primária fornecido pelo usuário para identificar as tuplas.

11.2.3 Criando tabelas baseadas nos UDTs

Para cada UDT especificado para ser instanciável por meio da frase **INSTANTIABLE** (ver Figura 11.4(b)), uma ou mais tabelas podem ser criadas. Isso é ilustrado na Figura 11.4(d), onde criamos uma tabela PESSOA com base no UDT TIPO_PESSOA. Observe que os UDTs da Figura 11.4(a) são não instanciáveis e, portanto, só podem ser usados como tipos para atributos, e não como base para a criação de tabela. Na Figura 11.4(d), o atributo ID_PESSOA manterá o identificador de objeto gerado pelo sistema sempre que um novo registro (objeto) PESSOA for criado e inserido na tabela.

11.2.4 Encapsulamento de operações

Em SQL, um **tipo definido pelo usuário** pode ter a própria especificação comportamental ao definir métodos (uma especificação de UDT com métodos é a seguinte:

```
CREATE TYPE <NOME-TIPO> (
    <LISTA DE ATRIBUTOS DE COMPONENTE E
     SEUS TIPOS>
    <DECLARACAO DE FUNCOES (METODOS)>
);
```

Por exemplo, na Figura 11.4(b), declaramos um método Idade( ) que calcula a idade de um objeto individual do tipo TIPO_PESSOA.

O código para implementar o método ainda precisa ser escrito. Podemos nos referir à implementação do método especificando o arquivo que contém o código para o método, ou podemos escrever o código real na própria declaração de tipo (ver Figura 11.4(b)).

A SQL oferece certas funções embutidas para os tipos definidos pelo usuário. Para um UDT chamado TYPE_T, a **função construtora** TYPE_T(.) retorna um novo objeto desse tipo. No novo objeto UDT, cada atributo é inicializado para seu valor default. Uma **função observadora** *A* é criada implicitamente para cada atributo *A* a fim de ler seu valor. Logo, *A*(*X*) ou *X*.*A* retorna o valor do atributo *A* de TYPE_T se *X* é do tipo TYPE_T. Uma **função alteradora (mutator function)** para atualizar um atributo define um novo valor para o atributo. A SQL permite que essas funções sejam bloqueadas contra uso público. Um privilégio EXECUTE é necessário para se ter acesso a essas funções.

Em geral, um UDT pode ter uma série de funções definidas pelo usuário associadas a ele. A sintaxe é

INSTANCE METHOD <NOME>
(<LISTA_ARGUMENTOS>) **RETURNS**
<TIPO_RETORNO>;

Dois tipos de funções podem ser definidos: SQL interna e externa. Funções internas são escritas na linguagem PSM estendida de SQL (ver Capítulo 13). Funções externas são escritas em uma linguagem hospedeira (host), com apenas sua assinatura (interface) aparecendo na definição do UDT. Uma definição de função externa pode ser declarada da seguinte maneira:

DECLARE EXTERNAL <NOME_FUNCAO>
<ASSINATURA> **LANGUAGE** <NOME_
LINGUAGEM>;

Atributos e funções nos UDTs são divididos em três categorias:

- PUBLIC (visíveis na interface do UDT).
- PRIVATE (não visíveis na interface do UDT).
- PROTECTED (visíveis apenas aos subtipos).

Também é possível definir atributos virtuais como parte dos UDTs, que são calculados e atualizados usando funções.

11.2.5 Especificando herança e sobrecarga de funções

Lembre-se de que já discutimos muitos dos princípios de herança na Seção 11.1.5. A SQL tem regras para lidar com **herança de tipo** (especificada por meio da palavra-chave UNDER). Em geral, atributos e métodos (operações) de instância são herdados. A frase NOT FINAL precisa ser incluída em um UDT se os subtipos puderem ser criados sob esse UDT (ver Figura 11.4(b) e (c), onde TIPO_PESSOA, TIPO_ALUNO e TIPO_FUNCIONARIO são declarados como NOT FINAL). Associadas à herança de tipo estão as regras para a sobrecarga de implementações de função e para resolução de nomes de função. Essas regras de herança podem ser resumidas da seguinte forma:

- Todos os atributos são herdados.
- A ordem dos supertipos na cláusula UNDER determina a hierarquia de herança.
- Uma instância de um subtipo pode ser usada em cada contexto em que uma instância do supertipo é utilizada.
- Um subtipo pode redefinir qualquer função que é definida em seu supertipo, com a restrição de que a assinatura seja a mesma.
- Quando uma função é chamada, a melhor combinação é selecionada com base nos tipos de todos os argumentos.
- Para a ligação dinâmica, a execução dos tipos de parâmetros são considerados.

Considere os seguintes exemplos para ilustrar a herança de tipo, que são ilustrados na Figura 11.4(c). Suponha que queiramos criar dois subtipos de TIPO_PESSOA: TIPO_FUNCIONARIO e TIPO_ALUNO. Além disso, também criamos um subtipo TIPO_GERENTE que herda todos os atributos (e métodos) de TIPO_FUNCIONARIO, mas tem um atributo adicional DEP_GERENCIADO. Esses subtipos são mostrados na Figura 11.4(c).

Em geral, especificamos os atributos locais e quaisquer métodos específicos para o subtipo, que herda os atributos e operações de seu supertipo.

Outra facilidade em SQL é a **herança de tabela** por meio da facilidade de supertabela/subtabela. Isso também é especificado usando a palavra-chave UNDER (ver Figura 11.4(d)). Aqui, um novo registro que é inserido em uma subtabela, digamos, a tabela GERENTE, também é inserido em suas supertabelas FUNCIONARIO e PESSOA. Observe que, quando um registro é inserido em GERENTE, temos de oferecer valores para todos os seus atributos herdados. Operações INSERT, DELETE e UPDATE são propagadas corretamente.

11.2.6 Especificando relacionamentos por referência

Um atributo componente de uma tupla pode ser uma **referência** (especificada usando a palavra-chave **REF**) a uma tupla de outra tabela (ou possivelmente a mesma). Um exemplo é mostrado na Figura 11.4(e).

A palavra-chave **SCOPE** especifica o nome da tabela cujas tuplas podem ser referenciadas pelo atributo de referência. Observe que isso é semelhante a uma chave estrangeira, exceto que o valor gerado pelo sistema é usado, em vez do valor da chave primária.

A SQL usa uma **notação de ponto** para montar **expressões de caminho** que se referem aos atributos componentes de tuplas e tipos de linha. Porém, para um atributo cujo tipo é REF, o símbolo de desreferência —> é utilizado. Por exemplo, a consulta a seguir recupera os funcionários que trabalham na empresa chamada 'ABCXYZ' consultando a tabela EMPREGO:

SELECT *F*.Funcionario—>NOME
FROM EMPREGO **AS** E
WHERE *E*.Empresa—>NOME_EMP = 'ABCXYZ';

Em SQL, —> é usado para **desreferenciar** e tem o mesmo significado atribuído a ele na linguagem de programação C. Assim, se *r* é uma referência a uma tupla e *a* é um atributo componente nela, então *r* —> *a* é o valor do atributo *a* nessa tupla.

Se existirem várias relações do mesmo tipo, a SQL oferece a palavra-chave SCOPE, pela qual um atributo de referência pode ser feito para apontar para uma tupla dentro de uma tabela específica daquele tipo.

11.3 O modelo de objeto ODMG e a Object Definition Language (ODL)

Conforme discutimos na introdução ao Capítulo 4, um dos motivos para o sucesso dos SGBDs relacionais é o padrão SQL. A falta de um padrão para os SGDOs por vários anos pode ter evitado que alguns usuários em potencial convertessem para a nova tecnologia. Subsequentemente, um consórcio de vendedores e usuários de SGDO, chamado ODMG (*Object Data Management Group*), propôs um padrão conhecido como padrão ODMG-93 ou ODMG 1.0. Este foi revisado para o ODMG 2.0 e, mais tarde, para o ODMG 3.0. O padrão é composto de várias partes, incluindo o **modelo de objeto**, a **Object Definition Language** (**ODL**), a **Object Query Language** (**OQL**) e os **bindings** com as linguagens de programação orientadas a objeto.

Nesta seção, vamos descrever o modelo de objeto ODMG e a ODL. Na Seção 11.4, discutimos como projetar um BDO com base em um esquema conceitual EER. Mostraremos uma visão geral da OQL na Seção 11.5 e o vínculo da linguagem C++ na Seção 11.6. Alguns exemplos de como usar ODL, OQL e o binding da linguagem C++ utilizarão o exemplo de banco de dados UNIVERSIDADE introduzido no Capítulo 8. Em nossa descrição, seguiremos o modelo de objeto ODMG 3.0, conforme descrito em Cattell et al. (2000).[21] É importante observar que muitas das ideias incorporadas no modelo de objeto ODMG são baseadas em duas décadas de pesquisa em modelagem conceitual e bancos de dados de objeto por muitos pesquisadores.

A incorporação de conceitos de objeto no padrão de banco de dados relacional SQL, levando à tecnologia objeto-relacional, foi apresentada na Seção 11.2.

11.3.1 Visão geral do modelo de objeto do ODMG

O **modelo de objeto ODMG** é o modelo de dados no qual a linguagem de definição de objeto (ODL) e a linguagem de consulta de objeto (OQL) são baseadas. Ele serve para oferecer um modelo de dados padrão para os bancos de dados de objeto, assim como a SQL descreve um modelo de dados padrão para bancos de dados relacionais. Ele também oferece uma terminologia padrão em um campo onde os mesmos termos às vezes eram usados para descrever diferentes conceitos. Tentaremos aderir à terminologia ODMG neste capítulo. Muitos dos conceitos no modelo ODMG já foram discutidos na Seção 11.1, e consideramos que o leitor leu essa seção. Faremos a indicação sempre que a terminologia do ODMG diferir daquela usada na Seção 11.1.

Objetos e literais Objetos e literais são os blocos básicos de montagem do modelo de objeto. A principal diferença entre os dois é que um objeto tem um identificador de objeto e um **estado** (ou valor atual), enquanto um literal tem um valor (estado), mas *nenhum identificador de objeto*.[22] Nos dois casos, o valor pode ter uma estrutura complexa. O estado do objeto pode mudar com o tempo, modificando seu valor. Um literal é basicamente um valor constante, com possibilidade de ter uma estrutura complexa, no entanto ele não muda.

Um **objeto** tem cinco aspectos: identificador, nome, tempo de vida, estrutura e criação.

[21] As versões mais antigas do modelo de objeto foram publicadas em 1993 e 1997.

[22] Usaremos os termos *valor* e *estado* para indicar a mesma coisa aqui.

1. O **identificador do objeto** é um identificador único de todo o sistema (ou **Object_id**).[23] Todo objeto precisa ter um identificador de objeto.
2. Alguns objetos podem opcionalmente receber um **nome** único dentro de um SGDO em particular — esse nome pode ser usado para localizar o objeto, e o sistema deve retornar o objeto que recebeu tal nome.[24] Obviamente, nem todos os objetos individuais possuem nomes exclusivos. Com frequência, alguns objetos, principalmente aqueles que mantêm coleções de objetos de um tipo de objeto em particular — como extensões —, terão um nome. Esses nomes são utilizados como **pontos de entrada** para o banco de dados; ou seja, ao localizar esses objetos por seu nome único, o usuário pode então localizar outros objetos que são referenciados com base nesses. Outros objetos importantes na aplicação também podem ter nomes exclusivos, e é possível dar mais de um nome a um objeto. Todos os nomes em um SGDO em particular precisam ser exclusivos.
3. O **tempo de vida** de um objeto especifica se ele é um *objeto persistente* (ou seja, um objeto do banco de dados) ou um *objeto transiente* (ou seja, um objeto em um programa em execução, que desaparece após o término do programa). Os tempos de vida são independentes dos tipos; ou seja, alguns objetos de um tipo em particular podem ser transientes, enquanto outros podem ser persistentes.
4. A **estrutura** de um objeto especifica como ele é moldado usando os construtores de tipo. A estrutura especifica se um objeto é *atômico* ou não. Um **objeto atômico** refere-se a um único objeto que segue um tipo definido pelo usuário, como Funcionario ou Departamento. Se um objeto não é atômico, então ele será composto de outros objetos. Por exemplo, um *objeto de coleção* não é um objeto atômico, pois seu estado será uma coleção de outros objetos.[25] O termo *objeto atômico* é diferente de como definimos o *construtor de átomo* na Seção 11.1.3, que se referia a todos os valores de tipos de dados embutidos. No modelo ODMG, um objeto atômico é qualquer *objeto individual definido pelo usuário*. Todos os valores dos tipos de dados embutidos básicos são considerados *literais*.
5. A **criação** do objeto refere-se à maneira como ele pode ser criado. Isso normalmente é realizado por meio de uma operação *new* para uma interface. Vamos descrever isso com detalhes mais adiante nesta seção.

No modelo de objeto, um **literal** é um valor que *não tem* um identificador de objeto. Porém, o valor pode ter uma estrutura simples ou complexa. Existem três tipos de literais: atômicas, estruturadas e de coleção.

1. **Literais atômicas**[26] correspondem aos valores dos tipos de dados básicos e são predefinidas. Os tipos de dados básicos do modelo de objeto incluem long, short e números inteiros sem sinal (estes são especificados pelas palavras-chave **long**, **short**, **unsigned long** e **unsigned short** em ODL), números de ponto flutuante de precisão normal e dupla (**float**, **double**), valores booleanos (**boolean**), caracteres isolados (**char**), cadeias de caracteres (**string**) e tipos de enumeração (**enum**), entre outros.
2. **Literais estruturadas** correspondem aproximadamente aos valores que são construídos usando o construtor de tupla descrito na Seção 11.1.3. As literais estruturadas embutidas incluem Date, Interval, Time e Timestamp (ver Figura 11.5(b)). Outras literais estruturadas definidas pelo usuário podem ser estabelecidas conforme a necessidade de cada aplicação.[27] As estruturas definidas pelo usuário são criadas usando a palavra-chave **STRUCT** em ODL, assim como nas linguagens de programação C e C++.
3. **Literais de coleção** especificam um valor literal que é uma coleção de objetos ou valores, mas a coleção em si não tem um OID. As coleções no modelo de objeto podem ser definidas pelos *geradores de tipo* **set**<*T*>, **bag**<*T*>, **list**<*T*> e **array**<*T*>, onde *T* é o tipo dos objetos ou valores na coleção.[28] Outro tipo de coleção é **dictionary**<*K*, *V*>, que é uma coleção de associações <*K*, *V*>, onde *K* é uma chave (um valor de pesquisa exclusivo) associada a um valor *V*. Este pode ser usado para criar um índice sobre uma coleção de valores *V*.

[23] Isso corresponde ao OID da Seção 11.1.2.

[24] Isso corresponde ao mecanismo de nomeação para persistência, descrito na Seção 11.1.4.

[25] No modelo ODMG, *objetos atômicos* não correspondem a objetos cujos valores são tipos de dados básicos. Todos os valores básicos (inteiros, reais etc.) são considerados *literais*.

[26] O uso da palavra *atômica* em *literal atômica* corresponde ao modo como usamos o construtor de átomo na Seção 11.1.3.

[27] As estruturas para Date, Interval, Time e Timestamp podem ser usadas para criar valores literais ou objetos com identificadores.

[28] Estes são semelhantes aos construtores de tipo correspondentes na Seção 11.1.3.

```
(a) interface Object {
        ...
        boolean         same_as(in object other_object);
        object          copy();
        void            delete();
    };

(b) Class Date : Object {
        enum            Weekday
                        { Sunday, Monday, Tuesday, Wednesday,
                          Thursday, Friday, Saturday };
        enum            Month
                        { January, February, March, April, May, June,
                          July, August, September, October, November,
                          December };
        unsigned short  year();
        unsigned short  month();
        unsigned short  day();
        ...
        boolean         is_equal(in Date other_date);
        boolean         is_greater(in Date other_date);
        ...   };
    Class Time : Object {
        ...
        unsigned short  hour();
        unsigned short  minute();
        unsigned short  second();
        unsigned short  milisecond();
        ...
        boolean         is_equal(in Time a_time);
        boolean         is_greater(in Time a_time);
        ...
        Time            add_interval(in Interval an_interval);
        Time            subtract_interval(in Interval an_interval);
        Interval        subtract_time(in Time other_time);   };
    class Timestamp : Object {
        ...
        unsigned short  year();
        unsigned short  month();
        unsigned short  day();
        unsigned short  hour();
        unsigned short  minute();
        unsigned short  second();
        unsigned short  milisecond();
        ...
        Timestamp       plus(in Interval an_interval);                    (continua)
```

Figura 11.5
Visão geral das definições de interface para parte do modelo de objeto ODMG. (a) A interface object, herdada por todos os objetos. (b) Algumas interfaces-padrão para literais estruturadas.

```
        Timestamp           minus(in Interval an_interval);
        boolean             is_equal(in Timestamp a_timestamp);
        boolean             is_greater(in Timestamp a_timestamp);
        ... };
        class Interval:     Object {
        unsigned short      day();
        unsigned short      hour();
        unsigned short      minute();
        unsigned short      second();
        unsigned short      milisecond();
        ...
        Interval            plus(in Interval an_interval);
        Interval            minus(in Interval an_interval);
        Interval            product(in long a_value);
        Interval            quotient(in long a_value);
        boolean             is_equal(in interval an_interval);
        boolean             is_greater(in interval an_interval);
        ... };

(c) interface Collection : Object {
        ...
        exception           ElementNotFound{Object element;};
        unsigned long       cardinality();
        boolean             is_empty();
        ...
        boolean             contains_element(in Object element);
        void                insert_element(in Object element);
        void                remove_element(in Object element)
                                raises(ElementNotFound);
        iterator            create_iterator(in boolean stable);
        ... };

    interface Iterator {
        exception           NoMoreElements();
        ...
        boolean             at_end();
        void                reset();
        Object              get_element() raises(NoMoreElements);
        void                next_position() raises(NoMoreElements);
        ... };
    interface set : Collection {
        set                 create_union(in set other_set);
        ...
        boolean             is_subset_of(in set other_set);
        ... };
```

(*continua*)

Figura 11.5 (continuação)
Visão geral das definições de interface para parte do modelo de objeto ODMG. (b) (continuação) Algumas interfaces-padrão para literais estruturadas. (c) Interfaces para coleções e objetos de iteração.

```
interface bag : Collection {
        unsigned long           occurrences_of(in Object element);
        bag                     create_union(inBagother_bag);
        ... };
interface list : Collection {
        exception               Invalid_Index{unsigned_longindex;);
        void                    remove_element_at(inunsignedlongindex)
                                    raises(InvalidIndex);
        Object                  retrieve_element_at(in unsigned long index)
                                    raises(InvalidIndex);
        void                    replace_element_at(in Object element, in unsiged long index)
                                    raises(InvalidIndex);
        void                    insert_element_after(in Object element, in unsigned long index)
                                    raises(InvalidIndex);
        ...
        void                    insert_element_first(in Object element);
        ...
        void                    remove_first_element( ) raises(ElementNotFound);
        ...
        Object                  retrieve_first_element( ) raises(ElementNotFound);
        ...
        list                    concat(inlistother_list);
        void                    append(inlistother_list);
};
interface array : Collection {
        exception               Invalid_Index{unsigned_longindex;};
        exception               Invalid_Size{unsigned_long size;};
        void                    remove_element_at(inunsignedlongindex)
                                    raises(InvalidIndex);
        Object                  retrieve_element_at(in unsigned long index)
                                    raises(InvalidIndex);
        void                    replace_element_at(in unsigned long index, in Object element)
                                    raises(InvalidIndex);
        void                    resize(inunsignedlongnew_size)
                                    raises(InvalidSize);
};
struct association { Object key; Object value; };
interface dictionary : Collection {
        exception               DuplicateName{stringkey;};
        exception               KeyNotFound{Objectkey;};
        void                    bind(inObjectkey,inObjectvalue)
                                    raises(DuplicateName);
        void                    unbind(inObjectkey)raises(KeyNotFound);
        Object                  lookup(in Objectkey)raises(KeyNotFound);
        boolean                 contains_key(inObjectkey);
};
```

A Figura 11.5 mostra uma visão simplificada dos tipos básicos e geradores de tipo do modelo de objeto. A notação do ODMG usa três conceitos: interface, literal e class. Seguindo a terminologia do ODMB, usamos a palavra **comportamento** para nos referirmos às *operações* e **estado** para nos referirmos às *propriedades* (atributos e relacionamentos). Uma interface especifica apenas o comportamento de um tipo de objeto e normalmente é **não instanciável** (ou seja, nenhum objeto é criado correspondente a uma interface). Embora uma interface possa ter propriedades de estado (atributos e relacionamentos) como parte de suas especificações, estas *não* podem ser herdadas da interface. Logo, uma interface serve para definir operações que podem ser *herdadas* por outras interfaces, assim como por classes que definem os objetos definidos pelos usuários para determinada aplicação. Uma class especifica tanto o estado (atributos) quanto o comportamento (operações) de um tipo de objeto, e é **instanciável**. Assim, objetos de banco de dados e aplicação normalmente são criados com base nas declarações de classe especificadas pelo usuário que formam um esquema de banco de dados. Finalmente, uma declaração **literal** especifica o estado, mas não o comportamento. Dessa forma, uma instância literal mantém um valor estruturado simples ou complexo, mas não tem um identificador de objeto nem operações encapsuladas.

A Figura 11.5 é uma versão simplificada do modelo de objeto. Para especificações completas, consulte Cattell et al. (2000). Descreveremos algumas das construções mostradas na Figura 11.5 à medida que descrevermos o modelo de objeto. Em um modelo de objeto, todos os objetos herdam as operações de interface básicas de Object, mostradas na Figura 11.5(a); estas incluem operações como copy (cria uma cópia do objeto), delete (exclui o objeto) e same_as (compara a identidade do objeto com outro objeto).[29] Em geral, as operações são aplicadas aos objetos usando a **notação de ponto**. Por exemplo, dado um objeto *O*, para compará-lo com outro objeto *P*, escrevemos

O.same_as(*P*)

O resultado retornado por essa operação é booleano e seria verdadeiro se a identidade de *P* fosse a mesma de *O*, e falso em caso contrário. De modo semelhante, para criar uma cópia *P* do objeto *O*, escrevemos

P = *O*.copy( )

Uma alternativa à notação de ponto é a **notação de seta:** *O*–>same_as(*P*) ou *O*–>copy( ).

11.3.2 Herança no modelo de objeto de ODMG

No modelo de objeto ODMG, existem dois tipos de relacionamento de herança: herança apenas de comportamento e herança de estado mais comportamento. A **herança de comportamento** também é conhecida como *herança ISA* ou de *interface*, e é especificada pela notação de dois pontos (:).[30] Logo, no modelo de objeto ODMG, a herança de comportamento requer que um supertipo seja uma interface, enquanto o subtipo poderia ser uma classe ou outra interface.

O outro relacionamento de herança, chamado **herança ESTENDIDA**, é especificado pela palavra-chave **extends**. Ele é usado para herdar estado e comportamento estritamente entre classes, de modo que o supertipo e o subtipo devem ser classes. A herança múltipla por extends não é permitida. Porém, a herança múltipla é permitida para a herança de comportamento por meio da notação de dois pontos (:). Logo, uma interface pode herdar comportamento de várias outras interfaces. Uma classe também herda comportamento de diversas interfaces por meio da notação de dois pontos (:), além de herdar o comportamento e o estado de *no máximo uma* classe por meio de extends. Na Seção 11.3.4, daremos exemplos de como esses dois relacionamentos de herança — : e extends — podem ser usados.

11.3.3 Interfaces e classes embutidas no modelo de objeto

A Figura 11.5 mostra as interfaces e classes embutidas do modelo de objeto. Todas as interfaces, como Collection, Date e Time, herdam a interface Object básica. No modelo de objeto, existe uma distinção entre objetos de coleção, cujo estado contém múltiplos objetos ou literais, *versus* objetos atômicos (e estruturados), cujo estado é um objeto ou literal individual. **Objetos de coleção** herdam a interface Collection básica, mostrada na Figura 11.5(c), que mostra as operações de todos os objetos de coleção. Dado um objeto de coleção *O*, a operação *O*.cardinality( ) retorna o número de elementos na coleção. A operação *O*.is_empty( ) retorna verdadeira se a coleção *O* for vazia, e retorna falsa em caso contrário. As operações *O*.insert_element(E) e *O*.remove_element(*E*) inserem e

[29] Operações adicionais são definidas sobre objetos para fins de ***bloqueio***, mas não aparecem na Figura 11.5. Discutiremos os conceitos de bloqueio para bancos de dados no Capítulo 22.

[30] O relatório do ODMG também chama a herança de interface de relacionamentos tipo/subtipo, 'é-um' e generalização/especialização, embora, na literatura, esses termos tenham sido usados para descrever a herança tanto de estado quanto de operações (ver Capítulo 8 e Seção 11.1).

removem um elemento *E* da coleção *O*, respectivamente. Por fim, a operação *O*.contains_element(*E*) retorna verdadeira se a coleção *O* incluir o elemento *E*, e retorna falsa em caso contrário. A operação *I* = *O*.create_iterator( ) cria um **objeto de iteração** *I* para o objeto de coleção *O*, que pode percorrer cada elemento na coleção. A interface para os objetos de iteração também aparece na Figura 11.5(c). A operação *I*.reset( ) define o objeto de iteração para o primeiro elemento em uma coleção (para uma coleção desordenada, esse seria um elemento qualquer), e *I*.next_position( ) define o objeto de iteração para o próximo elemento. *I*.get_element( ) recupera o **elemento atual**, que é o elemento em que o objeto de iteração está posicionado atualmente.

O modelo de objeto ODMG usa **exceções** para informar erros ou condições particulares. Por exemplo, a exceção ElementNotFound (elemento não encontrado) na interface Collection seria lançada pela operação *O*.remove_element(*E*) se *E* não fosse um elemento na coleção *O*. A exceção NoMoreElements (sem mais elementos) na interface do objeto de iteração seria lançada pela operação *I*.next_position( ) se tal objeto estivesse atualmente posicionado no último elemento da coleção, e, portanto, não houvesse mais elementos a serem apontados por ele.

Objetos Collection são especializados ainda mais em set, list, bag, array e dictionary, que herdam as operações da interface Collection. Um gerador de tipo set<*T*> pode ser usado para criar objetos de modo que o valor do objeto *O* seja um *conjunto cujos elementos são do tipo T*. A interface Set inclui a operação adicional *P* = *O*.create_union(*S*) (ver Figura 11.5(c)), que retorna um novo objeto *P* do tipo set<*T*> que é a união dos dois conjuntos *O* e *S*. Outras operações semelhantes a create_union (não mostradas na Figura 11.5(c)) são create_intersection(*S*) e create_difference(*S*). Operações para comparação de conjunto incluem a operação *O*.is_subset_of(*S*), que retorna verdadeira se o objeto set *O* for um subconjunto de algum outro objeto set *S*, e retorna falsa em caso contrário. Operações semelhantes (não mostradas na Figura 11.5(c)) são is_proper_subset_of(*S*), is_superset_of(*S*) e is_proper_superset_of(*S*). O gerador de tipo bag<*T*> permite elementos duplicados na coleção e também herda a interface Collection. Ele tem três operações, create_union(*b*), create_intersection(*b*) e create_difference(*b*), que retornam um novo objeto do tipo bag<*T*>.

Um tipo de objeto list<*T*> herda as operações de Collection e pode ser usado para criar coleções onde a ordem dos elementos é importante. O valor de cada objeto *O* desse tipo é uma *lista ordenada cujos elementos são do tipo T*. Logo, podemos nos referir ao primeiro, último e *i*-ésimo elemento na lista. Além disso, quando acrescentamos um elemento à lista, temos de especificar a posição em que o elemento é inserido. Algumas das operações de list são exibidas na Figura 11.5(c). Se *O* é um objeto do tipo list<*T*>, a operação *O*.insert_element_first(*E*) insere o elemento *E* antes do primeiro elemento na lista *O*, de modo que *E* se torna o primeiro elemento na lista. Uma operação semelhante (não mostrada) é *O*.insert_element_last(*E*). A operação *O*.insert_element_after(*E*, *I*), na Figura 11.5(c), insere o elemento *E* após o *i*-ésimo elemento na lista *O* e lançará a exceção InvalidIndex se não houver um *i*-ésimo elemento em *O*. Uma outra operação semelhante (não mostrada) é *O*.insert_element_before(*E*, *I*). Para remover elementos da lista, as operações são *E* = *O*.remove_first_element( ), *E* = *O*.remove_last_element( ) e *E* = *O*.remove_element_at(*I*); essas operações removem o elemento indicado da lista e retornam o elemento como o resultado da operação. Outras operações recuperam um elemento sem removê-lo da lista. Estas são *E* = *O*.retrieve_first_element( ), *E* = *O*.retrieve_last_element( ) e *E* = *O*.retrieve_element_at(*I*). Além disso, duas operações para manipular listas são definidas. São elas *P* = *O*.concat(*I*), a qual cria uma nova lista *P* que é a concatenação das listas *O* e *I* (os elementos da lista *O* seguidos por aqueles da lista *I*), e *O*.append(*I*), que anexa os elementos da lista *I* ao final da lista *O* (sem criar um novo objeto de lista).

O tipo de objeto array<*T*> também herda as operações de Collection, e é semelhante à lista. Operações específicas para um objeto de array *O* são *O*.replace_element_at(*I*, *E*), que substitui o elemento de array na posição *I* pelo elemento *E*; *E* = *O*.remove_element_at(*I*), que recupera o *i*-ésimo elemento e o substitui por um valor NULL; e *E* = *O*.retrieve_element_at(*I*), que simplesmente recupera o *i*-ésimo elemento do array. Qualquer uma dessas operações pode lançar a exceção InvalidIndex se *I* for maior do que o tamanho do array. A operação *O*.resize(*N*) muda o número dos elementos do array para *N*.

O último tipo de objetos de coleção são do tipo dictionary<*K*,*V*>, que permite a criação de uma coleção de pares associados <*K*,*V*>, em que todos os valores *K* (chave) são únicos. Isso, por sua vez, permite a recuperação associativa de determinado par dado seu valor de chave (semelhante a um índice). Se *O* é um objeto de coleção do tipo dictionary<*K*,*V*>, então *O*.bind(*K*,*V*) vincula o valor *V* à chave *K* como uma associação <*K*,*V*> na coleção, enquanto *O*.unbind(*K*) remove a associação com chave *K* de *O*, e *V* = *O*.lookup(*K*) retorna o valor *V* associado à chave *K* em *O*. As duas últimas operações podem lançar a exceção KeyNotFound. Por fim, *O*.contains_key(*K*) retorna verdadeiro se a chave *K* existir em *O*, e retorna falso em caso contrário.

A Figura 11.6 é um diagrama que ilustra a hierarquia de herança das construções embutidas do mo-

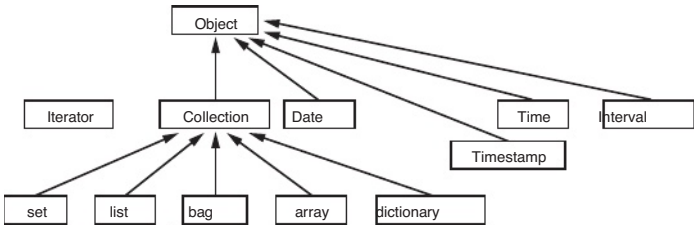


Figura 11.6
Hierarquia de herança para as interfaces embutidas do modelo de objeto.

delo de objeto. As operações são herdadas do supertipo para o subtipo. As interfaces de coleção descritas anteriormente *não são diretamente instanciáveis*, ou seja, não se pode criar diretamente objetos com base nessas interfaces. Em vez disso, as interfaces podem ser usadas para gerar tipos de coleção definidos pelo usuário — do tipo set, bag, list, array ou dictionary — para uma aplicação de banco de dados em particular. Se um atributo ou classe tem um tipo de coleção, digamos, um set, então ele herdará as operações da interface set. Por exemplo, em uma aplicação de banco de dados UNIVERSIDADE, o usuário pode especificar um tipo para set<ALUNO>, cujo estado seria conjuntos de objetos ALUNO. O programador pode, então, usar as operações para set<*T*> para manipular uma instância do tipo set<ALUNO>. A criação de classes de aplicação normalmente é feita utilizando a linguagem de definição de objeto — ODL (ver Seção 11.3.6).

É importante observar que todos os objetos em uma coleção em particular *precisam ser do mesmo tipo*. Logo, embora a palavra-chave any apareça nas especificações das interfaces de coleção na Figura 11.5(c), isso não significa que os objetos de qualquer tipo podem ser mesclados dentro da mesma coleção. Ao contrário, isso significa que qualquer tipo pode ser usado quando se especifica o tipo dos elementos para determinada coleção (incluindo outros tipos de coleção!)

11.3.4 Objetos atômicos (definidos pelo usuário)

A seção anterior descreveu os tipos de coleção embutidos do modelo de objeto. Agora, vamos discutir como os tipos de objeto para *objetos atômicos* podem ser construídos. Estes são especificados usando a palavra-chave class na ODL. No modelo de objeto, qualquer objeto definido pelo usuário, que não é um objeto de coleção, é chamado de **objeto atômico**.[31]

Por exemplo, em uma aplicação de banco de dados UNIVERSIDADE, o usuário pode especificar o tipo de objeto (class) para objetos ALUNO. A maior parte desses objetos será de **objetos estruturados**. Por exemplo, um ALUNO terá uma estrutura complexa, com muitos atributos, relacionamentos e operações, mas ainda é considerado atômico porque não é uma coleção. Esse tipo de objeto atômico definido pelo usuário é estabelecido como uma classe ao especificar suas **propriedades** e **operações**. As propriedades definem o estado do objeto e são distinguidas ainda mais em **atributos** e **relacionamentos**. Nesta subseção, detalhamos os três tipos de componentes — atributos, relacionamentos e operações — que um tipo de objeto definido pelo usuário para objetos atômicos (estruturados) pode incluir. Ilustramos nossa discussão com as duas classes, FUNCIONARIO e DEPARTAMENTO, mostradas na Figura 11.7.

Um **atributo** é uma propriedade que descreve algum aspecto de um objeto. Atributos possuem valores (os quais normalmente são literais com uma estrutura simples ou complexa) que são armazenados dentro do objeto. Porém, os valores de atributo também podem ser OIDs de outros objetos. Os valores de atributo podem até mesmo ser especificados por meio de métodos que são utilizados para calcular o valor do atributo. Na Figura 11.7,[32] os atributos para FUNCIONARIO são Nome, Cpf, Data_nascimento, Sexo e Idade, e para DEPARTAMENTO são Dnome, Dnumero, Ger, Localizacoes e Projs. Os atributos Ger e Projs de DEPARTAMENTO possuem estrutura complexa e são definidos por meio de struct, que corresponde ao *construtor de tuplas* da Seção 11.1.3. Logo, o valor de Ger em cada objeto DEPARTAMENTO terá dois componentes: Gerente, cujo valor é um OID que referencia o objeto FUNCIONARIO que gerencia o DEPARTAMENTO e Data_inicio, cujo valor é uma date.

[31] Como dissemos anteriormente, essa definição de *objeto atômico* no modelo de objeto ODMG é diferente da definição do construtor de átomo da Seção 11.1.3, que é a definição usada em grande parte da literatura de banco de dados orientada a objeto.

[32] Estamos usando a notação da Object Definition Language (ODL) na Figura 11.7, que será discutida com mais detalhes na Seção 11.3.6.

O atributo localizacoes de DEPARTAMENTO é definido por meio do construtor set, pois cada objeto DEPARTAMENTO pode ter um conjunto de locais.

Um relationship é uma propriedade que especifica que dois objetos no banco de dados estão relacionados. No modelo de objeto do ODMG, somente relacionamentos binários (ver Seção 7.4) são representados explicitamente, e cada relacionamento binário é representado por um *par de referências inversas* especificadas por meio do relationship de palavra-chave. Na Figura 11.7, existe um relacionamento que relaciona cada FUNCIONARIO ao DEPARTAMENTO em que trabalha — o relacionamento Trabalha_para de FUNCIONARIO. Na direção inversa, cada DEPARTAMENTO está relacionado ao conjunto de FUNCIONARIOS que trabalha no DEPARTAMENTO — o relacionamento Tem_funcs de DEPARTAMENTO. A palavra-chave inverse especifica que essas duas propriedades definem um único relacionamento conceitual nas direções inversas.[33]

```
class FUNCIONARIO
(       extent          TODOS_FUNCIONARIOS
        key             Cpf)
{
        attribute       string                  Nome;
        attribute       string                  Cpf;
        attribute       date                    Data_nascimento;
        attribute       enum Genero {M,F}       Sexo;
        attribute       short                   Idade;
        relationship    DEPARTAMENTO            Trabalha_para
                        inverse DEPARTAMENTO::Tem_funcs;
        void            realoca_func(in string Novo_dnome)
                        raises(dnome_invalido);
};
class DEPARTAMENTO
(       extent          TODOS_DEPARTAMENTOS
        key                                     Dnome, Dnumero)
{
        attribute       string                  Dnome;
        attribute       short                   Dnumero;
        attribute       struct Ger_proj {FUNCIONARIO Gerente,
                        date Data_inicio} Ger;
        attribute       set<string>             Localizacoes;
        attribute       struct Projs {string Proj_nome,
                        time Horas_semana) Projs;
        relationship    set<FUNCIONARIO>        Tem_funcs inverse FUNCIONARIO::Trabalha_para;
        void            adiciona_func(in string Novo_fnome) raises(fnome_invalido);
        void            troca_gerente(in string Novo_ger_nome; in date
                        Data_inicio);
};
```

Figura 11.7
Os atributos, relacionamentos e operações em uma definição de classe.

[33] A Seção 7.4 discute como um relacionamento pode ser representado por dois atributos em direções inversas.

Ao especificar inversos, o sistema de banco de dados pode manter a integridade referencial do relacionamento automaticamente. Ou seja, se o valor de Trabalha_para para determinado FUNCIONARIO *F* refere-se ao DEPARTAMENTO *D*, então o valor de Tem_funcs para o DEPARTAMENTO *D* precisa incluir uma referência a *F* em seu conjunto de referências de FUNCIONARIO . Se o projetista de banco de dados quiser ter um relacionamento para ser representado em *apenas uma direção* , então ele precisa ser modelado como um atributo (ou operação). Um exemplo é o componente Gerente do atributo Ger em DEPARTAMENTO.

Além dos atributos e relacionamentos, o projetista pode incluir **operações** nas especificações de tipo de objeto (class). Cada tipo de objeto pode ter uma série de **assinaturas de operação** , que especificam o nome da operação, seus tipos de argumento e seu valor retornado, se for o caso. Os nomes de operação são exclusivos dentro de cada t ipo de objeto, mas eles podem ser sobrecarregados, fazendo que o mesmo nome de operação apareça em tipos de objeto distintos. A assinatura da operação também pode especificar os nomes das **exceções** sujeitas a ocorrer durante a execução da operação. A implementação da operação incluirá o código para lançar essas exceções. Na Figura 11.7, a classe FUNCIONARIO tem uma operação: realoca_func, e a classe DEPARTAMENTO tem duas operações: adiciona_func e troca_gerente .

11.3.5 Extensões, chaves e fábrica de objetos

No modelo de objeto ODMG, o projetista de banco de dados pode declarar uma *extensão* (usando a palavra-chave **extent**) para qualquer tipo de objeto que seja definido por meio de uma declaração **class**. A extent recebe um nome e terá todos os objetos persistentes dessa classe. Logo, a extent comporta-se como um *objeto de conjunto* que mantém todos os objetos persistentes da classe. Na Figura 11.7, as classes FUNCIONARIO e DEPARTAMENTO possuem extensões chamadas TODOS_FUNCIONARIOS e TODOS_DEPARTAMENTOS, respectivamente. Isso é semelhante a criar dois objetos — um do tipo set<FUNCIONARIO> e outro do tipo set<DEPARTAMENTO> — e torná-los persistentes chamando-os de TODOS_FUNCIONARIOS e TODOS_DEPARTAMENTOS. As extensões também são usadas para impor automaticamente o relacionamento de conjunto/subconjunto entre as extensões de um supertipo e seu subtipo. Se duas classes A e B possuem extensões TODOS_A e TODOS_B, e a classe B é um subtipo da classe A (ou seja, a classe B **extends** class A), então a coleção de objetos em TODOS_B precisa ser um subconjunto daqueles em TODOS_A em qualquer ponto. Essa restrição é imposta automaticamente pelo sistema de banco de dados.

Uma classe com uma extensão pode ter uma ou mais chaves. Uma **chave** consiste em uma ou mais propriedades (atributos ou relacionamentos), cujos valores são restritos a serem únicos para cada objeto na extensão. Por exemplo, na Figura 11.7, a classe FUNCIONARIO tem o atributo Cpf como chave (cada objeto FUNCIONARIO na extensão precisa ter um valor de Cpf único), e a classe DEPARTAMENTO tem duas chaves distintas: Dnome e Dnumero (cada DEPARTAMENTO deve ter um Dnome único e um Dnumero único). Para uma chave composta[34] que é feita de várias propriedades, as propriedades que formam a chave estão contidas em parênteses. Por exemplo, se uma classe VEICULO com uma extent TODOS_VEICULOS tem uma chave composta por uma combinação de dois atributos Estado e Placa, eles poderiam ser colocados entre parênteses, como em (Estado, Placa) na declaração de chave.

Em seguida, apresentamos o conceito de **fábrica de objeto** — um objeto que pode ser usado para gerar ou criar objetos individuais por meio de suas operações. Algumas das interfaces da fábrica de objetos que fazem parte do modelo de objeto ODMG aparecem na Figura 11.8. A interface ObjectFactory tem uma única operação, new( ), que retorna um novo objeto com um OID. Ao herdar essa interface, os usuários podem criar as próprias interfaces de fábrica para ca da tipo de objeto definido pelo usuário (atômico), e o programador pode implementar a operação *new* de forma diferente para cada tipo de objeto. A Figura 11.8 também mostra uma interface DateFactory, que tem operações adicionais para a criação de um valor de current_date, calendar_date, e para outras operações (não mostradas na Figura 11.8). Como podemos ver, uma fábrica de objeto basicamente oferece as **operações construtoras** para novos objetos.

[34] Uma chave composta é chamada de *compound key* no relatório do ODMG.

```
interface ObjectFactory {
        Object          new();
};
interface SetFactory : ObjectFactory {
        Set             new_of_size(in long size);
};
interface ListFactory : ObjectFactory {
        List            new_of_size(in long size);
};

interface ArrayFactory : ObjectFactory {
        Array           new_of_size(in long size);
};
interface DictionaryFactory : ObjectFactory {
        Dictionary      new_of_size(in long size);
};
interface DateFactory : ObjectFactory {
        exception       InvalidDate{};
        ...
        Date            calendar_date(  in unsigned short year,
                                        in unsigned short month,
                                        in unsigned short day)
                        raises(InvalidDate);
        ...
        Date            current();
};
interface DatabaseFactory {
        Database        new();
};
interface Database {
        ...
        void            open(in string database_name)
                            raises(DatabaseNotFound, DatabaseOpen);
        void            close() raises(DatabaseClosed, ...);
        void            bind(in Object an_object, in string name)
                            raises(DatabaseClosed, ObjectNameNotUnique, ...);
        Object          unbind(in string name)
                            raises(DatabaseClosed, ObjectNameNotFound, ...);
        Object          lookup(in string object_name)
                            raises(DatabaseClosed, ObjectNameNotFound, ...);
... };
```

Figura 11.8
Interfaces para ilustrar fábrica de objetos e objetos de banco de dados.

Finalmente, discutimos o conceito de um **banco de dados.** Como um SGBDO pode criar muitos bancos de dados diferentes, cada um com o próprio esquema, o modelo de objeto ODMG tem interfaces para objetos DatabaseFactory e Database, como mostra a Figura 11.8. Cada banco de dados tem o próprio *nome de banco de dados*, e a operação bind pode ser usada para atribuir nomes únicos individuais a objetos persistentes em um banco de dados em particular. A operação **lookup** retorna um objeto do banco de dados que tem o object_name especificado, e a operação unbind remove o nome de um objeto dito persistente do banco de dados.

11.3.6 A linguagem de definição de objeto ODL

Depois de nossa visão geral do modelo de objeto ODMG na seção anterior, agora vamos mostrar como esses conceitos podem ser utilizados para criar um esquema de banco de dados de objeto usando a linguagem de definição de objeto ODL.[35]

A ODL é projetada para dar suporte às construções semânticas do modelo de objeto ODMG e é independente de qualquer linguagem de programação em particular. Seu uso principal é para criar especificações de objeto — ou seja, classes e interfaces. Logo, a ODL não é uma linguagem de programação completa. Um usuário pode especificar um esquema de banco de dados na ODL independentemente de qualquer linguagem de programação, e depois usar os bindings da linguagem específica para indicar como as construções ODL podem ser mapeadas em construções nas linguagens de programação específicas, como C++, Smalltalk e Java. Daremos uma visão geral do binding com a C++ na Seção 11.6.

A Figura 11.9(b) mostra um esquema de objeto possível para parte do banco de dados UNIVERSIDADE, que foi apresentado no Capítulo 8. Descreveremos os conceitos da ODL usando esse exemplo, e aquele da Figura 11.11. A notação gráfica para a Figura 11.9(b) é mostrada na Figura 11.9(a) e pode ser considerada uma variação dos diagramas EER (ver Capítulo 8) com o conceito adicionado de herança de interface, mas sem vários conceitos de EER, como categorias (tipos de união) e atributos de relacionamentos.

A Figura 11.10 mostra um conjunto possível de definições de classes ODL para o banco de dados UNIVERSIDADE. Em geral, pode haver diversos mapeamentos possíveis a partir de um diagrama de esquema de objeto (ou diagrama de esquema EER) para classes ODL. Discutiremos melhor sobre essas opções na Seção 11.4.

A Figura 11.10 mostra o modo direto de mapear parte do banco de dados UNIVERSIDADE do Capítulo 8. Os tipos de entidade são mapeados para classes ODL, e a herança é feita usando **extends**. Porém, não existe um modo direto de mapear categorias (tipos de união) ou realizar a herança múltipla. Na Figura 11.10, as classes PESSOA, DOCENTE, ALUNO e ALUNO_POSGRADUACAO têm as extensões PESSOAS, DOCENTES, ALUNOS e ALUNO_POSGRADUACAO, respectivamente. Tanto DOCENTE quanto ALUNO **extends** PESSOA e ALUNO_POSGRADUACAO **extends** ALUNO. Logo, a coleção de ALUNOS (e a coleção de DOCENTES) será restrita a um subconjunto da coleção de PESSOA a qualquer momento. De modo semelhante, a coleção de ALUNO_POSGRADUACAO será um subconjunto de ALUNO. Ao mesmo tempo, objetos ALUNO e DOCENTE individuais herdarão as propriedades (atributos e relacionamentos) e operações de PESSOA, e objetos ALUNO_POSGRADUACAO individuais herdarão aquelas de ALUNO.

As classes DEPARTAMENTO, DISCIPLINA, TURMA e TURMA_ATUAL da Figura 11.10 são mapeamentos diretos dos tipos de entidade correspondentes da Figura 11.9(b). Porém, a classe NOTA requer alguma explicação. A classe NOTA corresponde ao relacionamento M:N entre ALUNO e TURMA na Figura 11.9(b). O motivo para isso ter sido feito em uma classe separada (em vez de em um par de relacionamentos inversos) é porque inclui o atributo de relacionamento Nota.[36]

Assim, o relacionamento M:N é mapeado para a classe NOTA, e um par de relacionamentos 1:N, um entre ALUNO e NOTA e o outro entre TURMA e NOTA.[37] Esses relacionamentos são representados pelas seguintes propriedades de relacionamento: Turma_concluida de ALUNO; Turma e Aluno de NOTA; e Alunos de TURMA (ver Figura 11.10). Por fim, a classe TITULO_ACADEMICO é usada para representar os graus de atributo compostos, multivalorados, de ALUNO_POSGRADUACAO (ver Figura 8.10).

Como o exemplo anterior não inclui quaisquer interfaces, apenas classes, agora utilizamos um exemplo diferente para ilustrar interfaces e herança de interface (comportamento). A Figura 11.11(a) faz parte de um esquema de banco de dados para armazenar objetos geométricos. Uma interface GeometriaObjeto é especificada, com operações para calcular o perímetro e a area de um objeto geométrico, mais operações para translacao (mover) e rotacao (girar) um objeto.

[35] A sintaxe e os tipos de dados ODL têm como propósito serem compatíveis com IDL — Linguagem de Definição de Interface (Interface Definition Language) de CORBA (Common Object Request Broker Architecture), com extensões para relacionamentos e outros conceitos de banco de dados.

[36] Discutiremos mapeamentos alternativos para atributos de relacionamentos na Seção 11.4.

[37] Isso é semelhante ao modo como um relacionamento M:N é mapeado no modelo relacional (ver Seção 9.1) e no modelo de rede legado (ver Apêndice E).

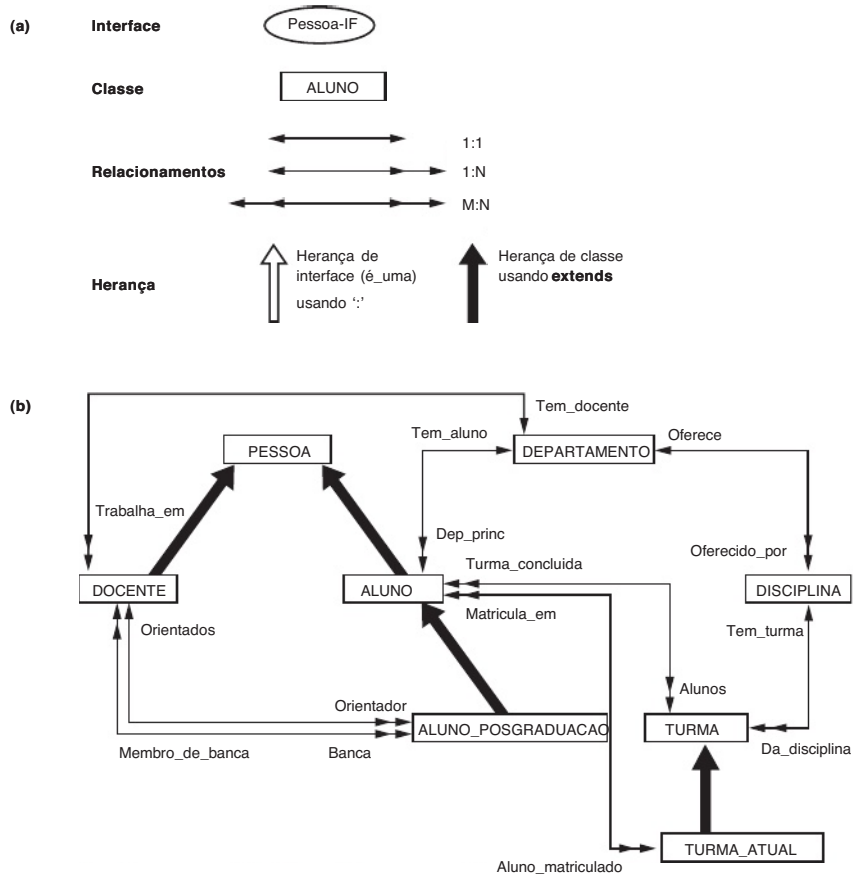


Figura 11.9
Exemplo de um esquema de banco de dados. (a) Notação gráfica para representar esquemas ODL. (b) Um esquema gráfico de banco de dados de objeto para parte do banco de dados UNIVERSIDADE (as classes NOTA e TITULO_ACADEMICO não aparecem).

Várias classes (RETANGULO, TRIANGULO, CIRCULO, ...) herdam a interface GeometriaObjeto. Como GeometriaObjeto é uma interface, ela é *não instanciável* — ou seja, nenhum objeto pode ser criado com base nessa interface diretamente. No entanto, objetos do tipo RETANGULO, TRIANGULO, CIRCULO, ... podem ser criados, e esses objetos herdam todas as operações da interface GeometriaObjeto. Observe que, com a herança de interface, somente operações são herdadas, e não propriedades (atributos, relacionamentos). Logo, se uma propriedade for necessária na classe de herança, ela precisa ser repetida na definição da classe, como no atributo Ponto_referencia da Figura 11.11(b). Observe que as operações herdadas podem ter diferentes implementações em cada caso. Por exemplo, as implementações das operações area e perimetro podem ser diferentes para RETÂNGULO, TRIANGULO e CIRCULO.

A *herança múltipla* das interfaces por uma classe é permitida, assim como a herança múltipla de interfaces por outra interface. Contudo, com a herança **extends** (classe), a herança múltipla *não é permitida*. Logo, uma classe pode herdar por **extends** até no máximo uma classe (além de herdar de zero ou mais interfaces).

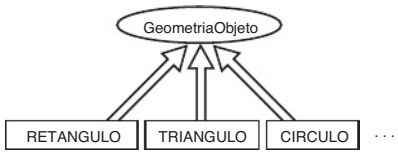
```
class PESSOA
(       extent          PESSOAS
        key             Cpf)
{       attribute       struct Projnome{        string          Pnome,
                                                string          Mnome,
                                                string          Unome}          Nome;
        attribute       string                                                  Cpf;
        attribute       date                                                    Data_nascimento;
        attribute       enum Genero{M,F}                                        Sexo;
        attribute       struct Endereco{        short           Nr,
                                                string          Rua,
                                                short           Nr_apto,
                                                string          Cidade,
                                                string          Estado,
                                                short           Cep}            Endereco;
        short           Idade();};
class DOCENTE extends PESSOA
(       extent          DOCENTES)
{       attribute       string                  Nível;
        attribute       float                   Salario;
        attribute       string                  Escritorio;
        attribute       string                  Telefone;
        relationship    DEPARTAMENTO            Trabalha_em inverse DEPARTAMENTO::Tem_docente;
        relationship    set<ALUNO_POSGRADUACAO> Orientados inverse ALUNO_
                        POSGRADUACAO::Orientador;
        relationship    set<ALUNO_POSGRADUACAO> Membro_de_banca inverse ALUNO_
                        POSGRADUACAO::Banca;
        void            dar_aumento(in float aumento);
        void            promocao(in string novo_nivel);  };
class NOTA
(       extent          NOTAS)
{
        attribute       enum Valores Nota{A,B,C,D,F,I,P} Nota;
        relationship    TURMA Turma inverse TURMA::Alunos;
        relationship    ALUNO Aluno inverse ALUNO::Turma_concluida;};
class ALUNO extends PESSOA
(       extent                  ALUNOS)
{       attribute string        Tipo_aluno;
        attribute       DEPARTAMENTO    Dep_secund;
        relationship    DEPARTAMENTO Dep_princ inverse DEPARTAMENTO::Tem_aluno;
        relationship    set<NOTA> Turma_concluida inverse NOTA::Aluno;
        relationship    set<TURMA_ATUAL> Matricula_em inverse TURMA_ATUAL::Aluno_matriculado;
        void            troca_dep_princ(in string dnome) raises(departamento_invalido);
        float           coeficiente();
        void            matricula(in short nr_turma) raises(turma_invalida);
        void            aloca_nota(in short nr_turma; IN ValorNota nota)
                        raises(turma_invalida,nota_invalida); };
class TITULO_ACADEMICO
```

Figura 11.10
Esquema ODL possível para o banco de dados UNIVERSIDADE da Figura 11.8(b).

```
{       attribute                               string                  Faculdade;
        attribute                               string                  Titulo;
        attribute                               string                  Ano);
class ALUNO_POSGRADUACAO
        extends         ALUNO
(       extent          DEPARTAMENTOS
{       attribute       set<TITULO_ACADEMICO>                           Titulos;
        relationship    DOCENTE Orientador inverse DOCENTE::Orientados;
        relationship    set<DOCENTE> Banca inverse DOCENTE::Membro_de_banca;
        void            aloca_orientador (in string Unome; in string Pnome)
                        raises(docente_invalido);
        void            aloca_membro_banca (in string Unome; in string Pnome)
                        raises(docente_invalido); };
class DEPARTAMENTO
(       extent          DEPARTAMENTOS
        key             Dnome)
{       attribute       string                  Dnome;
        attribute       string                  Dtelefone;
        attribute       string                  Descritorio;
        attribute       string                  Faculdade;
        attribute       DOCENTE                 Diretor;
        relationship    set<DOCENTE> Tem_docente inverse DOCENTE::Trabalha_em;
        relationship    set<ALUNO> Tem_aluno inverse ALUNO::Dep_princ;
        relationship    set<DISCIPLINA> Oferece inverse DISCIPLINA::Oferecida_por; };
class DISCIPLINA
(       extent          DISCIPLINAS
        key             Dnr)
{       attribute       string                  Dnome;
        attribute       string                  Dnr;
        attribute       string                  Descrição;
        relationship    set<TURMA> Tem_turma inverse TURMA::Da_disciplina;
        relationship    <DEPARTAMENTO> Oferecido_por inverse DEPARTAMENTO::Oferece; };
class TURMA
(       extent          TURMAS)
{       attribute       short                   Num_turma
        attribute       string                  Ano;
        attribute       enum Periodo {Primeiro, Segundo}
                        Semestre;
        relationship    set<NOTA> Alunos inverse Nota::Turma;
        relationship    DISCIPLINA Da_disciplina inverse DISCIPLINA::Tem_turma; };

class TURMA_ATUAL extends TURMA
(       extent          TURMAS_ATUAIS)
{       relationship    set<ALUNO> Aluno_matriculado
                        inverse ALUNO::Matricula_em
        void            matricular_aluno(in string Cpf)
                        raises(aluno_invalido, turma_cheia); };
```

(a)



(b)

```
interface GeometriaObjeto
{       attribute   enum        Forma{RETANGULO,TRIANGULO,CIRCULO,...}
                                Forma;
        attribute   struct      Ponto{short x, short y} Ponto_referencia;
        float       perimetro( );
        float       area( );
        void        translacao(in short translacao_x; in short translacao_y);
        void        rotacao(in float angulo_rotacao); };
class RECTANGLE : GeometriaObjeto
(       extent      RETANGULOS )
{       attribute   struct      Ponto{shortx,shorty}Ponto_referencia;
        attribute   short       Comprimento;
        attribute   short       Altura;
        attribute   float       Angulo_orientacao; };
class TRIANGULO : GeometriaObjeto
(       extent      TRIANGULOS  )
{       attribute   struct      Ponto{shortx,shorty}Ponto_referencia;
        attribute   short       Lado_1;
        attribute   short       Lado_2;
        attribute   float       Angulo_lado1_lado2;
        attribute   float       Angulo_orientacao_lado1; };
class CIRCULO : GeometriaObjeto
(       extent      CIRCULOS  )
{       attribute   struct      Ponto{shortx,shorty}Ponto_referencia;
        attribute   short       Raio};
```

Figura 11.11
Uma ilustração da herança de interface por meio de ':'. (a) Representação de esquema gráfico. (b) Definições de interface e classe correspondentes em ODL.

11.4 Projeto conceitual de banco de dados de objeto

A Seção 11.4.1 discute como o projeto de banco de dados de objeto (BDO) difere do projeto de banco de dados relacional (BDR). A Seção 11.4.2 esboça um algoritmo de mapeamento que pode ser usado para criar um esquema de BDO, feito de definições de classe ODL do ODMG, com base em um esquema EER conceitual.

11.4.1 Diferenças entre o projeto conceitual do BDO e do BDR

Uma das principais diferenças entre o projeto de BDO e BDR é o modo como os relacionamentos são tratados. No BDO, eles normalmente são tratados como tendo propriedades de relacionamento ou atributos de referência que incluem OID(s) dos objetos relacionados. Estes podem ser considerados *referências de OID* aos objetos relacionados. Tanto refe-

rências isoladas quanto coleções de referências são permitidas. As referências para um relacionamento binário podem ser declaradas em uma única direção, ou nas duas direções, dependendo dos tipos de acesso esperados. Se declaradas nas duas direções, elas podem ser especificadas como inversas uma da outra, impondo assim o equivalente BDO da restrição de integridade referencial do modelo relacional.

No BDR, os relacionamentos entre tuplas (registros) são especificados por atributos com valores que combinam. Estes podem ser considerados *referências de valor* e são especificados por meio de *chaves estrangeiras*, que são valores de atributos de chave primária repetidos em tuplas da relação que referencia. São limitados a serem de único valor em cada registro, pois atributos multivalorados não são permitidos no modelo relacional básico. Assim, relacionamentos M:N devem ser representados não diretamente, mas como uma relação (tabela) separada, conforme discutimos na Seção 9.1.

O mapeamento de relacionamentos binários que contém atributos não é direto nos BDOs, pois o projetista precisa escolher em que direção os atributos devem ser incluídos. Se eles forem incluídos nas duas direções, então haverá redundância no armazenamento, podendo ocasionar dados inconsistentes. Logo, às vezes é preferível usar a técnica relacional de criação de uma tabela separada ao criar uma classe separada para representar o relacionamento. Essa técnica também pode ser usada para relacionamentos *n*-ários, com grau $n > 2$.

Outra área importante da diferença entre o projeto de BDO e BDR é o modo como a herança é tratada. No BDO, essas estruturas são embutidas no modelo, de modo que o mapeamento é alcançado usando as construções de herança, como derived (:) e ***extends***. No projeto relacional, conforme discutimos na Seção 9.2, existem várias opções para escolher, pois não existe uma construção embutida para a herança no modelo relacional básico. É importante observar, porém, que os sistemas objeto-relacional e relacional estendido estão acrescentando recursos para modelar essas construções diretamente, bem como para incluir especificações de operação nos tipos de dados abstratos (ver Seção 11.2).

A terceira diferença importante é que, no projeto do BDO, é necessário especificar as operações desde cedo no projeto, pois elas fazem parte das especificações de classe. Embora seja essencial especificar operações durante a fase de projeto para todos os tipos de bancos de dados, isso pode ser adiado no projeto do BDR, pois não é estritamente exigido até a fase de implementação.

Existe uma diferença filosófica entre o modelo relacional e o modelo de objeto dos dados em relação à especificação comportamental. O modelo relacional *não* exige que os projetistas de banco de dados predefinam um conjunto de comportamentos ou operações válidas, enquanto esse é um requisito implícito no modelo de objeto. Uma das vantagens alegadas do modelo relacional é o suporte de consultas e transações ocasionais, ao passo que são contra o princípio de encapsulamento.

Na prática, está se tornando comum ter equipes de projeto de banco de dados aplicando metodologias baseadas em objeto nos estágios iniciais do projeto conceitual, de modo que tanto a estrutura quanto o uso ou operações dos dados sejam considerados, e uma especificação completa seja desenvolvida durante o projeto conceitual. Essas especificações são então mapeadas para esquemas relacionais, restrições e artefatos comportamentais, como triggers ou procedimentos armazenados (ver seções 5.2 e 13.4).

11.4.2 Mapeando um esquema EER para um esquema BDO

É relativamente simples projetar as declarações de tipo das classes de objeto para um SGBDO com base em um esquema EER que não contém *nem* categorias *nem* relacionamentos *n*-ários com $n > 2$. Porém, as operações das classes não são especificadas no diagrama EER e devem ser acrescentadas às declarações de classe após o término do mapeamento estrutural. Um esboço do mapeamento de EER para ODL é o seguinte:

Etapa 1. Crie uma *classe* ODL para cada tipo de entidade ou subclasse EER. O tipo da classe ODL deve incluir todos os atributos da classe EER. [38] *Atributos multivalorados* normalmente são declarados usando os construtores set, bag ou list. [39] Se os valores do atributo multivalorado para um objeto tiverem de ser ordenados, o construtor list é escolhido; se duplicatas forem permitidas, o construtor bag deverá ser escolhido; caso contrário, o construtor set é escolhido. *Atributos compostos* são mapeados para um construtor de tupla (usando uma declaração struct em ODL).

[38] Isso implicitamente usa um construtor de tupla no nível superior da declaração de tipo, mas, em geral, o construtor de tupla não é mostrado de maneira explícita nas declarações de classe ODL.

[39] É preciso haver uma análise melhor do domínio da aplicação para decidir qual construtor usar, pois essa informação não está disponível no esquema EER.

Declare uma extensão para cada classe e especifique quaisquer atributos de chave como chaves da extensão. (Isso só é possível se uma facilidade de extensão e declarações de restrição de chave estiverem disponíveis no SGBDO.)

Etapa 2. Inclua propriedades de relacionamento ou atributos de referência para cada *relacionamento binário* nas classes ODL que participam do relacionamento. Estas podem ser criadas em uma ou nas duas direções. Se um relacionamento binário for representado por referências nas *duas* direções, declare as referências às propriedades de relacionamento que são inversas uma da outra, se tal facilidade existir.[40] Se um relacionamento binário for representado por uma referência em apenas *uma* direção, declare a referência para que seja um atributo na classe que referencia, cujo tipo é o nome da classe referenciada.

Dependendo da razão de cardinalidade do relacionamento binário, as propriedades de relacionamento ou atributos de referência podem ser tipos de único valor ou de coleção. Eles serão de único valor para relacionamentos binários nas direções 1:1 ou N:1; e são tipos de coleção (valor de conjunto ou valor de lista[41]) para relacionamentos na direção 1:N ou M:N. Um modo alternativo de mapear os relacionamentos binários M:N é discutido na etapa 7.

Se houver atributos de relacionamento, um construtor de tupla (struct) pode ser usado para criar uma estrutura na forma < referência, atributos de relacionamento> , que pode ser incluída no lugar do atributo de referência. Contudo, isso não permite o uso da restrição inversa. Além disso, se essa escolha for representada nas *duas direções*, os valores de atributo serão representados duas vezes, criando redundância.

Etapa 3. Inclua operações apropriadas para cada classe. Estas não estão disponíveis no esquema EER e precisam ser acrescentadas ao projeto do banco de dados referenciando os requisitos srcinais. Um método construtor deverá incluir o código de programa que verifica quaisquer restrições que deverão existir quando um novo objeto for criado. Um método destruidor deve verificar quaisquer restrições que possam ser violadas quando um objeto for excluído. Outros métodos deverão incluir quaisquer outras verificações de restrição que sejam relevantes.

Etapa 4. Uma classe ODL que corresponde a uma subclasse no esquema EER herda (por **extends**) o tipo e os métodos de sua superclasse no esquema ODL. Seus atributos *específicos* (não herdados), referências de relacionamento e operações são especificados, conforme discutimos nas etapas 1, 2 e 3.

Etapa 5. Tipos de entidade fraca podem ser mapeados da mesma maneira que os tipos de entidade regulares. Um mapeamento alternativo é possível para tipos de entidade fraca que não participam de quaisquer relacionamentos, exceto seu relacionamento de identificação. Estes podem ser mapeados como se fossem *atributos multivalorados compostos* do tipo de entidade proprietário, usando os construtores set<struct<... >> ou list<struct<... >>. Os atributos da entidade fraca são incluídos na construção struct<... >, que corresponde a um construtor de tupla. Os atributos são mapeados conforme discutimos nas etapas 1 e 2.

Etapa 6. As categorias (tipos de união) em um esquema EER são difíceis de mapear para ODL. É possível criar um mapeamento semelhante ao EER-para-relacional (ver Seção 9.2), declarando uma classe para representar a categoria e definindo relacionamentos 1:1 entre a categoria e cada uma de suas superclasses. Outra opção é usar um *tipo de união*, se estiver disponível.

Etapa 7. Um relacionamento *n*-ário com grau $n > 2$ pode ser mapeado para uma classe separada, com referências apropriadas a cada classe participante. Essas referências são baseadas no mapeamento de um relacionamento 1:N de cada classe que representa um tipo de entidade participante para a classe que representa o relacionamento *n*-ário. Um relacionamento binário M:N, especialmente se tiver atributos de relacionamento, também pode usar essa opção de mapeamento, se for desejado.

Na Figura 8.10, o mapeamento foi aplicado a um subconjunto do esquema de banco de dados UNIVERSIDADE no contexto do padrão de banco de dados de objeto ODMG. O esquema de objeto mapeado usando a notação ODL é mostrado na Figura 11.10.

11.5 A linguagem de consulta de objeto (OQL — Object Query Language)

A linguagem de consulta de objeto OQL é a linguagem proposta para o modelo de objeto ODMG.

[40] O padrão ODL provê a definição explícita dos relacionamentos inversos. Alguns produtos de SGBDO podem não oferecer esse suporte; nesses casos, os programadores precisam manter cada relacionamento explicitamente, codificando os métodos que atualizam os objetos de forma apropriada.

[41] A decisão sobre usar set ou list não está disponível no esquema EER, e precisa ser determinada com base nos requisitos.

Ela foi projetada para trabalhar de perto com as linguagens de programação para as quais um binding ODMG é definido, como C++, Smalltalk e Java. Logo, uma consulta OQL embutida em uma dessas linguagens de programação pode retornar objetos que combinam com o sistema de tipos dessa linguagem. Além disso, as implementações de operações de classe em um esquema ODMG podem ter seu código escrito nessas linguagens de programação. A sintaxe OQL para consultas é semelhante à sintaxe da linguagem de consulta do padrão relacional, SQL, com recursos adicionais para os conceitos ODMG, como identidade de objeto, objetos complexos, operações, herança, polimorfismo e relacionamentos.

Na Seção 11.5.1, discutiremos a sintaxe das consultas OQL simples e o conceito de usar objetos nomeados ou extensões como pontos de entrada do banco de dados. Depois, na Seção 11.5.2, discutiremos a estrutura dos resultados de consulta e o uso de expressões de caminho para atravessar relacionamentos entre objetos. Outras características da OQL para tratamento de identidade de objeto, herança, polimorfismo e outros conceitos orientados a objeto são discutidos na Seção 11.5.3. Os exemplos para ilustrar consultas OQL são baseados no esquema de banco de dados UNIVERSIDADE dado na Figura 11.10.

11.5.1 Consultas em OQL simples, pontos de entrada do banco de dados e variáveis de iteração

A sintaxe OQL básica é uma estrutura select ... from ... where ..., assim como para a SQL. Por exemplo, a consulta para recuperar os nomes de todos os departamentos na faculdade de 'Engenharia' pode ser escrita da seguinte forma:

```
C0:  select  D.Dnome
     from    D in DEPARTAMENTOS
     where   D.Faculdade = 'Engenharia';
```

Em geral, um **ponto de entrada** para o banco de dados é necessário para cada consulta, que pode ser qualquer *objeto persistente nomeado*. Para muitas consultas, o ponto de entrada é o nome da extent de uma classe. Lembre-se de que o nome da extensão é considerado o nome de um objeto persistente cujo tipo é uma coleção (na maioria dos casos, um set) de objetos da classe. Ao examinar os nomes de extensão na Figura 11.10, o objeto nomeado DEPARTAMENTOS é do tipo set<DEPARTAMENTO>; PESSOAS é do tipo set<PESSOA>; DOCENTE é do tipo set<DOCENTE>; e assim por diante.

O uso de um nome de extensão — DEPARTAMENTOS em C0 — como um ponto de entrada refere-se a uma coleção persistente de objetos. Sempre que uma coleção é referenciada em uma consulta OQL, devemos definir uma **variável de iteração**[42] — *D* em C0 — que percorre cada objeto na coleção. Em muitos casos, como em C0, a consulta selecionará certos objetos da coleção, com base nas condições especificadas na cláusula where. Em C0, somente objetos persistentes *D* na coleção de DEPARTAMENTOS que satisfaçam a condição *D*.Faculdade = 'Engenharia' são selecionados para o resultado da consulta. Para cada objeto selecionado *D*, o valor de *D*.Dnome é recuperado no resultado da consulta. Assim, o *tipo do resultado* para C0 é bag<string> porque o tipo de cada valor Dnome é string (embora o resultado real seja um set, pois Dnome é um atributo-chave). Em geral, o resultado de uma consulta seria do tipo bag para select ... from ... e do tipo set para select distinct ... from ..., como na SQL (a inclusão da palavra-chave distinct elimina duplicatas).

Usando o exemplo em C0, existem três opções sintáticas para especificar variáveis de iteração:

```
D in DEPARTAMENTOS

DEPARTAMENTOS D
DEPARTAMENTOS AS D
```

Usaremos a primeira construção em nossos exemplos.[43]

Os objetos nomeados usados como pontos de entrada de banco de dados para consultas OQL não são limitados aos nomes das extensões. Qualquer objeto persistente nomeado, não importa se ele se refere a um objeto atômico (único) ou a um objeto de coleção, pode ser usado como um ponto de entrada do banco de dados.

11.5.2 Resultados de consulta e expressões de caminho

Em geral, o resultado de uma consulta pode ser de qualquer tipo expresso no modelo de objeto ODMG. Uma consulta não precisa seguir a estrutura select ... from ... where ...; no caso mais simples, qualquer nome persistente por si só é uma consulta, cujo resultado é uma referência a esse objeto persistente. Por exemplo, a consulta

[42] Isso é semelhante às variáveis de tupla que percorrem as tuplas nas consultas SQL.

[43] Observe que as duas últimas opções são semelhantes à sintaxe para especificar variáveis de tupla nas consultas SQL.

C1: DEPARTAMENTOS;

retorna uma referência à coleção de todos os objetos persistentes de DEPARTAMENTO, cujo tipo é set<DEPARTAMENTO>. De modo semelhante, suponha que tenhamos dado (pela operação de vínculo do banco de dados, ver Figura 11.8) um nome persistente CC_DEPARTAMENTO a um único objeto DEPARTAMENTO (o departamento de Ciência da Computação); então, a consulta

C1A: CC_DEPARTAMENTO;

retorna uma referência a esse objeto individual do tipo DEPARTAMENTO. Quando um ponto de entrada for especificado, o conceito de uma **expressão de caminho** poderá ser usado para especificar um *caminho* aos atributos e objetos relacionados. Uma expressão de caminho normalmente começa em um *nome de objeto persistente*, ou na variável de iteração que percorre os objetos individuais em uma coleção. Esse nome será seguido por zero ou mais nomes de relacionamento ou nomes de atributo conectados que usam a *notação de ponto*. Por exemplo, referindo-se ao banco de dados UNIVERSIDADE da Figura 11.10, a seguir estão exemplos de expressões de caminho, que também são consultas válidas em OQL:

C2: CC_DEPARTAMENTO.Diretor;

C2A: CC_DEPARTAMENTO.Diretor.Nivel;

C2B: CC_DEPARTAMENTO.Tem_docente;

A primeira expressão C2 retorna um objeto do tipo DOCENTE, pois esse é o tipo do atributo Diretor da classe DEPARTAMENTO. Esta será uma referência ao objeto DOCENTE que está relacionado ao objeto DEPARTAMENTO, cujo nome persistente é CC_DEPARTAMENTO por meio do atributo Diretor; ou seja, uma referência ao objeto DOCENTE que é diretor do departamento de Ciência da Computação. A segunda expressão C2A é semelhante, exceto que retorna o Nivel desse objeto DOCENTE (a cadeira de Ciência da Computação) em vez da referência ao objeto; logo, o tipo retornado por C2A é string, que é o tipo de dado para o atributo Nivel da classe DOCENTE.

As expressões de caminho C2 e C2A retornam valores isolados porque os atributos Diretor (de DEPARTAMENTO) e Nivel (de DOCENTE) são ambos valores isolados e aplicados a um único objeto. A terceira expressão, C2B, é diferente; ela retorna um objeto do tipo set<DOCENTE> mesmo quando aplicada a um único objeto, pois esse é o tipo do relacionamento Tem_docente da classe DEPARTAMENTO. A coleção retornada incluirá referências a todos os objetos DOCENTE que estão relacionados ao objeto DEPARTAMENTO, cujo nome persistente é CC_DEPARTAMENTO por meio do relacionamento Tem_docente; ou seja, as referências a todos os objetos DOCENTE que estão trabalhando no departamento de Ciência da Computação. Agora, para retornar as pontuações do corpo docente de Ciência da Computação, *não podemos* escrever

C3': CC_DEPARTAMENTO.Tem_docente.Nivel;

porque não está claro se o objeto retornado seria do tipo set<string> ou bag<string> (sendo o último mais provável, pois vários membros do corpo docente podem compartilhar o mesmo nível). Devido a esse tipo de problema de ambiguidade, a OQL não permite expressões como a C3. Em vez disso, é preciso usar uma variável de iteração sobre quaisquer coleções, como em C3A ou C3B, a seguir:

C3A: **select** *D*.Nivel
from *D* **in** CC_DEPARTAMENTO.Tem_docente;

C3B: **select distinct** *F*.Nivel
from *D* **in** CC_DEPARTAMENTO.Tem_docente;

Aqui, C3A retorna bag<string> (valores de níveis duplicados aparecem no resultado), enquanto C3B retorna set<string> (duplicatas são eliminadas por meio da palavra-chave distinct). Tanto C3A quanto C3B ilustram como uma variável de iteração pode ser definida na cláusula from para percorrer uma coleção restrita especificada na consulta. A variável *D* em C3A e C3B varia sobre os elementos da coleção CC_DEPARTAMENTO.Tem_docente, que é do tipo set<DOCENTE> e inclui apenas corpo docente, e o qual é membro do departamento de Ciência da Computação.

Em geral, uma consulta OQL pode retornar um resultado com uma estrutura complexa especificada na própria consulta e utilizar a palavra-chave struct. Considere os seguintes exemplos:

C4: CC_DEPARTAMENTO.Diretor.Orientados;

C4A: **select struct**(nome: **struct** (ultimo_nome: *A*.nome.Unome, primeiro_nome: *A*.nome.Pnome),
titulo:(**select struct** (tit: T.Titulo, ano: *T*.Ano,
faculdade: *T*.faculdade)
from *T* **in** *A*.Titulos))
from *S* **in** CC_DEPARTAMENTO.Diretor.Orientados;

Aqui, a C4 é direta, retornando um objeto do tipo set<ALUNO_POSGRADUACAO> como seu resultado. Essa é a coleção de alunos formados que são orientados pelo diretor do departamento de Ciência

da Computação. Agora, suponha que seja necessária uma consulta para recuperar o nome e sobrenome desses alunos de pós-graduação, mais a lista de títulos anteriores de cada um. Isso pode ser escrito como na C4A, em que a variável *A* percorre a coleção de alunos de pós-graduação orientados pelo diretor, e a variável *T* percorre os títulos de cada aluno *A*. O tipo do resultado de C4A é uma coleção de structs (de primeiro nível) onde cada struct tem dois componentes: nome e titulo.[44]

O componente nome é um outro struct composto de ultimo_nome e primeiro_nome, cada um sendo uma única cadeia. O componente de títulos é definido por uma consulta embutida e por si só é uma coleção de outros structs (segundo nível), cada um com três componentes de string: tit, ano e faculdade.

Observe que OQL é *ortogonal* com relação à especificação de expressões de caminho. Ou seja, atributos, relacionamentos e nomes de operação (métodos) podem ser usados um no lugar do outro dentro das expressões de caminho, desde que o sistema de tipo da OQL não seja comprometido. Por exemplo, pode-se escrever as seguintes consultas para recuperar a média de notas de todos os alunos sênior que estão se formando em Ciência da Computação, com o resultado ordenado por coeficiente, e dentro disso por sobrenome e primeiro nome.

```
C5A: select struct ( ultimo_nome: A.nome.Unome,
                     primeiro_nome: A.nome.Pnome,
                     coef: A.coeficiente )
       from        S in CC_DEPARTAMENTO.
                   Tem_aluno
       where       A.Tipo_aluno = 'senior'
       order by    media desc, ultimo_nome asc,
                   primeiro_nome asc;
Q5B: select struct ( ultimo_nome: A.nome.Unome,
                     primeiro_nome: A.nome.Pnome,
                     coef: A.coeficiente )
       from        A in ALUNOS
       where       A.Dep_princ.Dnome = 'Ciencia
                   da Computacao' and
                   A.Tipo_aluno = 'senior'
       order by    coef desc, ultimo_nome asc,
                   primeiro_nome asc;
```

A C5A usou o ponto de entrada nomeado CC_DEPARTAMENTO para localizar diretamente a referência ao departamento de Ciência da Computação e depois os alunos por meio do relacionamento Tem_aluno, ao passo que C5B procura a extensão ALUNOS para localizar todos os alunos que estão se formando nesse departamento. Observe como os nomes de atributo, nomes de relacionamento e nomes de operação (método) são todos usados de maneira intercambiável (ortogonal) nas expressões de caminho: coeficiente é uma operação; Dep_princ e Tem_aluno são relacionamentos; e Tipo_aluno, Nome, Dnome, Unome e Pnome são atributos. A implementação da operação coeficiente calcula a coeficiente de pontos e retorna seu valor como um tipo de ponto flutuante para cada ALUNO selecionado.

A cláusula order by é semelhante à construção SQL correspondente, e especifica em que ordem o resultado da consulta deve ser exibido. Logo, a coleção retornada por uma consulta com uma cláusula order by é do tipo *list*.

11.5.3 Outros recursos da OQL

Especificando visões como consultas nomeadas O mecanismo de visão (ou *view*) em OQL utiliza o conceito de **consulta nomeada**. A palavra-chave **define** é usada para especificar um identificador da consulta nomeada, que precisa ser um nome exclusivo entre todos os objetos nomeados, nomes de classe, nomes de método e nomes de função no esquema. Se o identificador tem o mesmo nome de uma consulta nomeada existente, então a nova definição substitui a anterior. Uma vez definida, a consulta é persistente até que seja redefinida ou excluída. Uma visão também pode ter parâmetros (argumentos) em sua definição.

Por exemplo, a visão V1 a seguir define uma consulta nomeada Tem_dep_secund para recuperar o conjunto de objetos para alunos que tenham departamento secundário:

```
V1:  define Tem_dep_secund(Nome_dep) as
     select A
     from   A in ALUNOS
     where  A.Dep_secund.Dnome = Nome_dep;
```

Como o esquema ODL na Figura 11.10 só forneceu um atributo Minors_in unidirecional para um ALUNO, podemos usar a visão acima para representar seu inverso sem ter de definir um relacionamento explicitamente. Esse tipo de visão pode ser utilizado para representar relacionamentos inversos que não deverão ser usados com frequência. O usuário agora pode utilizar a visão citada para escrever consultas como

```
Tem_dep_secund('Ciencia da Computacao');
```

[44] Como dissemos anteriormente, struct corresponde ao construtor de tupla discutido na Seção 11.1.3.

que retornariam uma bag de alunos que tenham departamento secundário de Ciência da Computação. Observe que, na Figura 11.10, definimos Tem_dep_princ como um relacionamento explícito, possivelmente porque se espera que seja usado com mais frequência.

Extraindo elementos isolados de coleções singulares. Uma consulta OQL, em geral, retorna uma coleção como seu resultado, tal como uma bag, set (se distinct for especificado) ou list (se a cláusula order by for usada). Se o usuário solicitar que uma consulta retorne apenas um único elemento, existe um operador element na OQL que garante o retorno de um único elemento E de uma coleção singular C, que contém apenas um elemento. Se C tiver mais de um elemento ou se for vazia, então o operador de elemento dança uma exceção. Por exemplo, C6 retorna a única referência de objeto ao departamento de Ciência da Computação:

```
C6: element ( select   D
    from       D in DEPARTAMENTOS
    where      D.Dnome = 'Ciencia da
               Computacao' );
```

Como um nome de departamento é único entre todos os departamentos, o resultado deve ser um departamento. O tipo do resultado é D:DEPARTAMENTO.

Operadores de coleção (funções de agregação, quantificadores). Como muitas expressões de consulta especificam coleções como seu resultado, diversos operadores foram definidos para serem aplicados a tais coleções. Estes incluem operadores de agregação, bem como condição de membro e quantificação (universal e existencial) sobre uma coleção.

Os operadores de agregação (min, max, count, sum, avg) operam sobre uma coleção.[45] O operador count retorna um tipo inteiro. Os demais operadores de agregação (min, max, sum, avg) retornam o mesmo tipo do tipo do operando da coleção. Vejamos dois exemplos. A consulta C7 retorna o número de alunos quem tenham departamento secundário em Ciência da Computação e C8 retorna o coeficiente médio de todos os formandos em Ciência da Computação.

```
C7: count( A in Tem_dep_princ('Ciencia da Computa
              cao'));
C8: avg ( select   A.Coeficiente
          from     A in ALUNOS
          where    A.Dep_princ.Dnome = 'Ciencia da
                   Computacao' and
                   A.Tipo_aluno = 'Senior');
```

Observe que as operações de agregação podem ser aplicadas a qualquer coleção do tipo apropriado e usadas em qualquer parte de uma consulta. Por exemplo, a consulta para recuperar todos os nomes de departamento que possuem mais de cem alunos como departamento principal pode ser escrita como em C9:

```
C9:  select  D.Dnome
     from    D in DEPARTAMENTOS
     where   count (D.Tem_aluno) > 100;
```

As expressões de *condição de membro* e *quantificação* retornam um tipo booleano — ou seja, verdadeiro ou falso. Considere que V seja uma variável, C, uma expressão de coleção; B, uma expressão do tipo Boolean (ou seja, uma condição booleana); E, um elemento do tipo dos elementos na coleção C. Então:

(*E* **in** *C*) retorna verdadeiro se o elemento E é um membro da coleção C.

(**for all** *V* **in** *C* : *B*) retorna verdadeiro se *todos* os elementos da coleção C satisfizerem B.

(**exists** *V* **in** *C* : *B*) retorna verdadeiro se houver pelo menos um elemento em C satisfazendo B.

Para ilustrar a condição de membro, suponha que queiramos recuperar os nomes de todos os alunos que completaram a disciplina chamada 'Sistemas de Bancos de Dados I'. Isso pode ser escrito como em C10, em que a consulta aninhada retorna a coleção de nomes de disciplina que cada ALUNO A completou, e a condição de membro retorna verdadeira se 'Sistemas de Bancos de Dados I' estiver na coleção para determinado ALUNO A:

```
C10: select A.nome.Unome,A.nome.Pnome
     from   A in ALUNOS
     where  'Sistemas deBancos de Dados I' in
            ( select C.Da_disciplina.
              from   C in A.Turma_concluida);
```

C10 também ilustra um modo mais simples de especificar a cláusula select das consultas que retornam uma coleção de structs. O tipo retornado por C10 é bag<struct(string,string)>.

Também é possível escrever consultas que retornam resultados verdadeiro/falso. Como exemplo, vamos supor que haja um objeto nomeado, chamado JEREMIAS, do tipo ALUNO. Então, a consulta C11 responde à seguinte pergunta: *Jeremias tem departamento secundário em Ciência da Computação?* De modo semelhante, C12 responde à pergunta: *Todos os alunos formados em Ciência da Computação são orientados pelo corpo docente de Ciência da Computação?* Tanto C11 quanto C12 retornam verdadeira ou falsa, que são interpretadas como respostas sim ou não para as perguntas acima:

[45] Estes correspondem às funções de agregação em SQL.

```
C11: JEREMIAS in Tem_dep_secund('Ciencia da Computacao');

C12: for all G in
        ( select A
          from A in ALUNOS_POSGRADUACAO
          where A.Dep_princ.Dnome = 'Ciencia da Computacao' )
        : G.Orientador in CC_DEPARTAMENTO.Tem_docente;
```

Observe que a consulta C12 também ilustra como a herança de atributo, relacionamento e operação se aplica às consultas. Embora *A* seja um objeto de iteração que percorre a extent ALUNOS_POSGRADUACAO, podemos escrever *A*.Dep_princ porque o relacionamento dep_princ é herdado por ALUNO_POSGRADUACAO de ALUNO por meio de extends (ver Figura 11.10). Por fim, para ilustrar o quantificador exists, a consulta C13 responde à seguinte pergunta: *Algum formado em Ciência da Computação tem um COEFICIENTE 4,0?* Aqui, novamente, a operação coeficiente é herdada por ALUNO_POSGRADUACAO de ALUNO por meio de extends.

```
C13: exists G in
        ( select A
          from A in ALUNOS_POSGRADUACAO
          where A.Dep_princ.Dnome = 'Ciencia da Computacao' )
        : G.Coeficiente = 4;
```

Expressões de coleção ordenada (indexada). Conforme discutimos na Seção 11.3.3, as coleções que são listas e arrays possuem operações adicionais, tais como recuperar o *i*-ésimo, primeiro e último elementos. Além disso, existem operações para extrair uma subcoleção e concatenar duas listas. Logo, as expressões de consulta que envolvem listas ou arrays podem invocar essas operações. Ilustraremos algumas dessas operações usando exemplos de consulta. A C14 recupera o sobrenome do membro do corpo docente que ganha o maior salário:

```
C14: first ( select   struct(docnome: D.nome. Unome, salario: D.Salario)
             from     D in DOCENTE
             order by salario desc );
```

C14 ilustra o uso do operador **first** sobre uma coleção de lista que contém os salários dos membros do corpo docente classificados em ordem decrescente. Portanto, o primeiro elemento nessa lista classificada contém o membro do corpo docente com o salário mais alto. Essa consulta considera que apenas um membro do corpo docente ganha o salário máximo. A próxima consulta, C15, recupera os três melhores coeficientes do departamento de Ciência da Computação.

```
C15: ( select   struct( ultimo_nome: A.nome. Unome, primeiro_nome: A.nome. Pnome, coeficiente: A.Coeficiente )
       from     A in CC_DEPARTAMENTO.Tem_alunos
       order by coef desc ) [0:2];
```

A consulta select-from-order-by retorna uma lista de alunos de Ciência da Computação ordenados pelo COEFICIENTE em ordem decrescente. O primeiro elemento de uma coleção ordenada tem a posição de índice 0, de modo que a expressão [0:2] retorna uma lista com o primeiro, o segundo e o terceiro elementos do resultado de select ... from ... order by ...

O operador de agrupamento A cláusula **group by** em OQL, embora semelhante à cláusula correspondente em SQL, oferece referência explícita à coleção de objetos dentro de cada *grupo* ou *partição*. Primeiro, vamos dar um exemplo e, depois, descrever a forma geral dessas consultas.

C16 recupere o número de alunos em cada departamento. Nessa consulta, os alunos são agrupados na mesma partição (grupo) se tiverem o mesmo nome do departamento; ou seja, o mesmo valor para *A*.Dep_princ.Dnome:

```
C16: ( select   struct( Nome_dep, numero_de_alunos: count (partition) )
       from     A in ALUNOS
       group by Nome_dep: A.Dep_princ.Dnome;
```

O resultado da especificação de agrupamento é do tipo set<struct(Nome_dep: string, partition: bag<struct(*S*:ALUNO>)>), que contém uma struct para cada grupo (partition), o qual tem dois componentes: o valor do atributo de agrupamento (Nome_dep) e a bag dos objetos ALUNO no grupo (partition). A cláusula select retorna o atributo de agrupamento (nome do departamento) e uma contagem do número de elementos em cada partição (ou seja, o número de alunos em cada departamento), em que partition é a palavra-chave usada para se referir a cada partição. O tipo de resultado da cláusula select é set<struct(Nome_dep: string, numero_de_aluno: integer)>. Em geral, a sintaxe para a cláusula group by é

group by F_1: E_1, F_2: E_2, ..., F_k: E_k

onde F_1: E_1, F_2: E_2, ..., F_k: E_k é uma lista de atributos de particionamento (agrupamento) e cada especificação de atributo de particionamento F_i: E_i define um nome de atributo (campo) F_i e uma expressão E_i. O resultado da aplicação do agrupamento (especificado na cláusula group by) é um conjunto de estruturas:

set<struct(F_1: T_1, F_2: T_2, ..., F_k: T_k, partition: bag<B>)>

onde T_i é o tipo retornado pela expressão E_i, partition é um nome de campo distinto (uma palavra-chave) e B é uma estrutura cujos campos são as variáveis de iteração (A em C16) declaradas na cláusula from que tem o tipo apropriado.

Assim como em SQL, uma cláusula having pode ser utilizada para filtrar os conjuntos particionados (ou seja, selecionar apenas alguns dos grupos com base nas condições do grupo). Em C17, a consulta anterior é modificada para ilustrar a cláusula having (e também mostra a sintaxe simplificada para a cláusula select). C17 recupera, para cada departamento com mais de cem alunos, o coeficiente de seus formandos. A cláusula having em C17 seleciona apenas as partições (grupos) que possuem mais de cem elementos (ou seja, departamentos com mais de cem alunos).

```
C17: select    nome_dep, media_coef: avg (
                select P.coeficiente from P in
                partition)
     from      A in ALUNOS
     group by  nome_dep: A. Dep_princ.Dnome
     having    count (partition) > 100;
```

Observe que a cláusula select de C17 retorna o coeficiente médio dos alunos na partição. A expressão

select P.Coeficiente **from** P in **partition**

retorna uma bag de coeficientes de aluno para essa partição. A cláusula from declara uma variável de iteração P sobre a coleção da partição, que é do tipo bag<struct(A: ALUNO)>. Então, a expressão de caminho P.media é usada para acessar o coeficiente de cada aluno na partição.

11.6 Visão geral de binding da linguagem C++ no padrão ODMG

O binding da linguagem C++ especifica como as construções ODL são mapeadas para construções C++. Isso é feito por meio de uma biblioteca de classes C++ que oferece classes e operações que implementam as construções da ODL. Uma linguagem de manipulação de objeto (OML) é necessária para especificar como os objetos de banco de dados são recuperados e manipulados em um programa C++, e isso está baseado na sintaxe e semântica da linguagem de programação C++. Além dos bindings da ODL/OML, um conjunto de construções, chamado *pragmas físicas*, é definido para permitir que o programador tenha algum controle sobre aspectos de armazenamento físico, como o agrupamento de objetos, a utilização de índices e o gerenciamento de memória.

A biblioteca de classes acrescentada à C++ para o padrão ODMG usa o prefixo d_ para declarações de classe que lidam com conceitos de banco de dados.[46] O objetivo é que o programador pense que apenas uma linguagem está sendo usada, e não duas linguagens separadas. Para o programador se referir aos objetos do banco de dados em um programa, uma classe D_Ref<T> é definida para cada classe de banco de dados T no esquema. Logo, variáveis de programa do tipo D_Ref<T> podem se referir tanto a objetos persistentes quanto a transientes de classe T.

Para utilizar os vários tipos embutidos no modelo de objeto do ODMG, como tipos de coleção, várias classes genéricas (template class) são especificadas na biblioteca. Por exemplo, uma classe abstrata D_Objeto<T> especifica as operações a serem herdadas por todos os objetos. De modo semelhante, uma classe abstrata D_Collection<T> especifica as operações das coleções. Essas classes não são instanciáveis, mas somente especificam as operações que podem ser herdadas por todos os objetos e por objetos de coleção, respectivamente. Uma classe genérica (template class) é especificada para cada tipo de coleção; estas incluem D_Set<T>, D_List<T>, D_Bag<T>, D_Varray<T> e D_Dictionary<T>, e correspondem aos tipos de coleção no modelo de objeto (ver Seção 11.3.1). Assim, o programador pode criar classes de tipos como D_Set<D_Ref<ALUNO>>, cujas instâncias seriam conjuntos de referências a objetos ALUNO, ou D_Set<string>, cujas instâncias seriam conjuntos de strings. Além disso, uma classe d_Iterator corresponde à classe Iterator do modelo de objeto.

A ODL C++ permite que um usuário especifique as classes de um esquema de banco de dados usando as construções da C++, bem como as construções oferecidas pela biblioteca de banco de dados do objeto. Para especificar os tipos de dados dos atributos,[47] tipos básicos como d_Short (inteiro curto), d_Ushort (inteiro curto sem sinal), d_Long (inteiro longo) e d_Float (número de ponto flutuante) são fornecidos. Além dos tipos de dados básicos, vários tipos literais estruturados são fornecidos para corresponderem aos tipos literais estruturados do modelo de objeto ODMG. Estes incluem d_String, d_Interval, d_Date, d_Time e d_Timestamp (ver Figura 11.5(b)).

[46] Presume-se que d_ indique classes de ***database*** (banco de dados).

[47] Ou seja, ***variáveis membro*** na terminologia da programação orientada a objeto.

Para especificar relacionamentos, a palavra-chave rel_ é usada no prefixo dos nomes de tipo; por exemplo, ao escrever

```
d_Rel_Ref<DEPARTAMENTO, Tem_aluno> Dep_
princ;
```

na classe ALUNO, e

```
d_Rel_Set<ALUNO, Dep_princ> Tem_aluno;
```

na classe DEPARTAMENTO, estamos declarando que Dep_princ e Tem_aluno são propriedades de relacionamento inversas uma à outra e, portanto, representam um relacionamento binário 1:N entre DEPARTAMENTO e ALUNO.

Para a OML, o binding sobrecarrega a operação *new* de modo que pode ser usado para criar objetos persistentes ou transientes. Para criar objetos persistentes, deve-se fornecer o nome do banco de dados e o nome persistente do objeto. Por exemplo, ao escrever

```
D_Ref<ALUNO>A = new(BD1, 'João_Silva')ALUNO;
```

o programador cria um objeto persistente nomeado, do tipo ALUNO, no banco de dados BD1, com nome persistente João_Silva. Outra operação, delete_object( ), pode ser utilizada para excluir objetos. A modificação de objeto é feita pelas operações (métodos) definidas em cada classe pelo programador.

O binding C++ também permite a criação de extensões usando a classe de biblioteca d_Extent. Por exemplo, ao escrever

```
D_Extent<PESSOA> TODAS_PESSOAS(BD1);
```

o programador criaria um objeto de coleção nomeado TODAS_PESSOAS — cujo tipo seria D_Set<PESSOA> — no banco de dados BD1, que manteria objetos persistentes do tipo PESSOA. Porém, as restrições de chave não são aceitas no binding C++, e quaisquer verificações de chave devem ser programadas nos métodos da classe.[48] Além disso, o vínculo com C++ não admite persistência por meio da acessibilidade. O objeto precisa ser declarado estaticamente para ser persistente no momento em que é criado.

Resumo

Neste capítulo, começamos na Seção 11.1 com uma visão geral dos conceitos utilizados nos bancos de dados de objeto, e discutimos como esses conceitos foram derivados dos princípios gerais da orientação a objeto. Os conceitos principais que discutimos foram: identidade e identificadores de objeto; encapsulamento de operações; herança; estrutura complexa de objetos por meio de aninhamento de construtores de tipo; e como os objetos se tornam persistentes. Depois, na Seção 11.2, mostramos como muitos desses conceitos foram incorporados ao modelo relacional e ao padrão SQL, levando à funcionalidade expandida do banco de dados relacional. Esses sistemas eram chamados de bancos de dados objeto-relacional.

Depois, discutimos o padrão ODMG 3.0 para bancos de dados de objeto. Começamos descrevendo as diversas construções do modelo de objeto na Seção 11.3. Os vários tipos embutidos, como Object, Collection, Iterator, set, list, e assim por diante, foram descritos por suas interfaces, que especificam as operações embutidas de cada tipo. Esses tipos embutidos são o alicerce sobre o qual a linguagem de definição de objeto (ODL) e a linguagem de consulta de objeto (OQL) são baseadas. Também descrevemos a diferença entre objetos, que possuem um identificador de objeto, e literais, que são valores sem OID. Os usuários podem declarar classes para sua aplicação que herdam operações das interfaces embutidas apropriadas. Dois tipos de propriedades podem ser especificados em uma classe definida pelo usuário — atributos e relacionamentos — além das operações que podem ser aplicadas a objetos da classe. A ODL permite que os usuários especifiquem interfaces e classes, e dois tipos diferentes de herança — herança de interface através de ':' e herança de classe através de extends. Uma classe pode ter uma extensão e chaves. Uma descrição da ODL foi vista em seguida, e um exemplo de esquema para o banco de dados UNIVERSIDADE foi usado para ilustrar as construções da ODL.

Após a descrição do modelo de objeto ODMG, abordamos uma técnica geral para projetar esquemas de banco de dados de objeto na Seção 11.4. Discutimos como os bancos de dados de objeto diferem dos bancos de dados relacionais em três áreas principais: referências para representar relacionamentos, inclusão de operações e herança. Por fim, mostramos como mapear um projeto de banco de dados conceitual no modelo EER para as construções dos bancos de dados de objeto.

Na Seção 11.5, apresentamos uma visão geral da linguagem de consulta de objeto (OQL). A OQL segue o conceito de ortogonalidade na construção de consultas, significando que uma operação pode ser aplicada ao resultado de outra operação, desde que o tipo do resultado seja do tipo de entrada correto para a operação. A sintaxe da OQL segue muitas das construções da SQL, mas inclui conceitos adicionais, como expressões de caminho, herança, métodos, relacionamentos e coleções. Mostramos alguns exemplos de como usar a OQL no banco de dados UNIVERSIDADE.

Em seguida, na Seção 11.6, fizemos um rápido apanhado do binding da linguagem C++, que estende as declarações de sua classe com os construtores de tipo ODL, mas permite a integração transparente da C++ com o SGBDO.

[48] Só fornecemos uma breve visão geral do binding C++. Para mais detalhes, consulte Cattell e Barry, eds. (2000), Cap. 5.

Em 1997, a Sun aprovou o API ODMG (**API — Application Program InterfaceODMG**). A O2 Technologies foi a primeira empresa a oferecer um SGBD compatível com ODMG. Muitos fornecedores de SGBDO, incluindo a Object Design (agora eXcelon), a Gemstone Systems, a POET Software e a Versant Object Technology, aprovaram o padrão ODMG.

Perguntas de revisão

11.1. Quais são as srcens da abordagem orientada a objeto?

11.2. Quais características principais um OID deve possuir?

11.3. Discuta sobre os diversos construtores de tipo. Como eles são usados para criar estruturas de objeto complexas?

11.4. Discuta o conceito de encapsulamento e diga como ele é utilizado para criar tipos de dados abstratos.

11.5. Explique o que significam os seguintes termos na terminologia de banco de dados orientado a objeto: *método*, *assinatura*, *mensagem*, *coleção*, *extensão*.

11.6. Qual é o relacionamento entre um tipo e seu subtipo em uma hierarquia de tipos? Qual é a restrição imposta sobre as extensões correspondentes aos tipos na hierarquia de tipos?

11.7. Qual é a diferença entre objetos persistentes e transientes? Como a persistência é tratada nos sistemas típicos de banco de dados OO?

11.8. Qual é a diferença entre a herança normal, a herança múltipla e a herança seletiva?

11.9. Discuta o conceito de polimorfismo/sobrecarga de operador.

11.10. Discuta como cada um dos seguintes recursos é realizado na SQL 2008: *identificador de objeto*, *herança de tipo*, *encapsulamento de operações e estruturas de objeto complexas*.

11.11. No modelo relacional tradicional, a criação de uma tabela definia tanto o tipo de tabela (esquema ou atributos) quanto a própria tabela (extensão ou conjunto de tuplas atuais). Como esses conceitos podem ser separados na SQL 2008?

11.12. Descreva as regras de herança na SQL 2008.

11.13. Quais são as diferenças e semelhanças entre os objetos e literais no modelo de objeto ODMG?

11.14. Liste as operações básicas das seguintes interfaces embutidas do modelo de objeto ODMG: Object, Collection, Iterator, Set, List, Bag, Array e Dictionary.

11.15. Descreva as literais estruturadas embutidas do modelo de objeto ODMG e as operações de cada uma.

11.16. Quais são as diferenças e semelhanças das propriedades de atributo e relacionamento de uma classe (atômica) definida pelo usuário?

11.17. Quais são as diferenças e semelhanças da herança de classe por extends e a herança de interface por ':' no modelo de objeto ODMG?

11.18. Discuta como a persistência é especificada no modelo de objeto ODMG no binding C++.

11.19. Por que os conceitos de extensões e chaves são importantes nas aplicações de banco de dados?

11.20. Descreva os seguintes conceitos de OQL: *pontos de entrada de banco de dados*, *extensões de caminho*, *variáveis de iteração*, *consultas nomeadas* (*visões*), *funções de agregação*, *agrupamento* e *quantificadores*.

11.21. O que significa a ortogonalidade de tipo da OQL?

11.22. Discuta os princípios gerais por trás do binding C++ do padrão ODMG.

11.23. Quais são as principais diferenças entre projetar um banco de dados relacional e um banco de dados de objeto?

11.24. Descreva as etapas do algoritmo para o projeto de banco de dados de objeto pelo mapeamento EER para OO.

Exercícios

11.25. Converta o exemplo de GEOMETRIA_OBJETO dado na Seção 11.1.5 da notação funcional para a notação dada na Figura 11.2, que distingue atributos e operações. Use a palavra-chave INHERIT para mostrar que uma classe herda de outra classe.

11.26. Compare a herança no modelo EER (ver Capítulo 8) com a herança no modelo OO, descrito na Seção 11.1.5.

11.27. Considere o esquema EER UNIVERSIDADE da Figura 8.10. Pense em quais operações são necessárias para os tipos de entidade/classes no esquema. Não considere operações construtoras e destruidoras.

11.28. Considere o esquema ER EMPRESA da Figura 7.2. Pense em quais operações são necessárias para os tipos de entidade/classes no esquema. Não considere operações construtoras e destruidoras.

11.29. Projete um esquema OO para uma aplicação de banco de dados em que você esteja interessado. Construa um esquema EER para a aplicação e depois crie as classes correspondentes em ODL. Especifique uma série de métodos para cada classe, e depois especifique consultas em OQL para sua aplicação de banco de dados.

11.30. Considere o banco de dados AEROPORTO descrito no Exercício 8.21. Especifique uma série de operações/métodos que você acredita que deveriam ser adequados à aplicação. Especifique as classes e métodos ODL para o banco de dados.

11.31. Mapeie o esquema ER EMPRESA da Figura 7.2 para classes ODL. Inclua métodos apropriados para cada classe.

11.32. Especifique em OQL as consultas dos exercícios dos capítulos 7 e 8 que se aplicam ao banco de dados EMPRESA.

11.33. Usando mecanismos de busca e outras fontes, determine até que ponto os diversos produtos comerciais de SGBDO são compatíveis com o padrão ODMG 3.0.

Bibliografia selecionada

Os conceitos de banco de dados orientado a objeto são uma combinação de conceitos das linguagens de programação OO e dos sistemas de banco de dados e modelos de dados conceituais. Diversos livros-texto descrevem as linguagens de programação OO — por exemplo, Stroustrup (1997) para C++, e Goldberg e Robson (1989) para Smalltalk. Livros de Cattell (1994) e Lausen e Vossen (1997) descrevem os conceitos de banco de dados OO. Outros livros sobre modelos OO incluem uma descrição detalhada do SGBDOO desenvolvido na Microelectronic Computer Corporation, chamado Orion, e relacionados aos tópicos de OO por Kim e Lochovsky (1989). Bancilhon et al. (1992) descrevem a história da criação do SGBDOO O2 com uma discussão detalhada das decisões de projeto e implementação de linguagem. Dogac et al. (1994) oferecem uma discussão profunda sobre tópicos de banco de dados OO por especialistas em um workshop da OTAN.

Há uma vasta bibliografia sobre bancos de dados OO, de modo que só podemos oferecer uma amostra representativa aqui. A edição de outubro de 1991 da CACM e a edição de dezembro de 1990 da *IEEE Computer* descrevem os conceitos e sistemas de banco de dados OO. Dittrich (1986) e Zaniolo et al. (1986) analisam os conceitos básicos dos modelos de dados OO. Um artigo inicial sobre a implementação de sistema de banco de dados OO é o de Baroody e DeWitt (1981). Su et al. (1988) apresentam um modelo de dados OO que foi usado em aplicações de CAD/CAM. Gupta e Horowitz (1992) discutem aplicações de OO para CAD, Gerenciamento de Redes e outras áreas. Mitschang (1989) estende a álgebra relacional para abranger objetos complexos. Linguagens de consulta e interfaces gráficas com o usuário para OO são descritas em Gyssens et al. (1990), Kim (1989), Alashqur et al. (1989), Bertino et al. (1992), Agrawal et al. (1990) e Cruz (1992).

O *Object-Oriented Manifesto* de Atkinson et al. (1990) é um artigo interessante que relata sobre a posição de um painel de especialistas com relação aos recursos obrigatórios e opcionais do gerenciamento de banco de dados OO. O polimorfismo nos bancos de dados e nas linguagens de programação OO são discutidos em Osborn (1989), Atkinson e Buneman (1987) e Danforth e Tomlinson (1988). A identidade de objeto é discutida em Abiteboul e Kanellakis (1989). Linguagens de programação OO para bancos de dados são discutidas em Kent (1991). Restrições de objeto são discutidas em Delcambre et al. (1991) e Elmasri, James e Kouramajian (1993). Autorização e segurança em bancos de dados OO são examinados em Rabitti et al. (1991) e Bertino (1992).

Cattell e Barry (2000) descrevem o padrão ODMG 3.0, que é descrito neste capítulo. Cattell et al. (1993) e Cattell et al. (1997) descrevem as versões anteriores do padrão. Bancilhon e Ferrari (1995) oferecem uma apresentação tutorial dos aspectos importantes do padrão ODMG. Vários livros descrevem a arquitetura Corba — por exemplo, Baker (1996).

O sistema O2 é descrito em Deux et al. (1991), e Bancilhon et al. (1992) incluem uma lista de referências a outras publicações que descrevem vários aspectos do O2. O modelo O2 foi formalizado em Velez et al. (1989). O sistema ObjectStore é descrito em Lamb et al. (1991). Fishman et al. (1987) e Wilkinson et al. (1990) discutem o Iris, um SGBD orientado a objeto desenvolvido nos laboratórios da Hewlett-Packard.

Maier et al. (1986) e Butterworth et al. (1991) descrevem o projeto do GEM-STONE. O sistema ODE, desenvolvido na AT&T Bell Labs, é descrito em Agrawal e Gehani (1989). O sistema ORION desenvolvido no MCC é descrito em Kim et al. (1990). Morsi et al. (1992) descrevem um ambiente de testes OO.

Cattell (1991) analisa conceitos de bancos de dados relacional e de objeto, e discute vários protótipos de sistemas de banco de dados baseados em objeto e relacional estendido. Alagic (1997) indica discrepâncias entre o modelo de dados ODMG e seus bindings de linguagem, propondo algumas soluções. Bertino e Guerrini (1998) propõem uma extensão do modelo ODMG para dar suporte a objetos compostos. Alagic (1999) apresenta vários modelos de dados pertencentes à família ODMG.

capítulo 12

XML: Extensible Markup Language

Muitas aplicações de comércio eletrônico (e-commerce) e outras da Internet oferecem interfaces Web para acessar informações armazenadas em um ou mais bancos de dados. Esses bancos de dados normalmente são conhecidos como **fontes de dados**. É comum usar arquiteturas cliente/servidor de duas e três camadas para aplicações da Internet (ver Seção 2.5). Em alguns casos, outras variações do modelo cliente/servidor são usadas. O e-commerce e outras aplicações de banco de dados da Internet são projetadas para interagir com o usuário por meio de interfaces Web que exibem páginas Web. O método comum de especificar o conteúdo e a formatação das páginas Web é com o uso de **documentos de hipertexto** Existem várias linguagens para escrever esses documentos, sendo que a mais comum é a HTML (*Hypertext Markup Language*). Embora seja bastante usada para formatação e estrutura de *documentos* da Web, ela não é adequada para especificar *dados estruturados* que são extraídos de bancos de dados. Uma nova linguagem — a saber, XML (*Extensible Markup Language*) — surgiu como padrão para estruturação e troca de dados pela Web. A XML pode ser usada para oferecer informações sobre a estrutura e o significado dos dados nas páginas Web, em vez de apenas especificar como elas são formatadas para exibição na tela. Os aspectos de formatação são especificados separadamente — por exemplo, usando uma linguagem de formatação como a XSL (*Extensible Stylesheet Language*) ou uma linguagem de transformação como a XSLT *Extensible Stylesheet Language for Transformations*, ou simplesmente XSL *Transformations*). Recentemente, a XML também foi proposta como um possível modelo para armazenamento e recuperação de dados, embora apenas alguns sistemas de banco de dados experimentais, baseados em XML nessa linguagem, tenham sido desenvolvidos até o momento.

A HTML básica é útil para gerar páginas Web *estáticas*, com texto fixo e outros objetos, mas a maioria das aplicações de e-commerce exige páginas Web que oferecem recursos interativos com o usuário. Por exemplo, considere o caso de um cliente de companhia aérea que deseja verificar a informação de hora de chegada e portão de determinado voo. O usuário pode inserir informações como uma data e número de voo em certos campos de formulário da página Web. O programa da Web precisa, primeiro, submeter uma consulta ao banco de dados da companhia aérea para recuperar essa informação, e depois exibi-la. Essas páginas Web, onde parte da informação é extraída de bancos de dados e outras fontes de dados, são chamadas páginas Web *dinâmicas*, pois os dados extraídos e exibidos a cada vez serão para diferentes voos e datas.

Neste capítulo, vamos nos concentrar na descrição do modelo de dados XML e suas linguagens associadas, e como os dados extraídos dos bancos de dados relacionais podem ser formatados como documentos XML para serem trocados pela Web. A Seção 12.1 discute a diferença entre dados estruturados, semiestruturados e não estruturados. A Seção 12.2 apresenta o modelo de dados da XML, que é baseado em estruturas de árvore (hierárquicas), em comparação com as estruturas planas do modelo de dados relacional. Na Seção 12.3, verificamos a estrutura dos documentos XML e as linguagens para especificar a estrutura desses documentos, como DTD (*Document Type Definition*) e XML Schema. A Seção 12.4 mostra o relacionamento entre a XML e os bancos de dados relacionais. A Seção 12.5 descreve algumas das linguagens associadas à XML, como XPath e XQuery. A Seção 12.6 discute como os dados extraídos dos bancos de dados relacionais

podem ser formatados como documentos XML. Por fim, apresentamos um resumo do capítulo.

12.1 Dados estruturados, semiestruturados e não estruturados

A informação armazenada nos bancos de dados é conhecida como **dados estruturados** porque é representada em um formato estrito. Por exemplo, cada registro em uma tabela de banco de dados relacional — como cada uma das tabelas no banco de dados EMPRESA da Figura 3.6 — segue o mesmo formato dos outros registros nessa tabela. Para dados estruturados, é comum projetar cuidadosamente o esquema de banco de dados usando técnicas como as descritas nos capítulos 7 e 8 a fim de definir a estrutura do banco de dados. O SGBD então verifica para ter certeza de que todos os dados seguem as estruturas e restrições especificadas no esquema.

No entanto, nem todos os dados são coletados e inseridos em bancos de dados estruturados projetados cuidadosamente. Em algumas aplicações, os dados são coletados de uma maneira casual antes que se saiba como serão armazenados e gerenciados. Esses dados podem ter uma estrutura, mas nem toda a informação coletada terá a estrutura idêntica. Alguns atributos podem ser compartilhados entre as diversas entidades, mas outros podem existir apenas em algumas entidades. Além disso, atributos adicionais podem ser introduzidos em alguns dos itens de dados mais novos a qualquer momento, e não existe esquema predefinido. Esse tipo de dados é conhecido como **dados semiestruturados**. Diversos modelos de dados foram introduzidos para representar dados semiestruturados, geralmente com base no uso de estruturas de dados de árvore ou grafo, em vez das estruturas do modelo relacional plano.

A principal diferença entre dados estruturados e semiestruturados diz respeito a como as construções do esquema (como os nomes de atributos, relacionamentos e tipos de entidade) são tratadas. Nos dados semiestruturados, a informação do esquema é *misturada* com os valores dos dados, já que cada objeto de dado pode ter atributos diferentes que não são conhecidos antecipadamente. Logo, esse tipo de dados às vezes é chamado de **dados autodescritivos**. Considere o exemplo a seguir. Queremos coletar uma lista de referências bibliográficas relacionadas a certo projeto de pesquisa. Algumas delas podem ser livros ou relatórios técnicos, outras podem ser artigos de pesquisa em jornais ou eventos de conferência, e ainda outras podem se referir a edições completas de jornal ou atas de conferência. Nitidamente, cada uma pode ter diferentes atributos e diversos tipos de informação. Até para o mesmo tipo de referência — digamos, artigos de conferência —, podemos ter diferentes informações. Por exemplo, uma citação de artigo pode ser bastante completa, com todas as informações sobre nomes de autor, título, eventos, números de página, e assim por diante, enquanto outra citação pode não ter toda a informação disponível. Novos tipos de fontes bibliográficas podem aparecer no futuro — por exemplo, referências a páginas web ou tutoriais da conferência —, e estes podem ter novos atributos que os descrevem.

Os dados semiestruturados podem ser exibidos como um grafo direcionado, como mostra a Figura

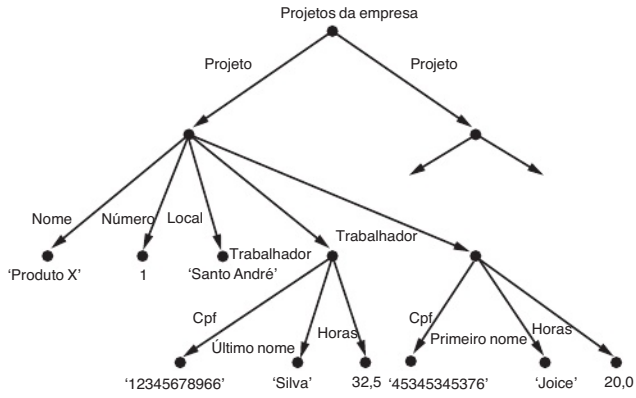


Figura 12.1
Representando dados semiestruturados como um grafo.

12.1. A informação exibida corresponde a alguns dos dados estruturados mostrados na Figura 3.6. Como podemos ver, esse modelo é um pouco semelhante ao modelo de objeto (ver Seção 11.1.3) em sua capacidade de representar objetos complexos e estruturas aninhadas. Na Figura 12.1, os **rótulos** ou **marcas** (labels ou tags) nas arestas direcionadas representam os nomes de esquema: os *nomes de atributos*, *tipos de objeto* (ou *tipos de entidade* ou *classes*) e *relacionamentos*. Os nós internos representam objetos individuais ou atributos compostos. Os nós da folha representam valores de dados reais de atributos simples (atômicos).

Existem duas diferenças principais entre o modelo semiestruturado e o modelo de objeto que discutimos no Capítulo 11:

1. A informação do esquema — nomes de atributos, relacionamentos e classes (tipos de objeto) no modelo semiestruturado é misturado com os objetos e seus valores de dados na mesma estrutura de dados.
2. No modelo semiestruturado, não existe requisito para um esquema predefinido ao qual os objetos de dados precisam se adequar, embora seja possível definir um esquema, se necessário.

Além de dados estruturados e semiestruturados, existe uma terceira categoria conhecida como dados não estruturados porque existe indicação muito limitada sobre o tipo de dados. Um exemplo típico é um documento de texto que contém informações incorporadas a ele. As páginas Web em HTML que contêm alguns dados são consideradas dados não estruturados. Considere parte de um arquivo HTML, mostrado na Figura 12.2. O texto que aparece entre os sinais < ... > é uma **tag HTML**. Uma tag com uma barra, </...>, indica uma **tag de fim**, que representa o encerramento do efeito de uma **tag de início** correspondente. As tags **marcam** o documento[1] a fim de instruir um processador HTML sobre como exibir o texto entre uma tag de início e uma tag de fim correspondente. Portanto, as tags especificam a formatação do documento, e não o significado dos diversos elementos de dados no documento. As tags HTML especificam informações, como as tags de fonte e espaçamento (negrito, itálico, assim por diante), cores, níveis de cabeçalho nos documentos etc. Algumas tags oferecem estruturação de texto nos documentos, como na especificação de lista numerada ou não numerada, ou de tabela. Até mesmo essas tags de estruturação especificam que os dados textuais embutidos devem ser exibidos de certa maneira, em vez de indicar o tipo de dado representado na tabela.

A HTML usa um grande número de tags predefinidas, que servem para especificar uma série de comandos para formatação de documentos Web para exibição. As tags de início e fim especificam o intervalo de texto a ser formatado por cada comando. Estes são alguns exemplos das tags mostradas na Figura 12.2:

- As tags <HTML> ... </HTML> especificam os limites do documento.
- A informação de **cabeçalho do documento** — dentro das tags <HEAD> ... </HEAD> — especifica diversos comandos que serão usados em outra parte do documento. Por exemplo, ela pode especificar diversas **funções de script** em uma linguagem como JavaScript ou PERL, ou certos **estilos de formatação** (fontes, estilos de parágrafo, estilos de cabeçalho, e assim por diante) que podem ser utilizados no documento. Ela também pode especificar um título para indicar para que serve o arquivo HTML, e outras informações semelhantes que não serão exibidas como parte do documento.
- O **corpo** do documento — especificado dentro das tags <BODY> ... </BODY> — inclui o texto do documento e as tags de marcação que especificam como o texto deve ser formatado e exibido. Também pode incluir referências a outros objetos, como imagens, vídeos, mensagens de voz e outros documentos.
- As tags <H1> ... </H1> especificam que o texto deve ser exibido como um cabeçalho de nível 1. Existem muitos níveis de cabeçalho (<H2>, <H3>, e assim por diante), cada um exibindo texto em um formato de cabeçalho proeminente.
- As tags <TABLE> ... </TABLE> especificam que o texto seguinte deve ser exibido como uma tabela. Cada *linha de tabela* é delimitada por tags <TR> ... </TR>, e os elementos de dados individuais da tabela, dentro de uma linha, são exibidos dentro de tags <TD> ... </TD>.[2]
- Algumas tags podem ter **atributos**, que aparecem dentro da tag de início e descrevem propriedades adicionais da tag.[3]

[1] É por isso que ela é conhecida como linguagem de *marcação* de hipertexto
[2] <TR> significa table row (linha da tabela) e <TD> significa table data (dado da tabela).
[3] É assim que o termo atributo é usado em linguagens de marcação de documento, que diferem do modo como é usado nos modelos de banco de dados.

```
<HTML>
    <HEAD>
    ...
    </HEAD>
    <BODY>
        <H1>Listta de projetos da empresa e os funcionários em cada projeto</H1>
      <H2>O projeto ProdutoX:</H2>
       <TABLE width="100%" border=0 cellpadding=0 cellspacing=0>
              <TR>
               <TD width="50%"><FONT size="2" face="Arial">João Silva:</FONT></TD>
              <TD>32,5 horas por semana</TD>
              </TR>
              <TR>
               <TD width="50%"><FONT size="2" face="Arial">Joice Leite:</FONT></TD>
              <TD>20,0 horas por semana</TD>
              </TR>
         </TABLE>
      <H2>O projeto ProdutoY:</H2>
       <TABLE width="100%" border=0 cellpadding=0 cellspacing=0>
              <TR>
               <TD width="50%"><FONT size="2" face="Arial">João Silva:</FONT></TD>
              <TD>7,5 horas por semana</TD>
              </TR>
              <TR>
               <TD width="50%"><FONT size="2" face="Arial">Joice Leite:</FONT></TD>
              <TD>20,0 horas por semana</TD>
              </TR>
              <TR>
                <TD width= "50%"><FONTsize="2" face="Arial">FernandoWong:</FONT></TD>
              <TD>10,0 horas por semana</TD>
              </TR>
         </TABLE>
    ...
    </BODY>
</HTML>
```

Figura 12.2
Parte de um documento HTML representando dados não estruturados.

Na Figura 12.2, a tag de início <TABLE> tem quatro atributos que descrevem diversas características da tabela. As tags de início <TD> e <FONT> seguintes têm um e dois atributos, respectivamente.

A HTML tem um número muito grande de tags predefinidas, e livros inteiros são dedicados a descrever como usá-las. Se projetados corretamente, os documentos HTML podem ser formatados de modo que os humanos consigam entender facilmente seu conteúdo, e sejam capazes de navegar pelos documentos Web resultantes. Porém, os documentos de texto HTML fonte são muito difíceis de interpretar automaticamente por *programas de computador*, pois eles não incluem informações de esquema sobre o tipo de dado nos documentos. À medida que o comércio eletrônico e outras aplicações da Internet se tornam cada vez mais automatizadas, está se tornando essencial a capacidade de trocar documentos Web entre diversos sites de computador e interpretar seu conteúdo de maneira automática. Essa necessi-

dade foi um dos motivos que levaram ao desenvolvimento da XML. Além disso, uma versão extensível da HTML, chamada XHTML, foi desenvolvida para permitir que os usuários estendessem as tags da HTML para diferentes aplicações, permitindo que um arquivo XHTML seja interpretado pelos programas de processamento XML padrão. Nossa discussão focalizará apenas a XML.

O exemplo da Figura 12.2 ilustra uma página HTML **estática**, pois toda a informação a ser exibida está escrita explicitamente como um texto fixo no arquivo HTML. Em muitos casos, as páginas Web são exibidas para mostrar dados extraídos de um banco de dados. Por exemplo, os nomes de projeto e os funcionários que trabalham em cada um deles podem ser extraídos do banco de dados da Figura 3.6 por meio da consulta SQL apropriada. Podemos querer usar as mesmas tags de formatação HTML para exibir cada projeto e os funcionários que trabalham nele, mas podemos querer mudar os projetos em particular (e funcionários) que estão sendo exibidos. Por exemplo, podemos querer ver uma página Web exibindo a informação para *ProjetoX* e, mais tarde, uma página exibindo a informação para o *ProjetoY*. Embora as duas páginas sejam exibidas usando as mesmas tags de formatação HTML, os itens de dados reais exibidos serão diferentes. Essas páginas Web são chamadas de **dinâmicas** porque as partes dos dados da página podem ser diferentes toda vez que ela é exibida, embora a aparência da tela seja a mesma.

12.2 Modelo de dados hierárquico (em árvore) da XML

Agora, vamos apresentar o modelo de dados usado em XML. O objeto básico em XML é o documento XML. Dois conceitos de estruturação principais são usados para construir um documento XML: **elementos** e **atributos**. É importante observar que o termo *atributo* em XML não é usado da mesma maneira que na terminologia de banco de dados, mas sim como é usado em linguagens de descrição de documento como HTML e SGML.[4] Os atributos em XML oferecem informações adicionais que descrevem elementos, conforme veremos. Existem conceitos adicionais na XML, como entidades, identificadores e referências, mas primeiro vamos nos concentrar na descrição de elementos e atributos para mostrar a essência do modelo XML.

A Figura 12.3 mostra um exemplo de elemento XML chamado <Projeto>. Assim como na HTML, os elementos são identificados em um documento por sua tag de início e tag de fim. Os nomes de tag são delimitados por sinais < ... >, e as tags de fim são identificadas ainda por uma barra, </ ... >[5].

```
<?xml version= "1.0" standalone="yes"?>
  <Projetos>
    <Projeto>
      <Nome>ProdutoX</Nome>
      <Numero>1</Numero>
      <Localizacao>Santo_Andre</Localizacao>
      <Dept_nr>5</Dept_nr>
      <Trabalhador>
          <Cpf>12345678966</Cpf>
          <Ultimo_nome>Silva</Ultimo_nome>
          <Horas>32,5</Horas>
      </Trabalhador>
      <Trabalhador>
          <Cpf>45345345376</Cpf>
          <Primeiro_nome>Joice</Primeiro_nome>
          <Horas>20,0</Horas>
      </Trabalhador>
    </Projeto>
    <Projeto>
      <Nome>ProdutoY</Nome>
      <Numero>2</Numero>
      <Localizacao>Itu</Localizacao>
      <Dept_nr>5</Dept_nr>
      <Trabalhador>
          <Cpf>12345678966</Cpf>
          <Horas>7,5</Horas>
      </Trabalhador>
      <Trabalhador>
          <Cpf>45345345376</Cpf>
          <Horas>20,0</Horas>
      </Trabalhador>
      <Trabalhador>
          <Cpf>33344555587</Cpf>
          <Horas>10,0</Horas>
      </Trabalhador>
    </Projeto>
  ...
  </Projetos>
```

Figura 12.3

Um elemento XML complexo, chamado <Projeto>.

[4] A SGML (*standard generalized markup language*) é uma linguagem mais geral para descrever documentos e oferece capacidades para especificar novas tags. Porém, ela é mais complexa do que a HTML e a XML.

[5] Os caracteres < e > são caracteres reservados, assim como o &, o apóstrofo (') e a aspa simples ("). Para incluí-los no texto de um documento, eles precisam ser codificados com escapes, como <, >, &, ', e ", respectivamente.

Elementos complexos são construídos com base em outros elementos hierarquicamente, enquanto **elementos simples** contêm valores de dados. Uma diferença importante entre XML e HTML é que os nomes de tag XML são definidos para descrever o significado dos elementos de dados no documento, em vez de descrever como o texto deve ser exibido. Isso possibilita processar os elementos de dados no documento XML de maneira automática pelos programas de computador. Além disso, os nomes de tag (elemento) XML podem ser definidos em outro documento para dar um significado semântico aos nomes de tag que podem ser trocados entre vários usuários. Em HTML, todos os nomes de tag são predefinidos e fixos; e por isso eles não são extensíveis.

É fácil ver a correspondência entre a representação textual da XML mostrada na Figura 12.3 e a estrutura de árvore mostrada na Figura 12.1. Na representação de árvore, os nós internos representam elementos complexos, enquanto os nós de folha representam elementos simples. É por isso que o modelo XML é conhecido como um **modelo de árvore** ou um **modelo hierárquico**. Na Figura 12.3, os elementos simples são aqueles com nomes de tag <Nome>, <Numero>, <Localizacao>, <Dept_nr>, <Cpf>, <Ultimo_nome>, <Primeiro_nome> e <Horas>. Os elementos complexos são aqueles com nomes de tag <Projetos>, <Projeto> <Trabalhador>.

Em geral, não existe limite sobre os níveis de aninhamento de elementos.

É possível caracterizar três tipos principais de documentos XML:

- **Documentos XML centrados em dados.** Esses documentos possuem muitos itens de dados pequenos que seguem uma estrutura específica e, portanto, podem ser extraídos de um banco de dados estruturado. Eles são formatados como documentos XML a fim de trocá-los pela Web ou exibi-los nela. Estes normalmente seguem um *esquema predefinido*, que define os nomes de tag.
- **Documentos XML centrados no documento.** Estes são documentos com grande quantidade de texto, como artigos de notícias ou livros. Há poucos ou nenhum elemento de dado estruturado nesses documentos.
- **Documentos XML híbridos.** Esses documentos podem ter partes que contêm dados estruturados e outras partes que são predominantemente textuais ou não estruturadas. E podem ou não ter um esquema predefinido.

Documentos XML que não seguem um esquema predefinido de nomes de elemento e estrutura de árvore correspondente são conhecidos como **documentos XML sem esquema**. É importante observar que os documentos XML centrados nos dados podem ser considerados dados semiestruturados ou estruturados, conforme definido na Seção 12.1. Seum documento XML obedece a um esquema XMLpredefinido ou DTD (ver Seção 12.3), então o documento pode ser considerado *dados estruturados*. Além disso, a XML permite documentos que não obedecem a qualquer esquema. Estes seriam considerados *dados semiestruturados* e são *documentos XML sem esquema* Quando o valor do atributo standalone em um documento XML éyes, como na primeira linha da Figura 12.3, o documento é independente e sem esquema.

Os atributos XML geralmente são usados de uma maneira semelhante à da HTML (ver Figura 12.2), a saber, para descrever propriedades e características dos elementos (tags) nas quais eles aparecem. Também é possível usar atributos XML para manter os valores de elementos de dados simples; porém, isso não costuma ser recomendado. Uma exceção a essa regra se dá em casos que precisam **referenciar** outro elemento em outra parte do documento XML. Para fazer isso, é comum usar valores de atributo em um elemento como referências. Isso é semelhante ao conceito de chaves estrangeiras nos bancos de dados relacionais, e é um modo de contornar o modelo hierárquico estrito que o modelo de árvore XML implica. Discutiremos mais sobre os atributos XML na Seção 12.3 quando falarmos sobre esquema XML e DTD.

12.3 Documentos XML, DTD e esquema XML

12.3.1 Documentos XML bem formados e válidos e XML DTD

Na Figura 12.3, vimos como um documento XML simples poderia se parecer. Um documento XML é **bem formado** se seguir algumas condições. Em particular, ele precisa começar com uma **declaração XML** para indicar a versão da linguagem que está sendo usada, bem como quaisquer outros atributos relevantes, com mostra a primeira linha da Figura 12.3. Ele também precisa seguir as diretrizes sintáticas do modelo de dados de árvore. Isso significa que deve haver um *único elemento raiz*, e cada elemento precisa incluir um par correspondente de tags de início e de fim *dentro* das tags de início e de fim *do elemento pai*. Isso garante que os elementos aninhados especificam uma estrutura de árvore bem formada.

Um documento XML bem formado é sintaticamente correto. Isso permite que ele seja processado por processadores genéricos, que percorrem o docu-

mento e criam uma representação de árvore interna. Um modelo-padrão com um conjunto associado de funções de API (*Application Programming Interface*), chamado **DOM** (*Document Object Model*) permite que os programas manipulem a representação de árvore resultante correspondente a um documento XML bem formado. No entanto, o documento inteiro precisa ser analisado de antemão quando se usa DOM, para converter o documento para a representação na estrutura de dados interna DOM padrão. Outra API, chamada **SAX** (*Simple API for XML*) permite o processamento de documentos XML no qual o programa de processamento pode definir chamadas de eventos sempre que uma tag de início ou fim for encontrada. Isso facilita o processamento de grandes documentos e permite o dos chamados **documentos XML de streaming** em que o programa de processamento pode processar as tags à medida que forem encontradas. Isso também é conhecido como **processamento baseado em evento**

Um documento XML bem formado pode não ter esquema; ou seja, ele pode ter quaisquer nomes de tag para os elementos do documento. Nesse caso, não existe um conjunto predefinido de elementos (nomes de tag) que um programa processando o documento saiba esperar. Isso dá ao criador do documento a liberdade de especificar novos elementos, mas limita as possibilidades para interpretar automaticamente o significado ou a semântica dos elementos do documento.

Um critério mais forte é que um documento XML seja **válido**. Nesse caso, o documento deverá ser bem formado e seguir um esquema específico. Ou seja, os nomes de elemento usados nos pares de tag de início e de fim devem seguir a estrutura especificada em um arquivo XML **DTD** (***Document Type Definition***) separado, ou arquivo de esquema XML. Primeiro, vamos discutir aqui a XML DTD, e depois daremos uma visão geral do esquema XML na Seção 12.3.2. A Figura 12.4 mostra um arquivo XML DTD simples, que especifica os elementos (nomes de tag) e suas estruturas aninhadas. Quaisquer documentos válidos em conformidade com essa DTD devem seguir a estrutura especificada.

Em DTD, com a sintaxe regular da Figura 12.4, primeiro, um nome é dado à **tag raiz** do documento, que é chamada de Projetos na primeira linha da Figura 12.4. Depois, os elementos e sua estrutura aninhada são especificados.

Ao especificar elementos, a notação a seguir é usada:

- Um * após o nome do elemento significa que ele pode ser repetido zero ou mais vezes no documento. Esse tipo de elemento é conhecido como um *elemento multivalorado (repetitivo) opcional*.
- Um + após o nome do elemento significa que ele pode ser repetido uma ou mais vezes no documento. Esse tipo de elemento é conhecido como um *elemento multivalorado (repetitivo) obrigatório*.
- Um ? após o nome do elemento significa que ele pode ser repetido zero ou uma vez. Esse

```
<!DOCTYPE Projects [
    <!ELEMENT Projetos (Projeto+)>
      <!ELEMENT Projeto (Nome, Numero, Localizacao, Dept_nr?, Trabalhadores)
      <!ATTLIST Projeto
            ProjId ID #REQUIRED>
            >
        <!ELEMENT Nome (#PCDATA)>
    <!ELEMENT Numero (#PCDATA)
    <!ELEMENT Localizacao (#PCDATA)>
    <!ELEMENT Dept_nr (#PCDATA)>
    <!ELEMENT Trabalhadores (Trabalhador*)>

    <!ELEMENT Trabalhador (Cpf, Ultimo_nome?, Primeiro_nome?, Horas)>
    <!ELEMENT Cpf (#PCDATA)>
    <!ELEMENT Ultimo_nome (#PCDATA)>
    <!ELEMENT Primeiro_nome (#PCDATA)>
    <!ELEMENT Horas (#PCDATA)>
  ] >
```

Figura 12.4
Um arquivo XML DTD chamado *Projetos*

tipo é um *elemento de único valor (não repetitivo) opcional*.

- Um elemento sem qualquer um dos três símbolos anteriores precisa aparecer exatamente uma vez no documento. Esse tipo é um *elemento de único valor (não repetitivo) obrigatório*.
- O *tipo* do elemento é especificado por parênteses após ele mesmo. Se os parênteses incluírem nomes de outros elementos, estes são os filhos do elemento na estrutura de árvore. Se os parênteses incluírem a palavra-chave #PCDATA ou um dos outros tipos de dados disponíveis em XML DTD, o elemento é um nó folha. PCDATA significa Parsed Character Data (dados de caractere analisados), que é mais ou menos equivalente a um tipo de dados de string.
- A lista de atributos que podem aparecem em um elemento também pode ser especificada por meio da palavra-chave !ATTLIST. Na Figura 12.3, o elemento Projeto tem um atributo ProjId. Se o tipo de um atributo é ID, então ele pode ser referenciado com base em outro atributo cujo tipo é IDREF dentro de outro elemento. Observe que os atributos também podem ser usados para manter os valores de elementos de dados simples do tipo #PCDATA.
- Os parênteses podem ser aninhados quando se especifica elementos.
- Um símbolo de barra (e_1 | e_2) especifica que e_1 ou e_2 podem aparecer no documento.

Podemos ver que a estrutura de árvore na Figura 12.1 e o documento XML na Figura 12.3 estão em conformidade com a XML DTD da Figura 12.4. Para exigir que um documento XML seja verificado por conformidade com uma DTD, temos de especificar isso na declaração do documento. Por exemplo, poderíamos mudar a primeira linha da Figura 12.3 para:

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE Projetos SYSTEM "proj.dtd">
```

Quando o valor do atributo standalone em um documento XML é "no", o documento precisa ser verificado contra um documento DTD ou um documento de esquema XML separado (ver a seguir). O arquivo DTD mostrado na Figura 12.4 deve ser armazenado no mesmo sistema de arquivos do documento XML, e receber o nome de arquivo proj.dtd. Como alternativa, poderíamos incluir o texto do documento DTD no início do próprio documento XML, para permitir a verificação.

Embora a XML DTD seja bastante adequada para especificar estruturas de árvore com elementos obrigatórios, opcionais e repetitivos, e com vários tipos de atributos, ela tem diversas limitações. Primeiro, os tipos de dados na DTD não são muito genéricos. Em segundo lugar, a DTD tem a própria sintaxe especial e, portanto, requer processadores especializados. Seria vantajoso especificar documentos de esquema XML usando as regras de sintaxe da própria XML, de modo que os mesmos processadores usados para documentos XML pudessem processar descrições de esquema XML. Em terceiro lugar, todos os elementos DTD são sempre forçados a seguir a ordenação especificada do documento, de modo que elementos não ordenados não são permitidos. Essas desvantagens levaram ao desenvolvimento do esquema XML, uma linguagem mais genérica, mas também mais complexa, para especificar a estrutura e os elementos dos documentos XML.

12.3.2 Esquema XML

A **linguagem de esquema XML** é um padrão para especificar a estrutura de documentos XML. Ela usa as mesmas regras de sintaxe dos documentos XML normais, de modo que os mesmos processadores podem ser utilizados em ambos. Para distinguir os dois tipos de documentos, usaremos o termo *documento de instância XML* ou *documento XML* para um documento XML normal, e *documento de esquema XML* para um documento que especifica um esquema XML. A Figura 12.5 mostra um documento de esquema XML correspondente ao banco de dados EMPRESA exibido nas figuras 3.5 e 7.2. Embora seja improvável que queiramos exibir o banco de dados inteiro como um único documento, há propostas para armazenar dados em formato *XML nativo* como uma alternativa ao armazenamento dos dados em bancos de dados relacionais. O esquema da Figura 12.5 atenderia à finalidade de especificar a estrutura do banco de dados EMPRESA se ela fosse armazenada em um sistema XML nativo. Vamos discutir melhor esse assunto na Seção 12.4.

Assim como a XML DTD, o esquema XML é baseado no modelo de dados de árvore, com elementos e atributos como os conceitos de estruturação principais. Contudo, ele utiliza conceitos adicionais dos modelos de banco de dados e objeto, como chaves, referências e identificadores. Aqui, descrevemos os recursos do esquema XML de uma maneira passo a passo, referindo-nos à um documento de esquema XML de exemplo na Figura 12.5 para fins de ilustração. Apresentamos e descrevemos alguns dos conceitos de esquema na ordem em que eles são usados na Figura 12.5.

1. **Descrições de esquema e namespaces XML.** É necessário identificar o conjunto específico de elementos da linguagem de esquema XML (tags) sendo usado ao especificar um arqui-

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <xsd:annotation>
        <xsd:documentation xml:lang="en">Esquema Empresa (Definição Elemento) - Criado por Babak
            Hojabri</xsd:documentation>
    </xsd:annotation>
<xsd:element name="empresa">
    <xsd:complexType>
        <xsd:sequence>
        <xsd:elementname="departamento" type="Departamento" minOccurs="0" maxOccurs="unbounded"/>
            <xsd:element name="funcionario" type="Funcionario" minOccurs="0" maxOccurs= "unbounded">
             <xsd:unique name="UnicoNomedependente">
                        <xsd:selector xpath="dependenteFuncionario" />
                        <xsd:field xpath="nomeDepartamento" />
                    </xsd:unique>
                </xsd:element>
              <xsd:element name="projeto" type="Projeto" minOccurs="0" maxOccurs="unbounded" />
        </xsd:sequence>
    </xsd:complexType>
  <xsd:unique name="unicoNomeDepartamento">
        <xsd:selector xpath="departamento" />
      <xsd:field xpath="nomeDepartmento" />
    </xsd:unique>
  <xsd:unique name="unicoNomeProjeto">
        <xsd:selector xpath="projeto" />
      <xsd:field xpath="nomeProjeto" />
    </xsd:unique>
  <xsd:key name="chaveNumeroProjeto">
      <xsd:selector xpath="projeto" />
      <xsd:field xpath="numeroProjeto" />
    </xsd:key>
  <xsd:key name="chaveNumeroDepartamento">
      <xsd:selector xpath="departamento" />
      <xsd:field xpath="numeroDepartamento" />
    </xsd:key>
  <xsd:key name="chaveCPFFuncionario">
      <xsd:selector xpath="funcionario" />
      <xsd:field xpath="CPFfuncionario" />
    </xsd:key>

  <xsd:keyref name="refChaveCPFGerenteDepartamento"
        refer="chaveCPFFuncionario">
        <xsd:selector xpath="departamento" />
      <xsd:field xpath="CPFGerenteDepartamento" />
    </xsd:keyref>
  <xsd:keyref name="refChaveNumeroDepartamentoFuncionario"
```

Figura 12.5 (*continua*)
Um arquivo de esquema XML chamado *empresa*.

```
        refer="chaveNumeroDepartamento">
          <xsd:selector xpath="funcionario" />
          <xsd:field xpath="numeroDepartamentoFuncionario" />
        </xsd:keyref>
      <xsd:keyref name="refChaveCPFGerenteFuncionario" refer="ChaveCPFFuncionario">
          <xsd:selector xpath="funcionario" />
          <xsd:field xpath="CPFGerenteFuncionario" />
        </xsd:keyref>
      <xsd:keyref name="refChaveNumeroDepartamentoProjeto" refer="ChaveNumeroDepartamento">
            <xsd:selector xpath="projeto" />
          <xsd:field xpath="numeroDepartamentoProjeto" />
        </xsd:keyref>
      <xsd:keyref name="refChaveCPFTrabalhadorProjeto refer="chaveCPFFuncionario">
            <xsd:selector xpath="projeto/trabalhadorProjeto" />
          <xsd:field xpath="CPF" />
        </xsd:keyref>
      <xsd:keyref name="refChaveNumeroProjetoTrabalhaemFuncionario" refer="chaveNumeroP">
          <xsd:selector xpath="funcionario/trabalhaemFuncionario" />
            <xsd:field xpath="numeroProjeto" />
        </xsd:keyref>
    </xsd:element>
    <xsd:complexType name="Departamento">
        <xsd:sequence>
          <xsd:element name="nomeDepartamento" type="xsd:string" />
            <xsd:element name="numeroDepartamento" type="xsd:string" />
            <xsd:element name="CPFGerenteDepartamento" type="xsd:string" />
            <xsd:element name="dataInicioGerenteDepartamento" type="xsd:date" />
           <xsd:element name="localizacaoDepartamento" type="xsd:string" minOccurs="0" maxOccurs="unbounded"
/>
        </xsd:sequence>
    </xsd:complexType>
    <xsd:complexType name="Funcionario">
        <xsd:sequence>
          <xsd:element name="nomeFuncionario" type="Nome" />
            <xsd:element name="CPFFuncionario" type="xsd:string" />
            <xsd:element name="sexoFuncionario" type="xsd:string" />
            <xsd:element name="salarioFuncionario" type="xsd:unsignedInt" />
            <xsd:element name="dataNascimentoFuncionario" type="xsd:date" />
          <xsd:element name="numeroDepartamentoFuncionario" type="xsd:string" />

            <xsd:element name="CPFGerenteFuncionario" type="xsd:string" />
            <xsd:element name="enderecoFuncionario" type="Address" />
           <xsd:element name="trabalhaemFuncionario" type="TrabalhaEm" minOccurs="1" maxOccurs="unbounded" />
            <xsd:element name="dependenteFuncionario" type="Dependente" minOccurs="0" maxOccurs="unbounded" />
        </xsd:sequence>
    </xsd:complexType>
```

Figura 12.5 (continuação) (*continua*)
Um arquivo de esquema XML chamado empresa.

```
<xsd:complexType name="Projeto">
    <xsd:sequence>
      <xsd:element name="nomeProjeto" type="xsd:string" />
      <xsd:element name="numeroProjeto" type="xsd:string" />
      <xsd:element name="localizacaoProjeto" type="xsd:string" />
      <xsd:element name="numeroDepartamentoProjeto" type="xsd:string" />
       <xsd:elementname="trabalhadorProjeto"type="Worker"minOccurs="1"maxOccurs="unbounded"/>
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Dependente">
    <xsd:sequence>
      <xsd:element name="nomeDependente" type="xsd:string" />
        <xsd:element name="sexoDependente" type="xsd:string" />
        <xsd:element name="dataNascimentoDependente" type="xsd:date" />
        <xsd:element name="parentescoDependente" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Endereco">
    <xsd:sequence>
      <xsd:element name="numero" type="xsd:string" />
        <xsd:element name="rua" type="xsd:string" />
        <xsd:element name="cidade" type="xsd:string" />
        <xsd:element name="estado" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Nome">
    <xsd:sequence>
      <xsd:element name="primeiroNome" type="xsd:string" />
      <xsd:element name="nomeMeio" type="xsd:string" />
      <xsd:element name="ultimoNome" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Trabalhador">
    <xsd:sequence>
      <xsd:element name="CPF" type="xsd:string" />
        <xsd:element name="horas" type="xsd:float" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType  name="TrabalhaEm">

    <xsd:sequence>
      <xsd:element name="numeroProjeto" type="xsd:string" />
      <xsd:element name="horas" type="xsd:float" />
    </xsd:sequence>
</xsd:complexType>
</xsd:schema>
```

Figura 12.5 (continuação)
Um arquivo de esquema XML chamado empresa.

vo armazenado em um local de Website. A segunda linha da Figura 12.5 especifica o arquivo utilizado neste exemplo, que é <http://www.w3.org/2001/XMLSchema>. Esse é um padrão normalmente usado para comandos de esquema XML. Cada definição desse tipo é chamada de **namespace XML**, pois define o conjunto de comandos (nomes) que podem ser usados. O nome de arquivo é atribuído à variável xsd (descrição de esquema XML, ou *XML Schema Description*) usando o atributo xmlns (XML namespace), e essa variável é utilizada como um prefixo para todos os comandos (nomes de tag) de esquema XML. Por exemplo, na Figura 12.5, quando escrevemos xsd:element ou xsd:sequence, estamos nos referindo às definições das tags element e sequence conforme definidas no arquivo <http://www.w3.org/2001/XMLSchema>.

2. **Anotações, documentação e linguagem usada.** As duas linhas seguintes da Figura 12.5 ilustram os elementos (tags) do esquema XML xsd:annotation e xsd:documentation, que são usadas para oferecer comentários e outras descrições no documento XML. O atributo xml:lang do elemento xsd:documentation especifica o idioma (*language*) sendo usado, no qual en significa *english* (inglês).
3. **Elementos e tipos.** Em seguida, especificamos o *elemento raiz* de nosso esquema XML. No esquema XML, o atributo name da tag xsd:element especifica o nome do elemento, que se chama empresa para o elemento raiz em nosso exemplo (ver Figura 12.5). A estrutura do elemento raiz empresa pode então ser especificada, que em nosso exemplo é xsd:complexType. Isso é especificado ainda mais como uma sequência de departamentos, funcionários e projetos usando a estrutura xsd:sequence do esquema XML. É importante observar aqui que essa não é a única maneira de especificar um esquema XML para o banco de dados EMPRESA. Discutiremos outras opções na Seção 12.6.
4. **Elementos de primeiro nível no banco de dados EMPRESA.** Em seguida, especificamos os três elementos de primeiro nível sob o elemento raiz empresa da Figura 12.5. Esses elementos são chamados funcionarios, departamento e projeto, e cada um é especificado em uma tag xsd:element. Observe que, se uma tag tem apenas atributos e não mais subelementos ou dados dentro dela, ela pode ser encerrada com o símbolo de contrabarra (/>) diretamente, em vez de ter uma tag de fim correspondente. Estes são chamados **elementos vazios**. Alguns exemplos são os elementos de xsd:element chamados departamento e projeto na Figura 12.5.
5. **Especificando tipo de elemento e ocorrências mínima e máxima.** No esquema XML, os atributos type, minOccurs e maxOccurs na tag xsd:element especificam o tipo e a multiplicidade de cada elemento em qualquer documento que esteja em conformidade com as especificações de esquema. Se especificarmos um atributo type em uma estrutura xsd:element, a estrutura do elemento precisa ser descrita separadamente, em geral usando o elemento xsd:complexType do esquema XML. Isso é ilustrado pelos elementos funcionario, departamento e projeto na Figura 12.5. Mas, se nenhum atributo type for especificado, a estrutura de elementos pode ser definida diretamente após a tag, conforme ilustrado pelo elemento raiz empresa da Figura 12.5. As tags minOccurs e maxOccurs são usadas para especificar os limites inferior e superior sobre o número de ocorrências de um elemento em qualquer documento XML que esteja em conformidade com as especificações do esquema. Se eles não forem especificados, o padrão é exatamente uma ocorrência. Estes têm um papel semelhante aos símbolos *, + e ? da XML DTD.
6. **Especificando chaves.** No esquema XML, é possível especificar restrições que correspondem a restrições únicas e de chave primária em um banco de dados relacional (ver Seção 3.2.2), bem como restrições de chaves estrangeiras (ou integridade referencial) (ver Seção 3.2.4). A tag xsd:unique especifica elementos que correspondem a atributos únicos em um banco de dados relacional. Podemos dar um nome a cada restrição única, e devemos especificar tags xsd:selector e xsd:field para ela, a fim de identificar o tipo de elemento que contém o elemento único e o nome do elemento dentro dela que é único por meio do atributo xpath. Isso é ilustrado pelos elementos unicoNomeDepartamento e unicoNomeProjeto da Figura 12.5. Para especificar **chaves primárias**, a tag xsd:key é usada no lugar de xsd:unique, conforme ilustrado pelos elementos chaveNumeroProjeto, chaveNumeroDepartamento e chaveCPFFuncionario da Figura 12.5. Para especificar **chaves estrangeiras**, a tag xsd:keyref é usada, conforme ilustrado pelos seis elementos xsd:keyref da Figura 12.5. Ao especificar uma chave estrangeira, o atributo refer da tag xsd:keyref especifica a chave primária referenciada, enquanto

as tags xsd:selector e xsd:field especificam o tipo de elemento referenciando e a chave estrangeira (ver Figura 12.5).

7. **Especificando as estruturas de elementos complexos por meio de tipos complexos.** A próxima parte de nosso exemplo especifica as estruturas dos elementos complexos Departamento, Funcionario, Projeto e Dependente, usando a tag xsd:complexType (ver Figura 12.5). Especificamos cada um deles como uma sequência de subelementos correspondentes aos atributos de banco de dados de cada tipo de entidade (ver Figura 3.7) ao usar as tags xsd:sequence e xsd:element do esquema XML. Cada elemento recebe um nome e tipo por meio dos atributos name e type de xsd:element. Também podemos especificar os atributos minOccurs e maxOccurs se precisarmos mudar o padrão de exatamente uma ocorrência. Para atributos de banco de dados (opcionais) em que o nulo é permitido, precisamos especificar minOccurs = 0, ao passo que, para atributos de banco de dados multivalorados, precisamos especificar maxOccurs = "unbounded" no elemento correspondente. Observe que, se não fôssemos especificar quaisquer restrições de chave, poderíamos ter embutido os subelementos nas definições do elemento pai diretamente sem ter de especificar tipos complexos. Contudo, quando restrições únicas, de chave primária e de chave estrangeira precisam ser especificadas, temos de definir tipos complexos para especificar as estruturas de elemento.
8. **Atributos compostos.** Os atributos compostos da Figura 7.2 também são especificados como tipos complexos na Figura 12.7, conforme ilustrado pelos tipos complexos Endereco, Nome, Trabalhador e Trabalha_em. Estes poderiam ter sido embutidos diretamente em seus elementos pai.

Este exemplo ilustra alguns dos principais recursos do esquema XML. Existem outros recursos, mas eles estão além do escopo de nossa apresentação. Na próxima seção, vamos discutir as diferentes técnicas para criar documentos XML baseando-se em bancos de dados relacionais e armazenar documentos XML.

12.4 Armazenando e extraindo documentos XML de bancos de dados

Várias técnicas de organização do conteúdo de documentos XML, para facilitar sua subsequente consulta e recuperação, foram propostas. A seguir estão as mais comuns:

1. **Usar um SGBD para armazenar os documentos como texto.** Um SGBD relacional ou de objeto pode ser utilizado para armazenar os documentos XML inteiros como campos de texto nos registros ou objetos do SGBD. Essa técnica pode ser usada se o SGBD tiver um módulo especial para processamento de documento, e funcionaria para armazenar documentos XML sem esquema e centrados no próprio documento.
2. **Usar um SGBD para armazenar conteúdos de documento como elementos de dados.** Essa técnica funcionaria para armazenar uma coleção de documentos que segue uma XML DTD específica ou um esquema XML. Como todos os documentos têm a mesma estrutura, pode-se projetar um banco de dados relacional (ou de objeto) para armazenar os elementos de dados em nível de folha nos documentos XML. Essa técnica exigiria algoritmos de mapeamento para projetar um esquema de banco de dados compatível com a estrutura do documento XML, conforme especificada no esquema XML ou DTD, para recriar os documentos XML com base nos dados armazenados. Esses algoritmos podem ser implementados como um módulo de SGBD interno ou como middleware separado, que não faz parte do SGBD.
3. **Projetar um sistema especializado para armazenar dados XML nativos.** Um novo tipo de sistema de banco de dados, baseado no modelo hierárquico (de árvore) poderia ser projetado e implementado. Esses sistemas estão sendo chamados de **SGBDs XML nativos**. O sistema incluiria técnicas especializadas para indexação e consulta, e funcionaria para todos os tipos de documentos XML. Ele também poderia incluir técnicas de compactação de dados, para reduzir o tamanho dos documentos para armazenamento. O Tamino, da Software AG, e a Dynamic Application Platform, da eXcelon, são dois produtos populares que oferecem capacidade de SGBD XML. A Oracle também oferece uma opção de armazenamento XML nativo.
4. **Criar ou publicar documentos XML personalizados de bancos de dados relacionais pré-existentes.** Como há grande quantidade de dados já armazenados em bancos de dados relacionais, partes desses dados podem ter de ser formatadas como documentos para a troca ou exibição pela Web. Essa técnica usaria uma camada separada de software de mid-

dleware para tratar das conversões necessárias entre os documentos XML e o banco de dados relacional. A Seção 12.6 discute essa técnica, em que os documentos XML centrados nos dados são extraídos dos bancos de dados existentes, com mais detalhes. Em particular, mostramos como os documentos estruturados em árvore podem ser criados a partir de bancos de dados estruturados em grafo. A Seção 12.6.2 discute o problema de ciclos e como lidar com ele.

Todas essas técnicas receberam bastante atenção. Focalizamos a quarta técnica na Seção 12.6, pois ela oferece um bom entendimento conceitual das diferenças entre o modelo de dados em árvore da XML e os modelos de banco de dados tradicionais baseados em arquivos planos (modelo relacional) e representações gráficas (modelo ER). Mas, primeiro, vamos dar uma visão geral das linguagens de consulta XML na Seção 12.5.

12.5 Linguagens XML

Houve várias propostas para linguagens de consulta XML, e dois padrões de linguagens de consulta se destacaram. O primeiro é o **XPath**, que oferece construções da linguagem para especificar expressões de caminho a fim de identificar certos nós (elementos) ou atributos em um documento XML que combina padrões específicos. O segundo é o **XQuery**, que é uma linguagem de consulta mais geral. A XQuery usa expressões XPath, mas tem construções adicionais. Vamos apresentar uma visão geral de cada uma dessas linguagens nesta seção. Depois, discutiremos algumas linguagens adicionais relacionadas à HTML na Seção 12.5.3.

12.5.1 XPath: especificando expressões de caminho em XML

Uma expressão XPath geralmente retorna uma sequência de itens que satisfazem certo padrão, conforme especificado pela expressão. Esses itens podem ser valores (ou nós de folha), elementos ou atributos. O tipo mais comum de expressão XPath retorna uma coleção de nós de elemento ou atributo que satisfaz certos padrões especificados na expressão. Os nomes na expressão XPath são nomes de nó na árvore de documentos XML que são também nomes de tag (elemento) ou de atributo, possivelmente com **condições qualificadoras** adicionais, para restringir ainda mais os nós que satisfazem o padrão. Dois **separadores** principais são usados ao se especificar um caminho: barra simples (/) e barra dupla (//). Uma barra simples antes de uma tag especifica que esta precisa aparecer como um filho direto da tag anterior (pai), enquanto uma barra dupla especifica que a tag pode aparecer como um descendente da tag anterior, *em qualquer nível*. Vamos examinar alguns exemplos da XPath conforme mostrados na Figura 12.6.

A primeira expressão XPath da Figura 12.6 retorna o nó raiz empresa e todos os nós descendentes, o que significa que retorna o documento XML inteiro. Devemos notar que é comum incluir o nome do arquivo na consulta XPath. Isso nos permite especificar qualquer nome de arquivo local ou mesmo qualquer nome de caminho que determine um arquivo na Web. Por exemplo, se o documento XML EMPRESA está armazenado no local

www.empresa.com/info.XML

então a primeira expressão XPath da Figura 12.6 pode ser escrita como

doc(www.empresa.com/info.XML)/empresa

Esse prefixo também seria incluído nos outros exemplos de expressões XPath.

O segundo exemplo da Figura 12.6 retorna todos os nós (elementos) de departamento e suas subárvores descendentes. Observe que os nós (elementos) em um documento XML são ordenados, de modo que o resultado XPath que retorna vários nós fará isso na mesma ordem em que os nós são ordenados na árvore do documento.

A terceira expressão XPath da Figura 12.6 ilustra o uso de //, que é conveniente se não soubermos o nome do caminho completo que estamos procuran-

1. /empresa
2. /empresa/departamento
3. //funcionario [salarioFuncionario gt 70.000]/nomeFuncionario
4. /empresa/funcionario[salarioFuncionario gt 70.000]/nomeFuncionario
5. /empresa/projeto/trabalhadorProjeto [horas ge 20,0]

Figura 12.6
Alguns exemplos de expressões XPath em documentos XML que seguem o arquivo de esquema XML *empresa* na Figura 12.5.

do, mas sabemos o nome de algumas tags de interesse no documento XML. Isso é útil particularmente para documentos XML sem esquema ou para documentos com muitos níveis de nós aninhados.[6]

A expressão retorna todos os nós nomeFuncionario que são filhos diretos de um nó funcionario, de modo que o nó funcionario tem outro elemento filho salarioFuncionario cujo valor é maior que 70000. Isso ilustra o uso de condições qualificadoras, que restringem os nós selecionados pela expressão XPath àqueles que satisfazem a condição. A XPath tem uma série de operações de comparação para uso nas condições qualificadoras, incluindo operações de comparação aritmética padrão, de string e de conjunto.

A quarta expressão XPath da Figura 12.6 deve retornar o mesmo resultado da anterior, exceto que especificamos o nome do caminho completo nesse exemplo. A quinta expressão da Figura 12.6 retorna todos os nós trabalhadorProjeto e seus nós descendentes, que são filhos sob um caminho /trabalhador/projeto e têm um nó filho horas com um valor maior que 20,0 horas.

Quando precisamos incluir atributos em uma expressão XPath, o nome do atributo é iniciado pelo símbolo @ para distingui-lo dos nomes de elemento (tag). É possível usar o símbolo **curinga** *, que representa qualquer elemento, como no exemplo a seguir, que recupera todos os elementos que são elementos filho da raiz, independentemente de seu tipo de elemento. Quando são usados curingas, o resultado pode ser uma sequência de diferentes tipos de itens.

/empresa/*

Os exemplos anteriores ilustram expressões XPath simples, nas quais só podemos descer na estrutura da árvore de determinado nó. Um modelo mais geral para expressões de caminho já foi proposto. Nesse modelo, é possível mover em várias direções a partir do nó atual na expressão de caminho. Estes são conhecidos como **eixos** de uma expressão XPath. Nossos exemplos usaram apenas *três desses eixos*: filho do nó atual (/), descendente ou ele mesmo em qualquer nível do nó atual (//) e atributo do nó atual (@). Outros eixos incluem pai, ancestral (em qualquer nível), irmão anterior (qualquer nó no mesmo nível à esquerda na árvore) e irmão seguinte (qualquer nó no mesmo nível à direita na árvore). Esses eixos permitem expressões de caminho mais complexas.

A principal restrição de expressões de XPath é que o caminho que especifica o padrão também especifica os itens a serem recuperados. Logo, é difícil especificar certas condições sobre o padrão enquanto se especifica separadamente quais itens do resultado devem ser recuperados. A linguagem XQuery separa esses dois problemas, e oferece construções mais poderosas para especificar consultas.

12.5.2 XQuery: especificando consultas em XML

A XPath nos permite escrever expressões que selecionam itens de um documento XML estruturado em árvore. A XQuery possibilita a especificação de consultas mais gerais sobre um ou mais documentos XML. O formulário típico de uma consulta em XQuery é conhecido como **expressão FLWR**, que indica as quatro cláusulas principais da XQuery e tem a seguinte forma:

FOR <vínculos de variável para nós (elementos) individuais>
LET <vínculos de variável para coleções de nós (elementos)>
WHERE <condições qualificadoras>
RETURN <especificação de resultado da consulta>

Pode haver zero ou mais instâncias da cláusula FOR, bem como da cláusula LET, em uma única XQuery. A cláusula WHERE é opcional, mas pode aparecer no máximo uma vez, e a cláusula RETURN deve aparecer exatamente uma vez. Vamos ilustrar essas cláusulas com o seguinte exemplo simples de uma XQuery.

```
LET $d := doc(www.empresa.com/info.xml)
FOR $x IN $d/empresa/projeto[NumeroProjeto = 5]/
    trabalhadorProjeto, $y IN $d/empresa/funcio
    nario
WHERE $x/horas gt 20.0 AND $y.cpf = $x.cpf
RETURN <res> $y/nomeFuncionario/primeiroNome,
             $y/nomeFuncionario/ultimoNome,
             $x/horas </res>
```

1. As variáveis são iniciadas com o sinal $. No exemplo, $d, $x e $y são variáveis.
2. A cláusula LET atribui uma variável a uma expressão em particular para o restante da consulta. Neste exemplo, $d é atribuída ao nome do arquivo de documento. É possível ter uma consulta que se refere a vários documentos ao atribuir diversas variáveis dessa forma.
3. A cláusula FOR atribui uma variável ao intervalo sobre cada um dos itens individuais em uma sequência. Em nosso exemplo, as sequências são especificadas por expressões de caminho. A variável $x percorre os ele-

[6] Usamos os termos nó, tag e elemento para indicar a mesma coisa aqui.

mentos que satisfazem a expressão de caminho $d/empresa/projeto[NumeroProjeto = 5]/trabalhadorProjeto. A variável $y percorre os elementos que satisfazem a expressão de caminho $d/empresa/funcionario. Logo, $x percorre os elementos trabalhadorProjeto, enquanto $y percorre os elementos funcionario.

4. A cláusula WHERE especifica condições adicionais sobre a seleção de itens. Nesse exemplo, a primeira condição seleciona apenas os elementos trabalhadorProjeto que satisfazem a condição (horas gt 20,0). A segunda condição especifica uma condição de junção que combina um funcionario com um trabalhadorProjeto somente se eles tiverem o mesmo valor de cpf.
5. Finalmente, a cláusula RETURN especifica quais elementos ou atributos devem ser recuperados dos itens que satisfazem as condições de consulta. Neste exemplo, ela retornará uma sequência de elementos, cada um contendo <primeiroNome, ultimoNome, horas> para funcionários que trabalham mais de 20 horas por semana no projeto número 5.

A Figura 12.7 inclui alguns exemplos adicionais de consultas em XQuery, as quais podem ser especificadas nos documentos de instância XML que seguem o documento de esquema XML na Figura 12.5. A primeira consulta recupera os nomes e sobrenomes dos funcionários que ganham mais de R$70.000. A variável $x está ligada a cada elemento nomeFuncionario, que é um filho de um elemento funcionario, mas somente para elementos de funcionario que satisfazem o qualificador de que seu valor de salarioFuncionario é maior do que R$70.000. O resultado recupera os elementos filhos primeiroNome e ultimoNome dos elementos nomeFuncionario selecionados. A segunda consulta é um modo alternativo de recuperar os mesmos elementos recuperados pela primeira consulta.

A terceira consulta ilustra como uma operação de junção pode ser realizada usando mais de uma variável. Aqui, a variável $x está ligada a cada elemento trabalhadorProjeto, que é um filho do projeto número 5, enquanto a variável $y está ligada a cada elemento funcionario. A condição de junção combina valores cpf a fim de recuperar os nomes de funcionário. Observe que esse é um modo alternativo de especificar a mesma consulta em nosso exemplo anterior, mas sem a cláusula LET.

A XQuery possui construções muito poderosas para especificar consultas complexas. Em particular, ela pode especificar quantificadores universais e existenciais nas condições de uma consulta, funções de agregação, ordenação dos resultados da consulta, seleção baseada na posição em uma sequência, e até mesmo desvio condicional. Portanto, de algumas maneiras, ela se qualifica como uma linguagem de programação completa.

Isso conclui nossa breve introdução à XQuery. O leitor interessado deverá consultar <www.w3.org>, que contém documentos descrevendo os padrões mais recentes relacionados à XML e XQuery. A próxima seção vai discutir rapidamente algumas linguagens e protocolos adicionais relacionados à XML.

```
1. FOR $x IN
     doc(www.empresa.com/info.xml)
   //funcionario [salarioFuncionario gt 70.000]/nomeFuncionario
   RETURN <res> $x/primeiroNome, $x/ultimoNome </res>
2. FOR $x IN
     doc(www.empresa.com/info.xml)/empresa/funcionario
   WHERE $x/salarioFuncionario gt 70.000
   RETURN <res> $x/nomeFuncionario/primeiroNome, $x/nomeFuncionario/ultimoNome </res>
3. FOR $x IN
   doc(www.empresa.com/info.xml)/empresa/projeto[numeroProjeto = 5]/trabalhadorProjeto,
   $y IN doc(www.empresa.com/info.xml)/empresa/funcionario
    WHERE $x/horas gt 20,0 AND $y.cpf = $x.cpf
   RETURN <res> $y/nome_funcionario/primeiroNome, $y/nomeFuncionario/ultimoNome, $x/horas</res>
```

Figura 12.7
Alguns exemplos de consultas XQuery em documentos XML que seguem o arquivo de esquema XML *company* da Figura 12.5.

12.5.3 Outras linguagens e protocolos relacionados a XML

Existem várias outras linguagens e protocolos relacionados à tecnologia XML. O objetivo em longo prazo destas e de outras linguagens e protocolos é oferecer a tecnologia para a realização da Web semântica, na qual toda informação na Web possa ser inteligentemente localizada e processada.

- A *Extensible Stylesheet Language* (XSL) pode ser usada para definir como um documento deve ser renderizado para exibição por um navegador Web.
- A *Extensible Stylesheet Language for Transformations* (XSLT) pode ser usada para transformar uma estrutura em outra. Logo, ela pode converter documentos de uma forma para outra.
- A *Web Services Description Language* (WSDL) permite a descrição de Web Services em XML. Isso torna o Web Service disponível para usuários e programas pela Web.
- O *Simple Object Access Protocol* (SOAP) é um protocolo independente de plataforma e de linguagem de programação para transmissão de mensagens e chamadas de procedimento remoto.
- O *Resource Description Framework* (RDF) oferece linguagens e ferramentas para trocar e processar descrições de metadados (esquema) e especificações pela Web.

12.6 Extraindo documentos XML de bancos de dados relacionais

12.6.1 Criando visões XML hierárquicas sobre dados planos ou baseados em grafos

Esta seção aborda as questões de representação que surgem quando se converte dados de um sistema de banco de dados para documentos XML. Conforme discutimos, a XML usa um modelo hierárquico (em árvore) para representar documentos. Os sistemas de banco de dados com uso mais difundido seguem o modelo de dados relacional plano. Quando acrescentamos restrições de integridade referencial, um esquema relacional pode ser considerado uma estrutura gráfica (por exemplo, ver Figura 3.7). De modo semelhante, o modelo ER representa dados que usam estruturas tipo grafo (por exemplo, ver Figura 7.2). Vimos no Capítulo 9 que existem mapeamentos diretos entre os modelos ER e relacional, de modo que podemos conceitualmente representar um esquema de banco de dados relacional usando o esquema ER correspondente. Embora usemos o modelo ER em nossa discussão e exemplos para esclarecer as diferenças conceituais entre os modelos de árvore e grafo, as mesmas questões se aplicam à conversão de dados relacionais para XML.

Usaremos o esquema ER UNIVERSIDADE simplificado mostrado na Figura 12.8 para ilustrar nossa discussão. Suponha que uma aplicação precise extrair documentos XML para informações sobre aluno, disciplina e nota do banco de dados UNIVERSIDADE. Os dados necessários para esses documentos estão contidos nos atributos do banco de dados dos tipos de entidade DISCIPLINA, TURMA e ALUNO da Figura 12.8, e nos relacionamentos T-A e D-T entre eles. Em geral, a maioria dos documentos extraídos de um banco de dados só usará um subconjunto dos atributos, tipos de entidade e relacionamentos no banco de dados. Neste exemplo, o subconjunto do banco de dados que é necessário aparece na Figura 12.9.

Pelo menos três hierarquias de documento possíveis podem ser extraídas do subconjunto do banco de dados da Figura 12.9. Primeiro, podemos escolher DISCIPLINA como a raiz, conforme ilustramos na Figura 12.10. Aqui, cada entidade de disciplina tem o conjunto de suas turmas como subelementos, e cada turma tem seus alunos como subelementos. Podemos ver uma consequência da modelagem da informação em uma estrutura de árvore hierárquica. Se um aluno tiver realizado diversas turmas, a informação desse aluno aparecerá várias vezes no documento — uma vez sob cada turma. Um esquema XML simplificado possível para essa visão é mostrado na Figura 12.11. O atributo de banco de dados Nota no relacionamento T-A é migrado para o elemento ALUNO. Isso porque ALUNO torna-se um filho de TURMA nessa hierarquia, de modo que cada elemento ALUNO sob um elemento TURMA específico pode ter uma nota específica nessa turma. Nessa hierarquia de documentos, um aluno que está em mais de uma turma terá várias réplicas, uma sob cada turma, e cada réplica terá a nota específica dada nessa turma em particular.

Na segunda visão de documento hierárquico, podemos escolher ALUNO como raiz (Figura 12.12). Nessa visão hierárquica, cada aluno tem um conjunto de turmas como seus elementos filhos, e cada turma está relacionada a uma disciplina como seu filho, pois o relacionamento entre TURMA e DISCIPLINA é N:1. Assim, podemos mesclar os elementos DISCIPLINA e TURMA nesta visão, como mostra a Figura 12.12. Além disso, o atributo de banco de

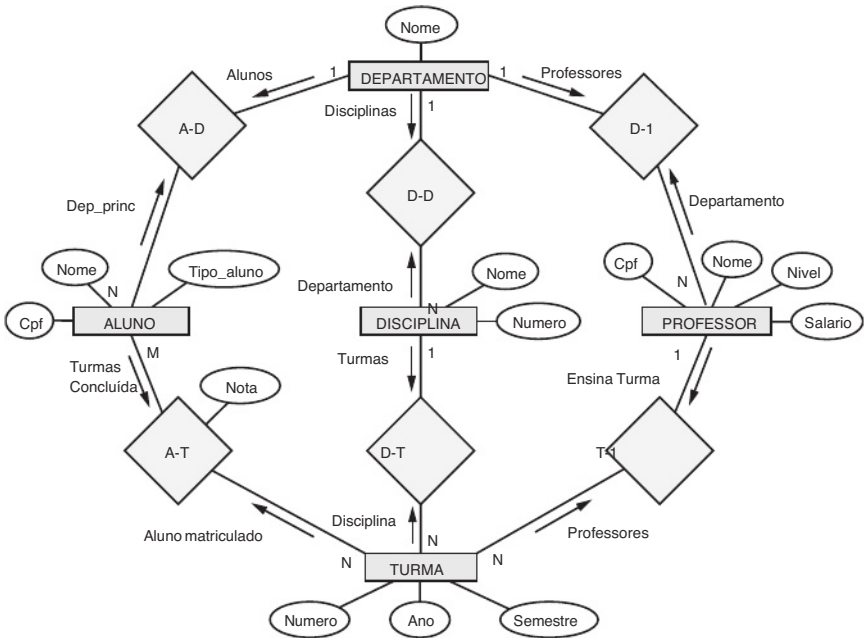


Figura 12.8
Um diagrama de esquema ER para um banco de dados UNIVERSIDADE simplificado.

dados NOTA pode ser migrado para o elemento TURMA. Nessa hierarquia, a informação combinada de DISCIPLINA/TURMA é replicada sob cada aluno que concluir a turma. Um esquema XML simplificado possível para essa visão aparece na Figura 12.13.

A terceira maneira possível é escolher TURMA como a raiz, conforme mostra a Figura 12.14. Semelhante à segunda visão hierárquica, a informação de DISCIPLINA pode ser mesclada no elemento TURMA. O atributo de banco de dados NOTA pode ser migrado para o elemento ALUNO. Como podemos ver, até mesmo nesse exemplo simples pode haver diversas visões de documento hierárquicas, cada uma correspondendo a uma raiz diferente e uma estrutura de documento XML diferente.

12.6.2 Quebrando ciclos para converter grafos em árvores

Nos exemplos anteriores, o subconjunto do banco de dados de interesse não tinha ciclos. É possível ter um subconjunto mais complexo com um ou mais ciclos, indicando múltiplos relacionamentos entre as entidades. Nesse caso, é mais difícil decidir como criar as hierarquias de documento. Umaduplicação adicio-

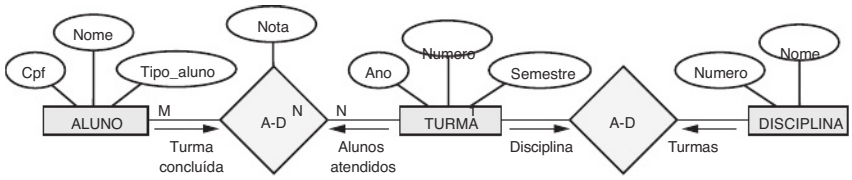


Figura 12.9
Subconjunto do esquema de banco de dados UNIVERSIDADE necessário para a extração de documento XML.

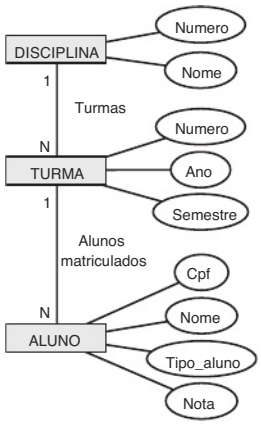


Figura 12.10
Visão hierárquica (em árvore) com DISCIPLINA como a raiz.

```
<xsd:element name="root">
    <xsd:sequence>
    <xsd:element name="disciplina" minOccurs="0" maxOccurs="unbounded">
        <xsd:sequence>
          <xsd:element name="dnome" type="xsd:string" />
          <xsd:element name="numdiscip" type="xsd:unsignedInt" />
          <xsd:element name="turma" minOccurs="0" maxOccurs="unbounded">
                <xsd:sequence>
              <xsd:element name="numturma" type="xsd:unsignedInt" />
              <xsd:element name="ano" type="xsd:string" />
              <xsd:element name="semestre" type="xsd:string" />
              <xsd:element name="aluno" minOccurs="0" maxOccurs="unbounded">
                    <xsd:sequence>
                  <xsd:element name="cpf" type="xsd:string" />
                  <xsd:element name="anome" type="xsd:string" />
                  <xsd:element name="tipoaluno" type="xsd:string" />
                  <xsd:element name="nota" type="xsd:string" />
                    </xsd:sequence>
                </xsd:element>
            </xsd:sequence>
          </xsd:element>
        </xsd:sequence>
</xsd:element>
</xsd:sequence>
</xsd:element>
```

Figura 12.11
Documento de esquema XML com *disciplina* como a raiz.

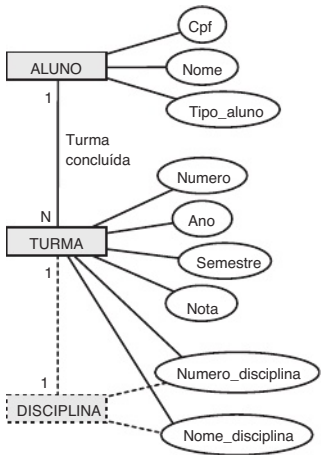


Figura 12.12
Visão hierárquica (em árvore) com ALUNO como a raiz.

```
<xsd:element name="root">
<xsd:sequence>
<xsd:element name="aluno" minOccurs="0" maxOccurs="unbounded">
      <xsd:sequence>
         <xsd:element name="cpf" type="xsd:string" />
         <xsd:element name="anome" type="xsd:string" />
         <xsd:element name="tipoaluno" type="xsd:string" />
         <xsd:element name="turma" minOccurs="0" maxOccurs="unbounded">
                  <xsd:sequence>
                  <xsd:element name="numturma" type="xsd:unsignedInt" />
                  <xsd:element name="ano" type="xsd:string" />
                  <xsd:element name="semestre" type="xsd:string" />
                  <xsd:element name="numdiscip" type="xsd:unsignedInt" />
                  <xsd:element name="dnome" type="xsd:string" />
                  <xsd:element name="nota" type="xsd:string" />
                  </xsd:sequence>
            </xsd:element>
      </xsd:sequence>
</xsd:element>

</xsd:sequence>
</xsd:element>
```

Figura 12.13
Documento de esquema XML com *aluno* como a raiz.

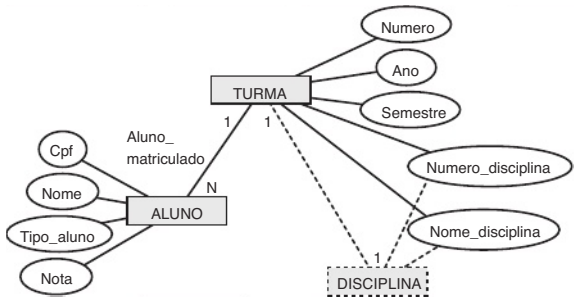


Figura 12.14
Visão hierárquica (em árvore) com TURMA como a raiz.

nal de entidades pode ser necessária para representar os múltiplos relacionamentos. Ilustraremos isso com um exemplo que usa o esquema ER da Figura 12.8.

Suponha que precisemos da informação em todos os tipos de entidade e relacionamentos da Figura 12.8 para um documento XML em particular, com ALUNO como elemento raiz. A Figura 12.15 ilustra como uma possível estrutura em árvore hierárquica pode ser criada para esse documento. Primeiro, obtemos um reticulado com ALUNO como raiz, conforme mostra a Figura 12.15(a). Essa não é uma estrutura em árvore verdadeira por causa dos ciclos. Um modo de quebrar os ciclos é replicar os tipos de entidade neles envolvidos. Primeiro, replicamos PROFESSOR, como mostra a Figura 12.15(b), chamando a réplica para o PROFESSOR1 à direita. A réplica de PROFESSOR à esquerda representa o relacionamento entre professores e as turmas que eles lecionam, enquanto a réplica PROFESSOR1 à direita representa o relacionamento entre professores e o departamento em que cada um trabalha. Depois disso, ainda temos o ciclo envolvendo DISCIPLINA, de modo que podemos replicar DISCIPLINA de uma maneira semelhante, levando à hierarquia mostrada na Figura 12.15(c). A réplica DISCIPLINA1 à esquerda representa o relacionamento entre disciplinas e suas turmas, ao passo que a réplica de DISCIPLINA à direita representa o relacionamento entre disciplinas e o departamento que oferece cada uma delas.

Na Figura 12.15(c), convertemos o grafo inicial em uma hierarquia. Podemos fazer outra mesclagem, se for desejado (como em nosso exemplo anterior) antes de criar a hierarquia final e a estrutura do esquema XML correspondente.

12.6.3 Outras etapas para extrair documentos XML de bancos de dados

Além de criar a hierarquia XML apropriada e o documento de esquema XML correspondente, várias

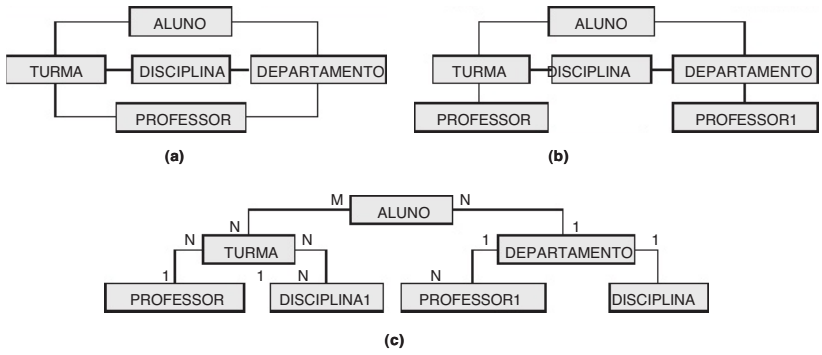


Figura 12.15
Convertendo um grafo com círculos em uma estrutura hierárquica (em árvore).

outras etapas são necessárias para extrair um documento XML em particular de um banco de dados:

1. É necessário criar a consulta correta em SQL para extrair a informação desejada para o documento XML.
2. Quando a consulta é executada, seu resultado deve ser reestruturado da forma relacional para a estrutura em árvore da XML.
3. A consulta pode ser personalizada para selecionar tanto um único objeto como vários objetos no documento. Por exemplo, na visão da Figura 12.13, a consulta pode selecionar uma única entidade de aluno e criar um documento correspondente a esse único aluno, ou pode selecionar vários — ou mesmo todos os alunos — e criar um documento com múltiplos alunos.

Resumo

Este capítulo forneceu uma visão geral do padrão XML para representação e troca de dados pela Internet. Primeiro, discutimos algumas das diferenças entre diversos tipos de dados, classificando três tipos principais: estruturados, semiestruturados e não estruturados. Os dados estruturados são armazenados em bancos de dados tradicionais. Os dados semiestruturados misturam nomes de tipos de dados e valores de dados, mas nem todos eles precisam seguir uma estrutura predefinida fixa. Os dados não estruturados referem-se à informação exibida na Web, especificada pela HTML, em que a informação sobre os tipos dos itens de dados não existe. Descrevemos o padrão XML e seu modelo de dados estruturado em árvore (hierárquico), e discutimos os documentos XML e as linguagens para especificar a estrutura desses documentos, a saber, XML DTD (*Document Type Definition*) e esquema XML. Demos uma visão geral das diversas técnicas para armazenar documentos XML, seja no formato nativo (texto), no formato compactado ou nos bancos de dados relacionais e de outros tipos. Por fim, oferecemos uma visão geral das linguagens XPath e XQuery, propostas para a consulta de dados XML, e discutimos as questões de mapeamento que surgem quando é necessário converter dados armazenados nos bancos de dados relacionais tradicionais para documentos XML.

Perguntas de revisão

12.1. Quais são as diferenças entre dados estruturados, semiestruturados e não estruturados?

12.2. Sob qual das categorias em 12.1 os documentos XML se encontram? E os dados autodescritivos?

12.3. Quais são as diferenças entre o uso de tags em XML *versus* HTML?

12.4. Qual é a diferença entre documentos XML centrados nos dados e centrados nos documentos?

12.5. Qual é a diferença entre os atributos e os elementos na XML? Liste alguns dos atributos importantes usados para especificar elementos no esquema XML.

12.6. Qual é a diferença entre esquema XML e XML DTD?

Exercícios

12.7. Crie parte de um documento de instância XML para corresponder aos dados armazenados no banco de dados relacional mostrado na Figura 3.6, tal que o documento XML corresponda ao documento de esquema XML da Figura 12.5.

12.8. Crie documentos de esquema XML e XML DTDs para corresponderem às hierarquias mostradas nas figuras 12.14 e 12.15(c).

12.9. Considere o esquema de banco de dados relacional BIBLIOTECA da Figura 4.6. Crie um documento de esquema XML que corresponda a esse esquema de banco de dados.

12.10. Especifique as visões a seguir como consultas em XQuery sobre o esquema XML *empresa* mostrado na Figura 12.5.

a. Uma visão que tem nome de departamento, nome de gerente e salário de gerente para cada departamento.
b. Uma visão que tem o nome do funcionário, nome do supervisor e salário de cada funcionário que trabalha no departamento Pesquisa.
c. Uma visão que tem o nome do projeto, nome do departamento de controle, número de funcionários e total de horas trabalhadas por semana para cada projeto.
d. Uma visão que tem o nome do projeto, nome do departamento de controle, número de funcionários e total de horas trabalhadas por semana para cada projeto com mais de um funcionário trabalhando nele.

Bibliografia selecionada

Existem tantos artigos e livros sobre vários aspectos da XML que seria impossível fazer até mesmo uma lista modesta. Mencionaremos um livro: Chaudhri, Rashid e Zicari (eds.), de 2003. Esse livro discute diversos aspectos da XML e contém uma lista de algumas referências a pesquisa e prática em XML.

parte 5

Técnicas de programação de banco de dados

capítulo 13

Introdução às técnicas de programação SQL

Nos capítulos 4 e 5, descrevemos vários aspectos da linguagem SQL, que é o padrão para bancos de dados relacionais. Descrevemos as instruções SQL para definição de dados, modificação de esquema, consultas, visões e atualizações. Também descrevemos como são especificadas diversas restrições sobre o conteúdo do banco de dados, como restrições de chave e integridade referencial.

Neste capítulo e no próximo, vamos discutir alguns dos métodos que foram desambolvidos para acessar bancos de dados de programas. A maioria das técnicas é realizada por meio de programas de software que implementam **aplicações de banco de dados**. Esse software normalmente é desambolvido em uma linguagem de programação de uso geral, como Java, C/C++/C#, COBOL ou alguma outra linguagem de programação. Além disso, muitas linguagens de scripting, como PHP e JavaScript, também estão sendo usadas para programação de acesso do banco de dados em aplicações Web. Neste capítulo, focalizamos como os bancos de dados podem ser acessados das linguagens de programação tradicionais C/C++ e Java, enquanto o próximo capítulo vai mostrar como os bancos de dados são acessados com linguagens de scripting, como PHP e JavaScript. Lembre-se, da Seção 2.3.1, que quando as instruções do banco de dados são incluídas em um programa, a linguagem de programação de uso geral é chamada de *linguagem hospedeira*, ao passo que a linguagem de banco de dados — SQL, em nosso caso — é chamada de *sublinguagem de dados*. Em alguns casos, *linguagens de programação de banco de dados* especiais são desambolvidas especificamente para a escrita de aplicações de banco de dados. Embora muitas delas tenham sido desambolvidas como protótipos de pesquisa, algumas linguagens de programação de banco de dados notáveis possuem uso generalizado, como a PL/SQL (Programming Language/SQL) da Oracle.

É importante observar que a programação de banco de dados é um assunto muito amplo. Existem livros-texto inteiros dedicados a cada técnica de programação de banco de dados e como essa técnica é realizada em um sistema específico. Novas técnicas são desambolvidas o tempo todo, e as mudanças nas técnicas existentes são incorporadas

ao acesso ao banco de dados em aplicações práticas é realizada por meio de programas de software que implementam **aplicações de banco de dados**. Esse software normalmente é desambolvido em uma linguagem de programação de uso geral, como Java, C/C++/C#, COBOL ou alguma outra linguagem de programação. Além disso, muitas linguagens de scripting, como PHP e JavaScript, também estão sendo usadas para programação de acesso do banco de dados em aplicações Web. Neste capítulo, focalizamos como os bancos de dados podem ser acessados das linguagens de programação tradicionais C/C++ e Java, enquanto o próximo capítulo vai mostrar como os bancos de dados são acessados com linguagens de scripting, como PHP e JavaScript. Lembre-se, da Seção 2.3.1, que quando as instruções do banco de dados são incluídas em um programa, a linguagem de programação de uso geral é chamada de *linguagem hospedeira*, ao passo que a linguagem de banco de dados — SQL, em nosso caso — é chamada de *sublinguagem de dados*. Em alguns casos, *linguagens de programação de banco de dados* especiais são desambolvidas especificamente para a escrita de aplicações de banco de dados. Embora muitas delas tenham sido desambolvidas como protótipos de pesquisa, algumas linguagens de programação de banco de dados notáveis possuem uso generalizado, como a PL/SQL (Programming Language/SQL) da Oracle.

É importante observar que a programação de banco de dados é um assunto muito amplo. Existem livros-texto inteiros dedicados a cada técnica de programação de banco de dados e como essa técnica é realizada em um sistema específico. Novas técnicas são desambolvidas o tempo todo, e as mudanças nas técnicas existentes são incorporadas

diversos sistemas de linguagens reais nessa área. Uma dificuldade adicional na apresentação desse tópico é que, embora existam padrões de SQL, eles mesmos estão continuamente evoluindo, e cada vendedor de SGBD pode ter algumas variações do padrão. Por causa disso, escolhemos fazer uma introdução a alguns dos principais tipos de técnicas de programação de banco de dados e compará-las, em vez de estudar um método ou sistema em particular com detalhes. Os exemplos que damos servem para ilustrar as principais diferenças que um programador enfrentaria ao usar cada uma dessas técnicas de programação de banco de dados. Tentaremos usar os padrões de SQL em nossos exemplos no lugar de descrever um sistema específico. Ao usar um sistema específico, os materiais neste capítulo podem servir como uma introdução, mas devem ser expandidos com os manuais do sistema e com livros que descrevem o sistema específico.

Iniciamos nossa apresentação da programação de banco de dados na Seção 13.1 com uma visão geral das diferentes técnicas desambolvidas para acessar um banco de dados de programas. Depois, na Seção 13.2, discutimos as regras para embutir instruções SQL em uma linguagem de programação de uso geral, comumente conhecida como *SQL embutida*.

Esta seção também discute rapidamente a *SQL dinâmica*, em que as consultas podem ser construídas dinamicamente em tempo de execução, e apresenta os fundamentos da variante SQLJ da SQL embutida, que foi desambolvida especificamente para a linguagem de programação Java. Na Seção 13.3, discutimos a técnica conhecida como *SQL/CLI* (*Call Level Interface*), em que uma biblioteca de procedimentos e funções é fornecida para acessar o banco de dados. Diversos conjuntos de funções de biblioteca foram propostos. O conjunto de funções da SQL/CLI é aquele dado no padrão SQL. Outra biblioteca de funções é *ODBC* (*Open Data Base Connectivity*). Não descrevemos a ODBC porque ela é considerada predecessora da SQL/CLI. Uma terceira biblioteca de funções — que descrevemos — é a *JDBC*; esta foi desambolvida especificamente para acessar bancos de dados baseados na linguagem Java. Na Seção 13.4, vamos discutir sobre *SQL/PSM* (*Persistent Stored Modules*), que é uma parte do padrão SQL que permite que módulos de programa — procedimentos e funções — sejam armazenados pelo SGBD e acessados pela SQL. Comparamos rapidamente as três técnicas de programação de banco de dados na Seção 13.5, e oferecemos um resumo do capítulo no final.

13.1 Programação de banco de dados: técnicas e problemas

Agora, vamos voltar nossa atenção para as técnicas que foram desambolvidas para acessar bancos de dados de programas e, em particular, para a questão de como acessar bancos de dados SQL de programas de aplicação. Nossa apresentação da SQL nos capítulos 4 e 5 focalizou as construções da linguagem para diversas operações do banco de dados — da definição do esquema e especificação de restrição até a consulta, atualização e especificação de visões. A maioria dos sistemas de banco de dados possui uma **interface interativa** na qual esses comandos SQL podem ser digitados diretamente em um monitor para execução pelo sistema de banco de dados. Por exemplo, em um sistema de computador em que o SGBD Oracle é instalado, o comando SQLPLUS inicia a interface interativa. O usuário pode digitar comandos ou consultas SQL diretamente em várias linhas, terminadas com um ponto e vírgula e uma tecla Enter (ou seja, '; <cr>'). Como alternativa, um **arquivo de comandos** pode ser criado e executado por meio da interface interativa ao digitar @<*nomearquivo*>. O sistema executará os comandos escritos no arquivo e exibirá os resultados, se houver.

A interface interativa é muito conveniente para a criação de esquema e restrição ou para consultas *ad hoc* ocasionais. Porém, na prática, a maioria das interações de banco de dados é executada por programas que foram cuidadosamente projetados e testados. Esses programas costumam ser conhecidos como **programas de aplicação** ou **aplicações de banco de dados**, e são usados como *transações programadas* pelos usuários finais, conforme discutimos na Seção 1.4.3. Outro uso comum da programação de banco de dados é para acessar um banco de dados por meio de um programa de aplicação que implementa uma **interface Web**, por exemplo, quando se faz reservas ou compras de uma companhia aérea. De fato, a grande maioria das aplicações de comércio eletrônico na Web inclui alguns comandos de acesso a banco de dados. O Capítulo 14 nos dará uma visão geral da programação de banco de dados Web usando PHP, uma linguagem de scripting que recentemente se tornou bastante utilizada.

Nesta seção, primeiro damos uma visão geral das principais técnicas de programação de banco de dados. Depois, discutimos alguns dos problemas que ocorrem quando se tenta acessar um banco de dados com base em uma linguagem de programação de uso geral, e a sequência típica de comandos para interagir com um banco de dados de um programa de software.

13.1.1 Técnicas para a programação de banco de dados

Existem várias técnicas para incluir interações do banco de dados nos programas de aplicação. As principais técnicas para programação de banco de dados são as seguintes:

1. **Embutir comandos do banco de dados em uma linguagem de programação de uso geral.** Nessa técnica, os comandos do banco de dados são **embutidos** na linguagem de programação hospedeira, mas eles são identificadas por um prefixo especial. Por exemplo, o prefixo para a SQL embutida é a string EXEC SQL, que precede todos os comandos SQL em um programa de linguagem hospedeira[1]. Um **pré-compilador** ou **pré-processador** varre o código do programa fonte para identificar os comandos de banco de dados e extraí-los para processamento pelo SGBD. Eles são substituídos no programa por chamadas de função ao código gerado pelo SGBD. Essa técnica geralmente é conhecida como **SQL embutida**.

[1] Outros prefixos às vezes são usados, mas este é o mais comum.

2. **Usar uma biblioteca de funções de banco de dados.** Uma **biblioteca de funções** se torna disponível à linguagem de programação hospedeira para chamadas de banco de dados. Por exemplo, poderia haver funções para conectar com um banco de dados, executar uma atualização, e assim por diante. Os comandos reais de consulta e atualização do banco de dados e quaisquer outras informações necessárias são incluídos como parâmetros nas chamadas de função. Essa técnica oferece o que é conhecido como **uma interface de programação de aplicação** (**API** — **Application Programming Interface**) para acessar um banco de dados de programas de aplicação.
3. **Projetar uma linguagem totalmente nova.** Uma **linguagem de programação de banco de dados** é projetada do zero para ser compatível com o modelo de banco de dados e a linguagem de consulta. Estruturas de programação adicionais, como loops e instruções condicionais, são acrescentadas à linguagem de banco de dados para convertê-la em uma linguagem de programação completa. Um exemplo dessa técnica é a PL/SQL da Oracle.

Na prática, as duas primeiras técnicas são mais comuns, pois muitas aplicações já são escritas em linguagens de programação de uso geral, mas exigem algum acesso ao banco de dados. A terceira técnica é mais apropriada para aplicações que possuem intensa interação com o banco de dados. Um dos principais problemas com as duas primeiras técnicas é a *divergência de impedância*, que não ocorre na terceira técnica.

13.1.2 Divergência de impedância

Divergência de impedância é o termo usado para se referir aos problemas que ocorrem devido às diferenças entre o modelo de banco de dados e o modelo da linguagem de programação. Por exemplo, o modelo relacional prático tem três construções principais: colunas (atributos) e seus tipos de dados, linhas (também chamadas de tuplas ou registros) e tabelas (conjuntos ou multiconjuntos de registros). O primeiro problema que pode ocorrer é que os *tipos de dados da linguagem de programação* diferem dos *tipos de dados de atributo* que estão disponíveis no modelo de dados. Logo, é necessário ter um **vínculo** para cada linguagem de programação hospedeira que especifica, para cada tipo de atributo, os tipos de linguagem de programação compatíveis. Um vínculo diferente é necessário *para cada linguagem de programação* , pois diferentes linguagens possuem diversos tipos de dados. Por exemplo, os tipos de dados disponíveis em C/C++ e Java são diferentes, e ambos diferem dos tipos de dados SQL, que são os tipos de dados padrão para bancos de dados relacionais.

Outro problema ocorre porque os resultados da maioria das consultas são conjuntos ou multiconjuntos de tuplas (linhas), e cada tupla é formada por uma sequência de valores de atributo. No programa, normalmente é necessário acessar os valores de dados individuais nas tuplas individuais para impressão ou processamento. Logo, é preciso que haja um vínculo para mapear a *estrutura de dados do resultado da consulta*, que é uma tabela, para uma estrutura de dados apropriada na linguagem de programação. É necessário que haja um mecanismo para percorrer as tuplas em um **resultado de consulta** a fim de acessar uma única tupla de cada vez e extrair valores individuais dela. Os valores de atributo extraídos costumam ser copiados para as variáveis de programa apropriadas para que o programa continue processando. Um **cursor** ou **variável de iteração** normalmente é usado para percorrer as tuplas em um resultado de consulta. Os valores individuais dentro de cada tupla são então extraídos para variáveis de programa distintas do tipo apropriado.

A divergência de impedância é um problema menor quando uma linguagem de programação de banco de dados especial é projetada para usar o mesmo modelo e tipos de dados do banco de dados. Um exemplo dessa linguagem é a PL/SQL da Oracle. O padrão SQL também tem uma proposta para tal linguagem de programação de banco de dados, conhecida como *SQL/PSM*. Para bancos de dados de objeto, o modelo de dados de objeto (ver Capítulo 11) é muito semelhante ao modelo de dados da linguagem de programação Java, de modo que a divergência de impedância é bastante reduzida quando Java é usada como linguagem hospedeira para acessar um banco de dados de objeto compatível com ela. Diversas linguagens de programação de banco de dados foram implementadas como protótipos de pesquisa (ver a bibliografia selecionada).

13.1.3 Sequência de interação típica na programação de banco de dados

Quando um programador ou engenheiro de software escreve um programa que exige acesso a um banco de dados, é muito comum que o programa esteja rodando em um sistema de computador enquanto o banco de dados é instalado em outro. Lembre-se, da Seção 2.5, de que uma arquitetura comum para o acesso ao banco de dados é o modelo cliente/servidor,

no qual um **programa cliente** trata da lógica de uma aplicação de software, mas inclui algumas chamadas para um ou mais **servidores de banco de dados** para acessar ou atualizar os dados.[2] Ao escrever tal programa, uma sequência comum de interação é a seguinte:

1. Quando o programa cliente requer acesso a determinado banco de dados, o programa precisa primeiro *estabelecer* ou *abrir* uma **conexão** com o servidor de banco de dados. Normalmente, isso ambolve especificar o endereço da Internet (URL) da máquina onde o servidor de banco de dados está localizado, além de fornecer um nome de conta de login e senha para o acesso ao banco de dados.
2. Quando a conexão é estabelecida, o programa pode interagir com o banco de dados submetendo consultas, atualizações e outros comandos do banco de dados. Em geral, a maioria dos tipos de instruções SQL pode ser incluída em um programa de aplicação.
3. Quando o programa não precisar mais acessar determinado banco de dados, ele deverá *terminar* ou *fechar* essa conexão.

Um programa pode acessar vários bancos de dados, se for preciso. Em algumas técnicas de programação de banco de dados, somente uma conexão pode estar ativa de uma só vez, enquanto em outras, várias conexões podem ser estabelecidas simultaneamente.

Nas próximas três seções, vamos discutir exemplos de cada uma das três principais técnicas de programação de banco de dados. A Seção 13.2 descreve como a SQL é *embutida* em uma linguagem de programação. A Seção 13.3 discute como as *chamadas de função* são usadas para acessar o banco de dados, e a Seção 13.4 discute uma extensão à SQL chamada SQL/PSM, que permite *construções de programação de uso geral* para definir módulos (procedimentos e funções) que são armazenados no sistema de banco de dados.[3] A Seção 13.5 compara essas técnicas.

13.2 SQL embutida, SQL dinâmica e SQLJ

Nesta seção, fornecemos uma visão geral da técnica que demonstra como as instruções SQL podem ser embutidas em uma linguagem de programação de uso geral. Focalizamos duas linguagens: C e Java. Os exemplos usados com a linguagem C, conhecida como **SQL embutida**, são apresentados nas seções 13.2.1 a 13.2.3, e podem ser adaptados a outras linguagens de programação. Os exemplos que usam Java, conhecidos como **SQLJ**, são apresentados nas seções 13.2.4 e 13.2.5. Nesta técnica embutida, a linguagem de programação é chamada de **linguagem hospedeira** (ou **host**). A maioria das instruções SQL — incluindo definições de dados ou restrições, consultas, atualizações ou definições de visão — pode ser embutida em um programa na linguagem hospedeira.

13.2.1 Recuperando tuplas isoladas com SQL embutida

Para ilustrar os conceitos da SQL embutida, usaremos C como linguagem de programação hospedeira.[4] Ao usar C dessa maneira, uma instrução SQL embutida é distinguida das instruções da linguagem de programação pelas palavras-chave de prefixo EXEC SQL, de modo que um **pré-processador** (ou **pré-compilador**) possa separar as instruções SQL embutidas do código da linguagem hospedeira. As instruções SQL em um programa terminam com um END-EXEC correspondente ou com um ponto e vírgula (;). Regras semelhantes se aplicam à SQL embutida em outras linguagens de programação.

Em um comando SQL embutido, podemos nos referir a variáveis de programação C especialmente declaradas. Estas são chamadas de **variáveis compartilhadas** porque são usadas tanto no programa C quanto nas instruções SQL embutidas. As variáveis compartilhadas são iniciadas com um sinal de dois pontos (:) *quando aparecem em uma instrução SQL*. Isso distingue os nomes de variável do programa dos nomes das construções do esquema de banco de dados, como atributos (nomes de coluna) e relações (nomes de tabela). Isso também permite que as variáveis do programa tenham os mesmos nomes que os atributos, pois podem ser distinguidos pelo sinal de dois pontos de prefixo na instrução SQL. Os nomes de construções do esquema de banco de dados — como atributos e relações — só podem ser usados nos comandos SQL, mas as variáveis de programa compartilhadas podem ser usadas em qualquer lugar no programa C sem o prefixo de dois pontos.

Suponha que queiramos escrever programas C para processar o banco de dados FUNCRESA da Figura 3.5. Precisamos declarar variáveis de programa

[2] Conforme discutimos na Seção 2.5, existem arquiteturas de duas e três camadas; para simplificar nossa discussão, vamos considerar aqui uma arquitetura cliente/servidor de duas camadas.

[3] A SQL/PSM ilustra como as construções típicas da linguagem de programação de uso geral — como loops e estruturas condicionais — podem ser incorporadas à SQL.

[4] Nossa discussão aqui também se aplica à linguagem de programação C++, pois não usamos nenhum dos recursos orientados a objeto, mas focalizamos o mecanismo de programação de banco de dados.

que correspondam aos tipos dos atributos do banco de dados que o programa processará. O programador pode escolher os nomes das variáveis de programa, que podem ou não ter nomes idênticos a seus atributos correspondentes no banco de dados. Usaremos as variáveis de programa C declaradas na Figura 13.1 para todos os nossos exemplos, mostrando os segmentos de programa C sem declarações de variável. As variáveis compartilhadas são declaradas em uma seção de declaração no programa, como mostra a Figura 13.1 (linhas 1 a 7).[5] Alguns dos vínculos comuns dos tipos SQL C entre os tipos SQL INTEGER, SMALLINT, REAL e DOUBLE são mapeados para os tipos C long, short, float e double, respectivamente. Strings de tamanho fixo e de tamanho variável (CHAR[*i*], VARCHAR[*i*]) em SQL podem ser mapeados para *vetores* de caracteres (char [i+1], varchar [i+1]) em C, que possuem um caractere a mais que o tipo SQL, pois as strings em C terminam com um caractere NULL (\0), que não faz parte da string de caracteres em si.[6] Embora varchar não seja um tipo de dado C padrão, ele é permitido quando C é usada para a programação de banco de dados SQL.

Observe que os únicos comandos SQL embutidos na Figura 13.1 são as linhas 1 e 7, que dizem ao pré-compilador para atentar para os nomes de variável C entre BEGIN DECLARE e END DECLARE, pois podem ser incluídos em instruções SQL embutidas — desde que precedidas por um sinal de dois pontos (:). As linhas 2 a 5 são declarações normais de programa C. As variáveis de programa C declaradas nas linhas 2 a 5 correspondem aos atributos das tabelas FUNCIONARIO e DEPARTAMENTO do banco de dados EMPRESA da Figura 3.5, que foi declarado pela SQL DDL na Figura 4.1. As variáveis declaradas na linha 6 — SQLCODE e SQLSTATE — são usadas para comunicar erros e condições de exceção entre o sistema de banco de dados e o programa em execução. Alinha 0 mostra uma variável de programa loop que não será usada em qualquer instrução SQL embutida, de modo que é declarada fora da seção de declaração da SQL.

```
0) int loop ;
1) EXEC SQL BEGIN DECLARE SECTION ;
2) varchar dnrme [16], pnome [16],
   unome [16], endereco [31] ;
3) char cpf [10], datanasc [11], sex [2], minicial [2] ;
4) float salario, aumento ;
5) int dno, dnumero ;
6) int SQLCODE ; char SQLSTATE [6] ;
7) EXEC SQL END DECLARE SECTION ;
```

Figura 13.1
Variáveis do programa C utilizadas nos exemplos E1 e E2 da SQL embutida.

Conectando ao banco de dados. O comando SQL para estabelecer uma conexão com um banco de dados tem a seguinte forma:

CONNECT TO <nome do servidor> **AS** <nome da conexão> **AUTHORIZATION** <nome de conta do usuário e senha> ;

Em geral, como um usuário ou programa podem acessar vários servidores de banco de dados, diversas conexões podem ser estabelecidas, mas somente uma pode estar ativa em qualquer ponto no tempo. O programador ou usuário podem usar o <nome da conexão> para mudar da conexão atualmente ativa para uma diferente utilizando o comando a seguir:

SET CONNECTION <nome da conexão> ;

Quando uma conexão não é mais necessária, ela pode ser terminada pelo seguinte comando:

DISCONNECT <nome da conexão> ;

Nos exemplos deste capítulo, consideramos que a conexão apropriada já foi estabelecida com o banco de dados FUNCRESA e que ela é a conexão atualmente ativa.

Comunicação entre o programa e o SGBD usando SQLCODE e SQLSTATE. As duas **variáveis de comunicação** especiais que são usadas pelo SGBD para comunicar condições de exceção ou erro ao programa são SQLCODE e SQLSTATE. A variável **SQLCODE**, mostrada na Figura 13.1, é uma variável inteira. Após cada comando do banco de dados ser executado, o SGBD retorna um valor em SQLCODE. Um valor 0 indica que a instrução foi executada com sucesso pelo SGBD. Se SQLCODE > 0 (ou, mais especificamente, se SQLCODE = 100), isso indica que não há mais dados (registros) disponíveis em um resultado de consulta. Se SQLCODE< 0, isso indica que houve algum erro. Em alguns sistemas — por exemplo, no SGBDR da Oracle —, SQLCODE é um campo em uma estrutura de registro chamada SQLCA (SQL Communication Area), de modo que é referenciado como SQLCA.SQLCODE. Nesse caso, a definição da SQLCA precisa ser incluída no programa C utilizando a seguintelinha:

EXEC SQL include SQLCA ;

[5] Usamos números de linha em nossos segmentos de código apenas para facilitar a referência; esses números não fazem parte do código real.
[6] Strings SQL também podem ser mapeadas para tipos char* em C.

Em versões mais recentes do padrão SQL, uma variável de comunicação chamada SQLSTATE foi acrescentada, que é uma string de cinco caracteres. Um valor '00000' em SQLSTATE indica nenhum erro ou exceção; outros valores indicam diversos erros ou exceções. Por exemplo, '02000' indica 'sem mais dados' quando se usa SQLSTATE. Atualmente, tanto SQLSTATE quanto SQLCODE estão disponíveis no padrão SQL. Muitos dos códigos de erro e exceção retornados em SQLSTATE supostamente estão padronizados para todos os vendedores e plataformas SQL,[7] enquanto os códigos retornados em SQLCODE não estão padronizados, mas são definidos pelo vendedor do SGBD. Logo, em geral é melhor usar SQLSTATE, pois isso torna o tratamento de erro nos programas de aplicação independente de um SGBD em particular. Como um exercício, o leitor deverá reescrever os exemplos dados mais adiante neste capítulo usando SQLSTATE em vez de SQLCODE.

Exemplo de programação SQL embutida. Nosso primeiro exemplo para ilustrar a programação SQL embutida é um segmento repetitivo (loop) do programa que recupera como entrada o número do Cpf de um funcionário e imprime algumas informações com base no registro de FUNCIONARIO correspondente no banco de dados. O código de programa em C aparece como o segmento de programa E1 na Figura 13.2. O programa lê (entradas) um valor de Cpf e depois recupera a tupla de FUNCIONARIO com esse Cpf do banco de dados por meio do comando SQL embutido. A cláusula INTO (linha 5) especifica as variáveis do programa em que os valores de atributo do registro de banco de dados são recuperados. As variáveis do programa em C na cláusula INTO são iniciadas com um sinal de dois pontos (:), conforme discutimos anteriormente. A cláusula INTO só pode ser usada desse modo quando o resultado da consulta é um único registro; se vários registros forem recuperados, será gerado um erro. Veremos como múltiplos registros são tratados na Seção 13.2.2.

A linha 7 em E1 ilustra a comunicação entre o banco de dados e o programa por meio da variável especial SQLCODE. Se o valor retornado pelo SGBD em SQLCODE for 0, a instrução anterior foi executada sem erros ou condições de exceção. A linha 7 verifica isso e assume que, se ocorreu um erro, foi porque não existia nenhuma tupla FUNCIONARIO com o Cpf dado; portanto, ela gera uma mensagem de saída indicando isso (linha 8).

Em E1, um *único registro* é selecionado pela consulta SQL embutida (porque Cpf é um atributo de chave de FUNCIONARIO). Quando um único registro é recuperado, o programador pode designar seus valores de atributo diretamente às variáveis do programa em C na cláusula INTO, como na linha 5. Em geral, uma consulta SQL pode recuperar muitas tuplas. Nesse caso, o programa em C costuma percorrer as tuplas recuperadas e as processa uma de cada vez. O conceito de um *cursor* é usado para permitir o processamento de uma tupla por vez no resultado de uma consulta pelo programa da linguagem hospedeira. A seguir, vamos descrever os cursores.

13.2.2 Recuperando múltiplas tuplas com SQL embutida usando cursores

Podemos imaginar um **cursor** como um ponteiro que aponta para uma *única tupla* (*linha*) do resultado de uma consulta que recupera múltiplas tuplas. O cursor é declarado quando o comando de consulta

```
   //Segmento de programa E1:
0)  loop = 1 ;
1)  while (loop) {
2)     prompt("Digite um CPF: ", cpf) ;
3)     EXEC SQL
4)          select Pnome, Minicial, Unome, Endereco, Salario
5)          into :pnome, :minicial, :unome, :endereco, :salario
6)          from FUNCIONARIO where Cpf = :cpf ;
7)     if (SQLCODE == 0) printf(pnome, minicial, unome, endereco, salario)
8)          else printf("CPF não existe: ", cpf) ;
9)     prompt("MaisCPF (digite 1 para Sim, 0 para Não): ", loop) ;
10)    }
```

Figura 13.2
Segmento de programa E1, um segmento de programa em C com SQL embutida.

[7] Em particular, códigos de SQLSTATE começando com os caracteres 0 a 4 ou A a H supostamente são padronizados, enquanto outros valores podem ser definidos pela implementação.

SQL é declarado no programa. Mais adiante no programa, um comando **OPEN CURSOR** busca o resultado da consulta no banco de dados e define o cursor para uma posição *antes da primeira linha* no resultado da consulta. Esta se torna a **linha atual** para o cursor. Depois, comandos **FETCH** são emitidos no programa. Cada FETCH move o cursor para a *próxima linha* no resultado da consulta, tornando-a a linha ativa e copiando seus valores de atributo para as variáveis do programa em C (linguagem hospedeira) especificadas no comando FETCH por uma cláusula INTO. A variável do cursor é basicamente um **iterador** (iterator) que percorre as (por loop) tuplas no resultado da consulta — uma tupla de cada vez.

Para determinar quanto todas as tuplas no resultado da consulta foram processadas, a variável de comunicação SQLCODE (ou, como alternativa, SQLSTATE) é verificada. Se um comando FETCH for emitido e resultar na movimentação do cursor além da última tupla no resultado da consulta, um valor positivo (SQLCODE > 0) é retornado em SQLCODE, indicando que nenhum dado (tupla) foi encontrado (ou a string '02000' é retornada em SQLSTATE). O programador usa isso para terminar um loop sobre tuplas no resultado da consulta. Em geral, diversos cursores podem ser abertos ao mesmo tempo. Um comando **CLOSE CURSOR** é emitido para indicar que terminamos com o processamento do resultado da consulta associada a esse cursor.

Um exemplo de uso de cursores para processar um resultado de consulta com múltiplos registros é mostrado na Figura 13.3, onde um cursor chamado FUNC é declarado na linha 4. O cursor FUNC é associado à consulta SQL declarada nas linhas 5 a 6, mas a consulta não é executada até que o comando OPEN FUNC (linha 8) seja processado. O comando OPEN <nome cursor> executa a consulta e busca seu resultado como uma tabela no workspace do programa, onde o programa pode percorrer as linhas (tuplas) individuais por comandos FETCH <nome cursor> subsequentes (linha 9). Considera-se que as variáveis apropriadas no programa em C foram declaradas, como na Figura 13.1. O segmento de programa em E2 lê (entrada) um nome de departamento (linha 0), recupera o número de departamento correspondente do banco de dados (linhas 1 a 3) e depois recupera os funcionários que trabalham nesse departamento por meio do cursor FUNC. Um loop (linhas 10 a 18) passa por cada registro no resultado da consulta, um de cada vez, e imprime o nome do funcionário. O programa então lê (entrada) um valor de aumento para esse funcionário (linha 12) e atualiza o salário dele no banco de dados pelo valor do aumento que foi oferecido (linhas 14 a 16).

Este exemplo também ilustra como o programador pode *atualizar* registros do banco de dados. Quando um cursor é definido para linhas que devem ser modificadas (**atualizadas**), temos de acrescentar a

```
    //Segmento de programa E2:
0)  prompt("Digite o Nome do Departamento: ", dnome) ;
1)  EXEC SQL
2)      select Dnumero into :dnumero
3)      from DEPARTAMENTO where Dnome = :dnome ;
4)  EXEC SQL DECLARE FUNC CURSOR FOR
5)      select Cpf, Pnome, Minicial, Unome, Salario
6)      from FUNCIONARIO where Dnr = :dnumero
7)      FOR UPDATE OF Salario ;
8)  EXEC SQL OPEN FUNC ;
9)  EXEC SQL FETCH from FUNC into :cpf, :pnome, :minicial, :unome, :salario ;
10) while (SQLCODE == 0) {
11)     printf("O nome do funcionario é:", Pnome, Minicial, Unome) ;
12)     prompt("digite o valor de aumento: ", aumento) ;
13)     EXEC SQL
14)         update FUNCIONARIO
15)         set Salario = Salario + :aumento
16)         where CURRENT OF FUNC ;
17)     EXEC SQL FETCH from FUNC into :cpf, :pnome, :minicial, :unome, :salario ;
18)     }
19) EXEC SQL CLOSE FUNC ;
```

Figura 13.3
Segmento de programa E2, um segmento de programa em C que usa cursores com SQL embutida para fins de atualização.

cláusula **FOR UPDATE OF** na declaração do cursor e listar os nomes de quaisquer atributos que serão atualizados pelo programa. Isso é ilustrado na linha 7 do segmento de código E2. Se as linhas tiverem de ser **excluídas**, as palavras-chave **FOR UPDATE** devem ser acrescentadas sem especificar quaisquer atributos. No comando embutido UPDATE (ou DELETE), a condição **WHERE CURRENT OF** <nome cursor> especifica que a tupla atual referenciada pelo cursor é aquela a ser atualizada (ou excluída), como na linha 16 de E2.

Observe que declarar um cursor e associá-lo a uma consulta (linhas 4 a 7 em E2) não executa a consulta. A consulta é realizada somente quando o comando OPEN <nome cursor> (linha 8) é executado. Observe também que não é preciso incluir a cláusula FOR UPDATE OF na linha 7 de E2 se os resultados da consulta tiverem de ser usados *apenas para fins de recuperação* (sem atualização ou exclusão).

Opções gerais para uma declaração de cursor. Várias opções podem ser especificadas quando se declara um cursor. O formato geral de uma declaração de cursor é o seguinte:

DECLARE <nome cursor> [**INSENSITIVE**] [**SCROLL**] **CURSOR** [**WITH HOLD**] **FOR** <especificação da consulta>

[**ORDER BY** <especificação de ordenação>] [**FOR READ ONLY** | **FOR UPDATE** [**OF** <lista de atributos>]] ;

Já discutimos rapidamente as opções listadas na última linha. O padrão é que a consulta seja para fins de recuperação (FOR READ ONLY). Se algumas das tuplas no resultado da consulta tiverem de ser atualizadas, precisamos especificar FOR UPDATE OF <lista atributos> e listar os atributos que podem ser atualizados. Se algumas tuplas tiverem de ser excluídas, precisamos especificar FOR UPDATE sem quaisquer atributos.

Quando a palavra-chave opcional SCROLL é especificada em uma declaração de cursor, é possível posicionar o cursor de outras maneiras além de simplesmente para acesso sequencial. Uma **orientação de busca** pode ser acrescentada ao comando FETCH, cujo valor pode ser NEXT, PRIOR, FIRST, LAST, ABSOLUTE *i* e RELATIVE *i*. Nos dois últimos comandos, *i* precisa ser avaliado como um valor inteiro que especifica uma posição de tupla absoluta no resultado da consulta (para ABSOLUTE *i*) ou uma posição de tupla relativa à posição atual do cursor (para RELATIVE *i*). A orientação de busca padrão, que usamos em nossos exemplos, é NEXT. A orientação de busca permite que o programador movimente o cursor pelas tuplas no resultado da consulta com maior flexibilidade, oferecendo acesso aleatório por posição ou acesso na ordem inversa. Quando SCROLL é especificado no cursor, o formato geral de um comando FETCH é o seguinte, com as partes entre colchetes sendo opcionais:

FETCH [[<orientação de busca>] **FROM**] <nome do cursor> **INTO** <lista de destino da busca> ;

A cláusula ORDER BY ordena as tuplas de modo que o comando FETCH as buscará na ordem especificada. Ela é determinada de modo semelhante à cláusula correspondente para consultas SQL (ver Seção 4.3.6). As duas últimas opções quando se declara um cursor (INSENSITIVE e WITH HOLD) referem-se a características de transação dos programas de banco de dados, que discutiremos no Capítulo 21.

13.2.3 Especificando consultas em tempo de execução usando a SQL dinâmica

Nos exemplos anteriores, as consultas SQL embutidas foram escritas como parte do código fonte do programa hospedeiro. Logo, quando quisermos escrever uma consulta diferente, temos de modificar o código do programa e passar por todas as etapas envolvidas (compilação, depuração, teste etc.). Em alguns casos, é conveniente escrever um programa que possa executar diferentes consultas ou atualizações SQL (ou outras operações) *dinamicamente em tempo de execução*. Por exemplo, podemos querer escrever um programa que aceite uma consulta SQL digitada pelo terminal, execute-a e apresente seu resultado, como as interfaces interativas disponíveis para a maioria dos SGBDs relacionais. Outro exemplo é quando uma interface de fácil utilização gera consultas SQL de maneira dinâmica para o usuário com base em operações do tipo apontar e clicar em um esquema gráfico (por exemplo, uma interface tipo QBE; ver Apêndice C). Nesta seção, fazemos uma rápida visão geral da **SQL dinâmica**, que é uma técnica para escrever esse tipo de programa de banco de dados, dando um exemplo simples para ilustrar como essa linguagem pode funcionar. Na Seção 13.3, descreveremos outra técnica para lidar com consultas dinâmicas.

O segmento de programa E3 da Figura 13.4 lê uma string que é inserida pelo usuário (essa string poderia ser um comando de atualização SQL) para a variável de string do programa sqlupdatestring na linha 3. Depois, ele prepara isso como um comando SQL na linha 4, associando-o à variável SQL sqlcommand.

```
    //Segmento de programa E3:
0)  EXEC SQL BEGIN DECLARE SECTION ;
1)  varchar sqlupdatestring [256] ;
2)  EXEC SQL END DECLARE SECTION ;
    ...
3)  prompt("Digite o comando Update: ",
    sqlupdatestring) ;
4)  EXEC SQL PREPARE sqlcommand FROM
    :sqlupdatestring ;
5)  EXEC SQL EXECUTE sqlcommand ;
    ...
```

Figura 13.4
Segmento de programa E3, um segmento de programa em C que usa a SQL dinâmica para atualizar uma tabela.

A linha 5 então executa o comando. Observe que, nesse caso, nenhuma verificação de sintaxe ou outros tipos de verificações sobre o comando são possíveis *em tempo de compilação*, pois o comando SQL não está disponível em tempo de execução. Isso contrasta com nossos exemplos anteriores de SQL embutida, em que a consulta podia ser verificada em tempo de compilação, pois seu texto estava no código fonte do programa.

Embora a inclusão de um comando de atualização dinâmica seja relativamente simples na SQL dinâmica, uma consulta dinâmica é muito mais complicada. Isso porque em geral não conhecemos os tipos ou o número de atributos a serem recuperados pela consulta SQL quando estamos escrevendo o programa. Uma estrutura de dados complexa às vezes é necessária para permitir diferentes números e tipos de atributos no resultado da consulta se nenhuma informação anterior for conhecida sobre a consulta dinâmica. Técnicas semelhantes às que discutimos na Seção 13.3 podem ser usadas para atribuir resultados da consulta (e parâmetros da consulta) às variáveis do programa hospedeiro.

Em E3, o motivo para separar PREPARE e EXECUTE é que, se o comando tiver de ser executado várias vezes em um programa, ele pode ser preparado apenas uma vez. A preparação do comando costuma ambolver sintaxe e outros tipos de verificações pelo sistema, bem como a geração do código para executá-lo. É possível combinar os comandos PREPARE e EXECUTE (linhas 4 e 5 em E3) em um único comando ao escrever

```
EXEC SQL EXECUTE IMMEDIATE :sqlupdatestring ;
```

Isso é útil se o comando tiver de ser executado apenas uma vez. Como alternativa, o programador pode separar as duas instruções para recuperar quaisquer erros após a instrução PREPARE, se houver algum.

13.2.4 SQLJ: embutindo comandos SQL em Java

Nas subseções anteriores, demos uma ideia de como os comandos SQL podem ser embutidos em uma linguagem de programação tradicional, usando a linguagem C em nossos exemplos. Agora, voltamos nossa atenção para como a SQL pode ser embutida em uma linguagem de programação orientada a objeto,[8] em particular, a linguagem Java. A SQLJ é um padrão que foi adotado por diversos vendedores para embutir SQL em Java. Historicamente, a SQLJ foi desambolvida após a JDBC, a qual é usada para acessar bancos de dados SQL com a linguagem Java usando chamadas de função. Vamos discutir a JDBC na Seção 13.3.2. Nesta seção, focamos em SQLJ e como ela é usada no SGBDR Oracle. Um tradutor de SQLJ geralmente converterá comandos SQL para Java, que poderão então ser executados por meio da interface JDBC. Logo, é necessário instalar um *driver JDBC* ao usar a SQLJ.[9] Nesta seção, mostramos como usar conceitos de SQLJ para escrever SQL embutida em um programa Java.

Antes de ser capaz de processar SQLJ com Java em Oracle, é necessário importar várias bibliotecas de classe, mostradas na Figura 13.5. Estas incluem as classes JDBC e IO (linhas 1 e 2), mais as classes adicionais listadas nas linhas 3, 4 e 5. Além disso, o programa precisa primeiro se conectar ao banco de dados desejado usando a chamada de função getConnection, que é um dos métodos da classe oracle da linha 5 da Figura 13.5. O formato dessa chamada de função, que retorna um objeto do tipo *contexto default*,[10] é o seguinte:

```
public static DefaultContext getConnection
(String url,String user, String password,
    Boolean autoCommit) throws SQLException ;
```

Por exemplo, podemos escrever as instruções nas linhas 6 a 8 da Figura 13.5 para conectar a um banco de dados Oracle localizado no url <nome url> usando

[8] Esta seção assume alguma familiaridade com conceitos orientados a objeto (ver Capítulo 11) e conceitos básicos de Java.

[9] Discutiremos sobre drivers JDBC na Seção 13.3.2.

[10] Um *contexto default*, quando definido, se aplica a comandos subsequentes no programa até que ele seja mudado.

```
1) import java.sql.* ;
2) import java.io.* ;
3) import sqlj.runtime.* ;
4) import sqlj.runtime.ref.* ;
5) import oracle.sqlj.runtime.* ;
   …
6) DefaultContext cntxt =
7) oracle.getConnection("<urlname>", "<user name>",
   "<password>", true) ;
8) DefaultContext.setDefaultContext(cntxt) ;
   …
```

Figura 13.5
Importando classes necessárias para incluir SQLJ em programas Java no Oracle e estabelecendo uma conexão e um contexto default.

o login de <nome usuário> e <senha> com confirmação automática de cada comando,[11] e depois definir essa conexão como o **contexto default** para comandos seguintes.

Nos exemplos a seguir, não mostraremos as classes ou programas Java completos, pois não é nossa intenção ensinar Java. Em vez disso, mostraremos segmentos de programa que ilustram o uso da SQLJ. A Figura 13.6 mostra as variáveis de programa Java usadas em nossos exemplos. O segmento de programa J1 na Figura 13.7 lê o Cpf de um funcionário e imprime algumas das informações do funcionário do banco de dados.

Observe que, como Java já usa o conceito de **exceções** para tratamento de erro, uma exceção especial, chamada SQLException, é utilizada para

```
1) string dnome, cpf, pnome, pn, unome, un,
   datanasc, endereco;
2) char sexo, minicial, mi ;
3) double salario, sal ;
4) integer dnr, dnumero ;
```

Figura 13.6
Variáveis de programa Java usadas nos exemplos de J1 e J2 de SQLJ.

retornar erros ou condições de exceção depois de executar um comando de banco de dados SQL. Isso desempenha um papel semelhante a SQLCODE e SQLSTATE na SQL embutida. A Java tem muitos tipos de exceções predefinidas. Cada operação (função) Java deve especificar as exceções que podem ser **lançadas** — ou seja, as condições de exceção que podem ocorrer enquanto se executa o código Java dessa operação. Se ocorrer uma exceção definida, o sistema transfere o controle ao código Java especificado para tratamento da exceção. Em J1, o tratamento da exceção para uma SQLException é especificado nas linhas 7 e 8. Em Java, a estrutura a seguir

```
try {<operacao>} catch (<excecao>) {<codigo de
tratamento de excecao>} <continuacao de codigo>
```

é usada para lidar com exceções que ocorrem durante a execução da <operacao>. Se não houver exceção, o <continuacao de codigo> é processado diretamente. As exceções que podem ser lançadas pelo código em determinada operação devem ser especificadas como parte da declaração da operação ou *interface* — por exemplo, no formato a seguir:

```
//Segmento de programa J1:
1)  cpf = readEntry("Digite o número do CPF: ") ;
2)  try {
3)      #sql { select Pnome, Minicial, Unome, Endereco, Salario
4)          into :pnome, :minicial, :unome, :endereco, :salario
5)          from FUNCIONARIO where Cpf = :cpf} ;
6)  } catch (SQLException se) {
7)          System.out.println("Número do CPF não existe: " + cpf) ;
8)          Return ;
9)      }
10) System.out.println(pnome + " " + minicial + " " + unome + " " + endereco + " " + salario)
```

Figura 13.7
Segmento de programa J1, um segmento de programa Java com SQLJ.

[11] *Confirmação automática* significa mais ou menos que cada comando é aplicado ao banco de dados depois de ser executado. A alternativa é que o programador queira executar vários comandos de banco de dados relacionados e, depois, os confirme juntos. Discutiremos conceitos de confirmação (*commit*) no Capítulo 21, quando descrevermos transações do banco de dados.

<tipo de retorno de operacao> <nome da operacao> (<parametros>) throws SQLException, IOException ;

Em SQLJ, os comandos SQL embutidos em um programa Java são precedidos por #sql, conforme ilustrado na linha 3 de J1, de modo que possam ser identificados pelo pré-processador. O #sql é usado no lugar das palavras-chave EXEC SQL que são utilizadas na SQL embutida com a linguagem de programação C (ver Seção 13.2.1). A SQLJ usa uma *cláusula* INTO — semelhante àquela da SQL embutida — para retornar os valores de atributo recuperados do banco de dados por uma consulta SQL em variáveis de programa Java. As variáveis de programa são precedidas por sinais de dois pontos (:) na instrução SQL, assim como na SQL embutida.

Em J1, uma *única tupla* é recuperada pela consulta SQLJ embutida. É por isso que podemos atribuir seus valores de atributo diretamente a variáveis do programa Java na cláusula INTO, na linha 4 da Figura 13.7. Para consultas que recuperam muitas tuplas, a SQLJ usa o conceito de um *iterador* (iterator), que é semelhante a um cursor na SQL embutida.

13.2.5 Recuperando múltiplas tuplas em SQLJ usando iteradores

Em SQLJ, um **iterador** é um tipo de objeto associado a uma coleção (conjunto ou multiconjunto) de registros em um resultado de consulta.[12] O iterador está associado às tuplas e atributos que aparecem em um resultado de consulta. Existem dois tipos de iteradores:

1. Um **iterador nomeado** é associado a um resultado de consulta ao listar os *nomes e tipos* de atributo que aparecem no resultado dela. Os nomes de atributo devem corresponder a variáveis de programa Java apropriadamente declarados, como mostra a Figura 13.6.
2. Um **iterador posicional** lista apenas os *tipos de atributo* que aparecem no resultado da consulta.

Nos dois casos, a lista deveria estar *na mesma ordem* dos atributos que são listados na cláusula SELECT da consulta. No entanto, o looping sobre um resultado de consulta é diferente para os dois tipos de iteradores, conforme veremos. Primeiro, mostramos um exemplo de uso de um iterador *nomeado* na Figura 13.8, segmento de programa J2A. A linha 9 na Figura 13.8 mostra como um *tipo de iterador nomeado* Func é declarado. Observe que os nomes dos atributos em um tipo de iterador nomeado precisam combinar com os nomes dos atributos no resultado da consulta SQL. A linha 10 mostra como um *objeto iterador* e do tipo Func é criado no programa e depois associado a uma consulta (linhas 11 e 12).

Quando o objeto iterador é associado a uma consulta (linhas 11 e 12 da Figura 13.8), o programa busca o resultado da consulta do banco de dados e define o iterador para uma posição *antes da primeira linha* no resultado da consulta. Esta torna-se a **linha atual** para o iterador. Subsequentemente, operações **next** são emitidas sobre o objeto iterador. Cada next move o iterador para a *próxima linha* no resultado da consulta, tornando-a a linha atual. Se a linha existir, a operação recupera os valores de atributo para essa linha nas variáveis de programa correspondente. Se não houver mais linhas, a operação next retorna NULL, e pode assim ser usada para controlar o looping. Observe que o iterador nomeado não precisa de uma cláusula INTO, pois as variáveis do programa correspondentes aos atributos recuperados já estão especificadas quando o tipo iterador é declarado (linha 9 da Figura 13.8).

Na Figura 13.8, o comando (e.next( )) na linha 13 realiza duas funções: ele recupera a próxima tupla no resultado da consulta e controla o loop while. Quando o programa termina o processamento do resultado da consulta, o comando e.close( ) (linha 16) fecha o iterador.

A seguir, considere o mesmo exemplo usando iteradores *posicionais*, como mostra a Figura 13.9 (segmento de programa J2B). A linha 9 da Figura 13.9 mostra como um *tipo iterador posicional* Funcpos é declarado. A principal diferença entre ele e o iterador nomeado é que não existem nomes de atributo (correspondentes a nomes de variável de programa) no iterador posicional — apenas tipos de atributo. Isso pode oferecer mais flexibilidade, mas torna o processamento do resultado da consulta ligeiramente mais complexo. Os tipos de atributo ainda devem ser compatíveis com os tipos de atributo no resultado da consulta SQL e na mesma ordem. A linha 10 mostra como um *objeto iterador posicional* f do tipo Funcpos é criado no programa e depois associado a uma consulta (linhas 11 e 12).

O iterador posicional se comporta de uma maneira mais parecida com a SQL embutida (ver Seção 13.2.2). Um comando FETCH **<variável de iteração>** INTO **<variáveis do programa>** é necessário para colocar a próxima tupla em um resultado da consulta. Na primeira vez em que fetch é executado, ele recupera a primeira tupla (linha 13 na Figura 13.9). A

[12] Discutimos sobre iteradores com mais detalhes no Capítulo 11, quando apresentamos os conceitos de banco de dados de objeto.

```
    //Segmento de programa J2A:
0)  dnome = readEntry("Digite o nome do departamento: ") ;
1)  try {
2)      #sql { select Dnumero into :dnumero
3)          from DEPARTAMENTO where Dnome = :dnome} ;
4)  } catch (SQLException se) {
5)      System.out.println("Departamento não existe: " + dnome) ;
6)      Return ;
7)      }
8)  System.out.println("Informação do funcionário para departamento: " + dnome) ;
9)  #sql iterator Func(String cpf, String pnome, String minicial, String unome, double salario) ;
10) Func f = null ;
11) #sql f = { select cpf, pnome, minicial, unome, salario
12)     from FUNCIONARIO where Dnr = :dnumero} ;
13) while (f.next( )) {
14)     System.out.println(f.cpf + " " + f.pnome + " " + f.minicial + " " + f.unome + " " + f.salario) ;
15) } ;
16) f.close( ) ;
```

Figura 13.8
Segmento de programa J2A, um segmento de programa Java que usa um iterador nomeado para imprimir informações de funcionário em determinado departamento.

```
    //Segmento de programa J2B:
0)  dnome = readEntry("Digite o nome do departamento: ") ;
1)  try {
2)      #sql { select Dnumero into :dnumero
3)          from DEPARTAMENTO where Dnome = :dnome} ;
4)  } catch (SQLException se) {
5)      System.out.println("Departamento não existe: " + dnrme) ;
6)      Return ;
7)      }
8)  System.out.println("Informação do funcionário para departamento: " + dnome) ;
9)  #sql iterator Funcpos(String, String, String, String, double) ;
10) Funcpos f = null ;
11) #sql e = { select cpf, pnome, minicial, unome, salario
12)     from FUNCIONARIO where Dnr = :dnumero} ;
13) #sql { fetch :f into :cpf, :pn, :mi, :un, :sal} ;
14) while (!f.endFetch( )) {
15)     System.out.println(cpf + " " + pn + " " + mi + " " + un + " " + sal) ;
16)     #sql { fetch :f into :cpf, :pn, :mi, :un, :sal} ;
17) } ;
18) f.close( ) ;
```

Figura 13.9
Segmento de programa J2B, um segmento de programa em Java que usa um iterador posicional para imprimir informações de funcionário em determinado departamento.

linha 16 recupera a próxima tupla até que não haja mais tuplas no resultado da consulta. Para controlar o loop, uma função de iterador posicional, `f.endFetch( )`, é utilizada. Essa função é definida para um valor TRUE quando o iterador é associado inicialmente a uma consulta SQL (linha 11), e é definida como FALSE toda vez que um comando de busca retorna uma tupla válida do resultado da consulta. Ela é definida

como TRUE novamente quando um comando de busca não encontra mais tuplas. A linha 14 mostra como o looping é controlado pela negação.

13.3 Programação de banco de dados com chamadas de função: SQL/CLI e JDBC

A SQL embutida (ver Seção 13.2) às vezes é chamada de técnica de programação de banco de dados **estática**, pois o texto da consulta é escrito no código fonte do programa e não pode ser alterado sem uma nova compilação ou reprocessamento do código fonte. O uso de chamadas de função é uma técnica mais **dinâmica** para programação de banco de dados do que a SQL embutida. Já vimos uma técnica de programação de banco de dados dinâmico — SQL dinâmica — na Seção 13.2.3. As técnicas discutidas aqui oferecem outro enfoque para a programação dinâmica de banco de dados. Uma **biblioteca de funções**, também conhecida como uma **interface de programação de aplicação** (**API**), é usada para acessar o banco de dados. Embora isso ofereça mais flexibilidade, pois nenhum pré-processador é necessário, uma desvantagem é que a sintaxe e outras verificações sobre comandos SQL precisam ser feitas em tempo de execução. Outra desvantagem é que isso às vezes requer uma programação mais complexa para acessar resultados da consulta, pois os tipos e números de atributos em um resultado de consulta podem não ser conhecidos previamente.

Nesta seção, damos uma visão geral de duas interfaces de chamada de função. Primeiro, discutimos a **SQL Call Level Interface** (**SQL/CLI**), que é parte do padrão SQL. Ela foi desambolvida como um complemento para a técnica mais antiga conhecida como ODBC (*Open Database Connectivity*). Usamos C como linguagem hospedeira em nossos exemplos de SQL/CLI. Depois, oferecemos uma visão geral da **JDBC**, que é a interface de chamada de função para acessar bancos de dados com Java. Embora normalmente se considere que JDBC signifique Java Database Connectivity, trata-se apenas de uma marca registrada da Sun Microsystems, *não* um acrônimo.

A principal vantagem do uso de uma interface de chamada de função é que ela facilita o acesso a múltiplos bancos de dados no mesmo programa de aplicação, mesmo que eles sejam armazenados sob diferentes pacotes de SGBD. Discutiremos isso melhor na Seção 13.3.2, quando abordaremos a programação de banco de dados Java com JDBC, embora essa vantagem também se aplique à programação de banco de dados com SQL/CLI e ODBC (ver Seção 13.3.1).

13.3.1 Programação de banco de dados com SQL/CLI usando C como linguagem hospedeira

Antes de usar as chamadas de função na SQL/CLI, é necessário instalar os pacotes de bibliotecas apropriados no servidor de banco de dados. Esses pacotes são obtidos com o vendedor do SGBD em uso. Agora, vamos apresentar uma visão geral de como a SQL/CLI pode ser usada em um programa em C.[13] Ilustraremos nossa apresentação com um exemplo de segmento de programa CLI1, mostrado na Figura 13.10.

Ao usar a SQL/CLI, as instruções SQL são criadas dinamicamente e passadas como *parâmetros de string* nas chamadas de função. Logo, é necessário registrar as informações sobre as interações do programa hospedeiro com o banco de dados nas estruturas de dados em tempo de execução, pois os comandos do banco de dados são processados em tempo de execução. A informação é mantida em quatro tipos de registros, representados como *structs* em tipos de dados C. Um **registro de ambiente** é usado como um recipiente para registrar uma ou mais conexões de banco de dados e definir informações de ambiente. Um **registro de conexão** registra as informações necessárias para determinada conexão de banco de dados. Um **registro de instrução** registra as informações necessárias para uma instrução SQL. Um **registro de descrição** registra as informações sobre tuplas ou parâmetros — por exemplo, o número de atributos e seus tipos em uma tupla, ou o número e os tipos de parâmetros em uma chamada de função. Isso é necessário quando o programador não conhece essa informação sobre a consulta ao escrever o programa. Em nossos exemplos, supomos que o programador conheça a consulta exata, de modo que não mostramos quaisquer registros de descrição.

Cada registro é acessível ao programa por meio de uma variável de ponteiro C chamada **identificador** (ou *handle*) do registro. O identificador é retornado quando um registro é criado inicialmente. Para criar um registro e retornar seu identificador, a seguinte função SQL é usada:

SQLAllocHandle(<tipo_identificador>, <id_1>,<id_2>)

[13] Nossa discussão aqui também se aplica à linguagem de programação C++, pois não usamos nenhum dos recursos orientados a objeto, mas focalizamos o mecanismo de programação de banco de dados.

```
//Programa CLI1:
0)  #include sqlcli.h ;
1)  void imprimeSalario( ) {
2)  SQLHSTMT inst1 ;
3)  SQLHDBC con1 ;
4)  SQLHENV amb1 ;
5)  SQLRETURN ret1, ret2, ret3, ret4 ;
6)  ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &amb1) ;
7)  if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, amb1, &con1) else exit ;
8)  if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz", SQL_NTS) else exit ;
9)  if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &inst1) else exit ;
10) SQLPrepare(inst1,"select Unome,Salario fromFUNCIONARIO whereCpf = ?", SQL_NTS);
11) prompt("Digite um número de CPF: ", cpf) ;
12) SQLBindParameter(inst1, 1, SQL_CHAR, &cpf, 9, &fetchlen1) ;
13) ret1 = SQLExecute(inst1) ;
14) if (!ret1) {
15)     SQLBindCol(inst1, 1, SQL_CHAR, &unome, 15, &fetchlen1) ;
16)     SQLBindCol(inst1, 2, SQL_FLOAT, &salario, 4, &fetchlen2) ;
17)     ret2 = SQLFetch(inst1) ;
18)     if (!ret2) printf(cpf, unome, salario)
19)         else printf("O número do CPF não existe: ", cpf) ;
20)     }
21) }
```

Figura 13.10
Segmento de programa CLI1, um segmento de programa em C com SQL/CLI.

Nessa função, os parâmetros são os seguintes:

- <tipo_identificador> indica o tipo do registro que está sendo criado. Os valores possíveis para esse parâmetro são as palavras-chave SQL_HANDLE_ENV, SQL_HANDLE_DBC, SQL_HANDLE_STMT ou SQL_HANDLE_DESC, para um registro de ambiente, conexão, instrução ou descrição, respectivamente.
- <id_1> indica o recipiente dentro do qual o novo identificador está sendo criado. Por exemplo, para um registro de conexão, este seria o ambiente no qual a conexão está sendo criada, e para um registro de instrução, este seria a conexão para essa instrução.
- <id_2> é o ponteiro (identificador) para o registro recém-criado do tipo <tipo_identificador>.

Ao escrever um programa em C que incluirá chamadas de banco de dados por meio da SQL/CLI, as seguintes etapas típicas são tomadas. Ilustramos as etapas referindo-nos ao exemplo CLI1 da Figura 13.10, que lê um número de Cpf de um funcionário e imprime seu sobrenome e salário.

1. A *biblioteca de funções* compreendendo SQL/CLI deve ser incluída no programa C. Esta é chamada da Figura 13.10: sqlcli.h, incluída usando a linha 0.
2. Declare *variáveis de identificador* dos tipos SQLHSTMT, SQLHDBC, SQLHENV e SQLHDESC para as instruções, conexões, ambientes e descrições necessárias no programa, respectivamente (linhas 2 a 4).[14] Também declare variáveis do tipo SQLRETURN (linha 5) para manter os códigos de retorno das chamadas de função da SQL/CLI. Um código de retorno 0 (zero) indica *execução bem-sucedida* da chamada de função.
3. Um *registro de ambiente* deve ser configurado no programa usando SQLAllocHandle. A função para fazer isso aparece na linha 6. Como um registro de ambiente não está contido em qualquer outro registro, o parâmetro <id_1> é o identificador NULL SQL_NULL_HANDLE (ponteiro NULL) quando se cria um ambiente. O identificador (ponteiro) para o registro de ambiente recém-criado é retornado na variável amb1 na linha 6.
4. Um *registro de conexão* é configurado no programa usando SQLAllocHandle. Na linha 7, o registro de conexão criado tem o identifi-

[14] Para manter nossa apresentação simples, não mostraremos os registros de descrição aqui.

cador con1 e está contido no ambiente amb1. Uma **conexão** é então estabelecida em con1 para um banco de dados de servidor em particular usando a função SQLConnect da SQL/CLI (linha 8). Em nosso exemplo, o nome do servidor de banco de dados ao qual estamos nos conectando é *dbs* e o nome de conta e senha para login são *js* e *xyz*, respectivamente.

5. Um *registro de instrução* é configurado no programa usando SQLAllocHandle. Na linha 9, o registro de instrução criado tem um identificador inst1 e usa a conexão con1.
6. A instrução é *preparada* usando a função SQL/CLI SQLPrepare. Na linha 10, isso atribui a **string de instrução** SQL (a *consulta* em nosso exemplo) ao identificador inst1. O símbolo de ponto de interrogação (?) na linha 10 representa um **parâmetro de instrução**, que é um valor a ser determinado em tempo de execução — normalmente, por seu vínculo com uma variável de programa em C. Em geral, poderia haver vários parâmetros na string da instrução. Eles são distinguidos pela ordem de aparecimento dos pontos de interrogação na string de instrução (o primeiro ? representa o parâmetro 1, o segundo ? representa o parâmetro 2, e assim por diante). O último parâmetro em SQLPrepare deveria dar o tamanho da string da instrução SQL em bytes, mas, se entrarmos com a palavra-chave SQL_NTS, isso indica que a string que mantém a consulta é uma *string terminada em NULL*, de modo que a SQL pode calcular seu tamanho automaticamente. Esse uso de SQL_NTS também se aplica a *outros parâmetros de string* nas chamadas de função em nossos exemplos.
7. Antes de executar a consulta, quaisquer parâmetros na string de consulta devem ser ligados a variáveis do programa usando a função SQL/CLI SQLBindParameter. Na Figura 13.10, o parâmetro (indicado por ?) para a consulta preparada referenciada por inst1 é vinculado à variável do programa em C cpf na linha 12. Se houver *n* parâmetros na instrução SQL, devemos ter *n* chamadas de função SQLBindParameter, cada uma com uma *posição de parâmetro* diferente (1, 2, ..., *n*).
8. Após essas preparações, podemos executar a instrução SQL referenciada pelo identificador inst1 usando a função SQLExecute (linha 13). Observe que, embora a consulta seja executada na linha 13, seus resultados ainda não foram atribuídos a quaisquer variáveis do programa em C.
9. Para determinar onde o resultado da consulta é retornado, uma técnica comum é a abordagem de **colunas vinculadas**. Aqui, cada coluna em um resultado de consulta é vinculada a uma variável de programa em C usando a função SQLBindCol. As colunas são distinguidas por sua ordem de aparecimento na consulta SQL. Nas linhas 15 e 16 da Figura 13.10, as duas colunas na consulta (Unome e Salario) são vinculadas às variáveis do programa em C unome e salario, respectivamente.[15]
10. Finalmente, para recuperar os valores de coluna nas variáveis de programa em C, a função SQLFetch é usada (linha 17). Essa função é semelhante ao comando FETCH da SQL embutida. Se um resultado de consulta tem uma coleção de tuplas, cada SQLFetch recebe a próxima tupla e retorna seus valores de coluna para as variáveis do programa vinculadas. A SQLFetch retorna um código de exceção (diferente de zero) se não houver mais tuplas no resultado da consulta.[16]

Como podemos ver, o uso de chamadas de função dinâmicas requer muita preparação para configurar as instruções SQL e vincular parâmetros de instrução e resultados de consulta às variáveis de programa apropriadas.

Em CLI1, uma *única tupla* é selecionada pela consulta SQL. A Figura 13.11 mostra um exemplo da recuperação de múltiplas tuplas. Consideramos que as variáveis apropriadas do programa em C foram declaradas como na Figura 13.1. O segmento de programa em CLI2 lê (entrada) um número de departamento e depois recupera os funcionários que trabalham nesse departamento. Um loop, então, percorre cada registro de funcionário, um de cada vez, e imprime o último nome e o salário do funcionário.

13.3.2 JDBC: chamadas de função SQL para programação Java

Agora, vamos voltar nossa atenção para o modo como a SQL pode ser chamada com base na lingua-

[15] Uma técnica alternativa, conhecida como **colunas desvinculadas**, utiliza diferentes funções SQL/CLI, a saber, SQLGetCol ou SQLGetData, para recuperar colunas do resultado da consulta sem vinculá-las previamente; estas são aplicadas após o comando SQLFetch na linha 17.

[16] Se forem usadas variáveis de programa desvinculadas, a SQLFetch retorna a tupla em uma área de programa temporária. Cada SQLGetCol (ou SQLGetData) subsequente retorna um valor de atributo em ordem. Basicamente, para cada linha no resultado da consulta, o programa deve percorrer os valores de atributo (colunas) nessa linha. Isso é útil se o número de colunas no resultado da consulta for variável.

```
    //Segmento de programa CLI2:
0)  #include sqlcli.h ;
1)  void imprimeFuncsDepartamento( ) {
2)  SQLHSTMT inst1 ;
3)  SQLHDBC con1 ;
4)  SQLHENV amb1 ;
5)  SQLRETURN ret1, ret2, ret3, ret4 ;
6)  ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &amb1) ;
7)  if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, amb1, &con1) else exit ;
8)  if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz",
    SQL_NTS) else exit ;
9)  if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &inst1) else exit ;
10) SQLPrepare(inst1,"select Unome,Salario fromFUNCIONARIO whereDnr = ?", SQL_NTS);
11) prompt("Digite o número do Departamento: ", dnr) ;
12) SQLBindParameter(inst1, 1, SQL_INTEGER, &dnr, 4, &fetchlen1) ;
13) ret1 = SQLExecute(inst1) ;
14) if (!ret1) {
15)     SQLBindCol(inst1, 1, SQL_CHAR, &unome, 15, &fetchlen1) ;
16)     SQLBindCol(inst1, 2, SQL_FLOAT, &salario, 4, &fetchlen2) ;
17)     ret2 = SQLFetch(inst1) ;
18)     while (!ret2) {
19)         printf(unome, salario) ;
20)         ret2 = SQLFetch(inst1) ;
21)         }
22)     }
23) }
```

Figura 13.11
Segmento de programa CLI2, um segmento de programa em C que usa SQL/CLI para uma consulta com uma coleção de tuplas em seu resultado.

gem de programação orientada a objeto Java.[17] As bibliotecas de função para esse acesso são conhecidas como **JDBC**.[18] A linguagem de programação Java foi projetada para ser independente de plataforma — ou seja, um programa deve ser capaz de rodar em qualquer tipo de sistema de computador que tenha um interpretador Java instalado. Por causa dessa portabilidade, muitos fornecedores de SGBDR oferecem drivers JDBC de modo que seja possível acessar seus sistemas por meio de programas Java. Um **driver JDBC** é basicamente uma implementação das chamadas de função especificadas na interface de programação de aplicação (API) JDBC para o SGBDR de determinado fornecedor. Logo, um programa Java com chamadas de função JDBC pode acessar qualquer SGBDR que tenha um driver JDBC disponível.

Como a Java é orientada a objeto, suas bibliotecas de função são implementadas como **classes**. Antes de ser capaz de processar chamadas de função JDBC com Java, é necessário importar as **bibliotecas de classes JDBC**, que se chamam java.sql.*. Estas podem ser baixadas e instaladas pela Web.[19]

A JDBC foi elaborada para permitir que um único programa Java se conecte a vários bancos de dados diferentes. Estes às vezes são chamados de **fontes de dados** acessadas pelo programa Java. Essas fontes de dados poderiam ser armazenadas usando SGBDRs de diferentes vendedores, e poderiam ficar em diferentes máquinas. Logo, variados acessos a fontes de dados no mesmo programa Java podem exigir drivers JDBC de diferentes vendedores. Para alcançar essa flexibilidade, uma classe JDBC especial, chamada classe **gerenciadora de drivers**, é empregada, e registra os drivers instalados. Um driver deve ser *registrado* no gerenciador de driver antes de ser usado. As operações (métodos) da classe gerenciadora de driver incluem getDriver, registerDriver e deregisterDriver. Estas podem ser usadas para acrescentar e remover drivers dinami-

[17] Esta seção pressupõe uma familiaridade com conceitos orientados a objeto (ver Capítulo 11) e conceitos básicos de Java.

[18] Como já dissemos, JDBC é uma marca registrada da Sun Microsystems, embora normalmente seja considerado um acrônimo para *Java Database Connectivity*.

[19] Estas estão disponíveis em vários sites Web — por exemplo, em <http://industry.java.sun.com/products/jdbc/drivers>.

camente. Outras funções configuram e fecham conexões com fontes de dados, conforme veremos.

Para carregar um driver JDBC de maneira explícita, a função Java genérica para carregar uma classe pode ser usada. Por exemplo, para carregar o driver JDBC para o SGBDR da Oracle, o comando a seguir pode ser usado:

```
Class.forName("oracle.jdbc.driver.OracleDriver")
```

Isso registrará o driver no gerenciador e o tornará disponível ao programa. Também é possível carregar e registrar os drivers necessários na linha de comandos que executa o programa, por exemplo, ao incluir o seguinte na linha de comando:

```
-Djdbc.drivers = oracle.jdbc.driver
```

A seguir, vemos as etapas típicas que são realizadas ao escrever um programa de aplicação Java com acesso a banco de dados por meio de chamadas de função JDBC. Ilustramos as etapas nos referindo ao exemplo JDBC1 da Figura 13.12, que lê um número de Cpf de um funcionário e imprime o último nome e salário dele.

1. A *biblioteca de classes* JDBC precisa ser importada para o programa Java. Essas classes são chamadas de java.sql.*, e podem ser importadas usando a linha 1 da Figura 13.12. Quaisquer bibliotecas de classe Java adicionais necessárias pelo programa também devem ser importadas.
2. Carregar o driver JDBC conforme discutido anteriormente (linhas 4 a 7). A exceção Java na linha 5 ocorre se o driver não for carregado com sucesso.
3. Criar variáveis apropriadas conforme a necessidade no programa Java (linhas 8 e 9).
4. **O objeto Connection**. Um **objeto de conexão** é criado usando a função getConnection da classe DriverManager do JDBC. Nas linhas 12 e 13, o objeto Connection é criado usando a chamada de função getConnection(urlstring), na qual urlstring tem a forma

```
jdbc:oracle:<tipoDriver>:<conta_dba>/<senha>
```

```
   //Programa JDBC1:
0)  import java.io.* ;
1)  import java.sql.*
    ...
2)  class obterInfFunc {
3)      public static void main (String args []) throws SQLException, IOException {
4)          try { Class.forName("oracle.jdbc.driver.OracleDriver")
5)          } catch (ClassNotFoundException x) {
6)            System.out.println ("Driver não pode ser carregado") ;
7)          }
8)          String dbacct, senha, cpf, unome ;
9)          Double salario ;
10)         dbacct = readentry("Digite a conta do banco de dados:") ;
11)         senha = readentry("Digite a senha:") ;
12)         Connection con = DriverManager.getConnection
13)           ("jdbc:oracle:oci8:" + dbacct + "/" + senha) ;
14)         String inst1 = "select Unome, Salario from FUNCIONARIO where Cpf = ?" ;
15)         PreparedStatement p = conn.prepareStatement(inst1) ;
16)         cpf = readentry("Digite um número de CPF: ") ;
17)         p.clearParameters( ) ;
18)         p.setString(1, cpf) ;
19)         ResultSet r = p.executeQuery( ) ;
20)         while (r.next( )) {
21)           unome = r.getString(1) ;
22)           salario = r.getDouble(2) ;
23)           system.out.println(unome + salario) ;
24)     } }
25) }
```

Figura 13.12
Segmento de programa JDBC1, um segmento de programa em Java com JDBC.

Uma forma alternativa é

getConnection(url, conta_dba, senha)

Várias propriedades podem ser definidas para um objeto de conexão, mas elas são relacionadas principalmente a propriedades transicionais, que discutiremos no Capítulo 21.

5. **O objeto Statement** Um **objeto de instrução** é criado no programa. Em JDBC, existe uma classe de instrução básica, Statement, com duas subclasses especializadas: PreparedStatement e CallableStatement. O exemplo da Figura 13.12 ilustra como objetos PreparedStatement são criados e usados. O próximo exemplo (Figura 13.13) ilustra o outro tipo de objetos Statement. Na linha 14 da Figura 13.12, uma string de consulta com um único parâmetro — indicado pelo símbolo ? — é criada na variável de string inst1. Na linha 15, um objeto p do tipo PreparedStatement é criado com base na string de consulta em inst1 e usando o objeto de conexão con. Em geral, o programador deve usar objetos PreparedStatement se uma consulta tiver de ser executada *múltiplas vezes*, pois ela seria preparada, verificada e compilada apenas uma vez, economizando assim esse custo para execuções adicionais da consulta.

6. **Definindo os parâmetros de instrução.** O ponto de interrogação (?) na linha 14 representa um **parâmetro de instrução**, que é um valor a ser determinado em tempo de execução, normalmente vinculando-o a uma variável de programa Java. Em geral, poderia haver vários parâmetros, distinguidos pela ordem de aparecimento dos pontos de interrogação na string de instrução (o primeiro ? representa o parâmetro 1, o segundo ? representa o parâmetro 2, e assim por diante), conforme discutimos anteriormente.

```
//Program Segment JDBC2:
0)  import java.io.* ;
1)  import java.sql.*
    ...
2)  class imprimeFuncsDepartamento {
3)      public static void main (String args [ ]) throws SQLException, IOException {
4)          try { Class.forName("oracle.jdbc.driver.OracleDriver")
5)          } catch (ClassNotFoundException x) {
6)            System.out.println ("Driver não pode ser carregado") ;
7)          }
8)          String dbacct, senha, unome ;
9)          Double salario ;
10)         Integer dnr ;
11)         dbacct = readentry("Digite a conta do banco de dados:") ;
12)         senha = readentry("Digite a senha:") ;
13)         Connection con = DriverManager.getConnection
14)           ("jdbc:oracle:oci8:" + dbacct + "/" + senha) ;
15)         dnr = readentry("Digite o número do departamento: ") ;
16)         String q = "select Unome, Salario from FUNCIONARIO where Dnr = " + dnr.tostring( ) ;
17)         Statement s = con.createStatement( ) ;
18)         ResultSet r = s.executeQuery(q) ;
19)         while (r.next( )) {
20)           unome = r.getString(1) ;
21)           salario = r.getDouble(2) ;
22)           system.out.println(unome + salario) ;
23)     } }
24) }
```

Figura 13.13

Segmento de programa JDBC2, um segmento de programa que usa JDBC para uma consulta com uma coleção de tuplas em seu resultado.

7. Antes de executar uma consulta PreparedStatement, quaisquer parâmetros devem ser vinculados a variáveis do programa. Dependendo do tipo do parâmetro, diferentes funções, como setString, setInteger, setDouble, e assim por diante, podem ser aplicadas ao objeto PreparedStatement para definir seus parâmetros. A função apropriada deve ser usada para corresponder ao tipo de dado do parâmetro que está sendo definido. Na Figura 13.12, o parâmetro (indicado por ?) no objeto p é vinculado à variável de programa Java cpf na linha 18. A função setString é utilizada porque cpf é uma variável de string. Se houver *n* parâmetros na instrução SQL, devemos ter *n* funções set..., cada uma com uma posição de parâmetro diferente (1, 2, ..., *n*). Geralmente, é aconselhável limpar todos os parâmetros antes de definir quaisquer valores novos (linha 17).
8. Após essas preparações, podemos executar a instrução SQL referenciada pelo objeto p usando a função executeQuery (linha 19). Existe uma função genérica execute em JDBC, mais duas funções especializadas: executeUpdate e executeQuery. A executeUpdate é utilizada para instruções insert, delete ou update da SQL, e retorna um valor inteiro indicando o número de tuplas que foram afetadas. A executeQuery é empregada para instruções de recuperação SQL, e retorna um objeto do tipo ResultSet, que vamos discutir na sequência.
9. **O objeto ResultSet** Na linha 19, o resultado da consulta é retornado em um *objeto* r do tipo **ResultSet** Isso é semelhante a um array bidimensional ou a uma tabela, na qual as tuplas são as linhas e os atributos retornados são as colunas. Um objeto **ResultSet** é semelhante a um cursor na SQL embutida e um iterador em SQLJ. Em nosso exemplo, quando a consulta é executada, r refere-se a uma tupla antes da primeira tupla no resultado da consulta. A função r.next( ) (linha 20) se move para a próxima tupla (linha) no objeto ResultSet e retorna NULL se não houver mais objetos. Isso serve para controlar o looping. O programador pode se referir aos atributos na tupla atual usando diversas funções get... que dependem do tipo de cada atributo (por exemplo, getString, getInteger, getDouble, e assim por diante). O programador pode usar tanto as posições de atributo (1, 2) como os nomes de atributo reais ("Unome", "Salario") com as funções get.... Em nossos exemplos, usamos a notação posicional nas linhas 21 e 22.

Em geral, o programador pode verificar exceções SQL depois de cada chamada de função JDBC. Não fizemos isso para simplificar os exemplos.

Observe que a JDBC não distingue consultas que retornam tuplas isoladas daquelas que retornam múltiplas tuplas, diferentemente de algumas outras técnicas. Isso é justificável porque um conjunto de resultados de única tupla é apenas um caso especial.

No Exemplo JDBC1, uma *única tupla* é selecionada pela consulta SQL, de modo que o loop nas linhas 20 a 24 é executado no máximo uma vez. O exemplo mostrado na Figura 13.13 ilustra a recuperação de múltiplas tuplas. O segmento de programa em JDBC2 lê (entrada) um número de departamento e depois recupera os funcionários que trabalham nesse departamento. Um loop, então, percorre cada registro de funcionário, um de cada vez, e imprime o sobrenome e salário de cada um. Esse exemplo também ilustra como podemos executar uma consulta diretamente, sem ter de prepará-la como no exemplo anterior. Essa técnica é preferível para consultas que serão executadas apenas uma vez, pois é mais simples de programar. Na linha 17 da Figura 13.13, o programador cria um objeto Statement (em vez de um PreparedStatement, como no exemplo anterior) sem associá-lo a uma string de consulta em particular. A string de consulta q é *passada ao objeto de instrução* s ao ser executada na linha 18.

Isso conclui nossa breve introdução à JDBC. O leitor interessado deve consultar o Website <http://java.sun.com/docs/books/tutorial/jdbc/>, que contém muitos outros detalhes sobre essa linguagem.

13.4 Procedimentos armazenados de banco de dados e SQL/PSM

Esta seção introduz dois tópicos adicionais relacionados à programação de banco de dados. Na Seção 13.4.1, vamos discutir o conceito de procedimentos armazenados, que são módulos de programa armazenados pelo SGBD no servidor de banco de dados. Depois, na Seção 13.4.2, vamos abordar as extensões à SQL que são especificadas no padrão para incluir construções de programação de uso geral em SQL. Essas extensões são conhecidas como SQL/PSM (SQL/*Persistent Stored Modules*) e podem ser usadas para escrever procedimentos armazenados. A SQL/PSM também serve como exemplo de uma linguagem de programação de banco de dados que estende um modelo de banco de dados e linguagem — a saber, a SQL — com algumas construções de programação, como instruções condicionais e loops.

13.4.1 Procedimentos armazenados e funções de banco de dados

Em nossa apresentação das técnicas de programação de banco de dados até aqui, houve uma suposição implícita de que o programa de aplicação de banco de dados estava rodando em uma máquina cliente ou, mais provavelmente, no *computador do servidor de aplicação* na camada intermediária de uma arquitetura cliente-servidor de três camadas (ver Seção 2.5.4 e Figura 2.7). Em ambos os casos, a máquina onde o programa está executando é diferente da máquina em que o servidor de banco de dados — e a parte principal do pacote de software de SGBD — está localizado. Embora isso seja adequado para muitas aplicações, às vezes é útil criar módulos de programa de banco de dados — procedimentos ou funções — que são armazenados e executados pelo SGBD no servidor de banco de dados. Estes são historicamente conhecidos como **procedimentos armazenados** (ou ***stored procedures***) do banco de dados, embora possam ser funções ou procedimentos. O termo usado no padrão SQL para os procedimentos armazenados é **módulos armazenados persistentes** porque esses programas são armazenados persistentemente pelo SGBD, de modo semelhante aos dados persistentes armazenados pelo SGBD.

Os procedimentos armazenados são úteis nas seguintes circunstâncias:

- Se um programa de banco de dados é necessário por várias aplicações, ele pode ser armazenado no servidor e invocado por qualquer um dos programas de aplicação. Isso reduz a duplicação de esforço e melhora a modularidade do software.
- A execução de um programa no servidor pode reduzir a transferência de dados e o custo de comunicação entre o cliente e o servidor em certas situações.
- Esses procedimentos podem melhorar o poder de modelagem fornecido pelas visões ao permitir que tipos mais complexos de dados derivados estejam disponíveis aos usuários do banco de dados. Além disso, eles podem ser usados para verificar restrições complexas que estão além do poder de especificação de asserções e triggers.

Com frequência, muitos SGBDs comerciais permitem que procedimentos armazenados e funções sejam escritos em uma linguagem de programação de uso geral. Como alternativa, um procedimento armazenado pode ser feito de comandos SQL simples, como recuperações e atualizações. O formato geral da declaração de procedimentos armazenados é o seguinte:

```
CREATE PROCEDURE<nome do procedimento>
(<parametros>)
<declaracoes de local>
<corpo do procedimento> ;
```

Os parâmetros e declarações locais são opcionais e especificados apenas se necessário. Para declarar uma função, um tipo de retorno é necessário, de modo que o formato da declaração é

```
CREATE FUNCTION<nome da funcao>
(<parametros>)
RETURNS <tipo de retorno>
<declaracoes de local>
<corpo da funcao> ;
```

Se o procedimento (ou função) for escrito em uma linguagem de programação de uso geral, é comum especificar a linguagem e também um nome de arquivo em que o código do programa é armazenado. Por exemplo, o formato a seguir pode ser utilizado:

```
CREATE PROCEDURE<nome do procedimento>
(<parametros>)
LANGUAGE<nome da linguagem de programacao>
EXTERNAL NAME<nome do caminho do arquivo> ;
```

Em geral, cada parâmetro deve ter um **tipo de parâmetro**, o qual é um dos tipos de dados da SQL. Cada parâmetro também deve ter um **modo de parâmetro**, que é um dentre IN, OUT ou INOUT. Estes correspondem a parâmetros cujos valores são apenas de entrada, apenas de saída (retornados) ou de entrada e saída, respectivamente.

Como os procedimentos e funções são armazenados de maneira persistente pelo SGBD, deve ser possível chamá-los das várias interfaces SQL e linguagens de programação. A **instrução CALL** no padrão SQL pode ser usada para chamar um procedimento armazenado — ou por uma interface interativa, ou SQLJ ou SQL embutida. O formato da instrução é o seguinte:

```
CALL <nome do procedimento ou funcao>
(<lista de argumentos>) ;
```

Se essa instrução for chamada da JDBC, ela deve ser atribuída a um objeto de instrução do tipo CallableStatement (ver Seção 13.3.2).

13.4.2 SQL/PSM: estendendo a SQL para especificar módulos armazenados persistentes

A SQL/PSM é a parte do padrão SQL que especifica como escrever módulos armazenados persis-

tentes. Ela inclui as instruções para criar funções e procedimentos que descrevemos na seção anterior. Também inclui construções de programação adicionais para melhorar o poder da SQL com a finalidade de escrever o código (ou corpo) dos procedimentos armazenados e funções.

Nesta seção, vamos discutir as construções SQL/PSM para instruções condicionais (desvio) e para instruções de looping. Estas darão uma ideia do tipo de construção que a SQL/PSM incorporou.[20] Depois, oferecemos um exemplo para ilustrar como essas construções podem ser usadas.

A instrução de desvio condicional na SQL/PSM tem a seguinte forma:

```
IF <condicao> THEN <lista de instrucoes>
    ELSEIF<condicao>THEN <listade instrucoes>
    ...
    ELSEIF<condição>THEN <listade instrucoes>
    ELSE <lista de instrucoes>
END IF ;
```

Considere o exemplo da Figura 13.14, que ilustra como a estrutura de desvio condicional pode ser usada em uma função SQL/PSM. A função retorna um valor de string (linha 1) descrevendo o tamanho de um departamento em uma empresa com base no número de funcionários. Existe um parâmetro inteiro IN, nrdep, que indica um número de departamento. Uma variável local Nr_de_funcs é declarada na linha 2. A consulta nas linhas 3 e 4 retorna o número de funcionários no departamento, e o desvio condicional nas linhas 5 a 8 então retorna um dos valores {'ENORME', 'GRANDE', 'MEDIO', 'PEQUENO'} com base no número de funcionários.

```
  //Função PSM1:
0)  CREATE FUNCTION Tam_dep(IN nrdep INTEGER)
1)  RETURNS VARCHAR [7]
2)  DECLARE Nr_de_funcs INTEGER ;
3)  SELECT COUNT(*) INTO Nr_de_funcs
4)  FROM FUNCIONARIO WHERE Dnr = nrdep ;
5)  IF Nr_de_funcs > 100 THEN RETURN "ENORME"
6)      ELSEIF Nr_de_funcs > 25 THEN RETURN
        "GRANDE"
7)      ELSEIF Nr_de_funcs > 10 THEN
        RETURN "MEDIO"
8)      ELSE RETURN "PEQUENO"
9)  END IF;
```

Figura 13.14
Declarando uma função em SQL/PSM.

A SQL/PSM tem várias construções para looping. Existem estruturas de looping padrão while e repeat, com as seguintes formas:

```
WHILE <condicao> DO
    <lista de instrucoes>
END WHILE ;
REPEAT
    <lista de instrucoes>
UNTIL <condicao>
END REPEAT ;
```

Há também uma estrutura de looping baseada em cursor. A lista de instrução em tal loop é executada uma vez para cada tupla no resultado da consulta. Esta tem a seguinte forma:

```
FOR <nome do loop> AS <nome do cursor>
CURSOR FOR <consulta> DO
    <lista de instrucoes>
END FOR ;
```

Os loops podem ter nomes, e existe uma instrução LEAVE <nome do loop> para parar um loop quando uma condição é satisfeita. A SQL/PSM tem muitos outros recursos, mas eles estão fora do escopo de nossa apresentação.

13.5 Comparando as três técnicas

Nesta seção, comparamos rapidamente as três técnicas para programação de banco de dados e discutimos as vantagens e desvantagens de cada uma.

1. **Técnica da SQL embutida.** A principal vantagem dessa técnica é que o texto da consulta faz parte do próprio código fonte do programa e, portanto, é possível verificar erros de sintaxe e validar contra o esquema do banco de dados em tempo de compilação. Isso também torna o programa bastante legível, pois as consultas são prontamente visíveis no código fonte. As principais desvantagens são a perda de flexibilidade na mudança da consulta em tempo de execução e o fato de que todas as mudanças nas consultas devem passar pelo processo inteiro de recompilação. Além disso, como as consultas são conhecidas de antemão, a escolha de variáveis de programa para manter os resultados da consulta é uma tarefa simples e, dessa forma, a programação da aplicação costuma ser mais fácil. Porém, para aplicações complexas em que as consultas precisam ser

[20] Só oferecemos uma breve introdução à SQL/PSM aqui. Existem muitos outros recursos no padrão SQL/PSM.

geradas em tempo de execução, a técnica de chamada de função será mais adequada.

2. **Técnica da biblioteca de chamadas de função.** Essa técnica oferece mais flexibilidade porque as consultas podem ser geradas em tempo de execução, se necessário. Contudo, isso leva a uma programação mais complexa, visto que as variáveis do programa que combinam com as colunas no resultado da consulta podem não ser conhecidas de antemão. Como as consultas são passadas como strings de instrução nas chamadas de função, menos uma verificação pode ser feita em tempo de compilação. Toda verificação de sintaxe e validação de consulta precisa ser feita em tempo de execução, e o programador deve verificar e levar em conta possíveis erros adicionais em tempo de execução no código do programa.
3. **Técnica da linguagem de programação de banco de dados.** Essa técnica não sofre do problema de divergência de impedância, pois os tipos de dados da linguagem de programação são os mesmos que os tiposde dados do banco de dados. Porém, os programadores precisam aprender uma nova linguagem de programação, em vez de usar uma linguagem com a qual já estão familiarizados. Além disso, algumas linguagens de programação de banco de dados são específicas do vendedor, ao passo que as de uso geral podem funcionar facilmente com sistemas de diversos vendedores.

Resumo

Neste capítulo, apresentamos recursos adicionais da linguagem de banco de dados SQL. Em particular, apresentamos uma visão geral das técnicas mais importantes para programação de banco de dados na Seção 13.1. Depois, discutimos as diversas técnicas para a programação de aplicação de banco de dados nas seções 13.2 a 13.4.

Na Seção 13.2, discutimos a técnica geral conhecida como SQL embutida, na qual as consultas fazem parte do código fonte do programa. Um pré-compilador normalmente é usado para extrair comandos SQL do programa, para processamento pelo SGBD, e substituindo-os pelas chamadas de função ao código compilado do SGBD. Apresentamos uma visão geral da SQL embutida, usando a linguagem de programação C como linguagem hospedeira em nossos exemplos. Também discutimos a técnica SQLJ para embutir SQL em programas Java. Os conceitos de cursor (para a SQL embutida) e iterador (para a SQLJ) foram apresentados e ilustrados com exemplos para mostrar como eles são usados para percorrer as tuplas em um resultado de consulta e extrair o valor do atributo de variáveis do programa, para processamento posterior.

Na Seção 13.3, discutimos como as bibliotecas de chamada de função podem ser usadas para acessar bancos de dados SQL. Essa técnica é mais dinâmica do que a SQL embutida, mas requer programação mais complexa porque os tipos e o número de atributos em um resultado de consulta podem ser determinados em tempo de execução. Uma visão geral do padrão SQL/CLI foi apresentada, com exemplos usando C como a linguagem hospedeira. Discutimos algumas das funções na biblioteca SQL/CLI, como as consultas são passadas como strings, quantos parâmetros são atribuídos em tempo de execução e como os resultados são retornados às variáveis do programa. Depois, demos uma visão geral da biblioteca de classes JDBC, que é usada em Java, e abordamos algumas de suas classes e operações. Em particular, a classe ResultSet é utilizada para criar objetos que mantêm os resultados da consulta, que podem então ser percorridos pela operação next(). As funções get e set, para recuperar valores de atributo e definir valores de parâmetro também foram discutidas.

Na Seção 13.4, falamos rapidamente sobre os procedimentos armazenados e discutimos a SQL/PSM como um exemplo de linguagem de programação de banco de dados. Finalmente, comparamos brevemente as três técnicas na Seção 13.5. É importante observar que escolhemos dar uma visão geral comparativa das três técnicas principais para programação de banco de dados, pois o estudo de uma técnica em particular em profundidade é um assunto que merece ser abordado em um livro inteiro.

Perguntas de revisão

13.1. O que é ODBC? Qual é sua relação com a SQL/CLI?

13.2. O que é JDBC? Ela é um exemplo de SQL embutida ou de uso de chamadas de função?

13.3. Liste as três técnicas principais para programação de banco de dados. Quais são as vantagens e desvantagens de cada uma?

13.4. O que é o problema da divergência de impedância? Qual das três técnicas de programação minimiza esse problema?

13.5. Descreva o conceito de um cursor e como ele é usado na SQL embutida.

13.6. Para que é usada a SQLJ? Descreva os dois tipos de iteradores disponíveis na SQLJ.

Exercícios

13.7. Considere o banco de dados mostradona Figura 1.2, cujo esquema é mostrado na Figura 2.1. Escreva um segmento de programa para ler o nome de um aluno e imprimir sua média de notas, considerando que A=4, B=3, C=2 e D=1 ponto. Use a SQL embutida com C como linguagem hospedeira.

13.8. Repita o Exercício 13.7, mas use a SQLJ com Java como linguagem hospedeira.

13.9. Considere o esquema de banco de dados relacional de biblioteca da Figura 4.6. Escreva um segmento de programa que recupere a lista de livros que ficaram em atraso ontem e que imprima o título do livro e o nome de quem pegou cada um emprestado. Use a SQL embutida e C como linguagem hospedeira.

13.10. Repita o Exercício 13.9, mas use SQLJ com Java como linguagem hospedeira.

13.11. Repita os exercícios 13.7 e 13.9, mas use SQL/CLI com C como linguagem hospedeira.

13.12. Repita os exercícios 13.7 e 13.9, mas use JDBC com Java como linguagem hospedeira.

13.13. Repita o Exercício 13.7, mas escreva uma função em SQL/PSM.

13.14. Crie uma função em PSM que calcule o salário médio para a tabela FUNCIONARIO mostrada na Figura 3.5.

Bibliografia selecionada

Existem muitos livros que descrevem os diversos aspectos da programação de banco de dados em SQL. Por exemplo, Sunderraman (2007) descreve a programação no SGBD Oracle 10g e Reese (1997) foca a JDBC e a programação Java. Muitos recursos também estão disponíveis na Web.

capítulo 14

Programação de banco de dados Web usando PHP

No capítulo anterior, fornecemos uma visão geral das técnicas de programação de banco de dados utilizando linguagens de programação tradicionais, e usamos as linguagens Java e C em nossos exemplos. Agora, vamos voltar nossa atenção para o modo como os bancos de dados são acessados com linguagens de scripting. Muitas aplicações de comércio eletrônico (e-commerce) e outras aplicações da Internet, que oferecem interfaces Web para acessar informações armazenadas em um ou mais bancos de dados, estão utilizando linguagens de scripting. Essas linguagens normalmente são usadas para gerar documentos HTML, que são então exibidos pelo navegador Web para interação com o usuário.

No Capítulo 12, tivemos uma visão geral da linguagem XML para representação e intercâmbio de dados na Web, e discutimos algumas das maneiras como ela pode ser usada. Apresentamos a HTML e discutimos como ela difere da XML. A HTML básica é útil para gerar páginas Web *estáticas* com texto fixo e outros objetos, mas a maioria das aplicações de e-commerce exige páginas Web que oferecem recursos interativos com o usuário. Por exemplo, considere o caso de um cliente de companhia aérea que deseja verificar as informações de hora e portão de chegada de determinado voo. O usuário pode inserir informações como uma data e um número de voo em certos campos de formulário da página Web. O programa Web primeiro precisa submeter uma consulta ao banco de dados da companhia aérea para recuperar essa informação, e depois exibi-la. Essas páginas Web, nas quais parte da informação é extraída de bancos de dados ou outras fontes de dados, são denominadas páginas Web *dinâmicas*. Os dados extraídos e exibidos a cada vez serão para diferentes voos e datas.

Existem várias técnicas para a programação de recursos dinâmicos nas páginas Web. Vamos focar uma técnica aqui, que é baseada no uso da linguagem de scripting de fonte aberto PHP. A PHP recentemente passou a ser bastante utilizada. Os interpretadores para PHP são fornecidos gratuitamente e escritos na linguagem C, de modo que estão disponíveis na maioria das plataformas de computador. Um interpretador PHP oferece um pré-processador de hipertexto, que executará comandos PHP em um arquivo de texto e criará o arquivo HTML desejado. Para acessar bancos de dados, uma biblioteca de funções PHP precisa ser incluída no interpretador PHP, conforme discutiremos na Seção 14.3. Os programas PHP são executados no computador servidor Web. Isso é diferente de algumas linguagens de scripting, como JavaScript, que são executadas no computador cliente.

Este capítulo é organizado da seguinte forma. A Seção 14.1 contém um exemplo simples para ilustrar como a PHP pode ser utilizada. A Seção 14.2 oferece uma visão geral da linguagem PHP e como ela é usada para programar algumas funções básicas para páginas Web interativas. A Seção 14.3 focaliza o uso da PHP para interagir com bancos de dados SQL por meio de uma biblioteca de funções conhecida como PEAR DB. Por fim, apresentamos um resumo do capítulo.

14.1 Um exemplo simples em PHP

A PHP é uma linguagem de scripting de uso geral com fonte aberto. O mecanismo interpretador da PHP é escrito na linguagem de programação C, de modo que pode ser utilizado em quase todos os tipos de computadores e sistemas operacionais. A PHP normalmente vem instalada com o sistema operacional UNIX. Para plataformas de computação com outros sistemas ope-

racionais, como Windows, Linux ou Mac OS, o interpretador PHP pode ser baixado em <http://www.php.net>. Assim como outras linguagens de scripting, a PHP é particularmente adequada para manipulação de páginas de texto, e em particular para manipulação de páginas HTML dinâmicas no computador servidor Web. Isso é diferente da JavaScript, que é baixada com as páginas Web para execução no computador cliente.

A PHP possui bibliotecas de funções para acessar bancos de dados armazenados em diversos tipos de sistemas de bancos de dados relacionais, como Oracle, MySQL, SQLServer e qualquer sistema que suporta o padrão ODBC (ver Capítulo 13). Sob a arquitetura de três camadas (ver Capítulo 2), o SGBD residiria no **servidor de banco de dados da camada inferior**. A PHP seria executada no **servidor Web da camada intermediária**, onde os comandos de programa em PHP manipulariam os arquivos HTML para criar as páginas Web dinâmicas personalizadas. A HTML é então enviada à **camada cliente** para exibição e interação com o usuário.

Considere o exemplo mostrado na Figura 14.1(a), que pede que um usuário informe o nome e último nome e depois imprime uma mensagem de boas-vindas a ele. Os números de linha não fazem parte do código do programa; eles são utilizados a seguir apenas como referência para a explicação:

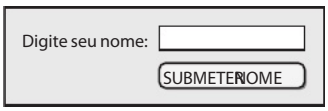
1. Suponha que o arquivo contendo o script em PHP no segmento de programa P1 esteja armazenado no seguinte local da Internet: <http://www.meuservidor.com/saudacao.php>. Então, se um usuário digita esse endereço no navegador, o interpretador PHP começaria a inter-

(a)

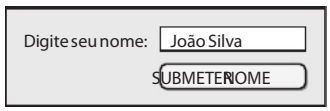
```
//Segmento de programa P1:
0)  <?php
1)  // Imprimindomensagem deboas-vindas seo usuáriosubmeteu
    // seu nome por este formulário HTML
2)  if ($_POST['nome_usuario']) {
3)          print("Bem-vindo, ") ;
4)          print($_POST['nome_usuario'
5)  }
6)  else            {

7)          /// Imprimindo o formulário para entrar com o nome do usuário
            pois nenhum nome foi informado ainda
8)          print <<<_HTML_
9)          <FORM method="post" action="$_SERVER['PHP_SELF']">
10)         Digite seu nome: <input type="text" name="nome_usuario">
11)         <BR/>
12)         <INPUT type="submit" value="SUBMETER NOME">
13)         </FORM>
14)         _HTML_;
15) }
16) ?>
```

(b)



(c)



(d)

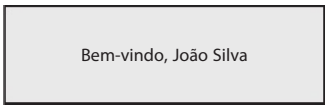


Figura 14.1
(a) Segmento de programa PHP para a entrada de uma saudação. (b) Formulário inicial exibido pelo segmento de programa PHP. (c) Usuário informa o nome *João Silva*. (d) Formulário imprime saudação para *João Silva*.

pretar o código e a produzir o formulário mostrado na Figura 14.1(b). Explicaremos como isso acontece enquanto examinamos as linhas no segmento de código P1.

2. A linha 0 mostra a tag de início da PHP <?php, que indica ao mecanismo interpretador PHP que ele deverá processar todas as linhas de texto seguintes até encontrar a tag de fim PHP ?>, mostrada na linha 16. O texto fora dessas tags é impresso tal como está. Isso permite que os segmentos de código PHP sejam incluídos em um arquivo HTML maior. Somente as seções no arquivo entre <?php e ?> são processadas pelo pré-processador PHP.
3. A linha 1 mostra um modo de postar comentários em um programa PHP em uma única linha iniciada por //. Comentários de uma linha também podem ser iniciados com #, e terminam ao final da linha em que são inseridos. Comentários em múltiplas linhas começam com /* e terminam com */.
4. A variável PHP **autoglobal** predefinida $_POST (linha 2) é um vetor que mantém todos os valores inseridos por meio de parâmetros do formulário. Vetores em PHP são *vetores dinâmicos*, sem um número fixo de elementos. Eles podem ser vetores indexados numericamente, cujos índices (posições) são numerados (0, 1, 2, ...), ou podem ser vetores associativos cujos índices podem ser quaisquer valores de string. Por exemplo, um vetor associativo indexado com base na cor pode ter os índices {"vermelho", "azul", "verde"}. Neste exemplo, $_POST é indexado de forma associativa pelo nome do valor postado nome_usuario que é especificado no atributo de nome da tag de entrada na linha 10. Assim, $_POST['nome_usuario'] terá o valor digitado pelo usuário. Discutiremos mais sobre vetores em PHP na Seção 14.2.2.
5. Quando a página Web em <http://www.meuservidor.com/saudacao.php> é aberta inicialmente, a condição if na linha 2 será avaliada como falsa porque ainda não existe valor em $_POST['nome_usuario']. Logo, o interpretador PHP processará as linhas 6 a 15, as quais criam o texto para um arquivo HTML que exibe o formulário mostrado na Figura 14.1(b). Este é então exibido no lado do cliente pelo navegador Web.
6. A linha 8 mostra uma maneira de criar **strings de texto longas** em um arquivo HTML. Discutiremos outras maneiras de especificar strings mais adiante nesta seção. Todo texto entre um <<<_HTML_ de abertura e um _HTML_; de fechamento é impresso no arquivo HTML tal como está. O _HTML_; de fechamento precisa estar sozinho, em uma linha separada. Assim, o texto acrescentado ao arquivo HTML enviado ao cliente será o texto entre as linhas 9 e 13. Isso inclui as tags HTML para criar o formulário mostrado na Figura 14.1(b).
7. **Nomes de variável** PHP começam com um sinal $ e podem incluir caracteres, números e o caractere de sublinhado _. A variável autoglobal (predefinida) do PHP $_SERVER (linha 9) é um vetor que inclui informações sobre o servidor local. O elemento $_SERVER['PHP_SELF'] no vetor é o nome do caminho do arquivo PHP que atualmente está sendo executado no servidor. Portanto, o atributo action da tag FORM (linha 9) instrui o interpretador PHP a reprocessar o mesmo arquivo, quando os parâmetros do formulário forem inseridos pelo usuário.
8. Logo que o usuário digita o nome *João Silva* na caixa de texto e clica no botão *SUBMETER NOME* (Figura 14.1(c)), o segmento de programa *P1* é reprocessado. Dessa vez, $_POST['nome_usuario'] incluirá a string "João Silva", de modo que as linhas 3 e 4 agora serão colocadas no arquivo HTML enviado ao cliente, que exibe a mensagem da Figura 14.1(d).

Como podemos ver por esse exemplo, o programa PHP pode criar dois comandos HTML diferentes, dependendo se o usuário acabou de entrar ou se ele já submeteu seu nome pelo formulário. Em geral, um programa PHP pode criar diversas variações de texto HTML em um arquivo HTML no servidor, dependendo dos caminhos condicionais particulares tomados no programa. Logo, a HTML enviada ao cliente será diferente, dependendo da interação com o usuário. Esse é um modo como a PHP é usada para criar páginas Web *dinâmicas*.

14.2 Visão geral dos recursos básicos da PHP

Nesta seção, fornecemos uma visão geral de alguns dos recursos da PHP que são úteis na criação de páginas HTML interativas. A Seção 14.3 focalizará como os programas em PHP podem acessar bancos de dados para consulta e atualização. Não podemos oferecer uma discussão abrangente sobre PHP, pois existem li-

vros inteiros dedicados a esse assunto. Em vez disso, focalizamos a ilustração de certas características da PHP que são particularmente adequadas para a criação de páginas Web dinâmicas que contêm comandos de acesso a banco de dados. Esta seção abrange alguns conceitos e recursos da PHP que serão necessários quando discutirmos o acesso a banco de dados na Seção 14.3.

14.2.1 Variáveis, tipos de dados e construções de programação em PHP

Os **nomes de variável** em PHP começam com o símbolo $ e podem incluir caracteres, letras e o caractere de sublinhado (_). Nenhum outro caractere especial é permitido. Os nomes de variável diferenciam maiúsculas de minúsculas, e o primeiro caractere não pode ser um número. O tipo das variáveis não precisa ser definido antecipadamente. Os valores atribuídos às variáveis determinam seu tipo. De fato, a mesma variável pode mudar seu tipo quando um novo valor for atribuído a ela. A atribuição é feita por meio do operador=.

Como a PHP é direcionada para processamento de textos, existem vários tipos diferentes de valores de string. Também há muitas funções disponíveis para o processamento de strings. Só vamos discutir algumas propriedades básicas dos valores de string e variáveis aqui. A Figura 14.2 ilustra alguns desses valores. Existem quatro formas principais de expressar strings literais:

```
0) print 'Bem-vindo ao meu Web site.';
1) print 'Eu disse a ele, "Bem-vindo à casa"';
2) print 'Vamos agora visitar o próximo site';
3) printf('O preço é $%.2f e o imposto é
   R$%.2f', $preço, $imposto) ;
4) print strtolower('AbCdE');
5) print ucwords(strtolower('JOAO silva'));
6) print 'abc' . 'efg'
7) print "envie seu e-mail para: $endereco_
   email"
8) print <<<FORM_HTML
9) <FORM method="post" action="$_
   SERVER['PHP_SELF']">
10) Digite seu nome: <input type="text"
    name="nome_usuario">
11) FORM_HTML
```

Figura 14.2
Ilustrando valores de string e texto básicos da PHP.

1. **Strings com aspas simples.** Delimite a string com aspas simples, como nas linhas 0, 1 e 2. Se uma aspa simples for necessária dentro da string, use o caractere de escape \' (ver a linha 2).
2. **Strings com aspas duplas.** Delimite strings com aspas duplas, como na linha 7. Nesse caso, *nomes de variável que aparecem dentro da string* são substituídos pelos valores que estão atualmente armazenados nessas variáveis. O interpretador identifica nomes de variável nas strings com aspas duplas por seu caractere inicial $ e os substitui pelo valor na variável. Isso é conhecido como **interpolação de variáveis** nas strings. A interpolação não ocorre nas strings com aspas simples.
3. **Here documents (heredoc).** Delimite uma parte de um documento entre um <<<DOCNAME e termine essa parte com uma única linha contendo o nome do documento DOCNAME. DOCNAME pode ser qualquer string, desde que seja usado tanto para iniciar quanto para terminar o heredoc. Isso é ilustrado nas linhas 8 a 11 da Figura 14.2. As variáveis também são interpoladas substituindo-as por seus valores de string, se aparecerem em heredoc. Esse recurso é utilizado de um modo semelhante às strings com aspas duplas, mas é mais conveniente para texto de múltiplas linhas.
4. **Aspas simples e duplas.** Aspas simples e duplas usadas pela PHP para delimitar strings devem ser aspas *retas* (" ") nos dois lados da string. O editor de textos que criar essas aspas não deverá produzir aspas *curvas* de abertura e fechamento (“ ”) em torno da string.

Há também um operador de concatenação de string especificado pelo símbolo de ponto (.), conforme ilustrado na linha 6 da Figura 14.2. Existem muitas funções de string. Ilustramos apenas algumas delas aqui. A função strtolower muda os caracteres alfabéticos na string para minúsculas, enquanto a função ucwords converte para maiúsculo o primeiro caracter de cada palavra. Estas são ilustradas nas linhas 4 e 5 da Figura 14.2.

A regra geral é usar strings com aspas simples para strings literais, que não contêm variáveis de programa em PHP, e os outros dois tipos (strings com aspas duplas e heredoc), quando os valores das variáveis precisarem ser interpolados na string. Para grandes blocos de texto em múltiplas linhas, o programa deverá usar o estilo de *here documents* para as strings.

A PHP também possui tipos de dados para números inteiros e ponto flutuante, e geralmente segue

as regras da linguagem de programação C para o processamento desses tipos. Os números podem ser formatados para impressão em strings ao especificar o número de dígitos que segue o ponto decimal. Uma variação da função print, chamada printf (print formatado) permite a formatação de números em uma string, conforme ilustra a linha 3 da Figura 14.2.

Existem as construções da linguagem de programação padrão para loops for, loops while e instruções if condicionais. Elas costumam ser semelhantes as suas equivalentes na linguagem C. Não vamos discuti-las aqui. De modo semelhante, *qualquer* valor é avaliado como verdadeiro se usado como uma expressão booleana, *exceto pelo* zero numérico (0) e a string vazia, que são avaliados como falso. Há também os valores literais true e false que podem ser atribuídos. Os operadores de comparação também geralmente seguem as regras da linguagem C. São eles == (igual), != (não igual), > (maior que), >= (maior ou igual), < (menor que) e <= (menor ou igual).

14.2.2 Vetores em PHP

Vetores são muito importantes em PHP, pois permitem listas de elementos. Eles são usados constantemente em formulários que empregam menus pull-down. Um vetor unidimensional serve para manter a lista de escolhas no menu pull-down. Para resultados de consulta de banco de dados, vetores bidimensionais são utilizados com a primeira dimensão representando *linhas* de uma tabela e a segunda dimensão representando *colunas* (atributos) em uma linha.

Existem dois tipos principais de vetores: numéricos e associativos. Vamos discutir cada um deles no contexto dos vetores unidimensionais a seguir.

Um **vetor numérico** associa um índice numérico (posição ou número de sequência) a cada elemento no vetor. Os índices são números inteiros que começam em zero e crescem de forma incremental. Um elemento no vetor é referenciado por meio de seu índice. Um **vetor associativo** oferece pares de elementos (chave => valor). O valor de um elemento é referenciado por meio de sua chave, e todos os valores de chave em determinado vetor precisam ser exclusivos. Os valores de elemento podem ser strings ou inteiros, ou eles mesmos podem ser vetores, levando assim a vetores de maior dimensão.

A Figura 14.3 oferece dois exemplos de variáveis de vetor: $ensinar e $disciplina. O primeiro vetor $ensinar é associativo (ver a linha 0 da Figura 14.3), e cada elemento associa um nome de disciplina (como chave) ao nome do professor da disciplina (como valor). Existem três elementos nesse vetor. A linha 1 mostra como o vetor pode ser atualizado. O primeiro comando na linha 1 atribui um novo professor à disciplina 'Grafico' atualizando seu valor. Como o valor-chave 'Grafico' já existe no vetor, nenhum elemento é criado, mas o valor existente é atualizado.

```
0) $ensinar = array('banco dados' => 'Silva',
   'SO' => 'Carrick', 'Grafico' => 'Kam');
1) $ensinar['Grafico'] = 'Benson';
   $ensinar['Mineração dados'] = 'Kam';
2) sort($ensinar);
3) foreach($ensinar as$chave => $valor){
4)         print " $chave : $valor\n";}
5) $disciplinas = array('Banco dados', 'SO',
   'Grafico', 'Mineração dados');
6) $alterna_cor = array('azul', 'amarelo');
7) for ($i = 0, $num = count($disciplinas); i <
   $num; $i++) {
8)         print '<TR bgcolor="' . $alterna_cor[$i
           % 2] . '">';
9)         print "<TD>Disciplina $i is</
           TD><TD>$disciplinas[$i]</TD></
           TR>\n";
10) }
```

Figura 14.3 Ilustrando o processamento de vetor básico em PHP.

O segundo comando cria um elemento, pois o valor-chave 'Mineração dados' não existia no vetor antes. Novos elementos são acrescentados ao final do vetor.

Se só oferecermos valores (não chaves) como elementos do vetor, as chaves são automaticamente numéricas e numeradas com 0, 1, 2, ... Isso é ilustrado na linha 5 da Figura 14.3, pelo vetor $disciplinas. Vetores associativos e numéricos não têm limites de tamanho. Se algum valor de outro tipo de dado, digamos, um inteiro, for atribuído a uma variável PHP que estava mantendo um vetor, a variável agora mantém o valor inteiro e o conteúdo do vetor é perdido. Basicamente, a maioria das variáveis pode receber valores de qualquer tipo de dado a qualquer momento.

Existem várias técnicas para percorrer vetores em PHP. Ilustramos duas delas na Figura 14.3. As linhas 3 e 4 mostram um método de percorrer todos os elementos em um vetor usando a construção foreach, e de imprimir a chave e o valor de cada elemento em uma linha separada. As linhas 7 a 10 mostram como uma construção de loop for tradicional pode ser usada. Um contador de função embutido (linha 7) retorna o número atual de elementos no vetor, que é atribuído à variável $num e utilizado para controlar o término do loop.

O exemplo nas linhas 7 a 10 ilustra como uma tabela HTML pode ser exibida com cores de linha alternadas, ao definir as cores em um vetor $alt_row_color (linha 8). A cada passada do loop, a função de resto $i % 2 muda de uma linha (índice 0) para a seguinte (índice 1) (ver a linha 8). A cor é atribuída ao atributo HTML *bgcolor* da tag <TR> (que em inglês significa *table row* ou linha de tabela).

A função count (linha 7) retorna o número atual de elementos no vetor. A função sort (linha 2) classifica o vetor com base nos valores de elemento nela contidos (não as chaves). Para vetores associativos, cada chave permanece associada ao mesmo valor de elemento após a classificação. Isso não ocorre quando se classifica vetores numéricos. Existem muitas outras funções que podem ser aplicadas a vetores PHP, mas uma discussão completa está fora do escopo de nossa apresentação aqui.

14.2.3 Funções em PHP

Assim como em outras linguagens de programação, as **funções** podem ser definidas em PHP para estruturar melhor um programa complexo e compartilhar seções comuns do código, que podem ser reutilizadas por várias aplicações. A versão mais nova da PHP, a PHP5, também possui recursos orientados a objeto, mas não discutiremos a respeito deles aqui, pois estamos focalizando os fundamentos das PHP. As funções básicas das PHP podem ter argumentos que são *passados por valor*. Variáveis globais podem ser acessadas nas funções. As regras de escopo padrão se aplicam a variáveis que aparecem em uma função e no código que chama a função.

Agora, vamos dar dois exemplos simples para ilustrar as funções básicas da PHP. Na Figura 14.4, mostramos como poderíamos rescrever o segmento de código P1 da Figura 14.1(a) usando funções. O segmento de código P1' da Figura 14.4 tem duas funções: exibir_saudacao( ) (linhas 0 a 3) e exibir_form_vazio( ) (linhas 5 a 13). Nenhuma dessas funções tem argumentos ou valores de retorno. As linhas 14 a 19 mostram como podemos chamar essas funções para produzir o mesmo efeito do segmento de código P1 na Figura 14.1(a). Como podemos ver neste exemplo, as funções podem ser utilizadas apenas para tornar o código PHP mais bem estruturado e mais fácil de acompanhar.

Um segundo exemplo é mostrado na Figura 14.5. Aqui, estamos usando o vetor $ensinar apresentado na Figura 14.3. A função professor_disciplina() nas linhas 0 a 8 da Figura 14.5 possui dois argumentos: $disciplina (uma string contendo um nome de disciplina) e $atividades_ensino (um vetor associativo contendo trabalhos de disciplina, semelhante ao vetor $ensinar mostrado na Figura 14.3). A função acha o nome do professor que leciona uma disciplina em particular. As linhas 9 a 14 da Figura 14.5 mostram como essa função pode ser usada.

```
//Segmento de programa P1 ':
0)  function exibir_saudacao( ) {
1)          print("Bem-vindo, ") ;
2)          print($_POST['nome_usuario'
3)  }
4)
5)  function exibir_form_vazio( ); {
6)  print <<<_HTML_
7)  <FORM method="post" action="$_
    SERVER['PHP_SELF']">
8)  Digite seu nome: <INPUT type="text"
    name="nome_usuario">
9)  <BR/>
10) <INPUT type="submit" value="Enviar
    nome">
11) </FORM>
12) _HTML_;
13) }
14) if ($_POST['nome_usuario']) {
15)         exibir_saudacao( );
16) }
17) else {
18)         exibir_form_vazio( );
19) }
```

Figura 14.4
Reescrevendo o segmento de programa P1 como P1' usando funções.

A chamada de função na linha 11 retornaria a string: *Silva está lecionando Banco de Dados* , pois a entrada de vetor com a chave 'Banco dados' tem o valor 'Silva' para professor. Por sua vez, a chamada de função na linha 13 retornaria a string: *não existe a disciplina de Arquitetura de Computadores*, pois não há uma entrada no vetor com a chave 'Arquitetura Computadores'. Alguns comentários sobre este exemplo e sobre as funções em PHP em geral:

- A função de vetor em PHP embutida ve-tor_key_exists($k, $a) retorna verdadeira se o valor na variável $k *existir como uma chave* no *vetor associativo* na variável $a. Em nosso exemplo, ela verifica se o valor $disciplina fornecida existe como uma chave no vetor $atividades_ensino (linha 1 da Figura 14.5).
- Os argumentos de função são passados por valor. Logo, neste exemplo, as chamadas nas

```
0)  function professor_disciplina ($disciplina, $atividades_ensino) {
1)          if (array_key_exists($disciplina, $atividades_ensino)) {
2)          $professor = $atividades_ensino [$disciplina];
3)          RETURN "$professor está lecionando $disciplina";
4)          }
5)          else {
6)          RETURN "não existe a disciplina $disciplina";
7)          }
8)  }
9)  $ensinar = array('Banco dados' => 'Silva', 'SO' => 'Carrick', 'Grafico' => 'Kam');
10) $ensinar[ Grafico ] = Benson ; $ensinar[ Mineracao dados ] = Kam ;
11) $x = professor_disciplina('Banco dados', $ensinar);
12) print($x);
13) $x = professor_disciplina('Arquitetura Computadores', $ensinar);
14) print($x);
```

Figura 14.5
Ilustrando uma função com argumentos e valor de retorno.

linhas 11 e 13 não poderiam mudar o vetor $ensinar fornecido como argumento para a chamada. Os valores fornecidos nos argumentos são passados (copiados) para os argumentos da função quando esta é chamada.

- Os valores de retorno de uma função são colocados após a palavra-chave RETURN. Uma função pode retornar qualquer tipo. Neste exemplo, ela retorna um tipo string. Duas strings diferentes podem ser retornadas em nosso exemplo, dependendo de o valor da chave $disciplina fornecido existir ou não no vetor.
- As regras de escopo para nomes de variável se aplicam como nas outras linguagens de programação. Variáveis globais fora da função não podem ser utilizadas a menos que sejam referenciadas usando o vetor embutido da PHP $GLOBALS. Basicamente, $GLOBALS['abc'] acessará o valor em uma variável global $abc definida fora da função. Caso contrário, as variáveis que aparecem em uma função são locais mesmo que haja uma variável global com o mesmo nome.

A discussão anterior oferece uma breve introdução às funções da PHP. Muitos detalhes não foram discutidos, pois não é nosso objetivo apresentar a PHP em detalhes.

14.2.4 Variáveis e formulários de servidor PHP

Existem várias entradas embutidas em uma variável de autoglobal de vetor embutida da PHP, chamada $_SERVER, que podem oferecer ao programador informações úteis sobre o servidor onde o interpretador PHP está rodando, bem como outras informações. Estas podem ser necessárias quando se constrói o texto em um documento HTML (por exemplo, veja a linha 7 da Figura 14.4). Aqui estão algumas dessas entradas:

1. **$_SERVER['SERVER_NAME'].** Essa fornece o nome do Website do computador servidor onde o interpretador PHP está rodando. Por exemplo, se o interpretador PHP estiver rodando no Website <http://www.uta.edu>, então essa string seria o valor em $_SERVER['SERVER_NAME'].
2. **$_SERVER['REMOTE_ADDRESS'].** Esse é o endereço IP (Internet Protocol) do computador usuário do cliente que está acessando o servidor, por exemplo, 129.107.61.8.
3. **$_SERVER['REMOTE_HOST'].** Esse é o nome do site Web do computador usuário do cliente, por exemplo, abc.uta.edu. Nesse caso, o servidor precisará traduzir o nome para um endereço IP para acessar o cliente.
4. **$_SERVER['PATH_INFO'].** Essa é a parte do endereço URL que vem após a barra (/) ao final do URL.
5. **$_SERVER['QUERY_STRING'].** Isso fornece a string que mantém os parâmetros em um URL após o ponto de interrogação (?) ao final do URL. Isso pode manter parâmetros de pesquisa, por exemplo.
6. **$_SERVER['DOCUMENT_ROOT'].** Esse é o diretório raiz que mantém os arquivos no servidor Web que são acessíveis aos usuários clientes.

Estas e outras entradas no vetor$_SERVER normalmente são necessárias ao se criar o arquivo HTML a ser enviado para exibição.

Outra importante variável autoglobal de vetor embutida da PHP é $_POST. Esta oferece ao programador os valores de entrada submetidos pelo usuário por meio de formulários HTML especificados na tag HTML <INPUT> e outras tags semelhantes. Por exemplo, na linha 14 da Figura 14.4, a variável $_POST['nome_usuario'] oferece ao programador o valor digitado pelo usuário no formulário HTML especificado pela tag <INPUT> na linha 8. As chaves para esse vetor são os nomes dos diversos parâmetros de entrada fornecidos por meio do formulário, por exemplo, usando o atributo name da tag <INPUT> da HTML, como na linha 8. Quando os usuários inserem dados nos formulários, os valores de dados podem ser armazenados nesse vetor.

14.3 Visão geral da programação de banco de dados em PHP

Existem várias técnicas para acessar um banco de dados por meio de uma linguagem de programação. Discutimos algumas delas no Capítulo 13, nas visões gerais sobre como acessar um banco de dados SQL usando as linguagens de programação C e Java. Em particular, discutimos SQL embutida, JDBC, SQL/CLI (semelhante à ODBC) e SQLJ. Nesta seção, oferecemos um panorama de como acessar o banco de dados usando a linguagem de scripting PHP, que é bastante adequada para a criação de interfaces Web para busca e atualização de bancos de dados, bem como páginas Web dinâmicas.

Existe uma biblioteca de funções de acesso a banco de dados PHP que faz parte do PHP Extension and Application Repository (PEAR), uma coleção de várias bibliotecas de funções para melhorar a PHP. A biblioteca PEAR DB oferece funções para acesso a banco de dados. Muitos sistemas de banco de dados podem ser acessados por essa biblioteca, incluindo Oracle, MySQL, SQLite e Microsoft SQLServer, entre outros.

Discutiremos várias funções que fazem parte da PEAR DB no contexto de alguns exemplos. A Seção 14.3.1 mostra como se conectar a um banco de dados usando a PHP. A Seção 14.3.2 discute como os dados coletados de formulários HTML podem ser usados para inserir um novo registro em uma tabela (relação) de banco de dados. A Seção 14.3.3 mostra como consultas de recuperação podem ser executadas e ter seus resultados exibidos em uma página Web dinâmica.

14.3.1 Conectando a um banco de dados

Para usar as funções de banco de dados em um programa PHP, o módulo de biblioteca PEAR DB, chamado DB.php, precisa ser carregado. Na Figura 14.6, isso é feito na linha 0 do exemplo. As funções de biblioteca DB agora podem ser acessadas usando DB::<function_name>. A função para conectar a um banco de dados é chamada de DB::connect('string'), na qual o argumento de string especifica a informação de banco de dados. O formato para 'string' é:

<software SGBD>://<conta do usuário>:<senha>@<servidor de banco de dados>

Na Figura 14.6, a linha 1 conecta ao banco de dados que está armazenado usando Oracle (especificado por meio da string oci8). A parte <software SGBD> da 'string' especifica o pacote de software de SGBD em particular que está sendo conectado. Alguns dos pacotes de software SGBD acessíveis por meio do PEAR DB são:

- **MySQL.** Especificado como mysql para versões mais antigas e mysqli para versões mais recentes, começando com a versão 4.1.2.
- **Oracle.** Especificado como oc8i para as versões 7, 8 e 9. Este é usado na linha 1 da Figura 14.6.
- **SQLite.** Especificado como sqlite.
- **Microsoft SQL Server.** Especificado como mssql.
- **Mini SQL.** Especificado como msql.
- **Informix.** Especificado como ifx.
- **Sybase.** Especificado como sybase.
- **Qualquer sistema compatível com ODBC.** Especificado como odbc.

Esta não é uma lista completa.

Após o <software SGDB> no argumento de string passado a DB::connect está o separador ://, seguido pelo nome da conta do usuário <conta do usuário>, seguido pelo separador : e a senha da conta <senha>. Estes são seguidos pelo separador @ e o nome e diretório do servidor <servidor de banco de dados> em que o banco de dados está armazenado.

Na linha 1 da Figura 14.6, o usuário está se conectando ao servidor em <www.host.com/db1> usando o nome de conta conta1 e a senha senha12 armazenada sob o SGBD Oracle oci8. A string inteira é passada usando DB::connect. A informação de conexão é mantida na variável de conexão do banco de dados $d, que é usada sempre que uma operação para esse banco de dados em particular é aplicada.

```
0)  require 'DB.php';
1)  $d = DB::connect('oci8://conta1:senha12@www.host.com/db1');
2)  if (DB::isError($d)) { die("não pode conectar – " . $d->getMessage( ));}
    ...
3)  $q = $d->query("CREATE TABLE FUNCIONARIO
4)              (Func_id INT,
5)              Nome VARCHAR(15),
6)              Cargo VARCHAR(10),
7)              Dnr INT)" );
8)  if (DB::isError($q)) { die("criacao de tabela sem sucesso – " .
    $d->getMessage( )); }

9)  $d->setErrorHandling(PEAR_ERROR_DIE);
    ...
10) $fid = $d->nextID('FUNCIONARIO');
11) $d = $d->query("INSERT INTO FUNCIONARIO VALUES
12)             ($eid, $_POST['func_nome'], $_POST['func_cargo'], $_POST['‘func_dnr'])" );
    ...
13) $fid = $d->nextID('FUNCIONARIO');
14) $q = $d->query('INSERT INTO FUNCIONARIO VALUES (?, ?, ?, ?)',
15) vetor($fid, $_POST[func_nome'], $_POST['func_cargo'], $_POST['func_dnr']) );
```

Figura 14.6
Conectando a um banco de dados, criando uma tabela e inserindo um registro.

A linha 2 da Figura 14.6 mostra como verificar se a conexão com o banco de dados foi estabelecida com sucesso ou não. A PEAR DB tem uma função DB::isError, que pode determinar se qualquer operação de acesso ao banco de dados foi bem-sucedida ou não. O argumento para essa função é a variável de conexão de banco de dados ($d neste exemplo). Em geral, o programador PHP pode verificar após cada chamada ao banco de dados para determinar se a última operação do banco de dados teve sucesso ou não, e terminar o programa (usando a função die) se não tiver obtido sucesso. Uma mensagem de erro também é retornada do banco de dados por meio da operação $d->get_message( ). Esta também pode ser exibida como mostra a linha 2 da Figura 14.6.

Na maioria das vezes, muitos comandos SQL podem ser enviados ao banco de dados quando uma conexão é estabelecida por meio da função query. A função $d->query usa um comando SQL como seu argumento de string e o envia ao servidor de banco de dados para execução. Na Figura 14.6, as linhas 3 a 7 enviam um comando CREATE TABLE para criar uma tabela chamada FUNCIONARIO com quatro atributos. Sempre que uma consulta é executada, seu resultado é atribuído a uma variável de consulta, que é chamada $q em nosso exemplo. A linha 8 verifica se a consulta foi executada com sucesso ou não.

A biblioteca PEAR DB da PHP oferece uma alternativa para verificar erros após cada comando do banco de dados. A função

```
$d–>setErrorHandling(PEAR_ERROR_DIE)
```

terminará o programa e imprimirá as mensagens de erro padrão se quaisquer erros subsequentes ocorrerem ao acessar o banco de dados por meio da conexão $d (ver a linha 9 da Figura 14.6).

14.3.2 Coletando dados de formulários e inserindo registros

É comum, em aplicações de banco de dados, coletar informações por meio da HTML ou de outros tipos de formulários Web. Por exemplo, ao adquirir uma passagem aérea ou solicitar um cartão de crédito, o usuário precisa entrar com informações pessoais como nome, endereço e número de telefone. Essa informação normalmente é coletada e armazenada em um registro do banco de dados em um servidor.

As linhas 10 a 12 da Figura 14.6 ilustram como isso pode ser feito. Neste exemplo, omitimos o código para criar o formulário e coletar os dados, que pode ser uma variação do exemplo na Figura 14.1. Assumimos que o usuário inseriu valores válidos nos parâmetros de entrada chamados func_nome, func_cargo e func_dnr. Estes seriam acessíveis por meio do vetor autoglobal em PHP $_POST, conforme discutido no final da Seção 14.2.4.

No comando SQL INSERT mostrado nas linhas 11 e 12 da Figura 14.6, as entradas de vetor $_POST['func_nome'], $_POST['func_cargo'] e $_POST['func_dnr'] manterão os valores coletados do usuário por meio do formulário de entrada de HTML. Estes são então inseridos como um novo registro de funcionário na tabela FUNCIONARIO.

Esse exemplo também ilustra outro recurso da PEAR DB. É comum, em algumas aplicações, criar um identificador de registro exclusivo para cada novo registro inserido no banco de dados.[1]

PHP tem uma função $d->nextID para criar uma sequência de valores exclusivos para determinada tabela. Em nosso exemplo, o campo Func_id da tabela FUNCIONARIO (ver Figura 14.6, linha 4) é criado para essa finalidade. A linha 10 mostra como recuperar o próximo valor exclusivo na sequência para a tabela FUNCIONARIO e inseri-lo como parte do novo registro nas linhas 11 e 12.

O código para inserção nas linhas 10 a 12 da Figura 14.6 pode permitir que strings maliciosas sejam inseridas, podendo alterar o comando INSERT. Um modo mais seguro de realizar inserções e outras consultas é por meio do uso de **marcadores de lugar** (especificados pelo símbolo ?). Um exemplo é ilustrado nas linhas 13 a 15, onde outro registro deve ser inserido. Nessa forma da função $d->query( ), existem dois argumentos. O primeiro argumento é a instrução SQL, com um ou mais símbolos? (marcadores de lugar). O segundo argumento é um vetor, cujos valores de elemento serão usados para substituir os marcadores de lugar na ordem em que são especificados.

14.3.3 Consultas de recuperação de tabelas do banco de dados

Agora, oferecemos três exemplos de consultas de recuperação por meio da PHP, mostradas na Figura 14.7. As primeiras linhas, de 0 a 3, estabelecem uma conexão de banco de dados $d e definem o tratamento de erro para o default, conforme discutimos na seção anterior. A primeira consulta (linhas 4 a 7) recupera o nome e o número do departamento de todos os registros de funcionários. A variável de consulta $q é usada para se referir ao **resultado da consulta**. Um loop while para percorrer cada linha no resultado aparece nas linhas 5 a 7. A função $q->fetchRow( ) na linha 5 serve para recuperar o próximo registro no resultado da consulta e controlar o loop. O looping começa no primeiro registro.

O segundo exemplo de consulta aparece nas linhas 8 a 13 e ilustra uma consulta dinâmica. Nesta consulta, as condições para seleção de linhas

```
0)  require 'DB.php';
1)  $d = DB::connect('oci8://conta1:senha12@www.host.com/dbname');
2)  if (DB::isError($d)) { die("não pode conectar – " . $d->getMessage( )); }
3)  $d->setErrorHandling(PEAR_ERROR_DIE);
    ...
4)  $q = $d->query('SELECT Nome, Dnr FROM FUNCIONARIO');
5)  while ($r = $q->fetchRow( )) {
6)          print "funcionario $r[0] trabalha para o departamento $r[1] \n" ;
7)  }
    ...
8)  $q = $d->query('SELECT Nome FROM FUNCIONARIO WHERE Cargo = ? AND Dnr = ?',
9)          vetor($_POST['func_cargo'], $_POST['func_dnr']) );
10) print "funcionarios no dep $_POST['func_dnr'] cujo cargo é $_POST['func_cargo']: \n"
11) while ($r = $q->fetchRow( )) {
12) print "funcionario $r[0] \n" ;
13) }
    ...
14) $todos resultados = $d->getAll('SELECT Nome, Cargo, Dnr FROM FUNCIONARIO');
15) foreach ($allresult as $r) {
16)         print "funcionario $r[0] tem cargo $r[1] e trabalha para o departamento $r[2] \n" ;
17) }
    ...
```

Figura 14.7
Ilustrando as consultas de recuperação do banco de dados.

[1] Este seria semelhante ao OID gerado pelo sistema, discutido no Capítulo 11, para sistemas de banco de dados de objeto e objeto-relacional.

são baseadas nos valores inseridos pelo usuário. Aqui, queremos recuperar os nomes dos funcionários que têm um cargo específico e trabalham para determinado departamento. O cargo e número de departamento em particular são inseridos por um formulário nas variáveis de vetor $POST['func_cargo'] e $POST['func_dnr']. Se o usuário tivesse inserido 'Engenheiro' para o cargo e 5 para o número do departamento, a consulta selecionaria os nomes de todos os engenheiros que trabalharam no departamento 5. Como podemos ver, essa é uma consulta dinâmica, cujos resultados diferem dependendo das escolhas que o usuário informa como entrada. Usamos dois marcadores de lugar ? neste exemplo, conforme discutido no final da Seção 14.3.2.

A última consulta (linhas 14 a 17) mostra uma forma alternativa de especificar uma consulta e percorrer suas linhas. Neste exemplo, a função $d->getAll mantém todos os registros de um resultado da consulta em uma única variável, chamada $todos_resultados. Para percorrer os registros individuais, um loop foreach pode ser usado, com a variável de linha $r percorrendo cada linha em $todos_resultados.[2]

Como podemos ver, a PHP é adequada tanto para acesso a banco de dados quanto para criação de páginas Web dinâmicas.

Resumo

Neste capítulo, tivemos uma visão geral de como converter alguns dados estruturados de bancos de dados para elementos a serem inseridos ou exibidos em uma página Web. Focalizamos a linguagem de scripting PHP, que está se tornando muito popular para a programação de banco de dados na Web. A Seção 14.1 apresentou alguns fundamentos de PHP para programação na Web por meio de um exemplo simples. A Seção 14.2 mostrou alguns dos fundamentos da linguagem PHP, incluindo seus tipos de dados de vetor e string, que são bastante utilizados. A Seção 14.3 apresentou um panorama de como a PHP pode ser usada para especificar diversos tipos de comandos de banco de dados, incluindo a criação de tabelas, inserção de novos registros e recuperação de registros de banco de dados. A PHP roda no computador servidor, em comparação com algumas outras linguagens de scripting, que rodam no computador cliente.

Fizemos apenas uma introdução muito básica à PHP. Existem muitos livros, além de sites Web, dedicados à programação PHP introdutória e avançada. Também existem diversas bibliotecas de funções para PHP, pois esse é um produto de fonte aberto.

Perguntas de revisão

14.1. Por que as linguagens de scripting são populares para a programação de aplicações Web? Onde, na arquitetura de três camadas, um programa em PHP é executado? Onde um programa em JavaScript é executado?

14.2. Que tipo de linguagem de programação é a PHP?

14.3. Discuta as diferentes maneiras de especificar strings em PHP.

14.4. Discuta os diferentes tipos de vetores em PHP.

14.5. O que são variáveis autoglobais da PHP? Dê alguns exemplos de vetores autoglobais em PHP e discuta como cada um costuma ser usado.

14.6. O que é PEAR? O que é PEAR DB?

14.7. Discuta as principais funções para acessar um banco de dados em PEAR DB e como cada uma é usada.

14.8. Discuta as diferentes maneiras de realizar um looping durante um resultado de consulta em PHP.

14.9. O que são marcadores de lugar? Como eles são usados na programação de banco de dados em PHP?

Exercícios

14.10. Considere o esquema de banco de dados BIBLIOTECA da Figura 4.6. Escreva um código em PHP para criar as tabelas desse esquema.

14.11. Escreva um programa em PHP que crie formulários Web para a entrada de informações sobre uma nova entidade USUARIO. Repita para uma nova entidade LIVRO.

14.12. Escreva interfaces Web em PHP para as consultas especificadas no Exercício 6.18.

Bibliografia selecionada

Existem muitas fontes para programação PHP, tanto impressas quanto na Web. Indicamos dois livros como exemplos. Uma introdução bastante boa à PHP é dada em Sklar (2005). Para o desenvolvimento avançado de site Web, o livro de Schlossnagle (2005) oferece muitos exemplos detalhados.

[2] A variável $r é semelhante aos cursores e variáveis de iteração discutidos nos capítulos 11 e 13.



Teoria e normalização de projeto de banco de dados

15 Fundamentos de dependências funcionais e normalização para bancos de dados relacionais

Nos capítulos 3 a 6, apresentamos diversos aspectos do modelo relacional e as linguagens associadas a ele. Cada *esquema de relação* consiste em uma série de atributos, e o *esquema de banco de dados relacional* consiste em uma série de esquemas de relação. Até aqui, assumimos que os atributos são agrupados para formar um esquema de relação usando o bom senso do projetista de banco de dados ou mapeando um projeto de esquema de banco de dados com base no modelo de dados conceitual, como o modelo de dados ER ou ER Estendido (EER). Esses modelos fazem o projetista identificar os tipos de entidade e de relacionamento e seus respectivos atributos, o que leva a um agrupamento natural e lógico dos atributos em relações quando os procedimentos de mapeamento discutidos no Capítulo 9 são seguidos. Porém, ainda precisamos de algum modo formal de análise porque um agrupamento de atributos em um esquema de relação pode ser melhor do que outro. Ao discutir o projeto de banco de dados nos capítulos 7 a 10, não desenvolvemos nenhuma medida de adequação ou *boas práticas* para medir a qualidade do projeto, além da intuição do projetista. Neste capítulo, vamos discutir parte da teoria que foi desenvolvida com o objetivo de avaliar esquemas relacionais para a qualidade do projeto — ou seja, para medir formalmente por que um conjunto de agrupamentos de atributos em esquemas de relação é melhor do que outro.

Existem dois níveis em que podemos discutir as boas práticas de esquemas de relação. O primeiro é o **nível lógico** (ou **conceitual**) — como os usuários interpretam os esquemas de relação e o significado de seus atributos. Ter bons esquemas de relação nesse nível permite que os usuários entendam claramente o significado dos dados nas relações, e daí formulem suas consultas corretamente. O segundo é o **nível de implementação** (ou **armazenamento físico**) — como as tuplas em uma relação da base são armazenadas e atualizadas. Esse nível se aplica apenas a esquemas das relações da base — que serão fisicamente armazenadas como arquivos —, enquanto no nível lógico estamos interessados em esquemas de relações da base e visões (relações virtuais). A teoria de projeto de banco de dados relacional desenvolvida neste capítulo se aplica principalmente a *relações da base*, embora alguns critérios de adequação também se apliquem a visões, como mostra a Seção 15.1.

Assim como em muitos problemas de projeto, o projeto de banco de dados pode ser realizado usando duas técnicas: de baixo para cima (bottom-up) ou de cima para baixo (top-down). Uma **metodologia de projeto de baixo para cima** (também chamada *projeto por síntese*) considera os relacionamentos básicos *entre atributos individuais* como ponto de partida e os utiliza para construir esquemas de relação. Essa técnica não é muito popular na prática,[1] pois sofre

[1] Uma exceção em que essa técnica é usada na prática é baseada em um modelo chamado *modelo relacional binário*. Um exemplo é a metodologia NIAM (Verheijen e VanBekkum, 1982).

do problema de ter que coletar um grande número de relacionamentos binários entre atributos como ponto de partida. Para situações práticas, é quase impossível capturar relacionamentos binários entre todos esses pares de atributos. Ao contrário, uma **metodologia de projeto de cima para baixo** (também chamada *projeto por análise*) começa com uma série de agrupamentos de atributos em relações que existem naturalmente juntas, por exemplo, em uma fatura, formulário ou relatório. As relações são então analisadas individual e coletivamente, levando a mais decomposição, até que todas as propriedades desejáveis sejam atendidas. A teoria descrita neste capítulo se aplica às técnicas de projeto de cima para baixo e de baixo par cima, mas é mais apropriada quando usada com a técnica de cima para baixo.

O projeto de banco de dados relacional por fim produz um conjunto de relações. Os objetivos implícitos da atividade de projeto são *preservação da informação* e *redundância mínima*. A informação é muito difícil de se quantificar — logo, consideramos a preservação de informação em matéria de manutenção de todos os conceitos, incluindo tipos de atributo, tipos de entidade e tipos de relacionamento, bem como os relacionamentos de generalização/especialização, que são descritos usando um modelo como o EER. Assim, o projeto relacional precisa preservar todos esses conceitos, que são capturados srcinalmente no projeto conceitual após o mapeamento do projeto conceitual para lógico. A minimização da redundância implica diminuir o armazenamento redundante da mesma informação e reduzir a necessidade de múltiplas atualizações para manter a consistência entre diversas cópias da mesma informação, em resposta a eventos do mundo real que exijam fazer uma atualização.

Começamos este capítulo discutindo informalmente alguns critérios para esquemas de relação bons e ruins na Seção 15.1. Na Seção 15.2, definimos o conceito de *dependência funcional*, uma restrição formal entre os atributos que é a principal ferramenta para medir formalmente a adequação dos agrupamentos de atributo em esquemas de relação. Na Seção 15.3, vamos discutir as formas normais e o processo de normalização usando dependências funcionais. As formas normais sucessivas são definidas para reduzir a incidência de certas dependências indesejáveis, expressas com dependências funcionais. O projeto de normalização consiste em aplicar uma série de testes às relações para atender a esses requisitos cada vez mais rígidos e decompor as relações quando necessário. Na Seção 15.4, discutimos definições mais gerais das formas normais, que podem ser aplicadas diretamente a qualquer projeto dado e não exigem análise e normalização passo a passo. As seções 15.5 a 15.7 discutem outras formas normais, até a quinta forma normal. Na Seção 15.6, apresentamos a dependência multivalorada (MVD), seguida pela dependência de junção (DJ) na Seção 15.7. No final há um resumo do capítulo.

O Capítulo 16 continuará o desenvolvimento da teoria relacionada ao projeto de bons esquemas relacionais. Discutimos as propriedades desejáveis da decomposição relacional — propriedade de junção não aditiva e propriedade de preservação da dependência funcional. Um algoritmo geral testa se uma decomposição tem ou não a propriedade de junção não aditiva (ou sem perdas). Depois, abordamos algoritmos projetados para satisfazer propriedades de dependências funcionais e o conceito de uma cobertura mínima de dependências. Consideramos a técnica de baixo para cima para o projeto de banco de dados que consiste em um conjunto de algoritmos para projetar relações em uma forma normal desejada. Esses algoritmos consideram como entrada determinado conjunto de dependências funcionais e alcançam um projeto relacional em uma forma normal de destino, enquanto aderem às propriedades desejáveis acima. No Capítulo 16, também definimos outros tipos de dependências que melhoram ainda mais a avaliação das *boas práticas* de esquemas de relação.

Se o Capítulo 16 não for incluído no curso, recomendamos uma rápida introdução às propriedades desejáveis de decomposição e à discussão da Propriedade NJB da Seção 16.2.

15.1 Diretrizes de projeto informais para esquemas de relação

Antes de discutirmos a teoria formal do projeto de banco de dados relacional, vamos abordar quatro *diretrizes informais* que podem ser usadas como *medidas para determinar a qualidade* de projeto do esquema da relação:

- Garantir que a semântica dos atributos seja clara no esquema.
- Reduzir a informação redundante nas tuplas.
- Reduzir os valores NULL nas tuplas.
- Reprovar a possibilidade de gerar tuplas falsas.

Essas medidas nem sempre são independentes uma da outra, conforme veremos.

15.1.1 Comunicando uma semântica clara aos atributos nas relações

Sempre que agrupamos atributos para formar um esquema de relação, consideramos que aqueles atributos pertencentes a uma relação têm certo significado no mundo real e uma interpretação apro-

priada associada a eles. A **semântica** de uma relação refere-se a seu significado resultante da interpretação dos valores de atributo em uma tupla. No Capítulo 3, discutimos como uma relação pode ser interpretada como um conjunto de fatos. Se o projeto conceitual descrito nos capítulos 7 e 8 for feito cuidadosamente e o procedimento de mapeamento do Capítulo 9 for seguido de maneira sistemática, o projeto do esquema relacional deverá ter um significado claro.

Em geral, quanto mais fácil for explicar a semântica da relação, melhor será o projeto do esquema de relação. Para ilustrar isso, considere a Figura 15.1, uma versão simplificada do esquema de banco de dados relacional EMPRESA da Figura 3.5, e a Figura 15.2, que apresenta um exemplo de estados de relação preenchidos desse esquema. O significado do esquema de relação FUNCIONARIO é muito simples: cada tupla representa um funcionário, com valores para o nome do funcionário (Fnome), número do Cadastro de Pessoa Física (Cpf), data de nascimento (Datanasc), endereço (Endereco) e o número do departamento para o qual o funcionário trabalha (Dnumero). O atributo Dnumero é uma chave estrangeira que representa um *relacionamento implícito* entre FUNCIONARIO e DEPARTAMENTO. A semântica dos esquemas DEPARTAMENTO e PROJETO é muito simples: cada tupla DEPARTAMENTO representa uma entidade de departamento, e cada tupla PROJETO representa uma entidade de projeto. O atributo Cpf_gerente de DEPARTAMENTO relaciona um departamento ao funcionário que é seu gerente, enquanto Dnum de PROJETO relaciona um projeto a seu departamento de controle; ambos são atributos de chave estrangeira. A facilidade com que o significado dos atributos de uma relação pode ser explicado é uma *medida informal* de quão bem a relação está projetada.

A semântica dos outros dois esquemas de relação da Figura 15.1 é ligeiramente mais complexa. Cada tupla em LOCALIZACAO_DEP gera um número de departamento (Dnumero) e *um dos* locais do departamento (Dlocalizacao). Cada tupla em TRABALHA_EM gera um número de Cadastro de Pessoa Física (Cpf), o número de projeto de *um dos* projetos em que o funcionário trabalha (Projnumero) e o número de horas por semana que o funcionário trabalha nesse projeto (Horas). Porém, os dois esquemas têm uma interpretação bem definida e não ambígua. O esquema LOCALIZACAO_DEP representa um atributo multivalorado de DEPARTAMENTO, enquanto TRABALHA_EM representa um relacionamento M:N entre FUNCIONARIO e PROJETO. Logo, todos os esquemas de relação na Figura 15.1 podem ser considerados fáceis de explicar e, portanto, bons do ponto de vista de ter uma semântica clara.

Assim, podemos formular as seguintes diretrizes de projeto informal.

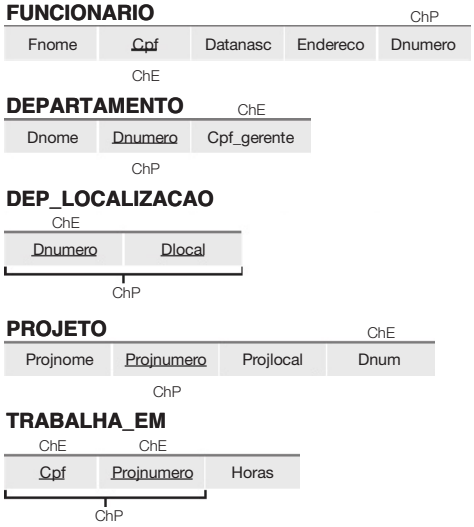


Figura 15.1
Um esquema de banco de dados relacional EMPRESA simplificado.

Diretriz 1

Projete um esquema de relação de modo que seja fácil explicar seu significado. Não combine atributos de vários tipos de entidade e de relacionamento em uma única relação. Intuitivamente, se um esquema de relação corresponde a um tipo de entidade ou um tipo de relacionamento, é simples interpretar e explicar seu significado. Caso contrário, se a relação corresponder a uma mistura de várias entidades e relacionamentos, haverá ambiguidades semânticas e a relação não poderá ser explicada com facilidade.

Exemplos de violação da diretriz 1. Os esquemas de relação das figuras 15.3(a) e 15.3(b) também têm semântica clara. (O leitor deve ignorar as linhas sob as relações por enquanto; elas são usadas para ilustrar a notação da dependência funcional, discutida na Seção 15.2.) Uma tupla no esquema de relação FUNC_DEP na Figura 15.3(a) representa um único funcionário, mas inclui informações adicionais — a saber, o nome (Dnome) do departamento para o qual o funcionário trabalha e o número do Cadastro de Pessoa Física (Cpf_gerente) do gerente de departamento. Para a relação FUNC_PROJ da Figura

FUNCIONARIO

Fnome	Cpf	Datanasc	Endereco	Dnumero
Silva,JoaoB.	12345678966	09-01-1965	RuadasFlores,751,SãoPaulo,SP	5
Wong,FernandoT.	33344555587	08-12-1955	RuadaLapa,34,SãoPaulo,SP	5
Zelaya,AliceJ.	99988777767	19-01-1968	RuaSouzaLima,35,Curitiba,PR	4
Souza,JenniferS.	98765432168	20-06-1941	Av.ArthurdeLima,54,SantoAndré,SP	4
Lima,RonaldoK.	66688444476	15-09-1962	RuaRebouças,65,Piracicaba,SP	5
Leite,JoiceA.	45345345376	31-07-1972	Av.LucasObes,74,SãoPaulo,SP	5
Pereira,AndréV.	98798798733	29-03-1969	RuaTimbira,35,SãoPaulo,SP	4
Brito,JorgeE.	88866555576	10-11-1937	RuadoHorto,35,SãoPaulo,SP	1

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente
Pesquisa	5	33344555587
Administração	4	98765432168
Matriz	1	88866555576

LOCALIZACAO_DEP

Dnumero	Dlocal
1	SãoPaulo
4	Mauá
5	SantoAndré
5	Itu
5	SãoPaulo

TRABALHA_EM

Cpf	Projnumero	Horas
12345678966	1	32,5
12345678966	2	7,5
66688444476	3	40,0
45345345376	1	20,0
45345345376	2	20,0
33344555587	2	10,0
33344555587	3	10,0
33344555587	10	10,0
33344555587	20	10,0
99988777767	30	30,0
99988777767	10	10,0
98798798733	10	35,0
98798798733	30	5,0
98765432168	30	20,0
98765432168	20	15,0
88866555576	20	NULL

PROJETO

Projnome	Projnumero	Projlocal	Dnum
ProdutoX	1	SantoAndré	5
ProdutoY	2	Itu	5
ProdutoZ	3	SãoPaulo	5
informatização	10	Mauá	4
Reorganização	20	SãoPaulo	1
Novosbenefícios	30	Mauá	4

Figura 15.2
Exemplo de estado de banco de dados para o esquema relacional da Figura 15.1.

15.3(b), cada tupla relaciona um funcionário a um projeto, mas também inclui o nome do funcionário (Fnome), nome do projeto (Projnome) e local do projeto (Projlocal). Embora não haja nada logicamente errado logicamente com essas duas relações, elas violam a Diretriz 1 ao misturar atributos de entidades distintas do mundo real: FUNC_DEP mistura atributos dos funcionários e departamentos, e FUNC_PROJ mistura atributos de funcionários e projetos e o relacionamento TRABALHA_EM. Logo, elas se saem mal contra a medida de qualidade de projeto citado. Elas podem ser usadas como visões, mas causam problemas quando utilizadas como relações da base, conforme discutiremos na próxima seção.

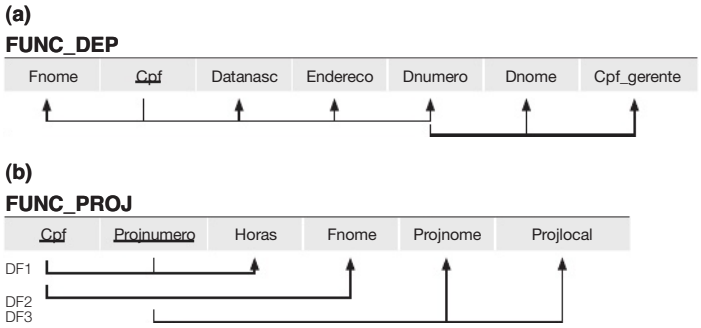


Figura 15.3
Dois esquemas de relação sofrendo de anomalias de atualização. (a) FUNC_DEP e (b) FUNC_PROJ.

15.1.2 Informação redundante nas tuplas e anomalias de atualização

Um objetivo do projeto de esquema é minimizar o espaço de armazenamento usado pelas relações (e, portanto, pelos arquivos correspondentes). O agrupamento de atributos em esquemas de relação tem um efeito significativo no espaço de armazenamento. Por exemplo, compare o espaço usado pelas duas relações da base FUNCIONARIO e DEPARTAMENTO da Figura 15.2 com o da relação da base FUNC_DEP da Figura 15.4, que é o resultado da aplicação da operação JUNCAO NATURAL em FUNCIONARIO e DEPARTAMENTO. Em FUNC_DEP, os valores de atributo pertencentes a determinado departamento (Dnumero, Dnome, Cpf_gerente) são repetidos para *cada funcionário que trabalha para esse departamento*. Ao contrário, a informação de cada departamento aparece apenas uma vez na relação DEPARTAMENTO da Figura 15.2. Somente o número do departamento (Dnumero) é repetido na relação FUNCIONARIO para cada funcionário que trabalha nesse departamento, como uma chave estrangeira. Comentários semelhantes se aplicam à relação FUNC_PROJ (ver Figura 15.4), que aumenta a relação TRABALHA_EM com atributos adicionais de FUNCIONARIO e PROJETO.

O armazenamento de junções naturais de relações da base leva a um problema adicional conhecido como **anomalias de atualização**. Estas podem ser classificadas em anomalias de inserção, anomalias de exclusão e anomalias de modificação.[2]

Anomalias de inserção. As anomalias de inserção podem ser diferenciadas em dois tipos, ilustrados pelos seguintes exemplos baseados na relação FUNC_DEP:

- Para inserir uma nova tupla de funcionário em FUNC_DEP, temos de incluir ou os valores de atributo do departamento para o qual o funcionário trabalha ou NULLs (se o funcionário ainda não trabalha para nenhum departamento). Por exemplo, para inserir uma nova tupla para um funcionário que trabalha no departamento 5, temos de inserir todos os valores de atributo do departamento 5 corretamente, de modo que eles sejam *coerentes* com os valores correspondentes para o departamento 5 em outras duplas de FUNC_DEP. No projeto da Figura 15.2, não temos de nos preocupar com esse problema de coerência, pois entramos apenas com o número do departamento na tupla do funcionário. Todos os outros valores de atributo do departamento 5 são registrados apenas uma vez no banco de dados, como uma única tupla na relação DEPARTAMENTO.
- É difícil inserir um novo departamento que ainda não tenha funcionários na relação FUNC_DEP. A única maneira de fazer isso é colocar valores NULL nos atributos para funcionário. Isso viola a integridade de entidade para FUNC_DEP, porque Cpf é sua chave primária. Além do mais, quando o primeiro funcionário é atribuído a esse departamento, não precisamos mais dessa tupla com valores NULL. Esse problema não ocorre no projeto da Figura 15.2, visto que um departamento é inserido na relação DEPARTAMENTO independentemente de haver ou não funcionários trabalhando para ele, e sempre que um funcionário é atribuído a esse departamento, uma tupla correspondente é inserida em FUNCIONARIO.

[2] Essas anomalias foram identificadas por Codd (1972a) para justificar a necessidade de normalização das relações, conforme discutiremos na Seção 15.3.

Redundância

FUNC_DEP

Fnome	Cpf	Datanasc	Endereco	Dnumero	Dnome	Cpf_gerente
Silva,JoãoB.	12345678966	09-01-1965	RuadasFlores,751,SãoPaulo,SP	5	Pesquisa	33344555587
Wong,FernandoT.	33344555587	08-12-1955	RuadaLapa,34,SãoPaulo,SP	5	Pesquisa	33344555587
Zelaya,AliceJ.	99988777767	19-01-1968	RuaSouzaLima,35,Curitiba,PR	4	Administração	98765432168
Souza,JenniferS.	98765432168	20-06-1941	Av.ArthurdeLima,54,SantoAndré,SP	4	Administração	98765432168
Lima,RonaldoK.	66688444476	15-09-1962	RuaRebouças,65,Piracicaba,SP	5	Pesquisa	33344555587
Leite,JoiceA.	45345345376	31-07-1972	Av.LucasObes,74,SãoPaulo,SP	5	Pesquisa	33344555587
Pereira,AndréV.	98798798733	29-03-1969	RuaTimbira,35,SãoPaulo,SP	4	Administração	98765432168
Brito,JorgeE.	88866555576	10-11-1937	RuadoHorto,35,SãoPaulo,SP	1	Matriz	88866555576

Redundância Redundância

FUNC_PROJ

Cpf	Projnumero	Horas	Fnome	Projnome	Projlocal
12345678966	1	32,5	Silva,JoãoB.	ProdutoX	SantoAndré
12345678966	2	7,5	Silva,JoãoB.	ProdutoY	Itu
66688444476	3	40,0	Lima,RonaldoK.	ProdutoZ	SãoPaulo
45345345376	1	20,0	Leite,JoiceA.	ProdutoX	SantoAndré
45345345376	2	20,0	Leite,JoiceA.	ProdutoY	Itu
33344555587	2	10,0	Wong,FernandoT.	ProdutoY	Itu
33344555587	3	10,0	Wong,FernandoT.	ProdutoZ	SãoPaulo
33344555587	10	10,0	Wong,FernandoT.	Informatização	Mauá
33344555587	20	10,0	Wong,FernandoT.	Reorganização	SãoPaulo
99988777767	30	30,0	Zelaya,AliceJ.	Novosbenefícios	Mauá
99988777767	10	10,0	Zelaya,AliceJ.	Informatização	Mauá
98798798733	10	35,0	Pereira,AndréV.	Informatização	Mauá
98798798733	30	5,0	Pereira,AndréV.	Novosbenefícios	Mauá
98765432168	30	20,0	Souza,JenniferS.	Novosbenefícios	Mauá
98765432168	20	15,0	Souza,JenniferS.	Reorganização	SãoPaulo
88866555576	20	Null	Brito,JorgeE.	Reorganização	SãoPaulo

Figura 15.4

Exemplos de estados para FUNC_DEP e FUNC_PROJ resultando na aplicação da JUNÇÃO NATURAL às relações da Figura 15.2. Estas podem ser armazenadas como relações da base por questões de desempenho.

Anomalias de exclusão. O problema das anomalias de exclusão está relacionado à segunda situação de anomalia de inserção que acabamos de discutir. Se excluirmos de FUNC_DEP uma tupla de funcionário que represente o último funcionário trabalhando para determinado departamento, a informação referente a esse departamento se perde do banco de dados. Esse problema não ocorre no banco de dados da Figura 15.2, pois as tuplas de DEPARTAMENTO são armazenadas separadamente.

Anomalias de modificação. Em FUNC_DEP, se mudarmos o valor de um dos atributos de determinado departamento — digamos, o gerentedo departamento 5 —, temos de atualizar as tuplas de*todos* os funcionários que trabalham nesse departamento; caso contrário, o banco de dados ficará incoerente. Se deixarmos de atualizar algumas tuplas, o mesmo departamento mostrará dois valores diferentes para o gerente em diferentes tuplas de funcionário, o que seria errado.

É fácil ver que essas três anomalias são indesejáveis e causam dificuldades para manter a coerência dos dados, bem como exigem atualizações desnecessárias que podem ser evitadas; logo, podemos declarar a próxima diretriz como se segue.

[3] Este não é tão sério quanto os outros problemas, pois todas as tuplas podem ser atualizadas por uma única consulta SQL.

Diretriz 2

Projete os esquemas de relação da base de modo que nenhuma anomalia de inserção, exclusão ou modificação esteja presente nas relações. Se houver alguma anomalia,[4] anote-as claramente e cuide para que os programas que atualizam o banco de dados operem corretamente.

A segunda diretriz é coerente com e, de certa forma, é uma reafirmação da primeira diretriz. Também podemos ver a necessidade de uma técnica mais formal para avaliar se um projeto atende a essas diretrizes. As seções 15.2 a 15.4 oferecem esses conceitos formais necessários. É importante observar que essas diretrizes às vezes *podem ter de ser violadas* a fim de *melhorar o desempenho* de certas consultas. Se FUNC_DEP for usado como uma relação armazenada (conhecida de outra forma como uma *visão materializada*) além das relações da base de FUNCIONARIO e DEPARTAMENTO, as anomalias em FUNC_DEP precisam ser observadas e consideradas (por exemplo, usando triggers ou procedimentos armazenados que fariam atualizações automáticas). Desse modo, sempre que a relação da base é atualizada, não ficamos com incoerências. Em geral, é aconselhável usar relações da base sem anomalias e especificar visões que incluem as junções para reunir os atributos frequentemente referenciados nas consultas importantes.

15.1.3 Valores NULL nas tuplas

Em alguns projetos de esquema, podemos agrupar muitos atributos em uma relação 'gorda'. Se muitos dos atributos não se aplicarem a todas as tuplas na relação, acabamos com muitos NULLs nessas tuplas. Isso pode desperdiçar espaço no nível de armazenamento e também ocasionar problemas com o conhecimento do significado dos atributos e com a especificação de operações JUNÇÃO no nível lógico.[5] Outro problema com NULLs é como considerá-los quando operações de agregação como CONTA ou SOMA são aplicadas. Operações SELEÇÃO e JUNÇÃO envolvem comparações; se valores NULL estiverem presentes, os resultados podem se tornar imprevisíveis.[6] Além do mais, os NULLs podem ter várias interpretações, como as seguintes:

- O atributo *não se aplica* a essa tupla. Por exemplo, Estado_visto pode não se aplicar a alunos do Brasil.
- O valor do atributo para essa tupla é *desconhecido*. Por exemplo, a Data_nascimento pode ser desconhecida para um funcionário.
- O valor é *conhecido, mas ausente*; ou seja, ele ainda não foi registrado. Por exemplo, o Numero_telefone_residencial para um funcionário pode existir, mas ainda não estar disponível e registrado.

Ter a mesma representação para todos os NULLs compromete os diferentes significados que eles podem ter. Portanto, podemos declarar outra diretriz.

Diretriz 3

Ao máximo possível, evite colocar atributos em uma relação da base cujos valores podem ser NULL com frequência. Se os NULLs forem inevitáveis, garanta que eles se apliquem apenas em casos excepcionais, e não à maioria das tuplas na relação.

Usar o espaço de modo eficaz e evitar junções com valores NULL são os dois critérios prioritários que determinam a inclusão das colunas que podem ter NULLs em uma relação ou que podem ter uma relação separada para essas colunas (com as colunas de chave apropriadas). Por exemplo, se apenas 15 por cento dos funcionários têm escritórios individuais, há pouca justificativa para incluir um atributo Numero_escritorio na relação FUNCIONARIO. Em vez disso, uma relação FUNC_ESCRITORIO (Cpf, Numero_escritorio) pode ser criada para incluir tuplas apenas para funcionários com escritórios individuais.

15.1.4 Geração de tuplas falsas

Considere os esquemas de duas relações FUNC_LOCAL e FUNC_PROJ1 da Figura 15.5(a), que podem ser usados no lugar da única relação FUNC_PROJ da Figura 15.3(b). Uma tupla em FUNC_LOCAL significa que o funcionário cujo nome é Fnome trabalha em *algum projeto* cujo local é Projlocal. Uma tupla em FUNC_PROJ1 refere-se ao fato de o funcionário cujo número de Cadastro de Pessoa Física é Cpf trabalhar Horas por semana no projeto cujo nome, número e localização são Projnome, Projnumero e Projlocal. A Figura 15.5(b) mostra os estados da relação de FUNC_LOCAL e FUNC_PROJ1 correspondentes à relação FUNC_PROJ da Figura 15.4, que são obtidos aplicando as operações PROJETO (π) apropriadas a FUNC_PROJ (ignore as linhas tracejadas na Figura 15.5(b) por enquanto).

[4] Outras considerações de aplicação podem determinar e tornar certas anomalias inevitáveis. Por exemplo, a relação FUNC_DEP pode corresponder a uma consulta ou a um relatório que é exigido com frequência.

[5] Isso porque as junções interna e externa produzem diferentes resultados quando NULLs são envolvidos nas junções. Assim, os usuários precisam estar cientes dos diferentes significados dos vários tipos de junções. Embora isso seja razoável para usuários sofisticados, pode ser difícil para outros.

[6] Na Seção 5.5.1, apresentamos comparações envolvendo valores NULL onde o resultado (na lógica de três valores) é TRUE, FALSE e UNKNOWN.

(a)

FUNC_LOCAL

Fnome	Projlocal

ChP

(b)

FUNC_PROJ1

Cpf	Projnumero	Horas	Projnome	Projlocal

ChP

(c)

FUNC_LOCAL

Fnome	Projlocal
Silva, João B.	Santo André
Silva, João B.	Itu
Lima, Ronaldo K.	São Paulo
Leite, Joice A.	Santo André
Leite, Joice A.	Itu
Wong, Fernando T.	Itu
Wong, Fernando T.	São Paulo
Wong, Fernando T.	Mauá
Zelaya, Alice J.	Mauá
Pereira, André V.	Mauá
Souza, Jennifer S.	Mauá
Souza, Jennifer S.	São Paulo
Brito, Jorge E.	São Paulo

FUNC_PROJ1

Cpf	Projnumero	Horas	Projnome	Projlocalizacao
12345678966	1	32,5	ProdutoX	Santo André
12345678966	2	7,5	ProdutoY	Itu
66688444476	3	40,0	ProdutoZ	São Paulo
45345345376	1	20,0	ProdutoX	Santo André
45345345376	2	20,0	ProdutoY	Itu
33344555587	2	10,0	ProdutoY	Itu
33344555587	3	10,0	ProdutoZ	São Paulo
33344555587	10	10,0	Computadorização	Mauá
33344555587	20	10,0	Reorganização	São Paulo
99988777767	30	30,0	Novosbenefícios	Mauá
99988777767	10	10,0	Computadorização	Mauá
98765432168	10	35,0	Computadorização	Mauá
98765432168	30	5,0	Novosbenefícios	Mauá
98765432168	30	20,0	Novosbenefícios	Mauá
98798798733	20	15,0	Reorganização	São Paulo
88866555576	20	NULL	Reorganização	São Paulo

Figura 15.5
Projeto particularmente fraco para a relação FUNC_PROJ da Figura 15.3(b). (a) Esquemas de duas relações FUNC_LOCAL e FUNC_PROJ1. (b) Resultado da projeção da extensão de FUNC_PROJ da Figura 15.4 para as relações FUNC_LOCAL e FUNC_PROJ1.

Suponha que usamos FUNC_PROJ1 e FUNC_LOCAL como relações da base em vez de FUNC_PROJ. Isso produz um projeto de esquema particularmente ruim, pois não podemos recuperar a informação que havia srcinalmente em FUNC_PROJ de FUNC_PROJ1 e FUNC_LOCAL. Se tentarmos uma operação JUNÇÃO NATURAL sobre FUNC_PROJ1 e FUNC_LOCAL, o resultado produz muito mais tuplas do que o conjunto srcinal de tuplas em FUNC_PROJ. Na Figura 15.6, mostramos o resultado da aplicação da junção apenas às tuplas *acima* das linhas tracejadas da Figura 15.5(b) (para reduzir o tamanho da relação resultante). Tuplas adicionais, que não estavam em FUNC_PROJ, são chamadas de **tuplas falsas**, pois representam informação falsa, que não é válida. As tuplas falsas são marcadas com asteriscos (*) na Figura 15.6.

A decomposição de FUNC_PROJ em FUNC_LOCAL e FUNC_PROJ1 é indesejável porque, quando as juntamos (JUNÇÃO) de volta usando JUNÇÃO NATURAL, não obtemos a informação srcinal correta. Isso porque, neste caso, Projlocal é o atributo que relaciona FUNC_LOCAL e FUNC_PROJ1, e Projlocal não é a chave primária nem uma chave estrangeira em FUNC_LOCAL ou FUNC_PROJ1. Agora podemos declarar informalmente outra diretriz de projeto.

Diretriz 4

Projete esquemas de relação de modo que possam ser unidos com condições de igualdade sobre os atributos que são pares relacionados corretamente (chave primária, chave estrangeira) de um modo que garanta que nenhuma tupla falsa será gerada. Evi-

Cpf	Projnumero	Horas	Projnome	Projlocal	Fnome
12345678966	1	32,5	ProdutoX	SantoAndré	SilvaJoãoB.
*12345678966	1	32,5	ProdutoX	SantoAndré	LeiteJoiceA.
12345678966	2	7,5	ProdutoY	Itu	SilvaJoãoB.
*12345678966	2	7,5	ProdutoY	Itu	LeiteJoiceA.
*12345678966	2	7,5	ProdutoY	Itu	WongFernandoT.
66688444476	3	40,0	ProdutoZ	SãoPaulo	LimaRonaldoK.
*66688444476	3	40,0	ProdutoZ	SãoPaulo	WongFernandoT.
*45345345376	1	20,0	ProdutoX	SantoAndré	SilvaJoãoB.
45345345376	1	20,0	ProdutoX	SantoAndré	LeiteJoiceA.
*45345345376	2	20,0	ProdutoY	Itu	SilvaJoãoB.
45345345376	2	20,0	ProdutoY	Itu	LeiteJoiceA.
*45345345376	2	20,0	ProdutoY	Itu	WongFernandoT.
*33344555587	2	10,0	ProdutoY	Itu	SilvaJoãoB.
*33344555587	2	10,0	ProdutoY	Itu	LeiteJoiceA.
33344555587	2	10,0	ProdutoY	Itu	WongFernandoT.
*33344555587	3	10,0	ProdutoZ	SãoPaulo	LimaRonaldoK.
33344555587	3	10,0	ProdutoZ	SãoPaulo	WongFernandoT.
33344555587	10	10,0	Computadorização	Mauá	WongFernandoT.
*33344555587	20	10,0	Reorganização	SãoPaulo	LimaRonaldoK.
33344555587	20	10,0	Reorganização	SãoPaulo	WongFernandoT.
			⋮		

Figura 15.6
Resultado da aplicação do NATURAL JOIN às tuplas acima das linhas tracejadas em FUNC_PROJ1 e FUNC_LOCAL da Figura 15.5. As tuplas falsas geradas são marcadas com asteriscos.

te relações com atributos correspondentes que não sejam combinações (chave estrangeira, chave primária), pois a junção sobre tais atributos pode produzir tuplas falsas.

Essa diretriz informal, obviamente, precisa ser declarada de maneira mais formal. Na Seção 16.2, discutiremos uma condição formal chamada propriedade de junção não aditiva (ou sem perda), que garante que certas junções não produzam tuplas falsas.

15.1.5 Resumo e discussão das diretrizes de projeto

Nas seções 15.1.1 a 15.1.4, discutimos informalmente situações que levam a esquemas de relação problemáticos e propusemos diretrizes informais para um bom projeto relacional. Os problemas que apontamos, que podem ser detectados sem ferramentas de análise adicionais, são os seguintes:

- Anomalias que causam trabalho redundante durante a inserção e modificação em uma relação, e que podem causar perda acidental de informação durante a exclusão de uma relação.
- Desperdício de espaço de armazenamento devido a NULLs e a dificuldade de realizar seleções, operações de agregação e junções por causa de valores NULL.
- Geração de dados inválidos e falsos durante as junções em relações da base com atributos correspondentes que possam não representar um relacionamento apropriado (chave estrangeira, chave primária).

No restante deste capítulo, apresentamos os conceitos formais e a teoria que pode ser usada para definir os pontos *positivos* e *negativos* dos esquemas de relação *individuais* com mais precisão. Primeiro, discutimos a dependência funcional como uma ferramenta para análise. Depois, especificamos as três formas normais e a Forma Normal de Boyce-Codd (FNBC) para esquemas de relação. A estratégia para alcançar um bom projeto é decompor de maneira correta uma relação mal projetada. Também intro-

duzimos rapidamente formas normais adicionais que lidam com dependências adicionais. No Capítulo 16, discutimos as propriedades da decomposição com detalhes, e oferecemos algoritmos que projetam relações de baixo para cima, usando as dependências funcionais como ponto de partida.

15.2 Dependências funcionais

Até aqui, lidamos com as medidas informais do projeto de banco de dados. Agora, vamos introduzir uma ferramenta formal para a análise de esquemas relacionais, que nos permite detectar e descrever alguns dos problemas mencionados em termos precisos. O conceito isolado mais importante na teoria de projeto de esquema relacional é o deuma dependência funcional. Nesta seção, definimos formalmente o conceito e, na Seção 15.3, veremos como ele pode ser usado para definir formas normais para esquemas de relação.

15.2.1 Definição de dependência funcional

Uma dependência funcional é uma restrição entre dois conjuntos de atributos do banco de dados. Suponha que nosso esquema de banco de dados relacional tenha n atributos $A_1, A_2, ..., A_n$. Vamos pensar no banco de dados inteiro sendo descrito por um único esquema de relação **universal** $R = \{A_1, A_2, ..., A_n\}$.[7] Não queremos dizer que realmente armazenaremos o banco de dados como uma única tabela universal — usamos esse conceito apenas no desenvolvimento da teoria formal das dependências de dados.[8]

> **Definição.** Uma **dependência funcional**, indicada por $X \rightarrow Y$, entre dois conjuntos de atributos X e Y que são subconjuntos de R, especifica uma *restrição* sobre possíveis tuplas que podem formar um estado de relação r de R. A restrição é que, para quaisquer duas tuplas t_1 e t_2 em r que tenham $t_1[X] = t_2[X]$, elas também devem ter $t_1[Y] = t_2[Y]$.

Isso significa que os valores do componente Y de uma tupla em r dependem dos (ou são *determinados pelos*) valores do componente X; como alternativa, os valores do componente X de uma tupla *determinam* exclusivamente (ou **funcionalmente**) os valores do componente Y. Também dizemos que existe uma dependência funcional de X para Y, ou que Y é **funcionalmente dependente** de X. A abreviação para a dependência funcional é **DF** ou **d.f.** O conjunto de atributos X é chamado de **lado esquerdo** da DF, e Y é chamado de **lado direito**.

Assim, a funcionalidade X determina Y em um esquema de relação R se, e somente se, sempre que duas tuplas de $r(R)$ combinarem sobre seu valor X, elas devem necessariamente combinar sobre seu valor Y. Observe o seguinte:

- Se uma restrição sobre R declarar que não pode haver mais de uma tupla com determinado valor X em qualquer instância de relação $r(R)$ — ou seja, X é uma **chave candidata** de R —, isso implica que $X \rightarrow Y$ para qualquer subconjunto de atributos Y de R (porque a restrição de chave implica que duas tuplas em qualquer estado válido $r(R)$ não terão o mesmo valor de X). Se X for uma chave candidata de R, então $X \rightarrow R$.
- Se $X \rightarrow Y$ em R, isso não quer dizer que $Y \rightarrow X$ em R.

Uma dependência funcional é uma propriedade da **semântica** ou **significado dos atributos**. Os projetistas de banco de dados usarão seu conhecimento da semântica dos atributos de R — ou seja, como eles se relacionam entre si — para especificar as dependências funcionais que devem ser mantidas em *todos* os estados de relação (extensões) r de R. Toda vez que a semântica de dois conjuntos de atributos em R indicar que uma dependência funcional deve ser mantida, especificamos a dependência como uma restrição. As extensões de relação $r(R)$ que satisfazem as restrições de dependência funcional são chamadas de **estados de relação válidos** (ou **extensões válidas**) de R. Logo, o uso principal das dependências funcionais é para descrever melhor um esquema de relação R ao especificar restrições sobre seus atributos que devem ser mantidas *o tempo todo*. Certas DFs podem ser especificadas sem que se refiram a uma relação específica, mas como uma propriedade desses atributos, dado seu significado comumente entendido. Por exemplo, {Estado, Num_habilitacao} → Cpf deve ser mantido para qualquer adulto no Brasil e, por isso, ser mantido sempre que esses atributos aparecerem em uma relação. Também é possível que certas dependências funcionais possam deixar de existir no mundo real se o relacionamento mudar. Por exemplo, a DF Cep → Codigo_area costumava existir como um relacionamento entre os códigos postais e os códigos de área de telefone no Brasil, mas, com a proliferação dos códigos de área telefônicos, isso não é mais verdadeiro.

Considere o esquema de relação FUNC_PROJ da Figura 15.3(b). Pela semântica dos atributos e da re-

[7] Esse conceito de uma relação universal será importante quando discutirmos os algoritmos para o projeto de banco de dados relacional no Capítulo 16.

[8] Esta suposição implica que cada atributo no banco de dados tenha um nome distinto. No Capítulo 3, iniciamos os nomes de atributo com nomes de relação, para obter exclusividade sempre que os atributos em relações distintas tinham o mesmo nome.

lação, sabemos que as seguintes dependências funcionais devem ser mantidas:

a. Cpf → Fnome
b. Projnumero →{Projnome, Projlocal}
c. {Cpf, Projnumero} → Horas

Essas dependências funcionais especificam que (a) o valor do número do Cadastro de Pessoa Física (Cpf) de um funcionário determina exclusivamente o nome do funcionário (Fnome), (b) o valor do número de um projeto (Projnumero) determina exclusivamente o nome do projeto (Projnome) e seu local (Projlocal) e (c) uma combinação de valores de Cpf e Projnumero determina exclusivamente o número de horas que o funcionário costuma trabalhar no projeto por semana (Horas). Como alternativa, dizemos que Fnome é determinado de maneira funcional por (ou dependente funcionalmente de) Cpf, ou *dado um valor de Cpf, sabemos o valor de Fnome*, e assim por diante.

Uma dependência funcional é uma *propriedade do esquema de relação R*, e não um estado de relação válido e específico *r* de *R*. Portanto, uma DF *não pode* ser deduzida automaticamente por determinada extensão de relação *r*, mas deve ser definida de maneira explícita por alguém que conhece a semântica dos atributos de *R*. Por exemplo, a Figura 15.7 mostra um estado em particular do esquema de relação ENSINA. Embora à primeira vista possamos pensar que Texto → Disciplina, não podemos confirmar isso a menos que saibamos que é verdadeiro para *todos os estados legais possíveis* de ENSINA. É, no entanto, suficiente demonstrar *um único contraexemplo* para refutar uma dependência funcional. Por exemplo, como 'Silva' leciona tanto 'Estruturas de Dados' e 'Gerenciamento de Dados', podemos concluir que o Professor *não* determina funcionalmente a Disciplina.

Dada uma relação preenchida, não se podem determinar quais DFs são mantidas e quais não são, a menos que o significado e os relacionamentos entre os atributos seja bem conhecido. Tudo o que se pode dizer é que certa DF *pode existir* se for mantida pela sua extensão em particular. Não se pode garantir sua existência até que o significado dos atributos correspondentes seja claramente compreendido. Porém, pode-se afirmar de modo enfático que certa DF *não se mantém* se houver tuplas que mostrem a violação de tal DF. Veja a relação de exemplo ilustrativa na Figura 15.8. Nela, as DFs a seguir *podem ser mantidas* porque as quatro tuplas na extensão atual não têm violação dessas restrições: $B \rightarrow C$; $C \rightarrow B$; $\{A, B\} \rightarrow C$; $\{A, B\} \rightarrow D$; e $\{C, D\} \rightarrow B$. No entanto, as seguintes *não* se mantêm porque já temos violações delas na extensão dada: $A \rightarrow B$ (tuplas 1 e 2 violam essa restrição); $B \rightarrow A$ (tuplas 2 e 3 violam essa restrição); $D \rightarrow C$ (tuplas 3 e 4 a violam).

ENSINA

Professor	Disciplina	Texto
Silva	EstruturasdeDados	Bartram
Silva	GerenciamentodeDados	Martin
Neto	Compiladores	Hoffman
Braga	EstruturasdeDados	Horowitz

Figura 15.7
Um estado de relação de ENSINA com uma possível dependência funcional TEXTO→ DISCIPLINA. Porém, PROFESSOR → DISCIPLINA está excluída.

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Figura 15.8
Uma relação *R* (A, B, C, D) com sua extensão.

A Figura 15.3 apresenta uma **notação diagramática** para exibir DFs: cada DF aparece como uma linha horizontal. Os atributos do lado esquerdo da DF são conectados por linhas verticais à linha que representa a DF, enquanto os atributos do lado direito são conectados pelas linhas com setas que apontam para os atributos.

Indicamos com *F* o conjunto de dependências funcionais que são especificadas no esquema de relação *R*. Em geral, o projetista do esquema especifica as dependências funcionais que são *semanticamente óbvias*. Porém, diversas outras dependências funcionais se mantêm em *todas* as instâncias de relação válidas entre conjuntos de atributos que podem ser derivados das (e satisfazem as) dependências em *F*. Essas outras dependências podem ser *deduzidas* das DFs em *F*. Adiaremos os detalhes das regras de inferência e propriedades das dependências funcionais para o Capítulo 16.

15.3 Formas normais baseadas em chaves primárias

Após introduzir as dependências funcionais, agora estamos prontos para usá-las na especificação de alguns aspectos da semântica dos esquemas de relação. Consideramos que um conjunto de dependências funcionais é dado para cada relação, e que cada

relação tem uma chave primária designada. Essa informação combinada com os testes (condições) para formas normais controla o *processo de normalização* para o projeto do esquema relacional. A maioria dos projetos relacionais práticos assume uma das duas técnicas a seguir:

- Realiza um projeto de esquema conceitual usando um modelo conceitual como ER ou EER e mapeia o projeto conceitual para um conjunto de relações.
- Projeta as relações com base no conhecimento externo derivado de uma implementação existente de arquivos, formulários ou relatórios.

Ao seguir uma dessas técnicas, é útil avaliar a virtude de relações e decompô-las ainda mais, conforme a necessidade, para obter formas normais mais altas, usando a teoria de normalização apresentada neste capítulo e no seguinte. Nesta seção, focalizamos as três primeiras formas normais para esquemas de relação e a intuição por trás delas, e discutimos como elas foram desenvolvidas historicamente. Definições mais gerais dessas formas normais, que levam em conta todas as chaves candidatas de uma relação, em vez de apenas a chave primária, são adiadas para a Seção 15.4.

Começamos discutindo informalmente as formas normais e a motivação por trás de seu desenvolvimento, bem como revisando algumas definições do Capítulo 3, que são necessárias aqui. Depois, discutimos a primeira forma normal (1FN) na Seção 15.3.4, e apresentamos as definições da segunda forma normal (2FN) e terceira forma normal (3FN), que são baseadas em chaves primárias, nas seções 15.3.5 e 15.3.6, respectivamente.

15.3.1 Normalização de relações

O processo de normalização, proposto inicialmente por Codd (1972a), leva um esquema de relação por uma série de testes para *certificar* se ele satisfaz certa **forma normal**. O processo, que prossegue em um padrão de cima para baixo, avaliando cada relação em comparação com os critérios para as formas normais e decompondo as relações conforme a necessidade, pode assim ser considerado *projeto relacional por análise*. Inicialmente, Codd propôs três formas normais, que ele chamou de primeira, segunda e terceira forma normal. Uma definição mais forte da 3FN — chamada Forma Normal Boyce-Codd (FNBC) — foi proposta posteriormente por Boyce e Codd. Todas essas formas normais estão baseadas em uma única ferramenta analítica: as dependências funcionais entre os atributos de uma relação. Depois, uma quarta forma normal (4FN) e uma quinta forma normal (5FN) foram propostas, com base nos conceitos de dependências multivaloradas e dependências de junção, respectivamente: estas serão discutidas rapidamente nas seções 15.6 e 15.7.

A **normalização de dados** pode ser considerada um processo de analisar os esquemas de relação dados com base em suas DFs e chaves primárias para conseguir as propriedades desejadas de (1) minimização da redundância e (2) minimização das anomalias de inserção, exclusão e atualização discutidas na Seção 15.1.2. Esse pode ser considerado um processo de 'filtragem' ou 'purificação' para fazer que o projeto tenha uma qualidade cada vez melhor. Esquemas de relação insatisfatórios, que não atendem a certas condições — os **testes de forma normal** —, são decompostos em esquemas de relação menores, que atendem aos testes e, portanto, possuem as propriedades desejáveis. Assim, o procedimento de normalização oferece aos projetistas de banco de dados o seguinte:

- Uma estrutura formal para analisar esquemas de relação com base em suas chaves e nas dependências funcionais entre seus atributos.
- Uma série de testes de forma normal que podem ser executados em esquemas de relação individuais, de modo que o banco de dados relacional possa ser **normalizado** para qualquer grau desejado.

Definição. A **forma normal** de uma relação refere-se à condição de forma normal mais alta a que ela atende e, portanto, indica o grau ao qual ela foi normalizada.

As formas normais, quando consideradas *isoladamente* de outros fatores, não garantem um bom projeto de banco de dados. Em geral, não é suficiente verificar em separado se cada esquema de relação no banco de dados está, digamos, na FNBC ou 3FN. Em vez disso, o processo de normalização pela decomposição também precisa confirmar a existência de propriedades adicionais que os esquemas relacionais, tomados juntos, devem possuir. Estas incluiriam duas propriedades:

- A **propriedade de junção não aditiva ou junção sem perdas**, que garante que o problema de geração de tuplas falsas, discutido na Seção 15.1.4, não ocorra com relação aos esquemas de relação criados após a decomposição.
- A **propriedade de preservação de dependência**, que garante que cada dependência funcional seja representada em alguma relação individual resultante após a decomposição.

- A propriedade de junção não aditiva é extremamente crítica e **deve ser alcançada a todo custo**, ao passo que a propriedade de preservação de dependência, embora desejável, às vezes é sacrificada, conforme discutiremos na Seção 16.1.2. Adiaremos a apresentação dos conceitos e técnicas formais que garantem as duas propriedades citadas para o Capítulo 16.

15.3.2 Uso prático das formas normais

A maioria dos projetos práticos adquire projetos existentes de bancos de dados anteriores, projetos em modelos legados ou de arquivos existentes. A normalização é executada na prática, de modo que os projetos resultantes sejam de alta qualidade e atendam às propriedades desejáveis indicadas anteriormente. Embora várias formas normais mais altas tenham sido definidas, como a 4FN e a 5FN, que discutiremos nas seções 15.6 e 15.7, a utilidade prática dessas formas normais torna-se questionável quando as restrições sobre as quais elas estão baseadas são raras, e difíceis de entender ou detectar pelos projetistas e usuários de banco de dados que precisam descobrir essas restrições. Assim, o projeto de banco de dados praticado na indústria hoje presta atenção particular à normalização apenas até a 3FN, FNBC ou, no máximo, 4FN.

Outro ponto que merece ser observado é que os projetistas de banco de dados *não precisam* normalizar para a forma normal mais alta possível. As relações podem ser deixadas em um estado de normalização inferior, como 2FN, por questões de desempenho, como aquelas discutidas ao final da Seção 15.1.2. Fazer isso gera as penalidades correspondentes de lidar com as anomalias.

Definição. **Desnormalização** é o processo de armazenar a junção de relações na forma normal mais alta como uma relação da base, que está em uma forma normal mais baixa.

15.3.3 Definições de chaves e atributos participantes em chaves

Antes de prosseguirmos, vejamos novamente as definições de chaves de um esquema de relação, do Capítulo 3.

Definição. Uma **superchave** de um esquema de relação $R = \{A_1, A_2, ..., A_n\}$ é um conjunto de atributos $S \subseteq R$ com a propriedade de que duas tuplas t_1 e t_2 em qualquer estado de relação válido r de R não terão $t_1[S] = t_2[S]$. Uma **chave** Ch é uma superchave com a propriedade adicional de que a remoção de qualquer atributo de Ch fará que Ch não seja mais uma superchave.

A diferença entre uma chave e uma superchave é que a primeira precisa ser *mínima*; ou seja, se tivermos uma chave $Ch = \{A_1, A_2, ..., A_k\}$ de R, então $Ch - \{A_i\}$ não é uma chave de R para qualquer A_i, $1 \leq i \leq k$. Na Figura 15.1, {Cpf} é uma chave para FUNCIONARIO, enquanto {Cpf}, {Cpf, Fnome}, {Cpf, Fnome, Datanasc} e qualquer conjunto de atributos que inclua Cpf são todos superchaves.

Se um esquema de relação tiver mais de uma chave, cada uma é chamada de **chave candidata**. Uma das chaves candidatas é *arbitrariamente* designada para ser a **chave primária**, e as outras são chamadas de chaves secundárias. Em um banco de dados relacional prático, cada esquema de relação precisa ter uma chave primária. Se nenhuma chave candidata for conhecida para uma relação, a relação inteira pode ser tratada como uma superchave padrão. Na Figura 15.1, {Cpf} é a única chave candidata para FUNCIONARIO, de modo que também é a chave primária.

Definição. Um atributo do esquema de relação R é chamado de **atributo principal** de R se ele for um membro de *alguma chave candidata* de R. Um atributo é chamado **não principal** se não for um atributo principal — ou seja, se não for um membro de qualquer chave candidata.

Na Figura 15.1, tanto Cpf quanto Projnumero são atributos principais de TRABALHA_EM, ao passo que outros atributos de TRABALHA_EM são não principais.

Agora, vamos apresentar as três primeiras formas normais: 1FN, 2FN e 3FN. Elas foram propostas por Codd (1972a) como uma sequência para conseguir o estado desejável de relações 3FN ao prosseguir pelos estados intermediários de 1FN e 2FN, se necessário. Conforme veremos, 2FN e 3FN atacam diferentes problemas. Contudo, por motivos históricos, é comum segui-los nessa sequência. Logo, por definição, uma relação 3FN *já satisfaz* a 2FN.

15.3.4 Primeira forma normal

A **primeira forma normal (1FN)** agora é considerada parte da definição formal de uma relação no modelo relacional básico (plano). Historicamente, ela foi definida para reprovar atributos multivalorados, atributos compostos e suas combinações. Ela afirma que o domínio de um atributo deve incluir apenas *valores atômicos* (simples, indivisíveis) e que o valor de qualquer atributo em uma tupla deve ser um *único valor* do domínio desse atributo. Logo, 1FN reprova ter um conjunto de valores, uma tupla de valores ou uma combinação de ambos como um valor de atributo para uma *única tupla*. Em outras palavras, a 1FN reprova *relações dentro de relações* ou *relações como*

valores de atributo dentro de tuplas. Os únicos valores de atributo permitidos pela 1FN são os **valores atômicos** (ou **indivisíveis**).

Considere o esquema de relação DEPARTAMENTO da Figura 15.1, cuja chave primária é Dnumero, e suponha que a estendamos ao incluir o atributo Dlocal, conforme mostra a Figura 15.9(a). Supomos que cada departamento pode ter *certo número de* locais. O esquema DEPARTAMENTO e um exemplo de estado de relação são mostrados na Figura 15.9. Como podemos ver, esta não está em 1FN porque Dlocal não é um atributo atômico, conforme ilustrado pela primeira tupla na Figura 15.9(b). Existem duas maneiras possíveis para examinar o atributo Dlocal:

- O domínio de Dlocal contém valores atômicos, mas algumas tuplas podem ter um conjunto desses valores. Nesse caso, Dlocal não é funcionalmente dependente da chave primária Dnumero.
- O domínio de Dlocal contém conjuntos de valores e, portanto, é não atômico. Nesse caso, Dnumero → Dlocal, pois cada conjunto é considerado um único membro do domínio de atributo.[9]

De qualquer forma, a relação DEPARTAMENTO da Figura 15.9 não está na 1FN; de fato, ela nem sequer se qualifica como uma relação, de acordo com nossa definição na Seção 3.1. Existem três técnicas principais para conseguir a primeira forma normal para tal relação:

1. Remover o atributo Dlocal que viola a 1FN e colocá-lo em uma relação separada LOCALIZACAO_DEP, junto com a chave primária Dnumero de DEPARTAMENTO. A chave primária dessa relação é a combinação {Dnumero, Dlocal}, como mostra a Figura 15.2. Existe uma tupla distinta em LOCALIZACAO_DEP para *cada local* de um departamento. Isso decompõe a relação não 1FN em duas relações 1FN.
2. Expandir a chave de modo que haverá uma tupla separada na relação srcinal DEPARTAMENTO para cada local de um DEPARTAMENTO, como mostra a Figura 15.9(c). Nesse caso, a chave primária torna-se a combinação {Dnumero, Dlocal}. Essa solução tem a desvantagem de introduzir a *redundância* na relação.
3. Se o *número máximo de valores* for conhecido para o atributo — por exemplo, se for conhecido que *no máximo três locais* poderão existir para um departamento —, substituir o atributo Dlocal pelos três atributos atômicos: Dlocal1, Dlocal2 e Dlocal3. Essa solução tem a desvantagem de introduzir *valores* NULL se a maioria dos departamentos tiver menos de três locais. Ela ainda introduz uma falsa semântica sobre a ordenação entre os valores de local, que não era intencionado srcinalmente. A consulta sobre esse atributo torna-se mais difícil. Por exemplo, considere como você escreveria a consulta: *Listar os departamentos que têm 'Santo André' como um de seus locais* nesse projeto.

Das três soluções anteriores, a primeira geralmente é considerada a melhor, pois não sofre de redundância e é completamente genérica, não tendo limite imposto sobre o número máximo de valores. De fato, se escolhermos a segunda solução, ela será

(a)

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Dlocal



(b)

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Dlocal
Pesquisa	5	33344555587	Santo André, Itu, São Paulo
Administração	4	98765432168	Mauá
Matriz	1	88866555576	SãoPaulo

(c)

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Dlocal
Pesquisa	5	33344555587	SantoAndré
Pesquisa	5	33344555587	Itu
Pesquisa	5	33344555587	SãoPaulo
Administração	4	98765432168	Mauá
Matriz	1	88866555576	SãoPaulo

Figura 15.9
Normalização na 1FN. (a) Um esquema de relação que não está em 1FN. (b) Exemplo de estado da relação DEPARTAMENTO. (c) Versão 1FN da mesma relação com redundância.

[9] Nesse caso, podemos considerar o domínio de Dlocalizacoes como sendo o **conjunto de potência** do conjunto de locais isolados; ou seja, o domínio é composto por todos os subconjuntos possíveis do conjunto de locais isolados.

decomposta ainda mais durante as etapas de normalização subsequentes para a primeira solução.

A primeira forma normal também desaprova atributos multivalorados que por si só sejam compostos. Estes são chamados de **relações aninhadas**, pois cada tupla pode ter uma relação *dentro dela*. A Figura 15.10 mostra como a relação FUNC_PROJ poderia aparecer se o aninhamento for permitido. Cada tupla representa uma entidade de funcionário, e a relação PROJS(Projnumero, Horas) *dentro de cada tupla* representa os projetos do funcionário e o número de horas por semana que ele trabalha em cada projeto. O esquema dessa relação FUNC_PROJ pode ser representado da seguinte forma:

FUNC_PROJ(Cpf, Fnome, {PROJS(Projnumero, Horas)})

O conjunto de chaves { } identifica o atributo PROJS como multivalorado, e listamos os atributos componentes que formam o PROJS entre parênteses ( ). O interessante é que as tendências recentes para dar suporte a objetos complexos (ver Capítulo 11) e dados XML (ver Capítulo 12) tentam permitir e formalizar as relações aninhadas nos sistemas de bancos de dados relacionais, que eram reprovadas inicialmente pela 1FN.

(a)

FUNC_PROJ

Cpf	Fnome	Projs	
		Projnumero	Horas

(b)

FUNC_PROJ

Cpf	Fnome	Projnumero	Horas
12345678966	Silva,João B.	1	32,5
		2	7,5
66688444476	Lima,Ronaldo K.	3	40,0
45345345376	Leite,Joice A.	1	20,0
		2	20,0
33344555587	Wong,Fernando T.	2	10,0
		3	10,0
		10	10,0
		20	10,0
99988777767	Zelaya,Alice J.	30	30,0
		10	10,0
98798798733	Pereira,André V.	10	35,0
		30	5,0
98765432168	Souza,Jennifer S.	30	20,0
		20	15,0
88866555576	Brito,Jorge E.	20	NULL

(c)

FUNC_PROJ1

Cpf	Fnome

FUNC_PROJ2

Cpf	Projnumero	Horas

Figura 15.10
Normalizando relações aninhadas para a 1FN. (a) Esquema da relação FUNC_PROJ com um atributo de relação aninhada PROJS. (b) Exemplo de extensão da relação FUNC_PROJ mostrando relações aninhadas dentro de cada tupla. (c) Decomposição de FUNC_PROJ nas relações FUNC_PROJ1 e FUNC_PROJ2 pela propagação da chave primária.

Observe que Cpf é a chave primária da relação FUNC_PROJ nas figuras 15.10(a) e (b), enquanto Projnumero é a chave **parcial** da relação aninhada; ou seja, dentro de cada tupla, a relação aninhada precisa ter valores únicos de Projnumero. Para normalizar isso para a 1FN, removemos os atributos da relação aninhada para uma nova relação e *propagamos a chave primária* para ela. A chave primária da nova relação combinará a parcial com a chave primária da relação srcinal. A decomposição e a propagação da chave primária resultam nos esquemas FUNC_PROJ1 e FUNC_PROJ2, como mostra a Figura 15.10(c).

Esse procedimento pode ser aplicado recursivamente a uma relação com aninhamento em nível multiplo para **desaninhar** a relação para um conjunto de relações 1FN. Isso é útil na conversão de um esquema de relação não normalizado com muitos níveis de aninhamento em relações 1FN. A existência de mais de um atributo multivalorado em uma relação deve ser tratada com cuidado. Como um exemplo, considere a seguinte relação não 1FN:

PESSOA (Cpf, {Placa}, {Telefone})

Essa relação representa o fato de uma pessoa ter vários carros e vários telefones. Se a estratégia 2 acima for seguida, ela resulta em uma relação com todas as chaves:

PESSOA_NA_1FN (Cpf, Placa, Telefone)

Para evitar a introdução de qualquer relacionamento estranho entre Placa e Telefone, todas as combinações de valores possíveis são representadas para cada Cpf, fazendo surgir a redundância. Isso leva aos problemas tratados pelas dependências multivaloradas e 4FN, que discutiremos na Seção 15.6. O modo certo de lidar com os dois atributos multivalorados em PESSOA mostrados anteriormente é decompô-los em duas relações separadas, usando

a estratégia 1 já discutida: P1(Cpf, Placa) e P2(Cpf, Telefone).

15.3.5 Segunda forma normal

A **segunda forma normal** (2FN) é baseada no conceito de *dependência funcional total* Uma dependência funcional $X \rightarrow Y$ é uma **dependência funcional total** se a remoção de qualquer atributo A de X significar que a dependência não se mantém mais; ou seja, para qualquer atributo $A \varepsilon X$, $(X - \{A\})$ *não* determina Y funcionalmente. Uma dependência funcional $X \rightarrow Y$ é uma **dependência parcial** se algum atributo $A \varepsilon X$ puder ser removido de X e a dependência ainda se mantiver; ou seja, para algum $A \varepsilon X$, $(X - \{A\}) \rightarrow Y$. Na Figura 15.3(b), {Cpf, Projnumero} → Horas é uma dependência total (nem Cpf → Horas nem Projnumero → Horas se mantêm). Contudo, a dependência {Cpf, Projnumero} → Fnome é parcial porque Cpf → Fnome se mantém.

> **Definição.** Um esquema de relação R está em 2FN se cada atributo não principal A em R for *total e funcionalmente dependente* da chave primária de R.

O teste para a 2FN envolve testar as dependências funcionais cujos atributos do lado esquerdo fazem parte da chave primária. Se a chave primária tiver um único atributo, o teste não precisa ser aplicado. A relação FUNC_PROJ na Figura 15.3(b) está na 1FN, mas não está na 2FN. O atributo não principal Fnome viola a 2FN por causa da DF2, assim como os atributos não principais Projnome e Projlocal, por causa da DF3. As dependências funcionais DF2 e DF3 tornam Fnome, Projnome e Projlocal parcialmente dependentes da chave primária {Cpf, Projnumero} de FUNC_PROJ, violando, assim, o teste da 2FN.

Se um esquema de relação não estiver na 2FN, ele pode ser *segundo normalizado* ou *normalizado pela 2FN* para uma série de relações 2FN em que os atributos não principais são associados com a parte da chave primária em que eles são total e funcionalmente dependentes. Portanto, as dependências funcionais DF1, DF2 e DF3 da Figura 15.3(b) levam à decomposição de FUNC_PROJ nos três esquemas de relação FP1, FP2 e FP3 mostrados na Figura 15.11(a), cada qual estando na 2FN.

15.3.6 Terceira forma normal

A **terceira forma normal** (**3FN**) é baseada no conceito de *dependência transitiva*. Uma dependência funcional $X \rightarrow Y$ em um esquema de relação R é uma **dependência transitiva** se houver um conjunto de atributos Z em R que nem sejam uma chave candidata nem um subconjunto de qualquer chave de R,[10] e tanto $X \rightarrow Z$ quanto $Z \rightarrow Y$ se mantiverem. A dependência Cpf → Cpf_gerente é transitiva por meio de Dnumero em FUNC_DEP na Figura 15.3(a), pois ambas as dependências Cpf → Dnumero e Dnumero → Cpf_gerente se mantêm *e* Dnumero não é nem uma chave por si só nem um subconjunto da chave de FUNC_DEP. Intuitivamente, podemos ver que a dependência de Cpf_gerente sobre Dnumero é indesejável em FUNC_DEP, pois Dnumero não é uma chave de FUNC_DEP.

> **Definição.** De acordo com a definição srcinal de Codd, um esquema de relação R está na 3FN se ele satisfizer a 2FN *e* nenhum atributo não principal de R for transitivamente dependente da chave primária.

O esquema de relação FUNC_DEP da Figura 15.3(a) está na 2FN, pois não existe dependência parcial sobre uma chave. Porém, FUNC_DEP não está na 3FN devido à dependência transitiva de Cpf_gerente (e também Dnome) em Cpf por meio de Dnumero. Podemos normalizar FUNC_DEP decompondo-o nos dois esquemas de relação 3FN DF1 e DF2 mostrados na Figura 15.11(b). Intuitivamente, vemos que DF1 e DF2 representam fatos de entidades independentes sobre funcionários e departamentos. Uma operação JUNÇÃO NATURAL sobre DF1 e DF2 recuperará a relação srcinal FUNC_DEP sem gerar tuplas falsas.

De maneira intuitiva, podemos ver que qualquer dependência funcional de que o lado esquerdo faz parte (é um subconjunto apropriado) da chave primária, ou qualquer dependência funcional de que o lado esquerdo é um atributo não chave, é uma DF *problemática*. A normalização 2FN e 3FN remove essas DFs problemáticas ao decompor a relação original em novas relações. Em relação ao processo de normalização, não é necessário remover as dependências parciais antes das dependências transitivas, porém, historicamente, a 3FN tem sido definida com a suposição de que uma relação é testada primeiro pela 2FN, antes de ser testada pela 3FN. A Tabela 15.1 resume informalmente as três formas normais com base nas chaves primárias, os testes usados em cada uma e a *solução* ou normalização realizada para alcançar a forma normal.

[10] Essa é a definição geral de dependência transitiva. Como estamos preocupados apenas com as chaves primárias nesta seção, permitimos dependências transitivas onde X é a chave primária, mas Z pode ser (um subconjunto de) uma chave candidata.